

FIT1058 FOUNDATIONS OF COMPUTING

COURSE NOTES

Graham Farr, Alexey Ignatiev, and Rebecca Robinson

Faculty of Information Technology

Monash University

August 19, 2025

PREFACE

Welcome to FIT1058 Foundations of Computing!

Computation uses abstract formal models of real objects and systems. This unit lays the theoretical foundations for working with the most fundamental abstract models used in computer science. It will develop skills in abstract modelling, logical reasoning, rigorous proof, and mathematical analysis of computational methods and structures. These skills will be used throughout your degree in both theoretical and practical settings. The material we cover is considered core knowledge for computer science students and graduates by the major national and international professional associations.

This document contains the course notes for the unit. It will be released in instalments through the semester.

PREREQUISITES

This unit assumes successful prior study in mathematics at least to the standard of *Mathematical Methods units 3 & 4* in the Victorian Certificate of Education (VCE). That VCE subject in turn builds on mathematics studied earlier in high school.

The specific topics from school mathematics that we make most use of in this unit are:

- sets (including Venn diagrams, subsets, supersets, complement, union, intersection, counting subsets);
- functions (incl. their domain, codomain, rule), inverse functions, relations;
- sequences and series (arithmetic and geometric, finite and infinite);
- numbers (natural numbers, integers, rational numbers, real numbers, representations in a base, divisors, factorisation, composite numbers, primes, arithmetic, remainders);
- counting (incl. using addition and multiplication, permutations, combinations, binomial coefficients);
- probability (incl. using counting, combinations of events, conditional probability, random variables, probability distributions, binomial distribution, normal distribution, probability density functions);
- exponentiation and logarithms (which are ubiquitous in computer science);

- calculus (not a lot, but we make some use of limits and integrals, and derivatives can give useful insight even in situations where we don't make specific use of them).

We also make pervasive use of standard high-school algebra when working with polynomials and other functions.

We review parts of some of these topics in the pre-reading (see next section), but the pace and level still assumes prior knowledge. So it is important that you work to fill in any gaps or hazy areas in your knowledge of these topics.

PRE-READING

Sections whose numbers have superscript α contain *pre-reading*. For example:

§ 2.5 $^\alpha$ Functions with multiple arguments and values

These pre-reading sections should be read and studied before the seminars on that topic. Some of them review work you did in school, but you should still read them, for several reasons: they establish the notation, terminology and other conventions we will use, which sometimes differ from those used in schools; they give some important computer science context to the concepts discussed, which you may not be aware of even if you have studied the concepts themselves before; and our experience is that most students benefit from some reminders and revision of this school material anyway. Other sections marked α may cover new material that is so fundamental to the topic to be discussed that reading about it before the seminar will significantly increase your ability to learn the material and master it.

The amount of pre-reading varies, depending on the topic. Some chapters spend a lot of their early sections reviewing concepts and topics covered in school (e.g., the chapters on sets and functions). These have more pages of pre-reading, but reading them should not take as long as reading completely new material. Other chapters contain material that is almost entirely new. These have fewer pages of pre-reading, but those pages will need to be read more slowly and carefully.

EXTRA MATERIAL

Sections whose numbers have superscript ω contain extra material that is beyond the FIT1058 curriculum. For example:

§ 2.11 $^\omega$ Loose composition

This may include discussion of alternative approaches that we don't study, or more advanced aspects of the topic, or some historical background. The specific content introduced in these extra sections won't be on tests or exams in this unit. But reading them may still be of indirect benefit, by consolidating the material studied in the other sections.

ACKNOWLEDGEMENTS

Thanks very much to those who have given feedback on earlier versions of these Course Notes, including: David Albrecht, Mathew Baker, Harald Bögeholz, Annalisa Calvi, Nathan Companez, Michael Gill, Thomas Hendrey, Alexey Ignatiev, Roger Lim, Rebecca Robinson, James Sherwood, Alejandro Stuckey de la Banda, Tham Weng Kee, Nelly Tucker, Joel Wills, and some anonymous student reviewers.

Thanks also to the FIT1058 students, including Tiancheng Cai, Eunice Chan, Joshua Ding, Rawad El Nemr, Shshank Jha, Ian Ko, Zhi Ying Lee, Cody Lincoln, Zijing Song, Eugene Sulisty, Timothy Tong, Jing Yap, and Michael Zeng, who have pointed out some errors in earlier versions of these notes, or in the sample solutions to the exercises, enabling us to fix them. Further error-spotting is very welcome. We look forward to acknowledging more students in future.

CONTENTS

	Preface	iii
1	Sets	1
1.1 ^α	Sets and elements	1
1.2 ^α	Specifying sets	2
1.3 ^α	Cardinality	4
1.4	Sets of numbers	5
1.5	Sets of strings	5
1.6	Subsets and supersets	7
1.7	Minimal, minimum, maximal, maximum	9
1.8	Counting all subsets	11
1.9	The power set of a set	11
1.10	Counting subsets by size using binomial coefficients	12
1.11	Complement and set difference	17
1.12	Union and intersection	19
1.13	Symmetric difference	23
1.14	Cartesian product	24
1.15	Partitions	26
1.16	Exercises	28
2	Functions	37
2.1 ^α	Definitions	37
2.1.1 ^α	Domain	38
2.1.2 ^α	Codomain & co.	39
2.1.3 ^α	Rule	41
2.2 ^α	Functions in computer science	44
2.2.1 ^α	Functions from analysis	44
2.2.2 ^α	Functions in mathematics and programming	44
2.3 ^α	Notation	45
2.4 ^α	Some special functions	46
2.5 ^α	Functions with multiple arguments and values	46
2.6	Restrictions	47
2.7	Injections, surjections, bijections	49
2.8	Inverse functions	50
2.9	Composition	51
2.10	Cryptosystems	59
2.11 ^ω	Loose composition	62

2.12	Counting functions	63
2.13	Binary relations	64
2.14	Properties of binary relations	68
2.15	Combining binary relations	69
2.16	Equivalence relations	71
2.17	Relations	74
2.18	Counting relations	75
2.19	Exercises	76
3	Proofs	85
3.1 ^α	Theorems and proofs	85
3.2	Logical deduction	88
3.3	Proofs of existential and universal statements	92
3.4	Finding proofs	93
3.5	Types of proofs	94
3.6	Proof by symbolic manipulation	95
3.7	Proof by construction	95
3.8	Proof by cases	96
3.9	Proof by contradiction	96
3.10	Proof by mathematical induction	97
3.11	Induction: more examples	100
3.12 ^ω	Induction: extended example	104
3.13	Mathematical induction and statistical induction	108
3.14	Programs and proofs	109
3.15	Exercises	110
4	Propositional Logic	121
4.1 ^α	Truth values	121
4.2 ^α	Boolean variables	122
4.3 ^α	Propositions	122
4.4 ^α	Logical operations	123
4.5	Negation	123
4.6	Conjunction	124
4.7	Disjunction	125
4.8	De Morgan's Laws	126
4.9	Implication	127
4.10	Equivalence	128
4.11	Exclusive-or	129
4.12	Tautologies and logical equivalence	132
4.13 ^ω	History	132
4.14	Distributive Laws	133
4.15	Laws of Boolean algebra	133
4.16	Disjunctive Normal Form	134

4.17	Conjunctive Normal Form	138
4.18	Representing logical statements	139
4.19	Statements about how many variables are true	142
4.20	Universal sets of operations	143
4.21	Exercises	144
5	Predicate Logic	155
5.1 ^α	Relations, predicates, and truth-valued functions	155
5.2 ^α	Variables and constants	157
5.3 ^α	Predicates and variables	158
5.4	Arguments of predicates	160
5.5	Building logical expressions with predicates	163
5.6	Existential quantifier	164
5.7	Restricting existentially quantified variables	167
5.8	Universal quantifier	168
5.9	Restricting universally quantified variables	169
5.10	Multiple quantifiers	170
5.11	Predicate logic expressions	172
5.12	Doing logic with quantifiers	173
5.13	Duality between quantifiers	175
5.14	Summary of rules for logic with quantifiers	175
5.15	Some examples	176
5.16	Exercises	178
6	Sequences & Series	183
6.1 ^α	Definitions and notation	184
6.2 ^α	Recursive definitions of sequences	185
6.3 ^α	Arithmetic sequences	187
6.4 ^α	Geometric sequences	188
6.5	Harmonic sequences	189
6.6	From recursive definitions to formulas	190
6.7	The Fibonacci sequence	192
6.7.1	Upper bounds	193
6.7.2	Lower bounds	196
6.7.3	Asymptotic behaviour	199
6.7.4	An exact formula	200
6.8	Limits of infinite sequences	204
6.9	Big-O notation	210
6.10	Sums and summation	214
6.11	Finite series	217
6.12	Finite arithmetic series	218
6.13	Finite geometric series	219
6.14	Infinite geometric series	221

6.15	Harmonic numbers	221
6.16	Exercises	223
7	Number Theory	233
7.1 ^α	Multiples and divisors	233
7.2 ^α	Prime numbers	235
7.3	Remainders and the mod operation	236
7.4	Parity	238
7.5	The greatest common divisor	239
7.6	The Euclidean algorithm	241
7.7	The gcd and integer linear combinations	243
7.8	The extended Euclidean algorithm	247
7.9	Coprimality	250
7.10	Modular arithmetic	253
7.11	Modular inverses	259
7.12	The Euler totient function	260
7.13	Fast exponentiation	262
7.14	Modular exponentiation	263
7.15	Primitive roots	266
7.16	One-way functions	268
7.17	Modular exponentiation with fixed base	269
7.18	Diffie-Hellman key agreement scheme	270
7.19	Exercises	274
8	Counting & Combinatorics	281
8.1 ^α	Counting by addition	281
8.2 ^α	Counting by multiplication	282
8.3	Inclusion-exclusion	283
8.4	Inclusion-exclusion: derangements	289
8.5	Selection	293
8.6	Ordered selection with replacement	294
8.7	Ordered selection without replacement	295
8.8	Unordered selection <u>without</u> replacement	295
8.9	Unordered selection <u>with</u> replacement	296
8.10	Exercises	299
9	Discrete Probability I	305
9.1 ^α	The nature of randomness	306
9.2 ^α	Probability	307
9.3 ^α	Choice of sample space	313
9.4 ^α	Mutually exclusive events	317
9.5	Operations on events	319
9.6	Inclusion-Exclusion for probabilities	325

9.7	Independent events	327
9.8	Conditional probability	333
9.9	Bayes' Theorem	337
9.10	Exercises	342
10	Discrete Probability II	347
10.1 ^α	Random variables	347
10.2 ^α	Probability distributions	349
10.3 ^α	Expectation	351
10.4	Median	358
10.5	Mode	360
10.6	Variance	360
10.7	Uniform distribution	365
10.8	Binomial distribution	367
10.9	Poisson distribution	370
10.10	Geometric distribution	374
10.11	The coupon collector's problem	380
10.12	Exercises	385
11	Graph Theory I	389
11.1 ^α	Basic definitions	390
11.2 ^α	Types of graphs	392
11.3 ^ω	Graphs and relations	393
11.4 ^α	Representing graphs	394
11.4.1	Edge list	394
11.4.2	Adjacency matrix	394
11.4.3	Adjacency list	396
11.4.4	Incidence matrix	397
11.5	Subgraphs	398
11.6	Some special graphs	398
11.7	Degree	399
11.8	Moving around	402
11.9	Connectivity	407
11.10	Bipartite graphs	408
11.11	Euler tours	411
11.12	Exercises	416
12	Graph Theory II	421
12.1 ^α	Trees	421
12.2	Properties of trees	423
12.3	Forests	426
12.4	Spanning trees	427
12.5	Planarity	431

12.6 ^ω Games on graphs	438
12.6.1 ^ω Shannon's Switching Game	438
12.6.2 ^ω Slither	444
12.7 Exercises	447

SETS

The raw material of computation and communication is information. This takes many different forms, due to the great variety of things we might want to represent and the many different ways of representing them.

In this unit and in other units in your degree, you will learn about many different structures that are used to represent information in order to store it, communicate it or compute with it.

We will start with sets because these are among the simplest possible information structures. Most other structures can be defined in terms of sets, so sets are a foundational topic.

Sets are used extensively to define the *types* of objects that we compute with. When we work with a whole number, we say it is of Integer *type* because it belongs to the set of integers and satisfies the various laws and properties of integers. When we work with a string of characters, we might say that it is of String *type* because it belongs to the set of all strings over a suitable alphabet and satisfies the properties expected of strings. Many programming languages take careful account of the *types* of the objects they work with, and sets always underlie any notion of type.

1.1^α SETS AND ELEMENTS

A **set** is just a collection of objects, without order or repetition. The objects in a set are called its **elements** or **members**. To specify a set, we can just give a comma-separated list of them between curly braces. So the following are all sets:

$$\begin{aligned} &\{\text{Harry, Ginny, Hermione, Ron, Hagrid}\} \\ &\{42, -273.15, 1729, 10^{100}\} \\ &\{\text{CSIRAC, Manchester Baby, EDSAC}\} \\ &\{\} \end{aligned}$$

In the last example, the set is empty. This set, simple as it is, is so fundamental that it has its own symbol, $\emptyset$, not to be confused with zero, 0 (which, in the early days of the computer industry, was often written with a slash through it to distinguish it from the letter O).

When we write a set by listing its elements in the above way, we will inevitably list the elements in some specific order. But different orders of writing do not affect the identity of the set. Our third set above contains three of the earliest computers ever built, but they are not listed there in chronological order. If we wrote them in chronological order, the set would be written

$$\{\text{Manchester Baby, EDSAC, CSIRAC}\}.$$

But it would still be *the same set*:

$$\{\text{CSIRAC, Manchester Baby, EDSAC}\} = \{\text{Manchester Baby, EDSAC, CSIRAC}\}.$$

We state that an object is an element of a set using the symbol $\in$:

$$\text{object} \in \text{set}.$$

For example,

$$\text{CSIRAC} \in \{\text{Manchester Baby, EDSAC, CSIRAC}\}.$$

To state that an object does not belong to a set, we use $\notin$. For example,

$$\text{SILLIAC} \notin \{\text{Manchester Baby, EDSAC, CSIRAC}\}.$$

1.2^α SPECIFYING SETS

We will be working with many sets that are far larger than these examples, and many will be infinite. So it is often not practical to write out all the elements. So we need a succinct way of specifying precisely the elements of a set. One way is to give a condition that, in general, is either true or false, with the members of the set being precisely those objects for which the condition is true. For example,

$$\{x : x \text{ is even}\}$$

is the set of all even numbers. The variable x here is simply a name for elements of this set, so that we can talk about them. The colon, “:”, separates the name x from the condition on x that must be satisfied in order for it to be an element of this set. We read this as “the set of all x such that x is even”. The choice of name, x , is not important; we could equally well write the set as

$$\{n : n \text{ is even}\}$$

In this definition, the reader will naturally infer that the variable (x or n) represents a whole number, since the concept of a number being even or not only applies to whole

numbers; it makes no sense, in general, for rational numbers or real numbers. But it is often preferable to spell out the kind of numbers we are talking about, so that the reader does not have to fill in any gaps in our description. In this example, we might also want to remove any doubt in the reader's mind as to whether we are working with integers in general or just natural numbers. So we might rewrite our definition as

$$\{x : x \in \mathbb{Z} \text{ and } x \text{ is even}\}$$

Set definitions of this type have the general form

$$\{name : condition\},$$

where *name* is a convenient name for an arbitrary member of the set and *condition* is a statement involving *name* which is true precisely when the object called *name* belongs to the set and is false otherwise.

It is a common convention to include, in our statement of the name (before the colon), a specification of a larger set that the object must belong to. For example, the set of even integers could be written

$$\{x \in \mathbb{Z} : x \text{ is even}\}.$$

This can be read as “the set of x in $\mathbb{Z}$ such that x is even” or “the set of integers x such that x is even”. In general we can write

$$\{name \in larger\ set : condition\}.$$

It is necessary that the *condition* be precise and clear. To ensure this, it will often be specified in a formal symbolic way. It is ok to use English text in the condition *provided* it is used clearly and precisely. It is also important for the text to be succinct, subject to ensuring precision and clarity.

Another way to specify a set is to give a rule by which each member is constructed. For example, the set of even integers could be written

$$\{2n : n \in \mathbb{Z}\}$$

We read this as “the set of $2n$ such that n belongs to $\mathbb{Z}$ ” or “the set of $2n$ such that n is an integer”. The rule is a formula for converting a named object into a member of the set, and after the colon we give a condition that the named object must satisfy in order for the formula to be used. Taking all objects that satisfy this condition, and applying the formula to each one of them, must give *all* members of the set. In general, we can write

$$\{rule \text{ expressed in terms of } name : condition \text{ on } name\}.$$

Since the curly braces are read as “the set of”, it's ok to write, for example, {even integers} for the set of even integers or {people on Earth} for the set of all people on

Earth. This way of defining sets — using just English text between the braces — is fine when the English is completely precise and not too long. But it should be used with care, because of the risk of imprecision, and only works well for sets that can be described very simply.

People sometimes describe large sets by listing a few of their elements and expecting readers to spot the pattern and infer what the entire set is. For example, the set of even integers might sometimes be written as

$$\{0, 2, -2, 4, -4, 6, -6, \dots\}$$

or

$$\{\dots, -6, -4, -2, 0, 2, 4, 6, \dots\}.$$

While this sort of description might help communicate ideas in an informal conversation, it is *not* a definition of the set, since it does not precisely specify which elements are in the set, but rather turns that task over to the reader by the use of “...”. Informal descriptions have their place, and we will use them sometimes, but they are not formal definitions.

1.3^α CARDINALITY

The **size** or **cardinality** of a set is just the number of elements it has. If A is a set, then its size is denoted by $|A|$ or sometimes $\#A$. When a set is specified by just listing its elements, we can determine its size by just counting those elements, which can be done manually if the set is small enough. For the above examples, we have

$$\begin{aligned} |\{\text{Harry, Ginny, Hermione, Ron, Hagrid}\}| &= 5, \\ |\{42, -273.15, 1729, 10^{100}\}| &= 4, \\ |\{\text{CSIRAC, Manchester Baby, EDSAC}\}| &= 3, \\ |\emptyset| = |\{\}| &= 0. \end{aligned}$$

Determining the cardinality of a set is a fundamental skill in computer science. For example, if a set represents all the objects that an algorithm must examine in order to find one that is best in some sense, then determining the size of that set helps determine how long the algorithm will take. If another set represents all the data items that must be stored in some memory in a device, then determining its size helps determine how much storage will be used by the data. If yet another set represents all the possible outcomes of some computation, then its size is an indicator of how uncertain you are about that outcome before you do the computation.

We will often have to deal with very large sets, due to the huge amounts of data that computers work with. It is sometimes useful to focus on the *logarithm* of the size of a set.

1.4 SETS OF NUMBERS

Some sets are so commonly used that they have special names. We have already met $\emptyset$ which denotes the empty set. There are names for some fundamental sets of numbers:

$\mathbb{N}$	the set of positive integers
$\mathbb{N}_0$	the set of nonnegative integers
$\mathbb{Z}$	the set of all integers
$\mathbb{Q}$	the set of rational numbers
$\mathbb{R}$	the set of real numbers

Usually, when we work with these fundamental number sets, we are not *only* interested in them as plain *sets*: we may also be interested in the natural *order* they have (with $\leq$), and in some *operations* we can do with their elements (like $+$, $-$, $\times$ and more). So, the symbol $\mathbb{Z}$ stands for the *set* of integers (as above), but it is also used to represent that same set *together with* some selection of operations that we are interested in at the time. We will not dwell on this point further; it would be too fussy to start using different names for a number set depending on what operations on it were being used at the time.

To restrict any of these sets to only its positive or negative members, we can use superscript $+$ or $-$. So $\mathbb{Z}^+$ is another way of denoting $\mathbb{N}$, and $\mathbb{R}^-$ is the set of negative real numbers. To denote the set of nonnegative members of one of these sets of numbers, we combine superscript $+$ with subscript 0 , as in $\mathbb{R}_0^+$ for the set of nonnegative real numbers (since the nonnegative real numbers are just the positive real numbers together with zero). Similarly, $\mathbb{Q}_0^-$ is the set of nonpositive rational numbers.

For intervals of real numbers, there is some standard notation to indicate which, if any, of the two endpoints of the interval are included:

notation	definition	terminology
$[a, b]$	$\{x \in \mathbb{R} : a \leq x \leq b\}$	closed interval
$[a, b)$	$\{x \in \mathbb{R} : a \leq x < b\}$	half-open half-closed interval
$(a, b]$	$\{x \in \mathbb{R} : a < x \leq b\}$	
(a, b)	$\{x \in \mathbb{R} : a < x < b\}$	open interval

Sometimes we want to restrict the contents of the interval to one of our other special sets of numbers. We will indicate this using a subscript on the interval notation. For example, if we only want integers within the interval $[a, b]$, we write $[a, b]_{\mathbb{Z}}$, which is an abbreviation for $[a, b] \cap \mathbb{Z}$.

1.5 SETS OF STRINGS

For data to be stored, processed, or communicated, it first needs to be encoded in some symbolic form. So we start by specifying the symbols we are allowed to use, which we

often call **characters** or **letters**. The set of allowed characters is called the **alphabet**. We only consider finite alphabets.

For example, depending on the context, we could use the 26-letter English alphabet $\{a, b, c, \dots, y, z\}$ (restricting here to words consisting entirely of lower-case letters, and ignoring accents and apostrophes), or the alphabet of the ten decimal **digits** $\{0, 1, 2, \dots, 9\}$, or the set of two **bits** $\{0, 1\}$.

To represent data symbolically, we use *strings* of characters, where the characters all belong to some alphabet. If A is an alphabet, a **string over** A is a finite sequence of characters, each of which is drawn from that alphabet. In other words, a string is anything you can get by taking a character from your alphabet, then taking another (which might be the same one, or might not be) and putting it after the first, and then taking a third (which may or may not be one of the characters you have already used) and putting it after the first two, and so on, for as long as you like (but it must be finite). The **length** of a string is its number of characters. So, for example, the length of the string “babbage” is 7.

We allow the **empty string**, which has no characters. Because the empty string is, by its nature, impossible to see, there is a special symbol for it to help us write about it: this is ε , the Greek letter epsilon. The length of the empty string is 0.

So, every English word (written in lower case and without accents) is a string over the 26-letter alphabet $\{a, b, c, \dots, y, z\}$. Every positive integer can be represented in decimal notation as a string over the ten-digit alphabet $\{0, 1, 2, \dots, 9\}$, or in binary notation as a string over the two-bit alphabet $\{0, 1\}$. Every file on your computer may be regarded as a string over an appropriate alphabet, which might be the latest Unicode alphabet of about 155,000 characters. Strands of DNA are modelled as strings over the alphabet $\{C, G, A, T\}$.

If A is an alphabet and $k \in \mathbb{N}_0$, then A^k denotes all the strings of exactly k characters in which each character belongs to A . So A^1 is just the alphabet itself, and A^2 is the set of two-characters strings from that alphabet, and so on. For example, if $A = \{0, 1\}$ then

$$A^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}.$$

For any alphabet A , the set A^0 is the set containing just the empty string: $A^0 = \{\varepsilon\}$. This is not to be confused with the empty set!

How many strings of length k over the alphabet A are there? In other words, what is $|A^k|$? If $k = 1$, then we are just counting strings of length 1, which is the same as the number of letters in the alphabet, so $|A^1| = |A|$. If $k = 2$, then we are counting strings of length 2. For each possible first letter, we have $|A|$ choices for the second letter, since there is no restriction on the second letter. Since each choice of first letter gives the *same* number of choices for the second letter, and since there are $|A|$ choices for the first letter, we find that the number of strings of length 2 is $|A| \times |A| = |A|^2$. If $k = 3$, then we have $|A|^2$ choices for the first two letters (as we just saw), with each such choice followed by $|A|$ choices for the third letter, with this number being independent of the

choice we made for the first two letters. So the total number of strings of length 3 is $|A|^2 \times |A| = |A|^3$. This reasoning extends to any value of k . So the number of strings over A of length k is given by

$$|A^k| = |A|^k.$$

We also write A^* for the set of all finite strings (of all possible lengths) over the alphabet A . This is always an infinite set (provided $A \neq \emptyset$). For $A = \{0, 1\}$, we give a few of its smallest members:

$$A^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, \dots\}.$$

1.6 SUBSETS AND SUPERSETS

A **subset** A of a set B is a set A with the property that every element of A is also an element of B . We write $A \subseteq B$. For example,

$$\begin{aligned} \{\text{Manchester Baby, CSIRAC}\} &\subseteq \{\text{Manchester Baby, EDSAC, CSIRAC}\}, \\ \mathbb{N} &\subseteq \mathbb{Z}, \\ \mathbb{Q} &\subseteq \mathbb{Q}, \\ \{0, 1\}^3 &\subseteq \{0, 1\}^*, \\ \emptyset &\subseteq [1, 3]. \end{aligned}$$

We can also write $A \not\subseteq B$ to mean that A is not a subset of B , as in

$$\begin{aligned} \{\pi, i, e\} &\not\subseteq \{\pi, i, \phi\}, \\ [0, 1) &\not\subseteq (0, 1], \\ \mathbb{Q}^+ &\not\subseteq \mathbb{Z}, \\ \{0, 1, 2\} &\not\subseteq \{0, 1\}^*, \\ \{\emptyset\} &\not\subseteq \emptyset. \end{aligned}$$

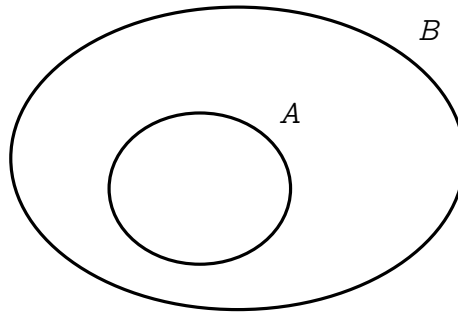
On a Venn diagram, we illustrate $A \subseteq B$ by drawing the set A entirely within the set B . See [Figure 1.1](#).

We can think of the subset relation as specifying a logical implication: $A \subseteq B$ means that membership of A implies membership of B . We can write this using the implication symbol $\Rightarrow$:

$$\text{membership of } A \Rightarrow \text{membership of } B$$

In other words, for all x , if $x \in A$ then $x \in B$:

$$x \in A \Rightarrow x \in B.$$

Figure 1.1: $A \subseteq B$.

This is sometimes read as “ $x \in A$ only if $x \in B$ ”.

Suppose we need to *prove* that $A \subseteq B$. This means we must prove that *every* member of A is also a member of B . In other words, we prove that every object that satisfies the condition for membership of A must also satisfy the condition for membership of B .

Every set is a subset of itself. Sometimes, we want to talk about subsets that are not the entire set. We say A is a **proper subset** of B if $A \subseteq B$ and $A \neq B$. So, if A is a proper subset of B , then there exists at least one element of B that is not also an element of A . We write $A \subset B$ to mean that A is a proper subset of B .

The empty set is a subset of every set (including itself). So, if B is any set, we can write $\emptyset \subseteq B$, and if B is nonempty then $\emptyset$ is a proper subset of B and we can write $\emptyset \subset B$.

If A and B are finite, then $A \subseteq B$ implies $|A| \leq |B|$. This explains the form of the subset symbol. But note that the converse does not hold in general: if $|A| \leq |B|$, it does not follow that $A \subseteq B$.

We say A is a **superset** of B , and write $A \supseteq B$, to mean that B is a subset of A . If in addition $A \neq B$ then A is a **proper superset** of B , written $A \supset B$.

All we are doing here is writing the subset relation in reverse. Similarly, we can write the implication in reverse too. So $A \supseteq B$ means that membership of A is implied by membership of B , which we can write using the reversed implication symbol:

$$\text{membership of } A \Leftarrow \text{membership of } B$$

In other words, for all x ,

$$x \in A \Leftarrow x \in B.$$

This is sometimes read as “ $x \in A$ if $x \in B$ ”.

If we need to *prove* that $A \supseteq B$, then we just need to prove that $B \subseteq A$. We discussed proving subset relations above.

If we have *both* $A \subseteq B$ and $A \supseteq B$, then the two sets must actually be identical: $A = B$. The converse certainly holds too: if $A = B$ then $A \subseteq B$ and $A \supseteq B$. This suggests a way of proving that two sets A and B are equal: prove that each is a subset of the other. So the task of proving set equality is broken down into two subtasks, each requiring proof of a subset relationship, which is usually easier to prove than equality. In fact, this is a very common strategy for proving set equality.

We can think of $A \subseteq B$ and $A \supseteq B$ as giving logical implication in both directions: membership of A implies membership of B (because $A \subseteq B$), *and is implied by* membership of B (because $A \supseteq B$). More succinctly: membership of A is equivalent to membership of B , just as we would expect because $A = B$ here. Because we have implication in both directions, $\Rightarrow$ and $\Leftarrow$, it is convenient to put them together in a single symbol, $\Leftrightarrow$, which means that implication goes in both directions:

$$\text{membership of } A \Leftrightarrow \text{membership of } B.$$

In other words, for all x ,

$$x \in A \Leftrightarrow x \in B.$$

This is often read as “ $x \in A$ if and only if $x \in B$ ”.

We will come across this “if and only if” wording often, so it is worth reflecting on its meaning. We think of it as stating two conditions that either both hold or both do *not* hold. In other words, either they both hold or neither of them holds. So the two conditions mean the same thing; they are *equivalent*.

In the case of “ $x \in A$ if and only if $x \in B$ ”, we have:

- the “**if** part”, saying that “ $x \in A$ if $x \in B$ ”, which means (writing the membership statements the other way round) that “if $x \in B$ then $x \in A$ ”, or equivalently, “ $x \in B \Rightarrow x \in A$ ”, or equivalently, “ $x \in A \Leftarrow x \in B$ ”;
- the “**only if** part”, saying that “ $x \in A$ only if $x \in B$ ”, which means that “if $x \in A$ then $x \in B$ ”, or equivalently, “ $x \in A \Rightarrow x \in B$ ”.

1.7 MINIMAL, MINIMUM, MAXIMAL, MAXIMUM

Suppose we have a set and we are interested in those subsets of it that have some specific property. For example, let B be a set of people. A **clique** in B is a set of people who all know each other. In other words, it's a subset $A \subseteq B$ such that, for each $x, y \in A$, person x and person y know each other. Given a set of people and their social links, we may wonder how “cliquey” they can be. To help us describe “peak cliques”, we make a precise distinction between the adjectives “maximum” and “maximal”.

- A **maximum clique** is a clique of maximum size; there is no larger clique.
- A **maximal clique** is a clique that is not a *proper* subset of any other clique.

Observe that these are *different concepts*, although they are related. Consider carefully the second of these, the concept of a maximal clique. Such a clique is not necessarily as large as the largest possible clique in B (although it might be). If A is a “maximal clique”, then it’s a clique with the extra property that, if we add any other person in B to the set, it’s no longer a clique: that new person will be a stranger to at least one person already in A . So A cannot be enlarged while preserving the clique property. But that does not mean it is as large as any clique in B can be. There may be other quite different cliques that are even larger than A . So a maximal clique may be smaller in size than a maximum clique.

On the other hand, a maximum clique is also a maximal clique. A clique that is largest, in size, among all possible cliques in B cannot possibly be enlarged; it cannot possibly be a proper subset of another clique, because then the latter clique would be larger in size than the former one.

So,

$$\text{maximum} \implies \text{maximal}.$$

The reverse implication does not hold in general. (Typically, there are maximal cliques that are not maximum cliques. See if you can construct an example social network where this happens. But there do exist unusual situations where every maximal clique is maximum; can you construct one?)

We make this distinction between the meanings of “maximum” and “maximal” whenever we are talking about subsets with some property.

- A **maximum subset** with the property has largest size among all subsets with the property.
- A **maximal subset** is a subset with the property that is not a *proper* subset of any other subset with the property. In other words, it cannot be enlarged while still maintaining the property.

We make a similar distinction between “minimum” and “minimal”.

- A **minimum subset** with some property has smallest size among all subsets with the property.
- A **minimal subset** with some property is a subset with the property that is not a *proper* superset of any other subset with the property. So, no proper subset has the property. In other words, if we remove anything from it, the property no longer holds.

In many situations in life, and especially if we are just talking about real numbers (rather than sets), this distinction between “maximum” and “maximal” is unnecessary (and likewise for “minimum” and “minimal”), and the terms are often treated as synonyms. What is the maximum numerical score you have ever made in your favourite game? You could replace “maximum” by “maximal” in this sentence, with no ambiguity (though it

would be less common wording in practice).¹ This is because real numbers are *totally ordered*; for every pair $x, y \in \mathbb{R}$, if $x \neq y$ then either $x < y$ or $y < x$. So, if a number has some property and cannot be increased while maintaining that property (i.e., it's maximal), then it's also the largest number with that property (i.e., it's maximum).

But the subset relation is different to the kind of order relation we are used to for real numbers. The subset relation does not give a total ordering; you can have two different sets A and B that are *incomparable* in the sense that $A \not\subseteq B$ and $B \not\subseteq A$, i.e., neither is a subset of the other. Such incomparability cannot occur among real numbers. But now that we are working with subsets, the terms “maximal” and “minimal” must be used with care, both in reading and writing. Unfortunately, they are often confused, even in technical publications in situations where the distinction matters.

From now on, we will mostly drop the underlining when using “maximum”, “maximal”, “minimum” and “minimal”. But be observant about which suffix, -um or -al, is being used, and what the usage implies.

1.8 COUNTING ALL SUBSETS

If B is a finite set, with $|B| = n$ say, how many subsets does it have? A subset of B is determined by a choice, for each element of B , of whether or not to include it in the subset. Now, B has n elements, and for each of these we have two choices. These choices are independent, in the sense that making a choice for one element puts no restrictions whatsoever on the choices we may make for other elements. So the total number of choices we make is

$$\underbrace{2 \times 2 \times 2 \times \cdots \times 2 \times 2}_{\substack{\text{for each element of } B, \\ \text{choose between two options}}}$$

which is just $2^{|B|} = 2^n$. This tells us that the number of subsets of a set grows very quickly — in fact, grows *exponentially* — as the size of the set increases.

1.9 THE POWER SET OF A SET

The **power set** of a set B is the set of all subsets of B . We denote it by $\mathcal{P}(B)$. The observations of the previous paragraph tell us that

$$|\mathcal{P}(B)| = 2^n. \quad (1.1)$$

¹ So, although a maximal clique is not necessarily a maximum clique, a maximal size clique is indeed just a maximum size clique. This is because, in “maximal/maximum size clique”, the adjective “maximal/maximum” is applied to the *size*, which is a number (and therefore part of a total order), rather than to the set itself. Nonetheless, we will avoid applying the term “maximal” to sizes and other numbers, since there we can use “maximum” which is more common.

This is true even if B is empty, when $n = 0$ and $2^n = 2^0 = 1$, in keeping with the fact that $\emptyset$ has one subset, namely itself. This expression for $|\mathcal{P}(B)|$ explains the term “power set”.

In algorithm design, we often need to find the “best” among all subsets of a set. Consider, for example, some social network analysis tasks, where we have a set of people and a set of pairs that know each other. Questions we might ask include: What is the largest clique, i.e., the largest set of people who all know each other? What is the largest set of mutual strangers? What is the smallest set of people who collectively know everyone? We could, in principle, solve these problems by examining all subsets of the set of people, or in other words, all members of its power set, provided we can easily determine, for each subset, whether or not it has the property we are interested in (being a clique, etc.). However, for reasonably large n , the number of sets to examine is prohibitive and the search would take too long. So we need to find smarter methods where we use the properties of networks and of the structures we are interested in to solve the problem without examining every single subset.

The power set of B is also often denoted by 2^B .

1.10 COUNTING SUBSETS BY SIZE USING BINOMIAL COEFFICIENTS

Sometimes we are focused on subsets of a specific size k . How many subsets of size k does a set B of size n have? This quantity is denoted by a **binomial coefficient**, written

$$\binom{n}{k}$$

and read as “ n choose k ” because we are interested in *choosing* k elements from n available elements. Between them, the binomial coefficients (taken over the full range of subset sizes, $k = 0, 1, 2, \dots, n$) count every subset of B exactly once, so we already have

$$\binom{n}{0} + \binom{n}{1} + \dots + \binom{n}{n-1} + \binom{n}{n} = 2^n.$$

This is an important and useful fact, but it does not yet give us a method for working out $\binom{n}{k}$. We now consider how to work this out.

We start with some simple cases. If $k = 0$, then we are choosing no elements at all, and this can be done in just one way, by doing nothing. (In this context, there’s only one way to do nothing!) So, for all n ,

$$\binom{n}{0} = 1.$$

At the other extreme, if $k = n$, then we choose *all* elements. Again, this can be done in only one way, because for each element of our set, we have no choice but to take it. So

$$\binom{n}{n} = 1.$$

Now suppose $k = 1$. We choose just one element from n elements, so we have n options:

$$\binom{n}{1} = n.$$

What about $k = n - 1$? This time, we are choosing one element *not* to include in our subset; once that choice is made, everything else is determined. So, again, we have n options:

$$\binom{n}{n-1} = n.$$

The symmetry we have seen here — firstly between $k = 0$ and $k = n$, and then between $k = 1$ and $k = n - 1$ — is more general. To see this, observe that deciding which elements are included also determines which elements are excluded, and vice versa. The number of ways of choosing k elements to include in our subset is the same as the number of ways of choosing k elements to exclude from our subset, which in turn is just the number of ways of choosing $n - k$ elements to include. Therefore we have

$$\binom{n}{k} = \binom{n}{n-k}. \quad (1.2)$$

We now turn to methods for counting k -element subsets in general.

Although order does not matter within a set, sometimes it is easier to count them as if order did matter, and then correct for the overcounting. It's a bit like counting pairs of socks in your drawer by counting all the socks and then dividing by two.

So let's first count all ways of choosing k different elements, *in order*, from a set of n elements. Our first element can be any one of the n elements, so we have n choices. For our second element, we must not choose the first, but our choice is otherwise unrestricted, so we have $n - 1$ choices. Our third element can be anything except the first two already-chosen ones, so we have $n - 2$ choices. This process continues, so that when we come to choose the i -th element (where $1 \leq i \leq k$), we have $n - i + 1$ choices. At the very end, for our k -th element, we have $n - k + 1$ choices. So, altogether we have

$$\# \text{ ways to choose } k \text{ elements in order} = n \cdot (n-1) \cdot (n-2) \cdots (n-k+1). \quad (1.3)$$

When $k = n$ we are just asking for the number of ways in which n elements can *all* be chosen in order, and that is just the **factorial** of n , written $n!$ and defined by

$$n! = n \cdot (n-1) \cdot (n-2) \cdots 3 \cdot 2 \cdot 1.$$

In fact, factorials allow a different way of writing (1.3):

$$\# \text{ ways to choose } k \text{ elements in order} = \frac{n!}{(n-k)!}. \quad (1.4)$$

Compare the number of arithmetic operations in each of these expressions, (1.3) and (1.4). It will be evident that the first expression, (1.3), is more efficient. The second is still important in understanding and using these counting problems, though.

We now return to our main aim of computing $\binom{n}{k}$, which counts the *unordered* choices of k elements from n elements.

Our ordered counting above will count each subset of size k some number of times. In fact, our sequence of choices was designed to count every possible ordering of the k elements exactly once. How many orderings are there? Since we drew these elements from a set (namely B), and each element of B is chosen at most once, all these chosen elements must be distinct. So there is no possibility of any of them looking identical to each other. So there are $k!$ ways to order the k elements, and therefore each subset of k elements gets counted $k!$ times by this process. Since this overcounting factor $k!$ is the same for all subsets of size k , we have

$$\# \text{ ways to choose } k \text{ elements in order} = k! \cdot (\# \text{ ways to choose a subset of } k \text{ elements}).$$

Since $\binom{n}{k}$ counts the subsets of size k in a set of n elements, it follows that

$$\begin{aligned} \binom{n}{k} &= \# \text{ ways to choose a subset of } k \text{ elements} \\ &= \frac{1}{k!} \cdot (\# \text{ ways to choose } k \text{ elements in order}) \\ &= \frac{n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot (n-k+1)}{k!} \quad (\text{using (1.3)}) \end{aligned} \quad (1.5)$$

$$= \frac{n!}{(n-k)!k!} \quad (\text{using (1.4)}). \quad (1.6)$$

Again, compare the number of arithmetic operations in the expressions (1.5) and (1.6), and consider which would be more efficient for computation. It is also worth thinking about the order in which the various multiplications and divisions are done. It makes no difference mathematically, but on a computer the order of operations can affect the accuracy of the result, because of limitations on the sizes and precision of numbers stored in the computer. In particular, the calculation works better, in general, if intermediate numbers used during the computation are not too large or small in magnitude. So, how can the computation be organised to best keep the sizes of those intermediate numbers under control?

A couple of special cases deserve special treatment because of their ubiquity in the analysis of algorithms and data structures.

$$\begin{aligned}\binom{n}{2} &= \frac{n(n-1)}{2}, \\ \binom{n}{3} &= \frac{n(n-1)(n-2)}{6}.\end{aligned}$$

Counting subsets of a given size can also be done *recursively*. A **recursive** method for doing a task is one based on breaking the task down into simpler tasks *of the same type*. In this case, our task is to count the subsets, of a given size, in a given set. How can we reduce this to simpler subset-counting tasks?

Consider again our set B of size n and suppose we want to determine the number $\binom{n}{k}$ of k -element subsets of B . Let $b \in B$. We divide the k -element subsets of B into those that include b and those that do not. How many of each kind do we have?

Let's work through an example. Suppose $B = \{1, 2, 3, 4, 5\}$, so $n = 5$, and $k = 3$. So we want the number $\binom{5}{3}$ of 3-element subsets of B . (This example is small enough that you can just list these by hand, so please do so! It will be a handy check on what we are about to do.) Pick $b \in B$, say $b = 1$. Some 3-element subsets of B include 1, others do not. The point is that

$$\begin{aligned}\text{total \# 3-element subsets} &= \quad \# \text{ 3-element subsets that include 1} \\ &+ \# \text{ 3-element subsets that do not include 1}.\end{aligned}$$

So we have expressed the answer to our subset-counting problem in terms of answers to *simpler* subset-counting problems. Furthermore, these simpler subset-counting problems are of *the same type*:

- Observe that choosing a 3-element subset that includes 1 is really just choosing the rest of the subset that isn't 1, and we need exactly two of those non-1 elements to make up three elements altogether. So, counting 3-element subsets that include 1 is the same as counting 2-element subsets of the four-element set $\{2, 3, 4, 5\}$. So

$$\# \text{ 3-element subsets that include 1} = \binom{4}{2}.$$

- Observe that choosing a 3-element subset that does not include 1 is really just choosing three elements from among the non-1 elements. So, counting 3-element subsets that don't include 1 is the same as counting 3-element subsets of the four-element set $\{2, 3, 4, 5\}$. So

$$\# \text{ 3-element subsets that don't include 1} = \binom{4}{3}.$$

So

$$\begin{aligned}
 \binom{5}{3} &= \text{total \# 3-element subsets of } B \\
 &= \text{\# 3-element subsets that include 1} \\
 &\quad + \text{\# 3-element subsets that do not include 1} \\
 &= \binom{4}{2} + \binom{4}{3}.
 \end{aligned}$$

Now let's look at how it works in general.

- For those k -element subsets that include b , we choose any $k-1$ elements from among all elements of B other than b . So we must choose $k-1$ elements from $n-1$ available elements. This can be done in $\binom{n-1}{k-1}$ ways. (We also choose b , to complete our k -element subset, but there's only one way to do that!)
- For those k -element subsets that do not include b , we choose all k elements for our subset from among all elements of B other than b . So we now choose k elements from $n-1$ available elements. This can be done in $\binom{n-1}{k}$ ways.

The total number of k -element subsets is obtained by adding these two quantities together. So we have

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}. \quad (1.7)$$

So we can compute $\binom{n}{k}$ by doing two simpler computations of the same type (each with $n-1$ instead of n) and adding the results. Those two simpler computations can, in turn, be done in terms of other even simpler computations (with $n-2$), and so on. Eventually, the numbers get so small that we can use the simple cases $k=0$ and $k=n$, which are so simple that they can be solved without reducing them any further. We call these the **base cases**: they sit at the “base” of the whole reduction process, ensuring that the process does stop eventually, instead of just “descending forever”.

It is worth comparing this method of computing $\binom{n}{k}$ with direct computation using (1.5) or (1.6).

This recursive method is especially useful when you want to compute $\binom{n}{k}$ for all n and k up to some limits. We start with base cases $\binom{n}{0} = \binom{n}{n} = 1$. The simplest case not covered by these is $\binom{2}{1}$, and applying (1.7) gives $\binom{2}{1} = \binom{1}{0} + \binom{1}{1} = 1 + 1 = 2$. The next simplest cases are $\binom{3}{1}$ and $\binom{3}{2}$. For the first of these, (1.7) gives $\binom{3}{1} = \binom{2}{0} + \binom{2}{1} = 1 + 2 = 3$; the second can be computed similarly, or even better, we can use symmetry: $\binom{3}{2} = \binom{3}{3-2}$, by (1.2), which we just calculated to be 3. Similar calculations for $n=4$, using the values we have just worked out, give $\binom{4}{1} = 4$, $\binom{4}{2} = 6$, and $\binom{4}{3} = 4$. And so on.

We can visualise the relation (1.7) using **Pascal's triangle**, shown symbolically in Figure 1.2a and with some actual values in Figure 1.2b. The binomial coefficients $\binom{n}{k}$ are arranged so that each one is the sum of the two immediately above it. In general, $\binom{n}{k}$ has $\binom{n-1}{k-1}$ and $\binom{n-1}{k}$ just above it, in that order, with $\binom{n-1}{k-1}$ to its upper left and

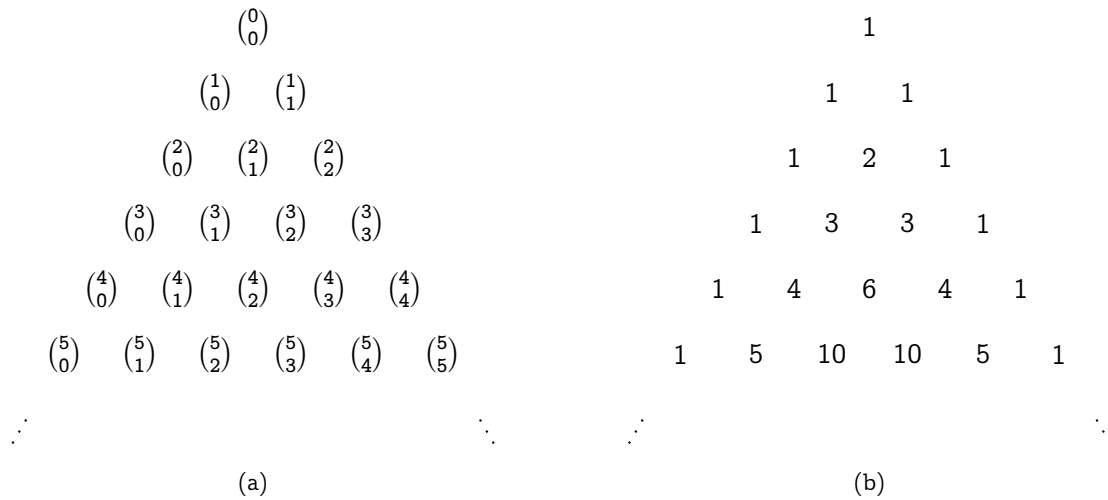


Figure 1.2: Pascal's triangle.

$\binom{n-1}{k}$ to upper right, and we saw in (1.7) that adding these two gives $\binom{n}{k}$. To take a specific example, consider $\binom{5}{2}$ in Figure 1.2a. We know from (1.7) that $\binom{5}{2} = \binom{4}{1} + \binom{4}{2}$, and we see in the triangular array in Figure 1.2a that $\binom{4}{1}$ and $\binom{4}{2}$ sit just above $\binom{5}{2}$. The actual values $\binom{5}{2} = 10$, $\binom{4}{1} = 4$ and $\binom{4}{2} = 6$ are shown in the corresponding positions in the triangular array in Figure 1.2b. The equation $\binom{5}{2} = \binom{4}{1} + \binom{4}{2}$ becomes $10 = 4 + 6$.

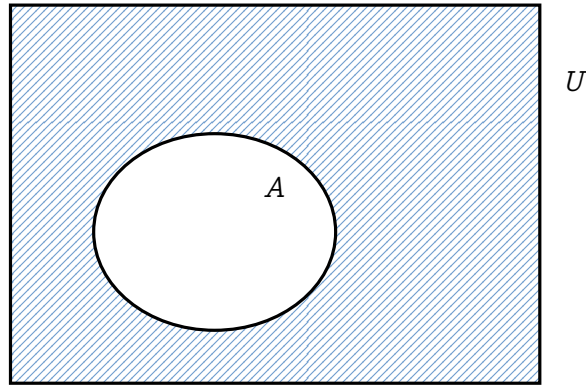
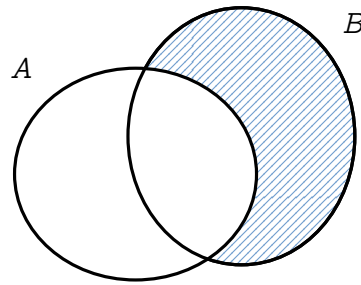
1.11 COMPLEMENT AND SET DIFFERENCE

Often, the sets we are discussing may all be subsets of some **universal set**, also called the **universe of discourse** or simply the **universe**.

For example, if we are working with various sets of integers (such as the even integers, or the odd integers, or the negative integers, or the primes), then the set $\mathbb{Z}$ of all integers can be used as the universal set. If we are working with sets of strings over the English alphabet A (such as the set of nouns, or the set of three-letter strings, or the set of names in the FIT1058 class list), then the set A^* of all strings over that alphabet may be a suitable universal set.

Suppose A is any set and U is some universal set, so that $A \subseteq U$. Then the **complement** of A , denoted by $\overline{A}$, is the set of all elements of U that are not in A . See Figure 1.3.

The notation $\overline{A}$ has the shortcoming that it does not include the universal set U , even though the definition depends on U . This is ok if the universal set has been clearly stated earlier or is clear from the context. But there is alternative notation that makes the dependence on U clear. We write $U \setminus A$ for everything (in the universal set) that is not in A . So $\overline{A} = U \setminus A$.

Figure 1.3: The complement $\bar{A}$, shaded.Figure 1.4: The set difference $B \setminus A$, shaded.

When A and U are finite sets, the size of $\bar{A}$ is given by

$$|\bar{A}| = |U| - |A|. \quad (1.8)$$

Taking the complement of the complement gives the original set:

$$\begin{aligned} \overline{\bar{A}} &= A, \\ U \setminus (U \setminus A) &= A. \end{aligned}$$

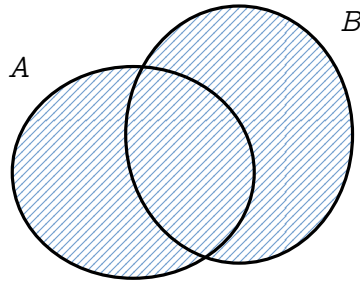
The operation $\setminus$ is called **set difference** and can be used between any two sets. So, if A and B are any sets, then $B \setminus A$ is the set of elements of B that are not in A :

$$B \setminus A = \{x \in B : x \notin A\}.$$

See [Figure 1.4](#).

If $A \subseteq B$, then

$$|B \setminus A| = |B| - |A|. \quad (1.9)$$

Figure 1.5: The union $A \cup B$, shaded.

In the special case when U is the universal set, this equation is just (1.8).

The size of the set difference does not satisfy (1.9) unless $A \subseteq B$. Why is this? How would you modify (1.9) so that it covers any set difference $B \setminus A$? What extra information about the sets would you need, in order to determine $|B \setminus A|$?

The subset and superset relations are complementary in a precise sense:

$$A \subseteq B \iff \overline{A} \supseteq \overline{B}. \quad (1.10)$$

This gives us another approach to proving that $A \subseteq B$ (as well as the approach described on p. 8 in § 1.6). Instead of taking a general member x of A and proving that it also belongs to B , we could take a general nonmember of B and prove that it also does not belong to A . In other words, we show that, every time the condition for membership of B is violated, then the condition for membership of A must be violated too.

1.12 UNION AND INTERSECTION

The **union** $A \cup B$ of two sets A and B is the set of all elements that belong to at least one of the two sets:

$$A \cup B = \{x : x \in A \text{ or } x \in B\}. \quad (1.11)$$

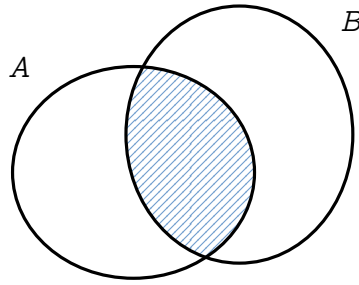
The “or” here is inclusive in the sense that it includes the possibility that $x \in A$ and $x \in B$ are *both* true. This is how we will use the word “or” in set definitions and logical statements, unless stated otherwise at the time.

The union is illustrated in Figure 1.5.

The **intersection** $A \cap B$ of two sets A and B is the set of all elements that belong to both the two sets:

$$A \cap B = \{x : x \in A \text{ and } x \in B\}. \quad (1.12)$$

See Figure 1.6.

Figure 1.6: The intersection $A \cap B$, shaded.

Intersection gives us another way of writing set difference.

$$B \setminus A = \overline{A} \cap B.$$

When we count all the elements of A and all the elements of B , we are counting everything in either set except that everything in both sets is counted twice. Therefore

$$|A| + |B| = |A \cup B| + |A \cap B|. \quad (1.13)$$

This means that, if we know $|A|$ and $|B|$, then knowing either one of $|A \cup B|$ and $|A \cap B|$ will enable us to determine the other.

An important special case is when two sets A and B are **disjoint**, meaning that $A \cap B = \emptyset$. In that case, the size of the union is just the sum of the sizes of the members. The notation $\sqcup$ is sometimes used for the **disjoint union** of A and B , which is the ordinary union when the sets are disjoint, but is undefined otherwise:

$$A \sqcup B = \begin{cases} A \cup B, & \text{if } A \cap B = \emptyset; \\ \text{undefined,} & \text{otherwise.} \end{cases}$$

See [Figure 1.7](#). There are some alternative symbols for disjoint union, the most common being obtained from the ordinary union symbol by placing a dot over it or $+$ inside it: $\dot{\cup}$ and $\uplus$.

When the disjoint union is defined, its size is just the sum of the sizes of the sets:

$$|A \sqcup B| = |A| + |B|. \quad (1.14)$$

We will use disjoint union occasionally, but mostly will focus on the normal, and more general, union.

The complement of the union of two sets is the intersection of their complements.

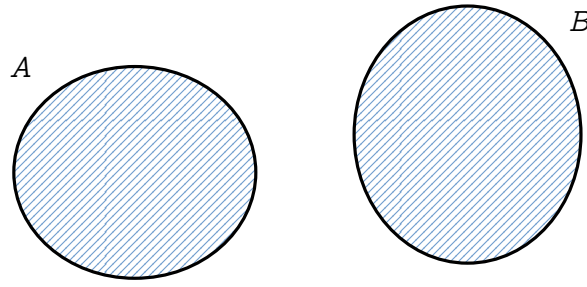


Figure 1.7: The disjoint union $A \sqcup B$, shaded. It is only defined when A and B are disjoint.

Theorem 1.

$$\overline{A \cup B} = \overline{A} \cap \overline{B}.$$

Proof.

$$\begin{aligned} x \in \overline{A \cup B} &\iff x \notin A \cup B \\ &\iff x \notin A \text{ and } x \notin B \\ &\iff x \in \overline{A} \text{ and } x \in \overline{B} \\ &\iff x \in \overline{A} \cap \overline{B} \end{aligned}$$

□

Similarly, the complement of the intersection of two sets is the union of their complements. We could prove this in a similar way, but we can prove it even more easily using [Theorem 1](#).

Corollary 2.

$$\overline{A \cap B} = \overline{A} \cup \overline{B}.$$

Proof.

$$\begin{aligned} \overline{A \cap B} &= \overline{\overline{\overline{A \cap B}}} \\ &= \overline{\overline{A \cup B}} && \text{(by Theorem 1)} \\ &= \overline{A \cup B}. \end{aligned}$$

□

[Theorem 1](#) and [Corollary 2](#) are known as **De Morgan's Laws for Sets**. They describe a *duality* between union and intersection. We will meet a similar duality later, when studying logic.

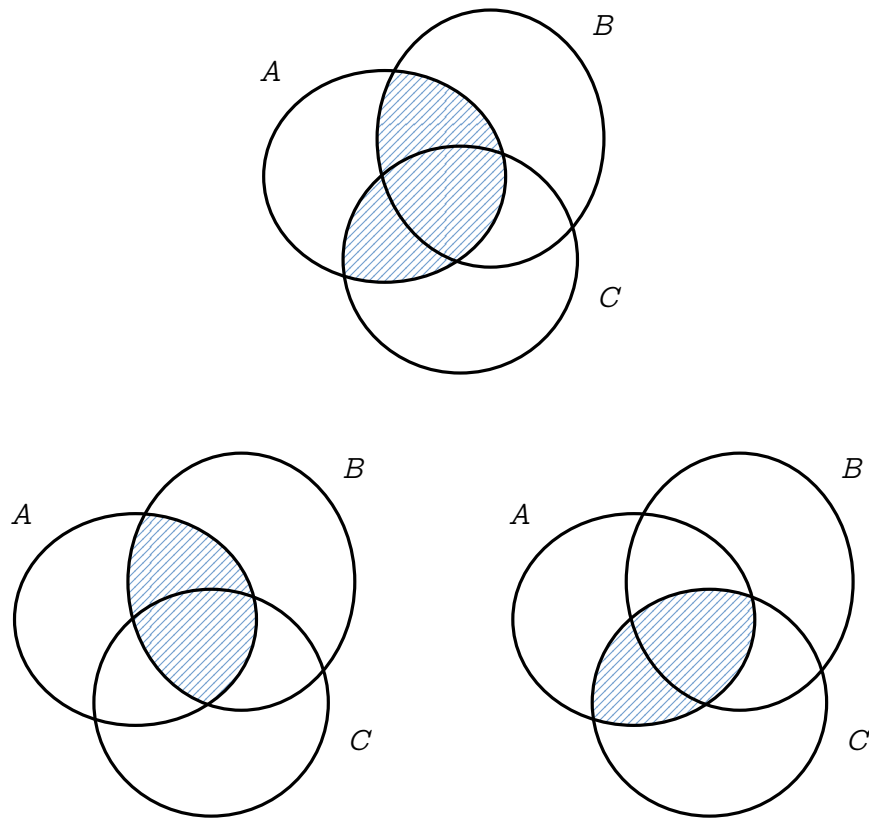


Figure 1.8: $A \cap (B \cup C)$ (top); compare with $A \cap B$ (left) and $A \cap C$ (right), and observe that $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.

How do union and intersection interact *with each other*? If we take the union of two sets, and then the intersection with a third, what happens? What about taking an intersection first, then a union?

Consider $A \cap (B \cup C)$, shown in [Figure 1.8](#). It is evident from the Venn diagrams that this is the same as $(A \cap B) \cup (A \cap C)$.

Now consider $A \cup (B \cap C)$. It is a good exercise to draw Venn diagrams to show how this relates to $(A \cap B) \cup (A \cap C)$.

In summary, we have the following.

Theorem 3. For any sets A , B and C ,

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C), \quad (1.15)$$

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C). \quad (1.16)$$

□

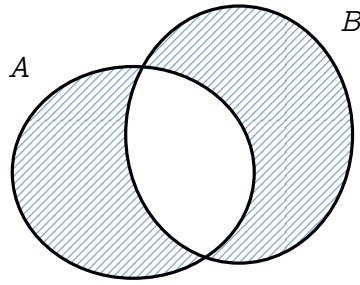


Figure 1.9: The symmetric difference $A \Delta B$, shaded.

Equations (1.15) and (1.16) are known as **the Distributive Laws for sets**. The first law, (1.15), is sometimes described as saying that “intersection distributes over union”. This means that, when taking the intersection of A with a union of several other sets, we can “distribute” the intersection among those other sets, taking all the intersections separately, and then take the union. Similarly, the second law, (1.16), is sometimes described as saying that “union distributes over intersection”. We will meet very similar Distributive Laws later, in logic. As for De Morgan’s Laws, the algebra of sets will be seen to mirror the algebra of logic.

Although this may be a new Distribute Law for you, the notion of a Distributive Law should be familiar. You already know a Distributive Law for *numbers*. For any real numbers a, b, c , we have

$$a \times (b + c) = (a \times b) + (a \times c).$$

So multiplication distributes over addition. But, for numbers, *addition does not distribute over multiplication*: in general,

$$a + (b \times c) \neq (a + b) \times (a + c).$$

(There are some cases where equality just happens to hold here, but they are atypical and very rare.) So it is refreshing to work with sets, where the two operations are distributive in all possible ways!

1.13 SYMMETRIC DIFFERENCE

The **symmetric difference** $A \Delta B$ of A and B is the set of elements that are in *exactly one* of A and B .

$$A \Delta B = \{x : x \in A \text{ or } x \in B \text{ but not both}\}.$$

The “or” here is now exclusive in the sense that the possibility of belonging to both sets is excluded. See Figure 1.9.

There are other ways of writing the symmetric difference in terms of our other operations.

$$A \Delta B = (A \setminus B) \cup (B \setminus A) \quad (1.17)$$

$$= (A \cap \bar{B}) \cup (\bar{A} \cap B),$$

$$A \Delta B = (A \cup B) \setminus (A \cap B). \quad (1.18)$$

The symmetric difference of a set with itself is the empty set,

$$A \Delta A = \emptyset.$$

This is the only situation where the symmetric difference of two sets is empty. So the symmetric difference enables a neat characterisation of when two sets are identical.

Theorem 4. For any two sets A and B , they are identical if and only if their symmetric difference is empty.

Proof.

$$\begin{aligned} A = B & \iff A \subseteq B \text{ and } B \subseteq A \\ & \iff A \setminus B = \emptyset \text{ and } B \setminus A = \emptyset \\ & \iff (A \setminus B) \cup (B \setminus A) = \emptyset \\ & \iff A \Delta B = \emptyset \end{aligned}$$

□

The symmetric difference of two sets is the same as the symmetric difference of the complements.

Theorem 5.

$$A \Delta B = \bar{A} \Delta \bar{B}.$$

Proof.

$$\begin{aligned} A \Delta B &= (\bar{A} \cap B) \cup (A \cap \bar{B}) \\ &= (\bar{A} \cap \bar{\bar{B}}) \cup (\bar{\bar{A}} \cap \bar{B}) \\ &= \bar{A} \Delta \bar{B} \end{aligned}$$

□

1.14 CARTESIAN PRODUCT

If we have two objects a and b , then the **ordered pair** (a, b) consists of both of them together, in that order. You have used ordered pairs many times, for example as co-ordinates of points in the x, y -plane, or as rows in a table with two columns.

The **Cartesian product** $A \times B$ of two sets A and B is the set of all ordered pairs consisting of an element of A followed by an element of B :

$$A \times B = \{(a, b) : a \in A, b \in B\}.$$

So, if A is the set of all possible values of x , and B is the set of all possible values of y , then $A \times B$ is the set of all possible ordered pairs (x, y) of these values.

For example, if $A = \{\text{King, Queen, Jack}\}$ and $B = \{\clubsuit, \heartsuit\}$, then

$$A \times B = \{(\text{King}, \clubsuit), (\text{King}, \heartsuit), (\text{Queen}, \clubsuit), (\text{Queen}, \heartsuit), (\text{Jack}, \clubsuit), (\text{Jack}, \heartsuit)\}.$$

The Cartesian product $\mathbb{R} \times \mathbb{R}$ is the set of all coordinates of points in the plane. If, for a given community of people, P is the set of all first (or personal) names and F is the set of all family names, then $P \times F$ is the set of all pairs (*first name, family name*). This would cover all pairings of names actually used by people in that community, but would typically include many unused pairings of names too.

If A and B are both finite sets, then the size of the Cartesian product is just the product of the sizes of the two sets:

$$|A \times B| = |A| \cdot |B|. \quad (1.19)$$

This is because we have $|A|$ possibilities for the first member of a pair, and $|B|$ possibilities for the second member, and these choices are made independently of each other. In more detail, each possibility for the first member gives $|B|$ possibilities for the second member, so the total number of pairs is

$$\underbrace{|B| + |B| + \cdots + |B|}_{|A| \text{ copies}}$$

which is just $|A| \times |B|$.

The Cartesian product of three sets gives the set of all triples of the appropriate kind:

$$A \times B \times C = \{(a, b, c) : a \in A, b \in B, c \in C\}$$

For example, $\mathbb{R} \times \mathbb{R} \times \mathbb{R}$ is the set of all coordinates of points in three-dimensional space.

More generally, if we have sets $A_1, A_2, \dots, A_n$, then their Cartesian product is the set of n -tuples, or sequences of length n , in which the first member (or co-ordinate) is in A_1 , the second is in A_2 , and so on, with the i -th member belonging to A_i for all $i \in \{1, 2, \dots, n\}$:

$$A_1 \times A_2 \times \cdots \times A_n = \{(a_1, a_2, \dots, a_n) : a_1 \in A_1, a_2 \in A_2, \dots, a_n \in A_n\}.$$

Again, if all the sets are finite then the size of the Cartesian product is the product of the sizes of all the sets:

$$|A_1 \times A_2 \times \cdots \times A_n| = |A_1| \cdot |A_2| \cdot \cdots \cdot |A_n|.$$

If the sets $A_1, \dots, A_n$ are all the same, then we can use an exponent to indicate how many of them are in the product:

$$\begin{aligned} A^n &= \underbrace{A \times A \times \cdots \times A}_{n \text{ factors}} \\ &= \{(a_1, a_2, \dots, a_n) : a_i \in A \text{ for all } i \in \{1, 2, \dots, n\}\}. \end{aligned}$$

For example,

$$\{0, 1\}^3 = \{(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0), (1, 0, 1), (1, 1, 0), (1, 1, 1)\}.$$

In the special case when A is an alphabet (i.e., a finite set of characters), we often write n -tuples in A^n as strings of length n . So, for the binary alphabet $\{0, 1\}$, we can write

$$\{0, 1\}^3 = \{000, 001, 010, 011, 100, 101, 110, 111\},$$

as we did in § 1.5.

For another example, the sets of coordinates of points in two- and three-dimensional space are $\mathbb{R}^2 = \mathbb{R} \times \mathbb{R}$ and $\mathbb{R}^3 = \mathbb{R} \times \mathbb{R} \times \mathbb{R}$, respectively. These are used extensively to model physical spaces, since the space around us is three-dimensional and we often deal with surfaces (terrain, paper, screens) that are two-dimensional. Higher-dimensional spaces are also useful. The set of coordinates in n -dimensional space is

$$\mathbb{R}^n = \underbrace{\mathbb{R} \times \mathbb{R} \times \cdots \times \mathbb{R}}_{n \text{ axes}}.$$

Spaces of more than three dimensions are hard to visualise, since the physical space we live in is only three-dimensional. But they are very useful and powerful. Models developed by machine learning programs can require millions or even billions of dimensions.

For the two smallest exponents, we have, for any set A ,

$$A^0 = \emptyset, \quad A^1 = A.$$

If A is finite, the size of A^n is just $|A|^n$.

1.15 PARTITIONS

A **partition** of a set A is a set of disjoint nonempty subsets of A whose union is A . These subsets are called the **parts** (or **blocks** or **cells**) of the partition.

Equivalently, a set of nonempty subsets of A is a partition of A if and only if every member of A belongs to exactly one of the parts.

Suppose $A = \{a, b, c\}$. One possible partition of A is

$$\{ \{a, c\}, \{b\} \}.$$

This partition has two parts: $\{a, c\}$ and $\{b\}$. These parts are each nonempty, and disjoint, and their union is A , so the definition is satisfied. Another partition of A is

$$\{ \{a\}, \{b\}, \{c\} \}.$$

This partition has three parts, namely the three sets $\{a\}$, $\{b\}$, $\{c\}$. At the other extreme, we have a partition of A with just one part:

$$\{ \{a, b, c\} \}.$$

Our set A is small enough that we can list all its five partitions:

partition	# parts
$\{ \{a, b, c\} \}$	1
$\{ \{a, b\}, \{c\} \}$	2
$\{ \{a, c\}, \{b\} \}$	2
$\{ \{b, c\}, \{a\} \}$	2
$\{ \{a\}, \{b\}, \{c\} \}$	3

There are several ways in which a collection of subsets of a set can fail to be a partition. For our set $A = \{a, b, c\}$, the collection $\{\{a, b\}, \{c\}, \emptyset\}$ fails because one of its members is empty. The collection $\{\{a, b\}, \{b, c\}\}$ fails because its members are not all disjoint, in particular $\{a, b\} \cap \{b, c\} = \{b\} \neq \emptyset$, so b belongs to two members of the collection instead of just one. The collection $\{\{a\}, \{b\}\}$ fails because the union of the collection's members is not the entire set A , i.e., $\{a\} \cup \{b\} = \{a, b\} \neq A$; in particular, c does not belong to any members of this collection.

Partitions have many applications. Consider, for example, classification. Suppose that A is a collection of plant specimens. We would like to classify the specimens according to their species: specimens from the same species are grouped together, while those from different species are kept in separate groups. (Some groups may have just one specimen, if there is no other specimen of the same species.) These groups form the parts of a partition of A , with each part corresponding to one of the species represented in the collection. Finding such classifications, from data obtained from specimens, is a major topic in machine learning.

The number of partitions of a finite set grows rapidly as the size of the set increases.

set size, n	1	2	3	4	5	6	7	8	9	10
# partitions	1	2	5	15	52	203	877	4140	21147	115975

We can also talk about partitions of infinite sets. For example, $\{\{\text{even numbers}\}, \{\text{odd numbers}\}\}$ is a partition of the set of nonnegative integers; this partition has two parts. Consider also the following partition of the set A^* of all strings over a finite alphabet A :

$$\{A^n : n \in \mathbb{N}_0\}.$$

This partition has infinitely many parts, one for each $n \in \mathbb{N}_0$. The parts are the sets of all strings of a given length.

Every set A has two partitions that might be thought of as “extreme”, but in opposite directions.

- The **coarsest partition** of A is the partition $\{A\}$ which has just one part, namely the entire set A itself. In effect, everything in A is “lumped together”.
- The **finest partition** of A is the partition $\{\{a\} : a \in A\}$ which has one part for each element of A , and each part contains only that element. If A is finite, then this partition has $|A|$ parts; if A is infinite, then this partition has infinitely many parts. Every part of this partition is as small as a part of a partition can be. In effect, all the elements of A are “kept apart” from each other.

If A has just one element, then the coarsest and finest partitions are the same, but if A is larger then they are different. If A has just two elements, then these are the only partitions of A , but if A is larger, then it has many other partitions too, with all the others being in a sense intermediate between these two extreme partitions.

1.16 EXERCISES

1. Why does the set difference only satisfy (1.9) when $A \subseteq B$? How would you modify (1.9) so that it covers any set difference $B \setminus A$? What extra information about the sets would you need, in order to determine $|B \setminus A|$?

2. We mentioned at the start of this chapter that sets are used to define *types* of objects in many programming languages. For example, in C, the statement

```
int monthNumber;
```

declares that the variable `monthNumber` has type `int`. The declaration also assigns a piece of memory to the variable, to contain the values that the variable has during the computation. Similarly, the statement

```
char monthName[10];
```

is C’s way of declaring that the variable `monthName` is a *string* of at most 9 characters; again, a piece of memory is allocated to it as well.

Let `Int` be the set of possible values for a variable of type `int`. Similarly, let `String` be the set of possible values for a variable that is declared to be a string of at most 9 letters.

In C, we can combine declarations in order to create more complex types.

(a) The following statement creates a new type, called `aNewType`, for representing objects consisting of any `int` followed by any `string`; it also sets aside consecutive pieces of memory, so that the `int` is followed by the `string` in memory. It also declares the variable `monthBothWays` to be of this type.

```
struct aNewType {
    int year;
    char monthName[10];
} monthBothWays;
```

Using the sets `Int` and `String`, together with a standard set operation, what set is represented by the type `aNewType`?

(b) The following statement creates another new type, called `anotherNewType`, for representing objects that can be either an `int` or a `string`. It sets aside a piece of memory that is large enough to contain either an `int` or a `string`; at any one time, it will contain just one of these. It also declares the variable `monthEitherWay` to be of this type.

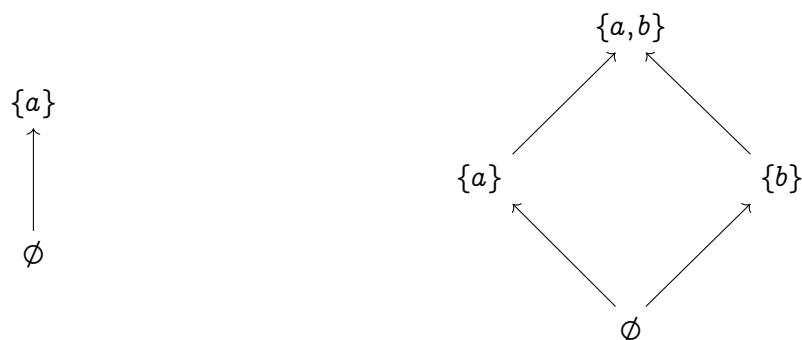
```
union anotherNewType {
    int year;
    char monthName[10];
} monthEitherWay;
```

Using the sets `Int` and `String`, together with a standard set operation, what set is represented by the type `anotherNewType`?

3. Suppose A and B are subsets of some universal set U .

- (a) If $A \subseteq B$, what is $\overline{A} \cup B$?
- (b) If $A \not\subseteq B$, what can you say about $\overline{A} \cup B$?
- (c) Complete the following: $A \subseteq B$ if and only if $\overline{A} \cup B = \underline{\hspace{1cm}}$.
- (d) Devise another equivalent condition for $A \subseteq B$ involving intersection instead of union.
- (e) Devise another equivalent condition for $A \subseteq B$ involving set difference.

4. Consider the following diagrams. The one on the left shows the set $\{a\}$ and its sole subset, $\emptyset$. The one on the right shows $\{a, b\}$ and all its subsets.



The requirements of this diagram are:

- Sets are just represented by writing them (in their usual textual form, with elements listed between curly braces). These are not Venn diagrams.
- Smaller sets are lower, on your page, than larger sets.
- Sets of the same size are shown on the same horizontal level.
- The arrows indicate when a lower set is a subset of another set that has just one extra element. If $X \subset Y$ and $|Y| = |X| + 1$ then there is an arrow from X to Y .
- The diagram is as neat and symmetric as possible.

(a) Draw a diagram of this type that shows all subsets of $\{a, b, c\}$ and the subset relation among them.

(b) For every pair of sets X, Y such that $X \subset Y$ and $|Y| = |X| + 1$, label the corresponding arrow in your diagram by the sole member of $Y \setminus X$.

(c) Suppose you are now liberated from the requirement to draw your diagram on a medium of only two dimensions such as paper or a computer screen. How could you draw this diagram in three dimensions in a natural way?

(d) For a set of n elements, how many sets and how many arrows does a diagram of this type have?

(e) For each element of the n -element set considered in (d), how many arrows are labelled by that element (if we label them as in (c))?

(f) In such a diagram, suppose we have two sets X, Y that satisfy $X \subseteq Y$. How many *directed paths* are there from X to Y ? Give an expression for this.

- A **directed path** is a path along arrows in which all arrows are directed forwards; you can't go backwards along an arrow. Paths are counted as different as long as they are not completely identical; they are allowed to have some overlap.

(g) With X, Y as in (f), what is the maximum number of *mutually internally disjoint* paths from X to Y ?

- An **internal set** on a path is a set on the path that is not the start or end of the path, i.e., it's not X or Y . Two paths are **internally disjoint** if no internal set on either path appears anywhere on the other path. If we have a collection of some number of paths (possibly more than two), then the paths are **mutually internally disjoint** if every pair of paths in the collection are internally disjoint.

(h) Explain how to use the diagram to find, for any two sets in it, their union and intersection.

5. Given $U = \{e_1, e_2, \dots, e_n\}$, the **characteristic string** of a subset A of U is the string of n bits $b_1 b_2 \dots b_n$ where the i -th bit indicates whether or not e_i belongs to A :

$$b_i = \begin{cases} 1, & \text{if } e_i \in A; \\ 0, & \text{if } e_i \notin A. \end{cases}$$

Write down the characteristic string of each of the subsets of a set of three elements. List them, one above the other, so that each differs from the one above it in just one bit.

See if you can extend this to subsets of a set of n elements.

This has algorithmic applications. Suppose we want to search through all subsets of a set, by looking at each of their characteristic strings in turn. If each characteristic string differs from its predecessor in only one bit, then moving from one characteristic string to the next requires fewer changes than may be required otherwise, which saves time.

6. In a Venn diagram, the closed curves representing the sets together divide the plane into regions. A single set divides the plane — or the portion of the plane within the rectangular box representing the universal set, if that is shown in the diagram — into two regions, its interior and exterior. See [Figure 1.3](#), where the regions correspond to A and $\bar{A}$ (with the latter shaded in that particular diagram, as it was being used to explain the complement, but that does not have to be done in general). Two intersecting sets, represented by two closed curves, divide the plane into four *basic regions*. (See [Figure 1.4](#), [Figure 1.5](#), and [Figure 1.6](#).) If the sets are A and B , then the basic regions correspond to $\bar{A}\bar{B}$, $A \cap B$, $A \setminus B$ and $B \setminus A$. Three sets can be drawn to divide the plane into eight regions.

(a) Label each basic region of a Venn diagram for three sets A, B, C with appropriate intersections of sets.

A **general Venn diagram** for n sets $A_1, A_2, \dots, A_n$ is one in which every possible intersection of the form

$$(A_1 \text{ or } \overline{A_1}) \cap (A_2 \text{ or } \overline{A_2}) \cap \dots \cap (A_n \text{ or } \overline{A_n})$$

corresponds to one of the basic regions of the diagram. This is intended to be an intersection of i sets, the i -th of which is either A_i or $\overline{A_i}$. For each i , there is a choice of two options, so the total number of basic regions is 2^n .

For example, the Venn diagrams in each of these figures are general Venn diagrams: [Figure 1.3](#), [Figure 1.4](#), [Figure 1.5](#), [Figure 1.6](#), [Figure 1.8](#), [Figure 1.9](#).

But the Venn diagram in [Figure 1.1](#) is not a *general* Venn diagram, because there is no region for $A \cap \overline{B}$ (which is fine in that case, because it is illustrating $A \subseteq B$). The Venn diagram in [Figure 1.7](#) is not a *general* Venn diagram, because there is no region for $A \cap B$ (which is fine in that case, because it is illustrating the disjoint union $A \sqcup B$).

- (b) What is the maximum number of sets for which a general Venn diagram can be drawn in which all the sets are circles of the same size?
- (c) Draw a general Venn diagram in the plane for four sets. You can use closed curves other than circles.
- (d) How could you draw a *three-dimensional* general Venn diagram using four sets, each represented as a sphere?

Drawing Venn diagrams (not necessarily general ones) to illustrate relationships among a collection of sets is one of many topics studied in *information visualisation*. There are many different criteria to aim for, including using only simple curves, not having too much variability in sizes of the basic regions, and symmetry. These criteria can conflict with each other. Algorithms have been developed and theorems have been proved.

7. Prove that if $A \subseteq B$ then $\mathcal{P}(A) \subseteq \mathcal{P}(B)$.

8. Consider the sequence of binomial coefficients $\binom{n}{r}$ with n fixed and r going from 0 to n :

$$\binom{n}{0}, \binom{n}{1}, \binom{n}{2}, \dots, \binom{n}{n-1}, \binom{n}{n}.$$

- (a) Using one of the formulas for $\binom{n}{r}$, prove that these binomial coefficients increase as r goes from 0 to $\lfloor n/2 \rfloor$ and then decrease as r goes from $\lceil n/2 \rceil$ to n .
 - Here, $\lfloor n/2 \rfloor$ is the “floor” of $n/2$, which is the greatest integer $\leq n/2$. If n is even, this is just $n/2$ itself, but if n is odd (which means $n/2$ is not an integer), its floor is the integer $(n-1)/2$.

Similarly, $\lceil n/2 \rceil$ is the “ceiling” of $n/2$, which is the least integer $\geq n/2$. If n is even, this is just $n/2$ again, but if n is odd, its ceiling is the integer $(n+1)/2$.

- So, what we are saying here is that:
 - If n is even, $\binom{n}{r}$ increases as r goes from 0 to $n/2$, then it decreases as r goes from $n/2$ to n .
 - If n is odd, $\binom{n}{r}$ increases as r goes from 0 to $(n-1)/2$, then it decreases as r goes from $(n+1)/2$ to n .
 - In the odd case, we haven't yet said anything about what happens when r goes from $(n-1)/2$ to $(n+1)/2$. See if you can work that out too.

(b) Now prove that, for every positive integer r in the range $1 \leq r \leq n-1$,

$$\binom{n}{r}^2 > \binom{n}{r-1} \binom{n}{r+1}.$$

This is an important property and means that the sequence is said to be *strictly log-concave*. (If the inequality is just $\geq$ instead of $>$, then it's *log-concave*.)

(c) Suppose you and a friend are considering a fixed set of size n . Suppose you get to choose an ordered pair of subsets of size r of the set of size n , with no restriction at all (so your two sets are allowed to overlap, or be disjoint, or be identical, or whatever; the only rule is that they both have to have size r). Suppose also that your friend gets to choose one subset of size $r-1$ and another of size $r+1$, with again no restriction on the sets apart from these size requirements. Who has more options, you or your friend? Does the answer depend on n and r in any way? If so, how? If not, why not? Is there any situation where the numbers of choices that you and your friend have are the same?

9. Express $|A \cup B|$ in terms of $|A|$, $|B|$, $|A \cap B|$.
10. Express $|A \cap B|$ in terms of $|A|$, $|B|$, $|A \cup B|$.
11. Express $|A \cup B \cup C|$ in terms of $|A|$, $|B|$, $|C|$, $|A \cap B|$, $|A \cap C|$, $|B \cap C|$, $|A \cap B \cap C|$.
12. Express $|A \cap B \cap C|$ in terms of $|A|$, $|B|$, $|C|$, $|A \cup B|$, $|A \cup C|$, $|B \cup C|$, $|A \cup B \cup C|$.
13. Suppose we have sets $A_1, A_2, \dots, A_n$. For each k , let

$$i_k := \text{sum of all sizes of } \underline{\text{intersections}} \text{ of } k \text{ of the sets.}$$

So i_1 gives just the sum of the sizes of the n sets, and i_2 gives the sum of the sizes of each intersection of two of the n sets, and so on.

For example, if $n = 3$, our sets are A_1, A_2, A_3 , and

$$\begin{aligned} i_1 &= |A_1| + |A_2| + |A_3|, \\ i_2 &= |A_1 \cap A_2| + |A_1 \cap A_3| + |A_2 \cap A_3|, \\ i_3 &= |A_1 \cap A_2 \cap A_3|. \end{aligned}$$

Rewrite your expressions for $|A \cup B|$ (for which $n = 2$) and $|A \cup B \cup C|$ (for which $n = 3$) in terms of $i_1, i_2, \dots$.

Then write out a general expression for $|A_1 \cup A_2 \cup A_3 \cup \dots \cup A_n|$. Pay careful attention to the sign of the coefficient of the last term in the sum.

For this exercise, you don't need to *prove* that your expression is correct. You will learn techniques for doing that later, in [Chapter 3](#). But, if you do try to do a proof (even if only briefly), you will be even better prepared for that later material.

14. Now let

$$u_k := \text{sum of all sizes of } \underline{\text{unions}} \text{ of } k \text{ of the sets.}$$

So, for example, if $n = 3$, with the same sets as before, we have

$$\begin{aligned} u_1 &= |A_1| + |A_2| + |A_3|, \\ u_2 &= |A_1 \cup A_2| + |A_1 \cup A_3| + |A_2 \cup A_3|, \\ u_3 &= |A_1 \cup A_2 \cup A_3|. \end{aligned}$$

Rewrite your expressions for $|A \cap B|$ and $|A \cap B \cap C|$ in terms of $u_1, u_2, \dots$.

Then write out a general expression for $|A_1 \cap A_2 \cap A_3 \cap \dots \cap A_n|$. Pay careful attention to the sign of the coefficient of the last term in the sum.

For this exercise, you don't need to prove that your expression is correct.

15. Express $|A \triangle B|$ in terms of the sizes of some other sets in three different ways.

16. Draw a Venn diagram for general sets A, B, C and shade the region(s) that form $A \triangle B \triangle C$.

17. Prove that, for any sets A and B ,

$$(A \cup B) \triangle (A \cap B) = A \triangle B.$$

18. Suppose $A_1, A_2, \dots, A_n$ are sets. How would you characterise $A_1 \triangle A_2 \triangle \dots \triangle A_n$? More specifically, the members of $A_1 \triangle A_2 \triangle \dots \triangle A_n$ are those that satisfy some specific condition on how many of the sets $A_1, A_2, \dots, A_n$ they belong to. What is that condition?

For this exercise, you don't need to construct a formal proof that your condition is correct.

19. If A and B are sets, does $\overline{A} \times \overline{B}$ equal $\overline{A \times B}$? If so, prove it. If not: give an example when it does not hold; characterise those cases when it does hold; and determine what can be said about the relationship between them.

20. Write down all partitions of $\{1,2,3,4\}$ into three parts.

21. Suppose $A, B, C \subseteq U$ are nonempty sets such that all basic regions in their general Venn diagram are nonempty (see Exercise 6). Then the three sets A, B, C define a partition of U into eight parts. What are those parts? Express each part in terms of one or more of A, B, C using set operations.

22. Does $\mathbb{R}$ have a partition in which all parts are

(a) open intervals?

(b) half-open half-closed intervals?

(c) closed intervals?

FUNCTIONS

Computation helps get things done. By “things”, we mean tasks where we take some information and transform it somehow. To specify such a task, we must specify:

- what we start with;
- what we end up with;
- what needs to be done.

For example, suppose we want to sort a list of names into alphabetical order. We first specify exactly what kinds of lists of names we are dealing with (which alphabet? any requirements or restrictions on the names to be considered? how long can the names be? how many names are we allowed to have in the list? etc.). Then we specify that we’ll end up with another list. Then we specify what we want to do to our first list, which is to sort it. So the list we end up with has all the same names as the list we started with, but now they are in alphabetical order.

Or suppose you want to determine the most goals kicked by any team in any season of your favourite football competition, from records of games. We first specify exactly what kinds of records we are working with: the information in them and how it is arranged. Then we specify that we’ll end up with a number (to be precise, a non-negative integer). Then we specify that this number must be the maximum, over all seasons and all teams, of the number of goals kicked by that team in that season.

To model such tasks precisely, we use *functions*.

2.1^α DEFINITIONS

A **function** f consists of

- a set called the **domain**,
- a set called the **codomain**,
- a specification, for each element x of the domain, of a unique member $f(x)$ of the codomain. This member, $f(x)$, is the **value** of the function for the **argument** x .

Informally, the argument x “goes in” and the value $f(x)$ “comes out”. These terms have some common synonyms.

- The *argument* x is also called the *parameter* or the *input*.
- The *value* $f(x)$ is also called the *output*.

But some care is needed with the use of “input” and “output” in this context, since in programming “input” is often used for extra information that a program reads from another source (such as a file), while “output” is often used for information a program writes to a file or screen, and these may be quite different to the argument and value.

Functions are ubiquitous in computer science, as well as in science, engineering, economics, finance, and any other field where symbolic objects of some type need to be transformed into something else in a precisely specified way.

We need to make some key points about the three parts of the definition of a function f .

2.1.1^α Domain

The **domain** of a function f can be any set, finite or infinite, provided that:

- Every member of the domain is considered to be a valid argument for the function. So, for every member x of the domain, its corresponding value $f(x)$ must be properly defined and “make sense”.
- For everything that does *not* belong to the domain, the function is considered to be undefined. So, if x does not belong to the domain then $f(x)$ has no meaning.

For our Sorting example, the domain is the set of all possible lists of names of the required type.

For the Maximum Goals example, it is the set of all possible records of all the games in a single season. Note that the domain is not just the set of all past seasons’ records, since we want our function to be able to determine the required information for any possible future season as well.

Suppose now that we have a function called `SumOfFourIntCubes` which takes any four integers and finds the sum of their cubes, according to the rule

$$\text{SumOfFourIntCubes}(w, x, y, z) = w^3 + x^3 + y^3 + z^3.$$

Then its domain is the set of all quadruples of integers, which we can write as $\mathbb{Z} \times \mathbb{Z} \times \mathbb{Z} \times \mathbb{Z}$. We could also envisage a function `SumOfFourRealCubes` that takes any four real numbers and finds the sum of their cubes. The rule looks the same:

$$\text{SumOfFourRealCubes}(w, x, y, z) = w^3 + x^3 + y^3 + z^3.$$

But its domain is now $\mathbb{R} \times \mathbb{R} \times \mathbb{R} \times \mathbb{R}$. Since this function has a different domain, it is considered to be a different function, even though the rule looks the same.

Why are functions with the same rule considered to be different if their domains are different? Why would we even need the function `SumOfFourIntCubes` when we can use the function `SumOfFourRealCubes` to do everything it does and more?

There are several reasons for this.

- Firstly, rules may *look* the same without actually *being* the same, because of details of how the operations in the rule depend on the types of objects being used. A multiplication symbol might be used to multiply numbers and also to multiply matrices, but they are very different operations.
- Secondly, the details of implementing functions in code will depend crucially on the specific types of objects they work with. In computing, real numbers are represented very differently to integers, even though we think of integers mathematically as a special case of real numbers. So some of the technical details of writing a program will be different for `SumOfFourIntCubes` and `SumOfFourRealCubes`. So, when we think of functions as specifications of tasks to be programmed, the specification of the domain is a crucial part of the definition.
- Thirdly, the purpose of the domain is not merely to help explain the rule; it is also a promise to the function's user that the function will work for every member of the domain. Sometimes, you may want a larger domain so that the function works for as many cases as possible. But bigger promises require more work to keep! Sometimes, it is better to use a more modest domain, provided it still captures everything that your function is supposed to work with; a more modest promise is easier to keep!

If f is a function, then we write $\text{dom}(f)$ for the domain of f . So

$$\begin{aligned}\text{dom}(\text{SumOfFourIntCubes}) &= \mathbb{Z} \times \mathbb{Z} \times \mathbb{Z} \times \mathbb{Z}, \\ \text{dom}(\text{SumOfFourRealCubes}) &= \mathbb{R} \times \mathbb{R} \times \mathbb{R} \times \mathbb{R}.\end{aligned}$$

2.1.2^α Codomain & co.

The **codomain** of a function f must include every possible value of $f(x)$ for every member x of the domain. But the codomain is allowed to be “loose”, in the sense that it is allowed to include other stuff too. We do not need to ensure that the *only* things in the codomain are things we can get by applying our function f to some member of its domain.

So, for the codomain, instead of specifying the possible function values *exactly*, we specify some *superset* of the set of possible function values.

The exact set of possible values of a function is called the **image** of the function. This is a subset of the codomain, but not necessarily a proper subset.

You may wonder at this point, why do we allow such “looseness” in the codomain when we were so insistent that the domain be the exact set of allowed arguments of the function? Why shouldn’t we use the image, instead of the codomain, when defining a function?

The reason for this is practical. It is often harder to know the image than it is to specify a natural codomain. Sometimes it’s impossible.

In our Maximum Goals example above, the codomain is the set $\mathbb{N} \cup \{0\}$. This does not mean that *every* nonnegative integer *must* arise as a possible maximum number of goals kicked by any team in a season. Indeed, some numbers (e.g., 10^{100}) could never arise in this way in practice. But it may be hard or impossible to know exactly which numbers are feasible values and which are not. So it is more practical to give a codomain, in the form of a simple, easily-described set which we know includes all possible function values (even if it has other things too). The set $\mathbb{N} \cup \{0\}$ works well.

In some cases, the difficulty may be computational. For example, consider the notorious **Pompous Python** function which takes, as its argument, any positive integer n and gives, as its value, the length (in characters) of the longest output that can be printed by *any* Python program which reads no input, eventually stops, and whose source code file has at most n characters.¹ On the face of it, this seems painful to compute, because there are so many programs that could be considered (once n is large enough to allow interesting programs of at most n characters to be written). In fact, it’s worse than that; it can be shown that this function is *impossible* to compute perfectly. (This fact is not obvious, but is a consequence of some famous results on uncomputability from the 1930s. Uncomputability is covered in detail in the unit FIT2014 Theory of Computation.) So it is impossible, in a precise sense, to know exactly which numbers can be possible values of this function. Therefore, in specifying the function, we would prefer not to have to specify, in advance, which numbers to allow for its possible values! So, instead, we specify a suitable codomain, and in this case $\mathbb{N} \cup \{0\}$ will do fine, even though only very few numbers can actually be values of the Pompous Python function.

In other cases, the difficulty may be the limits of our current knowledge. Consider again SumOfFourIntCubes. It is not yet known whether or not every integer can be written as the sum of four cubes, so we do not know if the values of this function are all integers or only some subset of them. But we can specify the codomain to be $\mathbb{Z}$ and know that this is a superset (but not necessarily a proper superset, as far as we currently know) of the actual set of possible values.

The codomain, then, represents a *promise* to users of the function that all the values they get from it will be in that set, but it gives no guarantee that *every* one of its members is an actual function value.

¹ Here we envisage an ideal computing environment where arbitrarily long programs can be run, arbitrarily long outputs can be printed, and programs can take an arbitrarily long time to run before stopping. If a program crashes or prints no output, then we define the length of its output to be 0. If a program runs forever, in an infinite loop, then we exclude it from consideration, regardless of how much output it produces.

It is good to know the image too, when that's possible. But we do not want our definition of the function concept to be hampered by the difficulties that are often associated with specifying the image. So we do not include a specification of the image in our specification of the function. Rather, we use the codomain as the “next best thing”.

Of course, if we do know the image, there is nothing to stop us from stating it as our codomain. But, even then, it is often neater to specify a simple codomain than it is to specify the image. For example, a function with domain $\mathbb{N}$ whose value is the n -th Fibonacci number has, as its image, the set of all Fibonacci numbers, and it's easy enough to write that down.² But it's even easier to specify $\mathbb{N}$ as a codomain.

A function in which the codomain is the same as the image of the function is said to map its domain **onto** its codomain, and may be said to be **onto**. Such a function is also said to be **surjective** and is called a **surjection**.

Finally, a word of warning! We have studiously avoided using the word “range”. This is because the term is, confusingly, used in two different ways: sometimes, it means the image, while at other times, it means the codomain. We will dodge this issue by not using “range” at all.

2.1.3^α Rule

The rule of a function must specify the relationship between each member of its domain and the corresponding value of the function. But it does not, in general, give an *algorithm* for computing the function.

In other words, the rule specifies *what* must be done, but it does not need to specify *how* it is to be done.

In our Sorting example, the rule is that the function's value is the sorted list of names. This rule does not specify *how* the sorting is to be done. As computer science students, you will meet many different sorting algorithms, including Bucket Sort, Merge Sort, Insertion Sort, and Quick Sort. They all have their strengths and all could be used to compute our sorting function. But the *function itself* does not include a choice of which algorithm we will use to compute it; that choice is a separate issue to the specification of the function.

Sometimes, a function's rule does give some information on how to compute it. For example, consider a function that squares integers. It has domain $\mathbb{Z}$, and we'll use the (loose) codomain $\mathbb{Z}$ too. Its rule is just that any integer argument is squared. This may be thought of as a small algorithm: it tells you what the value is in such a way that you also know how to work it out. Or do you? There is more than one algorithm for squaring an integer! To specify the rule, we don't need to specify which algorithm is to

² The **Fibonacci numbers** are the numbers you get by starting with two consecutive 1s and then repeatedly adding the two most recent numbers together to get the next number. So the **Fibonacci sequence** is: 1,1,2,3,5,8,13,21,34,55,89,144,233,....

be used; we only need to give enough information so that the reader can know, for each argument, what its corresponding value is.

A function's rule associates, to each argument in its domain, a unique value in its codomain. One way to specify this information is to give the set of all possible ordered pairs $(x, f(x))$. For example, consider a function *Employer*, defined as follows. Its domain is the set

$\{\text{Annie Jump Cannon, Henrietta Swan Leavitt, Muriel Heagney, Winsome Bellamy}\}$,

which consists of four human computers who worked at various astronomical observatories. For its codomain, we use the set of *all* astronomical observatories over the last two centuries. The rule gives values to arguments as follows.

$$\begin{aligned}\text{Employer}(\text{Annie Jump Cannon}) &= \text{Harvard College Observatory,} \\ \text{Employer}(\text{Henrietta Swan Leavitt}) &= \text{Harvard College Observatory,} \\ \text{Employer}(\text{Muriel Heagney}) &= \text{Melbourne Observatory,} \\ \text{Employer}(\text{Winsome Bellamy}) &= \text{Sydney Observatory.}\end{aligned}$$

We can specify this rule by giving the following set of ordered pairs $(\text{computer}, \text{Employer}(\text{computer}))$.

$$\begin{aligned}\{ & (\text{Annie Jump Cannon}, \text{Harvard College Observatory}), \\ & (\text{Henrietta Swan Leavitt}, \text{Harvard College Observatory}), \\ & (\text{Muriel Heagney}, \text{Melbourne Observatory}), \\ & (\text{Winsome Bellamy}, \text{Sydney Observatory}) \}\end{aligned}$$

Note that this function is not a surjection, since its image is only

$$\{ \text{Harvard College Observatory, Melbourne Observatory, Sydney Observatory} \}$$

which is a (very) proper subset of the codomain.

To do the same with our squaring function would require an infinite set of ordered pairs,

$$\{(0, 0), (1, 1), (-1, 1), (2, 4), (-2, 4), \dots\},$$

which we can write succinctly, using our standard conventions for writing sets, as

$$\{(x, x^2) : x \in \mathbb{Z}\}.$$

The set of all ordered pairs $(x, f(x))$ of a function f is called the **graph** of the function. This term reminds us that we often illustrate a function by drawing a plot of all points $(x, f(x))$, with the horizontal axis containing the domain and the vertical axis containing the codomain. We are used to referring to such a plot as a “graph” of the function, but the term *graph* as just defined is more abstract: it just refers to the set of pairs, without regard to how they might be displayed to a reader.

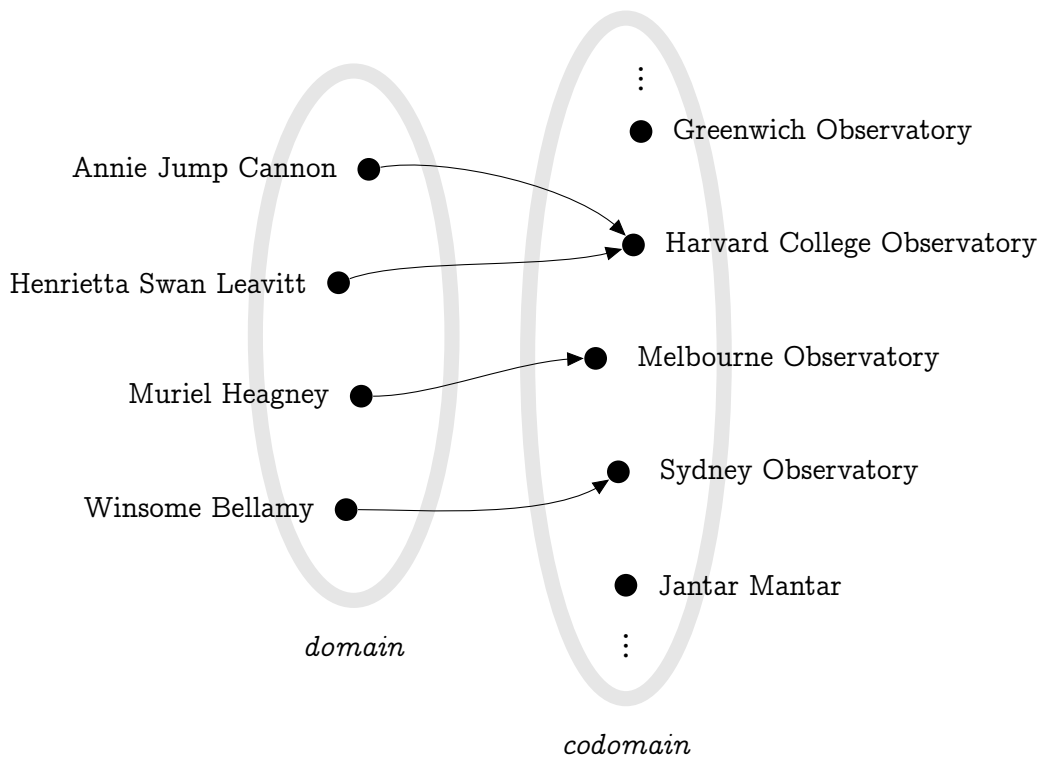


Figure 2.1: The Employer function.

Plots of functions, using horizontal and vertical axes, are convenient visual ways to display information about the function, but they also have their limitations. Many domains and codomains do not have an inherently one-dimensional structure, and increasing the number of dimensions — say, by using a 3D plot — does not always help. Some domains and codomains are not geometric in character at all. For example, the domain of Employer is a set of four people, and the domain of our Sorting function is the set of all possible lists of names, neither of which are defined in numerical or geometric terms.

There are other ways to depict functions. For example, we could start with a Venn diagram of the domain and codomain, draw points within the domain representing its members, and then, for every pair $(x, f(x))$, draw an arrow from x to $f(x)$ to indicate that the function sends x to $f(x)$. Because f is a function, every point in the domain has exactly one arrow going out of it. Our Employer function is depicted in this way in [Figure 2.1](#).

2.2^α FUNCTIONS IN COMPUTER SCIENCE2.2.1^α *Functions from analysis*

In software development, the first task is to work out what must be done. This process is traditionally called **analysis** and usually involves extensive communication with the owner of a problem (e.g., a client) in order to come up with a precise description of the task at hand. One possible outcome of this analysis process is a function.

At this stage, we have not yet worked out how to solve the problem at hand. But at least we have a precise statement of the task to be done (the “what”). With this, we can then try to **design** a method for doing this task (the “how”). If our task is specified by a function, then we will design an algorithm for the function.

Once we have an algorithm, we proceed to **implementation**, or *programming* the algorithm using a programming language such as Python.

Of course, this is a very simplified and incomplete view of software development. The process is seldom purely linear; each step often involves going back and re-doing parts of a previous stage. The design process might highlight problems, or gaps, in the specification, which may require further communication with the problem owners to sort out, leading to changes to the specification. Or the clients may simply change their minds about some aspect of the task! The implementation process may bring to light some problems with the design which must then be fixed. There are later stages we haven’t mentioned, notably **maintenance**. And not all problem analysis leads to a function specification. For example, some may lead to a specification of how the various components of some system must interact; some may lead to a specification of a database. But functions remain a very important product of analysis, partly because more complicated systems often contain functions as components.

2.2.2^α *Functions in mathematics and programming*

Our view of functions, as specifications of what rather than how, originated in mathematics although is now widespread in computer science and other disciplines. But, as you study programming, you will find that the term is used in another way too.

In many programming languages, and even in many pseudocode conventions for writing algorithms, a definition of a “function” has — in addition to the parts discussed here — some code in the programming language, or an algorithm, that specifies how the function is to be computed. In fact, the very word “function” is a reserved word in some programming languages and has this meaning, possibly with additional technical details.

By default, we use the term “function” in the mathematical sense, where there is no code or algorithm given. Occasionally we may use the term “mathematical function” to emphasise this, but even without that adjective, the term “function” will be used in this way.

If we wish to use the programming sense of the term, we will specify that explicitly, as in “Python function” or “programming function” or “algorithmic function”.

In *functional programming* languages, algorithmic functions are the most fundamental objects used, and all computation is done by manipulating them. They can be represented by variables and treated as both arguments and values of other functions. We do not consider the functional programming paradigm in this unit or in FIT1045. You can learn more about functional programming in FIT2102 Programming Paradigms.

2.3^α NOTATION

A function definition has the form

$$\begin{aligned} \textit{name} : \textit{domain} &\longrightarrow \textit{codomain}, \\ \textit{name}(x) &= \textit{precise description of the function value, in terms of } x. \end{aligned}$$

For example, a function f that gives squares of real numbers can be defined by

$$\begin{aligned} f : \mathbb{R} &\longrightarrow \mathbb{R}, \\ f(x) &= x^2. \end{aligned}$$

Here, the domain is $\mathbb{R}$, and the codomain is $\mathbb{R}$ too. Since we stated that our function would give the squares of *real numbers*, we really have no alternative but to specify $\mathbb{R}$ as the domain. But we have some more flexibility with the codomain. We could have used $\mathbb{R}_0^+$, since that is the image of this function: the squares of real numbers are precisely the nonnegative real numbers. We could, instead, have used $\mathbb{R} \cup \{0, -\sqrt{2}, -42\}$ as the codomain, which is perfectly valid mathematically, although this codomain has some extra detail that is irrelevant, useless, distracting, and shows poor user interface design!

The first line of these function definitions, such as $f : \mathbb{R} \longrightarrow \mathbb{R}$, is like a declaration in a program. It announces the name of the function and specifies the types of objects that it can take as its arguments and give as its values. The second line, such as $f(x) = x^2$, completes the definition by specifying the rule. A common form of wording is to say something like, “The function $f : \mathbb{R} \rightarrow \mathbb{R}$ is defined by $f(x) = x^2$.”

There is another common convention for specifying the rule of a function, where we just write

$$x \mapsto \textit{precise description of the function value, in terms of } x.$$

Note how the “mapping arrow” $\mapsto$ in the rule differs from the ordinary arrow $\rightarrow$ going from domain to codomain. It is important not to mix the two arrow types up.

If we use this second convention, then our squaring function would be defined by

$$\begin{aligned} f : \mathbb{R} &\longrightarrow \mathbb{R}, \\ x &\mapsto x^2. \end{aligned}$$

2.4^α SOME SPECIAL FUNCTIONS

The most trivial, vacuous, degenerate function of all is the **empty function**, denoted $\emptyset$. Its domain and codomain are each empty, and it has no rule because there is nothing in the domain for any rule to apply to. It can be defined simply as $\emptyset : \emptyset \rightarrow \emptyset$. It's pretty useless; we may hope never to see it again! Let's move on.

For any set A , the **identity function** i_A on A is defined by

$$\begin{aligned} i_A : A &\longrightarrow A, \\ i_A(x) &= x. \end{aligned}$$

This function just maps each member of A to itself.³

For any domain D , we can define for every subset $A \subseteq D$ the **indicator function** χ_A , which uses 1 and 0 to indicate, for each member of the domain, whether or not it is in A :

$$\begin{aligned} \chi_A : D &\longrightarrow \{0, 1\}, \\ \chi_A(x) &= \begin{cases} 1, & \text{if } x \in A; \\ 0, & \text{if } x \notin A. \end{cases} \end{aligned}$$

Although indicator function notation χ_A only mentions A , it must be kept in mind that a function's definition always includes a specification of its domain, which in this case is D . Different domains give rise to different indicator functions.

We can express the indicator functions of sets obtained from set operations on A and B using the indicator functions of A and B . For example, for all x we have

$$\chi_{A \cup B}(x) = \max\{\chi_A(x), \chi_B(x)\}.$$

See [Exercise 2](#).

For any domain D and any object a , the **constant function** c_a just maps everything to a :

$$\begin{aligned} c_a : D &\longrightarrow \{a\}, \\ c_a(x) &= a. \end{aligned}$$

2.5^α FUNCTIONS WITH MULTIPLE ARGUMENTS AND VALUES

Many functions you will meet have multiple arguments. If f is a function of two arguments x and y , then we write its value as $f(x, y)$. Suppose $x \in X$ and $y \in Y$, and that the value of the function belongs to a codomain C . Then the function definition would start by stating $f : X \times Y \rightarrow C$.

³ So, in a sense, it does nothing. But at least it does *nothing* to *something*, whereas the empty function does nothing to nothing!

It may seem that we are extending our definition of functions here, since we seem to have two domains, X and Y , for the first and second argument respectively. And no real harm can come from this view. But functions of two arguments may also be viewed functions of a *single* argument where that one argument happens to be a pair (x, y) and its domain is the Cartesian product $X \times Y$. So, when we start a function definition with $f : X \times Y \rightarrow C$, we are still just using our usual way of defining functions. When we write $f(x, y)$, indicating two arguments, we are using a shorthand for $f((x, y))$, where the argument that we put inside $f(\dots)$ is the ordered pair (x, y) . In accordance with usual practice, we will drop the second pair of parentheses from $f((x, y))$, writing $f(x, y)$ instead, and we will happily speak of its first argument x , its second argument y , and so on. But keep in the back of your mind that we can also view this as a function of just one argument whose sole argument happens to be the ordered pair (x, y) .

When we write $f(x, y)$ for applying function f to arguments x and y , we are using **prefix** notation, because we put the name of the function *before* the arguments. This is common practice and is the one we use when defining new functions. But there are also many well-known functions that use **infix** notation, where the function name is put *between* its arguments. Familiar examples include ordinary arithmetic functions $+$, $-$, $\times$, $/$ and many built-in operations in many programming languages. Far less common is **postfix** notation, where the name is placed *after* the arguments.

All these remarks extend readily to functions of three or more arguments. For example, a function of three arguments can also be regarded as a function of a single argument which happens to be a triple.

We also sometimes want the value of a function to be a tuple. For example, suppose the value of a function is to be a pair (y, z) where $y \in Y$ and $z \in Z$. If the domain of the function is X , then we may write $f : X \rightarrow Y \times Z$. This function may be regarded as giving two values, which we find it convenient to put in an ordered pair. We also view it as giving a single value which happens to be the ordered pair $(y, z) \in Y \times Z$.

Again, these comments extend readily to functions that return tuples of three or more values.

2.6 RESTRICTIONS

Sometimes we want to restrict a function to some subset of its domain, and to treat this restricted version of the function as a new function in its own right. For example, consider a function that assigns ID numbers to Monash students. Its domain is the set of all Monash students. If we want to use a function which only considers FIT1058 students, and assigns ID numbers to them, then this function is a restriction of the previous function just to the set of all FIT1058 students.

There are several reasons why we might want to focus just on the restriction of some function.

- If the domain is significantly smaller than the original function, then storing the restricted function as a list of ordered pairs takes up less space.

- As we discussed on page 39 in § 2.1.1^α, the domain of a function serves as a promise to users of the function that anything in the domain is valid, for that function, and is assigned a value by the function. As we noted, a bigger domain amounts to a bigger promise which may require more work to keep. If a function is given an algorithm and implemented as a program, then testing is required in order to be sure that it works for all members of its domain. The bigger the domain is, the more testing is required. So you may prefer to restrict the function to a subset of its domain, focusing on the arguments you *really* care about and reducing the work of testing and the size of the promise implied by the domain.
- Sometimes a function is simpler to describe, analyse or compute on some particular subset of its domain that we most care about.
- Sometimes a restricted version of a function may have stronger properties that enable it to be used in situations where the original function cannot be used.

Suppose we have a function $f : A \rightarrow B$ and that $X \subseteq A$. The **restriction** of f to X , denoted by $f|_X$ or $f \upharpoonright_X$, is the function with domain X , codomain B , and which agrees with f on X :

$$\begin{aligned} f|_X : X &\rightarrow B, \\ f|_X(x) &= f(x) \text{ for all } x \in X. \end{aligned}$$

For example, suppose f is our squaring function from § 2.3^α:

$$\begin{aligned} f : \mathbb{R} &\rightarrow \mathbb{R}, \\ f(x) &= x^2. \end{aligned}$$

Then its restriction $f|_{\mathbb{R}_0^+}$ to the nonnegative real numbers is given by

$$\begin{aligned} f|_{\mathbb{R}_0^+} : \mathbb{R}_0^+ &\rightarrow \mathbb{R}, \\ f|_{\mathbb{R}_0^+}(x) &= x^2. \end{aligned}$$

This has some properties that the original function f does not have. For example, $f|_{\mathbb{R}_0^+}$ is continually increasing as x increases across its domain, whereas $f(x)$ is decreasing along some of its domain (specifically, along $\mathbb{R}_0^-$) and increases elsewhere (along $\mathbb{R}_0^+$), so its behaviour is a bit more complicated. Each member y in the image of $f|_{\mathbb{R}_0^+}$ comes from a *unique* x in the domain $\mathbb{R}_0^+$, namely the positive square root $\sqrt{y}$ of y (or 0, in the case $y = 0$). By contrast, each nonzero member y of the image of f comes from *two different* values of x , namely the two square roots $\pm\sqrt{y}$ of y . This illustrates the point mentioned above that a restriction can be simpler and have stronger properties, which can make it more useful in some situations. (Later, in § 2.8, we discuss inverse functions. Then, we can say that $f|_{\mathbb{R}_0^+}$ is invertible but f is not.)

2.7 INJECTIONS, SURJECTIONS, BIJECTIONS

As we have seen from some of our examples, it is perfectly ok for different function arguments to give the same value. So it is ok for both Annie Jump Cannon and Henrietta Swan Leavitt to have the value Harvard College Observatory under the Employer function (p. 42 in § 2.1.3^α). There is no requirement for there to be a unique argument for each value. This contrasts with the requirement that there be a unique value for each argument, which is an essential property of *any* function. This specific Employer function only assigns one employer to each human computer.

Although it's ok in general for different arguments to be mapped to the same value, there are situations where we do not want that to happen. For example, a function that assigns an ID number to each student must ensure that different students get different ID numbers. A function that encrypts files must ensure that different files are encrypted differently, else the contents of a file cannot be recovered from its encrypted form.

A function with this property, that different arguments are always mapped to different values, is said to be **injective** and is called an **injection**. Mathematically, this property of a function $f: A \rightarrow B$ can be expressed as follows: for any two distinct $x_1, x_2 \in A$, we have $f(x_1) \neq f(x_2)$. Such a function gives a **one-to-one correspondence** between the domain and the image, but *not* between the domain and the codomain in general.

Injections have the virtue of *preserving information*: for every member y in the *image* of an injection f , there is a unique x in its domain such that $f(x) = y$. In every case, knowing y is logically sufficient for determining x (although we are not saying anything here about how much *work* it might be to recover x ; that depends on the details of the function). If a function is not an injection, then there must be at least one member y of its image such that there are two or more members x_1, x_2 of its domain which map to that image: $f(x_1) = f(x_2) = y$. So, in that case, knowing y still leaves you in doubt as to how it could have been produced by f .

Functions that aren't injections *lose information*. We might call them **lossy**. In fact, this term is used for data compression functions that are not injections. By contrast, an injective data compression function is called **lossless**.

Similarly, we can express mathematically the **onto** property of a function, which we defined in § 2.1.2^α. A function $f: A \rightarrow B$ is **surjective**, and is said to be a **surjection**, if for any value $y \in B$ there must be a value of the function's argument $x \in A$ such that $y = f(x)$.

A function that is both an injection and a surjection is said to be **bijective** and is called a **bijection**. Such a function is a **one-to-one correspondence** between the domain and the codomain (which is also the image in this case).

A bijection whose domain and codomain are the same set is also called a **permutation**, though the latter term is usually used only for bijections on finite sets.

Bijections preserve information, since they are injections. Furthermore, since they are also surjections, each member of the codomain may be thought of as encoding a

unique member of the domain. So a bijection establishes that the domain and codomain contain the same information, although it may be represented in different ways.

If a function has *finite* domain and codomain of the *same size*, and is an injection, then it must also be a surjection, and hence also a bijection. This is because there is no room in the codomain for the injection to avoid mapping to all its members. Similarly, a surjection with *finite* domain and codomain of the *same size* must also be an injection, and hence a bijection, since the need to map to all members of the codomain prevents any repetition of codomain elements. Both these assertions fail if the domain and codomain are infinite. Can you find examples to illustrate the failure in each case?

2.8 INVERSE FUNCTIONS

We think of a function as going *from* any member of its domain *to* its corresponding value in the codomain. But there are times when we may want to go backwards: given a value in the codomain, what argument in the domain does it come from? For example, which computer worked at Harvard College Observatory? Which number, when squared, gives 4? Which file corresponds to a particular encrypted file?

For functions in general, the answer may not be unique. We have just mentioned some cases of this: our Employer function mapped two different computers to Harvard College Observatory, and $2^2 = (-2)^2 = 4$.

This failure of uniqueness can happen in either of two different ways. Let $f: A \rightarrow B$ be a function. For a given value y in the codomain:

- There may be more than one argument that gives the value y under the function. So we may have $x_1, x_2 \in A$ such that $x_1 \neq x_2$ but $f(x_1) = f(x_2) = y$.
- There may be no argument that gives the value y . This happens when the image is a proper subset of the codomain and y lies in the codomain but not in the image.

But if neither of these occurs, then every $y \in B$ has a unique $x \in A$ such that $f(x) = y$. This means that, in giving each $y \in B$ a corresponding $x \in A$, we are actually defining a function from B to A , with domain B and codomain A . So the roles played by A and B are reversed, in keeping with the reversed “direction” of this new function. We call this new function the **inverse function** of f and denote it by f^{-1} . We can write its definition as follows.

$$\begin{aligned} f^{-1} : B &\longrightarrow A, \\ f^{-1}(y) &= \text{the unique } x \text{ such that } f(x) = y. \end{aligned}$$

If we want to write the rule of f^{-1} as a set of ordered pairs, then we just take all the ordered pairs in f and reverse them:

$$\{ (y, x) : (x, y) \text{ belongs to } f \}$$

or, to put it slightly differently,

$$\{ (f(x), x) : x \in A \}.$$

For a function to have an inverse function, it must be an injection (so that no value has two corresponding arguments) and it must also be a surjection (so that every value in the codomain is also in the image, i.e., has a corresponding argument). So, in fact, a function has an inverse function if and only if it is a bijection.

Our Employer function is not a bijection and therefore does not have an inverse function. The squaring function also does not have an inverse function, for the same reason. But we would want an encryption function to have an inverse function, so that an intended user of an encrypted file has no doubt about its contents. (In that context, there is the separate issue of how easy or hard it should be to actually compute the inverse. We would like that to be easy for intended users and hard for others. Achieving these competing aims is the fundamental challenge of cryptography.)

The role of the inverse function is to “undo” the function and get back what you started with. So, if you have $x \in A$ and apply f to get $y = f(x)$, then it does not matter much if you “lose” x , because you can recover it from y :

$$x = f^{-1}(y) = f^{-1}(f(x)).$$

Since the original function f is a bijection, its inverse f^{-1} is also a bijection. It follows that f^{-1} also has an inverse, which we could write as $(f^{-1})^{-1}$. But the inverse of the inverse is just the original function; if f^{-1} “undoes” f , then f also “undoes” f^{-1} . We have

$$y = f(x) = f(f^{-1}(y)).$$

2.9 COMPOSITION

It is common to use values obtained from one function as arguments to another function. For example, consider the functions Father and Mother which each have, as their domain and codomain, the set $\mathbb{P}$ of all people who have ever lived.

$$\begin{aligned} \text{Father} : \mathbb{P} &\longrightarrow \mathbb{P}, \\ \text{Father}(p) &= \text{the father of person } p. \\ \text{Mother} : \mathbb{P} &\longrightarrow \mathbb{P}, \\ \text{Mother}(p) &= \text{the mother of person } p. \end{aligned}$$

Starting with Alan Turing, the function *Mother* gives Alan's mother, Sara Turing. Applying the function *Father* to her gives Sara's father — Alan's maternal grandfather — Edward Stoney.

$$\begin{aligned}\text{Mother}(\text{Alan Turing}) &= \text{Sara Turing.} \\ \text{Father}(\text{Sara Turing}) &= \text{Edward Stoney.}\end{aligned}$$

This “chaining” of the two functions is called *composition*, and is denoted by stating the second function, followed by the composition symbol $\circ$, followed by the first function. (Note the order there.) In this case, the function is $\text{Father} \circ \text{Mother}$ and it is defined as follows.

$$\begin{aligned}\text{Father} \circ \text{Mother} : \mathbb{P} &\longrightarrow \mathbb{P}, \\ \text{Father} \circ \text{Mother}(p) &= \text{Father}(\text{Mother}(p)).\end{aligned}$$

Note now the order in which the functions are written in $\text{Father} \circ \text{Mother}$ is the same as the order in which they are written when we write one function as an argument of the other, i.e., in $\text{Father}(\text{Mother}(p))$. But our usual *order of reading and writing* (left to right) is the reverse of the *order of application* (right to left): we *apply* the function *Mother first*, and *then* we apply the function *Father*.

In this example, the codomain of the first function applied (*Mother*) equals the domain of the second function applied (*Father*); both are $\mathbb{P}$. This ensures that our first function application, using *Mother*, always produces something (or someone) that our second function, *Father*, can deal with. This is a general requirement for function composition.

In general, the **composition** $g \circ f$ of two functions $f : A \rightarrow B$ and $g : B \rightarrow C$ is defined by

$$\begin{aligned}g \circ f : A &\longrightarrow C, \\ g \circ f(x) &= g(f(x)).\end{aligned}$$

Note that, in this composition, f is applied *first*, and then the result is given to g . The order of doing things does matter; in general, function composition is not commutative, meaning that $g \circ f$ and $f \circ g$ are not the same. It is, however, associative: if $f : A \rightarrow B$, $g : B \rightarrow C$ and $h : C \rightarrow D$, then

$$h \circ (g \circ f) = (h \circ g) \circ f,$$

so we can write $h \circ g \circ f$ unambiguously.

Function composition is undefined if the codomain of the first function applied does not equal the domain of the second function applied. For example, suppose $\$$ is the set

of all students, $\mathbb{U}$ is the set of all Monash units, and the function `FavouriteUnit` is defined as follows.

$$\begin{aligned}\text{FavouriteUnit} : \mathbb{S} &\longrightarrow \mathbb{U}, \\ \text{FavouriteUnit}(p) &= \text{the favourite Monash unit of person } p.\end{aligned}$$

If you want to know your mother's favourite unit at Monash, you might be tempted to use the function $\text{FavouriteUnit} \circ \text{Mother}$. But not all mothers are students, and not all students are mothers. Formally, the codomain of `Mother` is $\mathbb{P}$, which does not equal the domain of `FavouriteUnit`, namely $\mathbb{S}$. So this function composition is undefined.

The requirement that the codomain of the first function f equals the domain of the second function g amounts to insisting that our guarantee about what f can produce (expressed in the form of its codomain) is the same as our guarantee about what g can handle (expressed in the form of its domain). So the two functions are compatible, in a precise sense.

You have seen composition before for mathematical functions. For example, the expression $(x-1)^2$ may be regarded as the rule for the composition $g \circ f$ of the two functions

$$\begin{aligned}f : \mathbb{R} &\longrightarrow \mathbb{R}, \\ f(x) &= x-1. \\ g : \mathbb{R} &\longrightarrow \mathbb{R}_0^+, \\ g(x) &= x^2.\end{aligned}$$

We saw a very special case of composition in § 2.8. If $f : A \rightarrow A$ is a bijection, then

$$\begin{aligned}f \circ f^{-1} &= i_A, \\ f^{-1} \circ f &= i_A.\end{aligned}$$

If $g : C \rightarrow D$ is any function, then composing it with the identity function makes no difference, provided we pick the correct identity:

$$\begin{aligned}g \circ i_C &= g, \\ i_D \circ g &= g.\end{aligned}$$

We can compose a function *with itself* provided its domain and codomain are the same. If $f : A \rightarrow A$ then the definition of composition tells us that $f \circ f : A \rightarrow A$ is defined for all $x \in A$ by $f \circ f(x) = f(f(x))$. We can then do iterated composition of f with itself, if we wish. We write $f^{(n)}$ for the composition of n copies of f :

$$f^{(n)} = \underbrace{f \circ f \circ \cdots \circ f}_{n \text{ copies of } f}$$

We introduce three iterated compositions: one from a practical application, one party trick, and one profound unsolved theoretical question.

- Consider the function $\text{LCG} : [0, 2^{31} - 1]_{\mathbb{Z}} \rightarrow [0, 2^{31} - 1]_{\mathbb{Z}}$ defined for all 31-bit nonnegative binary integers x by

$$\text{LCG}(x) = \text{the last 31 bits of } 1103515245x + 12345. \quad (2.1)$$

We always keep only the last 31 bits, to ensure that the numbers generated stay within our fixed interval. This function has been used to generate sequences of numbers that are *pseudorandom* in the sense that, superficially, they look random if you don't look too closely. Starting with some initial "seed" number s , the function LCG is applied repeatedly, and the successive numbers $\text{LCG}^{(n)}(s)$ should behave in a way that looks statistically random in some sense. (We have used this example as it is one of the simpler pseudorandom number generators that have been used in practice, but its randomness properties are imperfect and it should not be used by itself in this naive way. It is usually used in conjunction with other methods in order to increase the randomness.) The name LCG comes from the term *Linear Congruential Generator*, which is a type of pseudorandom number generator of which this is one example.

- Consider the function $K : [0, 9999]_{\mathbb{Z}} \rightarrow [0, 9999]_{\mathbb{Z}}$ defined for any nonnegative integer x with at most four (decimal) digits as follows. First, form a four-digit number by writing the four digits of x (using leading zeros if necessary) from smallest to largest. Then reverse that number, so that the digits now go from largest to smallest. Then $K(x)$ is defined to be the difference between these two numbers. For example,

$$K(1729) = 9721 - 1279 = 8442.$$

This function is not an injection (why?). It is clear that, if all the digits in x are the same, then $K(x) = 0$, and therefore $K^{(n)}(x) = 0$ for all $n \geq 1$. More surprisingly, in *all other cases* (i.e., when x has at least two different digits), iterated application of the function eventually yields 6174, and does so after at most seven iterations. For example, starting with 1729 as above, we have

$$1729 \xrightarrow{K} 8442 \xrightarrow{K} 5994 \xrightarrow{K} 5355 \xrightarrow{K} 1998 \xrightarrow{K} 8082 \xrightarrow{K} 8532 \xrightarrow{K} 6174 \xrightarrow{K} 6174.$$

So $K^{(7)}(x)$ is 0 if all digits are the same and 6174 otherwise. This was discovered by the Indian mathematician D. R. Kaprekar in 1946 and published in 1955.⁴

⁴ D. R. Kaprekar, An interesting property of the number 6174, *Scripta Mathematica* **21** (1955) 304. Martin Gardner, Mathematical Games, *Scientific American* (March 1975).

- Consider the function $\text{Collatz} : \mathbb{N} \rightarrow \mathbb{N}$ defined for all $x \in \mathbb{N}$ by

$$\text{Collatz}(x) = \begin{cases} 3x+1, & \text{if } x \text{ is odd;} \\ x/2, & \text{if } x \text{ is even.} \end{cases}$$

For example, if we start with 7 and keep iterating, we obtain

$$7 \mapsto 22 \mapsto 11 \mapsto 34 \mapsto 17 \mapsto 52 \mapsto 26 \mapsto 13 \mapsto 40 \mapsto 20 \mapsto 10 \mapsto 5 \mapsto 16 \mapsto 8 \mapsto 4 \mapsto 2 \mapsto 1 \mapsto 4 \mapsto 2 \mapsto 1 \mapsto \dots$$

Note how it eventually gets stuck in a loop, going from 4 to 2 to 1, then back to 4, and so on.

Iterated composition of this function is mysterious. It is conjectured that, for every x , there exists n such that $\text{Collatz}^{(n)}(x) = 1$. This has become known as **Collatz's Conjecture** or the **$3x+1$ problem**. Currently it is unsolved. It is a remarkable illustration that even simple questions about very simple algorithms can be very deep and hard to answer.

We now consider how the injective, surjective and bijective properties are affected by composition.

We start with injection.

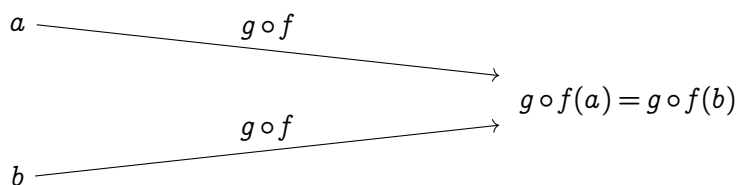
Theorem 6. If $f : A \rightarrow B$ and $g : B \rightarrow C$ are injections then $g \circ f$ is also an injection.

Proof. We prove this by proving the equivalent statement that

if $g \circ f$ is *not* an injection, then f and g are *not* both injections.

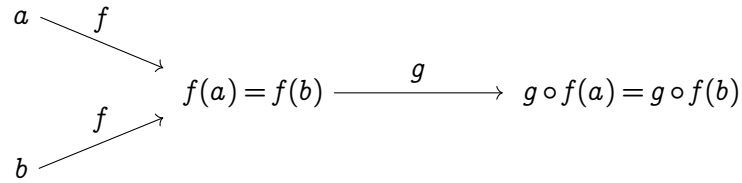
This is an example of the general logical principle that “ A implies B ” is equivalent to “not- B implies not- A ”. For example, the statement that “every rectangle has four sides” is equivalent to the statement that “everything that does not have four sides is not a rectangle”. See (1.10).

Suppose that $g \circ f$ is *not* an injection. Then there must exist $a, b \in A$ such that $a \neq b$ and $g \circ f(a) = g \circ f(b)$.

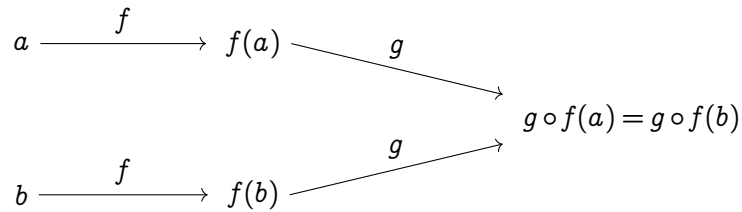


Consider $f(a)$ and $f(b)$. Either they are equal or unequal.

- If $f(a) = f(b)$, then we have $a \neq b$ and $f(a) = f(b)$, so f is not an injection.



- If $f(a) \neq f(b)$, then we have $f(a) \neq f(b)$ and $g(f(a)) = g(f(b))$, so g is not an injection (since we have two *distinct* members of its domain, namely $f(a)$ and $f(b)$, that are mapped by g to the same value).



So we see that, whatever happens with $f(a)$ and $f(b)$, at least one of f and g is not an injection. $\square$

The converse of this theorem does not hold: $g \circ f$ being an injection does *not* imply that *both* f and g are injections. For example, define $f: \{1, 2\} \rightarrow \{1, 2, 3\}$ by

$$\begin{aligned}
 f(1) &= 1, \\
 f(2) &= 2,
 \end{aligned}$$

and define $g: \{1, 2, 3\} \rightarrow \{1, 2\}$ by

$$\begin{aligned}
 g(1) &= 1, \\
 g(2) &= 2, \\
 g(3) &= 2.
 \end{aligned}$$

Then their composition $g \circ f: \{1, 2\} \rightarrow \{1, 2\}$ is defined by

$$\begin{aligned}
 g \circ f(1) &= g(f(1)) = g(1) = 1, \\
 g \circ f(2) &= g(f(2)) = g(2) = 2.
 \end{aligned}$$

So $g \circ f$ is an injection, but it is not the case that both f and g are injections. In fact, f is an injection, but g is not.

Now let's consider surjections.

Theorem 7. If $f: A \rightarrow B$ and $g: B \rightarrow C$ are surjections then $g \circ f$ is also a surjection.

Proof. Suppose f and g are surjections. Consider $g \circ f: A \rightarrow C$.

Let $c \in C$ be any member of its codomain C .

Since g is a surjection, there must exist $b \in B$ such that $g(b) = c$.

Since f is a surjection, there must exist $a \in A$ such that $f(a) = b$.

But then we have

$$g \circ f(a) = g(f(a)) = g(b) = c.$$

So, every member of the codomain of $g \circ f$ also belongs to its image. Therefore $g \circ f$ is a surjection. $\square$

For [Theorem 7](#), too, the converse does not hold. In fact, the same f and g we gave above, after the proof of [Theorem 6](#) and before stating [Theorem 7](#), shows this here too. In that example, $g \circ f$ is a surjection, but f is not a surjection (although g is a surjection).

[Theorem 6](#) and [Theorem 7](#) together give a similar statement for bijections.

Theorem 8. If $f: A \rightarrow B$ and $g: B \rightarrow C$ are bijections then $g \circ f$ is also a bijection.

Proof. Suppose $f: A \rightarrow B$ and $g: B \rightarrow C$ are bijections. Then, by definition, they are both injections and they are both surjections.

Since they are both injections, their composition is also an injection by [Theorem 6](#).

Since f and g are both surjections, their composition is also a surjection by [Theorem 7](#).

So $g \circ f$ is both an injection and a surjection. Therefore, by definition, it is a bijection. $\square$

Once again, the converse does not hold in general, and once again our little functions f and g show this, since $g \circ f$ is a bijection but neither f nor g is a bijection.

There is one important situation where the converse does hold as well.

Theorem 9. Let $f: A \rightarrow B$ and $g: B \rightarrow C$ be functions where A, B, C are finite sets of the same size. Then $g \circ f$ is a bijection if and only if both f and g are bijections.

Proof. The domains and codomains of f , g and $g \circ f$ are finite and of the same size, as stated. So each of them is a bijection if and only if it is an injection, by our remarks at the end of [§ 2.7](#). So it is enough to prove that

$g \circ f$ is an injection if and only if both f and g are injections.

We have already seen that, if f and g are injections, then so is $g \circ f$ ([Theorem 6](#)). So it remains to prove that

if $g \circ f$ is an injection then both f and g are injections.

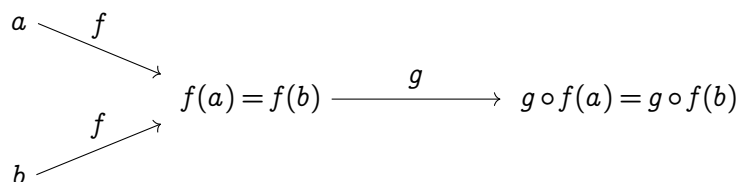
This we do now, by proving the equivalent statement that

if at least one of f and g is *not* an injection then $g \circ f$ is *not* an injection.

Our starting assumption here, that at least one of f and g is not an injection, divides naturally into two cases: (i) f is not an injection, and (ii) g is not an injection. These two cases overlap, which is ok.

Case (i):

If f is not an injection, then by definition there exist distinct $a, b \in A$ such that $f(a) = f(b)$. Then $g(f(a)) = g(f(b))$. So in fact our distinct a, b also give $g \circ f(a) = g \circ f(b)$, so $g \circ f$ is not an injection.



Case (ii):

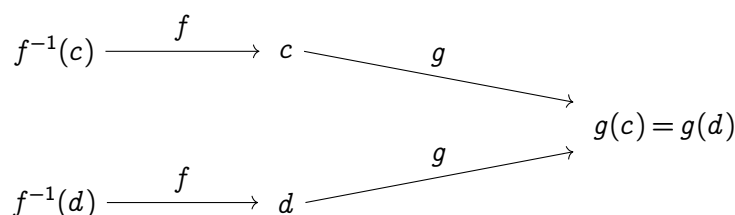
It remains to consider the possibility that g is not an injection. Within this case, we can restrict to cases where f is an injection, since we have just dealt with the possibility that f is not an injection. (Effectively, we are ignoring the overlap between the two cases, since that overlap is covered by Case (i).)

Suppose then that f is an injection. Since its domain and codomain are finite and have the same size, this means it is also a bijection, and therefore has an inverse.

If g is not an injection, then by definition there exist distinct $c, d \in B$ such that $g(c) = g(d)$. Now because f is a bijection, its inverse f^{-1} is defined, has the same domain B , and is also a bijection. So $f^{-1}(c)$ and $f^{-1}(d)$ are both defined and must be distinct since $c \neq d$. Furthermore, $c = f(f^{-1}(c))$ and $d = f(f^{-1}(d))$. So $g(c) = g(d)$ implies $g(f(f^{-1}(c))) = g(f(f^{-1}(d)))$, which may be rewritten

$$g \circ f(f^{-1}(c)) = g \circ f(f^{-1}(d)).$$

But $f^{-1}(c) \neq f^{-1}(d)$, so we have two distinct members of A which are mapped to the same thing by $g \circ f$. So $g \circ f$ is not an injection.



Summarising, we have found that if either of f, g is not an injection then $g \circ f$ is not an injection either. Therefore $g \circ f$ is not a bijection. $\square$

The restriction to finite sets is essential. Consider the following example.

Define $f : \mathbb{N} \rightarrow \mathbb{N}$ by $f(n) = 2n$, and define $g : \mathbb{N} \rightarrow \mathbb{N}$ by $f(n) = \lfloor n/2 \rfloor$. Neither of these is a bijection: f is an injection but not a surjection, and g is a surjection but not an injection. Yet their composition $g \circ f$ is a bijection, and in fact is the identity function on $\mathbb{N}$:

$$g \circ f(n) = g(f(n)) = \lfloor (2n)/2 \rfloor = \lfloor n \rfloor = n.$$

Now that we can compose two functions, it is natural to ask about the inverse of the composition. This turns out to be the *reverse* composition of their inverses. This aligns with everyday experience of doing and undoing sequences of tasks: if we wrap up a gift in multiple layers of wrapping paper, then the recipient unwraps the layers in reverse order.

Theorem 10. If $f : A \rightarrow B$ and $g : B \rightarrow C$ are bijections, then $(g \circ f)^{-1} = f^{-1} \circ g^{-1}$

Proof. Let $f : A \rightarrow B$ and $g : B \rightarrow C$ be bijections. By Theorem 8, their composition $g \circ f : A \rightarrow C$ is also a bijection, so its inverse $(g \circ f)^{-1} : C \rightarrow A$ exists. Now consider $f^{-1} \circ g^{-1}$. This exists, since the inverses f^{-1} and g^{-1} exist, since f and g are bijections. We check the two conditions that $f^{-1} \circ g^{-1}$ must satisfy in order to be the inverse of $g \circ f$.

$$\begin{aligned} (g \circ f) \circ (f^{-1} \circ g^{-1}) &= g \circ (f \circ f^{-1}) \circ g^{-1} && \text{(since function composition is associative)} \\ &= g \circ i_B \circ g^{-1} \\ &= g \circ g^{-1} \\ &= i_C. \end{aligned}$$

Similarly,

$$\begin{aligned} (f^{-1} \circ g^{-1}) \circ (g \circ f) &= f^{-1} \circ (g^{-1} \circ g) \circ f && \text{(again using associativity)} \\ &= f^{-1} \circ i_B \circ f \\ &= f^{-1} \circ f \\ &= i_A. \end{aligned}$$

So $f^{-1} \circ g^{-1}$ is indeed the inverse of $g \circ f$. □

2.10 CRYPTOSYSTEMS

One important application of function composition is to cryptosystems.

Suppose we have

- a **message space**, which is a finite set M of possible **messages** (in the form of strings over some alphabet),

- a **cypher space**, which is a finite set C of strings, which we call **cyphertexts**, and
- a **keyspace**, which is a finite set K whose members we call **keys**.

An **encryption function** is a function $e : M \times K \rightarrow C$, and a **decryption function** is a function $d : C \times K \rightarrow M$. These are functions of two arguments. Each also gives us a family of single-argument functions. For each key $k \in K$, define the function $e_k : M \rightarrow C$ by $e_k(m) = e(m, k)$, and define the function $d_k : C \rightarrow M$ by $d_k(c) = d(c, k)$. These are the functions of one argument that you get from e and d by fixing their second argument. So, for a given key $k \in K$, the function e_k encrypts messages just with that one key, while the function d_k decrypts just with that one key. We want decryption to undo encryption, provided the same key is used by each.

We say that (M, C, K, e, d) is a **cryptosystem** if, for all k ,

- (i) e_k and d_k are bijective over the image of the domain, and
- (ii) $d_k = e_k^{-1}$, for all $m \in M$.

Condition 2 is the most important and implies that for $m \in M$, $d_k(e_k(m)) = m$. In practice we will also want some conditions on how easy or hard it is to compute these functions or even to obtain partial information from them.

For convenience, we restrict ourselves to cryptosystems where the cypherspace and message space are the same, i.e., $M = C$. (Most real cryptosystems either have this property or can easily be modified so that they do.)

Suppose we have two cryptosystems with the same message spaces but with keyspaces and encryption/decryption maps that may be different. Call them $C = (M, M, K, e, d)$ and $C' = (M, M, K', e', d')$. We would like to *compose* them to make a more complex cryptosystem. For encryption, we want to first encrypt with e and then encrypt further with e' . This is shown in Figure 2.2. For decryption, we want to do the reverse: decrypt using d' , then decrypt further with d .

But our definition of function composition (§ 2.9) only applies to functions of one argument. So we need to extend this definition for our encryption and decryption functions.

The **keyed composition** of encryption functions $e : M \times K \rightarrow M$ and $e' : M \times K' \rightarrow M$ is the function $e' \bullet e : M \times (K \times K') \rightarrow M$ defined for all $m \in M$ and $(k, k') \in K \times K'$ by

$$(e' \bullet e)(m, (k, k')) = e'(e(m, k), k'). \quad (2.2)$$

Similarly, the **keyed composition** of decryption functions $d' : M \times K' \rightarrow M$ and $d : M \times K \rightarrow M$ is the function $d \bullet d' : M \times (K \times K') \rightarrow M$ defined for all $m \in M$ and $(k, k') \in K \times K'$ by

$$(d \bullet d')(m, (k, k')) = d(d'(m, k'), k). \quad (2.3)$$

Then the **composition** $C' \circ C$ of the cryptosystems C and C' is $(M, M, K \times K', e' \bullet e, d \bullet d')$.

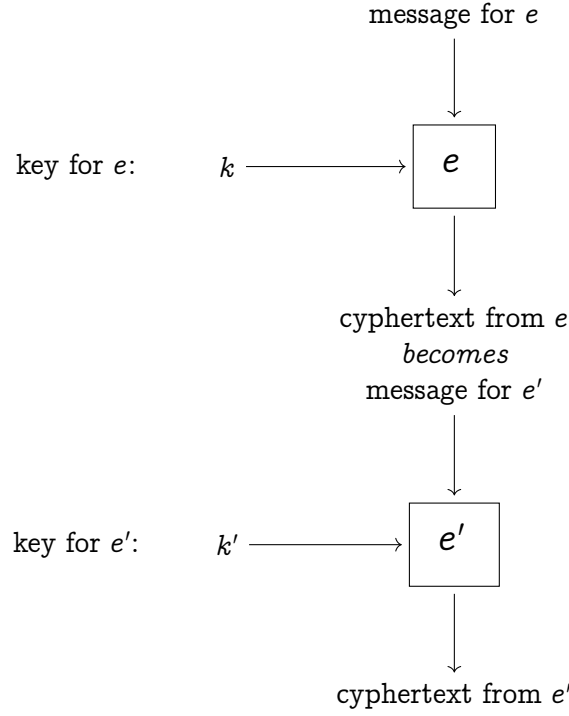


Figure 2.2: Composition of encryption functions

Observe that, in the composition $\mathcal{C}' \circ \mathcal{C}$, the encryption function $e' \bullet e$ gives rise to a family of single-argument functions, as follows. For each key pair (k, k') , the function $e_{(k, k')} : M \rightarrow M$ is defined for each $m \in M$ by

$$e_{(k, k')}(m) = (e' \bullet e)(m, (k, k')).$$

But, by (2.2), this is just $e'(e(m, k), k')$. And we can express this in terms of composition of our single-argument encryption functions:

$$e'(e(m, k), k') = e'_{k'}(e_k(m))$$

Therefore each single-argument encryption function $e_{(k, k')}$ in the composition $\mathcal{C}' \circ \mathcal{C}$ of the two cryptosystems is itself just the composition of the two single-argument encryption functions:

$$e_{(k, k')} = e'_{k'} \circ e_k.$$

Similarly, the decryption function $d \bullet d'$ yields the family of single-argument functions $d_{(k, k')} : M \rightarrow M$, defined for each $m \in M$ by

$$d_{(k, k')}(m) = (d \bullet d')(m, (k, k')) = d(d'(m, k'), k) = d_k(d'_{k'}(m)),$$

and

$$d_{(k,k')} = d_k \circ d'_{k'}.$$

Theorem 11. The composition of two cryptosystems is also a cryptosystem.

Proof. We need to prove that, for each key pair (k, k') , the encryption function $e_{(k,k')}$ and the decryption function $d_{(k,k')}$ are both bijections, and that the latter is the inverse of the former.

The fact that they are bijections follows from the fact that $e_k, e'_{k'}, d_k, d'_{k'}$ are all bijections for all k and k' (because $\mathcal{C}$ and $\mathcal{C}'$ are both cryptosystems) and [Theorem 8](#).

The fact that $d_{(k,k')} = (e_{(k,k')})^{-1}$ follows from [Theorem 10](#). $\square$

In a good cryptosystem, e_k and d_k should be easy to compute if the key k is known, otherwise the intended users will find it hard to use. But, in order for it to be secure, the encryption function should be hard to invert *without* the key: if all an eavesdropper knows is the encrypted text $c \in C$, it should be hard for them to recover either the original message $m \in M$ or the secret key $k \in K$.

Composition can be used to make decryption even harder for an eavesdropper. If each of e and e' is hard to invert, it is reasonable to hope that $e' \circ e$ might be even harder to invert, since an eavesdropper now has to undo the work of both encryption functions, not just one of them. There are many cryptosystems where this is indeed the case. But care is needed, because there are also systems where composition does not give any extra security.

2.11^ω LOOSE COMPOSITION

We might call the above type of function composition **tight composition**, because it requires an “exact fit” between the codomain of the first function to be applied and the domain of the second. There is also a looser form of function composition, which we’ll call “loose composition”, which is *always defined*, regardless of the domains and codomains of the functions, although in the worst case it might turn out to be the empty function.

Let us try again to define a function based on applying Mother followed by FavouriteUnit, which we considered in [§ 2.9](#). We’d like a function that equals $\text{FavouriteUnit}(\text{Mother}(p))$ whenever this makes sense. We saw in [§ 2.9](#) that the composition $\text{FavouriteUnit} \circ \text{Mother}$ is undefined. But we will define a new form of composition, called **loose composition** and denoted by $\circ$ (a larger, “looser” version of the symbol $\circ$), under which $\text{FavouriteUnit} \circ \text{Mother}$ is defined.

To properly define this loose composition, we must specify its domain and codomain. The codomain is easy, since any student’s favourite unit will always belong to $\mathbb{U}$ (although the image would be more complicated). But what should we use for this function’s domain? Not all mothers are students, so not every person $p \in \mathbb{P}$ gives a valid $\text{FavouriteUnit}(\text{Mother}(p))$.

The domain of a *loose composition* is the subset of the domain of the first function (i.e., the one that is applied first) containing everything for which the successive function applications are possible. In other words, it's everything that the first function maps into the domain of the second function. For `FavouriteUnit ◦ Mother`, this means every person whose mother is a student, since for any such person, the function `Mother` produces a member of `$`.

In general, the **loose composition** $g \circ f$ of two functions $f: A \rightarrow B$ and $g: C \rightarrow D$ is defined by

$$\begin{aligned} g \circ f : \{x \in A : f(x) \in C\} &\longrightarrow D, \\ g \circ f(x) &= g(f(x)). \end{aligned}$$

Note that, as usual for our composition notation, f is applied *first*, and then the result is given to g .

The loose composition of two functions is *always* defined. It might sometimes be useless: if $B \cap C = \emptyset$, then nothing that f produces is in the domain of g , so the composition $g \circ f$ is just the empty function.

It will be seen from the definition of loose composition that it is harder to work out the domain of loose composition than it is to work out the domain of tight composition. This makes it a bit harder to use in practice. Tight composition has the advantage that the question of whether the composition is defined can be answered solely by looking at the appropriate codomain and domain; you do not need to study the rule at all. This makes it much easier to work with, and from a computing perspective, much easier to use as a specification of a task based on combining two tasks.

In this unit, we will use tight composition rather than loose composition.

2.12 COUNTING FUNCTIONS

We often want to count functions of various types. You might want to determine the amount of time an algorithm takes, if the algorithm has to search through all functions of some type. You might want to determine the amount of space that a collection of data requires, if the data items correspond to functions. You might want to determine the probability that a random function has some particular property.

Suppose $f: A \rightarrow B$, where the sets have sizes $|A| = m$ and $|B| = n$. How many functions of this type are there? The domain has m elements, and each of them is mapped to one, and only one, member of the codomain B . So there are n possibilities for $f(x)$ for each element $x \in A$. These choices are independent; there is no requirement for the various values of x to differ from each other or to be related in any other way. So we have m independent choices, each being among n possibilities. This means we have n^m functions.

Suppose now we require f to be an injection. We still have m elements in the domain, and for each of these, we must still choose exactly one member of the codomain.

But now these choices are no longer independent, since as soon as one member of the codomain is chosen, that member is no longer available for any other member of the domain. Suppose we make these choices in order, and to help describe this, we suppose that the elements of A are enumerated as $a_1, a_2, \dots, a_m$. Now, a_1 can be mapped to any of the n elements of B . Then, a_2 can be mapped to any element of B except $f(a_1)$, so it has $n - 1$ choices. Then, a_3 can be mapped to any element of B except $f(a_1)$ and $f(a_2)$, so it has $n - 2$ choices. And so on. Finally, a_m can be mapped to any element of B except $f(a_1), f(a_2), \dots, f(a_{m-1})$, so it has $n - (m - 1)$ choices, which is $n - m + 1$ choices. So the total number of injections is

$$n(n-1)(n-2)\cdots(n-m+1).$$

This formula also copes nicely with the possibility that $m > n$. In that case, we know that the codomain is too small to allow any injections from the domain at all, so the answer should be 0, and that is indeed what the formula gives, since one of the factors will be 0.

Requiring f to be a surjection needs some more care and will be considered later.

If we require f to be a bijection, then that's the same as requiring f to be an injection whose codomain and domain are the same size, since both sets are finite. (See the end of § 2.7 on p. 50.) So this is the same as the injective case when $m = n$. So the number of bijections is just $n(n-1)(n-2)\cdots(n-n+1)$, which is $n!$.

Recall that, when a bijection maps a finite set to itself, it is called a *permutation* of that set (§ 2.7). So the number of permutations of an n -element set is $n!$.

2.13 BINARY RELATIONS

We often want to know when objects of one type are related in some specific way to objects of another type. For example, in considering sets of people, we might like to know who is a parent of whom. In mobile communications, systems that manage calls would use information on which smartphones contacted which cellphone towers. Some ecologists monitor predator-prey relationships among species in a geographic area. In timetabling Monash classes in a given semester, we want to know which pairs of units have at least one student in common, so that we can try to schedule classes in those units at different times.

These situations, and very many others, can be modelled by *binary relations*.

A **binary relation** consists of two sets A and B and a set of ordered pairs (a, b) where $a \in A$ and $b \in B$. The ordered pairs are used to state which members of A and B are related to each other in the required way.

Recall that the Cartesian product $A \times B$ is the set of *all* ordered pairs in which the first and second members of the pair belong to A and B respectively. This gives us a very succinct way to restate our definition. A **binary relation** consists of two sets A

and B and a subset of $A \times B$. We sometimes say that the binary relation is **from A to B** , and we may still call A the **domain** and B the **codomain**.

The two sets might be the same. A **binary relation on a set A** is a binary relation from A to itself. Each of the two sets is A , so that the relation is a subset of $A \times A$.

A binary relation is also called a **binary predicate** or a **predicate with two arguments**.

If R is the name of a binary relation, then we write xRy or $R(x, y)$ or $(x, y) \in R$ to mean that (x, y) is one of the ordered pairs in R . The notation xRy is an example of *infix* notation, where the name of the operation/function/relation is placed *between* the two things it links. The notation $R(x, y)$ is a further example of prefix notation. (Recall the discussion of prefix, infix and postfix notation on p. 47.)

For example, the Parent relation is a relation on the set $\mathbb{P}$ of all people (so the two sets are the same in this case), and the pair of people (p, q) belongs to this relation if q is a parent of p . Members of the Parent relation include:

(Ada Lovelace	,	George Gordon Byron),
(Annie Jump Cannon	,	Wilson Cannon),
(John Ferrier Turing	,	Sara Turing),
(John Ferrier Turing	,	Julius Turing),
(Alan Mathison Turing	,	Sara Turing),
(Alan Mathison Turing	,	Julius Turing),
⋮		⋮

So we may write, for example,

$$(Ada Lovelace, George Gordon Byron) \in \text{Parent},$$

or using infix notation,

$$Ada Lovelace \text{ Parent } George Gordon Byron,$$

or using prefix notation,

$$\text{Parent}(Ada Lovelace, George Gordon Byron).$$

For the mobile communications example, the two sets are different, namely a set of smartphones and a set of cellphone towers, and the relation includes pairs like

$$(Catherine Deneuve's \text{ phone}, \text{ the Eiffel Tower}).$$

It may be tempting to use binary relation names in the same way we use function names. Recall the functions *Mother* and *Father*, which enable us to write statements like

$$\begin{aligned}\text{Mother}(\text{Alan Turing}) &= \text{Sara Turing}, \\ \text{Father}(\text{Alan Turing}) &= \text{Julius Turing}.\end{aligned}$$

But we will not write “*Parent*(Alan Turing)”; such notation would treat *Parent* as a function of people, when in fact “the parent” of a person is not, in general, uniquely defined.

We saw in § 2.1.3^α that one way to specify the rule of a function $f : A \rightarrow B$ is to give its graph, i.e., to give all its ordered pairs $(x, f(x))$, which all belong to $A \times B$. So a function is a type of binary relation. To be precise, a function with domain A and codomain B is a binary relation on A and B in which, for every $a \in A$, there is a unique $b \in B$ such that (a, b) belongs to the relation. To put it the other way round (and less formally), a binary relation is a “function” where we drop the requirement that each member of the domain gives exactly one member of the codomain. With this latter viewpoint, a binary relation is sometimes called a “many-valued function” because a single member of the domain can yield many different values in the codomain (whereas normal *functions* are just single-valued). But we will not refer to “many-valued functions” because it is a contradiction in terms: a function, *by definition*, cannot be many-valued in that sense.

Examples of binary relations on sets of numbers include $=$, $\leq$, $\geq$. Examples of binary relations on sets of *sets* include $=$, $\subseteq$, $\supseteq$.

One common source of binary relations is network structures. Networks consist of nodes with links between some pairs of nodes. For example:

- In a social network, the nodes are people and the links represent which people know which other people.
- In a communications network, nodes represent the communicating devices and the links may represent messages sent from one device to another.
- In an ecological network, nodes represent species and links represent predator-prey relations among the species.
- In Monash timetabling, nodes represent the units running in a given semester, and links represent pairs of units that have at least one student in common for that semester. A small fragment of this network is shown in [Figure 2.3](#). In this example, MTH1030 and MTH1035 have no students in common (because it is prohibited to enrol in both of them), so their classes may be at the same time (or at different times), and we represent this lack of restriction by the absence of a link between the corresponding nodes. But every other pair of these units has some students that do both of them, so every other pair of nodes has a link between them.

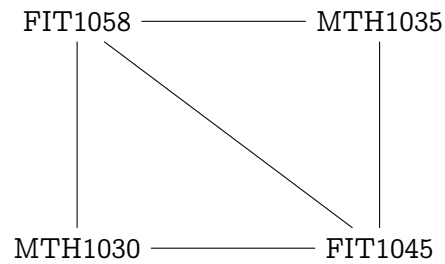


Figure 2.3: A fragment of the Monash timetabling network

- In a software system, nodes represent software components of some kind (modules, programs, ...), and the links represent some mechanism for passing information from one component to another.

In each of these cases, we have a set of nodes, and the set of links between them can be represented by a binary relation.

Another common source of binary relations is tables of data. Whenever you write a table with two columns, you are specifying a binary relation whose ordered pairs (x, y) correspond to the rows of the table, with x and y being the entries in the first and second column respectively. If you have a table with more than two columns, then often taking some (or perhaps any) pairs of columns will give you a binary relation in the same way. Similar remarks apply to data stored in other ways that may be thought of as tables, such as in spreadsheets and databases.

Important operations which you might want to do, when using a binary relation R on sets A and B , include:

- Determine the set of all members of A that actually appear as the first member of a pair in R . This is $\{x \in A : (x, y) \in R \text{ for some } y \in B\}$.
- Determine the set of all members of B that actually appear as the second member of a pair in R . This is $\{y \in B : (x, y) \in R \text{ for some } x \in A\}$.
- Given x , determine everything that is related to it by R . This is $\{y \in B : xRy\}$. This set is often denoted by $R(x)$, but we must not confuse this with the notation for the value of a function, since $R(x)$ is typically not just one single value in B , but rather a set of values, and it might even be empty.
- Given y , determine everything that is related to it by R . This is $\{x \in A : xRy\}$. This is often denoted by $R^{-1}(y)$. Again, take care to avoid confusion with the value of an inverse function, since $R^{-1}(y)$ is in general some set of values that are each related to y rather than just one value.
- For each of these sets, we may want to determine its size.

Let R be any binary relation from A to B . Its **inverse** R^{-1} is the binary relation from B to A defined by

$$yR^{-1}x \iff xRy.$$

So the inverse is constructed by just swapping the roles of the domain and codomain and reversing all the pairs. In the special case when R is actually a function, this is just the definition of the inverse of a function (see § 2.8). We can now talk of the inverse of *any* function, not just of bijections, but we have to remember that, if f is not a bijection, then f^{-1} is not a function (although it is a valid binary relation).

2.14 PROPERTIES OF BINARY RELATIONS

We now meet some important types of binary relations.

Let R be a binary relation on a set A .

We say R is **reflexive** if, for all $a \in A$, we have aRa . In other words, everything is related to itself.

Examples of reflexive binary relations include equality, $\leq$, $\geq$, $\subseteq$, $\supseteq$ (these latter two because these versions of the subset relation allow improper subsets).

Using the set of all people again, let *knows* be the binary relation that holds when one person knows another. It contains an ordered pair (p, q) of people precisely when person p knows person q . It seems reasonable to describe *knows* as reflexive, as everyone knows themselves (more or less).

Binary relations that are not reflexive include $<$, $>$, $\subset$, $\supset$. The relation *Parent* is also not reflexive.

We say R is **symmetric** if, for all $x, y \in A$, xRy implies yRx .

The relation $=$ is symmetric, but the relations $<$, $\leq$, $>$, $\geq$, $\subseteq$, $\subset$, $\supset$, $\supseteq$ are not symmetric. The relation *knows* is symmetric, assuming that we are using the word “knows” for relationships where each knows the other. The relation *Parent* is not symmetric. The hyperlink relation on the set of all webpages — consisting of all pairs (p, q) where webpage p has a hyperlink to webpage q — is also not symmetric.

We say R is **antisymmetric** if, for all $x, y \in A$, xRy and yRx together imply $x = y$. An equivalent way to say this is: for all $x, y \in A$, if xRy and $x \neq y$ then we *cannot* have yRx . So, if two *distinct* elements ($x \neq y$) are related *one way* (xRy), then they *cannot* be related the *other way* (yRx).

Being *antisymmetric* is not the same as just being *not symmetric*. In other words, “antisymmetric” and “asymmetric” are different terms meaning different things.

The relations $=$, $\leq$, $\geq$, $\subseteq$, $\supseteq$ are antisymmetric. For example, if two numbers x and y satisfy $x \leq y$ and $y \leq x$ then $x = y$. Similarly, if two sets X and Y satisfy $X \subseteq Y$ and $Y \subseteq X$ then $X = Y$. We often use this when proving that two sets are equal (see p. 9 in § 1.6): if we prove that $X \subseteq Y$ and also prove that $Y \subseteq X$ then we can conclude that $X = Y$.

It may seem slightly surprising that equality, $=$, is antisymmetric, since it is also symmetric and the two terms sound opposite. But this example illustrates that it is indeed possible for a binary relation to be both symmetric and antisymmetric. Are there any other cases where this happens?

The hyperlink relation on webpages is not antisymmetric. This illustrates the fact that it is possible for a binary relation to be neither symmetric nor antisymmetric.

The relation *knows* is also not antisymmetric.

We say R is **transitive** if, for all $x, y, z \in A$, xRy and yRz together imply xRz .

The relations $=$, $<$, $\leq$, $>$, $\geq$, $\subset$, $\subseteq$, $\supset$, $\supseteq$ are all transitive, but $\neq$ is not. The relation *knows* is not transitive: a “friend of a friend” is not necessarily your friend! The relation *Parent* is not transitive either.

2.15 COMBINING BINARY RELATIONS

Viewing a binary relation as a set of ordered pairs enables us to apply ordinary set operations on them. Suppose R and S are both binary relations from A to B . Then their **union** $R \cup S$ is the binary relation from A to B containing every pair that belongs to at least one of R and S .

$$R \cup S = \{(x, y) : xRy \text{ or } xSy\}.$$

For example,

$$\text{Parent} = \text{Mother} \cup \text{Father}.$$

The **intersection** $R \cap S$ is the binary relation from A to B containing every pair that belongs to both R and S .

$$R \cap S = \{(x, y) : xRy \text{ and } xSy\}.$$

For example,

$$\text{Employer} \cap \text{Parent}$$

is the set of pairs (x, y) such that y is both an employer *and* a parent of x . This includes, for example, the pairs

$$\begin{aligned} &(\text{Prince William}, \text{King Charles III}), \\ &(\text{Leonard Lauder}, \text{Estée Lauder}). \end{aligned}$$

If R is a binary relation from A to B , and S is a binary relation from B to C , then the **composition** $S \circ R$ is the binary relation from A to C whose set of ordered pairs is

$$\{(x, z) : \text{there exists } y \in B \text{ such that } xRy \text{ and } ySz\}.$$

For example, the composition

$$\text{Employer} \circ \text{Parent}$$

contains all pairs (x, z) such that z employs a parent of x . If you are x , then an employer of one of your parents is z , and the pair of you are related by $\text{Employer} \circ \text{Parent}$. But the *order matters*: this relation is not the same as the one that relates people to their own employers' parents.

In the special case when R and S are both functions, their composition is just their composition as functions, using the definition of function composition given in § 2.9.

Let's investigate the composition of any relation R from A to B with its inverse relation R^{-1} . The composition $R^{-1} \circ R$, which goes from A to itself, satisfies

$$\begin{aligned} (x, z) \in R^{-1} \circ R &\iff \text{there exists } y \text{ such that } xRy \text{ and } yR^{-1}z \\ &\iff \text{there exists } y \text{ such that } xRy \text{ and } zRy \\ &\quad \text{(using the definition of } R^{-1}\text{).} \end{aligned}$$

So, in the composition $R^{-1} \circ R$, two elements of A are related by $R^{-1} \circ R$ precisely when there is an element of B that they are both related to by R .

Similarly, the composition $R \circ R^{-1}$, which goes from B to itself, relates two elements of B precisely when there is an element of A that is related to both those elements of B .

Binary relations, like functions, can be composed with themselves. Consider again the binary relation *knows* on the set of all people. In the composition $\text{knows} \circ \text{knows}$, two people are related if they have a mutual acquaintance, i.e., they each know someone who knows the other. We can extend this. In the composition $\text{knows} \circ \text{knows} \circ \text{knows}$, one person is related to another if they know someone who knows someone who knows the other. The five-fold composition

$$\text{knows} \circ \text{knows} \circ \text{knows} \circ \text{knows} \circ \text{knows} \circ \text{knows}$$

is said to relate every pair of people on Earth. This is the principle known as “six degrees of separation”; it uses the *knows* relation *six* times, with *five* compositions.

As for function composition, we can use exponents in parentheses to denote composition of relations with themselves: $R^{(n)}$ is the composition of n copies of R . So the composition we wrote above, for six degrees of separation, could be written $\text{knows}^{(6)}$.

Sometimes, for a binary relation R on a set A , we may want to go further and identify every pair of elements of A that are linked by *any* chain, no matter how long, of pairs in R .

The **transitive closure** of a binary relation R on a set A is the unique binary relation R^+ on A such that, for all $x, y, z \in A$,

- (i) if xRy then xR^+y ;
- (ii) if xR^+y and yRz then xR^+z ;
- (iii) if xRy and yR^+z then xR^+z .

Actually, you can drop either the second or third (but not both) of these conditions from the definition. You cannot drop the first condition, which gets the transitive closure process started.

The transitive closure R^+ certainly contains all pairs in R , by condition (i) in its definition. It also contains all pairs in $R^{(2)}$, and all pairs in $R^{(3)}$, and so on. So the set of pairs in the transitive closure is given by

$$R^+ = R \cup R^{(2)} \cup R^{(3)} \cup R^{(4)} \cup \dots,$$

which may be written more succinctly as

$$R^+ = \bigcup_{i=1}^{\infty} R^{(i)}.$$

The transitive closure of *knows* identifies whenever two people can be linked by an arbitrarily long chain of social connections. Under the hypothesis of six degrees of separation,

$$\text{knows}^+ = \text{knows}^{(6)}$$

and every pair of people on Earth are linked in this way.

The transitive closure of a binary relation is always transitive, hence the name.

2.16 EQUIVALENCE RELATIONS

It seems fair to regard *equality* as the most fundamental binary relation. No matter what kind of objects we are working with, we want to be able to say whether or not two of them are really the same. If we cannot do even that, it is hard to imagine having any useful discussion about such objects at all. So, even if there is no other relationship to speak of among the objects we are discussing, there must be an equality relation. We will always feel free to use equality on any set at all, without announcing its existence beforehand.

In many situations, different objects may be treated as being equivalent for *some* purposes even though they must be treated differently for other purposes. For example, if two different students are in the same weekly tutorial class, then they can be treated as equivalent for FIT1058 timetabling purposes, even though their other subjects may be different so that they cannot be treated as equivalent for other timetabling purposes. Define the binary relation $\overset{1058}{\equiv}$ on the set $\mathcal{S}$ of all students by

$$p \overset{1058}{\equiv} q \iff p \text{ and } q \text{ are in the same FIT1058 tutorial,}$$

for any $p, q \in \mathcal{S}$.

If a binary relation is to capture some notion of “equivalence”, what properties should it have? We can use equality as a guide, since “equivalence” should be like equality but

a bit “looser” in that two things can be equivalent without being identical. We have just seen that the equality relation is reflexive, symmetric, and transitive. This is also what we would expect of a binary relation that tells us when two things are equivalent. Every object, of any kind, is certainly equivalent to itself; if one object is equivalent to another, then the latter object must also be equivalent to the former; and if an object is equivalent to another, which in turn is equivalent to a third object, then the first object must also be equivalent to the third object.

Equality is also antisymmetric. But we will not add this to our requirements of equivalence in general, since antisymmetry requires that if an object is equivalent to another, which in turn is equivalent to our first object, then the two objects must actually be equal. This would be tantamount to saying that equivalence implies equality, which would mean that we have no form of equivalence other than equality itself, which is too narrow.

With these thoughts in mind, we make the following definition.

An **equivalence relation** is a binary relation that is reflexive, symmetric, and transitive.

It is routine to check that the binary relation $\stackrel{1058}{\equiv}$ is an equivalence relation.

Other examples of equivalence relations:

- Let $m \in \mathbb{N}$. Two integers x, y are **congruent modulo m** if $x - y$ is a multiple of m (often written $m \mid (x - y)$). We write this equivalence as $x \equiv y \pmod{m}$.
- Two real numbers x, y are **equivalent in integer part** if, when each is rounded to the nearest integer, they become equal. We write this as $x \stackrel{\text{int}}{=} y$.
- Two triangles in the plane are **congruent** if one can be moved onto the other by some sequence of translations, rotations and reflections.
- Every function $f: A \rightarrow B$ defines an equivalence relation as follows. If $x, y \in A$, we write $x \sim_f y$ if $f(x) = f(y)$. This relation $\sim_f$ is an equivalence relation on A .
- The transitive closure of any reflexive symmetric binary relation is an equivalence relation.
- For any relation R , the relations $R \circ R^{-1}$ and $R^{-1} \circ R$ are equivalence relations.

Let R be an equivalence relation on a set A . An **equivalence class** of A under R is a nonempty subset $X \subseteq A$ in which

- (i) whenever $x, y \in X$, we have xRy ;
- (ii) there are no $x \in X$, $z \notin X$ such that xRz .

So all members of X are equivalent to each other, but no member of X is equivalent to anything outside X . This condition may be rephrased as saying that X is a maximal subset of A in which all elements are equivalent. (Reminder: the word “maximal” does not mean that X is largest in size among all those with this property. Rather, it means that X cannot be enlarged while maintaining this property; in other words, no proper superset of X has the property. See § 1.7.)

Recalling our earlier examples of equivalence relations:

- For the binary relation $\stackrel{1058}{\equiv}$, the equivalence classes are the FIT1058 tutorials (or, more precisely, the sets of students allocated to each tutorial).
- For congruence modulo m , the equivalence classes are the sets $\{km + r : k \in \mathbb{Z}\}$, for each $r \in \{0, 1, \dots, m-1\}$. These sets are called the **residue classes modulo m** . There is one such set for each $r \in \{0, 1, \dots, m-1\}$. For example, if $m = 2$, then $r \in \{0, 1\}$ and we have two equivalence classes: the set of all even integers (for $r = 0$), and the set of all odd integers (for $r = 1$).
- For the relation $\stackrel{\text{int}}{\equiv}$, the equivalence classes are the intervals $[n - \frac{1}{2}, n + \frac{1}{2})$ for all $n \in \mathbb{Z}$.
- For congruent triangles, the equivalence classes each contain all triangles of one particular shape and size (so they all have the same angles and side lengths, but otherwise can be in any location and orientation in the plane).
- For $\sim_f$, the equivalence classes are the preimages $f^{-1}(y) = \{x : f(x) = y\}$ of each $y \in \text{im } f$.

In each of these cases, the equivalence classes divide up the set A neatly, so that each element of A belongs to exactly one equivalence class. These are manifestations of a general phenomenon.

Theorem 12. The equivalence classes of an equivalence relation R on a set A form a partition of A .

Proof. To show that the equivalence classes form a partition of A , we need to show that (a) every member of A belongs to an equivalence class, and (b) no two different equivalence classes overlap. (Note that equivalence classes are nonempty by definition.)

(a)

Let $x \in A$. Define X to be the set of all members of A that are equivalent to x under our equivalence relation R :

$$X = \{y \in A : xRy\}.$$

We claim that this is an equivalence class of R that contains x . We prove parts (i) and (ii) of the definition of equivalence relation, in turn.

Firstly, suppose $y, z \in X$. By definition of X , this implies that xRy and xRz . By symmetry and transitivity of R , it follows that yRz . So every pair of members of X are equivalent, satisfying the first condition for an equivalence class.

Now suppose that $v \in X$, $w \notin X$ such that vRw . The fact that $v \in X$ implies that xRv . This, combined with vRw and transitivity, gives xRw . Therefore $w \in X$. This contradicts the assumption that $w \notin X$. So there cannot be any $v \in X$, $w \notin X$ such that vRw . So the second condition for an equivalence class is satisfied.

We conclude that X is an equivalence class.

Furthermore, X clearly contains x , since xRx by reflexivity.

So every member of A belongs to an equivalence class.

(b)

Suppose X and Y are overlapping equivalence classes of R . So $X \cap Y \neq \emptyset$. Let $x \in X \cap Y$.

We claim that $X = Y$.

To do this, we consider $Y \setminus X$ and $X \setminus Y$ and show that they are both empty, which implies that $X = Y$. (See [Theorem 4](#) and (1.17).)

Consider first the possibility that $Y \setminus X \neq \emptyset$. Suppose $y \in Y \setminus X$. Because X is an equivalence class and $y \notin X$, it follows that $(x, y) \notin R$. But because Y is an equivalence class and $x, y \in Y$, it follows that $(x, y) \in R$. So we have a contradiction in this case. Therefore $Y \setminus X \neq \emptyset$ is impossible, so $Y \setminus X = \emptyset$.

It remains to consider the possibility that $X \setminus Y \neq \emptyset$. But the argument here is the same as that of the previous paragraph, with X and Y interchanged. So $X \setminus Y = \emptyset$.

So we have $Y \setminus X = X \setminus Y = \emptyset$. This implies that $X = Y$.

So the only way two equivalence classes can overlap is if they are identical.

So we have shown that every member of A belongs to exactly one equivalence class. So the equivalence classes of R form a partition of A . $\square$

2.17 RELATIONS

Binary relations are *binary* in the sense that they consist of ordered *pairs*. It is often useful to consider relations whose members have more objects.

A **ternary relation** on sets A, B, C is a set of triples (x, y, z) such that $x \in A$, $y \in B$ and $z \in C$. (Triples written in parentheses are always considered to be ordered.) In other words, it is a subset of $A \times B \times C$. For example, a set of points in real three-dimensional space is a ternary relation on $\mathbb{R}$. A list of names in which each is listed as

first name middle name surname

could be represented as a ternary relation in which each triple has the form

$(\text{first name}, \text{middle name}, \text{surname})$.

Many Chinese names could be represented in a ternary relation with triples

(family name, personal name, generation name).

Dates can be represented by (day, month, year) triples, so they too can be represented naturally in a ternary relation.

More generally, an **n -ary relation** on sets $A_1, A_2, \dots, A_n$ consists of n -tuples $(x_1, x_2, \dots, x_n)$ where $x_i \in A_i$ for all $i \in \{1, \dots, n\}$. In other words, it is a subset of $A_1 \times A_2 \times \dots \times A_n$. The set A_i is called the **i -th domain**.

When you use a table with n columns, its rows represent the n -tuples in an n -ary relation. For each i , the set A_i is a set that contains all objects appearing in the i -th column (and is allowed to contain more, much as a codomain of a function is allowed to contain more than just the image). So, when you work with spreadsheets, you are usually working with n -ary relations whose members represent the rows of the spreadsheet. Relations are fundamental in databases, so much so that the term *relational database* is used for one of the most widely used types of database.

An n -ary relation is also called an **n -ary predicate** or a **predicate with n arguments**.

2.18 COUNTING RELATIONS

How many binary relations from A to B are there, where A and B are finite sets?

Put $m := |A|$ and $n := |B|$.

A binary relation is just a subset of $A \times B$, so the number of binary relations from A to B is just the number of subsets of $A \times B$. This is just the size of the power set $\mathcal{P}(A \times B)$, which is

$$2^{|A \times B|},$$

by (1.1). But $|A \times B| = mn$, by (1.19). So

$$\# \text{ binary relations from } A \text{ to } B = 2^{mn}.$$

In the special case when $A=B$, with $|A| = m$, we have

$$\# \text{ binary relations on } A = 2^{m^2}.$$

Now let's look at more general relations. How many n -ary relations are there on sets $A_1, A_2, \dots, A_n$?

Put $m_i := |A_i|$ for each $i = 1, 2, \dots, n$.

An n -ary relation is just a set of n -tuples from $A_1 \times A_2 \times \dots \times A_n$. So the number of n -ary relations is just the size of the power set of $A_1 \times A_2 \times \dots \times A_n$:

$$\# \text{ } n\text{-ary relations on } A_1, \dots, A_n = 2^{|A_1 \times A_2 \times \dots \times A_n|} = 2^{m_1 m_2 \dots m_n}.$$

In the special case when all sets are the same (say, $A_i = A$ for all i) and have size m , we have

$$\# \text{ } n\text{-ary relations on } A = 2^{m^n}.$$

2.19 EXERCISES

1. Let A be a finite set. How many indicator functions with domain A are there?
2. Let A and B be any sets. Express the indicator functions of each of the following in terms of the indicator functions χ_A and χ_B .

(a) $\overline{A}$

(b) $A \cap B$

(c) $A \setminus B$

(d) $A \Delta B$

3. If two functions $f: A \rightarrow \mathbb{R}$ and $g: A \rightarrow \mathbb{R}$ have the same domain A and give real numbers as their values, then their **sum** $f + g: A \rightarrow \mathbb{R}$ is defined for all $x \in A$ by

$$(f + g)(x) = f(x) + g(x).$$

What is the sum of all the indicator functions of all the subsets of a finite set?

4. Draw a Venn diagram showing each of the following sets of functions and the relationships between them: functions, injections, surjections, bijections, identity functions, binary relations, ternary relations, relations.

5. Functions can be viewed as sets of ordered pairs, so we can combine functions using set operations. The result will be a binary relation but, depending on the operation, it may or may not be another function.

Suppose $f: A \rightarrow B$ and $g: C \rightarrow D$ are functions. Which of the following is always a function?

$$f \cap g; \quad f \cup g; \quad f \setminus g; \quad g \setminus f; \quad f \Delta g; \quad f \times g.$$

For each that is always a function, give its definition in the usual form, including showing how the rule depends on f and g . For each that is not necessarily a function, explain why this is the case (e.g., with the help of examples for f and g under which the operation does not give a function).

6. Suppose that $f: A \rightarrow B$ and $g: C \rightarrow D$ are functions with disjoint domains: $A \cap C = \emptyset$. Is the disjoint union $f \sqcup g$ always a function? If so, give its definition in terms of f and g . If not, why not?

7. Give an example of a function $f: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ which is an injection. Even better, give one which is a bijection.

8. A **bitstring** is a string over the binary alphabet $\{0,1\}$; in other words, it is a member of $\{0,1\}^*$.

(a) Give a bijection from $\{0,1\}^*$ to $\mathbb{N}$.

Now let $\{0,1\}^{*+}$ denote the set of all finite nonempty tuples (or sequences) of bitstrings. Examples of members of $\{0,1\}^{*+}$ include:

$$(011, 1, 10001), \quad (10, 10, 10, 10), \quad (\epsilon, 1011), \quad (1010010001).$$

(b) (Challenge) Give a bijection from $\{0,1\}^{*+}$ to $\{0,1\}^*$.

9. Let A be a set of n elements and let $x \in A$. We say that a function $f: A \rightarrow A$ **fixes** x if $f(x) = x$. We also say in this case that x is a **fixed point** of f .

This says nothing about what f does to other elements of A ; it is possible that some other elements are fixed by f too, or that none are. All this definition requires is that f sends x to itself.

Now let $X \subseteq A$. The function f **fixes** X if it fixes all elements of x . So, for all $x \in X$, we have $f(x) = x$. In other words, the restriction of f to X is just the identity function on X :

$$f|_X = i_X.$$

Members of $A \setminus X$ may or may not be fixed by f .

Let $\text{Fix}(X)$ be the set of bijections on A that fix X , and let F be the set of all bijections on A that fix at least one element of A .

Using Exercise 1.13 (i.e., Exercise 13 in Chapter 1), express $|F|$ in terms of the set sizes $|\text{Fix}(X)|$ using all $X \subseteq A$.

10. For each $k \in \{1, \dots, n\}$ and any subset $X \subseteq A$ with $|X| = k$, write f_k for the number of bijections on A that fix X .

(a) Why don't we include X in the notation f_k , given that its definition refers to X ?

(b) Give an expression for f_k .

(c) Express $|F|$ in terms of $f_1, f_2, \dots, f_n$, and then use your expression for f_k to give an expression for $|F|$.

(d) Hence express the number of fixed-point-free bijections on A in terms of $f_1, f_2, \dots, f_n$. A function is **fixed-point-free** if it has no fixed points.

- (e) For each of the following sets, what *proportion* of all bijections on the set are fixed-point-free? $\{0, 1\}$; $\{\spadesuit, \clubsuit, \diamondsuit, \heartsuit\}$; the set of ten decimal digits; the 26-letter English alphabet.

You may need to use a program like Wolfram Alpha, or a spreadsheet, to help calculate the latter two.

- (f) What is the largest set for which you can determine this proportion, and what is the value of the proportion for that set? (Use a spreadsheet or a program if you wish.)

11.

- (a) Write down all surjections from $\{1, 2, 3, 4\}$ to $\{a, b, c\}$. Compare your answer with Exercise 1.20.

- (b) Define a surjection from

$$\{\text{surjections from } \{1, 2, 3, 4\} \text{ to } \{a, b, c\}\}$$

to

$$\{\text{partitions of } \{1, 2, 3, 4\} \text{ into three parts}\}.$$

- (c) What sizes can the preimages of members of its image be?

12. Suppose $f: A \rightarrow B$ and $g: C \rightarrow D$ are bijections, $A \cap C = \emptyset$ and $B \cap D = \emptyset$. Since $A \cap C = \emptyset$, the disjoint union $f \sqcup g$ exists. Show that $f \sqcup g$ is a bijection and express the inverse of $f \sqcup g$ in terms of the inverses of f and g .

13. If the function f has an inverse function, which of its iterated compositions $f^{(k)}$ have inverse functions (where $k \in \mathbb{N}$)?

14. Determine all functions from $\{1, 2, 3\}$ to itself whose indefinitely iterated composition is a constant function (i.e., $f^{(k)}$ is a constant function if k is large enough).

For a challenge: investigate when this happens in general. Try to characterise those functions $f: \{1, 2, \dots, n\} \rightarrow \{1, 2, \dots, n\}$ such that, for large enough k , the iterated composition $f^{(k)}$ is constant.

15. This exercise is about the **Caesar slide** cryptosystem, one of the oldest and simplest cryptosystems. It is not secure, but its core operation is used in many stronger and more complex cryptosystems.

Caesar slide encrypts a message by sliding each letter along the alphabet, rightwards, by some fixed number of steps. This fixed number is the key, k , and the same amount of sliding is done to each letter of the message. The sliding is done with wrap-around, so if sliding ever takes you beyond the end of the alphabet, then you resume sliding at the

start of the alphabet. In effect, we treat the alphabet as a circle rather than a straight line.

For example, if the message is⁵

thefamilyofdashwoodhadlongbeen

and the key is $k = 3$ (Julius Caesar's favourite), then the cyphertext is

wkhidplobrigdvkzrrgkdgorqjehhq

Here, sliding t along the alphabet by 3 steps gives w . We see the wrap-around when encrypting the letter y , since sliding it step-by-step gives z , then (wrapping around) a , then b .

Decryption is the reverse of encryption, meaning that we slide k steps leftwards instead of rightwards, again with wrap-around.

We now give a formal definition of this cryptosystem, using the definition of cryptosystems given in §2.10.

- The message space M is the set of all strings of English lower case letters (without blanks).
- The key space K is the English lower-case alphabet, $\{a, b, c, \dots, y, z\}$.
- The cypher space C is the same as the message space: $C = M$.
- The encryption and decryption functions, to be defined below, need to use the **letter addition** operation defined by the following table.

⁵ from the first sentence of *Sense and Sensibility* by Jane Austen, first published by Thomas Egerton in London in 1811.

+	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
a	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
b	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a
c	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b
d	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c
e	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d
f	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e
g	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f
h	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g
i	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h
j	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i
k	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j
l	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k
m	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l
n	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m
o	o	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n
p	p	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
q	q	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p
r	r	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q
s	s	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r
t	t	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s
u	u	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t
v	v	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u
w	w	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v
x	x	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w
y	y	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x
z	z	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y

In effect, letters correspond to numbers in $\{0, 1, 2, \dots, 24, 25\}$, and when we do the letter addition $\alpha + \beta$, we start at α and slide to the right (with wrap-around as needed) by a number of steps given by β . Letter addition is commutative and associative. We can also define letter subtraction, where we slide to the left instead of the right.

- The Caesar slide encryption function $e : M \times K \rightarrow M$ is defined for any message $m \in M$ and key $k \in K$ as follows. Let n be the length of m and let its letters be $m_1, m_2, \dots, m_n$, so we can write $m = m_1 m_2 \dots m_n$. Then $e(m, k)$ is the string $c = c_1 c_2 \dots c_n$ of n letters where, for each i , the i -th letter c_i is obtained from the i -th letter of the message by adding the key letter to it, using the wrap-around method described above. Formally,

$$\begin{aligned} e(m, k) &= c_1 c_2 \dots c_n, \\ c_i &= m_i + k. \end{aligned}$$

The addition here is *letter addition*. Note that, for each i , the *same* key letter is used. Although message and cypher letters may (and usually do) change as you go along the message, the key letter used for encryption does not change.

- The Caesar slide decryption function $d : M \times K \rightarrow M$ is defined for any cyphertext $c = c_1c_2 \cdots c_n \in M$ and key $k \in K$ as follows. Its value $d(c, k)$ is the string $m = m_1m_2 \cdots m_n$ where, for each i , the i -th letter m_i is obtained from the i -th letter of the cyphertext by subtracting the key letter from it, again using wrap-around. Formally,

$$\begin{aligned} d(c, k) &= m_1m_2 \cdots m_n, \\ m_i &= c_i - k. \end{aligned}$$

Again, we use the same key letter at all positions.

Now refer to the definitions of *embeddable*, *equivalent* and *idempotent* given in Assignment 1.

Prove that the Caesar slide cryptosystem is idempotent.

16. If f is an injection and f^{-1} is its inverse *relation*, what can you say about $f \circ f^{-1}$ and $f^{-1} \circ f$? What kinds of functions or relations are they, and what are their domains and codomains?
17. If a binary relation is symmetric, what can you say about its inverse relation?
18. Recall the Parent relation from § 2.13.
Suggest a good name for its transitive closure.
Then use that relation (i.e., the transitive closure) to define formally the relation that links two people that are related to each other (however distantly).
19. Rephrase the Collatz Conjecture as a statement about the transitive closure of the Collatz function.
20. Below we give some binary relations on the set of all Python programs. The Python programs are denoted by P and Q , and the binary relations are denoted by $\simeq_1, \simeq_2, \dots, \simeq_{10}$. For each relation, determine whether or not it is an equivalence relation, and give reasons.

notation	definition of when the relation holds
$P \simeq_1 Q$	P and Q have the same number of characters
$P \simeq_2 Q$	P and Q compute the same function
$P \simeq_3 Q$	P and Q have the same set of variable names
$P \simeq_4 Q$	P and Q have at least one variable name in common
$P \simeq_5 Q$	P and Q have no variable names in common
$P \simeq_6 Q$	P and Q were written by the same set of programmers
$P \simeq_7 Q$	P and Q have at least one coauthor in common
$P \simeq_8 Q$	P was completed before Q
$P \simeq_9 Q$	P was completed on the same day as Q
$P \simeq_{10} Q$	P was completed after Q

21. What can you say about the transitive closure of an equivalence relation?

22. The anagram relation on the set of all English words is defined as follows. Let $x_1x_2\cdots x_m$ and $y_1y_2\cdots y_n$ be two English words of lengths m and n respectively, where the x_i and y_j are their letters ($1 \leq i \leq m$, $1 \leq j \leq n$). The ordered pair of words

$$(x_1x_2\cdots x_m, y_1y_2\cdots y_n)$$

belongs to the anagram relation if and only if there exists a bijection $f: \{1, 2, \dots, m\} \rightarrow \{1, 2, \dots, n\}$ such that, for all $i \in \{1, 2, \dots, m\}$, we have $x_i = y_{f(i)}$.

We assume all letters belong to the usual lower-case English alphabet $\{a, b, \dots, z\}$. The exact choice of dictionary does not matter for this exercise.

- (a) Find the largest set of three-letter words you can that are all related to each other by the anagram relation.
- (b) For each $k = 1, 2, 3, 4$, find an English word of length k that is not related to any other word by anagram.
- (c) Prove that anagram is an equivalence relation.
- (d) ^{ω} (Programming challenge) Using a standard open-access list of English words, determine the number of equivalence classes of anagram on the set of words in that list.

23. How many non-reflexive binary relations are there on a set of size n ?

24. Recall that when you have a table with n columns, its rows represent the n -tuples in an n -ary relation. For $1 \leq i \leq n$, let A_i be a set containing all the entries in column i (and possibly more elements), so that the columns of the table define an n -ary relation on $A_1 \times A_2 \times \cdots \times A_n$. Assuming the columns of this table are all distinct, how many

ternary relations can you construct by choosing columns from this table of n columns?

PROOFS

Why should a programmer learn to write mathematical proofs? The most obvious answer is that proofs give rigorous justification for properties of your programs or of the structures that they work with. A computer *scientist* is not just a computer *user* or *hobbyist* or *fan*. A computer scientist provides rational support for their claims. Sometimes, this can be done by computational experiments, where programs are run on a large number of different inputs that hopefully form a representative sample of the situations of interest, and the outputs are studied carefully and perhaps analysed statistically. But a proof gives a more fundamental kind of support for a claim. It is independent of the specific technology which is used to develop and run the program. It applies to a far wider range of scenarios than can ever actually be run in a set of computational experiments.

In this chapter, we learn about the nature of mathematical proofs and the art of writing them. We treat the main types of proof, with emphasis on proof by induction. We conclude in § 3.14 by reflecting further on the role of proofs in computer science. Proofs are not only a tool for proving statements about programs; proofs are, themselves, like programs in many ways, and writing them is like programming, and developing skill in proof-writing will make you a better programmer.

3.1^α THEOREMS AND PROOFS

A **theorem** is a mathematical statement that has been proved to be true.

You may also come across the terms “proposition”, “lemma” and “corollary”. They are also theorems, but the different terms tell us something about how the theorem is used. A **lemma** is a theorem whose sole purpose is to be used in the proof of a more significant theorem later.^{1,2} A **proposition** is a theorem that is (unlike lemmas) of interest in its own right, but the term is typically used for theorems that are easy to prove and are

¹ Lemmas can tend to be highly technical and are often forgotten even in cases where the theorem they help prove becomes famous. But some lemmas have found fame in their own right, e.g., the Handshaking Lemma, which you’ll meet later in this unit, and Burnside’s Lemma.

² These days, the plural of “lemma” is “lemmas”, as you would expect. But, traditionally, the plural was “lemmata”, which you may encounter in old books or papers. Are there other English words where the plural ending is -ta?

of less significance than other theorems being proved in the same article/chapter/book. We also use the term “proposition” in a more specific sense from next week onwards. A **corollary** is a theorem that follows almost immediately from another theorem that has just been stated.

A **proof** of a claim is a step-by-step argument that establishes, logically and with certainty, that the claim is true.

A proof consists of a finite sequence of statements, culminating in the claim that is being proved. These statements are often called the **steps** of the proof. Each statement in the proof must be one of the following:

- something you already knew before the start of the proof, i.e.,
 - a definition,
 - an axiom (i.e., some fundamental property that is always taken to be true for the objects under discussion, such as the distributive law $x(y + z) = xy + xz$ for numbers),
 - a previously-proved theorem;

or

- an assumption, as a start towards proving something that is true under that assumption;

or

- a logical consequence of some of the previous statements. In other words, there must be some previous statements which, together, imply the current statement.
- The last statement in the sequence should establish that the claim follows from some previous statements in the proof. If the last statement is, by that stage, an obvious consequence of the statements before it, it is often omitted.

A proof must be verifiable, so it must be written clearly. It must be able to be read sequentially, with each step depending only on what comes before it. It should not be necessary to read ahead to determine if a step is correct (although it’s ok to read ahead to help your understanding).

We announce the beginning of a proof with the heading Proof. The end-of-proof symbol $\square$ indicates the end of the proof.³

³ Another traditional way to indicate the end of a proof is using the acronym “QED”, which stands for the Latin phrase “quod erat demonstrandum”, meaning “which was to be proved”. Occasionally, the end of a proof is indicated by //.

We illustrate these concepts with the following theorem and proof, which you have seen before in [Exercise 7](#). We number the steps of the proof, and give each its own line, to help with later discussion. But we won't normally do this in proofs.

Theorem 13. For any sets A and B , if $A \subseteq B$ then $\mathcal{P}(A) \subseteq \mathcal{P}(B)$.

Proof.

- (1) Assume $A \subseteq B$.
- (2) Let $X \in \mathcal{P}(A)$.
- (3) Therefore $X \subseteq A$, by definition of $\mathcal{P}(A)$.
- (4) Therefore $X \subseteq B$.
- (5) Therefore $X \in \mathcal{P}(B)$.
- (6) So we have shown that $X \in \mathcal{P}(A)$ implies $X \in \mathcal{P}(B)$.
- (7) Therefore $\mathcal{P}(A) \subseteq \mathcal{P}(B)$.

□

Think about the role of each step in this proof.

- What type of proof step is it? (See our listing of types of proof steps above.)
- Does it make use of any *previous* steps? If not, why not? If so, how?
- Does it use any *subsequent* steps? If not, good! If so, we have a problem!

The following table repeats all the proof steps, slightly expanding some of them, and annotating each step to show how it relates to our discussion of proof steps above.

proof step	comment
(1) Assume $A \subseteq B$.	The theorem statement is about what happens under the assumption that $A \subseteq B$, so we start by making this assumption .
(2) Let $X \in \mathcal{P}(A)$.	This is a definition . Its purpose is to give a name to a general member of $\mathcal{P}(A)$, so we can talk about it.
(3) Therefore $X \subseteq A$, by definition of $\mathcal{P}(A)$.	$X \subseteq A$ is a logical consequence of (2) and the definition of power set.
(4) Therefore $X \subseteq B$, by (1) and (3).	$X \subseteq B$ is a logical consequence of (3) $X \subseteq A$ and (1) $A \subseteq B$.
(5) Therefore $X \in \mathcal{P}(B)$, by (4) and the definition of $\mathcal{P}(B)$.	This is a logical consequence of (4) $X \subseteq B$ and the definition of power set.
(6) So $X \in \mathcal{P}(A)$ implies $X \in \mathcal{P}(B)$.	This really just summarises what we've done over steps (2)–(5).
(7) Therefore $\mathcal{P}(A) \subseteq \mathcal{P}(B)$.	This is a logical consequence of (6) and the definition of the subset relation.

3.2 LOGICAL DEDUCTION

The backbone of any proof consists of its logical deductions. These are the steps where we deduce a **logical consequence** of previous steps. If a proof does not have logical deductions, then it's just a collection of known facts and assumptions, and we get nothing new. It's a kind of "logical jelly" without structure or substance.

When making a logical deduction from a previous statement in a proof, the fundamental principle is:

If you've previously established P
and also that P implies Q
then you can deduce Q .

This principle is known as **modus ponens**. We usually don't bother to refer to it by that name when we're doing proofs, though, as we use it very often and it is so natural (indeed, it's the very essence of logical deduction itself).

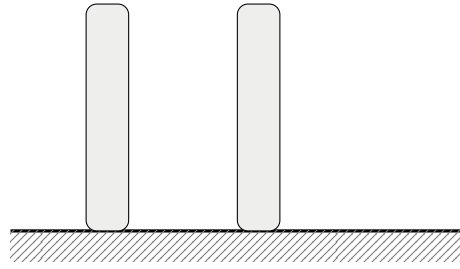


Figure 3.1: Two dominoes which could fall to the right

For example, consider the deduction we made at step (3) of the proof of [Theorem 13](#). At that stage, we know from earlier steps that

- $X \in \mathcal{P}(A)$, which we can treat as true simply because it's the definition of X (step (2));
- if $X \in \mathcal{P}(A)$ then $X \subseteq A$, which comes from the definition of $\mathcal{P}(A)$.⁴

If we let P stand for $X \in \mathcal{P}(A)$, and Q stand for $X \subseteq A$, then $P \Rightarrow Q$ represents the assertion that

$$\text{if } X \in \mathcal{P}(A) \text{ then } X \subseteq A.$$

So, step (2) is P , the definition of power set gives $P \Rightarrow Q$, and logical deduction (or *modus ponens*, if we want to practise our Latin) then gives Q .

The role of *implication* in logical deduction is crucial. An implication $P \Rightarrow Q$ is the link that translates the truth of P into the truth of Q . So let us consider it further.

Suppose we have two dominoes standing on their ends, a short distance apart and with their faces parallel, as shown side-on in [Figure 3.1](#).

Let P be the statement that the left domino falls to the right, and let Q be the statement that the right domino falls to the right. Each of these statements could be true or false. There is no requirement here for either domino to fall; it's fine for them both to remain standing. It's also fine for the left domino to remain standing but for the right one to fall (by whatever means). But if the left domino falls, then the right domino must fall too. It is impossible to have the left domino fall with the right domino remaining standing. So we have three possible situations, which we'll represent as ordered pairs:

(left stands, right stands)
 (left stands, right falls)
 (left falls, right falls)

⁴ That definition actually gives “if and only if” here: $X \in \mathcal{P}(A) \Leftrightarrow X \subseteq A$. But we do not need the reverse implication right now.

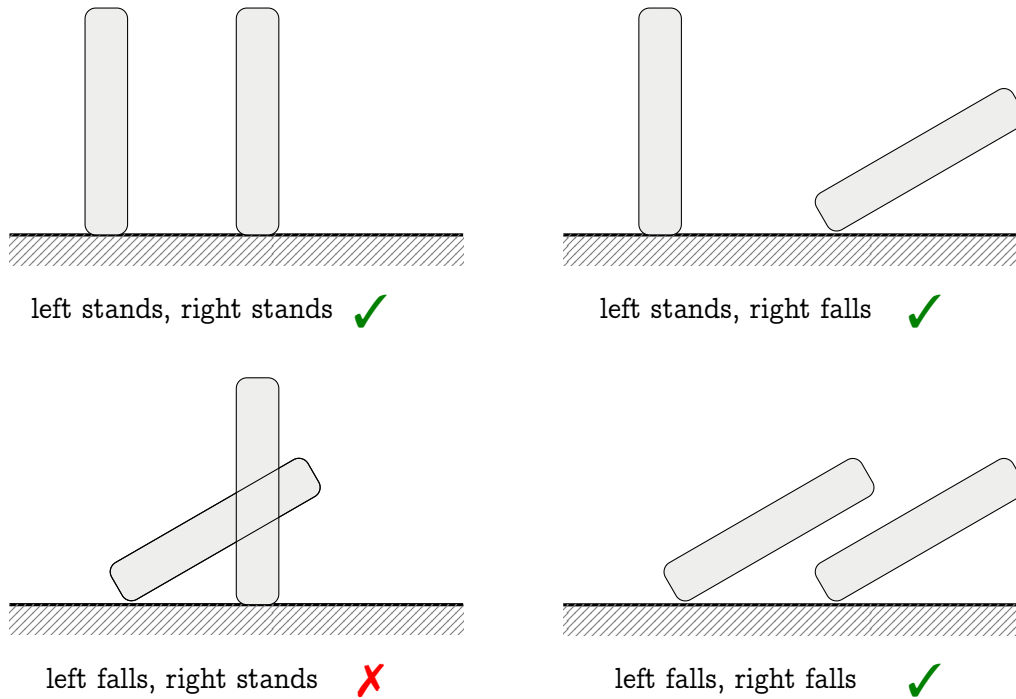


Figure 3.2: Two dominos, standing or falling: three possible situations, one impossible one.

We can depict these various possibilities using sets. Let P be the set of those situations where the left domino falls, and let Q be the set of those situations where the right domino falls. Then

$$\begin{aligned} P &= \{(\text{left falls, right falls})\}, \\ Q &= \{(\text{left falls, right falls}), (\text{left stands, right falls})\}. \end{aligned}$$

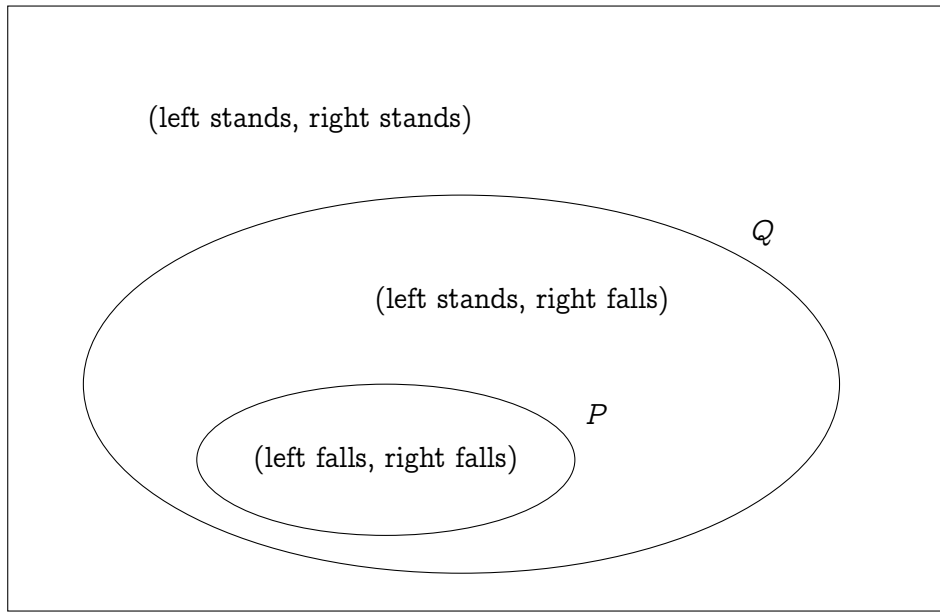
The various situations and the sets P and Q are shown in a Venn diagram in [Figure 3.3](#).

Observe that the impossible situation

(left falls, right stands)

is not shown on the Venn diagram. If it were possible, then it would belong to $P \setminus Q$ and then P would not be a subset of Q . But its impossibility means that $P \setminus Q = \emptyset$ and $P \subseteq Q$.

This example illustrates the general principle that the logical implication $P \Rightarrow Q$ between the statements P and Q corresponds to the subset relation $P \subseteq Q$ between the sets of situations they represent.

Figure 3.3: The sets P and Q .

We have seen this principle in action before, when the statements are framed as statements about set membership, on p. 7 in § 1.6. For any two sets A and B ,

$$A \subseteq B \text{ if and only if for all } x \text{ we have } x \in A \Rightarrow x \in B.$$

Our domino example illustrates that the link between the subset relation and logical implication is more general, and applies even where the logical implication is not stated in terms of set membership.

An important special case of implication is when the starting condition (on the left) is false. If we have an implication $X \Rightarrow Y$ when X is false, then the implication $X \Rightarrow Y$ is true regardless of what Y is. This corresponds to the fact that, if X is the empty set and Y is *any* set, then $X \subseteq Y$, since $\emptyset \subseteq Y$; the empty set is a subset of every set.

Keep in mind that the truth of an implication does not mean that either of its two parts is true. In our domino example, we know that $P \Rightarrow Q$, but this does not mean that P falls or that Q falls. It just gives a logical relationship between these two events, namely that if P falls then Q falls.

Note also that this is a purely logical relationship. In the domino scenario, there is also the ingredient of time. This plays a role in the physical mechanism by which the falling of P (if it happens) causes the falling of Q , and it follows from that mechanism that the fall of Q happens later, in time, than the fall of P . But this is a detail of the actual physical setting. Logical implication itself is not an assertion about time, but merely an assertion about the truth or falsehood of the two parts, in this case P and Q .

Indeed, it is entirely possible that logical implication can “go backwards” in time. Suppose X is the statement that you can see stars in the sky and Y is the statement that the sun has set. The sun setting does not itself mean that you can see stars, since it might be too cloudy. But if you can see stars, then you know the sun has set. So $X \Rightarrow Y$ holds, even though X happened after Y , and there is no suggestion that X *causes* Y .

Always keep in mind that implication is *not symmetric*. In the domino example, we have $P \Rightarrow Q$, but we do not have $Q \Rightarrow P$, because Q falling does not have to be because P falls (as we have supposed throughout that Q could be made to fall, by some external force, even if P remains standing). In the sunset example just given, we have $X \Rightarrow Y$, but we do not have $Y \Rightarrow X$, because the sun setting does not imply that you can see stars (as it might be cloudy).

The **converse** of an implication is the reverse implication, i.e., the implication you get by swapping the order of the two parts *or* by reversing the direction of the implication arrow symbol.⁵ So the converse of $P \Rightarrow Q$ is $Q \Rightarrow P$, which can also be written $P \Leftarrow Q$. When an implication holds, we cannot assume that the converse also holds. In the examples of the previous paragraph, we showed that, for the two example implications we have been discussing, the converse does not hold.

Sometimes, an implication and its converse *both* hold. For example, suppose you are holding a ball above the ground. Suppose R means that you release the ball and S means that the ball falls to the ground. Then $R \Rightarrow S$, and its converse $S \Rightarrow R$ holds too. In these situations, we can put the two implications together to make a two-way implication, written $R \Leftrightarrow S$, which means that R and S are logically equivalent, and we often say that R holds if and only if S holds. We have seen statements of this type before, in some of our Theorems.

3.3 PROOFS OF EXISTENTIAL AND UNIVERSAL STATEMENTS

We now give more examples of theorems and proofs, highlighting the relationship between the kind of statement you are trying to prove and the way you need to prove it.

Theorem 14. English has a palindrome.

Proof. ‘rotator’ is an English word and also a palindrome. □

An **existential** statement is a statement that asserts that something with a specified property exists. It may or may not be true.

The above theorem is an existential statement. This one happens to be true, and we have given a proof of it, so it is a theorem.

Proving an existential statement, such as ...

There exists a palindrome in English

⁵ But don’t do *both* of those, or you’ll get the implication you started with, just written differently! $P \Rightarrow Q$ and $Q \Leftarrow P$ are just different ways of writing the same thing.

...just requires one suitable example.

Most proofs are not this short ...

Theorem 15. Every English word has a vowel or a ‘y’.

Proof. This can be shown by listing all English words:

‘aardvark’ has a vowel.

‘aardwolf’ has a vowel.

‘aasvogel’ has a vowel.

...

...

‘syzygy’ has a ‘y’.

...

‘zygote’ has a vowel. □

We have only shown a few lines of this proof. The number of lines of the full proof equals the number of English words, which is several tens of thousands.

A **universal** statement is a statement that asserts that everything within some set has a specified property.

To prove a universal statement, such as ...

For every English word, it has a vowel or a ‘y’

... you need to cover every possible case.

One way is to go through all possibilities, in turn, and check each one. But the number of things to check may be huge, or infinite. So usually we want to reason in a way that can apply to many different possibilities at once.

3.4 FINDING PROOFS

There is no systematic method for finding proofs for theorems. There are deep theoretical reasons for this, based on work in the 1930s (Gödel, 1931; Church, 1936; Turing, 1936).

Discovering proofs is an art as well as a science. It requires

- skill at logical thinking and reasoning
- understanding the objects you’re working with
- practice and experience
- play and exploration
- creativity and imagination
- perseverance.

Although we can't give a recipe for discovering proofs, we will give some general advice on dealing with some common situations.

To prove subset relations, $A \subseteq B$ (where A and B are sets):

1. Take a general member of A , and give it a name. e.g., "Let $x \in A$ "
2. Use the definition of A to say something about x .
3. Follow through the logical consequences of that,
4. ... aiming to prove that x also satisfies the definition of B .

To prove **set** equality, $A = B$ (where A and B are **sets**):

1. Prove $A \subseteq B$
2. Prove $A \supseteq B$

To prove **numerical** equality, $A = B$ (where A and B represent **numbers**):

If symbolic manipulation using algebraic rules can transform expression A to expression B , then that's good;
but if not:

1. Prove $A \leq B$
2. Prove $A \geq B$

3.5 TYPES OF PROOFS

We now consider five types of proof, namely

- Proof by symbolic manipulation
- Proof by construction
- Proof by cases
- Proof by contradiction
- Proof by induction.

This list is not exhaustive.

Proofs can be quite individual in character and hard to classify, although many will follow one of the above patterns.

Many proofs are a mix of these types.

3.6 PROOF BY SYMBOLIC MANIPULATION

Some proofs proceed by a sequence of equations, where each equation uses some basic law (an axiom or some fundamental theorem) about the objects in question.

Many proofs you did in secondary school mathematics would have been of this type. For example, consider the difference of two squares:

$$x^2 - y^2 = (x + y)(x - y). \quad (3.1)$$

This is typically proved along the following lines.

$$\begin{aligned} (x - y)(x + y) &= x^2 + xy - yx - y^2 && \text{(expanding, using the distributive law for numbers)} \\ &= x^2 + xy - xy - y^2 && \text{(since multiplication is commutative)} \\ &= x^2 - y^2 && \text{(by additive cancellation).} \end{aligned}$$

Incidentally, this proof also illustrates that, if you want to prove an equation, you can start with *either side* of the equation and work towards the other side. You don't have to start with the left side, just because you read it first! In the above proof, we started with the right side. Any proof of this type can be done in either direction, but sometimes one direction seems more intuitive than the other.

Similarly, the basic laws of sets can be used to prove equality between some expressions involving sets. We have already seen some proofs of this type, in [Theorem 1](#), [Corollary 2](#), and [Theorem 5](#),

We can also use symbolic manipulation in parts of other proofs, as we did for example in the proof of [Theorem 10](#), where we used basic properties of function composition and inverse functions to do chains of equalities that establish the desired claims about inverses of compositions of functions.

3.7 PROOF BY CONSTRUCTION

...also known as **Proof by example**.

A **proof by construction** describes a specific object precisely and shows that it exists and that it satisfies the required conditions.

This can be used for some theorems that are existential statements, just asserting the existence of a certain object that satisfies some specified properties.

We saw a proof of this type in [Theorem 14](#).

Mistakes to avoid:

- attempting a proof by construction for a *universal* statement.
 - If a theorem asserts that *every* object has some property, then it's not enough to just construct *one* object with the property.

- constructing an example that has the claimed property, thinking that a convincing example is enough to prove that the property holds for other objects too.
 - An example may be useful in illustrating a proof or explaining the key ideas of a proof. But it is not, of itself, a proof.

3.8 PROOF BY CASES

...also known as **Proof by exhaustion** or (if lots of cases) “brute force”.

To do a proof by cases,

- identify a finite number of different cases which cover all possibilities,
- prove the theorem for each of these cases.
 - These separate proofs of the cases may be thought of as “subproofs” of the proof of the theorem.

We saw an example in [Theorem 15](#). That was not typical of proofs by cases, since the number of cases was so large (one case for each English dictionary word) and the number of possibilities to be covered was finite. More typically, a theorem asserts that every object from some infinite set has some property, and we divide the infinite set up into a small finite number of cases, and do a separate proof for each of the cases. In such situations, some of these cases must cover an infinite number of objects, and our reasoning in each case must be general enough to apply to all the objects covered by that case.

It’s ok if cases overlap (although it might indicate that the proof includes some unnecessary duplication of effort and is inefficient). But they must be *exhaustive* in the sense that every object considered by the theorem must belong to (at least) one of the cases.

3.9 PROOF BY CONTRADICTION

...also known as “reductio ad absurdum”.

A **proof by contradiction** works as follows.

- Start by assuming the negation of the statement you want to prove.
- Reason from this until you deduce a contradiction.
- This contradiction shows that the initial assumption was wrong.
- Therefore, the original statement must be true.

Our first proof by contradiction is somewhat whimsical, but has the structure of many proofs of this type and illustrates some important points about such proofs.

Theorem 16. Every natural number is interesting.⁶

Proof. Assume that not every natural number is interesting. So, there exists at least one uninteresting number. Therefore there exists a *smallest* uninteresting number. But that number must be interesting, by virtue of having this special property of being the smallest of its type. This is a contradiction, as this number is uninteresting. Therefore our original assumption was wrong. Therefore every natural number is interesting. $\square$

Comments:

That “theorem” and “proof” is really just an informal argument, as the meaning of “interesting” is imprecise and subjective.

But it illustrates the structure of proof by contradiction.

It also illustrates the point that, if you know an ordered set of objects is nonempty, then you can choose an element of *smallest* size in the set.

Often, the smallest object in a set may have special properties that can help you go further in the proof.

Can you always choose an object of *largest* size in a nonempty set?

Is every integer interesting?

Would the above proof still work, if applied to the set of all integers?

We now give a proof by contradiction of a fundamental theorem about prime numbers.

Theorem 17 (Euclid). There are infinitely many prime numbers.

Proof. **Suppose, by way of contradiction,** that there are only finitely many primes.

Let n be the number of primes.

Let $p_1, p_2, \dots, p_n$ be all the primes.

Define: $q := p_1 \cdot p_2 \cdot \dots \cdot p_n + 1$.

This is bigger than every prime p_i . Therefore q must be composite.

Therefore q is a multiple of some prime.

But, for each prime p_i , if you divide q by p_i , you get a remainder of 1.

So q cannot be a multiple of p_i .

So q cannot be a multiple of any prime. *This is a contradiction.*

So our initial assumption was wrong.

So there are infinitely many primes. $\square$

3.10 PROOF BY MATHEMATICAL INDUCTION

Suppose you want to prove that a statement $S(n)$ holds for every natural number n .

⁶ See, e.g., Ch. 14 (Fallacies), in: Martin Gardner, *The Scientific American Book of Mathematical Puzzles and Diversions*, Simon & Schuster, New York, 1959.

One powerful technique for proving theorems of this type is the **Mathematical Induction**. It is widely used across computer science and is particularly useful in proving theorems about the behaviour of algorithms and the discrete structures they work with including those considered in this unit. You will also use it extensively in later Computer Science units: FIT2004, FIT2014 and MTH3170/3175.

The **Principle of Mathematical Induction** is:

IF	$S(1)$ is true	(inductive basis)
AND	for all k : if $\underbrace{S(k) \text{ is true}}_{\text{inductive hypothesis}}$ then $S(k+1)$ is true	(inductive step)
THEN	for all n : $S(n)$ is true	

The intuition behind Induction is that:

- by the inductive basis, we know $S(1)$ is true;
- since $S(1)$ is true and the inductive step tells us that $S(1) \Rightarrow S(2)$, we deduce that $S(2)$ is true;
- since $S(2)$ is true and the inductive step tells us that $S(2) \Rightarrow S(3)$, we deduce that $S(3)$ is true;
- $\vdots$
- and so on, forever.

Mathematical Induction is a *single* logical principle that allows us to apply modus ponens repeatedly, forever, all the way along the number line, without “waving our hands” and saying “and so on”.

Our first use of Mathematical Induction is to prove an extension of De Morgan’s Law for Sets, [Theorem 1](#), to arbitrary numbers of sets. It is illustrated for three sets in [Figure 3.4](#).

Theorem 18. For all $n \in \mathbb{N}$,

$$\overline{A_1 \cup A_2 \cup \dots \cup A_n} = \overline{A_1} \cap \overline{A_2} \cap \dots \cap \overline{A_n}. \quad (3.2)$$

Proof. Let $S(n)$ be the statement that (3.2) holds for n . We must prove that, for all $n \in \mathbb{N}$, the statement $S(n)$ is true.

We prove it by induction on the number n of sets.

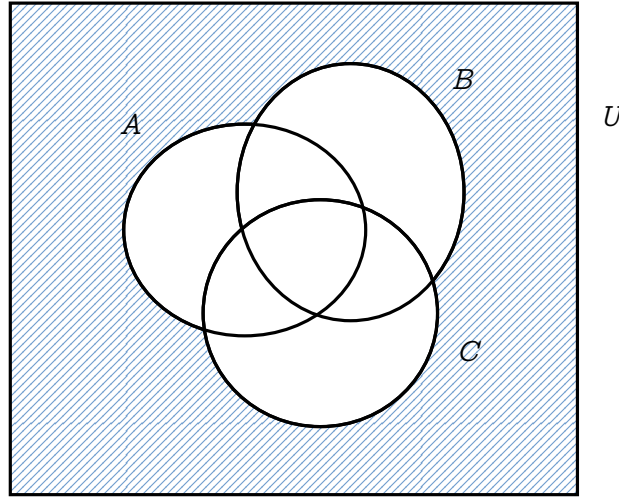


Figure 3.4: $\overline{A_1 \cup A_2 \cup A_3} = \overline{A_1} \cap \overline{A_2} \cap \overline{A_3}$, shaded.

Inductive basis:

$S(1)$ is trivially true. In that case, each side of (3.2) is just $\overline{A_1}$, so the equation holds.

Inductive step:

Let $k \geq 1$.

Suppose $S(k)$ is true:

$$\overline{A_1 \cup A_2 \cup \dots \cup A_k} = \overline{A_1} \cap \overline{A_2} \cap \dots \cap \overline{A_k}.$$

This our Inductive Hypothesis. We will use it later.

We have:

$$\begin{aligned} & \overline{A_1 \cup A_2 \cup \dots \cup A_{k+1}} \\ &= \overline{(A_1 \cup A_2 \cup \dots \cup A_k) \cup A_{k+1}} \quad (\text{just grouping ...}) \end{aligned} \tag{3.3}$$

$$= \overline{A_1 \cup A_2 \cup \dots \cup A_k} \cap \overline{A_{k+1}} \quad (\text{by De Morgan's Law for two sets, Theorem 1}) \tag{3.4}$$

$$= \overline{A_1} \cap \overline{A_2} \cap \dots \cap \overline{A_k} \cap \overline{A_{k+1}} \quad (\text{by Inductive Hypothesis})$$

Therefore $S(k+1)$ is true.

Conclusion:

So, by the Principle of Mathematical Induction, it's true for any number of sets. $\square$

Because induction helps prove theorems about statements that hold for all positive integers, it is a natural tool for proving statements about infinite sequences. In the next section (§ 3.11), we will use it to prove some statements about infinite sequences of

numbers. Because of this, it is a useful tool for proving statements about the behaviour of loops in programs; we will see this in the next section too.

A key thought process in the inductive step is to construct, from the “ $(k+1)$ -object”, a smaller object to which you can apply the inductive hypothesis. In the previous proof, our “ $(k+1)$ -object” is

$$\overline{A_1 \cup A_2 \cup \dots \cup A_{k+1}}.$$

Our aim is to show that this satisfies (3.2) (with $n = k+1$). In order to do this, we try to construct, from it, a “ k -object”, in this case

$$\overline{A_1 \cup A_2 \cup \dots \cup A_k}.$$

We first group the first k sets, in (3.3), as a step in this direction. Then applying De Morgan’s Law for two sets, in (3.4), gives us what we are aiming for: $\overline{A_1 \cup A_2 \cup \dots \cup A_k}$, our “ k -object”, shown in green in (3.4). Once we have the “ k -object”, we can apply the Inductive Hypothesis to it.

This technique, of reducing an object to a simpler object (or objects) of the same type, is known as **recursion**. It is one of the most fundamental problem solving strategies in computer science; you will encounter it and use it again and again and again. It is also provided for in most programming languages, when functions or methods can call themselves. By using this skill in proofs by induction, you are practising a core skill of your discipline. Moreover, proof by induction is the go-to proof technique for proving claims about the behaviour of recursive functions in programs.

3.11 INDUCTION: MORE EXAMPLES

For another use of induction, consider the following equation, which gives a formula for obtaining the sum of the first n positive integers, so that you can compute this sum without having to add up all those numbers.

$$1 + 2 + 3 + \dots + (n-1) + n = \frac{n(n+1)}{2}. \quad (3.5)$$

This is very useful in computer science and in fact throughout science, engineering and in many other fields. We illustrate this with an application.

Suppose you are trying to construct the Monash timetabling network described in § 2.13. You have access to a function which tells you, for any pair of units, how many students are doing both those units. You need to search all pairs of units, check how many students are doing both units in a pair, and if that number is nonzero, add a link between those units, to record the fact that their classes must be at different times.

A natural approach is to use two nested loops. The outer loop iterates over all units in some order (e.g., lexicographic order). For each unit considered in the outer loop, the inner loop iterates over all units that come later in the order. This saves duplication: if

the inner loop also iterated over *all* units, then each pair of units would be considered twice, once for each of the two possible ordered pairs based on those units.

If there are N units altogether, then these nested loops have the structure

```
for each  $i = 1, 2, \dots, N$ :
    for each  $j = i + 1, i + 2, \dots, N$ :
        if at least one student is doing both the  $i$ -th unit and the  $j$ -th unit,
            then make a link between these two units.
```

There are N iterations of the outer loop, but these have varying numbers of inner loop iterations. If we want to work out how long this computation takes, we need to know how many inner loop iterations are done altogether.

- For the first iteration of the outer loop, $i = 1$, and the inner loop starts at $i + 1 = 1 + 1 = 2$ and considers $j = 2, 3, \dots, N$, so it does $N - 1$ iterations.
- For the second iteration of the outer loop, $i = 2$, and the inner loop considers $j = 3, 4, \dots, N$, so it does $N - 2$ iterations.
- For the third iteration of the outer loop, the inner loop does $N - 3$ iterations.
- ...and so on ...
- For the $(N - 1)$ -th iteration of the outer loop, we have $i = N - 1$, so there is only one iteration of the inner loop, with $j = i + 1 = (N - 1) + 1 = N$.
- The N -th iteration of the outer loop actually has no inner loop iterations at all, because the range $i + 1 \leq j \leq N$ is empty, because $i + 1 > N$.

So the total number of inner loop iterations is

$$(N - 1) + (N - 2) + \dots + 2 + 1.$$

This expression is of the same type as the left-hand side of (3.5). We've written it in a different order, but that doesn't matter at all. The only other difference is that we're adding up the first $N - 1$ positive integers instead of the first n , but we can still use the right-hand side of (3.5) to give us the answer, with an appropriate substitution.

Nested loop structures along these lines are very common in programming, and equations like (3.5) enable us to give good estimates of how long they will take even before we run the program.

So, how do we *prove* (3.5)? There are many beautiful proofs of this fact, and we will see at least two proofs in this unit. The first we give now, using induction.

Theorem 19. For all n :

$$1 + \dots + n = \frac{n(n + 1)}{2}.$$

Proof. We prove it by induction on n .

Inductive basis:

When $n = 1$, LHS = 1 and RHS = $1(1+1)/2 = 1$.

Inductive step:

Let $k \geq 1$.

Suppose it's true for $n = k$:

$$1 + \cdots + k = k(k+1)/2.$$

We will deduce that it's true for $n = k + 1$.

$$\begin{aligned} 1 + \cdots + (k+1) &= (1 + \cdots + k) + (k+1) && \text{(preparing to use the Inductive Hypothesis)} \\ &= k(k+1)/2 + (k+1) && \text{(by the Inductive Hypothesis)} \\ &= (k+1)k/2 + (k+1) && \text{(algebra ...)} \\ &= (k+1)(k/2 + 1) \\ &= (k+1)(k+2)/2 \\ &= (k+1)((k+1)+1)/2 \end{aligned}$$

This is just the equation in the Theorem, for $n = k + 1$ instead of k .

So the inductive step is now complete.

Conclusion:

Therefore, by the Principle of Mathematical Induction, the equation holds for all n . □

Notice, in the inductive step, how we construct, from the “ $(k+1)$ -object” $1+2+\cdots+(k+1)$, a “ k -object” $1+2+\cdots+k$. In this case, it's just a matter of grouping, since the k -object just sits within the $(k+1)$ -object. As soon as we construct the k -object, we can apply the Inductive Hypothesis to it.

Often, as in our proof of [Theorem 18](#), we need to do some more work to construct the k -object from the larger $(k+1)$ -object.

There is a slightly different way of writing inductive proofs that you are likely to come across. We could make the inductive step go from $n = k - 1$ to $n = k$, instead of from $n = k$ to $n = k + 1$. Let us re-do the previous proof in this way.

Slightly different proof: We prove [Theorem 19](#) by induction on n .

Inductive basis:

When $n = 1$, LHS = 1 and RHS = $1(1+1)/2 = 1$.

Inductive step:

Let $k \geq 2$. [Note the change here!]

Suppose it's true for $n = k - 1$, where $k \geq 2$:

$$1 + \cdots + (k - 1) = (k - 1)k/2.$$

We will deduce that it's true for $n = k$.

$$\begin{aligned} 1 + \cdots + k &= (1 + \cdots + (k - 1)) + k && \text{(preparing to use the inductive hypothesis)} \\ &= (k - 1)k/2 + k && \text{(by the Inductive Hypothesis)} \\ &= k(k - 1)/2 + k && \text{(algebra ...)} \\ &= k((k - 1)/2 + 1) \\ &= k(k + 1)/2 \end{aligned}$$

This is just the equation in the Theorem, for $n = k$ instead of $k - 1$.

So the inductive step is now complete.

Conclusion:

Therefore, by the Principle of Mathematical Induction, the equation holds for all n .

□

We will mostly frame our inductive steps as going from $n = k$ (where we *assume* the statement is true) to $n = k + 1$ (where we *deduce* it's true, using the Inductive Hypothesis), as we did in our first proof of Theorem 19. But there is nothing wrong with doing it from $n = k - 1$ to $n = k$, as in the second proof above. If you read other books and resources, you will find that some authors do it one way, while others do it the other way. If you do it the second way (from $n = k - 1$ to $n = k$), then you need to take care that the inductive step is grounded in the inductive basis. In the first proof, the inductive step starts with "Let $k \geq 1$ "; in the second proof, the inductive step starts with "Let $k \geq 2$ " which ensures that $k - 1 \geq 1$ so that the inductive step is, indeed, grounded in the inductive basis.

Even though we have now proved Theorem 19, you are entitled to wonder where the expression, $n(n + 1)/2$, came from in the first place. There are many ways to derive this expression from scratch. We will discuss this in more detail later, in § 6.12.

When you first try to work out what the expression should be, you might *explore* the first few cases and try to discern a pattern. This may lead you to *conjecture* that the expression is $n(n + 1)/2$.

A pattern that works for small cases is not, in itself, a proof. But it might still help you come up with a proof, since now you have something to aim for: a conjecture that you can try to prove. And, in this case, mathematical induction can be used to *prove* the conjecture.

We will discuss this explore-conjecture-prove methodology, for discovering and proving formulae for mathematical expressions, in § 6.6.

Mathematical induction is a proof technique, and as such can only be used once you have a statement — in this case, an equation — to prove. It won't help you *discover* what the right equation should be; that's where exploration and conjecture come in. The upside is that you get a rigorous proof for a conjecture which you might otherwise have remained unsure about or been unable to justify fully.

Having used induction to prove that the sum of the first n positive integers is indeed $n(n+1)/2$, it is natural to ask, what about sums of higher powers of positive integers? To start with, what about the sum of the squares of the first n positive integers, i.e., $1^2 + 2^2 + \dots + n^2$? This gives the number of iterations in the following triply-nested loops:

```

for each  $i = 1, 2, \dots, n$ :
  for each  $j = i, i+1, \dots, n$ :
    for each  $k = i, i+1, \dots, n$ :
      [some action]
```

If you'd like a challenge, *explore* $1^2 + 2^2 + \dots + n^2$ by computing it for several small values of n and trying to understand its behaviour. How does it grow as n increases? Then try to *conjecture* a possible formula for it, and then try to *prove* it by induction.

3.12^ω INDUCTION: EXTENDED EXAMPLE

We now give another detailed example of proof by induction. This is quite involved but it does give extended practice at reasoning about functions and relations as well as induction.

Consider the following problem. Suppose there are m job vacancies to be filled from a pool of n applicants. For each vacant job, the employer has constructed a shortlist of applicants. Is it possible for each position to be filled by a different applicant, so that no position remains unfilled?

If $m > n$, then this is impossible: there are simply too many positions for the available applicants, and some number of positions must remain unfilled. If $m \leq n$, then it may or may not be possible, depending on the shortlists.⁷

We model this problem as follows. Let A be the set of positions, with $|A| = m$, and let B be the set of applicants, with $|B| = n$. Let $S \subseteq A \times B$ be the *shortlist relation*, a binary relation consisting of all pairs (a, b) such that the shortlist for position a includes

⁷ If $m < n$, then some applicants will not get a position, but that is permitted in this problem. One could also imagine a scenario where it is the applicants who have shortlists and the employers who have no choice.

applicant b . The full shortlist for position a is the set $S(a)$, which is a subset of B ; here, we use the notation introduced on p. 67 in § 2.13. If we have a set of positions $X \subseteq A$, then we can, if we wish, form a combined shortlist for the set of positions by taking the union of all the individual shortlists:

$$S(X) = \bigcup_{a \in X} S(a).$$

We want each position to be filled by exactly one applicant, so the set of pairs (a, b) that specify allocation of applicants to positions must be a *function*. Furthermore, we want each position to be filled by an applicant who does not also fill any other position; no applicant fills two positions. So we want, for each $a \in A$, a unique $b \in B$ such that $(a, b) \in S$. These pairs (a, b) that specify a valid allocation of applicants to positions, filling each position with a unique applicant, must therefore be an *injection*. So we are asking: does the shortlist relation S contain an injection?

Some situations can be dealt with easily.

If $m > n$, then B is too small to enable an injection to it from A , so S contains no injection. More generally, suppose there is a set $X \subseteq A$ of jobs for which the union of their shortlists $S(X)$ is smaller than X , i.e., $|X| > |S(X)|$. Then S cannot contain an injection, since an injection would ensure that the shortlists for X together include at least $|X|$ applicants. We have now proved:

Theorem 20. If a shortlist relation from A to B contains an injection with domain A then, for all $X \subseteq A$, we have $|X| \leq |S(X)|$.

(The condition here means that, for every set X of jobs, the union of their shortlists contains at least as many applicants as there are jobs in X .) $\square$

More surprisingly, the converse is true as well. We will prove this by induction.

Theorem 21. Let $S \subseteq A \times B$ be a binary relation. If, for all $X \subseteq A$ we have $|X| \leq |S(X)|$, then S contains an injection with domain A .

Proof. We prove this by induction on $|A|$.

Inductive basis:

When $|A| = 1$, the set A contains a single element a . Using $X = \{a\}$, if the shortlist for this one job has at least one applicant, then there exists $b \in B$ such that $(a, b) \in S$. So S contains the simple function whose sole ordered pair is (a, b) . This function is an injection with domain A . So the claim holds for $|A| = 1$.

Inductive step:

Let $m \geq 1$.

Assume that the following holds for every binary relation $T \subseteq C \times D$ in which $|C| \leq m$:

If, for each $X \subseteq C$ we have $|X| \leq |T(X)|$, then T contains an injection with domain C .

This is our Inductive Hypothesis. Note that this is an *implication*. Like any implication, it has a *condition* (namely, “for each $X \subseteq C$ we have $|X| \leq |T(X)|$ ”) and a *consequence* (namely, “ T contains an injection with domain C ”). In assuming this Inductive Hypothesis, we are not assuming that the condition is true, and we are not assuming that the consequence is true. We are merely assuming that, *if* the condition is true, *then* the consequence is true.

Let $S \subseteq A \times B$ where $|A| = m + 1$. Suppose that the following condition holds:

$$\text{for each } X \subseteq A \text{ we have } |X| \leq |S(X)| \quad (3.6)$$

We need to prove that S contains an injection with domain A .

Case 1: for each nonempty $X \subset A$ we have $|X| \leq |S(X)| - 1$. (Note here that X is a proper subset of A .)

Let (a, b) be any pair in S . Then consider the restriction of S to $(A \setminus \{a\}) \times (B \setminus \{b\})$, i.e., the subrelation consisting of all pairs of S that do not meet a OR b . Call this smaller relation T , and put $C := A \setminus \{a\}$ and $D := B \setminus \{b\}$. Let $X \subseteq C$. Then $|S(X)| \geq |X| + 1$, by the assumption on which this case is based. But, in T , the applicant b is excluded. Nevertheless, we can still say that, in T , the union $T(X)$ of X ’s shortlists has at least $|X|$ applicants (obtained from the $|X| + 1$ applicants by excluding b if necessary): $|T(X)| \geq |X|$. Now, T has one fewer position than S , since $|C| = |A| - 1$. So the Inductive Hypothesis applies to it. Therefore T contains an injection with domain C . Call that injection g . This injection, together with the pair (a, b) , gives a new binary relation $f := g \cup \{(a, b)\}$ which is a subset of S :

$$g \cup \{(a, b)\} \subseteq S,$$

since $g \subseteq T$ and $T \subseteq S$ and $(a, b) \in S$. Furthermore, this new relation f is actually an injection with domain A , since

- g is an injection, *and*
- a appears in no other pair of g (so there is no “one-to-many” violation of the definition of a function), *and*
- b appears in no other pair of g (so there is no “many-to-one” violation of the injection property), *and*
- the domain of f is A (since it is just the domain of g , namely $A \setminus \{a\}$, augmented by a which is mapped to b by f).

So S does indeed contain an injection with domain A in this case.

Case 2: for some nonempty $X \subset A$ we have $|X| = |S(X)|$.

Let $X \subset A$ be such a set of positions.

Consider the restriction of S to pairs (a, b) such that $a \in X$. Call this binary relation T . It is a subset of $X \times S(X)$. Since X is a proper subset of A , we have $|X| < |A|$, so

$|X| \leq m$. So we can apply the Inductive Hypothesis, with $C = X$ and $|C| \leq m$. Now, the Inductive Hypothesis is an implication, with condition and consequence as discussed above. We first establish that the condition holds in our current situation. Recall that every $Y \subseteq A$ satisfies $|Y| \leq |S(Y)|$, by (3.6). So, certainly every $Y \subseteq C$ satisfies $|Y| \leq |S(Y)|$ (since $C \subseteq A$). Also, T is just the restriction of S to $C \times B$, so $T(Y) = S(Y)$ (because $Y \subseteq C$). Therefore every $Y \subseteq C$ satisfies $|Y| \geq |T(Y)|$. Therefore T satisfies the condition of the Inductive Hypothesis. Therefore, by the Inductive Hypothesis, its consequence also holds, namely that T contains an injection with domain X , which we call g . It has domain $X = C$, and one suitable codomain is $T(C)$ which is the same as $S(C)$. So we can write $g: C \rightarrow S(C)$.

This injection g is a step towards constructing an injection in S . Its domain is X , so it fills the jobs in that set. But X is a proper subset of A , so there are other jobs that g does not fill.

So we need to find a way to fill jobs in $A \setminus X$. Again, we work towards applying the Inductive Hypothesis. Observe that $A \setminus X$ is a nonempty proper subset of A , since X is too.

Let $Z \subseteq A \setminus X$. We have

$$\begin{aligned}
 |X| + |Z| &= |Z \cup X| && \text{(since the union is disjoint)} \\
 &\leq |S(Z \cup X)| && \text{(by (3.6))} \\
 &= |S(X) \cup (S(Z) \setminus S(X))| && \text{(since } S(Z \cup X) = S(X) \cup (S(Z) \setminus S(X))\text{)} \\
 &= |S(X)| + |S(Z) \setminus S(X)| && \text{(since the union is disjoint)} \\
 &= |X| + |S(Z) \setminus S(X)| && \text{(since } |S(X)| = |X|, \text{ by the assumption on which this case is based).}
 \end{aligned}$$

Subtracting $|X|$ from the first and last expressions, we have

$$|Z| \leq |S(Z) \setminus S(X)|. \quad (3.7)$$

Let U be the binary relation obtained from X by restricting to those pairs (a, b) such that $a \notin X$ and $b \notin S(X)$. In other words, U is the restriction of S to $(A \setminus X) \times (B \setminus S(X))$. If $Z \subseteq A \setminus X$, then $U(Z)$ consists of those pairs (a, b) such that $a \in Z$, $b \in S(Z)$ and $b \notin S(X)$. So

$$U(Z) = S(Z) \setminus S(X).$$

This, together with (3.7), gives

$$|Z| \leq |U(Z)|.$$

We are now in a position to apply the Inductive Hypothesis. It can be used on U , because $|A \setminus X| \leq m$ (which follows from the fact that $A \setminus X$ is a proper subset of A). Its condition also holds, because, as we have just seen, $|Z| \leq |U(Z)|$ for all $Z \subseteq A \setminus X$. Therefore the consequence follows, namely that U contains an injection with domain $A \setminus X$. Call this injection h . Its domain is $A \setminus X$ and its codomain can be taken to be $B \setminus X$. So the domains of g and h are disjoint, and their codomains are disjoint too.

Since g and h are functions on disjoint domains and the union of their domains is A , the union $g \cup h$ is also a function and its domain is A . Also, since g and h are both injections and their codomains are disjoint, their union $g \cup h$ is also an injection.

We have constructed an injection $g \cup h$ with domain A . Since $g \subseteq T$ and $T \subseteq S$, and since also $h \subseteq U$ and $U \subseteq S$, we have $g \cup h \subseteq S$. So S contains an injection with domain A .

This completes Case 2. Since Cases 1 and 2 cover all possibilities, the Inductive Step is now complete.

Conclusion:

Therefore, by the Principle of Mathematical Induction, the theorem holds. $\square$

Putting [Theorem 20](#) and [Theorem 21](#) together we have

Corollary 22. Let $S \subseteq A \times B$ be a binary relation. The relation S contains an injection if and only if for all $X \subseteq A$ we have $|X| \leq |S(X)|$. $\square$

This gives us an important and useful characterisation of situations when there is an injection. It does not give us an algorithm, though it does contain ideas that are useful in the development of algorithms for this and related problems. What it does give us is succinct, easily-verified evidence for both positive and negative cases. We explain this now, for each case in turn.

- Positive case: if S does contain an injection, then the evidence of this is the injection itself. Once we have an injection f , we can easily check that it is an injection. First, check that every job $a \in A$ belongs to some pair $(a, b) \in f$, so that the domain of f is indeed A . Then check, for each $a \in A$, that it belongs to no more than one pair in f , so that f is indeed a function. Then check, for each $b \in B$, that it belongs to no more than one pair in f , so that f is indeed an injection.
- Negative case: if S does not contain an injection, then the evidence of this is some set $X \subseteq A$ such that $|X| > |S(X)|$. Once we have such a subset X , we can easily check that $|X|$ and $|S(X)|$ satisfy this inequality, as follows. For each $a \in X$ in turn, we find all pairs $(a, b) \in S$, and for each such pair, we add b to our set $S(X)$ if it is not already there. Once we have finished compiling $S(X)$, we check that its size is $< |X|$.

The details of these checking methods would depend on the specific data structure used to store the binary relation S . It would be a good programming exercise to write programs to do each of these two checking methods and to study how long they take to run, as a function of the sizes of A and B and the number of ordered pairs in S .

3.13 MATHEMATICAL INDUCTION AND STATISTICAL INDUCTION

The use of the term “induction” here is different to the use of “induction” in statistics, which is the process of drawing general conclusions from data. Statistical induction is

typically used in situations where there is some randomness in the data, and conclusions drawn can include some amount of error provided the conclusions drawn are significant enough that the error is unimportant. It is not a process of *logical deduction*; this, together with the presence of errors, means that it cannot be used as a step in a mathematical proof. By contrast, Mathematical Induction is a rigorous and very powerful tool for logical reasoning in mathematical proofs.

3.14 PROGRAMS AND PROOFS

At the start of this chapter, we discussed the role of proofs in computer science, in particular their importance in providing rigorous, rational support for general claims about programs and the structures they work with.

The different types of proofs we have introduced can each be used to prove statements about particular programming language constructs. If we want to prove something about an if-statement, then we would normally use proof by cases. If we want to prove something about a function that calls itself recursively, then we would normally use proof by induction. Proving statements about loops in programs is also often done by induction.

But there is a deeper reason for programmers to develop skills in writing proofs.

Programming is often thought of as a completely different activity to writing mathematical proofs. In fact, the two activities are surprisingly similar, and developing the skill of writing mathematical proofs will make you a better programmer.

Let's compare *programs* and *proofs*.

Each consists of a sequence of precise statements. Each of these statements obeys the rules of some language: for programs, this is the programming language; for proofs, this is the language of mathematics augmented by precise use of a natural language such as English.

Each uses variables, which are names that can refer to any object of some type. Variables can be combined into expressions using the operations that are available for objects of that type. One difference between programs and proofs is that, in programs, variables have memory set aside for them, but this does not happen in proofs.

Each statement must depend on previous statements in a precise way. In a program, when an operation is applied to a variable, it uses the most recently computed value of that variable; it does not “look ahead” and use some other value of the variable that will be computed later. In a proof, each statement is a logical consequence of *previous* statements (not of later statements), or it might be just an already-known fact or axiom.

In a program, we can use an if-statement to decide, according to some logical condition, which set of statements to execute next. In a proof, we can use a finite number of cases, which together cover all possible situations (§ 3.8). Each case pertains to situations that satisfy some logical condition.

Programs can call other programs, which may have been written by you or by other people. Proofs can use other theorems, which may have been written by you or by other

people. So building a proof, using theorems that have been proved previously, is like writing a function in your program that calls other functions.

In a program, a block of statements can be executed repeatedly, for as long as some logical condition is satisfied. Another way to do repetition is to use recursion, in which a function or method calls itself. The analogue in proofs is mathematical induction.

A program crashes if it encounters a situation that its statements cannot deal with. We say it has a bug, and this should be fixed. Ideally, the program never crashes. If a proof cannot deal with some situation it is supposed to cover, then we might also call this a bug, or a “hole” or a “gap”. This is more serious for proofs than for programs. A program with a bug might still be of some use, provided it crashes rarely and can correctly deal with the inputs that don’t cause it to crash (even though it *should* be fixed). But a “proof” with a hole is actually *not* a proof at all. It might be able to be repaired, to turn it into a proof, or it might not; in the latter case, the “theorem” being “proved” might actually be false, in which case it has no proof, and is therefore not a theorem after all.

Programming and proof-writing have differences as well as similarities, but the analogy is close. Writing mathematical proofs is like programming in a different “paradigm”. A *programming paradigm* is, loosely speaking, an approach to programming and a way of thinking about it. Important programming paradigms include procedural programming (historically the first), object-oriented programming, logic programming and functional programming. Learning to program in more than one *paradigm* (as well as in more than one language) improves your thinking about programming and makes you a better programmer in any language. Learning to write proofs — in the “programming paradigm” of mathematical reasoning — yields similar benefits to a programmer.

3.15 EXERCISES

1. Recall the Collatz function from p. 55 in § 2.9.

Prove by cases that, for all positive integers n ,

$$\text{Collatz}^{(2)}(n) \leq \frac{3}{2}n + 1.$$

- 2.

A set of strings L is called **hereditary** if it has the following property:

For every nonempty string x in L , there is a character in x which can be deleted from x to give another string in L .

Prove by contradiction that every nonempty hereditary set of strings contains the empty string.

3. Review the *statements* of all the theorems (including corollaries) in the previous chapters.

- (a) Which ones are implications?
- (b) Which ones are equivalences?
- (c) Which ones are existential statements?
- (d) Which ones are universal statements?

4. Review the *proofs* of all the theorems in the previous chapters.

- (a) Which ones use proof by construction?
- (b) Which ones prove an implication by proving its contrapositive?
- (c) Which ones use proof by cases?

5. Let E be a finite set and let $\mathcal{I}$ be a collection of subsets of E that satisfies the following three properties:

- (I1) $\emptyset \in \mathcal{I}$.
- (I2) Whenever $X \subseteq Y$ and $Y \in \mathcal{I}$, we also have $X \in \mathcal{I}$. This is called the **hereditary property**.
- (I3) Whenever $X, Y \in \mathcal{I}$ and $|X| < |Y|$, there exists $y \in Y \setminus X$ such that $X \cup \{y\} \in \mathcal{I}$. In other words, given two different-sized sets in $\mathcal{I}$, you can always enlarge the smaller set, using some new member from the larger set, to get another set in $\mathcal{I}$. This is called the **independence augmentation property**.

The sets in $\mathcal{I}$ are called **independent**.

For example, if $E = \{a, b, c\}$, then each of the following collections satisfies all three properties (I1)–(I3):

- $\mathcal{I} = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$
- $\mathcal{I} = \{\emptyset, \{a\}, \{b\}, \{c\}\}$
- $\mathcal{I} = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, c\}, \{b, c\}\}$
- $\mathcal{I} = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}\}$
- $\mathcal{I} = \{\emptyset\}$

By way of counterexample, using the same E , each of the following collections *fails* to satisfy all three properties (I1)–(I3):

- $\mathcal{I} = \{\{a\}, \{b\}, \{a, b\}\}$. This fails (I1) and (I2).
 - $\mathcal{I} = \{\emptyset, \{a\}, \{a, b\}\}$. This fails (I2).
 - $\mathcal{I} = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}\}$. This satisfies (I1) and (I2) but fails (I3).
 - $\mathcal{I} = \emptyset$. This fails (I1)
- (a) For each counterexample above that fails (I2) and/or (I3), give specific sets X and Y for which the property fails.
- (b) Prove that, for a social network (see § 1.7, § 2.14, § 2.13), the set of cliques satisfies (I1) and (I2) but not, in general, (I3).
- (c) Prove by contradiction that all maximal independent sets have the same size.

Suppose that each member $e \in E$ has a nonnegative real **weight** $w(e)$. So $w : E \rightarrow \mathbb{R}_0^+$. We use this weight function to also give weights to *subsets* of E , by defining the weight of a set to be the sum of the weights of its elements: for all $X \subseteq E$, define

$$w(X) := \sum_{e \in X} w(e).$$

Suppose we seek an independent set of maximum weight.

Consider the following **greedy algorithm**, so called because it always makes the choice that gives biggest immediate gain:

Input: a finite set E , a function $w : E \rightarrow \mathbb{R}_0^+$, a collection $\mathcal{I}$ of subsets of E .

Initialisation:

$X := \emptyset$

$i := 1$.

If there exists $y \in E \setminus X$ such that $X \cup \{y\} \in \mathcal{I}$:

From all these y , choose the one with largest weight $w(y)$. (This is where greed comes in.)

Call it e_i .

Add e_i to X .

else [we cannot enlarge X further]

Stop and output X .

Greedy algorithms are important because they are simple, easy to program, and efficient. In general, they do not give best-possible solutions. But it is important to recognise situations when they do give best-possible solutions, since then we attain a rare state of algorithmic bliss: simplicity, speed, optimality.

(d) Construct a simple social network, and give a nonnegative real weight to each person, such that the greedy algorithm, with $\mathcal{I}$ being the set of all cliques, does not find the maximum weight clique.

Now suppose $\mathcal{I}$ satisfies (I1)–(I3). The greedy algorithm chooses elements of E in a specific order, which we represent as

$$e_1, e_2, \dots, e_r.$$

(e) Prove by contradiction that the weights of the chosen elements are decreasing, i.e.,

$$w(e_1) \geq w(e_2) \geq \dots \geq w(e_r).$$

By “decreasing” we allow the possibility that some weights stay the same as we go along the list; we don’t require that they be *strictly* decreasing (which is a stronger property and would entail replacing each $\geq$ by $>$).

(f) Prove that the greedy algorithm finds a maximum weight independent set.

So, whenever (I1)–(I3) hold, we can confidently use the greedy algorithm, knowing that it will give us the best possible solution.

Less obviously, it turns out that properties (I1)–(I3) give a precise characterisation of situations where greedy algorithms give optimum solutions! To be specific, if $\mathcal{I}$ is a collection of sets satisfying (I1) and (I2), and the greedy algorithm *always* gives the maximum weight member of $\mathcal{I}$ (regardless of the weights of the elements of E), then $\mathcal{I}$ must satisfy (I3) too. It is an interesting but challenging exercise to try to prove this.

So, it is useful to be able to recognise structures where properties (I1)–(I3) hold, because they are precisely the situations where you can’t do better than a greedy algorithm. These structures are known as **matroids** and their role in optimisation goes beyond their connection with greedy algorithms. They also have many applications in the study of networks, vectors, matrices, and geometry.

6.

Prove the following statement, by mathematical induction:

For all k ,

$$(*) \quad \text{the sum of the first } k \text{ odd numbers equals } k^2.$$

(a) First, give a simple expression for the k -th odd number.

(b) Inductive basis: now prove the statement $(*)$ for $k = 1$.

Inductive Step:

Let $k \geq 1$.

Assume the statement $(*)$ true for k . This is our **Inductive Hypothesis**.

(c) Express the sum of the first $k + 1$ odd numbers ...

$$1 + 3 + \cdots + ((k + 1)\text{-th odd number})$$

...in terms of the sum of the first k odd numbers, plus something else.

(d) Use the inductive hypothesis to replace the sum of the first k odd numbers by something else.

(e) Now simplify your expression. What do you notice?

(f) When drawing your final conclusion, don't forget to briefly state that you are using the Principle of Mathematical Induction!

7. Prove by induction on n that, for all $n \in \mathbb{N}_0$,

$$1 + 2 + 2^2 + 2^3 + \cdots + 2^n = 2^{n+1} - 1.$$

8. Prove by induction that, for all n :

$$1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

9. Prove by induction that, for all n :

$$1^3 + 2^3 + \cdots + n^3 = (1 + 2 + \cdots + n)^2.$$

10.

Consider the following algorithm:

```

for each  $i = 1, 2, \dots, n$ :
  for each  $j = 1, 2, \dots, i$ :
    for each  $k = 1, 2, \dots, j$ :
      [some action]
```

- (a) For each pair i, j , how many times is the action inside the innermost loop performed? Express this in terms of i or j or both of them.
- (b) For each i , how many times is the action inside the innermost loop performed? Write this as both a sum of some number of terms, and (using a theorem in this chapter) a simple algebraic expression. Both these expressions should be in terms of i .
- (c) Now write a sum of some number of terms, expressed in terms of n , for the total number of times the action is performed.
- (d) Now work out this expression for $n = 1, 2, 3, 4, 5$, and for as many further values of n as you like.
- (e) *Explore* how these numbers behave.
- (f) *Conjecture* a simple algebraic expression, in terms of n , for the total number of times the action is performed.
- (g) *Prove*, by induction on n , that your expression is correct.

11.

The n -th *harmonic number* H_n is defined by

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}.$$

These numbers have many applications in computer science. We will meet at least one later in this unit.

In this exercise, we prove by induction that $H_n \geq \log_e(n+1)$. (It follows from this that the harmonic numbers increase without bound, even though the differences between them get vanishingly small so that H_n grows more and more slowly as n increases.)

- (i) Inductive basis: prove that $H_1 \geq \log_e(1+1)$.
- (ii) Let $n \geq 1$. Assume that $H_n \geq \log_e(n+1)$; this is our *inductive hypothesis*. Now, consider H_{n+1} . Write it recursively, using H_n . Then use the *inductive hypothesis* to obtain $H_{n+1} \geq \dots$ (where you fill in the gap). Then use the fact that $\log_e(1+x) \leq x$, and an elementary property of logarithms, to show that $H_{n+1} \geq \log_e(n+2)$.
- (iii) In (i) you showed that $H_1 \geq \log_e(1+1)$, and in (ii) you showed that if $H_n \geq \log_e(n+1)$ then $H_{n+1} \geq \log_e((n+1)+1)$. What can you now conclude, and why?

Advanced afterthoughts:

- The above inequality implies that $H_n \geq \log_e n$, since $\log_e(n+1) \geq \log_e n$. It is instructive to try to prove directly, by induction, that $H_n \geq \log_e n$. You will probably run into a snag. This illustrates that for induction to succeed, you sometimes need to prove something that is *stronger* than what you set out to prove.
- Would your proof work for logarithms to other bases, apart from e ? Where in the proof do you use the base e ?
- It is known that $H_n \leq (\log_e n) + 1$. Can you prove this?

12. (Challenge)

Let E be a finite set, and let $\mathcal{S}$ be a set of subsets of E . We say $\mathcal{S}$ is **linear under Δ** if, for every two sets $X, Y \in \mathcal{S}$, we have $X \Delta Y \in \mathcal{S}$. So, applying symmetric difference to sets in $\mathcal{S}$ always gives sets in $\mathcal{S}$; the operation never takes you out of $\mathcal{S}$.

The **span** of $\mathcal{S}$, written $\text{span}(\mathcal{S})$, is the set of all symmetric differences of any number of members of $\mathcal{S}$. In other words,

$$\text{span}(\mathcal{S}) := \{X_1 \Delta X_2 \Delta \cdots \Delta X_k : X_1, X_2, \dots, X_k \in \mathcal{S}, k \in \mathbb{N}\}.$$

By convention, the symmetric difference of zero sets is the empty set. Therefore, the span of any set $\mathcal{S}$ always contains the empty set.

$\mathcal{S}$ is **dependent under Δ** if, for some $X_1, X_2, \dots, X_k \in \mathcal{S}$ which are all distinct; i.e., $X_i \neq X_j$ whenever $1 \leq i < j \leq k$, we have

$$X_1 \Delta X_2 \Delta \cdots \Delta X_k = \emptyset.$$

If $\mathcal{S}$ is not dependent under Δ then we say it is **independent under Δ** .

$\mathcal{S}$ is a **circuit under Δ** if it is dependent under Δ and also *minimal* with respect to that property.

$\mathcal{S}$ is a **base under Δ** if it is independent under Δ and also *maximal* with respect to that property.

(Recall the precise usage of “minimal” and “maximal” from § 1.7.)

(a) Let $\mathcal{S}$ be linear under Δ . Prove, by induction on k , that for all k and all $X_1, X_2, \dots, X_k \in \mathcal{S}$,

$$X_1 \Delta X_2 \Delta \cdots \Delta X_k \in \mathcal{S}.$$

(b) Prove that, for every set $\mathcal{S}$ of subsets of E , the set $\mathcal{S}$ is linear under Δ if and only if it equals its own span, i.e.,

$$\mathcal{S} = \text{span}(\mathcal{S}).$$

(c) Prove by induction on k , that for all k and all $X_1, X_2, \dots, X_k \in \mathcal{S}$, either

$$X_1 \Delta X_2 \Delta \cdots \Delta X_k = \emptyset,$$

or there exist $Y_1, Y_2, \dots, Y_n \in S$ with $n \leq k$, such that $Y_i \neq Y_j$ for all $i \neq j$ and

$$X_1 \Delta X_2 \Delta \dots \Delta X_k = Y_1 \Delta Y_2 \Delta \dots \Delta Y_n.$$

(d) Prove that S is dependent under Δ if and only if there exists $X \in S$ such that

$$X \in \text{span}(S \setminus \{X\}).$$

(e) Prove that if S is a minimal dependent set under Δ , then for every $X \in S$ we have

$$X \in \text{span}(S \setminus \{X\}).$$

(f) Prove that every minimal spanning set under Δ is independent under Δ .

(g) Now prove that every minimal spanning set under Δ is a *maximal* independent set under Δ .

(h) Prove that every maximal independent set under Δ is also a spanning set under Δ .

(i) Now prove that every maximal independent set under Δ is also a *minimal* spanning set under Δ .

(j) Prove that S is a minimal spanning set under Δ if and only if it is a maximal independent set under Δ .

(k) Prove that, if S is independent under Δ then

$$|\text{span}(S)| = 2^{|S|}.$$

(l) Prove the *independence augmentation property*: if S and $\mathcal{T}$ are independent under Δ and $|\mathcal{T}| = |S| + 1$, then there exists $X \in \mathcal{T} \setminus S$ such that $S \cup \{X\}$ is independent under Δ .

(m) Prove that all bases under Δ have the same size.

(n) Do all minimal dependent sets under Δ have the same size? Give a proof of your claim.

(o) Prove that, for every base $\mathcal{B}$ of S under Δ , and every $X \in S$, there is a unique way to write X as a symmetric difference of members of $\mathcal{B}$, using each element of $\mathcal{B}$ at most once.

(p) For each previous part of this question, (a)–(o), state what type of proof you have used.

(q) Design a simple algorithm for the following problem.

INPUT: a collection S of subsets of E , together with a nonnegative real weight for each member of S .

OUTPUT: a subset $\mathcal{X}$ of S that is independent under Δ such that the sum of the weights of all members of $\mathcal{X}$ is maximum.

Does this algorithm always find the best possible solution? Justify your answer, using a previous exercise.

13. Identify the errors in the following “theorem” and “proof”, which are both incorrect.

The steps in the “proof” are numbered for convenience.

“Theorem.” For all $n \in \mathbb{N}$, a circular disc can be cut into 2^n pieces by n straight lines.

“Proof.”

1. Consider $n = 0$. The disc is not cut at all, so it is still in one piece. So the number of pieces is $2^0 = 1$. So the claim is true for $n = 0$.
2. Consider $n = 1$. A single line across a disc cuts it into two pieces, so the number of pieces is 2^1 . So the claim is true for $n = 1$.
3. Now consider $n = 2$. If the circle is cut by one line into two pieces, then we can cut it again by a line that crosses the first line. Such a line must cut right across both of the pieces created by the first cut. Therefore each of those pieces is divided in two, so the total number of pieces is $2 \times 2 = 2^2$, so the claim is true for $n = 2$.
4. It is clear from the cases considered so far that, once the circle is cut by some number of lines, then another line can be used to divide every piece into two pieces, thereby doubling the number of pieces.
5. So, if we use n lines, then the repeated doubling gives 2^n pieces. Therefore the claim is true for all n .

“□”

14. Identify the errors in the following “theorem” and “proof”, which are both incorrect.

“Theorem.” Every rational number is an integer.

“Proof.”

1. We need to show that the sets $\mathbb{Z}$ and $\mathbb{Q}$ are equal.
2. To prove that two sets are equal, we can prove the appropriate subset and superset relations between the two sets.
3. We first prove the subset relation between these two sets, $\mathbb{Z} \subseteq \mathbb{Q}$.
4. Let $x \in \mathbb{Z}$.
5. Since x is an integer, we have $x = \frac{x}{1}$.
6. So x is a quotient of integers.
7. Therefore x is rational, i.e., $x \in \mathbb{Q}$.
8. So we have proved that $\mathbb{Z} \subseteq \mathbb{Q}$.
9. Now we need to prove the superset relation, which is the reverse of subset.
10. So we will prove that $\mathbb{Q} \supseteq \mathbb{Z}$.
11. Let $x \notin \mathbb{Q}$.
12. Then x cannot be written in the form $\frac{p}{q}$ where p and q are integers.
13. So it certainly cannot be written in this form with $q = 1$.
14. Therefore x is not an integer, i.e., $x \notin \mathbb{Z}$.
15. So we have proved that $\mathbb{Q} \supseteq \mathbb{Z}$.
16. It follows from our subset and superset relations between the two sets that $\mathbb{Z} = \mathbb{Q}$.

“□”

“God made the integers; all else is the work of man.”

Leopold Kronecker (1823–1891)

15. Let G be the cryptosystem obtained from Caesar slide by restricting the keyspace to the first half of the alphabet. (See Exercise 2.15 for definitions relating to the Caesar slide cryptosystem.) So the keyspace K_G is

$$\{a, b, c, d, e, f, g, h, i, j, k, l, m\}.$$

Prove that G is not idempotent.

PROPOSITIONAL LOGIC

Computers are logical machines. This is not just a description of their behaviour, but also of their nature. Their hardware consists of huge numbers of tiny components that each do simple operations based on logic, and that do them very quickly. In their memory, information is stored as sequences of basic pieces which can each be True or False. The instructions used to get computers to do things — their programs — are expressed in a language that specifies what to do using formal, precise rules that use logic. Even before a program is written, a formal specification of the task to be done will make essential use of logic.

But logic is not just for machines. It is fundamental to clear thinking and precise communication. By learning about logic, you will improve your own thinking and communication, and therefore your prospects in life.

Logic plays a fundamental role in rigorous reasoning, especially in proofs. It will pervade everything we do in this unit and is fundamental to your future studies, not only in computer science but in mathematics, other sciences, engineering, economics, philosophy, and indeed any field of human activity where precise reasoning is important.

We will therefore now study the most fundamental types of logic. This week we study Propositional Logic, and next week we study Predicate Logic.

4.1^α TRUTH VALUES

Logic is built on just two values, called **truth values**: False and True.¹ These are abbreviated as F and T respectively.

Physically, truth values may correspond to the absence or presence of electrical current in a wire, or the absence or presence of magnetisation, or a switch being off or on, and so on. We use the abstraction of truth values in order to be able to discuss the logical workings of computers in a way that is independent of the particular technology used.

¹ There are forms of logic that use other values too, including three-valued logics which use the extra truth value Unknown. But we will focus entirely on classical two-valued logic, firstly because it is what computers are based on, and secondly because it is embedded within most other logics anyway, so that understanding two-valued logic is necessary in order to understand those other logics.

Another common abstraction of the notion of two possible states for something is the **bit**, which can be 0 or 1. This view is intimately related to truth values: in many situations, we can represent False by 0 and True by 1. It is worth exploring how the mathematical behaviour of 0 and 1 under basic arithmetic operations compares with that of False and True under logical operations. We will develop this link further in § 7.4.

4.2^α BOOLEAN VARIABLES

You are familiar with variables for numerical quantities, whose value can be any number from some set. We have already been using variables for other objects too. So it is natural to use variables whose values are truth values.

A **Boolean variable**, also called a **propositional variable**, is a variable whose value can be True or False.

Often, Boolean variables are used as names for statements which are either True or False.

4.3^α PROPOSITIONS

A **proposition** is a statement which is either true or false.

Examples

$1 + 1 = 2$	— a proposition which is true .
Xià Péisù designed the first computer in China	— a proposition which is true .
The earth is flat.	— a proposition which is false .
It will rain tomorrow.	— a proposition.

The left domino fell.	— a proposition.
See § 3.2.	

'Twas brillig, and the slithy toves did gyre and gimble in the wabe.	— <i>not</i> a proposition.
---	-----------------------------

From: Lewis Carroll, *Through the Looking Glass, and What Alice Found There*, Macmillan, London, 1871.

Come and work for us!	— <i>not</i> a proposition.
This statement is false.	— <i>not</i> a proposition.

For brevity, a proposition may be given a name, which is a Boolean variable. For example, let X be the proposition $1 + 1 = 2$. Then the truth value of X is True.

4.4^α LOGICAL OPERATIONS

Propositions may be combined to make other propositions using logical operations. Here are the basic logical operations we will use. We will define each of them shortly.

Not	$\neg$	$(\sim, \overline{}, -)$
And	$\wedge$	$(\&)$
Or	$\vee$	
Implies	$\Rightarrow$	$(\rightarrow)$
Equivalence	$\Leftrightarrow$	$(\leftrightarrow)$

Logical operations are also called **connectives**. Inside computers, they are implemented in electronic circuits called **logic gates**.

4.5 NEGATION

Logical negation is a unary operation which changes True to False and False to True. It is denoted by $\neg$, which is placed before its argument. So, $\neg\text{True} = \text{False}$ and $\neg\text{False} = \text{True}$. If a proposition P is True then $\neg P$ is False, and if P is False then $\neg P$ is True.

Example:

P : You have prepared for next week's tutorial.

$\neg P$: You have not prepared for next week's tutorial.

Other notation for $\neg P$ that you may come across: $\sim P$, $\overline{P}$, $-P$, $!P$

We can specify how $\neg$ works using a **truth table**, which lists all possible values of the arguments and, for each, states what the resulting value is. In this case, we only have one argument (let's call it P again), which has two possible values, so the table just has two rows (excluding the headings). The left column gives the possible values of the argument P and the right column gives the corresponding values of $\neg P$.

P	$\neg P$
F	T
T	F

Logical negation is reminiscent of set complementation (§ 1.11). In each case, you get something completely opposite, and doing it twice gets you back where you started. For logical negation, we have

$$\neg\neg P = P.$$

If a proposition asserts membership of a set, then its logical negation asserts membership of the complement. For example, consider the proposition $\sqrt{2} \in \mathbb{Q}$ (which is False) and

suppose our universal set is $\mathbb{R}$. Its logical negation $\neg(\sqrt{2} \in \mathbb{Q})$ may be written $\sqrt{2} \notin \mathbb{Q}$ or $\sqrt{2} \in \mathbb{R} \setminus \mathbb{Q}$ (which is True).

4.6 CONJUNCTION

Conjunction is a binary logical operation, i.e., it has two Boolean arguments. The result is True if and only if both its arguments are True. So, if at least one of its arguments is False, then the result is False.

We denote conjunction by $\wedge$ and read it as “and”. So the conjunction of P and Q is written $P \wedge Q$ and read as “ P and Q ”. Note that we are using the English word “and” in a strict, precise, logical sense here, which is narrower than the full range of meanings this word can have in English.

Example:

P Radhanath was a computer.

Q Radhanath was a person.

$P \wedge Q$ Radhanath was a computer and a person.



Radhanath Sikdar (1813–1870)

http://news.bbc.co.uk/2/hi/south_asia/3193576.stm

We can define conjunction symbolically using its truth table. It has two arguments (let's call them P and Q again), each of which can have two possible values (True and False), so there are $2^2 = 4$ combinations of arguments, hence four rows of the truth table. In each row, the corresponding value of $P \wedge Q$ is given in the last column.

P	Q	$P \wedge Q$
F	F	F
F	T	F
T	F	F
T	T	T

Conjunction is closely related to set intersection. If x is an object and A and B are sets, then the conjunction of the propositions $x \in A$ and $x \in B$ is the proposition $x \in A \cap B$:

$$(x \in A) \wedge (x \in B) = (x \in A \cap B).$$

Restating (1.12) using conjunction, we have

$$A \cap B = \{x : x \in A \wedge x \in B\}.$$

4.7 DISJUNCTION

Disjunction is another binary logical operation. Its result is True if and only if at least one of its arguments is True. So, if both of its arguments are False, then the result is False.

We denote disjunction by $\vee$ and read it as “or”. The disjunction of P and Q is written $P \vee Q$ and read as “ P or Q ”. Again, our use of English words is unusually specific: “or” is being used here in a strict, precise, logical sense, much narrower than its full range of English meanings. An analogous situation arose with “and” previously. Also, we are using the word “or” inclusively, so that a disjunction is True whenever *any one or more* of its arguments are True. For this reason, disjunction is sometimes called *inclusive-OR*. (This contrasts with the *exclusive-or* of two propositions, which is True precisely when *exactly one* of its two arguments is True; we discuss it in § 4.11.)

Example:

P I will study FIT3155 Advanced Data Structures & Algorithms.

Q I will study MTH3170 Network Mathematics.

$P \vee Q$ I’ll study FIT3155 **or** I’ll study MTH3170.

I’ll study *at least one of* FIT3155 and MTH3170.

Here is the truth table definition of disjunction.

P	Q	$P \vee Q$
F	F	F
F	T	T
T	F	T
T	T	T

Disjunction is closely related to set union. If x is an object and A and B are sets, then the disjunction of the propositions $x \in A$ and $x \in B$ is the proposition $x \in A \cup B$:

$$(x \in A) \vee (x \in B) = (x \in A \cup B).$$

Restating (1.11) using disjunction, we have

$$A \cup B = \{x : x \in A \vee x \in B\}.$$

At this point, it is worth reflecting on other set operations we discussed in [Chapter 1](#) and thinking about what logical operations they might correspond to. Feel free to invent your own logical operations if that helps.

4.8 DE MORGAN'S LAWS

Conjunction and disjunction seem somewhat opposite in character, but there is a close relationship between them, a kind of logical duality. This is captured by **De Morgan's Laws**.

$$\neg(P \vee Q) = \neg P \wedge \neg Q$$

$$\neg(P \wedge Q) = \neg P \vee \neg Q$$



Augustus De Morgan (1806–1871)
https://mathshistory.st-andrews.ac.uk/Biographies/De_Morgan/

These laws can be proved using truth tables. Consider the table below, which proves the first of De Morgan's Laws, $\neg(P \vee Q) = \neg P \wedge \neg Q$. We start out with the usual two columns giving all combinations of truth values of our variables P and Q . The overall approach is to gradually work along to the right, adding new columns that give some part of one of the expressions we are interested in, always using the columns we've already constructed in order to construct new columns. We'll first work towards constructing a column giving truth values for the left-hand side of the equation, $\neg(P \vee Q)$. As a step towards this, we make a column for $P \vee Q$: this becomes our third column. In fact, our first three columns are just the truth table for the disjunction $P \vee Q$, which we have seen before. Then we negate each entry in the third column to give the entries of the fourth column, which gives the truth table for $\neg(P \vee Q)$. So we've done the left-hand side of the equation. Then we start on the right-hand side of the equation, which is $\neg P \wedge \neg Q$. For this, we'll need $\neg P$ and $\neg Q$, which are obtained by negating the columns for P and Q respectively. This gives the fifth and sixth columns. Finally we form $\neg P \wedge \neg Q$ in the seventh column by just taking the conjunction of the corresponding entries in the fifth and sixth columns.

We now have columns giving the truth tables of both sides of the first of De Morgan's Laws: these are the fourth and seventh columns below, shown in **green**. These columns are identical! This shows that the two expressions $\neg(P \vee Q)$ and $\neg P \wedge \neg Q$ are logically equivalent, i.e., their truth values are the same for all possible assignments of truth values to their arguments P and Q . In other words, as Boolean expressions, they are equal. So $\neg(P \vee Q) = \neg P \wedge \neg Q$. This proves the first of De Morgan's Laws.

P	Q	$P \vee Q$	$\neg(P \vee Q)$	$\neg P$	$\neg Q$	$\neg P \wedge \neg Q$
F	F	F	T	T	T	T
F	T	T	F	T	F	F
T	F	T	F	F	T	F
T	T	T	F	F	F	F

We could prove the second of De Morgan's Laws by the same method. But, now that we know the first of De Morgan's Laws, it is natural to ask: can we use it to prove the second law more easily, so that we avoid doing the same amount of work all over again? In other words, *assuming* that $\neg(P \vee Q) = \neg P \wedge \neg Q$ holds for all P and Q , can you *prove* that $\neg(P \wedge Q) = \neg P \vee \neg Q$ holds for all P and Q without starting from scratch again and going through the same kind of detailed truth table argument? (Exercise 4.3)

De Morgan's Laws can also be proved by reasoning in a way that covers all possible combinations of truth values, rather than working laboriously through each combination of truth values separately (which is what the truth table proof does). We give such a proof now. For good measure, we do it for a more general version of the Law which caters for arbitrarily long conjunctions and disjunctions. This would not be possible just using the truth table approach, since we'd need a separate truth table for each n , which means we'd need infinitely many truth tables.

Theorem 23. For all n :

$$\neg(P_1 \vee \dots \vee P_n) = \neg P_1 \wedge \dots \wedge \neg P_n$$

Proof.

Left-Hand Side is True	if and only if	$P_1 \vee \dots \vee P_n$ is False
	if and only if	$P_1, \dots, P_n$ are all False
	if and only if	$\neg P_1, \dots, \neg P_n$ are all True
	if and only if	Right-Hand Side is True.

□

Again, we ask: having proved the first of De Morgan's Laws (in this more general form), can we use it to prove the second law more easily? How would you prove the second law?

There is a clear correspondence between De Morgan's Laws for Sets, in [Theorem 1](#) and [Corollary 2](#), and De Morgan's Laws for Logic.

4.9 IMPLICATION

Our next logical operation is **implication**, also called **conditional**, from § 3.2. Recall that, if P and Q are its arguments, it is written $P \Rightarrow Q$ and read " P implies Q " or "if P then Q ". It means that if P is True then Q must also be True. The only way its result can be False is if P is True and Q is False.

Example:

P Stars are visible.
 Q The sun has set.

$P \Rightarrow Q$ If stars are visible **then** the sun has set.
 Stars being visible **implies** the sun has set.
 Stars are visible **only if** the sun has set.
 Stars are visible is **sufficient** for the sun to have set.

Here is its truth table.

P	Q	$P \Rightarrow Q$
F	F	T
F	T	T
T	F	F
T	T	T

As we discussed in § 3.2, implication is closely related to the subset relation.

4.10 EQUIVALENCE

Our next binary logical operation is **equivalence**, also called **biimplication** or **biconditional**. We discussed this previously towards the end of § 1.6 on p. 9, and in the last paragraph of § 3.2 on p. 92.

For arguments P and Q , equivalence is written $P \Leftrightarrow Q$ and read “ P if and only if Q ” or “ P is equivalent to Q ”. It is true precisely when P and Q are either both True or both False. In other words, as Boolean variables, their values are always equal.

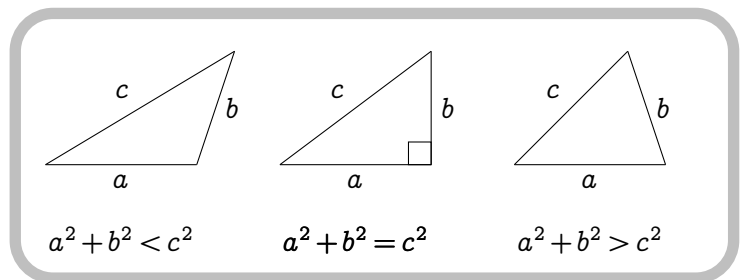
You can view $P \Leftrightarrow Q$ as the conjunction of $P \Rightarrow Q$ and $Q \Rightarrow P$:

$$P \Leftrightarrow Q = (P \Rightarrow Q) \wedge (Q \Rightarrow P).$$

$P \Leftrightarrow Q$ can be written the other way round, as $Q \Leftrightarrow P$. They have the same meaning.

Example:

P The triangle is right-angled.
 Q The side lengths satisfy
 $a^2 + b^2 = c^2$.



$P \Leftrightarrow Q$ The triangle is right-angled **if and only if** $a^2 + b^2 = c^2$.

The triangle being right-angled is a **necessary and sufficient condition** for $a^2 + b^2 = c^2$.

$a^2 + b^2 = c^2$ is a **necessary and sufficient condition** for the triangle being right-angled.

Here is its truth table.

P	Q	$P \Leftrightarrow Q$
F	F	T
F	T	F
T	F	F
T	T	T

Equivalence is closely related to set equality. If x is an object and A and B are sets, then the propositions $x \in A$ and $x \in B$ are equivalent if x belongs to both sets or neither of them. If this holds for all x then the two sets are identical. Conversely, if two sets are identical then the propositions $x \in A$ and $x \in B$ are always equivalent.

4.11 EXCLUSIVE-OR

Our final logical operation is **exclusive-or**. For arguments P and Q , exclusive-or is written $P \oplus Q$ (or sometimes $P \text{ XOR } Q$), and read “ P exclusive-or Q ” or “ P ex-or Q ”. It is true precisely when either P is True or Q is True, *but not both*.

Exclusive-or differs from inclusive-or (disjunction, § 4.7) in its value when both P and Q are True. In that case, exclusive-or is False but inclusive-or is True.

Here is its truth table.

P	Q	$P \oplus Q$
F	F	F
F	T	T
T	F	T
T	T	F

It is evident from their truth tables that exclusive-or is actually the logical negation of equivalence:

$$P \oplus Q = \neg(P \Leftrightarrow Q).$$

The exclusive-or of two propositions is True if and only if *exactly one* of them is True. What happens if we combine three propositions with exclusive-or, as in $P_1 \oplus P_2 \oplus P_3$? The exclusive-or of the first two, $P_1 \oplus P_2$, is True precisely when exactly one of P_1 and P_2 is True, and in that case $P_1 \oplus P_2 \oplus P_3$ can only be True if P_3 is False, so we still

have exactly one of the propositions P_1, P_2, P_3 being True. But there is another way for $P_1 \oplus P_2 \oplus P_3$ to be True, namely if $P_1 \oplus P_2$ is False and P_3 is True. For $P_1 \oplus P_2$ to be False, *neither* of P_1 and P_2 is True, or they *both* are. So we might again have exactly one of P_1, P_2, P_3 being True, or we might in fact have *all three* of P_1, P_2, P_3 being True.

We have covered all the possible things that can happen, if $P_1 \oplus P_2 \oplus P_3$ is to be True, and we have found that the number of P_1, P_2, P_3 that are True must be 1 or 3. So it needn't be *exactly* one; fortunately, we have avoided a common mistake there. In fact, what we can say is that the number of P_1, P_2, P_3 that are True must be odd.

This generalises to arbitrary numbers of propositions. You should play with some examples (say, with four propositions P_1, P_2, P_3, P_4), satisfy yourself that this does indeed happen in general, and try to understand why. Then you will get more out of reading the formal proof which we now present.

Theorem 24. For all $n \in \mathbb{N}$, and for any propositions $P_1, P_2, \dots, P_n$,

$$P_1 \oplus P_2 \oplus \dots \oplus P_n = \begin{cases} \text{True,} & \text{if an \underline{odd} number of } P_1, P_2, \dots, P_n \text{ is True;} \\ \text{False,} & \text{if an \underline{even} number of } P_1, P_2, \dots, P_n \text{ is True.} \end{cases} \quad (4.1)$$

Proof. We prove this by induction on n .

Inductive Basis:

When $n = 1$, the expression on the left of (4.1) is just P_1 , and this is True if if just one of P_1 is True, and False otherwise! So (4.1) holds in this case.

Inductive Step:

Let $k \geq 1$. Assume (4.1) holds for $n = k$; this is the Inductive Hypothesis.

Now consider $P_1 \oplus P_2 \oplus \cdots \oplus P_{k+1}$.

$$\begin{aligned}
& P_1 \oplus P_2 \oplus \cdots \oplus P_{k+1} \\
&= (P_1 \oplus P_2 \oplus \cdots \oplus P_k) \oplus P_{k+1} \\
&\quad \text{(identifying a smaller expression of the same type, within this expression)} \\
&= \left(\begin{cases} \text{True,} & \text{if an odd number of } P_1, P_2, \dots, P_n \text{ is True;} \\ \text{False,} & \text{if an even number of } P_1, P_2, \dots, P_n \text{ is True.} \end{cases} \right) \oplus P_{k+1} \\
&\quad \text{(by the Inductive Hypothesis)} \\
&= \begin{cases} \text{True} \oplus P_{k+1}, & \text{if an odd number of } P_1, P_2, \dots, P_n \text{ is True;} \\ \text{False} \oplus P_{k+1}, & \text{if an even number of } P_1, P_2, \dots, P_n \text{ is True.} \end{cases} \\
&= \begin{cases} \text{True} \oplus \text{False}, & \text{if an odd number of } P_1, P_2, \dots, P_n \text{ is True and } P_{k+1} \text{ is False;} \\ \text{True} \oplus \text{True}, & \text{if an odd number of } P_1, P_2, \dots, P_n \text{ is True and } P_{k+1} \text{ is True;} \\ \text{False} \oplus \text{False}, & \text{if an even number of } P_1, P_2, \dots, P_n \text{ is True and } P_{k+1} \text{ is False;} \\ \text{False} \oplus \text{True}, & \text{if an even number of } P_1, P_2, \dots, P_n \text{ is True and } P_{k+1} \text{ is True.} \end{cases} \\
&\quad \text{(breaking each option down according to the value of } P_{k+1} \text{)} \\
&= \begin{cases} \text{True,} & \text{if an odd number of } P_1, P_2, \dots, P_n \text{ is True and } P_{k+1} \text{ is False;} \\ \text{False,} & \text{if an odd number of } P_1, P_2, \dots, P_n \text{ is True and } P_{k+1} \text{ is True;} \\ \text{False,} & \text{if an even number of } P_1, P_2, \dots, P_n \text{ is True and } P_{k+1} \text{ is False;} \\ \text{True,} & \text{if an even number of } P_1, P_2, \dots, P_n \text{ is True and } P_{k+1} \text{ is True.} \end{cases} \\
&\quad \text{(evaluating the exclusive-ors)} \\
&= \begin{cases} \text{True,} & \text{if an odd number of } P_1, P_2, \dots, P_k, P_{k+1} \text{ is True;} \\ \text{False,} & \text{if an even number of } P_1, P_2, \dots, P_k, P_{k+1} \text{ is True.} \end{cases} \\
&\quad \text{(combining the True cases and combining the False cases, and} \\
&\quad \text{noting that their conditions also combine to capture precisely} \\
&\quad \text{the evenness or oddness of the number of the } P_i \text{ that are True).}
\end{aligned}$$

So (4.1) holds for $n = k + 1$ too. This completes the Inductive Step.

Conclusion:

Therefore, by Mathematical Induction, (4.1) holds for all $n \in \mathbb{N}$. □

Exclusive-or is closely related to the symmetric difference of sets. If x is an object and A and B are sets, then

$$x \in A \triangle B \quad \text{if and only if} \quad (x \in A) \oplus (x \in B).$$

More generally, if $A_1, A_2, \dots, A_n$ are sets, then

$$x \in A_1 \triangle A_2 \triangle \cdots \triangle A_n \quad \text{if and only if} \quad (x \in A_1) \oplus (x \in A_2) \oplus \cdots \oplus (x \in A_n).$$

4.12 TAUTOLOGIES AND LOGICAL EQUIVALENCE

A **tautology** is a statement that is always true. In other words, the right-hand column of its truth table has every entry True.

Two statements P and Q are **logically equivalent** if their truth tables are identical. In other words, P and Q are equivalent if and only if $P \Leftrightarrow Q$ is a tautology.

Examples:

$\neg\neg P$	is logically equivalent to	P
$\neg(P \vee Q)$	is logically equivalent to	$\neg P \wedge \neg Q$
$\neg(P \wedge Q)$	is logically equivalent to	$\neg P \vee \neg Q$
$P \Rightarrow Q$	is logically equivalent to	$\neg P \vee Q$
$P \Leftrightarrow Q$	is logically equivalent to	$(P \Rightarrow Q) \wedge (P \Leftarrow Q)$
	and to	$(\neg P \vee Q) \wedge (P \vee \neg Q)$
$P \oplus Q$	is logically equivalent to	$\neg(P \Leftrightarrow Q)$
	and to	$P \Leftrightarrow \neg Q$
	and to	$(P \Rightarrow \neg Q) \wedge (P \Leftarrow \neg Q)$

These can all be proved using truth tables.

We usually denote logical equivalence by “ $=$ ”. So we write $\neg\neg P = P$, etc.

We have already met logical equivalence. Each of De Morgan’s Laws states that its left-hand side is logically equivalent to its right-hand side.

4.13^ω HISTORY

George Boole (1815–1864)

<https://mathshistory.st-andrews.ac.uk/Biographies/Boole/>

4.14 DISTRIBUTIVE LAWS

$$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R) \quad (4.2)$$

$$P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R) \quad (4.3)$$

It is notable that we have two distributive laws in propositional logic. Conjunction is distributive over disjunction (first law above, (4.2)), and disjunction is distributive over conjunction (second law above, (4.3)).

This contrasts with the situation for ordinary algebra of numbers, when multiplication is distributive over addition, but addition is not distributive over multiplication.

$$p \times (q + r) = (p \times q) + (p \times r)$$

but

$$p + (q \times r) \neq (p + q) \times (p + r)$$

Just as for De Morgan's Laws, we see a correspondence between the algebra of sets and the algebra of logic. The Distributive Laws for sets, [Theorem 3](#), correspond to the Distributive Laws for Logic.

4.15 LAWS OF BOOLEAN ALGEBRA

Here is a full listing of the laws of Boolean algebra, which we may use to convert propositional expressions from one form to another. Reasons why we might do this include finding simpler forms for Boolean expressions and determining algebraically whether or not two given Boolean expressions are logically equivalent.

Because of the logical duality between conjunction and disjunction, these laws may be arranged in dual pairs. In the table below, each law involving conjunction or disjunction is written on the same line as its dual.

$\neg\neg P = P$	
$\neg\text{True} = \text{False}$	$\neg\text{False} = \text{True}$
$P \wedge Q = Q \wedge P$	$P \vee Q = Q \vee P$
$(P \wedge Q) \wedge R = P \wedge (Q \wedge R)$	$(P \vee Q) \vee R = P \vee (Q \vee R)$
$P \wedge P = P$	$P \vee P = P$
$P \wedge \neg P = \text{False}$	$P \vee \neg P = \text{True}$
$P \wedge \text{True} = P$	$P \vee \text{False} = P$
$P \wedge \text{False} = \text{False}$	$P \vee \text{True} = \text{True}$

$$\begin{array}{lcl}
 & \text{Distributive Laws} & \\
 P \wedge (Q \vee R) & = & (P \wedge Q) \vee (P \wedge R) \qquad P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R) \\
 & \text{De Morgan's Laws} & \\
 \neg(P \vee Q) & = & \neg P \wedge \neg Q \qquad \neg(P \wedge Q) = \neg P \vee \neg Q
 \end{array}$$

4.16 DISJUNCTIVE NORMAL FORM

We will introduce two standard ways of writing logical expressions. The first of these is *Disjunctive Normal Form (DNF)*, treated in this section. The second is *Conjunctive Normal Form (CNF)*, treated in the next section.

We treat DNF first, but CNF will be much more important for us. In brief, that's because CNF is more natural for encoding logical problems and real-world conditions. You can always convert one to the other, but at considerable cost in both time and space as we will see.

A **literal** is an appearance of a logical variable in which it is either unnegated or negated just once. So, if X is a logical variable, then its corresponding literals are X and $\neg X$. Separate appearances of a logical variable within a larger logical expression are counted as separate literals. For example, the expression $(\neg X \wedge \neg Y) \vee (\neg X \wedge Y) \vee (X \wedge Y)$ has six literals (even though some are equivalent to each other). We do not consider $\neg\neg X$ to be a literal, as it is, but it is equivalent to X , which is a literal. Similarly, $\neg\neg\neg X$ is not a literal but is equivalent to the literal $\neg X$.

A Boolean expression is in **Disjunctive Normal Form (DNF)** if

- it is written as a disjunction of some number of parts, *where*
- each part is a conjunction of some number of literals.

Examples:

$(\neg X \wedge \neg Y) \vee (\neg X \wedge Y) \vee (X \wedge Y)$	a disjunction of three parts, each of which is a conjunction of two literals.
$(A \wedge \neg B) \vee (\neg A \wedge \neg B \wedge C \wedge D)$	a disjunction of two parts, where the first part is a conjunction of two literals and the second part is a conjunction of four literals.
$P \vee Q \vee \neg R$	a disjunction of three parts, each containing a single literal.
$P \wedge Q \wedge R$	a disjunction of one part, which contains three literals.
Z	a disjunction of one part, which contains one literal.

You can convert any proposition into an equivalent one in DNF using its truth table. Consider the proposition P given by the following truth table.

X	Y	P
F	F	T
F	T	T
T	F	F
T	T	T

As a step towards designing a logical expression for the entire proposition P , we will design one logical expression for each row where P is True.

Consider the first row. This is for when $X = \text{False}$ and $Y = \text{False}$. We want the result to be True. We can do this using $\neg X \wedge \neg Y$. Satisfy yourself that this is True when $X = \text{False}$ and $Y = \text{False}$, and also that it is False *for every other combination of truth values* for X and Y . So it is True *only* in this first row, as its own truth table shows:

X	Y	$\neg X \wedge \neg Y$
F	F	T
F	T	F
T	F	F
T	T	F

Now consider the second row of the truth table for P , which is for when $X = \text{False}$ and $Y = \text{True}$. This time we will use $\neg X \wedge Y$, which is True for this row but False for all the other rows. We can add an extra column to include its truth table too.

X	Y	$\neg X \wedge \neg Y$	$\neg X \wedge Y$
F	F	T	F
F	T	F	T
T	F	F	F
T	T	F	F

The third row of the truth table for P has $P = \text{False}$, so we will ignore that.

The fourth row of the truth table for P has $P = \text{True}$ again. We will now use $X \wedge Y$, which is True for this row but for no other. We add a further column for its truth table.

X	Y	$\neg X \wedge \neg Y$	$\neg X \wedge Y$	$X \wedge Y$
F	F	T	F	F
F	T	F	T	F
T	F	F	F	F
T	T	F	F	T

Now look at what happens when we take the disjunction of the last three columns.

X	Y	$\neg X \wedge \neg Y$	$\neg X \wedge Y$	$X \wedge Y$	$(\neg X \wedge \neg Y) \vee (\neg X \wedge Y) \vee (X \wedge Y)$
F	F	T	F	F	T
F	T	F	T	F	T
T	F	F	F	F	F
T	T	F	F	T	T

This shows that the DNF expression $(\neg X \wedge \neg Y) \vee (\neg X \wedge Y) \vee (X \wedge Y)$ is equivalent to P .

$$P = \underbrace{(\overbrace{\neg X \wedge \neg Y}^{\text{conjunction}}) \vee (\overbrace{\neg X \wedge Y}^{\text{conjunction}}) \vee (\overbrace{X \wedge Y}^{\text{conjunction}})}_{\text{disjunction}}$$

DNF expressions like this can be read easily from truth tables. You don't have to add extra columns as we did above. In each row, look at the pattern of truth values for the variables. Then write down a conjunction of literals where variables that are True are written normally and variables that are False are written in negated form. For example, in the second row of our truth table, the variable X is False so we negate it, whereas Y is True so we just use it unchanged. Taking the conjunction of these two literals gives $\neg X \wedge Y$, which is the part we want for that row. Do this for every row in which the proposition P is True. This is shown for our current example in the following table.

X	Y	P	
F	F	T	$\neg X \wedge \neg Y$
F	T	T	$\neg X \wedge Y$
T	F	F	
T	T	T	$X \wedge Y$

When this is done, just take the disjunction of all the parts in the final column and you have your DNF expression for P .

Exercise: simplify the above expression P as much as possible, using Boolean algebra.

Here is another example of this method. The columns X, Y, Z correspond to three variables X, Y, Z , and the fourth column gives the truth table for a new proposition P . In the fifth column, we look at each row where $P = \text{True}$ and convert the first three truth values in that row into a conjunction of literals that follows the required pattern. For this particular P , this gives four conjunctions.

X	Y	Z	P	
F	F	F	T	$\neg X \wedge \neg Y \wedge \neg Z$
F	F	T	F	
F	T	F	T	$\neg X \wedge Y \wedge \neg Z$
F	T	T	F	
T	F	F	F	
T	F	T	T	$X \wedge \neg Y \wedge Z$
T	T	F	T	$X \wedge Y \wedge \neg Z$
T	T	T	F	

Taking the disjunction of all the conjunctions in the last column gives a DNF expression for P .

$$P = (\neg X \wedge \neg Y \wedge \neg Z) \vee (\neg X \wedge Y \wedge \neg Z) \vee (X \wedge \neg Y \wedge Z) \vee (X \wedge Y \wedge \neg Z)$$

The importance of this method is that it shows that every logical expression is equivalent to one in DNF. So DNF can be viewed as a “standard form” into which any logical expression can be transformed.

BUT this transformation comes at a price. The DNF expression has as many parts as there are truth table rows where the expression is True. An expression with k variables has 2^k rows in its truth table, which is exponential in the number of variables. If the expression is True for a large proportion of its truth table rows, then the number of parts in the DNF expression may also be exponentially large in the number of variables. So it may be too large and unwieldy to be useful, unless the number of variables is very small. (If a Boolean expression is actually provided in the form of its entire truth table, then constructing its equivalent DNF expression by the above method is ok, since the expression you get won’t be any larger than the truth table. But if the Boolean expression is provided as a compact formula, then the size of its equivalent DNF expression may be exponential in the size of the formula you started with.)

One apparent attraction of DNF is that it is easy to tell if a DNF expression is satisfiable, that is, if there is some assignment of truth values to its variables that makes the whole expression True. In fact, if you take any of the parts of a DNF expression, the pattern of the literals (i.e., whether each appears plainly or negated) tells you a truth assignment that makes that part True, and then the whole disjunction must also be True because a disjunction is True precisely when at least one of its parts is True. In effect, the parts of a DNF expression yield a kind of encoded listing of all the truth assignments that make the whole expression True.

On the other hand, it does not seem so easy to tell if a DNF expression is a tautology. For a DNF expression to *not* be a tautology, there would have to be some truth assignment to its variables that makes it False. There is no known way to do test for this efficiently.

The big problem with DNF, though, is that, in real life, logical rules are not usually specified in a form that is amenable to DNF. They are typically described by listing

conditions that must be satisfied together. In other words, they are described in a way that lends itself to expression as a conjunction rather than a disjunction.

4.17 CONJUNCTIVE NORMAL FORM

A Boolean expression is in **Conjunctive Normal Form (CNF)** if

- it is written as a conjunction of some number of parts (sometimes called **clauses**), *where*
- each part is a disjunction of some number of literals.

For example:

$$\underbrace{(\overbrace{-P \vee Q}^{\text{disjunction}}) \wedge (\overbrace{P \vee -Q}^{\text{disjunction}})}_{\text{conjunction}}$$

There is a close relationship — a kind of logical duality — between CNF and DNF. Suppose you have an expression P in CNF, and suppose you negate it, giving $\neg P$. So $\neg P$ is a negation of a conjunction of a disjunction of literals. But, by De Morgan's Law, a negation of a conjunction is a disjunction of negations. So $\neg P$ will then be expressed as a disjunction of negations of disjunctions of literals. But, again by De Morgan's Law, each negation of a disjunction is equivalent to a conjunction of negations. So $\neg P$ is now a disjunction of conjunctions of negations of literals. But the negation of a literal is always equivalent to another literal (since $\neg\neg X = X$). So we see that $\neg P$ is a disjunction of conjunctions of literals. In other words, it's in DNF.

We now have a way to convert any logical expression P in truth table form to a CNF expression. To do so, we just

1. negate all the truth values in the output column (turning it into a truth table for $\neg P$),
2. use the method of the previous subsection to construct a DNF expression for $\neg P$,
3. then negate the expression (so that it will now be an expression for P again),
4. and use De Morgan's Laws to transform the negated DNF expression into CNF.

This establishes the important theoretical point that every expression is equivalent to a CNF expression. But this is usually not a good way to construct CNF expressions in practice, because:

- Truth tables are too large. As discussed earlier, their size is exponential in the number of variables. Complex logical conditions in real computational problems usually contain enough variables for truth tables to be unusable. Furthermore, even for more modest-sized problems, the truth table approach for CNF uses a lot of time and space and, for manual work, is quite error-prone.

- Logical conditions are usually expressed in a way that makes CNF a natural way to represent them.

4.18 REPRESENTING LOGICAL STATEMENTS

Suppose we are given a set of rules or conditions that we want to model as a logical expression. These are often expressed as conditions that must be satisfied *together*. For example, the rules of a game must *all* be followed, to play the game correctly; you can't just ignore the rules you don't like on the grounds that you're following *some* of the rules! A software specification typically stipulates a set of conditions that must all be met, and similarly for legal contracts, acts of parliament, traffic regulations, itineraries, and so on. While some rules may offer choices as to how they can be satisfied, at the highest level a set of rules is usually best modelled as a conjunction.

So, if you are given some specifications and you want to model them by a logical expression, one first step you can take (working "top-down") is to identify how the rule is structured at the top level as a conjunction. What are the parts of this conjunction? You can then keep working top-down and try to decompose those parts as conjunctions too. A conjunction of conjunctions is just one larger conjunction.

Working from the other direction ("bottom-up"), you also have to think about what your most elementary logical "atoms" are. In other words, think about the simplest, most basic assertion that could hypothetically be made about this situation, without worrying about whether it might be True or False. In fact, in this kind of situation, you won't initially know what values your logical variables might have; you're merely encoding your problem in logical form, without solving it yet, so you avoid thinking about actual truth values for any of your variables. You are just trying to identify the kinds of "atomic assertions" that are needed to describe *any* hypothetical situation in your scenario.

Example:

You are planning a dinner party. Your guest list must have:

- at least one of: Harry, Ron, Hermione, Ginny
- Hagrid *only if* it also has Norberta
- none, or both, of Fred and George
- no more than one of: Voldemort, Bellatrix, Dolores.

At the top level, we can see that this is a conjunction:

(at least one of: Harry, Ron, Hermione, Ginny)
 $\wedge$ (Hagrid *only if* it also has Norberta)
 $\wedge$ (none, or both, of Fred and George)
 $\wedge$ (no more than one of: Voldemort, Bellatrix, Dolores).

Note that we're not trying to convert everything to logic at once. The four parts of our conjunction are not yet expressed in logical form; they're still just written in English text. That's ok, in this intermediate stage.

Early in the process, we should think about what Boolean variables to use, and what they should represent. In this case, that is fairly straightforward. The simplest logical statement we can make in this situation is that a specific person is on your guest list. So, for each person, we'll introduce a Boolean variable with the intended interpretation that the person is on your guest list. So, the variable Harry is intended to mean that the person Harry is on your guest list, and so on. This gives us eleven variables, one for each of our eleven guests. As far as we know at the moment, each of the variables might be True or False; it is the role of the logical expression we are constructing to ensure that the combinations of truth values for these eleven variables must correspond to valid guest lists. We will do that by properly representing the rules in logic. We will not try, at this stage, to enumerate all possible guest lists, or even to find one valid guest list. Our current task is to encode the problem's rules in logic, not to *solve* the problem. (That can be tackled later, and is a different skill.)

Now, let's look at each of the four parts of our conjunction, in turn, and see how they may be logically expressed using our variables.

- at least one of: Harry, Ron, Hermione, Ginny.
This is a *disjunction* of the four corresponding variables:

$$\text{Harry} \vee \text{Ron} \vee \text{Hermione} \vee \text{Ginny}$$

- Hagrid *only if* it also has Norberta.
This is modelled by *implication*:

$$\text{Hagrid} \Rightarrow \text{Norberta}$$

- none, or both, of Fred and George.
This is a job for the *biconditional*:

$$\text{Fred} \Leftrightarrow \text{George}$$

- no more than one of: Voldemort, Bellatrix, Dolores.
This requires some more thought! Unlike the previous parts, it doesn't correspond immediately to a logical operation we have already met. So we will try to see if it can be broken down further into a conjunction of simpler conditions. To do this, consider that "no more than one of ..." is the same as saying that every *pair* of them is forbidden. So, the pair Voldemort & Bellatrix is forbidden, and the pair

Voldemort & Dolores is forbidden, and the pair Bellatrix & Dolores is forbidden. See the logical structure emerging: "...and ...and ...". So we have:

$$\begin{aligned} & (\textit{not both Voldemort \& Bellatrix}) \wedge \\ & (\textit{not both Voldemort \& Dolores}) \wedge \\ & (\textit{not both Bellatrix \& Dolores}) \end{aligned}$$

So our expression is now:

$$\begin{aligned} & (\text{Harry} \vee \text{Ron} \vee \text{Hermione} \vee \text{Ginny}) \\ & \wedge (\text{Hagrid} \Rightarrow \text{Norberta}) \\ & \wedge (\text{Fred} \Leftrightarrow \text{George}) \\ & \wedge (\textit{not both Voldemort \& Bellatrix}) \\ & \wedge (\textit{not both Voldemort \& Dolores}) \\ & \wedge (\textit{not both Bellatrix \& Dolores}). \end{aligned}$$

We are now getting to the point where we can use logical manipulations (Boolean algebra) to transform each of the four parts into a disjunction of literals. The first part is already a disjunction. The second part can be written as the disjunction $\neg\text{Hagrid} \vee \text{Norberta}$, as we have already seen. The third part can be written

$$(\text{Fred} \Rightarrow \text{George}) \wedge (\text{Fred} \Leftarrow \text{George})$$

and turning each implication into an appropriate disjunction gives

$$(\neg\text{Fred} \vee \text{George}) \wedge (\text{Fred} \vee \neg\text{George}).$$

For the fourth part, requiring that Voldemort and Bellatrix are *not both* True is the same as requiring at least one of them to be False, which is the same as requiring at least one of their negations to be True, which is captured by $\neg\text{Voldemort} \vee \neg\text{Bellatrix}$. Treating the other two pairs the same way gives the conjunction of disjunctions

$$(\neg\text{Voldemort} \vee \neg\text{Bellatrix}) \wedge (\neg\text{Voldemort} \vee \neg\text{Dolores}) \wedge (\neg\text{Bellatrix} \vee \neg\text{Dolores})$$

Putting these altogether gives the expression

$$\begin{aligned} & (\text{Harry} \vee \text{Ron} \vee \text{Hermione} \vee \text{Ginny}) \\ & \wedge (\neg\text{Hagrid} \vee \text{Norberta}) \\ & \wedge (\neg\text{Fred} \vee \text{George}) \wedge (\text{Fred} \vee \neg\text{George}) \\ & \wedge (\neg\text{Voldemort} \vee \neg\text{Bellatrix}) \wedge (\neg\text{Voldemort} \vee \neg\text{Dolores}) \wedge (\neg\text{Bellatrix} \vee \neg\text{Dolores}) \end{aligned}$$

This is now in CNF.

Challenge: how long would an equivalent DNF expression be?

4.19 STATEMENTS ABOUT HOW MANY VARIABLES ARE TRUE

Given a collection of Boolean variables, we are often interested in how many of them are True. We might want to state that at least two of them are True, or that at most two of them are True, or exactly two of them are True. We might want to make similar statements with “two” replaced by some other number. Conditions of this kind are examples of *cardinality constraints*, since they are only about the number of variables with a given value.

We saw examples of this in the previous section. We wanted at least *one* of Harry, Ron, Hermione and Ginny to be True. We also wanted at most *one* of Voldemort, Bellatrix and Dolores to be True. So, to build your intuition about dealing with these situations, it would be worth pausing now and spending a few minutes reading that analysis again, and thinking through how the reasoning given there could be extended to situations where the number of True variables involved is some other number (i.e., not one).

If ever we want to specify that exactly k variables are True, we can express this as a conjunction:

$$(\text{at least } k \text{ are True}) \wedge (\text{at most } k \text{ are True}).$$

So we now focus our discussion on just the “at least” and “at most” cases.

Suppose we want to state that at most k of the n variables $x_1, x_2, \dots, x_n$ are True. This means that, for every set of $k+1$ variables, at least one of them is False, or in other words, at least one of their negations is True. So, for every set of $k+1$ variables, we form a disjunction of their negations (to say that at least one of these negations is True), and then we combine all these disjunctions into a larger conjunction. The number of disjunctions we use is just the number of subsets of $k+1$ variables chosen from our n variables, which is $\binom{n}{k+1}$.

For example, suppose we want to say that at most *two* of the four variables w, x, y, z is True (i.e., $k=2$ and $n=4$). This means that, for every *three* of the variables, at least one of them is False. So, at least one of w, x, y is False, and at least one of w, x, z is False, and so on. But saying that at least one of a set of variables is False is the same as saying that at least one of their *negations* is True. For example, at least one of w, x, y is False if and only if at least one of $\neg w, \neg x, \neg y$ is True. This is now a job for disjunction: at least one of $\neg w, \neg x, \neg y$ is True if and only if $\neg w \vee \neg x \vee \neg y$ is True. So, we create all disjunctions of *triples* ($k+1=3$) of *negated* literals, which gives the disjunctions

$$\neg w \vee \neg x \vee \neg y, \quad \neg w \vee \neg x \vee \neg z, \quad \neg w \vee \neg y \vee \neg z, \quad \neg x \vee \neg y \vee \neg z.$$

The number of these disjunctions is $\binom{n}{k+1} = \binom{4}{3} = 4$. These disjunctions are then combined by conjunction:

$$(\neg w \vee \neg x \vee \neg y) \wedge (\neg w \vee \neg x \vee \neg z) \wedge (\neg w \vee \neg y \vee \neg z) \wedge (\neg x \vee \neg y \vee \neg z).$$

Now suppose we want to state that at least k of the n variables $x_1, x_2, \dots, x_n$ are True. This means that at most $n - k$ of them are False. This means that, for every set of $n - k + 1$ variables, at least one of them is True. So, for every set of $n - k + 1$ variables, we form a disjunction of them (to say that at least one of them is True), and then we combine all these disjunctions into a larger conjunction.

For example, suppose we want to say that at least *two* of the four variables w, x, y, z is True (i.e., $k = 2$ and $n = 4$). We create all disjunctions of *triples* ($n - k + 1 = 4 - 2 + 1 = 3$) of literals (unnegated, this time), which gives the disjunctions

$$w \vee x \vee y, \quad w \vee x \vee z, \quad w \vee y \vee z, \quad x \vee y \vee z.$$

These are then combined by conjunction:

$$(w \vee x \vee y) \wedge (w \vee x \vee z) \wedge (w \vee y \vee z) \wedge (x \vee y \vee z).$$

Finally, if we want to say that exactly two of the four variables are True, then we take the conjunction of the expressions for “at least” and “at most”, giving

$$\begin{aligned} &(\neg w \vee \neg x \vee \neg y) \wedge (\neg w \vee \neg x \vee \neg z) \wedge (\neg w \vee \neg y \vee \neg z) \wedge (\neg x \vee \neg y \vee \neg z) \wedge \\ &(w \vee x \vee y) \wedge (w \vee x \vee z) \wedge (w \vee y \vee z) \wedge (x \vee y \vee z). \end{aligned}$$

4.20 UNIVERSAL SETS OF OPERATIONS

We saw in § 4.16 that every logical expression is equivalent to a DNF expression. Later, in § 4.17, we observed that every logical expression is also equivalent to a CNF expression.

One consequence of this is that every logical expression is equivalent to an expression that only uses the operations from the operation set $\{\wedge, \vee, \neg\}$.

We say that a set of operations X is **universal** if every logical expression P is equivalent to an expression Q that only uses operations in that set (together with variables from the original expression). So the set of operations that actually appear in Q must be a subset of the operation set X .

Our observations above, about DNF and CNF, demonstrate that the operation set

$$\{\wedge, \vee, \neg\}$$

is universal.

One reason for considering universal sets of operations relates to the construction of electronic circuits that compute the values of logical expressions. It is easier to make these circuits if we can put them together from simple components of only a few different types. Simple components are easier to construct, and the complexity of manufacturing is reduced if we don't have to make too many different types of components. We also gain economies of scale from making large numbers of these simple components instead of smaller numbers of more complex components.

The operation set $\{\wedge, \vee, \neg\}$ is not the smallest possible universal set of operations. De Morgan's Laws (§ 4.8) show how to express $\wedge$ in terms of $\vee$ and $\neg$, and dually how to express $\vee$ in terms of $\wedge$ and $\neg$. So either $\wedge$ or $\vee$ could be dropped from our universal set, as long as we retain the other as well as $\neg$. So the operation set

$$\{\wedge, \neg\}$$

is also universal, and so is

$$\{\vee, \neg\}.$$

Having just seen that we can drop either $\wedge$ or $\vee$ from our universal set $\{\wedge, \vee, \neg\}$, while still retaining the universal property, it is natural to ask if we can drop $\neg$ and keep the other two. In other words, is the set

$$\{\wedge, \vee\}$$

universal?

Having found universal sets of just two operations, we can ask if this is the smallest possible. Does there exist a universal set of just one operation?

Clearly the set $\{\neg\}$ is not universal, since $\neg$ is a unary operation that cannot be used to combine logical variables together. So, if we seek a single universal operation, we should start by looking at binary operations.

4.21 EXERCISES

1. Suppose we have the following propositions B, M, S regarding an overseas trip you are doing.

- B : you visit Brazil
- M : you visit Malaysia
- S : you visit Singapore

Use B , M and S to write a Boolean expression which is true if and only if you visit Brazil or both the other countries (but not all three).

2. Suppose we have the following propositions N, P, S about a certain unknown integer.

- N : the integer is negative
- P : the integer is prime
- S : the integer is a square

Use N , P , S to write a Boolean expression with the following meaning:

If the integer is a square, then it is neither negative nor prime.

3. Assuming the first of De Morgan's Laws,

$$\neg(P \vee Q) = \neg P \wedge \neg Q,$$

prove the second of De Morgan's Laws,

$$\neg(P \wedge Q) = \neg P \vee \neg Q,$$

as simply as possible.

4. Prove the two Distributive Laws ((4.2) and (4.3) in § 4.14).

5. Do you need parentheses in the expression $a \wedge b \vee c$? Investigate the two expressions

$$a \wedge (b \vee c) \quad \text{and} \quad (a \wedge b) \vee c.$$

Are they logically equivalent? If not, what can you say about the relationship between them? Does either imply the other?

- 6.

Prove that

$$(P_1 \wedge \cdots \wedge P_n) \Rightarrow C \quad \text{is logically equivalent to} \quad \neg P_1 \vee \cdots \vee \neg P_n \vee C$$

A disjunction of the form $\neg P_1 \vee \cdots \vee \neg P_n \vee C$ is called a *Horn clause*. These play a big role in the theory of logic programming.

7. Simplify the following expression as much as possible, using Boolean algebra:

$$(\neg X \wedge \neg Y) \vee (\neg X \wedge Y) \vee (X \wedge Y).$$

8. This question is about using Boolean algebra to describe the algebra of switching. An electrical switch is represented diagrammatically as follows.

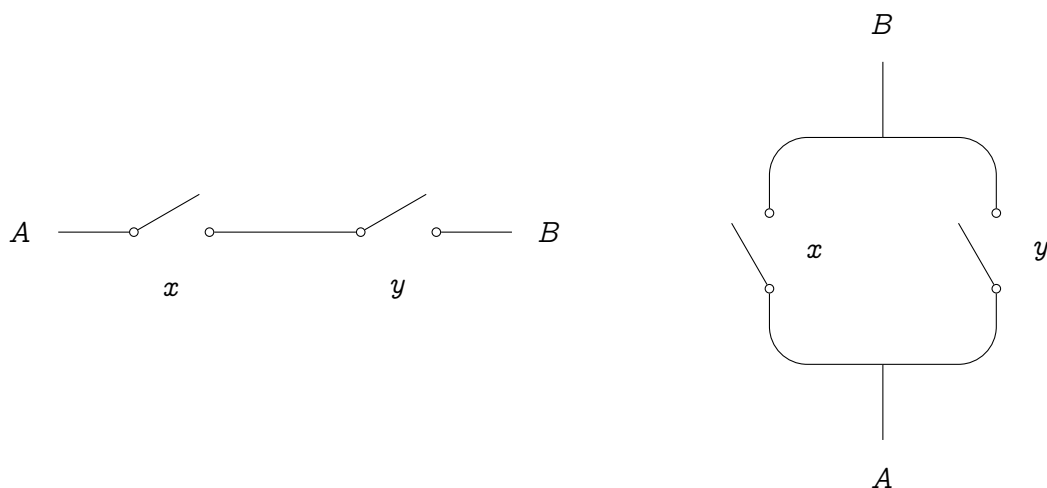


The state of a switch can be represented by a Boolean variable, with the states Off and On being represented by False and True respectively. Let x be a Boolean variable representing the state of a switch. Then $x = \text{True}$ represents the switch being On, so that electrical current can pass through it; $x = \text{False}$ represents the switch being Off, so that there is no electrical current through the switch.

We can put switches together to make more complicated circuits. Those circuits can then be described by Boolean expressions in the variables that represent the switches.

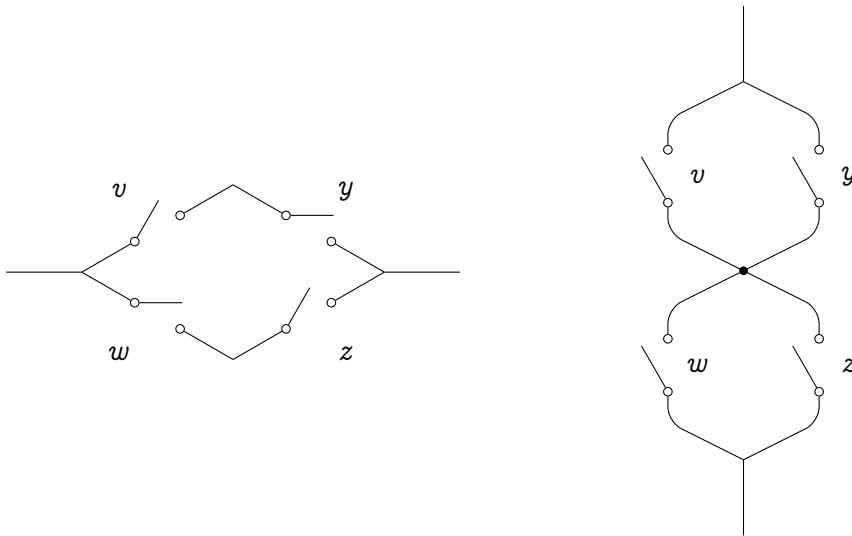
In the following circuits, v, w, x, y, z are Boolean variables representing the indicated switches.

- (a) For each of the two switching circuits below, write a Boolean expression in x and y for the proposition that current flows between the two ends of the circuit. (The ends are shown as A and B in the diagram. In effect, the diagrams only show part of a complete circuit. The rest of the circuit would contain a power supply, such as a battery, and an electrical device that operates when the current flows, such as a light or an appliance.)

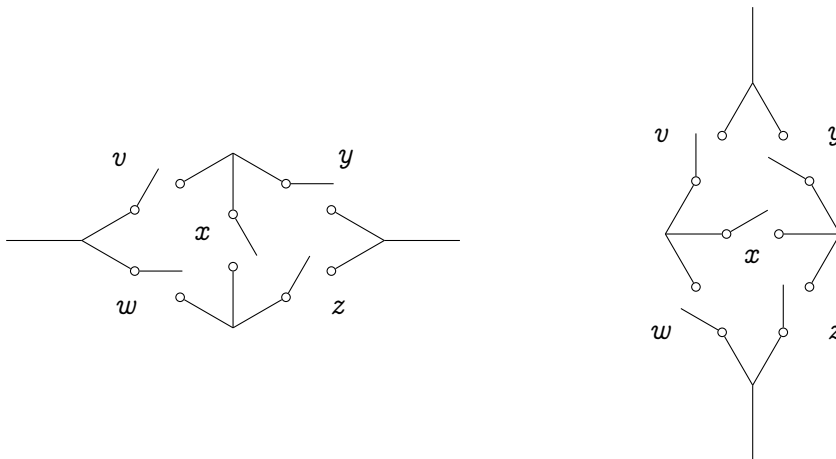


- (b) For each of the next two circuits, again construct a Boolean expression to represent the proposition that current flows between the two ends of the circuit.

Compare the two expressions you construct. What can you say about the relationship between them?



- (c) Similarly, construct Boolean expressions for the next two circuits. Compare the two expressions you construct. What can you say about the relationship between them?



- (d) For each of our circuit pairs in (a)–(c), discuss how they are related to each other, and what this means for how the Boolean expressions derived from them are related to each other.

9. Does logically negating an implication reverse it? In other words, are the expressions

$$\neg(P \Rightarrow Q) \quad \text{and} \quad P \Leftarrow Q$$

logically equivalent? If so, prove it; if not, determine (with proof) if either implies the other.

10. Prove that

$$(P \wedge (P \Rightarrow Q)) \Rightarrow Q$$

is a tautology. Comment on what this means for logical deduction.

11. Prove that

(a) $P \oplus Q = (P \wedge \neg Q) \vee (\neg P \wedge Q).$

(b) $P \oplus Q = (P \vee Q) \wedge (\neg P \vee \neg Q).$

12.

(a) Express $P \vee Q$ using just the operations $\oplus$ and $\wedge$, and no negation.

(b) Express $P \wedge Q$ using just the operations $\oplus$ and $\vee$, and no negation.

13. We saw two distributive laws for $\vee$ and $\wedge$ in § 4.14, and contrasted that with the situation for ordinary numbers under $+$ and $\times$, when only one distributive law holds.

Investigate the situation for $\oplus$ and $\wedge$. Do you have two distributive laws, or one (and which one?), or none? Give explanations for your answers.

14. Consider the following truth table for a proposition P in terms of X , Y and Z .

X	Y	Z	P
F	F	F	F
F	F	T	F
F	T	F	F
F	T	T	T
T	F	F	F
T	F	T	T
T	T	F	T
T	T	T	T

(a) Express P in Disjunctive Normal Form (DNF).

(b) Express $\neg P$ in DNF.

(c) Using (b) as your starting point, express P in Conjunctive Normal Form (CNF).

(d) An alternative way to try to express P in CNF might be to start from the DNF for P you found in (a) and then expand the whole thing using the Distributive Laws. If you start with your expression from (a) and do all this expansion without yet doing any further simplification, how many parts are combined together in the large conjunction

you construct? How many literals does it have? How does this approach compare with (c), with respect to ease and efficiency?

15.

A meeting about moon mission software is held at NASA in 1969. Participants may include Judith Cohen (electrical engineer), Margaret Hamilton (computer scientist), and Katherine Johnson (mathematician). Let Judith, Margaret and Katherine be propositions with the following meanings.

Judith	Judith Cohen is in the meeting.
Margaret	Margaret Hamilton is in the meeting.
Katherine	Katherine Johnson is in the meeting.

For each of the following statements, write a proposition in Conjunctive Normal Form with the same meaning.

- (a) Judith and Margaret are not both in the meeting.
- (b) Either Judith or Margaret, but not both of them, is in the meeting. (This is the “exclusive-OR”.)
- (c) At least one of Judith, Margaret and Katherine is in the meeting.
- (d) At most one of Judith, Margaret and Katherine is in the meeting.
- (e) Exactly one of Judith, Margaret and Katherine is in the meeting.
- (f) At least two of Judith, Margaret and Katherine are in the meeting.
- (g) At most two of Judith, Margaret and Katherine are in the meeting.
- (h) Exactly two of Judith, Margaret and Katherine are in the meeting.
- (i) Exactly three of Judith, Margaret and Katherine are in the meeting.
- (j) None of Judith, Margaret and Katherine is in the meeting.

16.

Recall the logical expression given on p. 141 in § 4.18 for your dinner party guest list:

(Harry $\vee$ Ron $\vee$ Hermione $\vee$ Ginny)

$\wedge (\neg \text{Hagrid} \vee \text{Norberta})$

$\wedge (\neg \text{Fred} \vee \text{George}) \wedge (\text{Fred} \vee \neg \text{George})$

$\wedge (\neg \text{Voldemort} \vee \neg \text{Bellatrix}) \wedge (\neg \text{Voldemort} \vee \neg \text{Dolores}) \wedge (\neg \text{Bellatrix} \vee \neg \text{Dolores})$

How long would an equivalent DNF expression be? Specifically, how many disjuncts — smaller expressions combined using $\vee$ to make the whole expression — would it have?

17. A Boolean function is in **algebraic normal form** if it is an exclusive-or of some number of parts, where each part is a conjunction of unnegated variables or the logical constant True, and we also allow the entire expression to be just the logical constant False (but otherwise False is not allowed to appear in the expression).

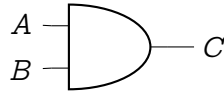


Figure 4.1: An AND gate. A and B are the inputs and C is the output (which is therefore true if and only if both A and B are true).

So, for example, the following expressions are in algebraic normal form:

$$x \oplus y \oplus (x \wedge y), \quad \text{True} \oplus y \oplus (x \wedge y \wedge z), \quad x, \quad \text{True} \oplus x, \quad x \wedge z, \quad \text{True}, \quad \text{False}.$$

But the following expressions are not in algebraic normal form:

$$x \oplus y \oplus (\neg x \wedge y), \quad \text{False} \oplus y \oplus (x \wedge y \wedge z), \quad \neg x, \quad x \vee z, \quad \text{True} \oplus \text{True}.$$

In [Exercise 12\(a\)](#), you expressed the disjunction of two variables in algebraic normal form.

Prove that every Boolean function can be written in algebraic normal form. (Use [Exercise 12\(a\)](#) and what you know about DNF.)

18. Suppose you have an unlimited supply of AND gates, which are logic gates for the binary operation $\wedge$. Each gate has two inputs, for the two arguments, and one output, which gives the conjunction of the inputs. The output of one gate can be used for the input of another gate.

To represent an AND gate in a circuit diagram we will use the symbol in [Figure 4.1](#).

Show how to put AND gates together to compute the conjunction of eight logical arguments $x_1, x_2, \dots, x_8$.

How many AND gates do you need?

Suppose each AND gate computes its output as soon as both its inputs are available, and that it takes time t to compute the output. Assume that all the initial inputs $x_1, x_2, \dots, x_8$ are available simultaneously, at time 0. How long does your combination of AND gates take to compute its output?

Try and put your AND gates together to minimise the total time taken to compute the final output.

19. Prove, by induction on n , that the conjunction of 2^n inputs can be computed in time nt by a suitable combination of AND gates.

Hence prove that the conjunction of m arguments (where m is not necessarily a power of 2) can be computed by a suitable combination of AND gates in time $\lceil \log_2 m \rceil nt$.

20. A Boolean expression is called **affine** if it is of one of the following forms:

$$x_1 \oplus x_2 \oplus \cdots \oplus x_n$$

or

$$x_1 \oplus x_2 \oplus \cdots \oplus x_n \oplus \text{True}.$$

Prove, by induction on n , that for all $n \in \mathbb{N}$, the number of satisfying truth assignments of an affine Boolean expression with n variables is 2^{n-1} .

It follows that half of all truth assignments are satisfying for the expression, and half are not.

Here, a **truth assignment** is just an assignment of a truth value to each variable, and it is **satisfying** if the assignment makes the whole expression True.

21. Let x_1x_0 be the bits of a two-bit binary number x , representing an integer in $\{0, 1, 2, 3\}$. So $x_1, x_0 \in \{0, 1\}$ and we have $x = 2x_1 + x_0$.

Let $y_2y_1y_0$ be the three-bit binary representation of the number $x + 1$ obtained from x by incrementing it. So $y \in \{1, 2, 3, 4\}$, and its leading bit is 0 except if $x = 3$, in which case $y = 4$ which in binary is 100.

In this exercise we use 0 and 1 for the truth values False and True (as mentioned on p. 122 in § 4.1^α).

Give Boolean expressions for each of the three bits of y , in terms of the two bits x_1 and x_0 of x .

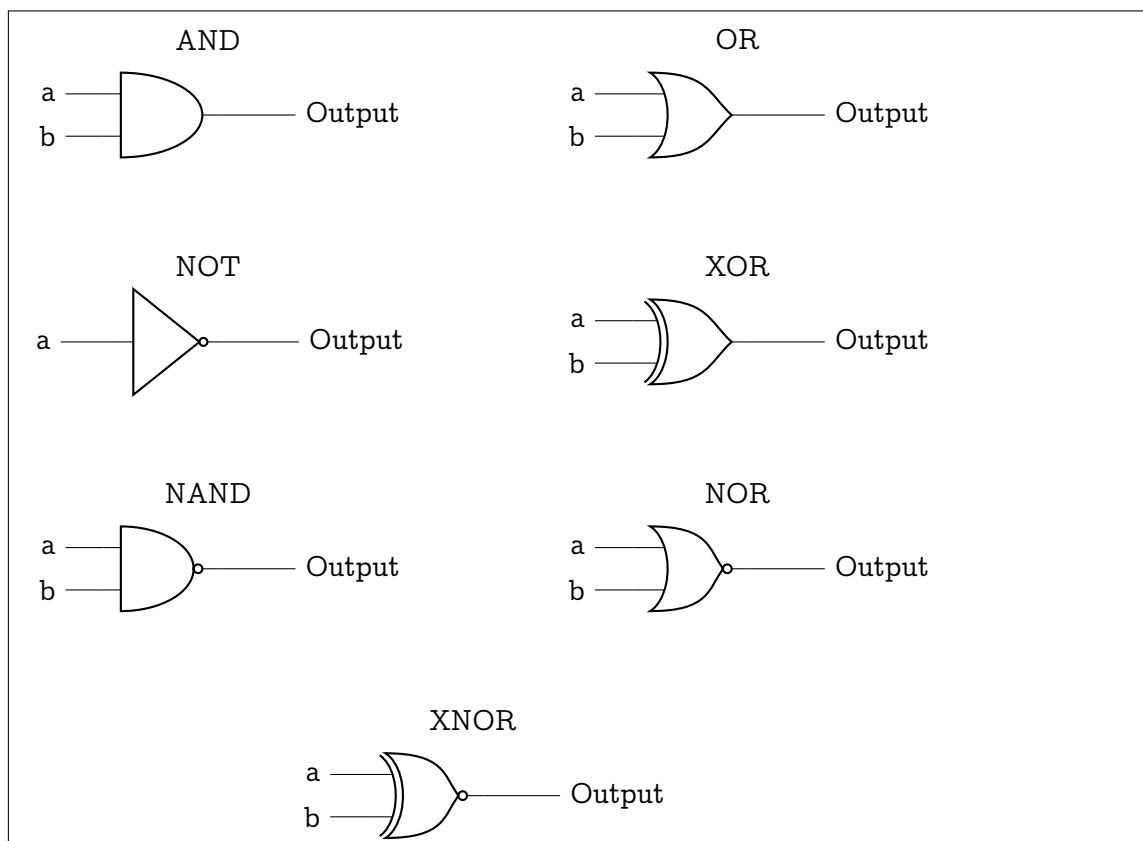
22. Is the operation set $\{\oplus, \neg\}$ universal? Justify your answer.

23. Prove or disprove the claim that there exists a universal operation set containing just a single operation.

You need not restrict consideration to the three operations $\wedge, \vee, \neg$. You can use another operation of at most two arguments (in other words, a binary operation) if you wish.

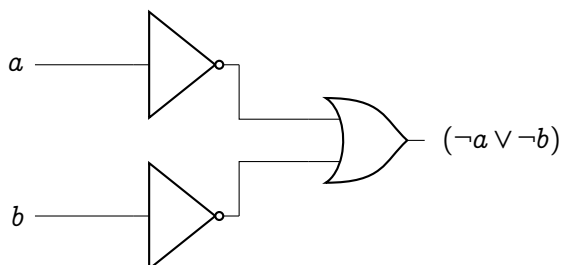
What type of proof did you use? (Recall the proof types discussed in Chapter 3.)

24. Computer circuits actually perform the very same Boolean logic that we have studied using *logic gates* (§ 4.4^α). The idea is that you have one or more wires coming into a gate, and usually one output wire. If an input wire has current flowing through, the variable it represents is set to True, and if not, then False. The logic gate then takes those signals and gives off an output signal if the result should be True, and no signal if the result is False. Below are the most common logic gates seen in circuits.



The AND, OR, and NOT gates function the same as what we have seen previously. If both signals of an AND gate are on, it will output a signal. For OR, only one input signal is required to be on, and for NOT, there will only be an output signal if the input signal is off (False).

XOR is “EXCLUSIVE OR”, which outputs a signal only if a or b are on, but not both. NAND, NOR, and XNOR are simply the inversions of AND, OR, and XOR respectively (i.e. NAND is “not both”, NOR is “neither”, and XNOR is “both or neither”). These gates can also be connected in series, like in the example below, which is equivalent to the logical expression $(\neg a \vee \neg b)$.



(a) What single logic gate is the above circuit equivalent to?

- (b) We saw in § 4.20, that $\{\vee, \wedge, \neg\}$ is a *universal set of operations*, meaning that any logical expression can be expressed with only these operations. How would you express the function of an XOR gate as a Boolean expression using only these operations?
- (c) Draw a circuit diagram which performs the same functionality as an XOR gate using only AND, OR, and NOT gates.
- (d) Do we even need three types of gates? Given that $\{\wedge, \vee, \neg\}$ is a universal set of operations, use De Morgan's Law to prove that $\{\wedge, \neg\}$ and $\{\vee, \neg\}$ are also a universal sets of operations. (*Hint: Can you express $\vee$ with the other two operators?*)
- (e) Draw a circuit diagram that performs an OR operation using only AND and NOT gates, and also one which performs AND using only OR and NOT gates.
- (f) Can you perform the function of a NOT gate, or an AND gate, with only NAND gates? What does this tell you about NAND gates?

PREDICATE LOGIC

Propositional logic, covered in the previous chapter, enables us to do logical reasoning with propositions, in which everything must be either True or False. Any variables we used were Boolean variables, and could only take the values True and False.

But we don't want to *only* do logical reasoning about Boolean variables and expressions. In computer science and mathematics, we want to reason about a huge variety of objects, such as numbers, strings, sets, sequences, networks, files, algorithms, programs, systems, and so on. And we want to be able to reason at a general level, so we can make statements that cover whole categories of objects without having to make separate statements about each and every individual object in the category. So we need *variables*, by which a name can be used to refer to any object of some type, and *predicates* which are a way of making assertions about variables and the objects they can represent. We use *sets* to specify the values that a variable can represent and allow *functions* that convert some objects into other objects. We want to be able to say whether or not a statement about variables is always true, or sometimes true, or never true.

Predicate logic gives us the logical concepts and machinery to do all these things. It is powerful and flexible enough that it can be used to express *any* precise mathematical statement and then do logical reasoning about it. It builds on the foundation of propositional logic but extends the realm of logic much further.

Mastery of predicate logic is not only important for *people* who work with mathematical statements and proofs. It is also used by *computer programs* that do automated reasoning about mathematical statements. As such, it is an important tool in Artificial Intelligence.

5.1^α RELATIONS, PREDICATES, AND TRUTH-VALUED FUNCTIONS

We have already met predicates. We mentioned in § 2.13 and § 2.17 that the term “predicate” is just a synonym for “relation”. Restating this as a definition, we have:

A **predicate** is a relation.

So, when discussing predicates, we can draw on all the terminology and theory that we developed for relations. For example, we can talk about the *arguments* of a predicate,

and the *domain* of each of its arguments. We can call it a *k-ary* predicate if it has *k* arguments.

Although a predicate is nothing more or less than a relation, the term tends to indicate an *intention* to do *logic* with it. So, we are interested in when things are true or false, and we might want to combine things using logical operations.

Consider the relation Parent from § 2.13. This is a subset of $\mathbb{P} \times \mathbb{P}$, where $\mathbb{P}$ is the set of all people who have ever lived. Some ordered pairs of people belong to this relation, others do not.

$$\begin{aligned} (\text{Ada Lovelace}, \text{George Gordon Byron}) &\in \text{Parent}, \\ (\text{George Boole}, \text{George Everest}) &\notin \text{Parent}. \end{aligned}$$

For the pairs that belong to it, we may state that fact using prefix notation:

$$\text{Parent}(\text{Ada Lovelace}, \text{George Gordon Byron}).$$

This is a *true* statement. In logic, we need to be able to work with both true and false statements. So we want to be able to make assertions of both types.

$$\begin{aligned} \text{Parent}(\text{Ada Lovelace}, \text{George Gordon Byron}) &\text{ is True,} \\ \text{Parent}(\text{George Boole}, \text{George Everest}) &\text{ is False.} \end{aligned}$$

In fact, if we give each argument of Parent a specific value from its domain, then we get a specific statement about those values which is either true or false. In other words, we get a proposition.

It is a short step from here to treat Parent as *returning* either True or False, so we can write

$$\begin{aligned} \text{Parent}(\text{Ada Lovelace}, \text{George Gordon Byron}) &= \text{True}, \\ \text{Parent}(\text{George Boole}, \text{George Everest}) &= \text{False}. \end{aligned}$$

In summary, we started with a relation, Parent, treating it as a subset of $\mathbb{P} \times \mathbb{P}$, and then treated it as a function that takes any pair of people and returns either True or False. So, we can view it as a truth-valued function, i.e., a function that returns a truth value.¹ In this case, the domain of the function is $\mathbb{P} \times \mathbb{P}$ and its codomain is $\{\text{True}, \text{False}\}$.

These are just different ways of viewing the same thing. When we use the word “predicate” for a relation, we are not really introducing anything new, but rather just signalling our intention that we want to use the relation in a logical context, and to use it to make logical statements, and maybe to build more complex logical statements from it.

¹ Predicates are occasionally called *propositional functions*, since they yield specific propositions about specific values of the various arguments.

Since predicates enable us to make true or false statements about their arguments, we can combine them using our usual logical operations: $\neg$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$.

statement	truth value
$\neg \text{Parent}(\text{George Boole}, \text{George Everest})$	True
$\text{Parent}(\text{Plato}, \text{Ariston}) \wedge \text{Parent}(\text{Hypatia}, \text{Theon})$	True
$\text{Parent}(\text{Plato}, \text{Ariston}) \wedge \text{Parent}(\text{Ariston}, \text{Plato})$	False
$\text{Parent}(\text{Plato}, \text{Ariston}) \vee \text{Parent}(\text{Ariston}, \text{Plato})$	True

When defining new predicates, it is conventional to use prefix notation, as we have done with `Parent`. But for some predicates based on binary relations, particularly those relating to ordering, containment or equivalence of some kind, infix notation is used. So, when using the predicate `<`, we place the name of the predicate, `<`, in between its arguments. (Recall the discussion of prefix and infix notation on p. 47 in § 2.5^α.)

The equality predicate, `=`, is always considered to be available. This is because you can't really say *anything* about a class of objects if you can't even tell when two objects are really the same object! You do not need to be told that you are allowed to use it; you always can, no matter what type of objects you are working with. It is the only predicate for which we make this sweeping assumption.

A predicate with one argument (i.e., a unary predicate) is also called a **property**. It corresponds naturally to a specific *set*, namely the set of elements of the domain of the argument for which the predicate is True.

For example, define the predicate `isNegative(X)` to be a unary predicate whose variable X has domain $\mathbb{R}$. It captures the *property* of being negative, and corresponds to the *set* $\mathbb{R}^-$ of negative real numbers.

Exercise: what could a predicate with no arguments be?

5.2^α VARIABLES AND CONSTANTS

A **variable** is a name for an object that allows us to refer to the object without specifying it exactly.

- Often, the name is a letter from some alphabet, like x or P or θ . It might also have subscripts, to distinguish it from other variables where we would like to use the same letter; the subscript is considered part of the name, so that x_1 and x_2 are different variables. We can also use words for names, like `myFavouriteColour` or `dateToday`. Using words for names is usual in programming; it is much less common in mathematics.²
- Often, we use a name for something when we don't know it, but still want to reason about it. This was one reason for introducing variables for unknown quantities

² This is for at least a couple of reasons. Using entire words for variable names can tend to lead to expressions that are cumbersome or cluttered, potentially obscuring the mathematical structure. Multi-letter names can sometimes look like products of many variables, due to the mathematical convention of denoting products of variables by juxtaposition (i.e., placing them next to each other).

when you first started doing algebra at school. But this need not be the only reason for introducing variables.

- Variables can be used when we want to talk about *any* member of some set in a general way. If we want to say that the logarithm of a product is the sum of the logarithms, we can let variables x and y represent any positive real numbers and write $\log(xy) = \log x + \log y$.
- Variables in programming and mathematics are similar in nature, but not exactly the same thing. In programming, a variable is associated with a piece of memory in which the object is stored. This does not happen in mathematics. In programming, variables can only refer to finite objects, but there is no such restriction in mathematics.

We have been using variables throughout this unit. We have done so every time we introduce a name for some object we are discussing. We often referred to sets $A, B, C, \dots$, functions $f, g, h, \dots$, and propositions $P, Q, \dots$. In the previous chapter we made extensive use of Boolean variables, which each refer to a truth value, False or True.

Every variable has an associated **domain**, which is the set of objects which that variable might represent. If we say, “Let $x \in \mathbb{Z}$ ”, then x is a variable with domain $\mathbb{Z}$. Every Boolean variable has domain $\{\text{False}, \text{True}\}$, in fact that’s the *definition* of a Boolean variable.

We can think of a variable as being *able* to take **values**, which are just the objects in its domain. But, when we use a variable, we are not implying that it *has* some specific value. If we define a variable x with domain $\mathbb{Z}$, then we are not assuming that x stands for some *specific* integer.

We also use the term **constant** for an object which belongs to the domain of some variable. So, if we have a variable x with domain $\mathbb{Z}$, then any specific integer is a constant. So, our constants include -273 , -1 , 0 , 1 , 8 , 1729 , and of course every other integer.

5.3^α PREDICATES AND VARIABLES

We saw in § 5.1^α that, for any predicate, you can obtain specific propositions by giving specific values to all its arguments.

In order to make more general logical statements, we can also giving *variables* to some, or all, of its arguments. When doing so, we must ensure that the domain of the variable equals the domain of that argument, keeping in mind that every variable has a domain (p. 158) and every argument of every predicate has a domain (p. 75).

For example: if X is a variable with domain $\mathbb{P}$, then we can write

$$\text{Parent}(\text{Alan Turing}, X). \quad (5.1)$$

The variable X in this statement is **free**, meaning that no value is (yet) given to it and it is available for values to be given to it. Because it contains a free variable, this statement does not yet have a truth value, so it is not yet a proposition.

We can, if we wish, assign values to one or more of the free variables in a logical statement. As usual, any value given to a variable must belong to its domain. If *every* free variable has been given a value from its domain, the statement becomes a proposition. Each combination of values you give to the variables in a statement creates a different specific proposition, potentially with different truth values.

In the statement (5.1), we can create specific propositions, with truth values, by giving values from $\mathbb{P}$ to the variable X .

value of X	proposition	truth value
Sara Turing	Parent(Alan Turing, Sara Turing)	True
Julius Turing	Parent(Alan Turing, Julius Turing)	True
Alonzo Church ³	Parent(Alan Turing, Alonzo Church)	False

For another example, consider the binary predicate $<$. We suppose its two arguments each have domain $\mathbb{R}$. This is just the usual $<$ relation on the real numbers. We can create propositions by giving values to all its arguments:

statement	truth value
$5 < 15$	True
$-5 < -15$	False
$\sqrt{2} < 1.618$	True
$\pi < e$	False

Suppose we have real variables x, y (i.e., variables with domain $\mathbb{R}$). If we plug these into the $<$ predicate's two arguments, then we get a logical statement about those variables:

$$x < y.$$

But the two variables remain free so the statement is not a proposition and has no truth value.

We can also use a variable for one argument and a value for the other. For example, using the real variable x and the value -3.7 for the first and second arguments respectively, we obtain the statement

$$x < -3.7.$$

Although it does include a substitution of a value for an argument, it also still has a free variable, so it is not a proposition and does not have a truth value.

For another example, consider the predicate `designedFirstComputerIn`, which has two arguments, the first having domain $\mathbb{P}$ and the second having as its domain the set $\mathbb{A}$ of

³ Alonzo Church (1903–1995) was Alan Turing's PhD supervisor at Princeton, 1936–1938. So he might be described informally as Turing's academic father! But he is not a parent of Turing's in the usual sense, so (Alan Turing, Alonzo Church) $\notin$ Parent.

all countries. As a relation, it consists of all pairs (p, c) such that person p designed the first computer that was designed and built in country c . We can create some specific propositions by assigning values to these arguments:

designedFirstComputerIn(Trevor Pearcey, Australia)	True
designedFirstComputerIn(Maston Beard, Australia)	True
designedFirstComputerIn(Xià Péisù, China)	True
designedFirstComputerIn(Blaise Pascal, France)	False
$\vdots$	$\vdots$

If variable P has domain $\mathbb{P}$ and variable Q has domain $\mathbb{A}$, we can make statements like

statement	free variables
designedFirstComputerIn(P, Q)	P, Q
designedFirstComputerIn(Trevor Pearcey, C)	C
designedFirstComputerIn(P , France)	P

But, in this case, it would be an error to write $\text{designedFirstComputerIn}(Q, P)$, with P and Q the other way round, because the domains do not match: variable P , with domain $\mathbb{P}$, is given to the second argument, which has domain $\mathbb{A}$ (and a similar mis-match for variable Q in the first argument).

5.4 ARGUMENTS OF PREDICATES

When *using* a predicate, we need to put something into each of its arguments.

We have seen in § 5.1^α and § 5.3^α that we can put constants and/or variables (or a mix of these) into the arguments of a predicate, provided the domains match appropriately.

But we also want to be able to put more complex expressions into the arguments. For example, we may want to make statements like

$$a^2 + b^2 < c^2$$

$$e^{i\pi} + 1 = 0$$

Parent(Caroline Herschel, MotherOf(William Herschel))
 knows(X, MotherOf(FatherOf(Y))).

To do this, the expressions we use for predicate arguments need to be able to use functions as well as constants and variables. But which functions can we use? Usually, we will work in settings where the available functions are specified up-front. But we still need to ensure that the expressions that use functions are properly constructed.

Informally, the expressions that can be put into an argument of a predicate can be any expression that makes sense in the domain of that argument and which only uses functions that are known to be available.

For example, if we are dealing with the predicate $<$ with real arguments, then the first statement above, $a^2 + b^2 < c^2$, is ok provided that

- the squaring operation is taken to be a function from $\mathbb{R}$ to $\mathbb{R}$, so that its codomain matches the domain of the second argument of $<$;

- the variables a, b, c each have domain $\mathbb{R}$, which ensures they can be used as arguments for the real squaring function;
- the addition operation, $+$, is taken to be a function from $\mathbb{R} \times \mathbb{R}$ to $\mathbb{R}$. Again, this ensures that its codomain matches the domain of the first argument of $<$. Its domain being $\mathbb{R} \times \mathbb{R}$ also ensures that we can plug a^2 and b^2 into its arguments, assuming again that we are using the squaring function on the reals.

The last example above, $\text{knows}(X, \text{MotherOf}(\text{FatherOf}(Y)))$, is ok provided

- variable X has domain $\mathbb{P}$ (the set of all people who have ever lived), so that its domain matches the domain of the first argument of knows ;
- the functions MotherOf and FatherOf each have $\mathbb{P}$ as their domain and codomain, to ensure that the composition $\text{MotherOf} \circ \text{FatherOf}$ is defined, and to ensure that the composition's codomain matches the domain of the second argument of knows (which it gets pluggged into);
- variable Y also has domain $\mathbb{P}$, to match the domain of the function FatherOf , since Y is plugged into the argument of the FatherOf .

In general, apart from constants and variables from appropriate domains, the other things we can put into a predicate's arguments are all expressions that use a function. In fact, such an expression might contain several uses of functions, like in $a^2 + b^2$ or $\text{MotherOf}(\text{FatherOf}(Y))$. But there will always be one function in the expression which was *applied last*, i.e., which we think of as producing something to be given to the argument of the predicate.

- In the expression $a^2 + b^2$, the addition function is applied last, *after* both the squarings have been done. We think of the addition as producing something which goes into the first argument of the predicate $<$.
- In the expression $\text{MotherOf}(\text{FatherOf}(Y))$, the function MotherOf is applied last, and we think of it as producing a person (in fact, a mother) who is then put into the second argument of the predicate knows .

But, as we saw from our examples above, it is not enough that the function's codomain matches the domain of the predicate's argument. The function also has arguments, each of which also has a domain, and anything put there must match the domain of the argument where it is put. What can we put into a function's arguments? The same rules apply here as apply to arguments of predicates. The things we can put into a function's arguments are constants from that argument's domain, variables with the same domain, and (again) functions whose codomain equals the domain of the argument where it is put. And those functions, too, have arguments, and the same rule applies to them! And so on, and so on, ...although we are not allowed to go on forever and construct infinite expressions! So, following these rules, we can construct, from constants, variables and

functions, any expression that is allowed to be put into the argument of any other function, and ultimately, any expression that is allowed to be put into the argument of a predicate.

Expressions that are allowed to be put into the arguments of predicates can be defined more formally, and we do so now for completeness. Such an expression is called a *term*.

We suppose, at the outset, that we have the following ingredients:

- a set of variables, each with an associated domain;
- a set of functions.

A **term** of **domain** D consists of any of the following:

- a constant from D ;
- a variable with domain D
- a function with codomain D , in which each argument has a term whose domain is the same as the domain of the argument.

We can illustrate this definition using a couple of our recent examples.

- The expression $a^2 + b^2$ is a term with domain $\mathbb{R}$ because it consists of a function, namely real addition, with codomain $\mathbb{R}$, whose two arguments (each with domain $\mathbb{R}$) have each been given a term with domain $\mathbb{R}$. The two terms given to the two arguments of $+$ are a^2 and b^2 , which are each formed from the squaring function with codomain $\mathbb{R}$ whose domain has been given a real variable, a or b .
- The expression $\text{MotherOf}(\text{FatherOf}(Y))$ is a term of domain $\mathbb{P}$ because it is formed from a function, MotherOf , whose codomain is $\mathbb{P}$, and whose argument (with domain $\mathbb{P}$) has been given a term of the same domain, $\text{FatherOf}(Y)$. And that, in turn, is a term of domain $\mathbb{P}$ because it has codomain $\mathbb{P}$ and its argument has the same domain as the variable it has been given.

If we are working with real numbers and we have functions $+, -, \times, /$, $\sqrt{}$ and real variables x and y , then some examples of terms are:

$$-5, \quad 22/7, \quad y, \quad 3 \times x + 1, \quad \sqrt{x \times x - 3 \times 3}, \quad 2 \times \pi \times x$$

But not everything is a term:

- The following six “expressions” are not terms because they have been incorrectly formed (either by breaking the rules for how to use the provided functions, or using objects, variables or functions that are not available):

$$2 + 3 +, \quad x + +, \quad \log x, \quad z, \quad e^{i\pi}, \quad x \wedge y$$

- In this scenario, character constants are not considered, so the expression $w+16$ is not a term. (Here, the letter w is not being used as a variable name, but is “just itself”, i.e., the 23rd letter of the alphabet and nothing more).

We will not usually fuss over these details of the construction of expressions we use inside predicates. Mostly, we will draw on our prior experience of what constitutes a valid expression using standard operations in domains we are used to. We know, for example, how to construct a valid expression involving constants, variables and functions involving numbers, and we know how to compose functions. But it is important to understand that expressions are constructed according to formal rules, and also that, in any given situation, we can only use the functions that are available to us, and usually these will be clearly specified.

Now that we know the kinds of expressions we can put into the arguments of predicates, we can recognise many instances of this in earlier chapters. For example, [Theorem 1](#) stated that $\overline{A \cup B} = \overline{A} \cap \overline{B}$. This uses the equality predicate for sets, with first argument $\overline{A \cup B}$ and second argument $\overline{A} \cap \overline{B}$. Each of these is an expression in sets, using the set functions complement, union and intersection and set variables A and B .

5.5 BUILDING LOGICAL EXPRESSIONS WITH PREDICATES

Once we have predicates whose arguments are terms with appropriate domains, we can combine them using all the Boolean operations (logical connectives) we studied in [Chapter 4](#). So we can construct expressions like those in the left column of the following table (with free variables listed in the right column).

logical expression	free variables
$(1 < i) \wedge (i < n)$	i, n
$\left(\frac{1}{3} < \frac{1+2}{3+5}\right) \wedge \left(\frac{1+2}{3+5} < \frac{2}{5}\right)$	none
$((x > 0) \wedge (y > 0)) \Rightarrow (\log(xy) = \log x + \log y)$	x, y
$\text{knows}(P, Q) \wedge \text{knows}(Q, R) \wedge \neg \text{knows}(P, R)$	P, Q, R
$\text{Parent}(X, Y) \Leftrightarrow (Y = \text{MotherOf}(X)) \vee (Y = \text{FatherOf}(X))$	X, Y

Formation of **predicate logic expressions** is done according to the following rules, noting that this list of rules is incomplete and we will meet more later (in [§ 5.11](#)).

- The truth values True and False are predicate logic expressions.
- If terms of the required domains are plugged into the arguments of a predicate, then the result is a predicate logic expression.
- If E and F is a predicate logic expression, then each of the following is a predicate logic expression:

$$(E), \quad \neg E, \quad E \wedge F, \quad E \vee F, \quad E \Rightarrow F, \quad E \Leftrightarrow F.$$

We can manipulate predicate logic expressions using the usual rules of Boolean algebra. This extends the realm of those rules. They were introduced in [Chapter 4](#) in the context of propositions, but now we are using them for predicate expressions, and these may have variables and therefore might not be propositions. But every assignment of values to all the variables makes a predicate expression True or False, so on that basis we can combine these expressions using logical operations just as we combined propositions using logical operations in [Chapter 4](#).

For example, consider again the first expression in the above table,

$$(1 < i) \wedge (i < n),$$

which is really just a more detailed way of writing $1 < i < n$ (in fact, it makes clear the exact logical meaning of that chain of two inequalities). If we want to say that the expression is *not* satisfied, then we can negate it, using De Morgan's Law:

$$\neg((1 < i) \wedge (i < n)) = \neg(1 < i) \vee \neg(i < n),$$

which accords with our understanding that if i does not lie between 1 and n then it must be ≤ 1 or $\geq n$.

5.6 EXISTENTIAL QUANTIFIER

At the start of this chapter, we mentioned that predicate logic enables us to make statements about variables that are always true, or sometimes true, or never true. We can do this with *quantifiers*. We now look at the one of these, which covers “sometimes true”.

The **existential quantifier** is written $\exists$ and read as “there exists”. It is placed before a variable to mean that there exists some value of that variable, within the variable's domain, that makes the subsequent statement True.

For example, consider the statement

There's a fly in my soup.

This may be written

$$\exists X : (X \text{ is a fly}) \wedge (X \text{ is in my soup}).$$

Suppose we have two unary predicates, Fly and InMySoup. For each of these, suppose that the domain of its sole argument is the set of everything on Earth, and that the variable X has this domain too. So $\text{Fly}(X)$ is a predicate logic expression meaning that “ X is a fly”, and $\text{InMySoup}(X)$ is a predicate logic expression meaning that “ X is in my soup”.

The conjunction of any two predicate logic expressions is another predicate logic expression (§ 5.5). So

$$\text{Fly}(X) \wedge \text{InMySoup}(X)$$

is another predicate logic expression, meaning that “ X is a fly and X is in my soup”. For any specific object on Earth, plugging it into X in this expression gives a specific proposition, which may be True or False. In this sense, this predicate logic expression may be viewed as representing many possible statements, one for each object on Earth.

Putting $\exists X$ in front of (i.e., at the very left of) this expression gives the statement

$$\exists X : \text{Fly}(X) \wedge \text{InMySoup}(X). \quad (5.2)$$

This asserts that

There exists X such that X is a fly and X is in my soup.

This is just a rewording of the statement we started with. And note that it is just *one single statement*, rather than representing many possible statements.

The colon after “ $\exists X$ ” in (5.2) is merely punctuation. It is usually read as “such that”, and provides convenient visual separation between “ $\exists X$ ” and the condition that X must satisfy. But it is fine to omit the colon, if the expression is still clear. Sometimes a full stop is used instead of the colon. So either of the following is also correct:

$$\begin{aligned} \exists X \text{ Fly}(X) \wedge \text{InMySoup}(X), \\ \exists X. \text{Fly}(X) \wedge \text{InMySoup}(X). \end{aligned}$$

The domain of each variable is often clear from the context. Alternatively, it might be specified as part of the written expression by specifying domain membership immediately after the variable being quantified, as in, $\exists Y \in \mathbb{Q} \dots$ (so in this case the domain of Y is $\mathbb{Q}$). The domain of a variable certainly affects the meaning of the expression it is part of, in general; changing the domain might change the truth value of the expression.

For example, consider the following expression, written as text on the left and as a logical expression on the right.

$$\text{There exists } W : W \text{ is negative.} \quad \exists W : W < 0.$$

If domain of W is $\mathbb{N}$, then we could write the expression as $\exists W \in \mathbb{N} : W < 0$, and it is False.

If domain of W is $\mathbb{Z}$, then we could write the expression as $\exists W \in \mathbb{Z} : W < 0$, and it is True.

There is an analogy between the existential quantifier and disjunction. In each case, the expression that uses them is True if and only if at least one of its “possibilities” is True. For a disjunction, we require that at least one of the parts of the disjunction is True; for an existential quantifier applied to some variable X , we require that at least

one value of X makes the entire expression True. In other words, at least one member of the domain of X may be assigned to X to make the expression True.

For example, suppose we have the statement “Someone did it”. We may write this as

$$\exists X : X \text{ did it.} \quad (5.3)$$

Suppose the domain of X is a large set of people,

$$\{\dots, \text{Annie, Edward, Henrietta, Radhanath, } \dots\}.$$

Then the quantified expression (5.3) is *sort-of* like a disjunction ...

$$\dots \vee (\text{Annie did it}) \vee (\text{Edward did it}) \vee (\text{Henrietta did it}) \vee (\text{Radhanath did it}) \vee \dots$$

However, the existential quantifier is not just a shorthand notation for disjunction. Firstly, if the domain of a variable is infinite, then existential quantification over that variable cannot be replaced by a disjunction because a disjunction is only allowed to have finitely many parts (and, indeed, logical expressions in general must be of finite size). Secondly, variables and their quantifiers allow us to do some reasoning that cannot be done in propositional logic.

Once a variable in an expression has had a quantifier applied to it, so that all occurrences of the variable come after the quantifier and are subject to it, the variable is said to be **bound**. You can no longer give specific values to the variable. So you can no longer create specific propositions by giving specific values to every free variable.

So, for example, in our statement “there’s a fly in my soup”, formalised in (5.2), there is *no free variable*. The variable X is bound. We now have one specific proposition, which has a specific truth value. The variable X has not been given any value, but that does not mean it is available to have values plugged into it. Now that X is bound, it is no longer free, and no longer available to receive values.

Normally, we put a quantifier in front of an expression containing the quantified variable, as in the example

$$\exists w : w < 0.$$

It’s also legal to put a quantifier, with its variable, in front of an expression that does not contain that variable. Here are some examples of this.

$$\exists w : z < 0, \quad \exists w : 1 + 1 = 2, \quad \exists w : 1 + 1 = 3.$$

In each case, the quantifier may as well not be there. The variable w is irrelevant to the truth of $z < 0$, so whether this first statement is true or not depends solely on the truth or otherwise of $z < 0$. In the second and third examples, w is also irrelevant, and in those cases there are no other variables so we can discard the quantifiers and conclude

that the second statement is True and the third is False. So these three statements are equivalent to

$$z < 0, \quad 1 + 1 = 2, \quad 1 + 1 = 3,$$

respectively. For another couple of examples, where the expression after $\exists x$ is just a logical constant:

$$\begin{aligned} \exists x \text{ True} & \text{ is equivalent to } \text{True}, \\ \exists x \text{ False} & \text{ is equivalent to } \text{False}. \end{aligned}$$

Note that quantifiers can only be used with *variables*. Using them with constant objects *makes no sense*. It is an error to write something like $\exists 5$, $\exists \text{Annie}$.

5.7 RESTRICTING EXISTENTIALLY QUANTIFIED VARIABLES

It is useful to consider how to incorporate a restriction on an existentially-quantified variable into a predicate logic statement.

Suppose we have the statement

$$\text{Some computer is human.} \quad \text{i.e.,} \quad \text{There exists a human computer.}$$

To begin with, suppose the domain of X is $\{\text{computers}\}$, and that we have the predicate $\text{human}(X)$ which is intended to mean that X is human.

Then our statement may be written

$$\exists X : \text{human}(X).$$

But what if the domain of X is $\{\text{everything on Earth}\}$?

In this case, suppose we have another predicate, $\text{computer}(X)$, which means that X is a computer.

The following table shows the correct expression on the left and an incorrect attempt at the expression on the right, together with different ways of writing each in words.

<i>Correct:</i> $\exists X : \text{computer}(X) \wedge \text{human}(X)$	<i>Incorrect:</i> $\exists X : \text{computer}(X) \Rightarrow \text{human}(X)$
• “There exists something that is both computer and human.” • “There exists a human computer.” • “Some computer is human.”	• “There exists something which is not a computer or is human.” • “There exists something which is not both a computer and non-human.” • “Not everything is a nonhuman computer.”

The general principle at work here is as follows. Let P be a unary predicate whose sole argument has domain D , and let X be a variable with the same domain, D . The existential statement

$$\exists X : P(X)$$

says that there is at least one $X \in D$ for which $P(X)$ holds. But suppose we want to make this assertion only for those X that also satisfy $R(X)$ (for some other predicate R , and with X still having domain D). In other words, we want to say that there is at least one X satisfying $R(X)$ that also satisfies $P(X)$. Then we can do this with the statement

$$\exists X : R(X) \wedge P(X).$$

So, the restriction to those X that satisfy $R(X)$ is done by putting $R(X)$ in conjunction with $P(X)$.

5.8 UNIVERSAL QUANTIFIER

Now we consider how to assert that a statements about variables is “always true”.

The **universal quantifier** is written $\forall$ and read as “for all” (or “for every” or “for each”⁴). It is placed before a variable to mean that for all values of that variable, within the variable’s domain, the subsequent statement is True.

For example, consider the statement

Everyone can pass. *i.e.*, For every X : X can pass.

This may be written

$$\forall X : \text{canPass}(X),$$

using `canPass` as a name for the unary predicate we are using. This statement happens to be True, provided the domain of X is the set of students who have the required prerequisite knowledge and have been allowed to enrol in the subject.

Again, the colon serves as convenient punctuation, but is not compulsory. It can be omitted, or sometimes a full stop is used instead. So each of the following is also allowed.

$$\begin{aligned} \forall X \text{ canPass}(X), \\ \forall X. \text{ canPass}(X). \end{aligned}$$

This time, we don’t read out the colon as “such that”, since that wording does not read naturally in this context.

Here is a famous statement, written in three equivalent ways, with the third using `isInteresting` as a name for the predicate we are using:

All numbers are interesting. $\forall X : X \text{ is interesting.}$ $\forall X : \text{isInteresting}(X).$

This statement is also True, and we proved it in [Theorem 16!](#)

⁴ but we do *not* read it as “for some”, because that means the same as “there exists”.

The following statement is False, whether the domain of W is $\mathbb{N}$ or $\mathbb{Z}$ or $\mathbb{R}$.

For all W : W is negative $\forall W : W < 0$.

Now that we’ve applied quantifiers to the variables in these expressions, the variables are all **bound**. Again, once a variable is bound, we can no longer assign values to it.

As with the existential quantifier, if we use the universal quantifier with a variable and then follow it with an expression that does not include that variable, then the quantification is irrelevant and can be dropped. It’s like saying, “for every dog, n is prime”, which is equivalent to just saying that “ n is prime” since the primality or otherwise of n has nothing to do with dogs. So the statements

$$\forall w : z < 0, \qquad \forall w : 1 + 1 = 2, \qquad \forall w : 1 + 1 = 3.$$

are equivalent to

$$z < 0, \qquad 1 + 1 = 2, \qquad 1 + 1 = 3,$$

respectively.

5.9 RESTRICTING UNIVERSALLY QUANTIFIED VARIABLES

We now consider how to incorporate a restriction on a universally-quantified variable into a predicate logic statement.

Again, we use our statement that “Every computer is human” and suppose we have the predicate $\text{human}(X)$ with the same meaning as before, in § 5.7.

If the domain of X is $\{\text{computers}\}$, then our statement may be written

$$\forall X : \text{human}(X).$$

But what if the domain of X is $\{\text{everything on Earth}\}$?

As before (§ 5.7), suppose we have the predicate $\text{computer}(X)$.

The following table shows the correct expression on the *right* (*not* the left, this time) and an incorrect attempt at the expression on the *left*. Again, we give a few different ways of writing each in words.

<i>Incorrect:</i>	<i>Correct:</i>
$\forall X : \text{computer}(X) \wedge \text{human}(X)$	$\forall X : \text{computer}(X) \Rightarrow \text{human}(X)$
• “Everything is both computer and human.” • “Everything is a human computer.”	• “For everything, if it’s a computer, then it’s human.” • “Everything that’s a computer is also human.” • “Every computer is human.”

The general principle is as follows. Let P be a unary predicate whose sole argument has domain D , and let X be a variable with the same domain, D . The universal statement

$$\forall X : P(X)$$

says that every $X \in D$ satisfies $P(X)$. But suppose we want to make this assertion only for those X that also satisfy $R(X)$ (for some other predicate R , and with X still having domain D). In other words, we want to say that every X satisfying $R(X)$ also satisfies $P(X)$. Then we can do this with the statement

$$\forall X : R(X) \Rightarrow P(X).$$

So, the restriction to those X that satisfy $R(X)$ is done by making $R(X)$ imply $P(X)$ in the expression after the quantified variable.

5.10 MULTIPLE QUANTIFIERS

It is permissible to have multiple quantifiers in an expression.

If there are two consecutive quantifiers at the start of an expression, then they must have different variables. (It makes no sense to write something like $\exists X \forall X \dots$.)

We illustrate how quantifiers work together using the binary predicate `hasVisited`, defined as follows. Its first argument can be any person and its second argument can be any country. If P is a person and C is a country, then `hasVisited( $P, C$ )` means that person P has visited country C .

Consider the statement

“Someone has visited every country”,

which may be written using our predicate as

$$\exists P \forall C \text{ hasVisited}(P, C). \quad (5.4)$$

After the first quantifier and its variable, $\exists P$, we have the expression

$$\forall C \text{ hasVisited}(P, C) \quad (5.5)$$

which asserts that P has visited every country. Two variables — P and C — appear in (5.5), but they play different roles, since P is free in (5.5) but C is bound by the $\forall C$ at the start. So the expression (5.5) has only one free variable. The presence of this free variable still prevents this expression (5.5) from being a proposition. In fact, we could use it as the definition of a new unary predicate, `hasVisitedEveryCountry`: if P is any person, then `hasVisitedEveryCountry( $P$ )` means that P has indeed visited every country:

$$\text{hasVisitedEveryCountry}(P) \quad \text{if and only if} \quad \forall C \text{ hasVisited}(P, C).$$

Then, we put $\exists P$ in front of (5.5) to make (5.4). This means P is now quantified and therefore bound, so the full expression in (5.4) has no free variable and becomes a proposition. That full expression has the same meaning as

$$\exists P \text{ hasVisitedEveryCountry}(P).$$

We summarise these remarks as follows.

$$\begin{array}{c} \text{hasVisitedEveryCountry}(P) \\ \hline \exists P \quad \forall C \quad \text{hasVisited}(P, C) \\ \hline \quad \quad \quad P, C \text{ both free} \\ \hline \quad \quad \quad P \text{ free, } C \text{ bound} \\ \hline \quad \quad \quad P, C \text{ both bound} \end{array}$$

The following table shows how we might represent various statements involving the predicate `hasVisited`, including the one we have just discussed in detail, with the statements on the left and the predicate logic expressions on the right.

Someone has visited some country.	$\exists P \exists C : \text{hasVisited}(P, C).$
Everyone has visited every country.	$\forall P \forall C : \text{hasVisited}(P, C).$
Someone has visited every country.	$\exists P \forall C : \text{hasVisited}(P, C).$
Everyone has visited some country.	$\forall P \exists C : \text{hasVisited}(P, C).$
Some country has been visited by everyone.	$\exists C \forall P : \text{hasVisited}(P, C).$
Every country has been visited by someone.	$\forall C \exists P : \text{hasVisited}(P, C).$

You should think about each of these examples carefully. Think about what they each mean and how they differ from each other. Some are obviously true, some are obviously false, some seem likely to be true although you may not know that for a fact.

One important issue to think about is whether the order of the quantifiers matters.

- We have given one example with two existential quantifiers, and one with two universal quantifiers. In each case, we could try putting them the other way round, e.g., we could try $\exists Y \exists X \dots$ instead of $\exists X \exists Y \dots$. Would that give us a new statement, or is it just another way of saying the same thing?
- We have given four examples that use a mix of our two quantifier types. Are they all logically different, or are some of them equivalent?

When you have two variables in successive identical quantifiers, i.e., $\exists X \exists Y \dots$ or $\forall X \forall Y \dots$, they can be replaced by a *single* quantifier of the same type, applied to a *pair* of variables. So $\exists X \exists Y \dots$ can be replaced by $\exists(X, Y) \dots$, which is sometimes written $\exists X, Y \dots$, and $\forall X \forall Y \dots$ can be replaced by $\forall(X, Y) \dots$, which is sometimes written $\forall X, Y \dots$. So, in the above example, $\exists P \exists C : \text{hasVisited}(P, C)$ can be rewritten as $\exists(P, C) : \text{hasVisited}(P, C)$, and so on.

For two consecutive uses of the *same* quantifier, it also does not matter which way round we put the quantified variables at the start. So, $\exists X \exists Y \dots$ and $\exists Y \exists X \dots$ amount to the same thing, and both are the same as $\exists(X, Y) \dots$, and $\exists X, Y \dots$, and $\exists Y, X \dots$. Similarly, $\forall X \forall Y \dots$ and $\forall Y \forall X \dots$ are equivalent, and both are the same as $\forall(X, Y) \dots$, and $\forall X, Y \dots$, and $\forall Y, X \dots$.

But the situation is different when the quantifiers are different. In fact, all the four possibilities

$$\begin{aligned} \exists X \forall Y \dots \\ \forall X \exists Y \dots \\ \exists Y \forall X \dots \\ \forall Y \exists X \dots \end{aligned}$$

are different in meaning, in general.

5.11 PREDICATE LOGIC EXPRESSIONS

Now that we have quantifiers, we are at last able to give a complete definition of predicate logic expressions, extending and finishing the work we began on p. 163 in § 5.5.

A **predicate logic expression** is any of the following.

- The truth values True and False are predicate logic expressions.
- If terms of the required domains are plugged into the arguments of a predicate, then the result is a predicate logic expression.
- If E and F is a predicate logic expression, then each of the following is a predicate logic expression:

$$(E), \quad \neg E, \quad E \wedge F, \quad E \vee F, \quad E \Rightarrow F, \quad E \Leftrightarrow F.$$

- If E is a predicate logic expression and X is a free variable of E , then each of the following is a predicate logic expression:

$$\begin{aligned} \exists X : E \\ \forall X : E \end{aligned}$$

But, in each case, X is no longer a free variable in the new expression. So, if V is the set of free variables of the expression E , then the set of free variables of $\exists X : E$, and the set of free variables of $\forall X : E$, are each $V \setminus \{X\}$.

5.12 DOING LOGIC WITH QUANTIFIERS

All the rules of Boolean algebra (§ 4.15) are available to us when working with predicate logic. We discussed this at the end of § 5.5.

There are also some other rules for doing logic involving quantifiers.

Let $P(X)$ be a predicate logic expression with a free variable X . If we know that

$$\forall X P(X)$$

and **obj** is any specific object (in the domain of X), then we can deduce that

$$P(\text{obj}).$$

In other words, if $P(X)$ is True for all X , then it's certainly true for any specific value from the domain of X :

$$(\forall X P(X)) \implies P(\text{obj})$$

In similar vein, if it's True for a specific value from the domain of X , then it's certainly True for *some* X .

$$P(\text{obj}) \implies (\exists X P(X)) \quad (5.6)$$

Universal quantifiers and conjunction mix in a natural way. For any predicates P and Q ,

$$\forall X (P(X) \wedge Q(X)) \text{ is logically equivalent to } (\forall X P(X)) \wedge (\forall X Q(X)) \quad (5.7)$$

Similarly, existential quantifiers and disjunction mix naturally too.

$$\exists X (P(X) \vee Q(X)) \text{ is logically equivalent to } (\exists X P(X)) \vee (\exists X Q(X)) \quad (5.8)$$

The expressions on the right, in (5.7) and (5.8), raise the important issue of the *scope* of variables. These expressions each contain two separate quantifications over X . Specifically, (5.7) contains $\forall X$ twice, and (5.8) contains $\exists X$ twice. To which parts of the entire expression do each of these quantifiers apply? Does the first $\forall X$ in (5.7) apply to every appearance of X in the rest of the expression? If so, how does that first $\forall X$ interact with the second $\forall X$? If not, how do we know that?

The **scope** of a quantified variable in a predicate logic expression is the portion of the expression (i.e., the “sub-expression”) in which that variable has meaning, and goes from its quantifier to *either*

- the end of the innermost pair of enclosing parentheses, if such a pair of parentheses exists, *or*
- the end of the entire expression, if there is no enclosing pair of parentheses.

So, in the left expression in (5.7), the scope of X is the entire expression, since $\forall X$ is not enclosed by any parentheses. (There are parentheses in the expression, but they do

not enclose $\forall X$ so they have no bearing on the scope of X .) But in the right expression in (5.7), we actually have *two variables with separate scopes*. The first $\forall X$ is enclosed in a pair of parentheses, and its scope is sub-expression $\forall X P(X)$ on the left of $\wedge$. The second $\forall X$ is enclosed in a different, and completely separate, pair of parentheses, and its scope is the sub-expression $\forall X Q(X)$ on the right of $\wedge$. These two scopes do not overlap; there is no appearance of X that belongs to both scopes, so there is no ambiguity over which quantifier governs each appearance of X . In effect, the two appearances of X , each with its own scope separate from the other's scope, are *local* to those scopes. It is up to the reader to see that these two variables are different, even though they have the same name, and keep track of their different scopes. Such variables in predicate logic are like *local variables* in programs, which many programming languages provide for, including Python.

Summarising for the examples in (5.7) and (5.8), we have

$$\begin{array}{ccc} \underbrace{\forall X (P(X) \wedge Q(X))}_{\text{scope of } X} & & (\underbrace{\forall X P(X)}_{\text{scope of } \underline{\text{first}} X}) \wedge (\underbrace{\forall X Q(X)}_{\text{scope of } \underline{\text{second}} X}) \\ \underbrace{\exists X (P(X) \vee Q(X))}_{\text{scope of } X} & & (\underbrace{\exists X P(X)}_{\text{scope of } \underline{\text{first}} X}) \vee (\underbrace{\exists X Q(X)}_{\text{scope of } \underline{\text{second}} X}) \end{array}$$

When a variable is bound, we can change its name, provided we do so to every appearance of that variable within its scope, and to no other variable of the same name. For example, consider the first X in

$$(\forall X P(X)) \wedge (\forall X Q(X)).$$

If we change the first X to W , taking care to do it throughout the scope of that first X and nowhere else, then we get the equivalent expression

$$(\forall W P(W)) \wedge (\forall X Q(X)).$$

So we can say that

$$\forall X (P(X) \wedge Q(X)) \quad \text{is logically equivalent to} \quad (\forall W P(W)) \wedge (\forall X Q(X)).$$

Having discussed how $\forall$ mixes well with $\wedge$, and how $\exists$ mixes well with $\vee$, it is natural to ask: how well do the other pairings mix? How does $\forall$ mix with $\vee$? How does $\exists$ mix with $\wedge$?

In detail, what can we say about the logical relationship between ...

$$\forall X (P(X) \vee Q(X)) \quad \text{and} \quad (\forall X P(X)) \vee (\forall X Q(X))$$

...? Are they equivalent, or is there an implication in one direction, or is there no direct logical relationship?

Similarly, what is the logical relationship between ...

$$\exists X (P(X) \wedge Q(X)) \text{ and } (\exists X P(X)) \wedge (\exists X Q(X))$$

...?

We consider this question further in [Exercise 14](#).

5.13 DUALITY BETWEEN QUANTIFIERS

If you have negation immediately to the left of a quantifier, then you may move it to the right of the quantifier (and its associated variable) provided you “flip” the quantifier as you do so ($\exists \longleftrightarrow \forall$).

$$\begin{aligned} \neg \forall Y & \text{ means the same as } \exists Y \neg \\ \neg \exists Y & \text{ means the same as } \forall Y \neg \end{aligned}$$

So, for any predicate logic expression $P(X)$ in which the variable X is free, we have the two laws

$$\begin{aligned} \neg \forall X P(X) &= \exists X \neg P(X), \\ \neg \exists X P(X) &= \forall X \neg P(X). \end{aligned}$$

These are **De Morgan’s Laws for quantifiers**. This terminology reflects the relationship between $\exists$ and disjunction, and the relationship between $\forall$ and conjunction.

So, for example, “Not all stars are visible” is the same as “There exists an invisible star”. To see this using logic, suppose we have unary predicates *star* and *visible*:

$$\begin{aligned} & \neg \forall X (\text{star}(X) \Rightarrow \text{visible}(X)) && \text{Not all stars are visible} \\ = & \exists X \neg (\text{star}(X) \Rightarrow \text{visible}(X)) && (\text{changing } \neg \forall \text{ to } \exists \neg) \\ = & \exists X \neg (\neg \text{star}(X) \vee \text{visible}(X)) && (\text{representing } \Rightarrow \text{ using } \neg \text{ and } \vee) \\ = & \exists X (\neg \neg \text{star}(X) \wedge \neg \text{visible}(X)) && (\text{by De Morgan’s Law}) \\ = & \exists X (\text{star}(X) \wedge \neg \text{visible}(X)) && \text{There exists an invisible star} \end{aligned}$$

Similarly,

$$\begin{aligned} \neg \forall Y \neg & \text{ means the same as } \exists Y \\ \neg \exists Y \neg & \text{ means the same as } \forall Y \end{aligned}$$

5.14 SUMMARY OF RULES FOR LOGIC WITH QUANTIFIERS

When doing predicate logic, we can use all the rules of propositional logic (§ 4.15) together with the principles for doing logic with quantifiers that we have introduced in this chapter. We summarise these principles below.

The notation we use in our list is:

- P and Q can be any predicates, and more generally they can be any predicate logic expressions in which the indicated variables appear (e.g., $P(x)$ can be any predicate logic expression in which x appears);
- R can be any predicate logic expression in which x does not appear as a free variable;
- x and y are free variables in the predicate expressions containing them;
- “obj” can be any specific object in the domain of x ;
- A is any subset of the domain of x , and Π_A is the predicate which is True if its argument is in A and False otherwise (so Π_A is just the set A turned into a predicate).

Here is the list of principles, with the sections discussing them on the right.

$$\begin{array}{llll} \exists x \in A P(x) & \text{is equivalent to} & \exists x (\Pi_A(x) \wedge P(x)) & \S 5.7 \\ \forall x \in A P(x) & \text{is equivalent to} & \forall x (\Pi_A(x) \Rightarrow P(x)) & \S 5.9 \end{array}$$

$$\begin{array}{llll} \exists x R & \text{is equivalent to} & R & \S 5.7 \\ \forall x R & \text{is equivalent to} & R & \S 5.9 \end{array}$$

$$\begin{array}{llll} \exists x \exists y P(x, y) & \text{is equivalent to} & \exists y \exists x P(x, y) & \S 5.10 \\ \forall x \forall y P(x, y) & \text{is equivalent to} & \forall y \forall x P(x, y) & \end{array}$$

$$\begin{array}{llll} \forall x P(x) & \text{implies} & P(\text{obj}) & \S 5.12 \\ P(\text{obj}) & \text{implies} & \exists x P(x) & \end{array}$$

$$\begin{array}{llll} \exists x (P(x) \vee Q(x)) & \text{is equivalent to} & (\exists x P(x)) \vee (\exists x Q(x)) & \S 5.12 \\ \forall x (P(x) \wedge Q(x)) & \text{is equivalent to} & (\forall x P(x)) \wedge (\forall x Q(x)) & \end{array}$$

$$\begin{array}{llll} \neg \exists x P(x) & \text{is equivalent to} & \forall x \neg P(x) & \S 5.13 \\ \neg \forall x P(x) & \text{is equivalent to} & \exists x \neg P(x) & \end{array}$$

5.15 SOME EXAMPLES

Suppose we have the property Prime, with domain $\mathbb{N}$, which is True if its argument is a prime number and False otherwise. Suppose also that we have the binary predicate $\leq$, and that any variables we use must have domain $\mathbb{N}$.

How might we use these to state [Theorem 17](#), that there are infinitely many primes?

At first glance, this might look like an existential statement, so we reach for an existential quantifier. The trouble is, we are asserting the existence of infinitely many

numbers of a particular type, but we are not allowed to use infinitely many quantifiers (or to do anything else that makes the statement infinitely long). Furthermore, the ingredients available to us here is quite limited; they do not allow us to describe arbitrarily long sequences. (There are richer settings in which that can be done, but that's a different puzzle!) So, what do we do?

Think about what it means for a set to be infinite. This means it goes on forever, i.e., it's *unbounded*; in other words, no matter what bound you might try to put on the numbers in this set, they eventually get bigger than the bound.

Let's focus on this, for the set of primes:

Every bound is exceeded by some prime.

We begin to see hints of quantifiers emerge:

Every bound is exceeded by some prime.

Rewording a bit:

For every bound, there exists a prime that is greater than the bound.

This is getting close enough to precise logical language that we can try writing it symbolically:

$$\forall b \exists n \in \{\text{primes}\} \quad n > b.$$

Let us move the condition on $\exists n$ to later in the expression, so there is no qualification on $\exists n$, and so that we can use our predicate *Prime* rather than using other symbols and relations we haven't been given in this scenario. To do this, we use the method of § 5.7 (see the first line of the list in § 5.14):

$$\forall b \exists n \text{ Prime}(n) \wedge (n > b).$$

We're almost there! The remaining detail is that $>$ has not been given to us as an available predicate (and nor has $<$). So we need to find a way of saying the same thing using $\leq$, which is one of our ingredients. This is straightforward, because $n > b$ if and only if $n \not\leq b$, which may be written using logical negation and $\leq$. So our final statement in predicate logic is

$$\forall b \exists n \text{ Prime}(n) \wedge \neg(n \leq b). \tag{5.9}$$

Now, think about the order of quantifiers here. We have seen previously that the order of *different* quantifiers *does* matter (§ 5.10). So, in this case, it would be a mistake to write the two quantifiers the other way round, as in

$$\exists n \forall b \text{ Prime}(n) \wedge \neg(n \leq b). \quad \text{X} \tag{5.10}$$

We will now look at this second statement in detail, to understand how it differs from (5.9), and so gain insight into the effect of the different orders of quantifiers. In the process, we will apply some other principles we have learned.

What does this second statement [Equation 5.10](#) say? There exists a number n such that, for *every* number b , n is prime and $n > b$. This certainly sounds different but its meaning may not yet be clear.

The rules of predicate logic can help make it clear.

After the existential quantifier in [\(5.10\)](#), we have $\forall b \text{ Prime}(n) \wedge \neg(n \leq b)$, which is of the form $\forall x(P(x) \wedge Q(x))$ so we can apply one of our principles of doing logic with quantifiers to deduce that

$$\forall b \text{ Prime}(n) \wedge \neg(n \leq b) \quad \text{is equivalent to} \quad (\forall b \text{ Prime}(n)) \wedge (\forall b \neg(n \leq b)).$$

Now, consider

$$\forall b \text{ Prime}(n).$$

The truth value of this depends entirely on the truth value of $\text{Prime}(n)$, because $\text{Prime}(n)$ does not depend on b . This is an instance of the principle that quantifying an expression over a variable that does not appear in the expression makes no difference to it, logically. You can include such a quantifier, or not, according to your preference; the expressions you get are equivalent. (See the end of [§ 5.9](#).) So, omitting this one superfluous universal quantifier from $\forall b \text{ Prime}(n)$, we see that

$$(\forall b \text{ Prime}(n)) \wedge (\forall b \neg(n \leq b)) \quad \text{is equivalent to} \quad \text{Prime}(n) \wedge (\forall b \neg(n \leq b)).$$

Substituting this back into the portion of [\(5.10\)](#) from $\forall b$ onwards, we see that it is equivalent to

$$\exists n (\text{Prime}(n) \wedge (\forall b \neg(n \leq b))).$$

Another way to write this, using the rule about restricting existential quantifiers (but going backwards), is

$$\exists n \in \{\text{primes}\} \quad \forall b \neg(n \leq b).$$

Rewriting $\neg(n \leq b)$ might make the meaning clearer:

$$\exists n \in \{\text{primes}\} \quad \forall b \quad n > b.$$

So, this is saying that there is a prime number that is greater than every number! This is clearly false, and in any case we see that the meanings of [\(5.9\)](#) and [\(5.10\)](#) are very different. This illustrates the fact that the order of two *different* quantifiers (one universal, one existential) does matter.

5.16 EXERCISES

1. What can a predicate with no arguments be?
2. Suppose you have the predicates `prolog` and `elvish`, with the following meanings:
`prolog(X)`: X knows the Prolog language.
`elvish(X)`: X knows the Elvish language.

- (a) Write a universal statement in predicate logic with the meaning:

“Nobody knows both Prolog and Elvish.”

- (b) Suppose that the statement in (a) is *False*. Starting with its negation, derive an existential statement meaning that someone knows both these languages.

3. The predicate supervised has two arguments, both of which are people. The meaning of supervised(X, Y) is that person X supervised the PhD of person Y .

Express each of the following three sentences in predicate logic.

- (a) Alonzo Church was the PhD supervisor of Alan Turing.

- (b) No person supervised their own PhD.

- (c) Not everyone has supervised someone else’s PhD.

- (d) Not every PhD supervisor had a PhD supervisor.

Write this one as an *existential statement*.

- (e) Someone had at least two PhD supervisors.

- (f) Alan Turing supervised the PhDs of exactly two people.

4. The predicate designedFirstComputerIn has two arguments. The first argument is a person, and the second argument is a country. The meaning of designedFirstComputerIn(X, Y) is that person X was one of the designers of the first computer in country Y .

Express each of the following three sentences in predicate logic.

- (a) Trevor Pearcey and Maston Beard designed the first computer in Australia.

- (b) No-one designed the first computer in two different countries.

5. (six degrees of separation)

Consider the binary predicate knows from p. 68 in § 2.14. The domain of each of its arguments is the set of all people.

It has been claimed that, in the human social network, the distance between any two people is at most 6. (See p. 70 in § 2.15.)

Write this claim in predicate logic, using just the predicate knows.

6. Suppose you have the equality predicate for sets, the symmetric difference function for sets, and set variables A and B . Write the statement of [Theorem 4](#) in predicate logic.

7. For each of the following predicate logic statements, (i) identify the predicates, functions, variables (including a suggested domain for each) and constants used; (ii) state whether or not it is a proposition, and if it is, whether it is true or false.

(a) $\forall x \, x^2 \geq 0$

(b) $x^2 < 0$

(c) $\forall x \, 2^x = x^2$

(d) $x < 0 \Rightarrow \forall y ((0 < y) \Rightarrow (x < y))$

(e) $\forall x \forall z \exists y (x < y) \wedge (y < z)$

(f) $\exists x \exists y \neg(x \Rightarrow y)$

8. Consider the following predicate logic statement.

$$\forall A \forall B (A \subseteq B \Rightarrow (\mathcal{P}(A) \subseteq \mathcal{P}(B)))$$

What predicates, functions and variables does this statement use? Restate, in words, the theorem that is being stated here.

9. Suppose that

- W is a variable whose domain is the set of all English words,
- L is a variable whose domain is the set of all English letters,
- `WordContainsLetter` is a binary predicate which takes an English word as its first argument, an English letter as its second argument, and is `True` whenever that word contains that letter. For example,

`WordContainsLetter(quizzical, z)` is `True`,
`WordContainsLetter(quizzical, e)` is `False`.

(a) Write the statement of [Theorem 15](#) in predicate logic.

(b) Now suppose you also have a unary predicate `Vowel` whose domain is the set of English letters and which is `True` when its argument is a vowel.

Now write the statement of [Theorem 15](#) in predicate logic again, but using this new predicate to write it more compactly this time.

10. Suppose you have the positive integer properties Even and Prime, which are true precisely for even numbers and prime numbers respectively, and that you also have the binary predicate $\leq$ on $\mathbb{N}$. Write a predicate logic expression for the statement

There are infinitely many odd prime numbers but only finitely many even prime numbers.

11. The ternary predicate Date is defined for any $(d, m, y) \in \mathbb{N} \times \mathbb{N} \times \mathbb{N}$, and is True if (d, m, y) represents a valid date in day-month-year format in the Gregorian calendar, and is False otherwise.

Using Date and $<$, write an expression in predicate logic which is True if and only if (d_1, m_1, y_1) and (d_2, m_2, y_2) are both valid dates and the first date comes chronologically before the second.

12. Suppose you have

- the positive integer predicate $\leq$;
- the function $t : \{\text{programs}\} \times A^* \rightarrow \mathbb{N}$ defined for all programs P and input strings $x \in A^*$ (where A is an alphabet (§ 1.5)) by

$t(P, x) =$ the time taken by program P when run on input x ;

- the function $\text{len} : A^* \rightarrow \mathbb{N}_0$ defined for any string $x \in A^*$ by

$\text{len}(x) =$ the length of the string x ;

- binary functions for multiplication and exponentiation on the positive integers;
- a string variable $x \in A^*$;
- positive integer variables c and k .

Write the following statement in predicate logic.

There exist integers c and k such that, for every input string x , the time taken by program P on input x is at most the product of c and the k -th power of the length of x .

13. There are many theorems that assert that there is a *unique* object that satisfies certain conditions. In this exercise, we look at how to make such statements using predicate logic.

Suppose you want to say that there is a unique x with property $P(x)$. It's not enough to just write

$$\exists x P(x),$$

because that only enforces existence without guaranteeing uniqueness. Sometimes, people use “ $\exists!$ ” as a shorthand for “there exists a unique”. With that shorthand,

$$\exists!x P(x)$$

means

There exists a unique x such that $P(x)$ holds.

How would you write this statement just using the tools available to us in predicate logic, i.e., without using “ $!$ ”?

14.

(a) Prove that

$$\exists X (P(X) \wedge Q(X)) \text{ implies } (\exists X P(X)) \wedge (\exists X Q(X))$$

(b) Prove that

$$\forall X (P(X) \vee Q(X)) \text{ is implied by } (\forall X P(X)) \vee (\forall X Q(X))$$

in two ways: (i) reason it through directly, (ii) use the result from part (a) and what you’ve learned about the relationship between existential and universal quantifiers.

SEQUENCES & SERIES

A sequence may be regarded as a list of objects. Their defining characteristic is that the objects are in some *order*, so there is a *first* object, then a *second* object, then a *third* object, and so on. We can use sequences to represent:

- items in a file, ordered by their position in the file, such as the words in a text file, or the lines in a text file, or the frames in a movie file, or the rows in a spreadsheet;
- items ordered by time, such as the children in a family (by birth order), or a person's posts to one of their social media platforms, or the population of a city in each year, or the dishes served in the successive courses of a banquet, or the world's early computers in order of when they first ran a program;
- items ordered in space along a line or curve, such as the houses along one side of a street, or the floors of a building, or the waterfalls along a river, or the amino acids along a protein molecule;
- items in some order of rank, such as the top ten songs according to some popularity poll, or the world's highest mountains in order, or the planets of the solar system in order of mean distance from the Sun, or the world's fastest computers in order of their speed on some suite of benchmark problems;
- the successive letters of a string, in fact a string is just a finite sequence whose members happen to be letters.

Inside a computer, data is always stored in some order, even in cases where the order may not be important. So, for example, if a set — in which order does not matter — is to be represented on a computer, then it must ultimately be represented as some sequence of items somewhere in the computer's memory.

Computation is a process that takes place over *time*. Any characteristic of a computation — such as the amount of memory it uses, or the amount of energy consumed, or some aspect of the information displayed — gives rise to a sequence that specifies how that characteristic changes over time.

We are also fundamentally concerned with how the time taken by some program depends on the size of the input. A sequence of time measures, in order of input size, shows how the time grows as the input size increases.

Sequences are therefore one of the most fundamental of all abstract models.

6.1^α DEFINITIONS AND NOTATION

A **sequence** is a function whose domain is the set of positive integers or some finite initial portion of it. So the domain of a sequence is either $\mathbb{N}$ or $[1, n]_{\mathbb{N}}$ for some $n \in \mathbb{N}$.

An **infinite sequence** is a sequence with domain $\mathbb{N}$.

A **finite sequence** is a sequence with domain $[1, n]_{\mathbb{N}}$ for some n . This is really the same as an n -tuple, and finite sequences are often written in tuple notation:

(first object, second object, third object, ..., last object).

For any set A , a **sequence over A** is a sequence whose codomain is A .

If $f: \mathbb{N} \rightarrow A$ is a sequence over A , then $f(n)$ is referred to as the n -th **term** of the sequence. The terms of the sequence f are the members of the image of f — remembering the definition of the *image* of a function from p. 39 in § 2.1.2^α — written in the order given by the domain $\mathbb{N}$.

The **length** of a finite sequence is the size of its domain, which is the number of terms of the sequence.

It is common to write a sequence as a comma-separated list of its terms, in order:

$$f(1), f(2), f(3), \dots \tag{6.1}$$

This is fine if the sequence is finite and very short, and it can be useful even for long sequences and infinite sequences provided the pattern is clear. But this way of writing a sequence is really just for informal exposition, *not* for defining the sequence. A sequence cannot be defined without either listing all its members in order (only practical if the sequence is very short) or giving a precise rule by which, for each $n \in \mathbb{N}$, the n -th term can be determined.

If f is a sequence, then its n -th term $f(n)$ is often denoted by f_n . This can be thought of as just a variation on the usual notation for function values, but one that is used more often in the context of sequences. With this notation, we might rewrite (6.1) as

$$f_1, f_2, f_3, \dots$$

We can always define a sequence over A by using the fact that it is simply a function $f: \mathbb{N} \rightarrow A$ and using one of our ways of defining functions (§ 2.3^α).

There is another common convention for defining sequences, which has a couple of variants:

$$(\text{formula for } n\text{-th term} : n \in \mathbb{N}) \quad \text{or} \quad (\text{formula for } n\text{-th term})_{n=1}^{\infty}.$$

The first of these is reminiscent of the way we used formulas to define sets, on p. 3 in § 1.2^α, except that now we use parentheses rather than curly braces because the order matters. This convention also applies to finite sequences:

$$(\text{formula for } n\text{-th term} : n \in \mathbb{N}, 1 \leq n \leq N) \quad \text{or} \quad (\text{formula for } n\text{-th term})_{n=1}^N.$$

For example, the infinite sequence of squares

$$1, 4, 9, 16, 25, \dots$$

can be written formally as

$$(n^2 : n \in \mathbb{N}) \quad \text{or} \quad (n^2)_{n=1}^{\infty}.$$

We will be especially interested in sequences over various sets of numbers, which we call **number sequences**. Typical codomains for number sequences include $\mathbb{N}$ or $\mathbb{Z}$ or $\mathbb{Q}$ or $\mathbb{R}$. The importance of these sequences is due not only to the fact that a lot of computation uses numbers, but also due to the use of numbers in measuring characteristics of computation, such as the time spent, the size of the input or output data, the amount of memory used, the locations of data in memory, energy consumption, and so on.

6.2^α RECURSIVE DEFINITIONS OF SEQUENCES

We have seen how to define sequences by expressing each term as a function of its position in the sequence. So, the n -th term f_n is given by a formula in n .

Another way to define sequences is to give an expression that uses a previous term in the sequence, or possibly several previous terms. For example, we could write

$$f_n = f_{n-1} + 2.$$

But this alone is not sufficient, because we have to specify how to get started. If we write

$$f_1 = 1, \quad f_n = f_{n-1} + 2,$$

then we have defined the sequence of odd numbers

$$1, 3, 5, 7, \dots \tag{6.2}$$

which could also be defined as $(2n - 1 : n \in \mathbb{N})$ or $(2n - 1)_{n=1}^{\infty}$. Starting with a different value would give a different sequence, even though the formula for f_n looks the same. For example,

$$f_1 = 0, \quad f_n = f_{n-1} + 2, \tag{6.3}$$

defines the sequence of even numbers

$$0, 2, 4, 6, \dots$$

For another example, consider the sequence of powers of 2,

$$2, 4, 8, 16, \dots$$

Previously, we might have defined this sequence as $(2^n : n \in \mathbb{N})$. Now, we can define it by

$$f_1 = 2, \quad f_n = 2f_{n-1}. \quad (6.4)$$

In general, a **recursive definition** of a family of objects consists of:

- the **base case**: a definition of a finite number of the simplest-possible objects in the family;
- the **general case**: a rule that defines a general object in the family in terms of simpler objects in the family.

The base cases, together with the general rule, must be sufficient so that, used together, any object in the family is defined uniquely, precisely and clearly.

For sequences, the base case gives some initial terms explicitly, and then the general case is given as an expression using previous terms in the sequence. A recursive definition of a number sequence is called a **recurrence relation**. We have seen three recurrence relations so far: for the odd numbers in (6.2), for the even numbers in (6.3), and for the powers of 2 in (6.4).

These sequences are all familiar and could be defined either by a formula in n or by a recurrence relation, according to taste or the needs of the situation in which they are being used. But there are many situations where a recurrence relation is the most natural way to define a sequence. For example, consider the factorials

$$1, 2, 6, 24, 120, \dots$$

This sequence may be defined by the recurrence relation

$$f_1 = 1, \quad f_n = n f_{n-1}. \quad (6.5)$$

This also has a formulaic definition, $(n! : n \in \mathbb{N})$, but $n!$ is really just a standard abbreviation for $n(n-1)(n-2)\cdots 3\cdot 2\cdot 1$, so the most succinct definition without using special abbreviations is the recurrence relation. This sequence also illustrates the point that, in the rule of a recurrence relation, the position n does not have to be confined to the subscripts; it can also be used, in its own right, in the expression, as it is on the right-hand side of the second equation in (6.5).

Exercise: Give recurrence relations for each of the following sequences: the positive integers; the negative even integers; the squares; the sequence whose n -th term is the sum of the first n positive integers; the sequence whose n -th term is the sum of the reciprocals of the first n positive integers.

So far, all our recurrence relations have rules that only use the previous term in the sequence. But rules can use terms that appear earlier than that. Consider the recurrence relation

$$f_1 = 2, f_2 = 4, f_n = 4f_{n-2}. \quad (6.6)$$

With the rule $f_n = 4f_{n-2}$, it is not sufficient to just define one base case, such as $f_1 = 2$. If we did that, then we could only compute every second term:

$$f_1 = 2, f_3 = 4 \cdot f_1 = 4 \cdot 2 = 8, f_5 = 4 \cdot f_3 = 4 \cdot 8 = 32, \text{ and so on.}$$

We need another base case to specify f_2 , and then the recurrence will give us values at all even positions too. If we set $f_2 = 4$, as in (6.6), then we again get the powers of 2, so in this case we have only come up with a more complicated way to define that sequence. But we can do other things too. For example, try

$$f_1 = 2, f_2 = 0, f_n = 4f_{n-2}. \quad (6.7)$$

This defines the sequence

$$2, 0, 8, 0, 32, 0, 128, 0, \dots$$

which can also be defined by

$$f_n = \begin{cases} 2^n, & \text{if } n \text{ is odd;} \\ 0, & \text{if } n \text{ is even.} \end{cases}$$

6.3^α ARITHMETIC SEQUENCES

An **arithmetic sequence**, also called an **arithmetic progression**, is a number sequence in which the difference between every pair of consecutive terms is the same. This means there is some **common difference** d such that every term is obtained by adding d to its predecessor. This, together with its first term which we'll call a , gives the following recurrence relation:

$$f_1 = a, f_n = f_{n-1} + d.$$

The successive terms are

$$a, a + d, a + 2d, \dots$$

with the n -th term being $a + (n - 1)d$.

An arithmetic sequence is specified by giving its first term a , its common difference d , whether it is finite or infinite and, if it is finite, its number of terms n . In the finite case, the sequence looks like

$$a, a + d, a + 2d, \dots, a + (n - 1)d. \quad (6.8)$$

Here, the i -th term is $a + (i - 1)d$, where $1 \leq i \leq n$.

Here are some old and new examples.

description	first few terms	a	d	length
even numbers	0, 2, 4, 6, ...	0	2	∞
odd numbers	1, 3, 5, 7, ...	1	2	∞
positive integers	1, 2, 3, ...	1	1	∞
launch countdown	10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0	10	-1	11
positive integers with remainder 2 after division by 3	2, 5, 8, 11, 14, ...	2	3	∞
years of the 21st century	2001, 2002, ..., 2100	2001	1	100
leap years this century	2004, 2008, ..., 2096 ¹	2004	4	24
day of each Monday in May 2025	5, 12, 19, 26	5	7	4
an account with one fixed regular bill	220, 120, 20, -80, -180	220	-100	5
annual balances (\$) of a 5-year investment with initial balance \$100 and <u>simple</u> interest accruing at 10% p.a.	100, 110, 120, 130, 140, 150	100	10	6

6.4^a GEOMETRIC SEQUENCES

A **geometric sequence**, also called a **geometric progression**, is a number sequence in which the *ratio* between every pair of consecutive terms is the same. This means there is some **common ratio** r such that every term is obtained by multiplying its predecessor by r .

Again, we call the first term a . We have the following recurrence relation:

$$f_1 = a, \quad f_n = f_{n-1} \cdot r.$$

¹ The year 2100 does indeed belong to this century, which goes from 2001 to 2100 inclusive. But 2100 is not a leap year, despite being a multiple of 4.

The successive terms are

$$a, ar, ar^2, \dots$$

with the n -th term being ar^{n-1} .

A geometric sequence is specified by giving its first term a , its common ratio r , whether it is finite or infinite and, if it is finite, its number of terms n . In the finite case, the sequence looks like

$$a, ar, ar^2, \dots, ar^{n-1}. \quad (6.9)$$

The i -th term is ar^{i-1} , where $1 \leq i \leq n$.

Examples:

description	first few terms	a	r	length
powers of 2	1, 2, 4, 8, 16, ...	1	2	∞
reciprocals of powers of 2	$1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \dots$	1	$\frac{1}{2}$	∞
decimal place values, going leftwards of decimal point	1, 10, 10 ² , 10 ³ , ...	1	10	∞
alternating ± 1	1, -1, 1, -1, ...	1	-1	∞
area (m ²) of paper sizes A4, A3, A2, A1	0.0625, 0.125, 0.25, 0.5	0.0625	2	4
number of transistors on an integrated circuit, each year from early 1980s (idealised, simplified model based on Moore's Law)	100 000, 141 000, 200 000, 282 000, ...	10 ⁵	1.41	?
annual balances (\$) of a 5-year investment with initial balance \$100 and <u>compound</u> interest accruing at 10% p.a.	100, 110, 121, 133.1, 146.41, 161.05	100	1.1	6
value (in \$) each year of an asset bought for \$100 which depreciates at 10% p.a. over five years	100, 90, 81, 72.9, 65.61, 59.05	100	0.9	6

6.5 HARMONIC SEQUENCES

A **harmonic sequence** is a sequence of terms whose reciprocals form an arithmetic sequence.

Harmonic sequences, like arithmetic sequences, can be finite or infinite. If a sequence is infinite, its set of indices is $\mathbb{N}$. If it's finite, its set of indices is the set $\{1, 2, \dots, n\}$ of the first n positive integers. We use M to denote the set of indices of the sequence.

So $(a_n : n \in M)$ is a harmonic sequence if and only if $(1/a_n : n \in M)$ is an arithmetic sequence.

The sequence 3, 4, 6, 12 is harmonic, because its sequence of reciprocals, $\frac{1}{3}, \frac{1}{4}, \frac{1}{6}, \frac{1}{12}$, is an arithmetic sequence. To see this, express all these fractions over a common denominator, and the sequence becomes

$$\frac{4}{12}, \frac{3}{12}, \frac{2}{12}, \frac{1}{12},$$

which is an arithmetic sequence of four terms with first term $a = \frac{1}{3}$ and common difference $d = -\frac{1}{12}$.

One particularly important harmonic sequence is the sequence of reciprocals of positive integers: $(1/n : n \in \mathbb{N})$. This is clearly harmonic because the positive integers form an arithmetic sequence. We discuss its properties further in § 6.15.

6.6 FROM RECURSIVE DEFINITIONS TO FORMULAS

We introduced recursive definitions of sequences in § 6.2^α. Recursive definitions arise very naturally, and are often the simplest way of writing down a rule for a sequence. But it is typically not easy to see, from the recursive rule, just how the sequence behaves as n increases. So we would like to find a formula for the n -th term, meaning an expression for the n -th term in the sequence that depends only on n , and not on any previous terms of the sequence. Even if we cannot find an exact formula for the n -th term, maybe we can find upper and/or lower bounds for it, and this can be very useful in analysing the behaviour of the sequence.

We studied arithmetic sequences in § 6.3^α; they have a recursive rule of the form $f_n = f_{n-1} + d$. Then we studied geometric sequences in § 6.4^α; these have a rule of the form $f_n = r f_{n-1}$. In each of these cases, we also have a formula for the n -th term: for the arithmetic sequence, $f_n = a + (n-1)d$, while for the geometric sequence, $f_n = ar^{n-1}$.

But many sequences are not of either of those types. Consider for example the following recursive definition.

$$f_1 = 1, \quad f_n = 2f_{n-1} + 1. \quad (6.10)$$

This sequence is neither arithmetic nor geometric, although it has both a constant multiplier and an additive constant, so in a way it seems like a mix of both types. Can we find, and prove, a formula for it?

We use this example to illustrate a very common approach:

1. **explore** the sequence by working out the first few terms;
2. study those terms, look for patterns, and try and **conjecture** an expression that fits the pattern you have observed so far;

3. **prove**, by induction on n , that your formula works for all n .

Let us apply this **explore-conjecture-prove** approach to our recursive definition in (6.10).

1. **explore**:

$$\begin{aligned} f_1 &= 1 && \text{(given in (6.10))} \\ f_2 &= 2f_1 + 1 = 2 \cdot 1 + 1 = 3 \\ f_3 &= 2f_2 + 1 = 2 \cdot 3 + 1 = 7 \\ f_4 &= 2f_3 + 1 = 2 \cdot 7 + 1 = 15 \end{aligned}$$

2. **conjecture**:

We can see that the values of f_n , for $n \leq 4$, are each one less than a power of 2. This trend looks likely to continue! We write this down as a formula, taking care to get the exponent of 2 correct and that the formula works correctly for the known initial case $n = 1$. So we propose

$$f_n = 2^n - 1. \tag{6.11}$$

3. **prove**:

Now we prove (6.11) holds for all $n \in \mathbb{N}$. It is natural to prove this by induction.

Inductive Basis:

If $n = 1$, then we know from the initial condition in (6.10) that $f_1 = 1$, and for $n = 1$ the formula (6.11) gives $f_1 = 2^1 - 1 = 2 - 1 = 1$, so the formula in (6.11) is correct for $n = 1$.

Inductive step:

Let $k \geq 1$.

Assume that (6.11) holds for $n = k$, i.e., that $f_k = 2^k - 1$. (This is the Inductive Hypothesis.)

Now consider f_{k+1} . The recursive rule from (6.10) gives

$$\begin{aligned} f_{k+1} &= 2f_k + 1 && \text{(by (6.10), and using } k \geq 1) \\ &= 2(2^k - 1) + 1 && \text{(by the Inductive Hypothesis)} \\ &= 2^{k+1} - 2 + 1 \\ &= 2^{k+1} - 1. \end{aligned}$$

So we have established that (6.11) holds for $n = k + 1$ too. This completes the Inductive Step.

Conclusion:

Therefore, by Mathematical Induction, (6.11) holds for all $n \in \mathbb{N}$.

6.7 THE FIBONACCI SEQUENCE

Now we consider another recursive definition, this time one where each term depends on the two previous terms:

$$f_1 = 1, \quad f_2 = 1, \quad f_n = f_{n-1} + f_{n-2} \quad (\text{for } n \geq 3). \quad (6.12)$$

This is the famous **Fibonacci sequence**, whose first ten terms are given in the second line of the following table.

n	1	2	3	4	5	6	7	8	9	10	...
f_n	1	1	2	3	5	8	13	21	34	55	...

In this section we study this sequence at length, showing how the explore-conjecture-prove methodology can be used to determine upper bounds, lower bounds, estimates of long-term behaviour, and eventually an exact formula for the n -th term. This extended case study illustrates the approaches and kinds of thinking that can be applied to many other problems of analysing recursively-defined sequences. We explain the process at some length, because we are illustrating not just the underlying mathematical principles, but also the process of investigating the sequence and developing the results and proofs.

For completeness, we include a series of proofs by induction which are each given in full even though they only differ slightly from each other. This takes up space, but there is not much in each proof that is new, and we indicate exactly what has changed in each case.

Dealing with sequences whose terms depend on two previous terms is more complex than when the dependence is on just one previous term (as was the case for the recursively-defined sequences we gave in § 6.2^α and § 6.6), so studying this case shows how to deal with this extra complexity.

The techniques described in this section can be extended to deal with sequences whose terms depend on three or more previous terms. We don't discuss such cases as they have more technical detail, although the underlying principles are the same.

There is also particular value in gaining a deep understanding of the Fibonacci sequence, since it arises throughout computer science and mathematics.

It is difficult to determine the behaviour of the Fibonacci sequence just from its recursive definition. So, again, we take the explore-conjecture-prove approach. Our exploration gives the terms shown above, and you can explore further using, say, a

spreadsheet or a program in order to develop a better understanding of its long-term behaviour. This suggests that the sequence grows quite quickly. But how quickly? The ratio f_{n+1}/f_n between successive terms is not constant, but that ratio does seem to be settling down. Although an exact formula for f_n is not yet evident, we can use the apparent settling-down of the ratio f_{n+1}/f_n to conjecture some simple upper and lower bounds, and this is still very useful information.

The ratio f_8/f_7 is $21/13 = 1.615\dots$, the ratio f_9/f_8 is $34/21 = 1.619\dots$, and the ratio f_{10}/f_9 is $55/34 = 1.617\dots$. While we do not yet know what happens for $n > 10$, we can look at the table of values and develop some hypotheses for upper and lower bounds for that ratio and for the terms of the sequence.

6.7.1 Upper bounds

The maximum ratio between any two consecutive terms is $f_3/f_2 = 2/1 = 2$, and our explorations suggest that it is likely to stay well below that. So we can conjecture the upper bound

$$f_n \leq 2^n. \quad (6.13)$$

We now prove this by induction on n .

Inductive Basis:

- When $n = 1$, we have $f_1 = 1$, and our upper bound is $2^1 = 2$, so we certainly have $f_1 \leq 2^1$.
- We need another base case too, for $n = 2$. The reason for this will become evident shortly. When $n = 2$, we have $f_2 = 1$, and our upper bound is $2^2 = 4$, so we have $f_2 \leq 2^2$ (which is an even looser upper bound than for $n = 1$).

Inductive Step:

Let $k \geq 2$.

Assume that $f_l \leq 2^l$ for all $l \leq k$. (This is the Inductive Hypothesis.)

Comment: This is a case of what is sometimes called “strong induction”, where we assume a claim holds not just for k , but for all positive integers $\leq k$. This is entirely valid, and in fact it is really still just normal induction, since it is *still a statement just about k* . In the Inductive Hypothesis, the variable l is *bound* by the universal quantifier, so l is not a free variable (unlike k), so the Inductive Hypothesis is a statement about k only, not about l .

Now consider f_{k+1} . Since $k \geq 2$, we have $k+1 \geq 3$, so we can apply the recursive rule (6.12):

$$\begin{aligned} f_{k+1} &= f_k + f_{k-1} && \text{(by (6.12), and using } k \geq 2) \\ &\leq 2^k + 2^{k-1} && \text{(by the Inductive Hypothesis, twice)} \\ &< 2^{k+1}. \end{aligned}$$

Comment: We can now see why we needed the Inductive Basis to cover $n = 2$ as well as $n = 1$: our recurrence only works for $n \leq 3$, so smaller cases need to be dealt with separately.

This completes the Inductive Step.

Conclusion:

Therefore, by Mathematical Induction, our upper bound (6.13) holds for all n .

There are several points worth noting about this upper bound and its proof.

Firstly, we can make the upper bound tighter by putting $n - 1$ in the exponent instead of n :

$$f_n \leq 2^{n-1}. \quad (6.14)$$

The proof would go through again with very little change, in fact the main changes needed are to the Inductive Basis. Now, $f_1 = 1 \leq 2^{1-1}$ and $f_2 = 1 \leq 2^{2-1}$, so this tighter upper bound still holds when $n = 1$ and $n = 2$. The rest of the proof, which only uses the recursive rule, is unchanged except that, when applying the Inductive Hypothesis, we insert the new upper bounds instead of the old ones.

Secondly, another way to try to refine the upper bound is to put a constant factor in front. Let's try

$$f_n \leq \alpha \cdot 2^n. \quad (6.15)$$

What is the best factor α to use? We need the Inductive Basis to still work, so we want

$$f_1 = 1 \leq \alpha 2^1 \quad \text{and} \quad f_2 = 1 \leq \alpha 2^2.$$

These tell us that $\alpha \geq 1/2$, so putting $\alpha = 1/2$ should work. This gives an upper bound

$$f_n \leq \frac{1}{2} 2^n.$$

But the right-hand side here equals 2^{n-1} , so this upper bound is really just a restatement of (6.14).

This use of a constant factor should be kept in mind, as it is a common technique for refining formulas for bounds for sequences given by recurrence relations.

Thirdly, the only algebraic property of 2^n that we used is

$$2^k + 2^{k-1} \leq 2^{k+1}. \quad (6.16)$$

Dividing each side by 2^{k-1} , we see that this is equivalent to

$$2 + 1 \leq 2^2. \quad (6.17)$$

This indicates that the Inductive Step would still work if, instead of using the upper bound 2^n , we used r^n where r is a smaller number than 2, *provided* we still have an inequality like (6.17) except for r rather than 2. So we could choose r to satisfy

$$r + 1 \leq r^2. \quad (6.18)$$

This will ensure that

$$r^k + r^{k-1} \leq r^{k+1}, \quad (6.19)$$

which is analogous to (6.16), and this should make the Inductive Step work. So we should be able to get the Inductive Step to work for a tighter upper bound of the form r^n for some suitable $r < 2$, namely one that satisfies (6.18).

For the whole proof to work, we also need the Inductive Basis to work. This should be possible in this case; happily, $f_1 = f_2 = 1$ are each $< r^n$ for any $r > 1$ and $n \geq 1$.

So suppose we have some r that satisfies (6.18), for example you could try $r = 1.62$, and let us re-do the proof by induction, this time proving that, for all $n \in \mathbb{N}$,

$$f_n \leq r^n. \quad (6.20)$$

In presenting the new proof by induction, we give it in full, even though most of it is unchanged. The parts that are changed are given in blue; this is really just replacing 2 by r throughout. (We have also abbreviated some of the explanations.)

Inductive Basis:

- When $n = 1$, we have $f_1 = 1$, and our upper bound is $r^1 = r$, so we certainly have $f_1 \leq r^1$ (since $r > 1$).
- When $n = 2$, we have $f_2 = 1$, and our upper bound is r^2 , so we have $f_2 \leq r^2$ (since $r > 1$).

Inductive Step:

Let $k \geq 2$.

Assume that $f_l \leq r^l$ for all $l \leq k$. (This is the Inductive Hypothesis.)

Now consider f_{k+1} . Since $k \geq 2$, we have $k+1 \geq 3$, so we can apply the recursive rule (6.12):

$$\begin{aligned} f_{k+1} &= f_k + f_{k-1} && \text{(by (6.12), and using } k \geq 2) \\ &\leq r^k + r^{k-1} && \text{(by the Inductive Hypothesis, twice)} \\ &\leq r^{k+1} && \text{(by (6.19), which in turn follows from (6.18)).} \end{aligned}$$

This completes the Inductive Step.

Conclusion:

Therefore, by Mathematical Induction, our upper bound (6.20) holds for all n .

We can actually do even better by choosing r to satisfy (6.18) *with equality*, i.e., by requiring it to satisfy

$$r + 1 = r^2.$$

Rearranging, this is equivalent to

$$r^2 - r - 1 = 0. \quad (6.21)$$

So r should be a root of this quadratic equation, and the usual formula for these roots gives

$$r = \frac{1 \pm \sqrt{5}}{2}$$

so the two roots are

$$r_1 = \frac{1 + \sqrt{5}}{2} = 1.61803\dots \quad \text{and} \quad r_2 = \frac{1 - \sqrt{5}}{2} = -0.61803\dots \quad (6.22)$$

Either of these would enable the Inductive Step to work, but only r_1^n works as an upper bound, since it works for the Inductive Basis too whereas r_2^n does not work as an upper bound for either $n = 1$ or $n = 2$.

6.7.2 Lower bounds

The approach we have taken to upper bounds, in § 6.7.1, is readily adapted to lower bounds, with a little adjustment.

The minimum ratio f_{n+1}/f_n between consecutive terms is $f_2/f_1 = 1$, but using 1^n as a lower bound for f_n is not very useful, even though it is clearly correct! So we look for a higher ratio and see what we can do with it.

Exploration shows that, from the second term onwards, the ratio f_{n+1}/f_n between consecutive terms is ≥ 1.5 . It is tempting to go from this ratio to a proposed lower bound for f_n of 1.5^n . After all, we did something similar for upper bounds at the start of the previous section (§ 6.7.1), when we observed that the ratio of consecutive terms is always ≤ 2 and then found that an upper bound of 2^n would work.

But bounds on ratios of consecutive terms do not always immediately yield bounds on the sequence itself. In this case, 1.5^n does not work as a lower bound, because it fails for $n = 1$ and $n = 2$. We need to get a lower bound that works for those cases too. Then, when we prove our lower bound later, we will be able to get the Inductive Basis to work.

So we introduce a constant factor α , much as we did in (6.15) in § 6.7.1 when seeking an upper bound:

$$f_n \geq \alpha \cdot 1.5^n.$$

For the Inductive Basis to work, we require

$$f_1 = 1 \geq \alpha \cdot 1.5^1 \quad \text{and} \quad f_2 = 1 \geq \alpha \cdot 1.5^2.$$

For these to hold, we need

$$\alpha \leq \frac{2}{3} \quad \text{and} \quad \alpha \leq \frac{4}{9}.$$

So we choose $\alpha = 4/9$ (noting that this implies $\alpha \leq 2/3$), and propose the lower bound

$$f_n \geq \frac{4}{9} \cdot 1.5^n. \quad (6.23)$$

We now prove this by induction on n . Again, we give the proof in full, with the new/changed parts in blue.

Inductive Basis:

- $n = 1$: we have $f_1 = 1$, and our lower bound is $\frac{4}{9} \cdot 1.5 = \frac{2}{3} \leq 1$, so the lower bound holds.
- $n = 2$: we have $f_2 = 1$, and our lower bound is $\frac{4}{9} \cdot 1.5^2 = \frac{4}{9} \cdot \frac{9}{4} = 1$, so the lower bound holds (with equality).

Inductive Step:

Let $k \geq 2$.

Assume that $f_l \geq \frac{4}{9} \cdot 1.5^l$ for all $l \leq k$. (This is the Inductive Hypothesis.)

Now consider f_{k+1} . Since $k \geq 2$, we have $k+1 \geq 3$, so we can apply the recursive rule (6.12):

$$\begin{aligned} f_{k+1} &= f_k + f_{k-1} && \text{(by (6.12), and using } k \geq 2) \\ &\geq \frac{4}{9} \cdot 1.5^k + \frac{4}{9} \cdot 1.5^{k-1} && \text{(by the Inductive Hypothesis, twice)} \\ &= \frac{4}{9} (1.5^k + 1.5^{k-1}) \\ &\geq \frac{4}{9} \cdot 1.5^{k+1}. \end{aligned}$$

This completes the Inductive Step.

Conclusion:

Therefore, by Mathematical Induction, our lower bound (6.23) holds for all n .

In the Inductive Step, we used the fact that

$$1.5^k + 1.5^{k-1} \geq 1.5^{k+1}.$$

The Inductive Step would still work for any bound of the form

$$f_n \geq \alpha r^n$$

provided r satisfies

$$r^k + r^{k-1} \geq r^{k+1}.$$

Dividing each side by r^{k-1} , this is equivalent to

$$r + 1 \geq r^2. \quad (6.24)$$

If we had equality, we would have the same quadratic equation we had previously, in (6.21), which has the two roots r_1 and r_2 given in (6.22). To ensure that the inequality (6.24) is satisfied, we require that r lies between those two roots:

$$r_2 \leq r \leq r_1.$$

We can let r be equal to r_1 or r_2 , and still the inequality is satisfied, which means the Inductive Step should still work, but we must still then choose α so that the Inductive Basis works too.

Suppose we use $r_1 = (1 + \sqrt{5})/2 = 1.61803\dots$ and let us determine α so that the proposed lower bound

$$f_n \geq \alpha r_1^n$$

works. For the Inductive Basis, we require

$$f_1 = 1 \geq \alpha r_1^1 \quad \text{and} \quad f_2 = 1 \geq \alpha r_1^2.$$

For these to hold, we need

$$\alpha \leq r_1^{-1} \quad \text{and} \quad \alpha \leq r_1^{-2}. \quad (6.25)$$

Now the fact that $r_1 > 1$ implies that $r_1^{-2} < r_1^{-1}$, so in fact the second of the two inequalities (6.25) implies the first, so we only need to require that

$$\alpha \leq r_1^{-2}$$

Now,

$$r_1^{-2} = \left(\frac{1 + \sqrt{5}}{2} \right)^{-2} = \frac{3 - \sqrt{5}}{2} = 0.381966\dots$$

We choose $\alpha = r_1^{-2}$ and propose the lower bound

$$f_n \geq r_1^{-2} r_1^n$$

which is just

$$f_n \geq r_1^{n-2}. \quad (6.26)$$

We now give the proof by induction that this holds for all $n \in \mathbb{N}$.

Inductive Basis:

- $n = 1$: we have $f_1 = 1$, and our lower bound is $r_1^{1-2} = r_1^{-1} = 0.61803\dots$, so the lower bound holds.
- $n = 2$: we have $f_2 = 1$, and our lower bound is $r_1^{2-2} = 1$, so the lower bound holds (with equality).

Inductive Step:

Let $k \geq 2$.

Assume that $f_l \geq r_1^{l-2}$ for all $l \leq k$. (This is the Inductive Hypothesis.)

Now consider f_{k+1} . Since $k \geq 2$, we have $k+1 \geq 3$, so we can apply the recursive rule (6.12):

$$\begin{aligned} f_{k+1} &= f_k + f_{k-1} && \text{(by (6.12), and using } k \geq 2\text{)} \\ &\geq r_1^{k-2} + r_1^{k-3} && \text{(by the Inductive Hypothesis, twice)} \\ &= r_1^{k-1} && \text{(by (6.21) and (6.22))} \\ &= r_1^{(k+1)-2} \end{aligned}$$

This completes the Inductive Step.

Conclusion:

Therefore, by Mathematical Induction, our lower bound (6.26) holds for all n .

6.7.3 Asymptotic behaviour

From § 6.7.1 and § 6.7.2 we have

$$r_1^{n-2} \leq f_n \leq r_1^n.$$

Dividing each side by r_1^n , we have

$$r_1^{-2} \leq \frac{f_n}{r_1^n} \leq 1.$$

This tells us that f_n always lies within a constant ratio of r_1^n . Although the exact ratio f_n/r_1^n varies, it is constrained to lie between lower and upper bounds that are constant, i.e., these bounds do not depend on n .

This tells us that the growth of f_n , as n increases, is very like the growth of r_1^n .

6.7.4 An exact formula

We have managed to “sandwich” f_n quite tightly between lower and upper bounds that grow in the same way as n increases. Encouraged by this success, we try now to get the two bounds to coincide, so that we obtain *an exact formula* for f_n .

When we looked for upper and lower bounds of the form r^n , we used the quadratic equation

$$r^2 - r - 1 = 0. \quad (6.27)$$

The intuition behind this is that any r that satisfies this equation also satisfies

$$r^n = r^{n-1} + r^{n-2}, \quad (6.28)$$

so that r^n actually satisfies the Fibonacci recurrence (where each term is the sum of the two previous terms), although dealing with the base case is a separate matter.

We took the largest root $r_1 = (1 + \sqrt{5})/2$, which is also the only positive root. For upper bounds, we chose r to be $\geq r_1$, while for lower bounds, we chose r to be $\leq r_1$. (For the lower bound, we also needed $r \geq r_2$, but this constraint did not rear its head at us, as the values we considered were so close to r_1 .) In each case, we observed that we could actually take r to be *equal* to r_1 , thereby using the same value, r_1 , in both upper and lower bounds. The distinction between upper and lower bounds came down to our choice of constant factors, α , and these were determined by the need for our bounds to work on the base cases $n = 1$ and $n = 2$.

This *suggests* that an expression of the form

$$\alpha_1 r_1^n,$$

with suitable choice of constant factor α_1 , might be a very good *estimate* for f_n . But it won't be exact, so we need something else too.

We have neglected the smaller root, r_2 , so far. But it does provide us with *something else* that satisfies the recurrence relation, because

$$r_2^n = r_2^{n-1} + r_2^{n-2},$$

by (6.28). So, if we add some multiple of r_2^n to any expression satisfying the Fibonacci recurrence, then the new expression will also satisfy the Fibonacci recurrence (although not necessarily the base cases).

Suppose, then, that our exact formula has contributions not just of the form r_1^n , using the larger root r_1 of (6.27), but also of the form r_2^n , using the smaller root r_2 . Give each of these contributions its own factor; call these factors α_1 and α_2 , respectively. Then we seek a formula of the form

$$f_n = \alpha_1 r_1^n + \alpha_2 r_2^n.$$

We now have two constants to vary, namely α_1 and α_2 , in our quest for an exact expression for f_n . This seems like progress, because the two base cases give us two conditions to satisfy. Previously, when we only looked for expressions of the form $\alpha_1 r_1^n$, we had only one constant to play with, yet we had two base cases to attend to, for f_1 and f_2 . That was ok when we only wanted bounds (lower or upper); in that situation, it's fine if we only get inequality (rather than equality) in one of the base cases, as long as the inequality goes in the right direction (and we were always able to achieve this). But if we want an exact formula for f_n , then we need the formula to work exactly for both the base cases. That's *two* equations ($f_1 = 1$, $f_2 = 1$), so it's a good idea to have *two* variables to solve for. Let's see what α_1 and α_2 can do for us.

What should α_1 and α_2 be? Consider the two base cases. When $n = 1$, we have $f_1 = 1$, so we need

$$\alpha_1 r_1 + \alpha_2 r_2 = 1.$$

When $n = 2$, we have $f_2 = 1$, so we need

$$\alpha_1 r_1^2 + \alpha_2 r_2^2 = 1.$$

Here we have two linear equations in the two unknowns α_1 and α_2 . We then use our favourite technique for solving such systems. Once we have done so, we find that

$$\alpha_1 = \frac{1}{\sqrt{5}}, \quad \alpha_2 = \frac{-1}{\sqrt{5}}.$$

So we propose the formula

$$f_n = \sqrt{5}^{-1} r_1^n - \sqrt{5}^{-1} r_2^n. \quad (6.29)$$

Given the fame of the Fibonacci numbers, it is worth giving this in full detail too:

$$f_n = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2} \right)^n. \quad (6.30)$$

We now prove by induction that it works. Once again, the new or modified parts of the proof are in [blue](#).

Inductive Basis:

- $n = 1$: we have $f_1 = 1$, and our [formula](#) gives $\sqrt{5}^{-1} r_1^1 - \sqrt{5}^{-1} r_2^1 = \sqrt{5}^{-1} (r_1 - r_2) = \sqrt{5}^{-1} \sqrt{5} = 1$, so the [formula](#) works [exactly](#).
- $n = 2$: we have $f_2 = 1$, and our [formula](#) gives $\sqrt{5}^{-1} r_1^2 - \sqrt{5}^{-1} r_2^2 = \sqrt{5}^{-1} (r_1^2 - r_2^2) = \sqrt{5}^{-1} (r_1 + r_2)(r_1 - r_2) = \sqrt{5}^{-1} \cdot 1 \cdot \sqrt{5} = 1$, so the [formula](#) works [exactly](#) again.

Inductive Step:

Let $k \geq 2$.

Assume that $f_l = \sqrt{5}^{-1}r_1^l - \sqrt{5}^{-1}r_2^l$ for all $l \leq k$. (This is the Inductive Hypothesis.)

Now consider f_{k+1} . Since $k \geq 2$, we have $k+1 \geq 3$, so we can apply the recursive rule (6.12):

$$\begin{aligned}
 f_{k+1} &= f_k + f_{k-1} && \text{(by (6.12), and using } k \geq 2) \\
 &= (\sqrt{5}^{-1}r_1^k - \sqrt{5}^{-1}r_2^k) + (\sqrt{5}^{-1}r_1^{k-1} - \sqrt{5}^{-1}r_2^{k-1}) \\
 &&& \text{(by the Inductive Hypothesis, twice)} \\
 &= (\sqrt{5}^{-1}r_1^k + \sqrt{5}^{-1}r_1^{k-1}) - (\sqrt{5}^{-1}r_2^k + \sqrt{5}^{-1}r_2^{k-1}) \\
 &&& \text{(rearranging, to collect like roots together)} \\
 &= \sqrt{5}^{-1}(r_1^k + r_1^{k-1}) - \sqrt{5}^{-1}(r_2^k + r_2^{k-1}) \\
 &= \sqrt{5}^{-1}r_1^{k+1} - \sqrt{5}^{-1}r_2^{k+1} && \text{(by (6.21) and (6.22)).}
 \end{aligned}$$

This completes the Inductive Step.

Conclusion:

Therefore, by Mathematical Induction, our formula (6.29) holds for all n .

We now have an exact formula for the n -th term of the Fibonacci sequence. The formula itself is certainly not obvious. In fact, if you only know the standard recursive definition and then this formula is presented to you “out of the blue”, then it would probably seem mysterious and complicated, and you would wonder how anyone came up with it. The appearance of the irrational numbers

$$\frac{1 + \sqrt{5}}{2}, \quad \frac{1 - \sqrt{5}}{2}$$

would probably seem strange, especially as all the terms in the sequence are positive integers.

Hopefully this section has removed some of the mystery, and equipped you with the skills to derive formulas for some other recursively-defined sequences. You can see now that those two strange irrational numbers are just the two roots of the equation

$$r^2 - r - 1 = 0, \tag{6.31}$$

which we used in (6.21) and (6.27). And this equation comes directly from the recurrence relation defining the sequence: a simple rearrangement of the recurrence $f_n = f_{n-1} + f_{n-2}$ gives

$$f_n - f_{n-1} - f_{n-2} = 0. \tag{6.32}$$

Compare (6.31) and (6.32). Although they are different kinds of equations (the former is a quadratic equation in a single real variable, the latter is a linear equation for any three successive terms in a sequence), they also have a common pattern. In each case, the coefficients on the left-hand side are the same: 1, -1 , -1 , and the right-hand side is 0. In each case, the three terms may be viewed as being “counted” or “indexed” by three consecutive decreasing integers: for (6.31), the exponents are 2, 1, 0, while for (6.32), the subscripts are n , $n-1$, $n-2$. We say that (6.31) is the **characteristic equation** for the recurrence relation (6.32).

This is one instance of a general method for solving recurrences: first convert the recurrence to its characteristic equation, then find the roots of that equation, and then try to express the n -th term as a sum, with suitable coefficients, of n -th powers of the roots of the characteristic equation. It does not always work quite like this; in particular, there are some complicating details to attend to when some root of the characteristic equation occurs more than once. But this link, between the recurrence and the roots of the characteristic equation, lies at the heart of a general method that can be used to solve any linear recurrence relation. We will not give the general method in full here; it is covered in more advanced courses on discrete mathematics.

An important aspect of the expression for the n -th Fibonacci number, (6.30), is the dominant role played by

$$\left(\frac{1+\sqrt{5}}{2}\right)^n.$$

Of the two roots $(1 \pm \sqrt{5})/2$ of (6.31), this one is the greater in size, so its n -th power will grow much larger than the n -th power of the other root. Furthermore, the size of that other root is < 1 , so its n -th power rapidly approaches 0, so it really provides only a small adjustment to the number calculated. So Fibonacci numbers will be well approximated, for large n , by

$$f_n \approx \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2}\right)^n.$$

This quantity is never an integer, but it does make the nature of the growth of f_n clear. It could be said that $(f_n : n \in \mathbb{N})$ is *approximately* geometric for large n , and it shows that the ratio between successive terms, as $n \rightarrow \infty$, tends to $(1 + \sqrt{5})/2$.

These observations align with our discussion in § 6.7.3, where we observed that the sequence grows like r_1^n , where $r_1 = (1 + \sqrt{5})/2$.

This *limiting ratio* of the Fibonacci numbers, $(1 + \sqrt{5})/2$, is the famous quantity known as the **golden ratio**, often denoted by φ or τ . In decimal form, it is

$$1.6180339887\dots$$

It is rich in mathematical properties and pops up all over the place, not only in computer science but also in nature and in art. It has an important geometric interpretation: if

sequence	list of terms	eventual behaviour
$\left(\frac{1}{n}\right)_{n=1}^{\infty}$	$1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \frac{1}{5}, \dots$	approaches 0 arbitrarily closely
$\left(\frac{(-1)^n}{n}\right)_{n=1}^{\infty}$	$-1, \frac{1}{2}, -\frac{1}{3}, \frac{1}{4}, -\frac{1}{5}, \dots$	approaches 0 arbitrarily closely
$\left(1 - \frac{1}{n}\right)_{n=1}^{\infty}$	$0, \frac{1}{2}, \frac{2}{3}, \frac{3}{4}, \frac{4}{5}, \dots$	approaches 1 arbitrarily closely
$\left(1 - \frac{1}{2^n}\right)_{n=1}^{\infty}$	$0, \frac{1}{2}, \frac{3}{4}, \frac{7}{8}, \frac{15}{16}, \dots$	approaches 1 arbitrarily closely
$\left((-1)^n - \frac{(-1)^n}{n}\right)_{n=1}^{\infty}$	$0, \frac{1}{2}, -\frac{2}{3}, \frac{3}{4}, -\frac{4}{5}, \frac{5}{6}, \frac{6}{7}, \dots$	odd terms approach -1 , even terms approach 1 .

Table 6.1: Some sequences and their eventual behaviour.

you take a rectangle whose side lengths are in this ratio (i.e., with the golden ratio as its aspect ratio), and you trim it by one straight cut to create a square based on the shorter side of the rectangle, then the smaller rectangle you trim off has the same proportions as the one you started with. Such a rectangle is called a **golden rectangle**.

6.8 LIMITS OF INFINITE SEQUENCES

Given a number sequence, we can ask, how does it behave *in the long run*? What can we say about how it grows (or declines) as n gets very large? We have considered this already for the sequence of reciprocals of positive integers (§ 6.5) and the Fibonacci sequence (§ 6.7).

Sequences are very diverse, and various kinds of behaviour are possible. Some sequences increase without bound, such as $(n)_{n=1}^{\infty}$ or $(n^2)_{n=1}^{\infty}$ or $(\log n)_{n=1}^{\infty}$. Others decrease without bound, such as $(-n)_{n=1}^{\infty}$ or $(-n^2)_{n=1}^{\infty}$ or $(-\log n)_{n=1}^{\infty}$. Others jump around; these may be bounded both above and below, like $((-1)^n)_{n=1}^{\infty}$, which starts $-1, 1, -1, 1, \dots$, or unbounded above and below, like $((-1)^n n)_{n=1}^{\infty}$, which starts $-1, 2, -3, 4, \dots$, or bounded above and unbounded below, or unbounded above and bounded below.

Some sequences eventually “settle down” in such a way that, beyond a certain point, the terms look like approximations to some specific number, with these approximations getting arbitrarily close. Consider the examples in Table 6.1.

Intuitively, the *limit* of a sequence is a number which the sequence’s terms get closer and closer to, forever, and the terms get *arbitrarily close*, meaning that however close

you want them to be, they eventually get that close or closer, and stay that close or closer from some point onwards.

Informally, the **limit** of a sequence $(a_n)_{n=1}^{\infty}$ is a number ℓ such that, however close to ℓ you want to get, there is a position in the sequence such that all terms beyond that position are within that distance of ℓ . You can set your required distance from ℓ , which we call ε , to be as small as you like (except it can't be 0), and there is always some position N such that *all terms* from that position onwards are within that distance ε of ℓ .

Formally, the sequence $(a_n)_{n=1}^{\infty}$ has **limit** ℓ , and we write

$$\lim_{n \rightarrow \infty} a_n = \ell,$$

if for all $\varepsilon > 0$ there exists $N \in \mathbb{N}$ such that for all $n \geq N$ we have

$$|a_n - \ell| < \varepsilon.$$

We can write this very succinctly and symbolically if we wish:

$$\lim_{n \rightarrow \infty} a_n = \ell \quad \Longleftrightarrow \quad \forall \varepsilon > 0 \exists N \in \mathbb{N} \forall n \geq N |a_n - \ell| < \varepsilon.$$

It might help to study carefully how the various parts of this formal definition align with the intuitive description we gave earlier.

$$\lim_{n \rightarrow \infty} a_n = \ell \quad \text{means:}$$

in symbols

in words

$\forall \varepsilon > 0$	<i>for any</i> desired distance from ℓ , no matter how small,
$\exists N \in \mathbb{N}$	<i>there is</i> a point in the sequence such that,
$\forall n \geq N$	<i>for any</i> position beyond that point,
$ a_n - \ell < \varepsilon.$	the term at that position is within the desired distance from ℓ .

For example, suppose $a_n = 1/n$. (See § 6.5 and, later, § 6.15 for discussion of this important sequence.) We claim that its limit, as $n \rightarrow \infty$, is 0. This makes sense intuitively, as these numbers $1/n$ get smaller and smaller, getting as close to 0 as you like. Let us see how this notion aligns with the definition given in the previous paragraph. Pick any positive real number ε , as our measure of how close to 0 we want to get. How far along the sequence do we have to go, to get *and stay* that close, or closer? This is the role of N , and this depends on ε . In this case, we can pick N to be any positive integer greater than $1/\varepsilon$:

$$N > \frac{1}{\varepsilon}, \quad N \in \mathbb{N}. \quad (6.33)$$

This ensures that, for any $n \geq N$, we also have

$$n > \frac{1}{\varepsilon},$$

which is equivalent to

$$\frac{1}{n} < \varepsilon.$$

Therefore, for any ε , there does indeed exist N — namely, any positive integer $> 1/\varepsilon$ — such that

$$\begin{aligned} n \geq N &\Rightarrow n > \frac{1}{\varepsilon} && (\text{because } N > \frac{1}{\varepsilon}, \text{ by (6.33)}) \\ &\Rightarrow \frac{1}{n} < \varepsilon \\ &\Rightarrow \left| \frac{1}{n} - 0 \right| < \varepsilon. \end{aligned}$$

The last inequality here says that the distance between the term $a_n = 1/n$ and 0 is $< \varepsilon$.

So we have shown that

$$\lim_{n \rightarrow \infty} \frac{1}{n} = 0. \quad (6.34)$$

For another example, suppose $a_n = n/(n+1)$. What is its limit as $n \rightarrow \infty$? Consider the first few terms:

$$\frac{1}{2}, \frac{2}{3}, \frac{3}{4}, \frac{4}{5}, \dots$$

From these, it looks like the terms approach 1 from below. In general, the numerator and denominator of $n/(n+1)$ are very similar, differing only by 1, and this difference should matter less and less as n gets larger and larger. So, intuitively, we might expect the limit as $n \rightarrow \infty$ to be 1. We now show that this is indeed the case.

Let $\varepsilon > 0$. We want to make n large enough to ensure that

$$\left| \frac{n}{n+1} - 1 \right| < \varepsilon. \quad (6.35)$$

On the left-hand side here, we are taking the absolute value of a negative quantity, since $n/(n+1) < 1$. This observation helps us remove the absolute value function and then we can simplify using algebra:

$$\begin{aligned} \left| \frac{n}{n+1} - 1 \right| &= 1 - \frac{n}{n+1} \\ &= \frac{(n+1) - n}{n+1} \\ &= \frac{1}{n+1}. \end{aligned}$$

So, to ensure that (6.35) holds, it would be sufficient to ensure that

$$\frac{1}{n+1} < \varepsilon.$$

But this is equivalent to

$$n+1 > \frac{1}{\varepsilon}$$

which in turn is equivalent to

$$n > \frac{1}{\varepsilon} - 1.$$

We can now write our limit proof. Given ε , we define N to be a positive integer $> (1/\varepsilon) - 1$. Then, for any $n \geq N$, we have $n > (1/\varepsilon) - 1$ too. But, as we have just seen, this is equivalent to

$$\frac{1}{n+1} < \varepsilon$$

which in turn is equivalent to

$$\left| \frac{n}{n+1} - 1 \right| < \varepsilon.$$

Since this inequality holds for all $n \geq N$, we have completed the proof that

$$\lim_{n \rightarrow \infty} \frac{n}{n+1} = 1.$$

For yet another example, suppose $a_n = 1/2^n$. Again, we claim the limit as $n \rightarrow \infty$ is 0. Pick any $\varepsilon > 0$. To get $1/2^n$ to be $< \varepsilon$, we need $1/2^n < \varepsilon$, which is equivalent to $2^n > 1/\varepsilon$. Taking logarithms of each side, we obtain $n > \log_2(1/\varepsilon)$. So we can take any positive integer N satisfying

$$N > \log_2(1/\varepsilon), \quad N \in \mathbb{N}. \quad (6.36)$$

Having chosen this N , any $n \geq N$ satisfies $n > \log_2(1/\varepsilon)$, and therefore satisfies $1/2^n < \varepsilon$. So

$$n \geq N \quad \Rightarrow \quad \left| \frac{1}{2^n} - 0 \right| < \varepsilon.$$

So we have

$$\lim_{n \rightarrow \infty} \frac{1}{2^n} = 0. \quad (6.37)$$

Now let r be any real constant in the range $-1 < r < 1$, and suppose $a_n = r^n$. Now what is $\lim_{n \rightarrow \infty} a_n$? The previous example is the case $r = 1/2$, and the argument given there can be adapted to handle any r in the range $-1 < r < 1$.

The algebra is neater when $0 < r < 1$, so we do that first. Given $\varepsilon > 0$, choose positive integer $N > \log_{1/r}(1/\varepsilon)$. Then, for any $n \geq N$, we have

$$n > \log_{1/r}(1/\varepsilon).$$

Raising $1/r$ to the power of each side, we have

$$(1/r)^n > 1/\varepsilon.$$

Taking the reciprocal of each side, which entails reversing the inequality, gives

$$r^n < \varepsilon$$

which immediately implies

$$|r^n - 0| < \varepsilon.$$

Since this holds for all $n \geq N$, we have proved that

$$\lim_{n \rightarrow \infty} r^n = 0. \quad (6.38)$$

The case $r = 0$ is trivial, since then we have $r^n = 0^n = 0$ for all n , so the limit is clearly 0 because the *value* is *always* 0.

If $-1 < r < 0$, then $0 < -r < 1$, so we can apply our earlier observation (6.38), with $-r$ instead of r , to deduce that

$$\lim_{n \rightarrow \infty} (-r)^n = 0.$$

This sequence $((-r)^n)_{n=1}^{\infty}$ consists of positive terms that decrease towards 0, approaching 0 in the limit as $n \rightarrow \infty$. It follows that the sequence of its negations, $(-(-r)^n)_{n=1}^{\infty}$, consists entirely of negative terms that increase towards 0, with

$$\lim_{n \rightarrow \infty} -(-r)^n = 0.$$

Now the sequence we are interested in, $(r^n)_{n=1}^{\infty}$, is “sandwiched” between these two sequences: each of its terms r^n satisfies

$$-(-r)^n \leq r^n \leq (-r)^n.$$

It follows that the terms r^n have no choice but to also converge on 0 as $n \rightarrow \infty$. They alternate between being positive and negative, but their sizes decrease towards 0 and approach it in the limit.:

$$\lim_{n \rightarrow \infty} r^n = 0.$$

In conclusion, we have

Theorem 25. If $-1 < r < 1$ then

$$\lim_{n \rightarrow \infty} r^n = 0.$$

We will use this fact later, when we consider sums of infinite geometric sequences (§ 6.14).

Another important limit, which takes a bit more effort to prove, is the limit of an n -th root (rather than an n -th power, as above). If $r > 0$ then we use $r^{1/n}$ to refer to the sole positive n -th root of r .

Theorem 26. If $r > 0$ then

$$\lim_{n \rightarrow \infty} r^{1/n} = 1.$$

The condition $r > 0$ is needed because for $r < 0$ taking n -th roots can involve complex numbers and there may be no positive root or no real root at all.

Once we have established some basic limits like these, we can use them to derive other limits using standard principles for combining limits. One of these principles is:

The limit of a sum is the sum of the limits.

If we have two sequences $(a_n : n \in \mathbb{N})$ and $(b_n : n \in \mathbb{N})$, and

$$\lim_{n \rightarrow \infty} a_n = a, \quad \lim_{n \rightarrow \infty} b_n = b,$$

then the sequence $(a_n + b_n : n \in \mathbb{N})$, formed by adding the corresponding terms in those two sequences together, has limit $a + b$:

$$\lim_{n \rightarrow \infty} (a_n + b_n) = a + b.$$

Similarly,

$$\begin{aligned} \lim_{n \rightarrow \infty} (a_n - b_n) &= a - b, \\ \lim_{n \rightarrow \infty} (a_n b_n) &= ab, \\ \lim_{n \rightarrow \infty} (a_n / b_n) &= a/b, \end{aligned}$$

with the last one requiring that $b_n \neq 0$ for all n and also $b \neq 0$.

For example,

$$\begin{aligned} \lim_{n \rightarrow \infty} (3^{1/n} + 3^{-n}) &= \lim_{n \rightarrow \infty} (3^{1/n} + (\tfrac{1}{3})^n) \\ &= \lim_{n \rightarrow \infty} 3^{1/n} + \lim_{n \rightarrow \infty} (\tfrac{1}{3})^n \\ &= 1 + 0 \quad (\text{by Theorem 25 and Theorem 26}) \\ &= 1. \end{aligned}$$

Some sequences increase without bound. They have no finite limit, and not even any finite upper bound. In such cases, we can say that their limit, as $n \rightarrow \infty$, is ∞ . For example,

$$\lim_{n \rightarrow \infty} n^2 = \infty.$$

If a sequence at some point goes below 0 and keeps decreasing without any finite lower bound, then we can say that its limit, as $n \rightarrow \infty$, is $-\infty$. For example,

$$\lim_{n \rightarrow \infty} -n^2 = -\infty.$$

We are often interested in sequences that grow without bound. This is the typical situation for the running time of a nontrivial algorithm, as a function of some positive integer measure n of the input size. (For example, n might be the number of bytes in an input file, or the number of names in a list to be sorted, or the number of digits in a number to be factorised.) In this kind of situation, just saying that the limit is ∞ doesn't say much and isn't very useful. For example, in different situations you may have algorithms that take time n , or n^2 , or $\sqrt{n}$, $\log n$, or $n \log n$, or 2^n , or $n!$, or 2^{2^n} . These are wildly different running times, but stating their limits does not reflect this:

$$\begin{aligned} \lim_{n \rightarrow \infty} \log n &= \infty, \\ \lim_{n \rightarrow \infty} \sqrt{n} &= \infty, \\ \lim_{n \rightarrow \infty} n &= \infty, \\ \lim_{n \rightarrow \infty} n \log n &= \infty, \\ \lim_{n \rightarrow \infty} n^2 &= \infty, \\ \\ \lim_{n \rightarrow \infty} 2^n &= \infty, \\ \lim_{n \rightarrow \infty} n! &= \infty, \\ \lim_{n \rightarrow \infty} 2^{2^n} &= \infty. \end{aligned}$$

To talk intelligently about the running time of an algorithm as n grows, it's not enough to just say "it keeps growing without bound", which is all we are saying here. We need to be able to say something that indicates *how* it grows.

6.9 BIG-O NOTATION

Many sequences we encounter in computer science describe quantities that grow without bound as n increases. As we saw at the end of the previous section, just stating that their limit is ∞ doesn't really capture *how* they grow. In this section, we describe

standard notation used throughout computer science for *the really important part* of the growth behaviour of a sequence.

Functions like n^2 and $\log n$ are simple and well-known, so that the growth of sequences based on them is well understood. But sequences in computer science are often significantly more complicated. In particular, the exact running times of a program can be very complicated indeed, in fact it is often not possible to write down an exact formula for it. But we still need to quantify how the costs of running the program grow as the input gets larger and larger.

Suppose we have a sequence $(t_n : n \in \mathbb{N})$ whose n -th term is given by

$$t_n = 100n + 10n^2 + 2^n + \log n.$$

This might conceivably be the running time of a program that has four successive stages, with these stages taking time $100n$, $10n^2$, 2^n and $\log n$, respectively (where n is the input size). This expression for t_n has four summands, each contributing to the growth of this sequence. You can use a spreadsheet or program to study how each summand grows as n increases. You'll find that, once $n \geq 10$, the summand 2^n is greater than each of the others. As n grows beyond that, 2^n rapidly dwarfs the others. For $n = 20$, this summand is $> 1,000,000$, but the other summands are each $\leq 4,000$.

So we can say that the growth of t_n as $n \rightarrow \infty$ is dominated by the growth of 2^n .

If this summand had been $3 \cdot 2^n$ instead of 2^n , then it would be even more dominant. But the constant factor 3 is relatively unimportant compared with 2^n . In fact, regardless of the constant, this would still have been the dominant summand. Even if the constant factor had been 0.01, as in

$$t_n = 100n + 10n^2 + 0.01 \cdot 2^n + \log n,$$

that summand $0.01 \cdot 2^n$ would eventually dominate the others. (Try $n \geq 20$.)

For sequences representing running times of programs, it might be infeasible to pin down the exact running time. It might just be too complicated, or it might depend on some information we don't know in advance, like the exact way the various data structures are laid out in memory. But we still need to be able to make well-founded quantitative statements about how long programs take.

It is important to be able to make *positive* statements, like

This program takes at most time 2^n .

This identifies something the program can be guaranteed to do, and in particular it guarantees that a certain quantity of a key resource — in this case, time — is *sufficient* for the program to do its job, even if it might actually use less. Such statements are useful when budgeting computational resources and when estimating how much more time the program might take as input sizes grow.

Another issue is that we don't want to be unduly distracted by behaviour for small n . It is common for some small inputs to need some kind of special treatment, which

means that the time taken to deal with them is not typical of behaviour on larger inputs. Sequences often take some time to “settle down” into their eventual pattern of behaviour; we have seen this already with the Fibonacci sequence (§ 6.7).

So, in summary, given a sequence of growing terms, we would like to make statements about its growth that

- focus on its eventual behaviour,
- focus on the dominant contribution, and
- give an upper bound.

With this motivation, we make the following definition.

Let $(a_n : n \in \mathbb{N})$ be any sequence of nonnegative real numbers, and let $f : \mathbb{N} \rightarrow \mathbb{R}$. We write

$$a_n = O(f(n)),$$

and read it as “ a_n is big-O $f(n)$ ”, if there is a constant c and a positive integer N such that, for all $n \geq N$,

$$a_n \leq c \cdot f(n).$$

We can express this using quantifiers:

$$a_n = O(f(n)) \quad \Longleftrightarrow \quad \exists c, N \forall n \geq N \quad a_n \leq c \cdot f(n).$$

For example, consider again

$$t_n = 100n + 10n^2 + 2^n + \log n.$$

We observed earlier that each of the other summands is $\leq 2^n$ when $n \geq 10$. So, for $n \geq 10$, we have

$$100n + 10n^2 + \cdot 2^n + \log n \leq 2^n + 2^n + 2^n + 2^n = 4 \cdot 2^n.$$

Referring to the definition of big-O notation, we can take $N = 10$ and $c = 4$, because for all $n > 10$, we have $t_n \leq 4 \cdot 2^n$. So we can write

$$t_n = O(2^n). \tag{6.39}$$

We could also have written

$$t_n = O(4 \cdot 2^n)$$

or

$$t_n = O(0.001 \cdot 2^n).$$

These are mathematically correct, but poor use of big-O notation because they include constant factors. One key point about big-O notation is that it “swallows up” constant factors, so there’s no need to state such factors explicitly. In this case, observe that

$$2^n = O(0.001 \cdot 2^n),$$

because $2^n \leq 1000 \cdot (0.001 \cdot 2^n)$, so the definition of big-O is satisfied in this case with $c = 1000$. Since any constant factor can be included inside the big-O, i.e.,

$$t^n = O(b \cdot 2^n)$$

for any constant b , it is simplest to just omit the constant factor (or use $b = 1$) and focus on the part that depends on n .

It is also correct to write

$$t_n = O(3^n),$$

since $2^n \leq 3^n$. This is unnecessarily loose, and in general we prefer tighter big-O bounds since they give us stronger claims. But it is sometimes very hard to give very tight upper bounds, so sometimes we have to be content with looser bounds (which is good for big-O because it is so forgiving towards constant factors).

Although we can replace 2^n by 3^n in (6.39) and still have a true statement (albeit a weaker one), we *cannot* replace 2^n by, say, 1.9^n (or use any other base smaller than 2). This is because there is no constant c such that $c \cdot 1.9^n$ eventually gets (and says) bigger than 2^n .

Coming up with big-O expressions for sequences is often a matter of picking the dominant summand and dropping its constant factor. We saw this in our study of t_n above, (6.39). Some other examples of this type:

$$\begin{aligned} \frac{1}{8}n^3 + \frac{1}{4}n^2 + \frac{1}{2}n + 1 &= O(n^3), \\ 3n^{1/3} + 2n^{1/2} &= O(n^{1/2}), \\ \log n + \log(\log n) + \log(\log(\log n)) &= O(\log n), \\ 100^n + 2^{n^2} &= O(2^{n^2}), \\ 5 + n^{-1} &= O(1), \\ \frac{1}{n^4} + \frac{1}{n^3} + \frac{1}{n^2} &= O\left(\frac{1}{n^2}\right). \end{aligned}$$

If the sequence term is a product of sums, then you can find the dominant term in each sum and then multiply them, simplifying as appropriate. For example:

$$(\sqrt{n} + 3n + 9^{1/n})(17 + \frac{1}{2}\log n) = O(n \log n).$$

Some useful big-O examples with logarithms:

$$\begin{aligned}\log(n^2) &= O(\log n) && (\text{because } \log(n^2) = 2\log n) \\ \log(n^2 + n^3 + n^4) &= O(\log n) \\ \log(3^n) &= O(n) && (\text{because } \log(3^n) = n\log 3, \text{ and } \log 3 \text{ is a constant}).\end{aligned}$$

6.10 SUMS AND SUMMATION

For number sequences, we are often interested in the sizes and behaviour of sums of terms, as well as just the individual terms themselves. For example, if the n -th term s_n of a sequence gives the energy consumed during the n -th second of some computation, then we may also want to study the total energy consumed so far, at each time. This can be done using the sum, $S_n = s_1 + s_2 + \cdots + s_n$, of the first n terms of the sequence. These cumulative sums S_n give us a new sequence, with each term S_n of the new sequence giving the total energy used during all the first n seconds of the computation.

Let M be a set of indices for a sequence, so $M = \mathbb{N}$ for an infinite sequence and $M = [1, k]_{\mathbb{N}}$ for a finite sequence of k terms.

If $(s_n)_{n \in M}$ is any number sequence, then its **sequence of partial sums** is the sequence $(S_n)_{n \in M}$ where, for each n , the term S_n is defined by

$$S_n = s_1 + \cdots + s_n. \quad (6.40)$$

Since we will be working with partial sums like this a lot now, it is time to introduce **summation notation**. The expression

$$\sum_{i=1}^n s_i$$

is an abbreviation for the sum on the right-hand side of (6.40). Let us study it closely.

- The $\sum$ is a large upper-case Greek sigma. It is called the **summation sign** and stands for sum.²
- At the base of the summation sign we see “ $i = 1$ ”. On the left-hand side of this equation we have i , which is the **index of summation** or **variable of summation**. This will be varied as part of a specification of what things are to be added up. On the right of “ $i = 1$ ” we have 1, which gives the initial value of i , which means the *very first value* we give to i when we are specifying the things to be added up.
- At the top of the summation sign we see n , which is the *very last value* we give to i when forming our sum.

² This notation was introduced by Leonhard Euler in 1755. The Greek letter sigma is the Greek equivalent of the English ‘s’, the first letter of “sum”.

- Taken together, this information specifies the **range of summation**, namely $[1, n]_{\mathbb{N}}$. So the understanding is always that the variable of summation is *incremented* from the first value, 1, up to the last value, n , and that every integer value in that range, $1 \leq i \leq n$, is a value of i that is to be used for the sum.
- But *how* is each of these values of i to be used in forming the sum? This is specified by the **summand** (i.e., the *thing being added*), which comes *after* the summation sign (i.e., to its right), and in this case is s_i . The summand includes the variable of summation, in this case in its subscript.
- By substituting all numbers in the range of summation into the variable of summation, we obtain different values of the summand, and we add all these different values up in order to obtain the whole sum.

In this case, the range of summation is $\{1, 2, \dots, n\}$, so the variable of summation i in the summand s_i is given each of these values. This gives the summand values

$$s_1, s_2, \dots, s_n,$$

and these are all added up, yielding

$$\sum_{i=1}^n s_i = s_1 + s_2 + \dots + s_n.$$

So we can rewrite (6.40) as

$$S_n = \sum_{i=1}^n s_i.$$

If summation notation seems new to you, be assured that, to a programmer, it is not as new as it may seem. You do something quite similar when you write a loop to add up some numbers in a program. This typically has structure like the following:

```
sum := 0
for each  $i$  from 1 to  $n$ :
    sum := sum +  $s_i$ 
 $S_n$  := sum
```

Just like summation notation, this specifies

- a variable of summation i ,
- a range of summation $\{1, \dots, n\}$, and
- a summand s_i .

There are also some differences with summation notation.

- What we have just given is really a short algorithm (or part of a program in some programming language), so it does more than just define a sum mathematically; it also specifies how to compute it. In particular, it specifies a particular order in which the summands are added, and uses a name (sum) for the partial sums, and initialises the partial sum at the start. By contrast, the summation notation does not specify an order of addition. Even though it is natural to add the summands in order of increasing i , there is no *requirement* to do it in that order, or any other order. The summation specifies the mathematical result of doing the sum without assuming anything about how it is computed. Since addition is associative and commutative, the sum is the same regardless of the order of the additions.
- Our summation algorithm, if implemented on a computer, would use computer representations of numbers rather than numbers themselves, and these introduce approximations and errors. These approximations also mean that the computer's version of addition is not necessarily associative, so that the order of addition *can* matter.

The name used for the variable of summation is not important, as long as it is used consistently. The variable used under the summation sign (the i in “ $i = 1$ ”) and the variable of that same name in the summand (the subscript i in s_i) are the same variable, as you'd expect. If you want to change the variable name, say to j , then you can, provided you change it everywhere: both under the summation sign, and in the summand. So the sum

$$\sum_{i=1}^n s_i$$

could, for example, be rewritten as

$$\sum_{j=1}^n s_j \quad \text{or} \quad \sum_{t=1}^n s_t \quad \text{or} \quad \sum_{\alpha=1}^n s_{\alpha} \quad \text{or} \quad \sum_{\heartsuit=1}^n s_{\heartsuit}.$$

It is common to use i, j, k, l, m, n as variables of summation, and adherence to conventions can help readers of your work, just as it does with programming. But it should not be stuck to, rigidly, just for its own sake, and there are certainly times when other variables make more sense in a particular context.

Variables of summation are *local to the summation*. In other words, their *scope* is restricted to that summation sign and summand, and if they are used outside the sum, then that outside use is *not the same variable* and should be avoided because of the confusion it can cause. So, in the expression

$$i + \sum_{i=1}^n s_i,$$

we really have two different variables called i : the local variable of summation used in the sum, and the i on the very left. In this case, we have not made the meaning of the first i clear, but even if we had, there is possibility of confusion about what the variables refer to, and this is to be avoided. There is no shortage of variable names: there are 26 letters of the English alphabet and 24 in the Greek alphabet, each with upper case versions too!³ The same issue arises if we write

$$\left(\sum_{i=1}^n s_i \right) + i. \quad (6.41)$$

Here, the i on the right is not the variable of summation. If for some reason we *must* use i on the right here, then it would be better to use a different variable of summation, such as j , and this is easily done. This particular expression (6.41) also raises another issue. The parentheses here make clear that the summand is only s_i . But what if the parentheses are omitted?

$$\sum_{i=1}^n s_i + i$$

Is this the same as (6.41), or is it a different sum in which the summand is now $s_i + i$? As it is, the expression is ambiguous, so we should use parentheses to make it clear. This was done in (6.41), imposing one interpretation, and if we wanted to impose the other possible interpretation, then we could write

$$\sum_{i=1}^n (s_i + i).$$

In this expression, each successive value of i in the range of summation, $\{1, \dots, n\}$ is substituted for i in *both places* in the summand where i appears. So what we obtain is the sum

$$(s_1 + 1) + (s_2 + 2) + \cdots + (s_n + n).$$

6.11 FINITE SERIES

A **finite series** is an expression obtained from a finite sequence by inserting $+$ between each pair of consecutive terms. We can think of this as replacing the commas in the list of terms by additions. So a finite sequence

$$a_1, a_2, a_3, \dots, a_n$$

gives rise to the finite series

$$a_1 + a_2 + a_3 + \cdots + a_n.$$

³ although there is significant overlap between the two upper-case alphabets.

The **value** of the series is the number given by the sum.

6.12 FINITE ARITHMETIC SERIES

A **finite arithmetic series** is a finite series obtained from an arithmetic sequence. If the arithmetic sequence has n terms, first term a , and common difference d , then, as we saw in (6.8), its list of terms is

$$a, \quad a+d, \quad a+2d, \quad \dots, \quad a+(n-1)d.$$

So the corresponding finite arithmetic series is

$$a + (a+d) + (a+2d) + \dots + (a+(n-1)d). \quad (6.42)$$

What is the sum S_n of this series?

One way to work this out starts with two copies of the series, one above the other, but with the second copy in reverse order, so high terms are vertically aligned with low terms:

$$\begin{array}{ccccccccc} S_n & = & a & + & (a+d) & + & \dots & + & (a+(n-2)d) & + & (a+(n-1)d) \\ S_n & = & (a+(n-1)d) & + & (a+(n-2)d) & + & \dots & + & (a+d) & + & a \end{array}$$

Now we add these two equations. The sum of the left-hand sides is $2S_n$. What is the sum of the right-hand sides? We have arranged the terms on the right so that each column of two terms adds to the same sum, namely $2a+(n-1)d$. For example, in the first column on the right, the sum is $a+(a+(n-1)d)=2a+(n-1)d$; in the second column on the right, the sum is $(a+d)+(a+(n-2)d)=2a+(n-1)d$; and so on. We do this for each of the n columns on the right. Each of these n column sums is $2a+(n-1)d$, so the sum of the two right-hand sides of the two equations is $n(2a+(n-1)d)$. Equating the sum of the two left-hand sides and the sum of the two right-hand sides, we obtain

$$2S_n = n(2a+(n-1)d).$$

Therefore

$$S_n = n \left(a + \frac{(n-1)d}{2} \right) = na + \frac{n(n-1)}{2}d. \quad (6.43)$$

At this point, we should check the sanity of our answer.

- In the special case $n = 1$, when the series just has the term a , then we should have $S_n = a$, and that is indeed what our expression for S_n in (6.43) gives.
- In the special case $d = 0$, when the series is constant, with every term equal to a , its sum should be $S_n = na$, and our expression agrees with this too.

- We have already seen an important finite arithmetic series, namely the sum of the first n positive integers, in (3.5) and Theorem 19. In that case, the sum should be $S_n = n(n+1)/2$. If we put $a = 1$ and $d = 1$ in (6.43), then we obtain

$$n + \frac{n(n-1)}{2} = n \left(1 + \frac{n-1}{2} \right) = n \left(\frac{2+n-1}{2} \right) = \frac{n(n+1)}{2}.$$

You should get in the habit of doing checks like these — against very simple special cases, and particular cases where you already know the answer — whenever you derive a new expression for something you are trying to compute. Passing these checks doesn't *prove* that your expression is correct, but in practice it often detects errors and helps you correct them.

In fact, the expression for the sum of the first n positive integers (Theorem 19), while only one particular case of a finite arithmetic series, can nonetheless be used to derive the expression for the general case. Look again at (6.42), and consider the coefficients of d throughout the right-hand side. These coefficients are 0, 1, 2, ..., $n-1$. So let us rearrange the right-hand side of (6.42) and collect all the parts involving d together.

$$\begin{aligned} S_n &= a + (a+d) + (a+2d) + \cdots + (a+(n-1)d) \\ &= \underbrace{(a+a+\cdots+a)}_{n \text{ copies}} + (d+2d+\cdots+(n-1)d) \\ &\quad \text{(collecting all the copies of } a \text{ together, then multiples of } d) \\ &= na + (1+2+\cdots+(n-1))d \\ &= na + \frac{n(n-1)}{2}d \quad \text{(by Theorem 19).} \end{aligned}$$

What happens to the expression for S_n as the number of terms grows? The two parts of the expression, na and $(n(n-1)/2)d$, grow at different rates; the first is linear in n , while the second is quadratic in n . For large n , quadratic functions of n grow faster *in size* than linear functions of n . What happens for large n depends on whether d is positive or negative. When $d > 0$, the sum S_n is eventually positive (even if $a < 0$) and becomes larger and larger, and is unbounded. When $d < 0$, the sum S_n is eventually negative (even if $a > 0$) and, although it grows *in size*, it grows in the *negative* direction, going lower and lower, with no lower bound.

6.13 FINITE GEOMETRIC SERIES

A **finite geometric series** is a finite series obtained from a geometric sequence. If the geometric sequence has n terms, first term a , and common ratio r , then, as in (6.9), its terms are

$$a, \quad ar, \quad ar^2, \quad \dots, \quad ar^{n-2}, \quad ar^{n-1}.$$

So the corresponding finite geometric series is

$$a + ar + ar^2 + \cdots + ar^{n-2} + ar^{n-1}.$$

Again we ask, what is its sum S_n ?

We can easily dispose of the case $r = 1$, for then we are just adding up n copies of a , so we have $S_n = na$.

So now suppose $r \neq 1$.

Probably the easiest way to work out S_n in general is to first notice that, if we multiply S_n by r , we get something that looks very similar:

$$rS_n = ar + ar^2 + ar^3 + \cdots + ar^{n-1} + ar^n.$$

The only difference with the earlier series is that we have now lost the first term, a , and gained a new last term, ar^n . Therefore

$$rS_n = S_n - a + ar^n.$$

Collecting the terms in S_n together on the left, we have

$$rS_n - S_n = -a + ar^n.$$

A little algebra now gives

$$(r - 1)S_n = a(r^n - 1).$$

Therefore

$$S_n = a \left(\frac{r^n - 1}{r - 1} \right). \quad (6.44)$$

Once again, having derived this expression, we should do some simple checks.

- If $n = 1$, then the sequence has the single term a so $S_1 = a$, and our expression for S_n agrees with this.
- If $a = 0$ then all terms are 0 so $S_n = 0$, and our expression agrees with this.
- In Exercise 3.7 we showed that

$$1 + 2 + 2^2 + 2^3 + \cdots + 2^n = 2^{n+1} - 1.$$

The sum on the left is a finite geometric series with $n + 1$ terms, first term $a = 1$ and common ratio $r = 2$. Our formula gives

$$S_{n+1} = 1 \cdot \left(\frac{2^{n+1} - 1}{2 - 1} \right) = 2^{n+1} - 1,$$

which is correct.

6.14 INFINITE GEOMETRIC SERIES

Consider our expression in (6.44) for the sum S_n of a finite geometric series of n terms. What happens to S_n as $n \rightarrow \infty$?

If $r > 1$ then $r^n > 0$ and r^n grows without bound, so the quotient inside the parentheses in (6.44) is also positive and grows without bound. The sign of S_n is the same as the sign of a . So, if $a > 0$ then S_n is positive and increases without bound, while if $a < 0$ then S_n is negative and decreases without bound (giving larger and larger negative numbers with no lower bound).

If $r < -1$ in (6.44) then r^n alternates in sign as n increases, while increasing in size. The same happens to the numerator of the quotient in (6.44). But the denominator $r-1$ is always negative when $r < -1$. For even n , we have $r^n > 1$ and $r^n - 1 > 0$, with r^n increasing without upper bound as n increases, while for odd n , we have $r^n < -1$ and $r^n - 1 < -2$, with r^n decreasing without lower bound as n increases. The sign of the whole expression for each n will also depend on the sign of a , but the general character of S_n as n increases is clear: alternating between positive and negative numbers, with each getting larger in size without bound, so S_n has no upper or lower bound.

We have already seen that, if $r = 1$, then $S_n = na$. As n increases, this increases without upper bound if $a > 0$ and decreases without lower bound if $a < 0$. If $r = -1$, we have $S_n = a - a + a - a + \cdots + (-1)^n a$, which is either a or 0 according as n is odd or even, respectively. Putting $r = -1$ in (6.44) gives

$$S_n = a \left(\frac{(-1)^n - 1}{-2} \right).$$

When n is even, this is 0 , and when n is odd, it is a , all as expected. So in this case, the sequence has no limit, although it is now bounded both above and below.

It remains to consider what happens if $-1 < r < 1$.

We saw in Theorem 25 that, in this case, $r^n \rightarrow 0$ as $n \rightarrow \infty$. So, in our expression for S_n in (6.44), the r^n in the numerator vanishes as $n \rightarrow \infty$. This means that, if $-1 < r < 1$, then

$$S_\infty = \lim_{n \rightarrow \infty} S_n = \frac{a}{1-r}, \quad (6.45)$$

where S_∞ represents the sum of the infinite geometric series.

This expression is of fundamental importance and appears throughout computer science, mathematics, other sciences, engineering, economics, finance, and indeed every quantitative discipline.

6.15 HARMONIC NUMBERS

In § 6.5 we discussed the sequence $(1/n : n \in \mathbb{N})$ as an important example of a harmonic sequence. We now consider this specific sequence in more detail, because it crops up a lot in computer science.

The terms in this sequence converge down to 0 as $n \rightarrow \infty$, as we saw in (6.34) in § 6.8.

Now consider the *partial sums* of this sequence. We call them the **harmonic numbers** and denote them by H_n :

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}.$$

Even though the individual terms of the sequence of reciprocals diminish to 0, their partial sums H_n grow arbitrarily large. Pick any positive number B , no matter how large, and you can find n such that

$$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} > B.$$

This property — terms converge to limit 0, but with unbounded partial sums — is a very notable one.

- There are many sequences that diminish to 0 and whose partial sums are *bounded*: no matter how large n is, the partial sum stays below some fixed upper bound, so in fact the sum of all terms in the sequence is finite. For example, the sequences $(1/n^2 : n \in \mathbb{N})$ and $(1/2^n : n \in \mathbb{N})$ both go to 0 as $n \rightarrow \infty$, and both have finite sums too. The finite-sum claim is not obvious for the former sequence, but we know the latter sequence has a finite sum from § 6.14.
- On the other hand, if a sequence of positive numbers does not diminish to 0, then it can be shown that its partial sums are unbounded. For example, the terms in the sequence $(1 + n^{-1} : n \in \mathbb{N})$ tend to 1 as $n \rightarrow \infty$, and their partial sums grow without bound, since the n -th partial sum is $\geq n$.

But the sequence of reciprocals $(1/n : n \in \mathbb{N})$ is not in either of these camps. Its terms tend to 0, yet it has unbounded partial sums. In fact, it is one of the more rapidly diminishing positive sequences with this property. (There are some sequences with this property that diminish even more quickly than $(1/n : n \in \mathbb{N})$, but they do not diminish *much* more quickly. Can you find some?)

We saw in Exercise 3.11 that the harmonic numbers behave very much like $\log n$. We proved that $H_n \geq \log_e(n+1)$, and mentioned in the solutions (as a suggested further exercise) that $H_n \leq (\log_e n) + 1$. It is known (although beyond the scope of these Course Notes to prove) that the difference between H_n and $\log_e n$ converges to a constant:

$$\lim_{n \rightarrow \infty} (H_n - \log_e n) = \gamma, \quad (6.46)$$

where $\gamma = 0.5772\dots$ is called **Euler's constant**. It is a famous and mysterious number that arises in a variety of contexts, but we do not even know whether or not it is rational.

The differences $H_n - \log_e n$ approach their limit γ from above, so we have

$$H_n \geq (\log_e n) + \gamma,$$

giving a stronger lower bound than the one from Exercise 3.11. For an upper bound, it is known that

$$H_n \leq (\log_e n) + \gamma + \frac{1}{2n}, \quad (6.47)$$

although we do not prove this. We can see that the difference between these two bounds diminishes to 0 as $n \rightarrow \infty$. For very large n , we have an approximation:

$$H_n \approx \log_e n + \gamma. \quad (6.48)$$

Harmonic numbers have many applications in computer science, especially in problems involving the analysis of algorithms and probability, and it is good to be aware of their basic properties.

6.16 EXERCISES

1. Give recurrence relations for each of the following sequences:

- (a) the positive integers;
- (b) the negative even integers;
- (c) the squares;
- (d) the sequence whose n -th term is the sum of the first n positive integers;
- (e) the sequence whose n -th term is the sum of the reciprocals of the first n positive integers.

2. Define the sequence $(a_n : n \in \mathbb{N})$ recursively by

$$a_1 = 2, \quad a_n = 2n a_{n-1}$$

- (a) Find the first five terms of this sequence.
- (b) Prove by induction on n that $a_n = 2^n n!$.

3. Define the sequence $(b_n : n \in \mathbb{N})$ recursively by

$$b_1 = 3, \quad b_n = b_{n-1}^2$$

- (a) Explore this sequence.
- (b) Conjecture a formula for the sequence.
- (c) Prove that the formula is correct, by induction.

4. Define the sequence $(q_n : n \in \mathbb{N})$ recursively by

$$q_1 = 3, \quad q_2 = 5, \quad q_n = 3q_{n-1} - 2q_{n-2}.$$

- (a) Explore this sequence. Find the first seven terms.
 (b) Conjecture a formula for the sequence.
 (c) Prove that the formula is correct, by induction.

5. Define the sequence $(g_n : n \in \mathbb{N})$ recursively by

$$g_1 = 0, \quad g_n = g_{n-1} + \log n.$$

Prove by induction on n that

$$g_n \leq n \log n - n + H_n,$$

where $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$ (see Exercise 3.11).

To do this, it will help to use the following inequality, which holds for all $x \geq -1$:

$$\log(1+x) \leq x$$

The following technique can be helpful in relating $\log n$ and $\log(n-1)$:

$$\log(n-1) = \log\left(n \cdot \frac{n-1}{n}\right) = \log\left(n\left(1 - \frac{1}{n}\right)\right) = \log n + \log\left(1 - \frac{1}{n}\right).$$

Throughout this question, logarithms are natural logarithms, i.e., to base e .

6. Use the explore-conjecture-prove method to develop the best upper bound you can for the n -th term r_n of the sequence defined by the recurrence relation

$$r_1 = 1, \quad r_n = 3r_{n-1} - 1,$$

and prove your bound by induction.

Even better, see if you can find a formula for r_n , and prove it by induction.

7. The **Lucas sequence** $(l_n : n \in \mathbb{N})$ is a close relative of the Fibonacci sequence, defined by using the same recurrence relation but starting out differently, as follows.

$$l_1 = 1, \quad l_2 = 3, \quad l_n = l_{n-1} + l_{n-2} \quad (\text{for } n \geq 3). \quad (6.49)$$

Adapt the approach of § 6.7.4 in order to develop a formula for l_n and then prove by induction that it works.

8. Define the sequence $(t_n : n \in \mathbb{N})$ by the recurrence relation

$$t_1 = 1, \quad t_n = 3 - \frac{1}{2}t_{n-1}.$$

- (a) Consider the following theorem, which is correct, and its “proof” by induction, which is incorrect.

Find the errors in this incorrect proof.

The lines of the proof are numbered, so you can refer to them.

Theorem. For all $n \in \mathbb{N}$,

$$t_n \leq \frac{5}{2}.$$

“Proof”.

1. For each $n \in \mathbb{N}$, let $S(n)$ be the statement that $t_n \leq 5/2$. We must prove that, for every $n \in \mathbb{N}$, the statement $S(n)$ is true.
2. Inductive basis: Let $n = 1$. Then $t_1 = 3 - \frac{1}{2} \cdot 1 \leq 5/2$. So $S(1)$ holds.
3. Inductive step:
4. Assume that for some k the statement $S(k)$ holds. (This is the Inductive Hypothesis.)
5. Consider $S(k+1)$.

$$\begin{array}{ll}
 6. & t_{k+1} = 3 - \frac{1}{2}t_k \\
 7. & \leq 3 - \frac{1}{2} \cdot \frac{5}{2} \quad (\text{by the Inductive Hypothesis}) \\
 8. & = 3 - \frac{5}{4} \\
 9. & = \frac{7}{4} \\
 10. & \leq \frac{5}{2}.
 \end{array}$$

11.

“□”

- (b) Write a correct proof by induction for the theorem.

There are (at least) two approaches you could try.

- (i) Prove, by induction on n , that for all $n \in \mathbb{N}$,

$$1 \leq t_n \leq \frac{5}{2}.$$

So this is a case where it's actually *easier* to prove a *stronger* statement.

- (ii) Prove firstly that $S(n)$ holds for all even n , and then that it holds for all odd n .

9. Define the sequence $(\varphi_n : n \in \mathbb{N})$ recursively by

$$\varphi_1 = 1, \quad \varphi_n = 1 + \frac{1}{\varphi_{n-1}}.$$

- (a) Explore this sequence. Find the first 20 terms, using a spreadsheet or program.
- (b) Conjecture a value for the limit φ of the sequence as $n \rightarrow \infty$.
- (c) Roughly speaking, if $\varphi_n \approx \varphi_{n-1}$, then we would expect that they are close to the limit, and so the limit φ would be expected to satisfy

$$\varphi = 1 + \frac{1}{\varphi}.$$

Solve this equation, in order to derive a *conjectured* exact value for the limit.

- (d) Suppose you want to prove that all sufficiently large n , the n -th term is within 0.001 of this limit. How large is “sufficiently large”? In other words, determine N so that, for all $n \geq N$,

$$|\varphi_n - \varphi| < 0.001.$$

It's fine to do this computationally.

- (e) Prove that, if

$$\varphi_n < \varphi + 0.001,$$

then

$$\varphi_{n+1} > \varphi - 0.001.$$

Note the change in direction of the inequality.

The following two points may help.

- When studying how a quantity changes under a small addition, it can sometimes be useful to convert that additive change to a multiplicative change. For example,

$$w + 0.001 = w \left(1 + \frac{0.001}{w} \right).$$

- The following inequality may help. For all $x > 0$,

$$\frac{1}{1+x} > 1-x.$$

(f) Prove that, if

$$\varphi_n > \varphi - 0.001,$$

then

$$\varphi_{n+1} < \varphi + 0.001.$$

Note, again, the change in direction of the inequality.

The following inequality may help. For all x satisfying $0 < x < \frac{1}{3}$,

$$\frac{1}{1-x} < 1 + \frac{3}{2}x.$$

(g) Deduce that, if

$$\varphi - 0.001 < \varphi_n < \varphi + 0.001,$$

then

$$\varphi - 0.001 < \varphi_{n+1} < \varphi + 0.001.$$

(h) Now use (g) to prove, by induction on n , that for all $n \geq N$,

$$\varphi - 0.001 < \varphi_n < \varphi + 0.001.$$

Here, N is the “threshold of sufficient largeness” from part (d).

10. The **Limit Game** is a simple two-player game that can be played on any sequence. We call the two players Lim and Una. Let $(a_n)_{n=1}^{\infty}$ be a sequence. Roughly speaking, Lim’s aim is to approximate a limit using a term from the sequence, while Una tries to ensure that the approximation is not as good as it should be. Lim and Una play the game as follows, with Lim having the first turn, and the two players then taking turns as follows.

1. **Lim’s first turn.** Lim chooses a number a , which is intended to specify a limit.
2. **Una’s first turn.** Una chooses a real number $\varepsilon > 0$, which is intended to specify how closely a must be approximated by terms in the sequence.

3. **Lim's second turn.** Lim chooses a positive integer N , which is intended to specify some position in the sequence.
4. **Una's second turn.** Una chooses a positive integer $n \geq N$, which is intended to specify some later position in the sequence, beyond position N .

The game then ends, and the winner is determined as follows.

- If $|a_n - a| < \varepsilon$ then **Lim wins**.
- If $|a_n - a| \geq \varepsilon$ then **Una wins**.

For example, suppose the sequence is $((-1)^{n+1}/n : n \in \mathbb{N})$. Here are one possible play of the Limit Game for this sequence.

1. Lim chooses $a = \frac{1}{2}$.
2. Una chooses $\varepsilon = \frac{1}{4}$.
3. Lim chooses $N = 3$.
4. Una chooses $n = 5$. (This satisfies $n \geq N$.)

Here, Una wins, because

$$|a_n - a| = |a_5 - a| = \left| \frac{1}{5} - \frac{1}{2} \right| = \frac{3}{10} \geq \frac{1}{4} = \varepsilon.$$

- (a) In this specific play of the game, could Una have possibly lost on her last move? In other words, is there a different choice she could have made at that stage, which would have lost her the game?
- (b) Now let's go back one move. So we just assume that Lim's and Una's first turns are as above. Now consider Lim's second turn. Could Lim have possibly chosen a value of N which would have guaranteed that he would win the game, no matter what Una did next?
- (c) Now let's go back further. Assume that Lim has just made his first move, as above. Consider Una's options. Are all her options equally good? Is she certain to win, no matter what ε she chooses? What advice would you give her about choosing ε ?
- (d) Now let's go back to the very first move. What should Lim choose, and why?
- (e) Play the game, with a classmate or someone else you know. You can pick whatever sequences you like.
- (f) In general, what property does a sequence need to have in order for Lim to have a winning strategy in the Limit Game?

- (g) Propose a sequence for which Una has a winning strategy in the Limit Game, and describe her winning strategy.
- (h) In general, what property does a sequence need to have in order for Una to have a winning strategy in the Limit Game?

11. The **Big-O Game** is another, shorter two-player game that can be played on any sequence. For this game, the two players are Oh-Yes and Oh-No. Let $(a_n)_{n=1}^{\infty}$ be a sequence and let $f: \mathbb{N} \rightarrow \mathbb{R}_0^+$ be a function. The aim of Oh-Yes is to help establish that

$$a_n = O(f(n)),$$

while the aim of Oh-No is to frustrate Oh-Yes. They play the game as follows, with Oh-Yes having the first turn, and each player having one turn as follows.

1. **Oh-Yes's turn.** Oh-Yes chooses two things:
 - a constant factor $c \in \mathbb{R}^+$;
 - a positive integer threshold N .
2. **Oh-No's turn.** Oh-No chooses a positive integer $n \geq N$, intended to specify some position at least this far along the sequence.

The game then ends, and the winner is determined as follows.

- If $a_n \leq c f(n)$ then **Oh-Yes wins**.
- If $a_n > c f(n)$ then **Oh-No wins**.

For example, suppose the sequence is $(n^2: n \in \mathbb{N})$ and the function is $f(n) = n^{1.5}$. Here is one possible play of the Big-O Game for this sequence.

1. Oh-Yes chooses constant factor $c = 100$ and threshold $N = 4$
2. Oh-No chooses $n = 1\,000\,000$.

Here, Oh-No wins, because

$$a_n = a_{1\,000\,000} = 1\,000\,000^2 = (10^6)^2 = 10^{12} > 10^{11} = 10^2(10^6)^{1.5} = 100(1\,000\,000)^{1.5} = c f(n).$$

- (a) In this play of the game, could Oh-No have possibly lost on their move?
- (b) Now let's go back to the first move. What could Oh-Yes have chosen in order to win the game, and why?
- (c) Play the game, with a classmate or someone else you know. You can pick whatever sequences and functions you like.

- (d) In general, what relationship must hold between the sequence and the function in order for Oh-Yes to have a winning strategy in the Big-Oh Game?
- (e) Propose a sequence and function for which Oh-No has a winning strategy in the Big-O Game, and describe their winning strategy.
- (f) In general, how do the sequence and function have to be related in order for Oh-No to have a winning strategy in the Big-O Game?

12. Give the simplest big-O expression you can for each of the following.

- (a) $1 + 4n + 6n^2 + 4n^3 + n^4$
- (b) $\binom{n}{2}$
- (c) $\binom{n}{3}$
- (d) the sum of the first n terms in an arithmetic series with first term 1 and common difference d
- (e) $2^n + 2^{n-1} + \cdots + 2 + 1$
- (f) $(n!)^{1/n}$
- (g) $n^{1/n}$

13.

- (a) How can the number

$$\underbrace{999 \dots 99}_{n \text{ digits}}$$

be interpreted as the sum of a finite geometric series? For that series, what are a , r and n ? What does the formula for S_n give in this case? Does this make sense?

- (b) What about the number

$$0.999999 \dots$$

14.

- (a) Using (6.47), show that

$$H_n = O(\log n).$$

- (b) Hence prove that

$$\frac{n}{1} + \frac{n}{2} + \frac{n}{3} + \cdots + \frac{n}{n-1} + \frac{n}{n} = O(n \log n).$$

15. Recall the sequence $(g_n : n \in \mathbb{N})$ from [Exercise 5](#). Its n -th term has the closed-form formula

$$g_n = \sum_{i=1}^n \log i.$$

(a) Use [Exercise 5](#) and (6.47) to show that

$$g_n \leq n \log n - n + \log n + c$$

for some positive constant c .

(b) Hence show that

$$g_n = O(n \log n).$$

(c) Use (a) to show that

$$e^{g_n} \leq \left(\frac{n}{e}\right)^n n e^c.$$

(d) Prove that

$$e^{g_n} = n!.$$

(e) Hence show that

$$n! = O\left(\left(\frac{n}{e}\right)^n n\right).$$

NUMBER THEORY

Numbers were arguably the first abstract objects on which people did computations. They are so central to computation that, for centuries, the term “computation” was assumed to refer to numerical computation. Although we now compute with many other abstract objects too, numbers remain fundamental. They appear in most algorithms and data structures in one way or another. They are essential to the design and analysis of algorithms and data structures, even those that don’t themselves contain numbers. Computational problems on numbers span the full range of difficulty, from very easy to very difficult or even totally intractable. For some hard problems, their difficulty is actually an asset, enabling them to be used to help keep information secure.

In this chapter we study the basics of number theory, both in a general way and as a specific tool used in modern cryptography.

7.1^α MULTIPLES AND DIVISORS

An integer n is a **multiple** of a positive integer d if there is another integer q such that $n = qd$. The set of all multiples of d is denoted by $d\mathbb{Z}$:

$$d\mathbb{Z} = \{\dots, -3d, -2d, -d, 0, d, 2d, 3d, \dots\}.$$

So, for example, the set of even integers is denoted by $2\mathbb{Z}$.

When n is a multiple of d , we also say that d is a **divisor** or **factor** of n . We write

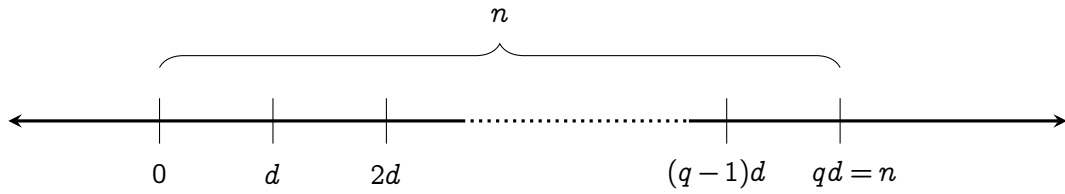
$$d \mid n$$

to say, symbolically, that d is a divisor of n .

For example, we can write that, for any integer n ,

$$n \text{ is even} \iff 2 \mid n.$$

You would have seen these ideas previously, except perhaps for the notation $d\mathbb{Z}$ and $d \mid n$, and would have seen it illustrated using a picture of the number line, with the length representing n being divided into q equal segments each of length d :

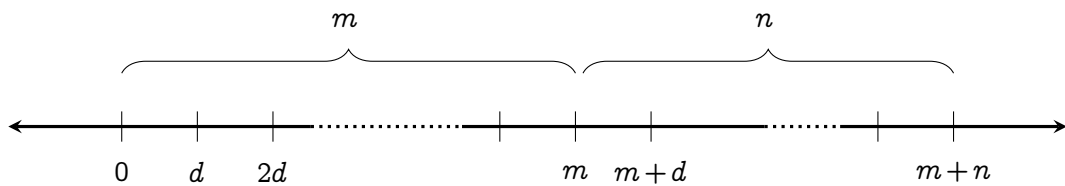


If d is not a divisor of n , then we can write $d \nmid n$.

If d is a divisor of integers m and n , then it is also a divisor of their sum:

$$d \mid m \wedge d \mid n \implies d \mid (m+n). \quad (7.1)$$

This can be illustrated as follows.



The converse does not hold. For example, $2 \mid (3+5)$ but $2 \nmid 3$ and $2 \nmid 5$.

A similar observation to (7.1) applies to differences:

$$d \mid m \wedge d \mid n \implies d \mid (m-n). \quad (7.2)$$

What about the product? If $d \mid m$ and $d \mid n$, does it follow that $d \mid mn$?

One technical point about divisors is that *every* integer d is a divisor of 0. This is because $0 = 0d$, so the definition is satisfied even though it seems slightly bizarre. So, at times, when dealing with divisors and multiples, we have to treat 0 as a special case.

For some positive integers d , there are simple tests you can use to determine if a number n is a multiple of d , using the decimal representation of n . For example

$$10 \mid n \iff \text{the decimal representation of } n \text{ ends in } 0.$$

You don't even need to look at the rest of n ; you only need to look at its rightmost (i.e., least significant) digit. Similarly,

$$5 \mid n \iff \text{the decimal representation of } n \text{ ends in } 0 \text{ or } 5,$$

$$2 \mid n \iff \text{the decimal representation of } n \text{ ends in one of } 0, 2, 4, 6, 8$$

$$\iff \text{the last digit of the decimal representation of } n \text{ gives a multiple of } 2,$$

$$4 \mid n \iff \text{the last two digits of the decimal representation of } n \text{ give a multiple of } 4.$$

We study these and other divisibility tests further in [Exercise 9](#).

There are other divisibility tests that require looking at all digits but which still save time over actually doing the division. Some of these are considered in [Exercise 10](#).

Divisibility tests along these lines are not restricted to the decimal number system.

$$2 \mid n \iff \text{the binary representation of } n \text{ ends in } 0.$$

Other divisibility tests for binary numbers are possible ([Exercise 10](#)).

7.2^α PRIME NUMBERS

Every $n \in \mathbb{N}$ is a multiple of 1 and of itself. A divisor of n is **proper** if it is neither 1 nor n . A **prime** number is a positive integer that is greater than 1 and has no proper divisors. So the first few prime numbers are

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, ...

A number that is not prime is **composite**.

We have already seen that there are infinitely many prime numbers: see [Theorem 17](#).

When considering multiplication of positive integers, prime numbers are like “atoms” in that they are indivisible (i.e., cannot be broken down further into a product of smaller positive integers) and every positive integer can be “put together” from these atoms.

Theorem 27. Every positive integer except 1 is a product of primes.

Proof. We prove, by induction on n , that for every positive integer $n \geq 2$ can be expressed as a product of primes.

Inductive basis:

If $n = 2$ then n is itself prime so it is a product of one prime, namely 2. So the claim holds in this case.

Inductive Step:

Let $n > 2$.

Assume that every $k \leq n$ is a product of primes. (This is our Inductive Hypothesis.)

Now consider $n + 1$. We consider two cases, according to whether $n + 1$ is prime or composite.

Case 1: If $n + 1$ is prime, then it is already a product of primes (with just one prime in the product, namely $n + 1$ itself).

Case 2: If $n + 1$ is composite, then it has a factor a which is neither 1 nor $n + 1$. Therefore $b = (n + 1)/a$ is also a positive integer, so it is also an integer factor of $n + 1$, so $n + 1 = ab$ is the product of these two positive integers. Furthermore, b is also neither 1 nor $n + 1$ (because if $b = 1$ then $a = n + 1$, and if $b = n + 1$ then $a = 1$, so either way we get a contradiction with what we already know about a).

Since $a \leq n$, we can apply the Inductive Hypothesis to it and deduce that a is a product of primes.

Similarly, since $b \leq n$, the Inductive Hypothesis tells us that b is also product of primes.

So, both a and b are products of primes. Therefore their product $n = ab$ is also a product of primes.

This completes the Inductive Step.

Conclusion: Therefore, by Mathematical Induction, the claim holds for all n . □

So far, we have not ruled out the possibility that some positive integers can be expressed as a product of primes in more than one way. But, in fact, that is impossible. Every positive integer can be expressed *uniquely* as a product of primes. We prove this later, in [Theorem 33](#) in § 7.9.

The theory of prime numbers has been developed over thousands of years. It has become one of the deepest and richest fields of mathematics. For most of its history, it was regarded as being among the “purest” areas of pure mathematics, fascinating and beautiful but very far removed from practical applications. The great Cambridge mathematician G.H. Hardy, a major contributor to the theory of prime numbers in the early 20th century, delighted in how “useless” his work was.¹ Yet today prime numbers are widely used in computer science and are central to information security. Whenever you do a secure electronic transaction, its security is probably based on the theory of prime numbers.

7.3 REMAINDERS AND THE MOD OPERATION

If n is a multiple of d , then we can divide n by d exactly, with nothing left over; the *remainder* is 0. In that case, the quotient

$$q = \frac{n}{d}$$

is an integer, and we have

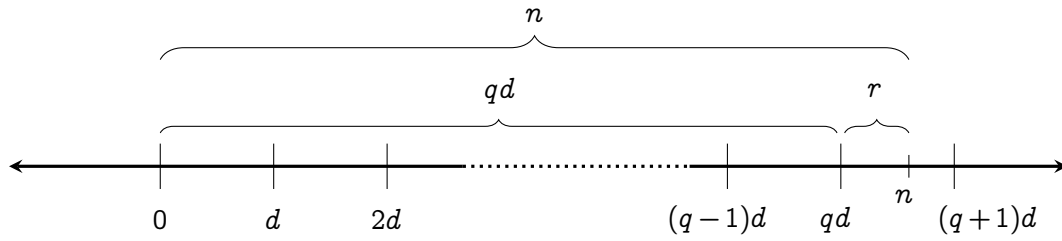
$$n = qd.$$

If, however, n is not a multiple of d , then dividing n by d does not give an integer. In that case, there is a nonzero *remainder*, which is the part of n that is left over after subtracting from n the highest multiple of d we can, without going over.

Formally, the **remainder** after division of n by d is the positive integer r that satisfies

$$n = qd + r, \quad 0 \leq r \leq d - 1. \quad (7.3)$$

¹ G. H. Hardy, *A Mathematician's Apology*, Cambridge University Press, 1940.



The computation of the remainder, r , of n after division by d is sufficiently important that it has its own binary operation notation. It is written

$$n \bmod d$$

and we read it as “ $n \bmod d$ ” or “ n modulo d ”. Note that, just like other arithmetic operations $+$, $-$, $\times$, $/$, this operation uses *infix* notation, with “mod” placed in between its arguments.

We have, for example:

$$\begin{aligned} 6 \bmod 4 &= 2 \\ 7 \bmod 4 &= 3 \\ 6 \bmod 2 &= 0 \\ 7 \bmod 2 &= 1 \\ 6 \bmod 5 &= 1 \\ 144 \bmod 100 &= 44 \\ 174 \bmod 48 &= 30 \end{aligned}$$

This operation extends to negative n too. We still require $0 \leq r \leq d - 1$, and q has the property that it is as large as possible while still having $qd \leq n$. So, for example,

$$-6 \bmod 5 = 4,$$

since the largest multiple of 5 which is ≤ -6 is $-2 \times 5 = -10$ (with $q = -2$), which gives $r = 4$, since $-10 + 4 = -6$.

Although we allow negative n , we always require d to be positive.

Programming languages have their own conventions for operations like mod. In many languages, % is used for applying mod to positive integers, so that, for example, $6 \% 4$ returns 2. But the meaning of % when one or both arguments is negative can vary across different languages or even different implementations of the same language. So it cannot be assumed to always mean the same thing as the mathematical remainder operation “mod”.

If we calculate n/d exactly, it has integer part q and fractional part $r/d \in [0, 1)$:

$$\frac{n}{d} = q + \frac{r}{d} = q + \frac{n \bmod d}{d}.$$

7.4 PARITY

Remainders modulo 2 are particularly simple and also particularly important. For any $x \in \mathbb{Z}$, its remainder mod 2 indicates whether x is even or odd:

$$x \bmod 2 = \begin{cases} 1, & \text{if } x \text{ is odd;} \\ 0, & \text{if } x \text{ is even.} \end{cases} \quad (7.4)$$

In fact, we can regard it as the indicator function of the set of odd numbers.

The remainder $x \bmod 2$ is called the **parity** of x . The term “parity” is also used more descriptively, to mean the identification of whether an integer is even or odd.

The parity of the sum, difference, product or quotient of two integers is completely determined by the parity of those two integers. The precise behaviour of arithmetic operations with respect to parity is given by the following tables:

+	even	odd	×	even	odd
even	even	odd	even	even	even
odd	odd	even	odd	even	odd

For example, the sum (or difference) of two odd numbers is even, while their product is odd. The table for subtraction has the same entries as for addition, while a table for division would look the same as that for multiplication with the caveat that division by zero is undefined.

Let us rewrite those tables with parities represented by 0 or 1, as in (7.4).

+	0	1	×	0	1
0	0	1	0	0	0
1	1	0	1	0	1

Addition here is the same as ordinary addition except that $1 + 1$ is not 2; instead, it is $2 \bmod 2$, i.e., 0. Multiplication is exactly the same as ordinary integer multiplication restricted to $\{0, 1\}$.

Let us lay these tables out differently, with headings along the top and each quantity having its own column, to help make links with some concepts we have studied previously.

x	y	$(x + y) \bmod 2$	x	y	$xy \bmod 2$
0	0	0	0	0	0
0	1	1	0	1	0
1	0	1	1	0	0
1	1	0	1	1	1

Do either of these tables look familiar? See if you can find tables in earlier chapters that have the same patterns of entries, even if the entries themselves are different.

It is worth developing the skill of recognising when two structures are the same except that things have been renamed. This skill is used throughout computer science and mathematics, and has been formalised as the concept of *isomorphism*. We will define and discuss isomorphism in one specific context later.

Now compare the left table above (for mod 2 addition) with the table on p. 129 (§ 4.11) for exclusive-or, $\oplus$. Compare the right table (for mod 2 multiplication) with the table on p. 124 (§ 4.6) for conjunction, $\wedge$.

We see, then, that the logical operations $\oplus$ and $\wedge$, using truth values False and True, may be treated as just addition and multiplication, respectively, modulo 2, using 0 and 1 to represent False and True respectively. This resolves a question we raised on p. 122 in § 4.1^α. It enables us to relate logic to arithmetic, indeed it underpins the use of logic to perform arithmetic in computers.

7.5 THE GREATEST COMMON DIVISOR

The **greatest common divisor (gcd)** of two integers a and b , written $\gcd(a, b)$, is the greatest integer d such that $d \mid a$ and $d \mid b$.² This is well defined provided a and b are not both 0.³

If $b = 1$ then clearly we have $\gcd(a, 1) = 1$.

More generally, if $b \mid a$ then $\gcd(a, b) = b$.

If p is prime, then $\gcd(a, p) = 1$ unless a is a multiple of p , say $a = kp$, in which case we have $\gcd(kp, p) = p$.

What if neither a nor b is prime and neither is a multiple of the other?

Consider 4 and 6. Neither is a multiple of the other, and each is a multiple of 2. Also, no number > 2 is a divisor of either of them. So $\gcd(4, 6) = \gcd(6, 4) = 2$.

Now can we work out the gcd, in general, in a systematic way?

The key observation is the following.

Theorem 28. For any integers a, b that are not both 0,

$$\gcd(a, b) = \gcd(b, a - b).$$

Proof. For any integers x, y , let $\text{CD}(x, y)$ be the set of their common divisors:

$$\text{CD}(x, y) = \{d : d \mid x \wedge d \mid y\}.$$

² This is also called the **highest common factor (hcf)**, but that term is less common these days. The gcd of a and b is often just denoted by (a, b) , but this seems to be unnecessary overloading of standard ordered pair notation, so we use $\gcd(a, b)$.

³ If $a = b = 0$, then every integer is a divisor of a and b , so they have no *greatest* common divisor.

We show that $\text{CD}(a, b) = \text{CD}(b, a - b)$, from which the Theorem will follow, since identical sets of integers have identical greatest elements (provided they do indeed have a greatest element, which they do in this case).

First, we show $\text{CD}(a, b) \subseteq \text{CD}(b, a - b)$. Suppose $d \in \text{CD}(a, b)$. Then $d \mid a$ and $d \mid b$. By (7.2), this implies that $d \mid (a - b)$. So $d \mid b$ and $d \mid (a - b)$. So $d \in \text{CD}(b, a - b)$. So every element of $\text{CD}(a, b)$ is also an element of $\text{CD}(b, a - b)$. Therefore $\text{CD}(a, b) \subseteq \text{CD}(b, a - b)$.

Second, we show $\text{CD}(b, a - b) \subseteq \text{CD}(a, b)$. Suppose $d \in \text{CD}(b, a - b)$. Then $d \mid a - b$ and $d \mid b$. By (7.1), this implies that $d \mid ((a - b) + b)$. But $(a - b) + b = a$. So $d \mid a$ and $d \mid b$. So $d \in \text{CD}(a, b)$. So every element of $\text{CD}(b, a - b)$ is also an element of $\text{CD}(a, b)$. Therefore $\text{CD}(b, a - b) \subseteq \text{CD}(a, b)$.

We established the subset relation in both directions. So, in fact,

$$\text{CD}(a, b) = \text{CD}(b, a - b).$$

The two sets are nonempty, since every integer has 1 as a divisor, so 1 belongs to each set. They also have an upper bound, since divisors are bounded above by the sizes of the numbers they are divisors of, i.e., $d \mid n \implies d \leq |n|$. The two sets therefore each have a greatest element.⁴

Since these two sets are identical, and since they each do have a greatest element, we deduce that their greatest elements are identical. Therefore $\text{gcd}(a, b) = \text{gcd}(b, a - b)$. $\square$

It does not matter which way round we write a and b in $\text{gcd}(a, b)$, because

$$\text{gcd}(a, b) = \text{gcd}(b, a).$$

So the equation of Theorem 28 could have been written

$$\text{gcd}(a, b) = \text{gcd}(a - b, b).$$

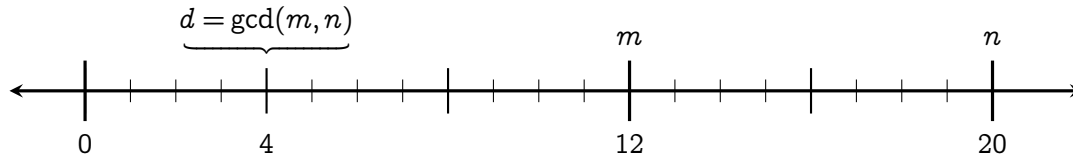
We will mostly adopt the habit of putting the larger number first, because that makes the description of some of our algorithms neater. But the *value* of the gcd does not care about the order of its arguments.

We illustrate the use of Theorem 28 in computing the gcd.

$$\begin{aligned} \text{gcd}(12, 20) &= \text{gcd}(20, 12) \\ &= \text{gcd}(12, 20 - 12) && \text{(by Theorem 28)} \\ &= \text{gcd}(12, 8) \\ &= \text{gcd}(8, 12 - 8) && \text{(by Theorem 28)} \\ &= \text{gcd}(8, 4) \\ &= 4. \end{aligned}$$

⁴ Here we are using the fact that every set of integers that has some upper bound has a unique greatest element. Note, though, that it is *not* the case that every set of real numbers that has some upper bound has a greatest element. Nor does it hold for the rational numbers.

Here is $\gcd(12, 20)$ pictured on a number line.



7.6 THE EUCLIDEAN ALGORITHM

If $a, b \in \mathbb{N}$ and $a > b$ then [Theorem 28](#) expresses $\gcd(a, b)$ in terms of the gcd of a smaller pair of positive integers. This can be used in a recursive algorithm for computing gcd. We also need a base case, but this can be provided by some of the special cases we gave just after defining the gcd on p. [239](#). So we have the following algorithm.

1. Input: $a, b \in \mathbb{N}$

2. If $a < b$ then swap them around (to ensure $a \geq b$):

$\text{new } a := \text{old } b,$
 $\text{new } b := \text{old } a.$

3. If $b \mid a$ then Output b .

4. Output $\gcd(b, a - b)$.

Trying it on $\gcd(12, 20)$ should give the calculation at the end of the previous section. Then try a larger pair. Here is the calculation for $\gcd(48, 174)$ (glossing over some of the swaps).

$$\begin{aligned}
 \gcd(48, 174) &= \gcd(174, 48) && \text{(swapping, Step 2)} \\
 &= \gcd(48, 174 - 48) && \text{(using Step 4, which is based on Theorem 28)} \\
 &= \gcd(126, 48) \\
 &= \gcd(126 - 48, 48) && \text{(Step 4)} \\
 &= \gcd(78, 48) \\
 &= \gcd(48, 78 - 48) && \text{(Step 4)} \\
 &= \gcd(48, 30) \\
 &= \gcd(30, 48 - 30) && \text{(Step 4)} \\
 &= \gcd(30, 18) \\
 &= \gcd(18, 30 - 18) && \text{(Step 4)} \\
 &= \gcd(18, 12) \\
 &= \gcd(12, 18 - 12) && \text{(Step 4)} \\
 &= \gcd(12, 6) \\
 &= 6 && \text{(finally reaching the base case, Step 3).}
 \end{aligned}$$

Consider the first few steps in this process. We repeatedly subtracted 48 from 174 until the amount left over was < 48 . But this is precisely the process of dividing by 48 a whole number of times and determining the remainder. In this case, after three subtractions of 48, we found that the remainder is 30. We can speed the algorithm up by doing this remainder calculation directly instead of by repeated subtraction. We now revise the algorithm to do this, with the modified part shown in blue.

Euclidean Algorithm (EA)

1. Input: $a, b \in \mathbb{N}$
2. If $a < b$ then swap them around (to ensure $a \geq b$):

$\text{new } a := \text{old } b,$
 $\text{new } b := \text{old } a.$
3. If $b \mid a$ then Output b .
4. $q := \lfloor a/b \rfloor$.
5. Output $\gcd(b, a - qb)$.

Note that, in the last step,

$$a - qb = a \bmod b,$$

and q plays no role except to work out $a \bmod b$. So, in the last step, we could write the output as $\gcd(b, a \bmod b)$. But we have made the role of q explicit because it will play a role later on, in an extension of this algorithm.

Here is the new computation for $(48, 174)$.

$$\begin{aligned} \gcd(48, 174) &= \gcd(174, 48) \\ &= \gcd(48, 174 \bmod 48) && \text{(Step 4, using remainder now)} \\ &= \gcd(48, 30) \\ &= \gcd(30, 48 \bmod 30) && \text{(Step 4)} \\ &= \gcd(30, 18) \\ &= \gcd(18, 30 \bmod 18) && \text{(Step 4)} \\ &= \gcd(18, 12) \\ &= \gcd(12, 18 \bmod 12) && \text{(Step 4)} \\ &= \gcd(12, 6) \\ &= 6 && \text{(base case, Step 3).} \end{aligned}$$

The Euclidean Algorithm is over two thousand years old and has been described as one of the oldest written mathematical algorithms.

When you meet a new algorithm, you should think about how many steps it takes, as a function of the input. Play around with some examples and see if you can conjecture some kind of bound on the number of steps required by the Euclidean Algorithm.

7.7 THE GCD AND INTEGER LINEAR COMBINATIONS

We saw in (7.1) and (7.2) that, if $d \mid m$ and $d \mid n$ then $d \mid m + n$ and $d \mid m - n$. It follows that, for all $x, y \in \mathbb{Z}$, we have

$$d \mid xm + yn.$$

The quantities $xm + yn$, with $x, y \in \mathbb{Z}$, are called **integer linear combinations** of m and n : “integer” because the *coefficients* x and y are integers, and “linear” because we are just taking multiples of m and n without raising them to powers, taking roots, or doing anything more exotic to them. We write $m\mathbb{Z} + n\mathbb{Z}$ for the set of all integer linear combinations of m and n .

Conversely, if $d \mid xm + yn$ for all $x, y \in \mathbb{Z}$, then it certainly holds for each of the following special cases:

- $x = 1$ and $y = 0$, which means $d \mid 1m + 0n$, i.e., $d \mid m$;

- $x = 0$ and $y = 1$, which means $d \mid 0m + 1n$, i.e., $d \mid n$.

So the set of common divisors of m and n is the same as the set of common divisors of all combinations $xm + yn$ with $x, y \in \mathbb{Z}$:

$$\{d : d \mid m \wedge d \mid n\} = \{d : \forall x, y \in \mathbb{Z} \ d \mid xm + yn\}.$$

Since these two sets are identical, so are their largest elements. The largest element of the left set is just $\gcd(m, n)$, while the largest element of the right set is the gcd of all integers $xm + yn$ with $x, y \in \mathbb{Z}$. So these two gcds — one being just the gcd of two numbers, the other being the gcd of the infinite set $m\mathbb{Z} + n\mathbb{Z}$ — are equal:

$$\gcd(m, n) = \gcd(m\mathbb{Z} + n\mathbb{Z}).$$

Let us see how this pans out for an example. Consider the case $m = 12$, $n = 20$. At the end of § 7.5, on p. 240, we worked out that $\gcd(12, 20) = 4$ and pictured it on the number line. In this case, what does the set of all integer linear combinations of 12 and 20 look like? For a start, we have

$$\begin{aligned} 1 \cdot 12 + 0 \cdot 20 &= 12, \\ 0 \cdot 12 + 1 \cdot 20 &= 20. \end{aligned}$$

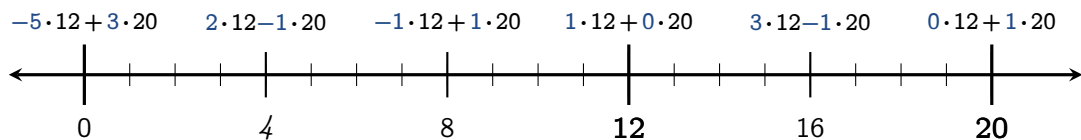
Other coefficients give other values, e.g.,

$$\begin{aligned} 1 \cdot 12 + 1 \cdot 20 &= 32, \\ -1 \cdot 12 + 1 \cdot 20 &= 8, \\ 2 \cdot 12 - 1 \cdot 20 &= 4, \\ -2 \cdot 12 + 1 \cdot 20 &= -4, \\ -5 \cdot 12 + 3 \cdot 20 &= 0, \\ 4 \cdot 12 - 2 \cdot 20 &= 8, \\ -3 \cdot 12 + 1 \cdot 20 &= -16. \end{aligned}$$

In fact, in this case, the set $12\mathbb{Z} + 20\mathbb{Z}$ of all integer linear combinations of 12 and 20 is the set of all multiples of 4 (including both positive and negative multiples, and zero). Symbolically, we can write

$$12\mathbb{Z} + 20\mathbb{Z} = 4\mathbb{Z}.$$

We illustrate this on the number line.



In fact, every multiple of 4 arises as an integer linear combination of 12 and 20 in multiple ways. We saw this above, with

$$-1 \cdot 12 + 1 \cdot 20 = 4 \cdot 12 - 2 \cdot 20 = 8.$$

This is actually an understatement: every multiple of 4 arises in *infinitely many* different ways as an integer linear combination of 12 and 20. This follows from the fact that 0 itself is an integer linear combination of 12 and 20, as we saw above:

$$-5 \cdot 12 + 3 \cdot 20 = 0. \quad (7.5)$$

We can take any equation expressing a multiple of 4 (like 8) as an integer linear combination of 12 and 20, such as

$$-1 \cdot 12 + 1 \cdot 20 = 8, \quad (7.6)$$

and then add our equation for 0, namely (7.5), to get another integer linear combinations for 8:

$$\begin{array}{rcl} \text{equation (7.6):} & -1 \cdot 12 + 1 \cdot 20 & = 8 \\ \text{plus equation (7.5):} & -5 \cdot 12 + 3 \cdot 20 & = 0 \\ \hline \text{equals:} & -6 \cdot 12 + 4 \cdot 20 & = 8 \end{array}$$

We can do this as many times as we like, to generate an infinite family of equations expressing 8 as an integer linear combination of 12 and 20:

$$\begin{array}{rcl} -1 \cdot 12 + 1 \cdot 20 & = & 8, \\ 4 \cdot 12 - 2 \cdot 20 & = & 8, \\ 9 \cdot 12 - 5 \cdot 20 & = & 8, \\ 14 \cdot 12 - 8 \cdot 20 & = & 8, \\ & \vdots & \\ -6 \cdot 12 + 4 \cdot 20 & = & 8, \\ -11 \cdot 12 + 7 \cdot 20 & = & 8, \\ & \vdots & \end{array}$$

The phenomena we have observed here are general, and not just specific to 12 and 20. Let m, n be any positive integers, and consider the set $m\mathbb{Z} + n\mathbb{Z}$ of all integers of the form $xm + yn$ for $x, y \in \mathbb{Z}$.

Firstly, $0 \in m\mathbb{Z} + n\mathbb{Z}$. This can be seen by putting $x = n$ and $y = -m$ so that

$$xm + yn = nm - mn = 0.$$

Secondly, the difference between any two members of $m\mathbb{Z} + n\mathbb{Z}$ is also in $m\mathbb{Z} + n\mathbb{Z}$. To see this, consider any two numbers in $m\mathbb{Z} + n\mathbb{Z}$, say $x_1m + y_1n$ and $x_2m + y_2n$, and find their difference:

$$(x_1m + y_1n) - (x_2m + y_2n) = (x_1 - x_2)m + (y_1 - y_2)n.$$

The coefficients $x_1 - x_2$ and $y_1 - y_2$ are both integers too, so the expression on the right, $(x_1 - x_2)m + (y_1 - y_2)n$, is of the same form, and is also a member of $m\mathbb{Z} + n\mathbb{Z}$.

Let Δ be the smallest positive difference between members of the set:

$$\Delta := \min\{z_1 - z_2 : z_1 - z_2 > 0, z_1, z_2 \in m\mathbb{Z} + n\mathbb{Z}\}.$$

Then we claim that $m\mathbb{Z} + n\mathbb{Z}$ consists precisely of all the multiples of Δ .

Theorem 29.

$$m\mathbb{Z} + n\mathbb{Z} = \Delta\mathbb{Z}.$$

Proof. This is an assertion of equality of two sets. So we divide the proof into two cases: first, we show that $m\mathbb{Z} + n\mathbb{Z} \supseteq \Delta\mathbb{Z}$, and then we show that $m\mathbb{Z} + n\mathbb{Z} \subseteq \Delta\mathbb{Z}$.

($\supseteq$)

To prove this superset relationship, we show that the right set, $\Delta\mathbb{Z}$, is a subset of the left set, $m\mathbb{Z} + n\mathbb{Z}$. To do this, we take a general member of $\Delta\mathbb{Z}$ and show that it also belongs to $m\mathbb{Z} + n\mathbb{Z}$.

Let $k\Delta$ be any multiple of Δ , where $k \in \mathbb{Z}$.

Since Δ is an integer linear combination of m and n (as explained above), there exist $x_0, y_0 \in \mathbb{Z}$ such that

$$\Delta = x_0m + y_0n.$$

Multiplying each side by k , we have

$$k\Delta = kx_0m + ky_0n.$$

Since kx_0 and ky_0 are integers, this establishes that $k\Delta$, too, is an integer linear combination of m and n .

($\subseteq$)

Now we take a general member of $m\mathbb{Z} + n\mathbb{Z}$ and show that it also belongs to $\Delta\mathbb{Z}$.

Let $xm + yn$ be an integer linear combination of m and n .

Consider what happens when we divide $xm + yn$ by Δ . The quotient q and remainder r are

$$\begin{aligned} q &= \left\lfloor \frac{xm + yn}{\Delta} \right\rfloor, \\ r &= (xm + yn) \bmod \Delta. \end{aligned}$$

- If $r = 0$ then $xm + yn$ is a multiple of Δ and we are done.
- If $r > 0$ then $0 < r < \Delta$ and

$$q\Delta < xm + yn < q\Delta + \Delta.$$

Then $q\Delta$ and $xm + yn$ differ by $< \Delta$, yet they both belong to $m\mathbb{Z} + n\mathbb{Z}$. (We saw earlier that $\Delta \in m\mathbb{Z} + n\mathbb{Z}$ and that therefore any integer multiple of it is also in $m\mathbb{Z} + n\mathbb{Z}$.) So we have a contradiction, because Δ is the smallest possible difference between two members of $m\mathbb{Z} + n\mathbb{Z}$. So this case, where $xm + yn$ is not a multiple of Δ , cannot arise.

□

We see therefore that Δ is a divisor of every member of $m\mathbb{Z} + n\mathbb{Z}$. It must also be the greatest of all such divisors, because any number $D > \Delta$ cannot be a divisor of Δ , which is a member of $m\mathbb{Z} + n\mathbb{Z}$, and therefore D cannot be a common divisor of the entire set $m\mathbb{Z} + n\mathbb{Z}$.

This establishes that Δ is actually the gcd of m and n . In other words, the gcd is the smallest integer linear combination of m and n that is greater than 0.

Theorem 30.

$$\gcd(m, n) = \text{the smallest positive value of } xm + yn \text{ with } x, y \in \mathbb{Z}.$$

As we will soon see, it is very important to be able to determine, for given m and n , two integers x, y such that

$$\gcd(m, n) = xm + yn. \quad (7.7)$$

To help see how to do this, consider that, at the outset, we have $m = 1 \cdot m + 0 \cdot n$ and $n = 0 \cdot m + 1 \cdot n$. So, at the very beginning, we already have simple equations expressing m and n as integer linear combinations of m and n . So if we can maintain equations of this type throughout the Euclidean algorithm, then hopefully we can end up with an appropriate equation (7.7). At this point, see if you can repeat one of our earlier gcd calculations — $\gcd(12, 20)$, at the end of § 7.5 on p. 240, or $\gcd(48, 174)$ at the end of § 7.6 on p. 243 — while keeping track of equations of this type as you go.

7.8 THE EXTENDED EUCLIDEAN ALGORITHM

To find such x and y such that

$$\gcd(m, n) = xm + yn,$$

it turns out to be enough to extend the Euclidean Algorithm so that it keeps track of some extra information.

Extended Euclidean Algorithm (EEA)

1. Input: $m, n \in \mathbb{N}$
2. If $m < n$ then swap them around (to ensure $m \geq n$):

$$\begin{aligned}\text{new } m &:= \text{old } n, \\ \text{new } n &:= \text{old } m.\end{aligned}$$

3. Initialise triples:

$$\begin{aligned}(a, x, y) &:= (m, 1, 0), \\ (b, z, w) &:= (n, 0, 1).\end{aligned}$$

4. If $b = 0$, Output (a, x, y) .
5. $q := \lfloor a/b \rfloor$.
6. Update the triples:

$$\begin{aligned}\text{new } (a, x, y) &:= (b, z, w), \\ \text{new } (b, z, w) &:= (a, x, y) - q \cdot (b, z, w),\end{aligned}$$

where the right-hand sides here use the old values of (a, x, y) and (b, z, w) .

7. Go back to Step 4.

In Step 6, we work out $(a, x, y) - q \cdot (b, z, w)$ using vector algebra. In this case, this means that we first multiply each member of (b, z, w) by q and then subtract each member of the resulting triple from the corresponding member of (a, x, y) . The result is $(a - qb, x - qz, y - qw)$.

At each step of the Extended Euclidean Algorithm, the triples (a, x, y) and (b, z, w) satisfy $a = xm + yn$ and $b = zm + wn$. It is easy to see that this holds for the initial triples $(m, 1, 0)$ and $(n, 0, 1)$. It is not difficult to show that the property is preserved by the updating in Step 6. It therefore holds for all triples used in the algorithm. It is intended that, when the algorithm stops, $a = \gcd(m, n)$, so the final a -triple will give us all the information we seek.

The final b -triple will have $b = 0$, so it does not give the gcd; we already have that from the final a -triple. But the final b -triple should still be calculated, when doing this manually, as a check. It is usually easy to check that the final b -triple satisfies

$$0 = zm + wn.$$

If this does not hold, then the earlier steps should be re-checked to identify the mistake.

Although there is more to do in the Extended Euclidean Algorithm than in the original Euclidean Algorithm, the extra work is really just bookkeeping. We keep track, in the triples, of exactly how the first member of each triple can be made up as an integer linear combination of m and n . But the *decisions* that we make, in the EEA, and the calculations that we do with the first member of each triple, are exactly the same in the two algorithms. So the EEA is really just the ordinary Euclidean Algorithm with extra accounting tasks.

Here is an example of using the Extended Euclidean Algorithm to compute $\gcd(27, 40)$ and also express it as an integer linear combination of its arguments. As we work out the triples, we write them underneath each other, forming a table with three columns. The left column has the numbers driving the calculation, the middle column has the value of x , and the right column has the value of y . Each row (t, x, y) satisfies $t = ax + by$. At each step, we do the appropriate integer division of the left numbers in the previous two rows to work out which multiple of the previous row has to be subtracted from the row above it.⁵

40	1	0	
27	0	1	
13	1	-1	(take previous row from the one above it)
1	-2	3	(take twice previous row from the one above it, since $\lfloor 27/13 \rfloor = 2$)
0	27	-40	(take $13 \times$ previous row from the one above it)

We could have stopped this calculation as soon as we obtained a row starting with 1, since no gcd can be < 1 . We continued the calculation one row further as a check, since if our calculations are correct, we end up with a row consisting of 0 followed by the two numbers x, y such that $ax + by = 0$, and this is usually easily checked. In the special case when the gcd is 1 (as here), the row starting with 0 contains the two numbers we started with, but with one of those negated. Note also the zig-zag pattern of negative numbers going down the last two columns. Checking that these patterns are followed is a handy way to pick up errors in manual calculations.

In this case, the calculation is correct and we find that

$$\gcd(27, 40) = 1,$$

and the algorithm also expresses this as an integer linear combination (from the row starting with 1):

$$1 = -2 \cdot 40 + 3 \cdot 27.$$

⁵ If you've done row operations in matrices, then you'll recognise that this is a similar process.

7.9 COPRIMALITY

Two integers a and b are **coprime**, or **relatively prime**, if $\gcd(a, b) = 1$. In other words, they are coprime if they have no factors in common apart from 1.

Note that coprimality is actually quite a different concept to primality, despite the similarity in name. Two integers can be coprime without either of them being prime. For example, 21 and 25 are coprime, but neither is prime. On the other hand, if a is prime, then a and b are coprime unless b is a multiple of a .

To determine if two integers are coprime, use Euclid's algorithm to determine if their gcd is 1. Again, this is different to the situation with ordinary primality, which is not so easy to test.

We saw in § 7.7, and in particular in Theorem 30, that the gcd has a characterisation in terms of integer linear combinations. In particular, $\gcd(m, n)$ is the smallest positive member of $m\mathbb{Z} + n\mathbb{Z}$. This enables a characterisation of coprimality.

Theorem 31. Integers m and n are coprime if and only if there exist $x, y \in \mathbb{Z}$ such that

$$xm + yn = 1.$$

Proof. Let $m, n \in \mathbb{Z}$. By definition, they are coprime if and only if their gcd is 1. By Theorem 30, this in turn is true if and only if 1 is the smallest positive member of $m\mathbb{Z} + n\mathbb{Z}$. But there is no smaller positive integer than 1, so 1 is the smallest positive member of $m\mathbb{Z} + n\mathbb{Z}$ if and only if $1 \in m\mathbb{Z} + n\mathbb{Z}$, which is another way of saying that there exist $x, y \in \mathbb{Z}$ such that $xm + yn = 1$. $\square$

For example, consider 21 and 25, which are coprime, as we noted above. So there must exist $x, y \in \mathbb{Z}$ such that $x \cdot 21 + y \cdot 25 = 1$. We could use $x = 6$ and $y = -5$, since

$$6 \cdot 21 - 5 \cdot 25 = 126 - 125 = 1.$$

Theorem 31 has many consequences.

Firstly, when a prime divides a product, then it must divide at least one of the factors in the product.

Theorem 32. Let p be a prime and let a and b be integers. Then

$$p \mid ab \implies p \mid a \vee p \mid b.$$

Proof. Let p, a and b be as in the statement of the theorem. Assume that $p \nmid ab$.

If $p \mid a$ then we are done already. So, suppose that $p \nmid a$. Since p is prime, this means that p and a must be coprime (since a prime is coprime to every positive integer except its own multiples).

Since p and a are coprime, there must exist $x, y \in \mathbb{Z}$ such that

$$xp + ya = 1,$$

by [Theorem 31](#). Multiplying each side by b , we have

$$xpb + yab = b.$$

Consider the two summands on the left. The first summand, xpb , is clearly a multiple of p because it has p as a factor. The second summand, yab , is a multiple of ab , but by our assumption, $p \mid ab$, so ab is also a multiple of p , so in fact the second summand is a multiple of p too. So, both summands on the left are multiples of p . So their sum is a multiple of p too. So the equation shows that b equals a multiple of p . $\square$

For example, the fact that $3 \mid (8 \times 9)$ implies that $3 \mid 8$ or $3 \mid 9$; in this case, it happens that $3 \mid 9$ (but $3 \nmid 8$).

The theorem won't work, in general, if p is not prime, though. For example, we know that $6 \mid (8 \times 9)$, because $8 \times 9 = 72$ and we know that $6 \mid 72$ (because $6 \times 12 = 72$). But we do *not* have $6 \mid 8$ or $6 \mid 9$; in fact, $6 \nmid 8$ and $6 \nmid 9$.

We are now in a position to keep a promise made on p. 236, after proving [Theorem 27](#), where we showed that every integer can be expressed as a product of primes. There, we stated that we would later show that this product of primes is always *unique*. We now prove this.

Theorem 33 (Fundamental Theorem of Arithmetic). Every positive integer has a unique factorisation as a product of primes.

Proof. We know from [Theorem 27](#) that every positive integer can be written as a product of primes. It remains to show that this can be done in only one way.

Assume, by way of contradiction, that there exists a positive integer n which can be written in two different ways as products of primes.

Among all positive integers n that can be written as a product of primes in two different ways, let's choose the smallest.

Let the primes that appear in one or both of these two products be $p_1, p_2, \dots, p_k$. Then we may suppose that the two different ways of writing n as a product of primes are

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}, \quad (7.8)$$

$$n = p_1^{f_1} p_2^{f_2} \cdots p_k^{f_k}, \quad (7.9)$$

where $e_i \in \mathbb{N}_0$ and $f_i \in \mathbb{N}_0$ for each $i \in \{1, 2, \dots, k\}$. We may assume that, for each i , the two exponents e_i and f_i are not both 0, since if they were then $p_i^{e_i} = p_i^{f_i} = 1$, so the prime p_i appears in neither of the two products, so we shouldn't have included it in the list of primes appearing in the products.

Since the two products are different, there must be at least one $i \in \{1, 2, \dots, k\}$ such that $e_i \neq f_i$.

Now, suppose one of the primes, say p_j , appears with *positive* exponents in each product: $e_j > 0$ and $f_j > 0$. (We may have $j = i$ or $j \neq i$; that doesn't matter.) We

will use this prime p_j to obtain, from n , a smaller number that can be expressed as a product of primes in two different ways, contradicting our earlier assumption that n is the smallest number of this type.

Define

$$g_j := \min\{e_j, f_j\},$$

and note that $g_j > 0$ too (since e_j, f_j are both > 0).

We can divide each side of (7.8) by $p_j^{g_j}$ to get an expression for $n/p_j^{g_j}$ as a product of all the primes in the same list. Similarly, we can divide each side of (7.9) by $p_j^{g_j}$ to get another expression for $n/p_j^{g_j}$ as a product using the same primes.

- Now, if $e_j \neq f_j$, then the prime p_j still has different exponents in the two products (but the two exponents of p_j have each been reduced by g_j).
- On the other hand, if $e_j = f_j$, then we know there is another prime p_i whose exponents are different ($e_i \neq f_i$), and the two products still have $p_i^{e_i}$ and $p_i^{f_i}$, so again we have a prime with different exponents in the two products.

So, either way, the two expressions for $n/p_j^{g_j}$ each have some prime p_i with different exponents in the two expressions. So we have two *different* expressions for $n/p_j^{g_j}$ as products of primes. Also, $g_j > 0$ implies $p_j^{g_j} > 1$, which in turn implies $n/p_j^{g_j} < n$.

The upshot of this is that we now have a *smaller* positive integer that can be written as a product of primes in two different ways. This contradicts our choice of n as the smallest positive integer with this property.

So our assumption, that one of the primes appears with a positive exponent in both the products in (7.8) and (7.9), is wrong. This, together with the fact that we earlier ruled out a prime having exponent 0 in both products, implies that every prime in our list appears with a positive exponent in one product and zero exponent in the other product. In other words, the two products use entirely separate sets of primes.

Let p be any prime appearing (with positive exponent) in the first expression for n , in (7.8). Let us write $q_1, q_2, \dots, q_l$ for the primes that appear (with positive exponent) in the second product, and let their exponents be $h_1, h_2, \dots, h_l$, respectively, so

$$n = q_1^{h_1} q_2^{h_2} \cdots q_l^{h_l}.$$

We know now, from the reasoning in the previous paragraph, that $p \notin \{q_1, q_2, \dots, q_l\}$, (because every prime appearing in the first product, in (7.8), does not appear at all in the second product, in (7.9)).

But, since p appears in the first product, we know that $p \mid n$. By Theorem 32, this implies that p divides at least one of $q_1, q_2, \dots, q_l$. But one prime cannot divide another prime unless they are the same prime. So p must equal one of $q_1, q_2, \dots, q_l$. This is a contradiction (see the end of the previous paragraph).

So our initial assumption, that there exists a positive integer that can be written as a product of two different ways, was wrong. Therefore every positive integer can only be written as a product of primes in one way. $\square$

We will see further consequences of [Theorem 31](#) in later sections, including applications in cryptography.

7.10 MODULAR ARITHMETIC

We discussed remainders and the mod operation in [§ 7.3](#). There are many situations where we are mainly interested in remainders, and all our calculations are done with them. For example:

- The seven days of the week can be regarded as names for the remainders, modulo 7, of the number of days since some reference date in the past.
- The hours in 24-hour clock time are remainders, modulo 24, of the number of hours since some reference time.
- When you press a toggle switch (which turns it on if it was off, and turns it off if it was already on), then the state of the switch (using 0 for “off” and 1 for “on”) is the remainder, modulo 2, of the number of times the switch has been pressed since it was last known to be off.
- If you are playing Monopoly, then (ignoring special situations that cause sudden jumps to other parts of the board, such as going to Jail or certain Chance and Community Chest cards) your position on the board (which is a square circuit of 40 positions) is the remainder modulo 40 of the sum of your dice throws so far.
- If you are playing a game where you move around a rectangular display with “wrap-around”, then whenever you change your coordinates by some amount, the new coordinates are obtained from the old ones by taking remainders modulo the appropriate dimensions of the display.
- The last digit of a nonnegative decimal number is its remainder modulo 10. The last bit of a binary number is its remainder modulo 2.
- Sometimes, calculations with very large numbers can be checked using calculations with some appropriate remainders of those numbers. Remainders are smaller, and in general do not contain all the information that the original numbers contain, so calculations with them cannot tell you everything about the larger numbers, and they won’t detect every possible error. But, used well, they can detect some common errors at modest cost.
- As we shall see, calculations with remainders underpin many cryptosystems including the most widely used ones today.

Let n be a positive integer. Two integers a and b are said to be **congruent modulo n** , or **congruent mod n** for short, if they differ by a multiple of n :

$$\exists k \in \mathbb{Z} \quad a - b = kn.$$

We can characterise congruence modulo n in terms of remainders.

Theorem 34. a and b are congruent modulo n if and only if their remainders modulo n are identical.

Proof. Let us find the quotients and remainders of a and b after division by n :

$$\begin{aligned} a &= q_a n + r_a, & q_a &\in \mathbb{Z}, \quad 0 \leq r_a \leq n-1; \\ b &= q_b n + r_b, & q_b &\in \mathbb{Z}, \quad 0 \leq r_b \leq n-1. \end{aligned}$$

Then the difference $a - b$ is given by

$$a - b = (q_a - q_b)n + (r_a - r_b).$$

Since $(q_a - q_b)n$ is already a multiple of n , this shows that $a - b$ is a multiple of n if and only if $r_a - r_b$ is a multiple of n . But $r_a - r_b$ is a difference between two nonnegative numbers that are each $< n$, and such a difference cannot be $\geq n$. So the only way $r_a - r_b$ can be a multiple of n is if it is actually 0. So the difference $a - b$ is a multiple of n if and only if their remainders modulo n are identical. $\square$

When a and b are congruent modulo n , we write

$$a \equiv b \pmod{n}.$$

Note that the use of “mod” here is different (but related) to the way we used it in § 7.3. There, it was a binary operation, with $a \bmod n$ giving the remainder of a after division by n . Now, used in parenthesis after an equation, it is no longer a binary operation; rather, it signifies that $\equiv$ means congruence mod n (for the n specified after “mod”). But the two uses of “mod” are closely related, since [Theorem 34](#) tells us that

$$a \equiv b \pmod{n} \quad \text{if and only if} \quad a \bmod n = b \bmod n.$$

Every integer a belongs to a family of all those integers that differ from a by a multiple of n , or in other words, all those integers that have the same remainder as a after division by n . This family may be denoted by $[a]$, provided the modulus n is clear from the context. It may also be denoted by $a + n\mathbb{Z}$, meaning the set of all numbers that can be obtained from a by adding an integer multiple of n . So

$$[a] = a + n\mathbb{Z} = \{a + kn : k \in \mathbb{Z}\} = \{\dots, a - 3n, a - 2n, a - n, a, a + n, a + 2n, a + 3n, \dots\}.$$

It is useful to describe these families using our knowledge of relations from [Chapter 2](#).

Congruence modulo n is a binary relation on $\mathbb{Z}$, so the discussion of § 2.13–§ 2.14 is applicable. Now recall our discussion of *equivalence relations* in § 2.16.

Theorem 35. For every $n \in \mathbb{N}$, congruence modulo n is an equivalence relation on $\mathbb{Z}$.

Proof. We show that congruence modulo n is reflexive, symmetric and transitive.

Reflexive:

For any $a \in \mathbb{Z}$, we have

$$a - a = 0 = 0n,$$

so

$$a \equiv a \pmod{n}.$$

Symmetric:

Suppose $a, b \in \mathbb{Z}$ are congruent modulo n :

$$a \equiv b \pmod{n}.$$

Then, by definition of congruence, there exists $k \in \mathbb{Z}$ such that

$$a - b = kn.$$

Negating each side, we have

$$b - a = -kn.$$

Since $-k \in \mathbb{Z}$, this tells us that

$$b \equiv a \pmod{n}.$$

Transitive:

Suppose $a, b, c \in \mathbb{Z}$ satisfy

$$a \equiv b \pmod{n},$$

$$b \equiv c \pmod{n}.$$

Then, by definition, there exist $k, l \in \mathbb{Z}$ such that

$$a - b = kn,$$

$$b - c = ln.$$

Adding these equations, we have

$$(a - b) + (b - c) = kn + ln.$$

Simplifying gives

$$a - c = (k + l)n.$$

Since $k + l \in \mathbb{Z}$, this tells us that

$$a \equiv c \pmod{n}.$$

□

Since congruence modulo n is an equivalence relation, it has equivalence classes. These are precisely the families $[a]$ which we introduced above. We call them **equivalence classes modulo n** , dropping the “modulo n ” when that is clear from the context, or **congruence classes**, or **residue classes**.

These equivalence classes partition the integers (as we would expect from [Theorem 12](#)). Since there are n different remainders modulo n (namely $0, 1, \dots, n-1$), there are exactly n different equivalence classes, namely $[0], [1], \dots, [n-1]$. Each of these equivalence classes can be written using others of their members, for example $[0]$ is the same class as $[n]$ and $[2n]$ and $[-n]$, and so on. But it is especially convenient to have each class represented, in this notation, by its unique member in the range $0, 1, \dots, n-1$.

We can do arithmetic with these equivalence classes. Suppose $a, b \in \mathbb{Z}$. Then the sum $[a] + [b]$ is the set containing all sums formed by adding a member of a to a member of b . This may sound complicated, because we have infinitely many sums to form! But it turns out that this is in fact another equivalence class:

$$\begin{aligned} [a] + [b] &= \{a + kn : k \in \mathbb{Z}\} + \{b + ln : l \in \mathbb{Z}\} \\ &= \{a + kn + b + ln : k, l \in \mathbb{Z}\} \\ &= \{a + b + (k + l)n : k, l \in \mathbb{Z}\} \\ &= \{a + b + hn : h \in \mathbb{Z}\}, && \text{(since } k + l \text{ ranges over all integers)} \\ &= [a + b]. \end{aligned}$$

We see, in fact, that when adding two equivalence classes $[a]$ and $[b]$ modulo n , we don't actually need to work out all possible sums; instead, we can take *just one representative* from each class, say a and b , and just add those two together (doing just one addition, instead of infinitely many), and the resulting equivalence class is just the class $[a + b]$ that the sum $a + b$ belongs to.

Similarly, it can be shown that

$$[a] - [b] = [a - b].$$

For multiplication, we can't say quite as much, but we do have

$$[a] \cdot [b] \subseteq [ab].$$

This is enough, for our purposes, since it means that, if we take any representatives $x \in [a]$ and $y \in [b]$, then their product xy belongs to $[ab]$, so $xy \equiv ab \pmod{n}$.

The question of division is more complex, partly because the integers are not closed under division. We return to this later.

Our observations so far show that, when doing arithmetic mod n , we can use any representative of each number's equivalence class (at least for addition, subtraction and multiplication). It is often most convenient to work with the representatives $0, 1, \dots, n-1$, since they seem to be the simplest possible representatives and they are the possible values of remainders modulo n .

We write $\mathbb{Z}_n$ for the set $\{0, 1, \dots, n-1\}$ endowed with addition, subtraction and multiplication but with all operations modified so that, whenever a number outside this set is produced, its remainder modulo n is used instead.

For example, consider $\mathbb{Z}_4$, which uses the set $\{0, 1, 2, 3\}$. To add 2 and 3, we start by doing so in the usual way, obtaining 5. But this is not in the set, so we replace it by its remainder modulo 4, which is 1. So,

$$2 + 3 = 1 \quad \text{in } \mathbb{Z}_4. \quad (7.10)$$

This mirrors the fact that

$$2 + 3 \equiv 1 \pmod{4}. \quad (7.11)$$

But these two statements are not saying exactly the same thing. The first statement, (7.10), is an *equation* in $\mathbb{Z}_4$, and $2+3$ cannot *equal* anything else in that system! But the second statement, (7.11), is about congruence modulo 4, and states that $2+3$ and 1 belong to the same equivalence class, but that equivalence class has many other members too. We could write (for example) $2 + 3 \equiv 37 \pmod{4}$, but we would not write “ $2 + 3 = 37$ in $\mathbb{Z}_4$ ” because 37 is not even a member of $\mathbb{Z}_4$.

The main message of this section is that, if we want to do a calculation with integers involving only addition, subtraction and multiplication, and if we only want a remainder at the end, then we can use remainders throughout. This simplifies the calculations a lot.

For example, suppose you want to calculate

$$(2360679774 - (7320508 + 41421356) * 109) \bmod 10 \quad (7.12)$$

Then, instead of doing working this out using ordinary arithmetic with these quite large integers and then finding the remainder at the end, we can take remainders as we go. First, we take the remainders mod 10 of each number in the expression (7.12):

$$\begin{aligned} 2360679774 \bmod 10 &= 4, \\ 7320508 \bmod 10 &= 8, \\ 41421356 \bmod 10 &= 6, \\ 109 \bmod 10 &= 9. \end{aligned}$$

Replacing each number in the expression (7.12) by its remainder mod 10 gives the expression

$$(4 - (8 + 6) * 9) \bmod 10$$

The first calculation is $8 + 6 = 14$, but we take remainders as we go, so we actually calculate

$$(8 + 6) \bmod 10 = 14 \bmod 10 = 4.$$

We keep doing these reductions. The full calculation is

$$\begin{aligned} (2360679774 - (7320508 + 41421356) \times 109) \bmod 10 &= (4 - (8 + 6) \times 9) \bmod 10 \\ &= (4 - (8 + 6 \bmod 10) \times 9) \bmod 10 \\ &= (4 - (14 \bmod 10) \times 9) \bmod 10 \\ &= (4 - 4 \times 9) \bmod 10 \\ &= (4 - (4 \times 9 \bmod 10)) \bmod 10 \\ &= (4 - (36 \bmod 10)) \bmod 10 \\ &= (4 - 6) \bmod 10 \\ &= -2 \bmod 10 \\ &= 8. \end{aligned}$$

We used remainders mod 10 in this example so that the remainders were clear throughout, using the fact that the remainder mod 10 is always just the final decimal digit. But the same principle holds for other moduli. For example,

$$\begin{aligned} ((705 + 48) \times 150 - 22) \bmod 7 &= ((5 + 6) \times 3 - 1) \bmod 7 \\ &\quad \text{(replacing each number by its remainder mod 7)} \\ &= (((5 + 6) \bmod 7) \times 3 - 1) \bmod 7 \\ &= ((11 \bmod 7) \times 3 - 1) \bmod 7 \\ &= (4 \times 3 - 1) \bmod 7 \\ &= (((4 \times 3) \bmod 7) - 1) \bmod 7 \\ &= ((12 \bmod 7) - 1) \bmod 7 \\ &= (5 - 1) \bmod 7 \\ &= 4. \end{aligned}$$

One application of doing calculation with remainders is as a check on ordinary integer calculations. If the final integer from a calculation does not have the same remainder as the one you get from doing the calculation entirely with remainders, then there must have been a mistake in the calculation. For example, the calculation “ $84 \times 32 = 2678$ ” cannot be right, because

$$(84 \times 32) \bmod 9 = ((84 \bmod 9) \times (32 \bmod 9)) \bmod 9 = (3 \times 5) \bmod 9 = 15 \bmod 9 = 6,$$

but

$$2678 \bmod 9 = 5.$$

This check is *one-sided* in the sense that a failure of the check (as in this example) indicates an error in the calculation but the converse does not hold. Not all errors in integer calculations will be detected this check. Furthermore, if the check detects an error, it does not tell you how to fix it.

Our choice of 9 as the modulus here was not random! The use of calculations mod 9 to check integer calculations is known as “casting out nines” and has a long history. We study this method further in [Exercise 11](#).

Although modular arithmetic works very smoothly for addition, subtraction and multiplication, there are some complications when doing division and when using exponents. We consider division in the next section (§ 7.11) and exponents in § 7.14.

7.11 MODULAR INVERSES

We omitted division from the our list of operations that you can do in $\mathbb{Z}_n$. Division is the opposite of multiplication. In ordinary arithmetic, finding x/y is the same as finding the product xy^{-1} . Here y^{-1} is the *multiplicative inverse* of y , which is *defined* by the equation

$$yy^{-1} = 1.$$

When dealing with ordinary real numbers, we call y^{-1} the *reciprocal* of y , and it exists for any nonzero real number.

Now consider inverses in $\mathbb{Z}_n$. Suppose firstly that $n = 7$: $\mathbb{Z}_7 = \{0, 1, 2, 3, 4, 5, 6\}$. What is the inverse of, for example, 2? Since $2 \times 4 = 8 \equiv 1 \pmod{7}$, we have $2^{-1} = 4$ in $\mathbb{Z}_7$. It can also be seen that: $1^{-1} = 1$, $3^{-1} = 5$, $4^{-1} = 2$, $5^{-1} = 3$, $6^{-1} = 6$. In fact, everything in $\mathbb{Z}_7$ except 0 has an inverse.

Things are not always so convenient.

Consider $\mathbb{Z}_6 = \{0, 1, 2, 3, 4, 5\}$. Which of these elements have inverses? Zero, of course, never does, and 1 is always its own inverse. What are the inverses for 2, 3, 4, 5? Suppose we try to find the inverse of 2. We want a number $z = 2^{-1}$ such that $2z \equiv 1 \pmod{6}$ — in other words, such that $2z$ is one plus a multiple of 6, i.e., 1, 7, 13, 19, ... This is impossible. So 2 has no inverse in $\mathbb{Z}_6$. Neither does 3 or 4. It can be seen that 5 is its own inverse, since $5 \times 5 = 25 \equiv 1 \pmod{6}$.

In fact, we can characterise those members of $\mathbb{Z}_n$ that have inverses.

Theorem 36. A positive integer $x \in \mathbb{Z}_n$ has an inverse in $\mathbb{Z}_n$ if and only if x and n are coprime.

Proof. Let $x \in \mathbb{Z}_n$.

If x has an inverse x^{-1} , then $xx^{-1} = 1$ in $\mathbb{Z}_n$. Therefore

$$\exists k \in \mathbb{Z} \quad xx^{-1} + kn = 1.$$

By [Theorem 31](#), this means that x and n are coprime.

Conversely, if x and n are coprime, then by [Theorem 31](#) there exist $y, z \in \mathbb{Z}$ such that

$$yx + zn = 1.$$

Let y' be the remainder of y modulo n . So $y = ln + y'$ for some $l \in \mathbb{Z}$ (recalling the definition of remainder with (7.3), but here n is the divisor). So we have

$$(ln + y')x + zn = 1,$$

which means

$$y'x + (lx + z)n = 1,$$

Since both x and y' are in $\mathbb{Z}_n$, this means that

$$y'x = 1 \quad \text{in } \mathbb{Z}_n.$$

But this is precisely the statement that y' is the multiplicative inverse of x in $\mathbb{Z}_n$. $\square$

[Theorem 36](#) shows one reason for the importance of coprimality: it determines which elements of $\mathbb{Z}_n$ have inverses. The Euclidean Algorithm can be used to determine whether or not a member x of $\mathbb{Z}_n$ is coprime and therefore whether or not it has an inverse. Furthermore, if we want to determine the inverse x^{-1} of x , then applying the Extended Euclidean Algorithm to x and n gives $y, z \in \mathbb{Z}$ such that

$$yx + zn = 1.$$

Then y (or its remainder modulo n) is the inverse of x in $\mathbb{Z}_n$.

This is the main reason we use the Extended Euclidean Algorithm: to find inverses in $\mathbb{Z}_n$.

We write $\mathbb{Z}_n^*$ for the set of members of $\mathbb{Z}_n$ that have multiplicative inverses. For example:

$$\begin{aligned}\mathbb{Z}_6^* &= \{1, 5\}, \\ \mathbb{Z}_7^* &= \{1, 2, 3, 4, 5, 6\}.\end{aligned}$$

7.12 THE EULER TOTIENT FUNCTION

The number of positive integers less than n that are coprime to n is denoted by $\varphi(n)$, and this function φ is called the **Euler totient function**. Putting it another way:

$$\varphi(n) = |\{x : 0 < x < n, \gcd(x, n) = 1\}|.$$

Exercise: determine $\varphi(6)$ and $\varphi(7)$.

From the definitions of the Euler totient function and $\mathbb{Z}_N^*$, together with [Theorem 36](#), we have

$$\varphi(n) = |\mathbb{Z}_n^*|.$$

The Euler totient function of a prime number p is given by

$$\varphi(p) = p - 1. \quad (7.13)$$

This is because every positive integer less than a prime must be coprime to it. So the maximum possible value of $\varphi(n)$ is $n - 1$, and that this is attained if and only if n is prime.

We can extend (7.13) to prime powers.

Theorem 37. For any prime p and any $m \in \mathbb{N}$,

$$\varphi(p^m) = p^{m-1}(p - 1). \quad (7.14)$$

Proof. Let p^m be a power of a prime p , where $m \in \mathbb{N}$. The total number of positive integers $< p^m$ is $p^m - 1$. How many of these are coprime to p^m ? The only proper divisors of p^m are $p, p^2, p^3, \dots, p^{m-1}$, which are all powers of p . So, if a positive integer x is not coprime to p^m , then its gcd with p^m must be a multiple of p (which includes the possibility of being a multiple of some higher power of p such as p^2, p^3 , etc.). How many multiples of p are there in the set $\{1, 2, \dots, p^m - 1\}$? The multiples of p in this set are $p, 2p, 3p, \dots, p^m - p$, noting that the last member of this sequence equals $(p^{m-1} - 1)p$. There are $p^{m-1} - 1$ of these. So we have

$$\begin{aligned} \varphi(n) &= \# \text{ positive integers } < p^m \text{ that are coprime to } p^m \\ &= \# \text{ positive integers } < p^m \text{ that are not multiples of } p \\ &= (\# \text{ positive integers } < p^m) - (\# \text{ multiples of } p \text{ that are } < p^m) \\ &= (p^m - 1) - (p^{m-1} - 1) \\ &= p^m - p^{m-1} \\ &= p^{m-1}(p - 1). \end{aligned}$$

□

To be able to compute $\varphi(n)$ when n is not a prime power, we need something more. Fortunately, we have

$$a, b \text{ coprime} \implies \varphi(ab) = \varphi(a)\varphi(b). \quad (7.15)$$

(In number theory, a function f is said to be **multiplicative** if $f(ab) = f(a)f(b)$ whenever a and b are coprime. So we have just stated that the Euler totient function is multiplicative.)

It is worth noting in passing that, if $n = pq$ is the product of two primes p and q , then

$$\varphi(n) = (p-1)(q-1). \quad (7.16)$$

This fact will be used later on, when we consider the RSA cryptosystem.

Properties (7.13)–(7.15) enable us to compute $\varphi(n)$ for any positive integer n whose complete prime factorisation is known. Here are its values for $n \leq 13$:

n :	2	3	4	5	6	7	8	9	10	11	12	13
$\varphi(n)$:	1	2	2	4	2	6	4	6	4	10	4	12

In fact, if you have a method for factorising any integer, then you can use it to compute the Euler totient function of any integer. Given a positive integer n , you first factorise it into a product of prime powers, say $p_1^{m_1} p_2^{m_2} \cdots p_k^{m_k}$, where $p_1, p_2, \dots, p_k$ are primes. Powers $p_i^{m_i}$ and $p_j^{m_j}$ of distinct primes p_i, p_j ($i \neq j$) are coprime, so

$$\varphi(n) = \varphi(p_1^{m_1} p_2^{m_2} \cdots p_k^{m_k}) = \varphi(p_1^{m_1}) \varphi(p_2^{m_2}) \cdots \varphi(p_k^{m_k}),$$

by (7.15). Then we can use (7.14) to work out each $\varphi(p_i^{m_i})$. This gives

$$\varphi(n) = (p_1^{m_1-1}(p_1-1)) (p_2^{m_2-1}(p_2-1)) \cdots (p_k^{m_k-1}(p_k-1)).$$

This can also be written as

$$\begin{aligned} \varphi(n) &= p_1^{m_1} \left(1 - \frac{1}{p_1}\right) p_2^{m_2} \left(1 - \frac{1}{p_2}\right) \cdots p_k^{m_k} \left(1 - \frac{1}{p_k}\right) \\ &= p_1^{m_1} p_2^{m_2} \cdots p_k^{m_k} \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_k}\right) \\ &= n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_k}\right). \end{aligned}$$

It is in general difficult to compute $\varphi(n)$ from scratch (i.e., if the prime factorisation of n is unknown).

7.13 FAST EXPONENTIATION

In ordinary arithmetic, raising a number a to a power m means forming the product of m copies of a :

$$a^m = \underbrace{a \times a \times \cdots a}_{m \text{ factors}}.$$

On the face of it, this means doing $m-1$ multiplications, which if m is large means a lot of work. Fortunately, there are much more efficient methods. If we express the exponent as a sum of powers of 2 (which is given by its binary representation), then we

can compute a^m in a way that does not use too many multiplications. For example, to calculate 3^{36} , observe that $36 = 2^5 + 2^2$, so

$$3^{36} = 3^{2^5+2^2} = 3^{2^5} \times 3^{2^2} = (((((3^2)^2)^2)^2)^2) \times (3^2)^2.$$

Careful accounting should show that the number of multiplications required to compute 3^{36} is 6. These are:

quantity to work out	calculation	result
3^2 :	$3^2 = 3 \times 3 =$	9
$(3^2)^2$:	$9^2 = 9 \times 9 =$	81
$((3^2)^2)^2$:	$81^2 = 81 \times 81 =$	6561
$((((3^2)^2)^2)^2)$:	$6561^2 = 6561 \times 6561 =$	43046721
$(((((3^2)^2)^2)^2)^2)$:	$43046721^2 = 43046721 \times 43046721 =$	1853020188851841
$(((((3^2)^2)^2)^2)^2 \times (3^2)^2)$:	$1853020188851841 \times 81 =$	14824161510814728

This compares well with the naïve method, which requires 35 multiplications.

The idea indicated here can be used as the basis of an algorithm for exponentiation. Such an algorithm can compute a^m with at most $2\lfloor \log_2 m \rfloor$ multiplications, which can be shown to be at most twice the number of bits in the binary representation of m .

7.14 MODULAR EXPONENTIATION

Modular exponentiation just means doing exponentiation in $\mathbb{Z}_n$.

The fast exponentiation method of § 7.13 can be used for modular exponentiation too, and again it significantly reduces the number of multiplications required. For example, to compute $3^{36} \bmod 25$, we can do the six multiplications described on page 263 in § 7.13 and obtain:

$$3^{36} \bmod 25 = 150094635296999121 \bmod 25 = 21.$$

But when doing exponentiation in $\mathbb{Z}_n$, we only ever need to multiply numbers that are $< n$, so we can save effort by reducing the factors mod n as we go. In the example of $3^{36} \bmod 25$, the six multiplications we need to do become much simpler:

quantity to work out	calculation	result
$3^2 \bmod 25$:	$3^2 = 3 \times 3 = 9$	9
	$3^2 \bmod 25 = 9 \bmod 25 = 9$	9
$(3^2)^2 \bmod 25$:	$9^2 = 9 \times 9 = 81$	81
	$9^2 \bmod 25 = 81 \bmod 25 = 6$	6
$((3^2)^2)^2 \bmod 25$:	$6^2 = 6 \times 6 = 36$	36
	$6^2 \bmod 25 = 36 \bmod 25 = 11$	11
$((((3^2)^2)^2)^2 \bmod 25$:	$11^2 = 11 \times 11 = 121$	121
	$11^2 \bmod 25 = 121 \bmod 25 = 21$	21
$(((((3^2)^2)^2)^2)^2 \bmod 25$:	$21^2 = 21 \times 21 = 441$	441
	$21^2 \bmod 25 = 441 \bmod 25 = 16$	16
$(((((3^2)^2)^2)^2)^2 \times (3^2)^2 \bmod 25$:	$16 \times 6 = 96$	96
	$21^2 \bmod 25 = 96 \bmod 25 = 21$	21

For modular exponentiation, an even more dramatic simplification is possible. This is a consequence of the following result, which is of fundamental importance.

Theorem 38. (Euler) If x is coprime to n , then

$$x^{\varphi(n)} \equiv 1 \pmod{n}. \quad (7.17)$$

Proof. Let x be coprime to n . Let the $\varphi(n)$ members of $\mathbb{Z}_n^*$ be

$$y_1, y_2, \dots, y_{\varphi(n)}. \quad (7.18)$$

If we multiply each of these by x , then they become

$$xy_1, xy_2, \dots, xy_{\varphi(n)}. \quad (7.19)$$

Now, these must all be distinct in $\mathbb{Z}_n$ too.

- If that were not the case, then we would have $xy_i \equiv xy_j \pmod{n}$ with $i \neq j$. Then we can multiply each side by x^{-1} (which exists since x is coprime to n) to obtain $y_i \equiv y_j \pmod{n}$, which means that $y_i = y_j$ (since y_i and y_j are both in $\mathbb{Z}_n$). So our distinct members of $\mathbb{Z}_n^*$ would not all be distinct after all, a contradiction.

Therefore our two lists, (7.18) and (7.19), each give all the $\varphi(n)$ elements of $\mathbb{Z}_n^*$, with the only difference between the lists being the order of the elements. In effect, multiplication of all elements by x just reorders the list.

Now let's form the products, in $\mathbb{Z}_n^*$, of the elements in each list, and also rearrange the second product by collecting all the x factors together:

$$\begin{aligned} \text{product in } \mathbb{Z}_n^* \text{ of elements in (7.18): } & y_1 y_2 \cdots y_{\varphi(n)} \\ \text{product in } \mathbb{Z}_n^* \text{ of elements in (7.19): } & x y_1 x y_2 \cdots x y_{\varphi(n)} = x^{\varphi(n)} y_1 y_2 \cdots y_{\varphi(n)} \end{aligned}$$

Since the two products come from the same elements being multiplied, they must be equal. Therefore

$$y_1 y_2 \cdots y_{\varphi(n)} \equiv x^{\varphi(n)} y_1 y_2 \cdots y_{\varphi(n)} \pmod{n}. \quad (7.20)$$

Now, since all the elements $y_1, y_2, \dots, y_{\varphi(n)}$ are elements of $\mathbb{Z}_n^*$, their product $y_1 y_2 \cdots y_{\varphi(n)}$ must be in $\mathbb{Z}_n^*$ too. Therefore $y_1 y_2 \cdots y_{\varphi(n)}$ has an inverse. Multiplying each side of (7.20) by this inverse, we obtain

$$x^{\varphi(n)} \equiv 1 \pmod{n},$$

which completes the proof.⁶ □

An important special case of Theorem 38 is when n is a prime p . Then (7.13) and (7.17) give

$$x^{p-1} \equiv 1 \pmod{p} \quad \text{for all } x \in \{1, \dots, p-1\}. \quad (7.21)$$

This is known as *Fermat's Little Theorem*. The generalisation above (Theorem 38) is due to Euler.

Theorem 38 is used at the heart of the RSA cryptosystem and Equation 7.21 is at the core of the Diffie-Hellman key exchange scheme (§ 7.18).

One of the main computational uses of Theorem 38 is to simplify modular exponentiation. The Theorem enables us to work mod $\varphi(n)$ in the exponent. So if we want to simplify $a^x \pmod{n}$, and we know that $x \equiv y \pmod{\varphi(n)}$, then we can write $a^x \equiv a^y \pmod{n}$. For example, suppose we want to simplify $2^9 \pmod{7}$. We compute $\varphi(7) = 7 - 1 = 6$ (by (7.13)). So the exponent 9 can be simplified mod 6, giving $9 \pmod{6} = 3$. Hence $2^9 \pmod{7} = 2^3 \pmod{7} = 8 \pmod{7} = 1$.

Here's how this works for $3^{36} \pmod{25}$. First, we find the Euler totient function of the modulus, 25.

$$\varphi(25) = \varphi(5^2) \stackrel{(\text{by 7.14})}{=} 5 \cdot (5 - 1) = 20.$$

Then we reduce the exponent mod $\varphi(n)$, in this case mod 20.

$$3^{36} \pmod{25} = 3^{36 \pmod{\varphi(25)}} \pmod{25} = 3^{36 \pmod{20}} \pmod{25} = 3^{16} \pmod{25}.$$

⁶ This theorem can be understood more deeply using *group theory*, which is beyond the scope of these Course Notes, but we give an outline of this connection here. It can be shown that $\mathbb{Z}_n^*$ is a *group* under multiplication. This means that it is *closed* (if two numbers a, b are coprime to n , then their product ab will be coprime to n as well), the multiplication operation is *associative*, there is a multiplicative *identity* (in this case, 1), and every element has a multiplicative *inverse* (Theorem 36). It is a basic theorem of group theory that, if g is a member of the group and k is the size of the group, then g^k equals the multiplicative identity of the group. Theorem 38 is simply the application of these results to the group $\mathbb{Z}_n^*$.

We now have a much simpler problem, with an exponent of 16 instead of 36. A simpler exponent will typically mean fewer multiplications are needed. In this case, we have

$$3^{36} \bmod 25 = 3^{16} \bmod 25 = 3^{2^4} \bmod 25 = (((3^2)^2)^2)^2 \bmod 25.$$

This requires just four multiplications:

quantity to work out	calculation	result
$3^2 \bmod 25$:	$3^2 = 3 \times 3 = 9$	9
	$3^2 \bmod 25 = 9 \bmod 25 = 9$	9
$(3^2)^2 \bmod 25$:	$9^2 = 9 \times 9 = 81$	81
	$9^2 \bmod 25 = 81 \bmod 25 = 6$	6
$((3^2)^2)^2 \bmod 25$:	$6^2 = 6 \times 6 = 36$	36
	$6^2 \bmod 25 = 36 \bmod 25 = 11$	11
$((((3^2)^2)^2)^2) \bmod 25$:	$11^2 = 11 \times 11 = 121$	121
	$11^2 \bmod 25 = 121 \bmod 25 = 21$	21

This can be compared with the longer calculation on p. 263 in § 7.14. The saving becomes much greater for higher exponents.

7.15 PRIMITIVE ROOTS

Suppose $\gcd(x, n) = 1$. Then Theorem 38 tells us that

$$x^{\varphi(n)} \equiv 1 \pmod{n}.$$

If, in addition,

$$x^k \not\equiv 1 \pmod{n} \quad \text{for all } k < \varphi(n),$$

then we say that x is a **primitive root** of n . Such an x has the property that its powers $x, x^2, x^3, \dots$ give *all the elements* of $\mathbb{Z}_n^*$, in some order, reaching 1 at $x^{\varphi(n)}$. In this sense, x **generates** $\mathbb{Z}_n^*$. If x is not a primitive root, then its powers will go through some proper subset of $\mathbb{Z}_n^*$.

We emphasise that, among the members of $\mathbb{Z}_n^*$, it is the primitive roots that have the *greatest possible range of values of their powers*. This will be important later on.

Example:

Suppose $n = 7$. Then $\varphi(7) = 6$, by (7.21), since 7 is prime, and $\mathbb{Z}_7^* = \{1, 2, 3, 4, 5, 6\}$. Consider the successive powers of 3 in $\mathbb{Z}_7^*$:

k :	1	2	3	4	5	6
$3^k \bmod 7$:	3	2	6	4	5	1

We see from this that $3^{\varphi(7)} = 3^6 \equiv 1 \pmod{7}$, and that $3^k \not\equiv 1 \pmod{7}$ for all $k < 6$. So the values of $3^k \bmod 7$, for $k = 1, \dots, 6$, are all the elements of $\mathbb{Z}_7^*$. So 3 is a primitive root of 7.

On the other hand, 2 is not a primitive root of 7. Consider its powers:

k :	1	2	3
$2^k \bmod 7$:	2	4	1

So $2^3 \equiv 1 \pmod{7}$, with the exponent $3 < \varphi(7)$, which means the definition of primitive root is not satisfied. Taking further powers, with $k = 4, 5, 6, \dots$, will just give the same numbers 2, 4, 1, ...; we will not get anything new.

Not all positive integers have primitive roots. For example, there are no primitive roots of 8. Let us consider all the candidates. Firstly, observe that $\varphi(8) = 4$ and $\mathbb{Z}_8^* = \{1, 3, 5, 7\}$. The following table shows that no member of $\mathbb{Z}_8^*$ can be a primitive root.

k :	1	2
$1^k \bmod 8$:	1	
$3^k \bmod 8$:	3	1
$5^k \bmod 8$:	5	1
$7^k \bmod 8$:	7	1

Note that it does not help to look at powers of some number x which is in $\mathbb{Z}_8$ but not in $\mathbb{Z}_8^*$. For example, try $x = 2$. Its powers in $\mathbb{Z}_8$ are 2, 4, 0, 0, 0, ...; it never reaches 1, and in fact once it reaches 0 it is stuck there. This is typical of what happens when taking successive powers of a member of $\mathbb{Z}_n \setminus \mathbb{Z}_n^*$. So a primitive root of n , if one exists, must in fact be a member of $\mathbb{Z}_n^*$.

Numbers that have primitive roots have been characterised.

Theorem 39. The numbers which have primitive roots are 1, 2, 4, and those of the form p^k or $2p^k$, where p is an odd prime and $k \in \mathbb{N}$. $\square$

The number 8 is not covered by this list of possibilities. (It is a power of the *even* prime, 2, rather than an odd one.) So it cannot have a primitive root.

Theorem 40. If n has a primitive root, then it has $\varphi(\varphi(n))$ of them. $\square$

This result can be proved by elementary methods, and insight into why it is true can be gained by playing with small examples.

Consider our earlier example of $p = 7$. We saw that 3 is a primitive root of 7, and found all its powers 3^i for $i \leq 6$, with $3^6 \equiv 1 \pmod{7}$. We then saw that 2 is not a primitive root of 7, because its powers reach 1 too soon: $2^3 \equiv 1 \pmod{7}$.

Now let's use the representation of 2 as a power of the primitive root 3, and look more closely at why 2 fails to be a primitive root.

We have $2 \equiv 3^2 \pmod{7}$, and the exponent here, also 2, is a divisor of 6, with $2 \times 3 = 6$, so

$$2^3 \equiv (3^2)^3 = 3^{2 \times 3} = 3^6 \equiv 1 \pmod{7}.$$

This illustrates that we can use the *exponents* to work out all the primitive roots. The exponent 2 is a factor of 6, and as we have just seen, this stops $3^2 \pmod{7} = 2$ from being another primitive root. The exponent 3 is also a factor of 6, so $3^3 \pmod{7} = 6$ won't be a primitive root either:

$$6^2 \equiv (3^3)^2 = 3^{3 \times 2} = 3^6 \equiv 1 \pmod{7}.$$

But it's not just about being a *factor* of 6. Consider exponent 4, which gives $3^4 \pmod{7} = 4$. This is not a factor of 6, but it does have a factor of 2 in common with 6. Because of this, it won't give us a primitive root either:

$$4^3 \equiv (3^4)^3 = 3^{4 \times 3} = 3^{12} = 3^{6 \times 2} = (3^6)^2 \equiv 1^2 = 1 \pmod{7}.$$

So, if the exponent is not coprime to 6, then it cannot yield a primitive root. This prevents 3^2 , 3^3 and 3^4 from being primitive roots of 7. So none of 2, 6, 4 are primitive roots of 7.

On the other hand, if the exponent is coprime to 6, it will yield a primitive root when our first primitive root 3 is raised to that exponent. This means that exponents 1 and 5 yield primitive roots. So the two primitive roots of 7 are $3^1 = 3$ and $3^5 \pmod{7} = 5$.

So the number of primitive roots of 7 is indeed

$$\varphi(\varphi(7)) = \varphi(7-1) = \varphi(6) = \varphi(2 \cdot 3) = \varphi(2)\varphi(3) = (2-1)(3-1) = 1 \cdot 2 = 2.$$

This illustrates [Theorem 40](#), and also that, once we have one primitive root a of n , the others have the form a^k where k is coprime to $\varphi(n)$.

There is, at present, no fast algorithm for finding a primitive root of a number.

7.16 ONE-WAY FUNCTIONS

Cryptography is about transforming information efficiently in such a way that reversing the transformation is difficult for someone who is not authorised to do so. So it is natural to consider functions that are easy to compute but hard to invert.

Informally, a function is **one-way** if it is:

1. "easy" to compute;
2. "hard" to invert, "usually".

This can be made precise, but doing so is beyond the scope of this unit.

One-way functions are *believed* to exist, and there is a number of functions that are *regarded* as one-way functions. We will see some examples shortly. However, no function has been rigorously proved to be one-way in the precise formal sense of that term.

Application of one-way functions: password checking

Let f be a one-way function. Suppose we have users U_i , $1 \leq i \leq N$, and that user U_i has password P_i , $1 \leq i \leq N$. In order to authenticate a user logging into a computer system, their password must be checked to ensure it is the correct one for that user. The obvious way to do this is to have a place in the system where users' passwords are stored, so that a password entered by a user logging in can be checked against the stored password to see if it is the same. However, storing passwords themselves is too much of a risk. Someone who gains access to the place where passwords are stored will then be able to find the passwords associated with usernames, which gives them the ability to pretend to be one of those users when logging into the system.

It is better for the system to store, with each username, the *image* of the password under the one-way function f . So the information stored is a list of pairs $(U_i, f(P_i))$, $1 \leq i \leq N$. When a user U_i logs in, the password P entered is first fed into the one-way function f , which computes $f(P)$. The system looks up the list of users, finds the pair $(U_i, f(P_i))$, and tests whether $f(P) = f(P_i)$. If so, the entered password is accepted and the user is admitted to the system. If not, the login attempt is rejected.

Under this system, if someone gains access to a pair $(U_i, f(P_i))$, they still do not know the password. To find the password, they would have to work out P_i just from knowledge of $f(P_i)$. But this is precisely the problem of inverting f . If f is a one-way function, then this should almost always be difficult. So the value $f(P_i)$ is of no help in logging in.

Note also that, since f is a one-way function, it is easy to compute, so *checking* passwords can be done quickly, even though *finding* a password, just from its image under f , is difficult.

Sometimes, this method of password verification is described as using "encryption", and the transformed password $f(P_i)$ is called an "encrypted password". Strictly speaking, this is inaccurate, since no key is involved: the password is always transformed in the same way. Furthermore, there is no-one to play the role played by the receiver of a secret message in an ordinary cryptosystem: there is no-one who is intended to be able to recover P_i from $f(P_i)$.

7.17 MODULAR EXPONENTIATION WITH FIXED BASE

We now consider an important candidate one-way function.

Suppose we fix a modulus n and a base $a < n$.

The function **modular exponentiation with fixed base** performs the mapping

$$x \mapsto a^x \bmod n.$$

We have seen that this can be computed efficiently, using the techniques of § 7.14. Here, we will focus on using exponents $x < \varphi(n)$, since we saw in § 7.14 that every other possible exponent can be reduced to such an x .

Although this function is easy to compute, it seems to be much harder to invert.

For the inverse, we are given $y < n$ and must find x such that $a^x \equiv y \pmod{n}$. If we want to make the inverse to be as hard as possible, we should ensure that there are as many potential values of x as possible, so that exhaustively searching through all possible x , to find the one which satisfies $a^x \equiv y \pmod{n}$, takes as long as possible. To this end, it is desirable to choose a to be a primitive root of n , because such numbers give the greatest range of possible values of their powers (as we observed early in § 7.15). The inverse problem is then the following.

Discrete Logarithm:

Fix: $n \in \mathbb{N}$, primitive root a of n .

Input: $y \in \mathbb{Z}_n^*$

Output: $x \leq \varphi(n)$ such that $a^x \equiv y \pmod{n}$.

When x and y are related in this way, we may write

$$x = \log_a y.$$

The current belief is that Discrete Logarithm has no fast algorithm. In fact, it seems to be of about the same difficulty as factorising integers. The best known algorithms for each take, very roughly, similar amounts of computation time.

Furthermore, Discrete Log is believed to be *almost always* hard.

Modular exponentiation with a fixed primitive root as the base is believed to be a one-way function, although this has not been proved, and proving it would solve a major open problem in computer science.

Recall that $\varphi(n)$ is maximised when n is prime. So, for the best possible candidate one-way function, we should choose a large prime p and then choose a to be a primitive root of p . This is exactly what we do when using this candidate one-way function to help distribute cryptographic keys securely. This is described in the next section.

7.18 DIFFIE-HELLMAN KEY AGREEMENT SCHEME

Suppose we have a large number of users who wish to be able to communicate with each other. Suppose that any pair of them who communicate want secrecy, so that no-one else (including others in our large set of users) can read their messages. With traditional cryptosystems (including the type discussed in § 2.10), they will need to

agree in advance on a shared secret key. This requires each pair of users to have their own secure communications channel. Not only is this expensive, but it requires time to arrange. This severely limits the ability of the users to communicate spontaneously, without having planned ahead of time to do so. It is clear that the demands of modern electronic communications and commerce are quite at odds with these limitations.

A remarkable solution to this problem was proposed by Diffie and Hellman in 1976, in a paper that marked the beginning of a revolution in cryptography.⁷ Their method uses the one-way function we have just met: modular exponentiation with fixed base, where the base is a primitive root of a large prime. It works as follows.

Firstly, we fix a large prime p and a primitive root a of p . These numbers are *public*, in that they are disseminated, without any encryption, to all users of the system, and *system-wide*, in that all users use the same values of p and a . Each user generates their own private random number $x \in \{1, \dots, p-1\}$, which is regarded as a member of $\mathbb{Z}_p^*$, and from it generates $y = a^x \bmod p$, which is made public. (Note that we are refraining from calling these numbers *keys* since, strictly speaking, they are not used directly to encrypt or decrypt messages.)

Observe that anyone who wants to determine some user's private number x is faced with the problem of inverting a candidate one-way function. They see p , a and $y = a^x \bmod p$, and from this must determine x . This is exactly the Discrete Log problem, i.e., the inverse of modular exponentiation with fixed base a . Since this appears to be a difficult problem, provided p is large, we will assume that the private number of each user is secure.

Suppose now that two users, Alice and Bob, want to communicate. Suppose that Alice's private and public numbers are x_A and y_A , respectively, while Bob's are x_B and y_B . Each of them knows the system-wide constants p and a , and each can read the other's public number. The exact means by which this is done is an implementation detail that does not concern us here. Perhaps they send the public numbers to each other (unencrypted), or perhaps the public numbers of all users are collected together in some central, publicly available file.

Now, when Alice reads Bob's public number y_B , she calculates her *key* k_{AB} by raising Bob's *public* number to the power of her own *private* number:

$$k_{AB} := y_B^{x_A} \bmod p.$$

Note that only she can do this calculation, since only she knows x_A .

Similarly, Bob reads Alice's *public* number y_A , and then raises it to the power of his own *private* number x_B , obtaining his *key* k_{BA} :

$$k_{BA} := y_A^{x_B} \bmod p.$$

⁷ W. Diffie and M. E. Hellman, New directions in cryptography, *IEEE Transactions on Information Theory* IT-22 (1976) 644–654.

Note that only he can do this, because no-one else knows x_B . Note also that Alice and Bob can do these computations independently of each other (provided each has made their public number available).

Although of Alice and Bob have each done a computation that only they could do, the keys they each compute turn out, remarkably, to be exactly the same:

$$k_{AB} = y_B^{x_A} = (a^{x_B})^{x_A} = a^{x_B x_A} = a^{x_A x_B} = (a^{x_A})^{x_B} = y_A^{x_B} = k_{BA}, \quad \text{all in } \mathbb{Z}_p^*,$$

with the exponentiations being done mod p using the methods of § 7.14.

The outcome of this process is that Alice and Bob have arrived at the same key, without that key itself having been sent anywhere. No secure channel is needed for key transmission. Neither party alone could determine the key they agree on, as it depends on the private numbers of both of them.

The problem facing a cryptanalyst is the following:

Diffie-Hellman problem:

Given: p, a, a^{x_A}, a^{x_B} (both mod p);

Find: $a^{x_A x_B} \bmod p$.

This is believed to be about as difficult as Discrete Log, although we only know that it cannot be significantly harder than Discrete Log:

Theorem 41. Any algorithm for Discrete Log can be used to construct an algorithm for the Diffie-Hellman problem. A fast algorithm for Discrete Log yields a fast algorithm for the Diffie-Hellman problem.

Proof. Suppose we have an algorithm for Discrete Log. The inputs to the Diffie-Hellman problem are p, a, a^{x_A} and a^{x_B} , the last three being members of $\mathbb{Z}_p^*$. If we give p, a and (say) a^{x_A} to the Discrete Log algorithm, then it will return x_A . We can then compute $k_{AB} = a^{x_A x_B} \bmod p$, just as Alice did above:

$$k_{AB} = (a^{x_B})^{x_A} = a^{x_A x_B},$$

with exponentiation mod p . This gives the desired output for the Diffie-Hellman problem.

If the algorithm for Discrete Log is fast, then this procedure for the Diffie-Hellman problem, which uses it, will be fast too, since there is not much extra work involved, and fast exponentiation techniques are used. $\square$

Open problem:

Does the algorithmic relationship between Discrete Log polynomial time and the Diffie-Hellman problem go the other way too? In other words, is there an efficient way to transform algorithms for the Diffie-Hellman problem into algorithms for Discrete Log?

It is widely believed that this is the case, but it has not yet been proved.

The purpose of the Diffie-Hellman system is to enable two parties to agree on a common key, over an insecure channel. Once this key is agreed, they can send and receive messages securely using a good secret-key cryptosystem.

The Diffie-Hellman system is not, itself, a cryptosystem. If used in isolation (i.e., without subsequent use of a secret-key cryptosystem), neither party can use it to send a message of their own choosing, in a secure form, to the other party. The only information either party receives from the other is the other's public number, which can in principle be read by anyone, and in any case is random since it is a function of that user's random private number. The only information that either party chooses for themselves is their private number, but the other party never sees this so it cannot be used as a message, and in any case it should be randomly chosen to make it as unpredictable as possible.

Example:

We use small numbers in this example, so the calculations can be done manually to help understand how it works, but of course you need very large numbers in real applications.

For our public global parameters, we use $p = 11$ and $a = 2$. In [Exercise 17](#) you will show that 2 is a primitive root of 11. We will take that as given, for now.

Suppose Alice chooses private number $x_A = 3$ and Bob chooses private number $x_B = 6$. Then they compute their public numbers y_A and y_B as follows.

Alice	Bob
$y_A = a^{x_A} \bmod p$	$y_B = a^{x_B} \bmod p$
$= 2^3 \bmod 11$	$= 2^6 \bmod 11$
$= 8.$	$= 9.$

So Alice sends Bob her public number $y_A = 8$ and Bob sends Alice his public number $y_B = 9$. Then Alice raises Bob's public number y_B to the power of her own private number x_A :

$$k_{AB} = y_A^{x_B} = 9^3 \equiv 3 \pmod{11}.$$

Meanwhile, Bob raises Alice's public number y_A to the power of his own private number x_B :

$$k_{AB} = y_B^{x_A} = 8^6 \equiv 3 \pmod{11}.$$

We see that $y_A^{x_B} \equiv y_B^{x_A} \pmod{11}$, which is what we expect, since both are equal to

$$a^{x_A x_B} = 2^{3 \cdot 6} = 2^{18} \equiv 3 \pmod{11}.$$

This last calculation, using both x_A and x_B , uses the private numbers of both Alice and Bob. So this particular calculation cannot be done by either of them individually, provided they are each able to keep their own private number secret. But we have seen that they can still each work out the final number $k_{AB} = 3$, even though they do not know the other's private number.

The final number, $k_{AB} = 3$ in this case, is then ready for use as a key in a proper cryptosystem.

7.19 EXERCISES

1.

- (a) Write the statement that integer d is a divisor of integer n as a predicate logic expression, using the integer multiplication function and *without* using the divisibility predicate $|$.
- (b) Write the statement that integer n is prime as a predicate logic expression. You may use the divisibility relation $|$ and the inequality relation $\leq$.
- (c) Write the statement that integer d is the greatest common divisor of integers m and n as a predicate logic expression, using $|$ and $\leq$.

2. A positive integer n is **perfect** if it equals the sum of its own proper divisors.

- (a) Show by direct calculation that the first two perfect numbers are 6 and 28.
- (b) The third perfect number is 496. Verify that 496 is perfect.

Over 50 perfect numbers are known, but it is not yet known if there are infinitely many. All known perfect numbers are even. It is a longstanding open problem to determine if any odd perfect numbers exist.

3. If you know the day of the week on which a given date falls in a given year, you can work out the day of the week of the same date in the next year.

Suppose the days of the week are represented by members of $\mathbb{Z}_7$, with Sunday represented by 0, Monday by 1, and so on. Let $d \in \mathbb{Z}_7$ be the day of the week of a given date this year.

- (a) Give an expression in terms of d , using the mod operation, for the day of the week for the same date next year.
- (b) How would you modify the expression if you knew there was a leap day (29 February) between that date in one year and the same date in the next year?

(c) Now let d be the day of the week of 29 February in some leap year. Assuming the next leap year is four years later, give an expression in terms of d for the weekday of the next 29 February, again using the mod operation.

4. In (2.1) on p. 54, we introduced a simple (and quite poor) random number generator based on a function that computes

$$\text{LCG}(x) = \text{the last 31 bits of } 1103515245x + 12345.$$

Restate this definition using the mod operation.

5. The online game Primel (<https://converged.yt/primel/>) is about guessing an unknown five-digit prime number and making the best use of information you are given after each guess. It was created by Hannah Park in 2022, and is based on Wordle (<https://www.nytimes.com/games/wordle/>).

Start by reading the rules at the Primel website and playing the game. Keep in mind that

- you are not penalised for guessing numbers that are *not* prime (although you don't get any information from non-prime guesses about the unknown prime number).
- You have a maximum of six *prime* guesses. So you must choose your guesses carefully, taking into account what you learn from previous guesses.

Think about what would make a good first guess in Primel, i.e., one which is likely to give as much information as possible about the unknown prime number.

- (a) Are there any prohibitions on certain digits in certain positions, in prime numbers in general? If so, what does this mean for the digits you might want in the five-digit prime number you use for your first guess?
- (b) The density of prime numbers — i.e., the proportion of numbers in a given interval that are prime — decreases as numbers get higher, although the pattern is a bit irregular and unpredictable. This effect is present even among five digit primes. So, for example, there are more with first digit 1 than with first digit 5.

With this in mind, what five digits would be best to use in your first guess? Which five-digit prime number do you recommend, as the first guess?

- (c) When playing the game, try asking ChatGPT for help with choosing your next guess. For example, if you know that the unknown prime does not have 1, 2 or 3, then ask ChatGPT to suggest a five-digit prime number that does not contain 1, 2 or 3. When it responds, check if the number it gives is prime or not, using an authoritative program or a table that includes all five-digit primes (e.g., <https://t5k.org/lists/>

[small/10000.txt](#), https://prime-numbers.de/blox29prime/blox_0.txt). How many attempts are necessary before ChatGPT gives an actual prime satisfying the conditions?

6. The rule for determining if a year is a leap year is as follows.

A positive integer is a leap year if and only if it is a multiple of 4, unless it is also a multiple of 100, in which case it is not a leap year unless it is a multiple of 400.

Define the unary predicate Leap with domain $\mathbb{N}$ to be True if and only if its argument is a leap year.

Using Leap and the divisibility predicate $|$, write the above rule for leap years in predicate logic.

7. Prove that the square of an odd number is an odd number and the square of an even number is an even number.

8. Restate the definitions of the Caesar slide encryption and decryption functions ([Exercise 15](#)) using the mod operation.

9. Prove each of the following statements.

- (a) $4 \mid n$ if and only if the last two digits of the decimal representation of n give a multiple of 4.
- (b) $8 \mid n$ if and only if the last three digits of the decimal representation of n give a multiple of 8.
- (c) $25 \mid n$ if and only if the last two digits of the decimal representation of n give a multiple of 25.
- (d) Extend (a) and (b) to higher powers of 2.
- (e) (Challenge) These divisibility tests, and those at the end of § 7.1^α, all have the property that the presence of divisor d can be determined just from some constant number of digits at the end of the number. In other words, the number of digits of n that you need to look at is *independent* of n . For example, to test divisibility by 4, you only look at the last two digits, regardless of how large n is.

Can you characterise when this happens? For which d is it the case that, to test divisibility of n by d , you only need to look at some constant number of digits at the very end of n ?

10. The **digital sum** of a positive integer n , written in standard decimal notation, is the sum of its digits. Let's denote it by $ds(n)$.

For example,

$$\text{ds}(1984) = 1 + 9 + 8 + 4 = 22.$$

- (a) Prove that, for all $n \in \mathbb{N}$,

$$n \bmod 3 = \text{ds}(n) \bmod 3.$$

This can be used repeatedly to compute $n \bmod 3$: just keep computing the digital sum of the digital sum of the digital sum ... until you get just a single digit, and then you can determine the remainder manually.

- (b) A similar method works for remainders modulo 9. Explain why, briefly.
- (c) The **alternating digital sum** of n is obtained by alternately adding and subtracting its digits, starting with addition at the right-hand end and moving to the left. We'll denote it by $\text{ads}(n)$.

For example,

$$\text{ads}(1984) = -1 + 9 - 8 + 4 = 4.$$

Prove that, for all $n \in \mathbb{N}$,

$$n \bmod 11 = \text{ads}(n) \bmod 11.$$

- (d) Devise a technique of similar type for working out $n \bmod 3$ from the bits of the binary representation of n .

11. For each of the following equations, apply the method from [Exercise 10\(b\)](#) to work out, by hand, the remainder mod 9 of each side. In each case, comment on what comparing these two remainders tells you.

(a) $92 \times 31 = 2847$

(b) $92 \times 31 = 2852$

(c) $92 \times 31 = 2861$

(d) $2718281828 \times 3141592653 = 8539734219628209634$

12. Use the Euclidean algorithm to find the greatest common divisor of the following pair of consecutive Fibonacci numbers: 34 and 55.

What do you notice about the other numbers found at each step of the algorithm?

Prove, by induction on n , that for all $n \geq 3$ the Euclidean algorithm uses $n - 2$ subtractions to compute the gcd of the consecutive Fibonacci numbers f_n and f_{n+1} .

and that every intermediate number found during the computation is also a Fibonacci number.

13. Use the Extended Euclidean Algorithm to show that 86 and 99 are coprime, and to express 1 as in integer linear combination of them, and to find the inverse of 86 in $\mathbb{Z}_{99}$.

14.

(a) Find $\varphi(n)$ for all n up to 20.

(b) Find primitive roots for a selection of values of n .

15. The **modular multiplication** cryptosystem works as follows. The message and cypher spaces are each the set of all strings over the 26-letter English alphabet. Each letter is treated as a member of $\mathbb{Z}_{26}$, with a,b,...,z being 0,1,...,25, respectively. The key is $k \in \mathbb{Z}_{26}$. A message m is encrypted using the key k to produce cyphertext c as follows, where m_i is the i -th letter of the message and c_i is the i -th letter of the cyphertext:

$$c_i = km_i \bmod 26.$$

(a) This definition of keyspace and encryption is not quite correct as it stands. Not all keys work properly. What restriction must be placed on members of $\mathbb{Z}_{26}$ so that they can work properly as keys for modular multiplication? Specify the keyspace using appropriate mathematical notation.

(b) Define the decryption function for modular multiplication.

(c) How would modular multiplication work in general, for an arbitrary alphabet size n ? Define its keyspace, encryption function and decryption function.

16. Compute

$$1752^{31415926535897932384626433832795028841971693993751058209749445923078164} \bmod 125$$

by hand, showing your working.⁸

17. In this exercise, we investigate the primitive roots of 11.

(a) If a positive integer a is a primitive root of 11, what is the least k such that $a^k \equiv 1 \pmod{11}$?

⁸ The base here is the year in which the Gregorian calendar was introduced in Britain. The exponent is $\lfloor 10^{70}\pi \rfloor$. But you do not need this information to do the computation.

- (b) Show that 2 is a primitive root of 11 by working out its powers 2^i for as far as necessary. You can do this by repeatedly multiplying by 2, taking remainders mod 11 as you go.
- (c) Study the exponents i for each 2^i in your list of powers from (b). Which of these exponents is coprime to $\varphi(11)$?
- (d) Using (c), list all the primitive roots of 11.

18. Alice and Bob are using the Diffie-Hellman scheme to agree on a key. Their public global parameters are prime $p = 11$ and primitive root $a = 7$. Their public numbers are

$$y_A = 2, \quad y_B = 8.$$

Play the role of the cryptanalyst: find their private numbers and the shared key they each compute.

19. You have been happily using the Diffie-Hellman scheme, with modulus $p = 17$ and base $a = 7$, and these parameters have met your (rather limited) security needs. However your General Manager has decided that “bigger is better” and that, from now on, you and your co-workers will be using $p = 18$ (with appropriate choice of a).

- (a) What value of a would you choose?
- (b) What are the security implications of the change? Try to be reasonably precise, e.g. by roughly estimating the percentage increase/decrease in the time a cryptanalyst would have to spend on some sort of exhaustive attack.

20. Explain how the security of the Diffie-Hellman scheme will be affected if user A selects private key $x_A = p - 1$ (where p is the prime modulus used)

21. Devise a variant of the Diffie-Hellman scheme to enable three people A, B and C to arrive at a common secret key, subject to the following conditions:

- they initially have no secret information in common;
- the common key itself is not sent anywhere by anyone;
- it is hard for an eavesdropper, or anyone only knowing public information, to determine the common key;
- it is impossible for any one or two of A, B, C to find this common key without the cooperation of the other(s) (i.e., if the other tells them nothing and has no public information).

COUNTING & COMBINATORICS

Counting is a fundamental human activity, dating back tens of thousands of years. It pervades all fields of human activity, especially those that rely on numbers and measurement. It especially pervades computer science, because computation works with symbols and proceeds in discrete steps, so that the things we want to measure are very often described by whole numbers.

If we have a program together with some input for it, we would like to know how long it will take to run, or at least to be able to estimate this time, or to provide a lower bound or upper bound. We may also want to know how much of the computer's memory will be used.

Consider also the structures that computation works with. These include numbers, strings, files¹, and sequences, all of which we have met in previous chapters. Structures we will meet later include trees and graphs. Each of these structures is represented as a string of symbols constructed according to certain rules. When we are doing computation with these structures, we want to answer questions like: How large is the structure? How many structures of a given size can be properly processed by our program?

Answering all these questions requires counting.

Counting is such a natural and ubiquitous activity that we have already done it many times, in previous chapters. This chapter will review some of the techniques we have already used, describe them more formally and with greater generality, and put some of them together into a broader framework.

8.1^a COUNTING BY ADDITION

Suppose we have a list of r numbers and a list of c numbers. In total, the two lists contain $r + c$ numbers. If each number requires 8 bytes, then the two lists together require $8(r + c)$ bytes.

Suppose now we want to add up all the numbers in both lists. We can do this using two loops, one after the other (so the first loop finishes before the second one starts).

```
sum = 0
```

¹ In terms of its *contents*, a file is just a string of symbols. The term “file” refers to the way it is stored and accessed within the computer; the term itself says nothing about its internal structure.

```

for each  $i$  from 1 to  $r$ 
    sum := sum + ( $i$ -th entry in the first list)
for each  $j$  from 1 to  $c$ 
    sum := sum + ( $j$ -th entry in the second list)

```

This computation requires one addition for each i and one addition for each j . Therefore we have $r + c$ additions altogether.

In general, if you are to choose one item from two *disjoint* sets of options, then the number of choices available to you is the number of options in the first set *plus* the number of options in the second set.

We met this principle in § 1.12, when we saw that the size of a disjoint union of sets is the sum of the sizes of the sets, see (1.14).

8.2^α COUNTING BY MULTIPLICATION

Suppose you want to store a *table* of numbers in memory. If the table has r rows and c columns, then you must store $r \times c$ numbers. If each number requires, say, 8 bytes of storage, then the table needs $r \times c \times 8$ bytes, or $8rc$ bytes for short.

Suppose now that you want to add up all the numbers in the table. We again use two loops, but this time they are nested rather than separate.

```

sum = 0
for each  $i$  from 1 to  $r$ 
    for each  $j$  from 1 to  $c$ 
        sum := sum + (entry in the  $i$ -th row and  $j$ -th column in the table)

```

This computation requires one addition for each pair (i, j) , and therefore rc additions altogether.

In each case here — whether we are determining storage requirements or counting additions — we are interested in the number of pairs (i, j) where $i \in \{1, 2, \dots, r\}$ and $j \in \{1, 2, \dots, c\}$. There are r choices for the first member of a pair, and c choices for the second member of a pair. Crucially, *these choices are independent*, meaning that the particular choice of the first member of the pair *has no effect* on the number of options there are for the second member of the pair, and vice versa. In such cases, the independence of the choices means that the numbers of the separate choices are multiplied.

We have seen this principle before, in § 1.14: see (1.19). The size of a Cartesian product is just the product of the sizes of the sets being combined. In the above examples, the pairs we are counting are precisely the members of the Cartesian product

$$\{1, 2, \dots, r\} \times \{1, 2, \dots, c\},$$

where the first set has r members and the second set has c members, so the number of them is rc .

8.3 INCLUSION-EXCLUSION

In § 8.1^α, we considered the size of a *disjoint* union of sets. Determining the size of a union of sets requires more care when the sets are not disjoint.

If $A \cap B \neq \emptyset$ then $|A \cup B|$ is no longer given by $|A| + |B|$, because $|A| + |B|$ double-counts everything in the intersection $A \cap B$. So we have to subtract $|A \cap B|$ once, so that its members are counted just once instead of twice:

$$|A \cup B| = |A| + |B| - |A \cap B|. \quad (8.1)$$

See § 1.12, especially (1.13) and Exercise 9.

Example

How many entries in the Monash library catalogue (<https://www.monash.edu/library>) contain at least one of “Babbage” and “Lovelace”?

Let B be the set of entries containing “Babbage”, and let L be the set of entries containing “Lovelace”. By doing a Basic Search just for “Babbage” (no need for quotes; we only use them here to identify the exact search term used), you may find that

$$|B| = 41,504.$$

A similar Basic Search just for “Lovelace” may show that

$$|L| = 124,045.$$

If you enter both terms “Babbage” and “Lovelace” in the search field (separated by a space, and without quotes), you get the number of items containing both terms:

$$|B \cap L| = 3,263.$$

You can now find the number of items containing at least one of these terms.

$$\begin{aligned} |B \cup L| &= |B| + |L| - |B \cap L| && \text{(by (8.1))} \\ &= 41,504 + 124,045 - 3,263 \\ &= 162,286. \end{aligned}$$

In the Monash library catalogue, you can actually check this answer using an Advanced Search, which enables you to combine searches using any of AND, OR and NOT.² But there are many other search tools for which union is either unavailable or significantly harder than intersection. This makes sense, since searching for two alternative terms involves two searches of the entire database (once for each term), whereas searching for

² But hang on, NOT is a unary operation, not a binary operation! Try using the NOT operation to combine two catalogue searches, which could be using the two searches we have used here or others you are interested in, and determine which set operation they mean by NOT.

joint occurrences of two terms (i.e., where both appear in the same item) can be done by doing just one search of the entire database followed by a second search restricted to the items found in the first search (with this second search usually being much more efficient than the first search, since by then there is much less data to sift through).

Now consider the size $|A \cup B \cup C|$ the union of three sets, A , B and C . Again, just adding the sizes of these three sets, obtaining $|A| + |B| + |C|$, overcounts elements that belong to more than one set.

- An element that belongs to A and B but not C — so it's in $(A \cap B) \setminus C$ — gets counted twice: once by A and once by B .
- An element that belongs to all three sets — so it's in $A \cap B \cap C$ — gets counted three times: once by each of $|A|$, $|B|$ and $|C|$.

We can try to compensate by subtracting $|A \cap B|$, $|A \cap C|$ and $|B \cap C|$, i.e., the sizes of the *pairwise intersections*. This corrects the overcounting of the elements that belong to exactly two of the three sets. But it *over*-corrects the overcounting of the elements that belong to all three sets! This is because any element that belongs to all three of A , B and C *also* belongs to *all three* of the pairwise intersections $A \cap B$, $A \cap C$ and $B \cap C$. The net result of this is that everything is now correctly counted *except* the elements of $A \cap B \cap C$, which are *not counted at all*! So we adjust by *adding* the size of that triple intersection, $|A \cap B \cap C|$, and then everything is counted exactly once as required. The upshot of this is

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|. \quad (8.2)$$

See also [Exercise 11](#) in [Chapter 1](#).

Example

Now let's determine the number of Monash library catalogue entries containing at least one of "Babbage", "Lovelace" and "Turing". Let the sets B and L be as before, and let T be the set of entries containing "Turing". We found $|B|$, $|L|$ and $|B \cap L|$ earlier, and we will use them again shortly. Further queries with Basic Search tell us that

$$\begin{aligned} |T| &= 484,719 \\ |B \cap T| &= 2,886 \\ |L \cap T| &= 2,518 \\ |B \cap L \cap T| &= 971 \end{aligned}$$

Therefore, using (8.2), the number of entries containing at least one of these three terms is given by

$$\begin{aligned}
 |B \cup L \cup T| &= |B| + |L| + |T| - |B \cap L| - |B \cap T| - |L \cap T| + |B \cap L \cap T| \\
 &= 41,504 + 124,045 + 484,719 - 3,263 - 2,886 - 2,518 + 971 \\
 &= 650,268 - 8,667 + 971 \\
 &= 642,572.
 \end{aligned}$$

Notice, in (8.2), how the *signs alternate* according to the number of intersecting sets: we *add* the sizes of single sets, *subtract* the sizes of pairwise intersections, and *add* the sizes of the triple intersections.

This alternation persists in expressions for the sizes of the unions of arbitrary numbers of sets in terms of the sizes of all possible intersections of them. Suppose the sets are $A_1, A_2, \dots, A_n$. Then the general expression has the form

$$\begin{aligned}
 |A_1 \cup A_2 \cup \dots \cup A_n| &= \text{sum of sizes of single sets} \\
 &\quad - \text{sum of sizes of all intersections of two sets} \\
 &\quad + \text{sum of sizes of all intersections of three sets} \\
 &\quad - \text{sum of sizes of all intersections of four sets} \\
 &\quad + \dots \\
 &\quad \vdots \\
 &\quad + (-1)^k \cdot \text{sum of sizes of all intersections of } k \text{ sets} \\
 &\quad \vdots \\
 &\quad + (-1)^n \cdot \text{the size of the intersection of all } n \text{ sets.}
 \end{aligned} \tag{8.3}$$

This can be written more succinctly, and we do so now in stating it as a theorem.

Theorem 42. For all n ,

$$|A_1 \cup A_2 \cup \dots \cup A_n| = \sum_{k=1}^n (-1)^{k+1} \cdot (\text{sum of sizes of all intersections of } k \text{ sets}) \tag{8.4}$$

Proof. We prove, by induction on n , that (8.4) holds for all $n \in \mathbb{N}$.

Base case:

When $n = 1$, there is only one set, A_1 , and the left and right sides of (8.4) are both $|A_1|$. So the equation holds in this case.

Inductive step:

Let $n \geq 1$. Assume that (8.4) holds. (This is our Inductive Hypothesis.)

Now consider the size of the union of $n + 1$ sets, i.e., $|A_1 \cup A_2 \cup \dots \cup A_{n+1}|$.

We first need to relate this somehow to the size of the union just of n sets. To this end, we can start by relating the union of $n+1$ sets to the union of the first n sets:

$$A_1 \cup A_2 \cup \cdots \cup A_{n+1} = A_1 \cup A_2 \cup \cdots \cup A_n \cup A_{n+1}$$

We can view this as a union of *two* sets, namely $A_1 \cup A_2 \cup \cdots \cup A_n$ and A_{n+1} . And we already know how to find the size of the union of two sets.

$$\begin{aligned} & |A_1 \cup A_2 \cup \cdots \cup A_{n+1}| \\ &= |(A_1 \cup A_2 \cup \cdots \cup A_n) \cup A_{n+1}| \\ &= |A_1 \cup A_2 \cup \cdots \cup A_n| + |A_{n+1}| - |(A_1 \cup A_2 \cup \cdots \cup A_n) \cap A_{n+1}| \\ &\quad \text{(by (8.1))} \\ &= |A_1 \cup A_2 \cup \cdots \cup A_n| + |A_{n+1}| - |(A_1 \cap A_{n+1}) \cup (A_2 \cap A_{n+1}) \cup \cdots \cup (A_n \cap A_{n+1})| \\ &\quad \text{(by the distributive law).} \end{aligned} \tag{8.5}$$

This is progress: instead of the size of a union of $n+1$ sets, we have two occurrences of a size of a union of n sets, and we can use the Inductive Hypothesis on each of these. Note also that the first of these occurrences is added, while the second is subtracted. Continuing from (8.5), we have

$$\begin{aligned} & |A_1 \cup A_2 \cup \cdots \cup A_{n+1}| \\ &= \left(\sum_{k=1}^n (-1)^{k+1} \cdot (\text{sum of sizes of all intersections of } k \text{ of the sets } A_1, A_2, \dots, A_n) \right) \\ &\quad + |A_{n+1}| \\ &\quad - \left(\sum_{k=1}^n (-1)^{k+1} \cdot \left(\text{sum of sizes of all intersections of } k \text{ of the sets } \right. \right. \\ &\quad \quad \left. \left. A_1 \cap A_{n+1}, A_2 \cap A_{n+1}, \dots, A_n \cap A_{n+1} \right) \right) \end{aligned} \tag{8.6}$$

(by applying the Inductive Hypothesis to the first and third terms in (8.5)).

We now consider the first and third terms in (8.6). It turns out that they are closely related, indeed complementary in a sense. To see this, we consider them each in turn.

The first term in (8.6) uses, inside the summation over k , the sum of the sizes of all intersections of k of the sets $A_1, A_2, \dots, A_n$. It will be convenient to describe this as the sum of the sizes of k of the sets $A_1, A_2, \dots, A_{n+1}$ excluding A_{n+1} . This is a bit more long-winded, but it does reflect the context that we are now considering $n+1$ sets, not just n sets. So the first term in (8.6) may be written

$$\sum_{k=1}^n (-1)^{k+1} \cdot \left(\text{sum of sizes of all intersections of } k \text{ of the sets } A_1, A_2, \dots, A_n, A_{n+1} \right. \\ \left. \text{excluding } A_{n+1} \right).$$

Now consider the second term in (8.6). What can we say about the intersection of k of the sets $A_1 \cap A_{n+1}, A_2 \cap A_{n+1}, \dots, A_n \cap A_{n+1}$? The intersection of any *two* of these sets is really an intersection of *three* sets (including A_{n+1}):

$$(A_i \cap A_{n+1}) \cap (A_j \cap A_{n+1}) = A_i \cap A_j \cap A_{n+1}.$$

Similarly, an intersection of k of the sets $A_1 \cap A_{n+1}, A_2 \cap A_{n+1}, \dots, A_n \cap A_{n+1}$ is really an intersection of $k+1$ sets, namely k of the sets $A_1, A_2, \dots, A_n$ and the set A_{n+1} . So the sum of the sizes of all intersections of k of the sets $A_1 \cap A_{n+1}, A_2 \cap A_{n+1}, \dots, A_n \cap A_{n+1}$ equals the sum of the sizes of all intersections of $k+1$ of the sets $A_1, A_2, \dots, A_{n+1}$ provided that A_{n+1} is one of the sets included when taking the intersection. So the last of the three main terms in (8.6), where the sum is being subtracted, is

$$- \sum_{k=1}^n (-1)^{k+1} \cdot \left(\text{sum of sizes of all intersections of } k+1 \text{ of the sets } A_1, A_2, \dots, A_n, A_{n+1} \right. \\ \left. \underline{\text{including } A_{n+1}} \right).$$

We can take the negation inside the sum, so that the whole sum is now added (instead of subtracted) but the coefficient of the sum of sizes is $-(-1)^{k+1}$, which may be rewritten as $(-1)^{k+2}$. Then it equals

$$\sum_{k=1}^n (-1)^{k+2} \cdot \left(\text{sum of sizes of all intersections of } k+1 \text{ of the sets } A_1, A_2, \dots, A_n, A_{n+1} \right. \\ \left. \underline{\text{including } A_{n+1}} \right).$$

Here, the number of sets being intersected is $k+1$, and this is also the exponent of -1 in the coefficient. The range of values that $k+1$ can take, in this sum, is from 2 to $n+1$ (since k ranges from 1 to n). Writing our sum in terms of $k+1$ rather than k gives

$$\sum_{k+1=2}^{n+1} (-1)^{(k+1)+1} \cdot \left(\text{sum of sizes of all intersections of } k+1 \text{ of the sets } A_1, A_2, \dots, A_n, A_{n+1} \right. \\ \left. \underline{\text{including } A_{n+1}} \right).$$

This sum covers all possible numbers of sets, $k+1$, except for the case of just a single set ($k+1=1$).

To enable easier comparison with the first term in (8.6), it will help if we consistently use k for the number of sets being intersected. So we will replace $k+1$ by k throughout the last sum in the previous paragraph. Again, the sum goes from 2 to $n+1$ instead of from 1 to n . So the expression becomes

$$\sum_{k=2}^{n+1} (-1)^{k+1} \cdot \left(\text{sum of sizes of all intersections of } k \text{ of the sets } A_1, A_2, \dots, A_n, A_{n+1} \right. \\ \left. \underline{\text{including } A_{n+1}} \right).$$

We have just expressed the size of a union of sets as a sum of appropriately-signed sizes of all possible intersections that can be formed from the sets.

We can also use Inclusion-Exclusion to express the size of an *intersection* of sets in terms of a sum of signed sizes of all possible *unions*. This can be proved by induction, just as we did above, but interchanging the roles of $\cup$ and $\cap$ throughout. It is a good exercise to use the above proof as a starting point and go through it, making all the changes needed to turn it into a proof of this new type of Inclusion-Exclusion. It can also be proved using the above theorem together with De Morgan's Law for sets ([Theorem 1](#) and [Corollary 2](#)); doing this is a more challenging exercise, though it gives a shorter proof.

Theorem 43. For all n ,

$$|A_1 \cap A_2 \cap \cdots \cap A_n| = \sum_{k=1}^n (-1)^{k+1} \cdot (\text{sum of sizes of all unions of } k \text{ sets}) \quad (8.8)$$

□

8.4 INCLUSION-EXCLUSION: DERANGEMENTS

A function $f: A \rightarrow A$ is said to **fix** an element $x \in A$ if $f(x) = x$. The element x is then said to be a **fixed point** of f . Other elements of A may or may not also be fixed by f .

Now let $X \subseteq A$. The function f **fixes** X if it fixes all elements of x . So, for all $x \in X$, we have $f(x) = x$. There may or may not be other fixed points of f that are not in X .

Given a finite set A of n elements, it is often of interest to know how many functions $f: A \rightarrow A$ are **fixed-point-free**, meaning that they have no fixed points.

This is reasonably straightforward if there is no restriction on the type of function we consider, i.e., no requirement for it to be an injection or a surjection. For each $x \in A$, its image $f(x)$ under f can be anything except x , so the number of options is $|A \setminus \{x\}|$ which is $n - 1$. We have the same number of choices for each $x \in A$, and the choices do not interfere with each other. So, for each $x \in A$, we can choose any of the $n - 1$ options for $f(x)$, with these choices being independent of each other. This is another application of the multiplicative counting principle discussed in § 8.2^α. So the total number of fixed-point-free functions is

$$(n - 1)^n.$$

But what if we are only interested in counting fixed-point-free *bijections*?³ This is the problem of counting *derangements*, where a **derangement** is just a fixed-point-free bijection from a finite set to itself.

This kind of problem crops up in a wide variety of contexts.

- After a written class test, the teacher in the classroom might tell each student to give their test answers to someone else for marking while she reads out the answers, with the requirement that each student marks exactly one other student's work, and no-one marks their own work.
- In the standard Enigma cypher machine used by the Nazi regime's army during the Second World War, at each position in the message the function sending plaintext letters to cyphertext letters (in the same alphabet) was a fixed-point-free bijection on the alphabet.

In this situation, we cannot just count the options available for each $x \in A$ and then multiply them as if they are independent. The problem with that is that, if $f(x) = y$ and $w \neq x$ then we must have $f(w) \neq y$, else f is not a bijection. So the choices we make for each $x \in A$ interfere with each other. So we need another approach. This is where inclusion-exclusion will prove useful.

We start by taking a complementary view of the problem. This is a general problem-solving strategy. It won't work in every situation, but it is worth keeping in mind.

We have

$$\begin{aligned} \# \text{ bijections with no fixed point} &= \text{total } \# \text{ bijections} - \\ &\quad \# \text{ bijections with at least one fixed point.} \end{aligned}$$

We already know the *total* number of bijections from A to A : this is just $n!$, as we saw in § 2.12. Therefore

$$\# \text{ bijections with no fixed point} = n! - \# \text{ bijections with at least one fixed point.} \quad (8.9)$$

So, to count fixed-point-free bijections, we first count bijections with at least one fixed point.

For convenience, denote the elements of A by $a_1, a_2, \dots, a_n$, so that

$$A = \{a_1, a_2, \dots, a_n\}.$$

For each $i \in \{1, 2, \dots, n\}$, let A_i be the set of all bijections on A that fix a_i . So the set of all bijections that fix at least one element of A is

$$A_1 \cup A_2 \cup \dots \cup A_n.$$

³ Since A is finite, any injection $f: A \rightarrow A$ is also a surjection, and *vice versa*. (See the end of § 2.7, on p. 50.) So, if we count fixed-point-free injections (or surjections) from a finite set to itself, then we will really be counting fixed-point-free bijections anyway.

So, to count these bijections, we need to determine

$$|A_1 \cup A_2 \cup \cdots A_n|.$$

This is a job for the Inclusion-Exclusion principle, in the form of [Theorem 42](#). To apply that theorem, we will need to work out, for each k , the sum of the sizes of all intersections of k of the sets $A_1, A_2, \dots, A_n$.

Consider, then, what an intersection of k of the sets $A_1, A_2, \dots, A_n$ looks like.

Let's start with $k = 1$. Then we have just a single set, say A_i . This is the set of all bijections on A that fix a_i . How many such bijections are there? If f is one of these bijections, then its value on a_i is determined by the fact that it fixes a_i , so for $f(a_i)$ we have no choice: it must equal a_i . Then none of the other elements of A can be mapped to a_i , because then f would not be a bijection. So f maps the elements of $A \setminus \{a_i\}$ to $A \setminus \{a_i\}$, and in fact it must be a bijection on that set, otherwise it can't be a bijection on A . So the number of bijections on A that fix a_i is just the number of bijections on $A \setminus \{a_i\}$, and there are $(n-1)!$ of these, since $|A \setminus \{a_i\}| = n-1$. Hence

$$|A_i| = (n-1)! \quad (8.10)$$

There are n of these sets, and all have the same size, so the sum of the sizes of all the sets A_i is given by

$$|A_1| + |A_2| + \cdots + |A_n| = n \cdot (n-1)! = n!.$$

Now consider what happens in general, for arbitrary k . Consider the intersection of the first k sets,

$$A_1 \cap A_2 \cap \cdots A_k.$$

This contains those bijections on A that fix a_1 and also fix a_2 and also a_3 and so on up to a_k . So, we want to count bijections $f: A \rightarrow A$ that fix every a_i with $1 \leq i \leq k$. The values of such a bijection f on $a_1, a_2, \dots, a_k$ is completely determined by the requirement that it fixes those elements. So it remains to consider what f does on the other elements of A , namely $a_{k+1}, a_{k+2}, \dots, a_n$. Now f cannot map any of these elements to the fixed points $a_1, a_2, \dots, a_k$, else it would not be a bijection, because each of those elements is already mapped to by itself. So f has to map the set $\{a_{k+1}, a_{k+2}, \dots, a_n\}$ into itself. Furthermore, it has to map this set *onto* itself too, else it isn't a surjection. So, in fact, the restriction of f to $\{a_{k+1}, a_{k+2}, \dots, a_n\}$ must be a bijection on that set, and it can be any bijection on that set at all. So, counting bijections that fix $a_1, a_2, \dots, a_k$ is the same as just counting bijections on $\{a_{k+1}, a_{k+2}, \dots, a_n\}$, which is a set of size $n-k$, so there are $(n-k)!$ bijections on this set. So we have

$$|A_1 \cap A_2 \cap \cdots A_k| = (n-k)!.$$

This is just one of the many possible intersections of k of these sets. Since no element of A has any special role, and since their names do not matter, the size of the intersection of k of them is always the same. We could pick any k distinct elements of A , say

$$a_{i_1}, a_{i_2}, \dots, a_{i_k},$$

where $i_1, i_2, \dots, i_k$ are any k distinct elements of A . Regardless of our choice, we still have

$$|A_{i_1} \cap A_{i_2} \cap \dots \cap A_{i_k}| = (n - k)!.$$

Note that the special case $k = 1$ agrees with the expression for that case we derived above, (8.10).

The number of ways of choosing k of the n sets $A_1, A_2, \dots, A_n$ is $\binom{n}{k}$. So

$$\text{sum of the sizes of all intersections of } k \text{ of the sets} = \binom{n}{k} (n - k)!.$$

We can now apply [Theorem 42](#) to obtain

$$|A_1 \cup A_2 \cup \dots \cup A_n| = \sum_{k=1}^n (-1)^{k+1} \binom{n}{k} (n - k)! \quad (8.11)$$

Since the left-hand side here is the number of bijections on A that fix at least one member of A , using (8.9) gives

$$\# \text{ fixed-point-free bijections} = n! - \sum_{k=1}^n (-1)^{k+1} \binom{n}{k} (n - k)!.$$

There are some instructive simplifications we can do here.

First, observe that the main sum starts at $k = 1$, so it does not include $k = 0$, but if we put $k = 0$ in its summand then (including the negation in front of the sum) it would be $-(-1)^{0+1} \binom{n}{0} (n - 0)!$, which is just $n!$. But this equals the solitary extra term $n!$ at the start. So, if we start the sum at $k = 0$ instead of $k = 1$, then we don't need that extra term $n!$ because it's then part of the sum. So

$$\# \text{ fixed-point-free bijections} = \sum_{k=0}^n (-1)^k \binom{n}{k} (n - k)!.$$

Next, observe that

$$\begin{aligned} \binom{n}{k} (n - k)! &= \frac{n!}{k! (n - k)!} \cdot (n - k)! && \text{(see § 1.10, especially (1.6))} \\ &= \frac{n!}{k!}. \end{aligned}$$

So

$$\# \text{ fixed-point-free bijections} = \sum_{k=0}^n (-1)^k \cdot \frac{n!}{k!}.$$

We have been focusing on the *number* of fixed-point-free bijections. But, often, we don't just want to know the *number* of structures of interest; we may also want to know what *proportion* they are, out of *all possible* structures. In this case, we may ask, what proportion of all bijections on A are fixed-point-free? Since there are $n!$ bijections on A altogether, the proportion that are fixed-point-free is

$$\begin{aligned} \frac{\# \text{ fixed-point-free bijections}}{\text{total } \# \text{ bijections}} &= \frac{1}{n!} \sum_{k=0}^n (-1)^k \cdot \frac{n!}{k!} \\ &= \sum_{k=0}^n (-1)^k \cdot \frac{1}{k!}. \end{aligned}$$

It is interesting that n no longer appears inside the sum; its only role is to determine how many terms ($n+1$, in fact) must be added up. Note also that these terms rapidly get smaller and smaller, and furthermore they alternate in sign. You can see the structure by writing the sum out:

$$1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \frac{1}{4!} - \cdots + (-1)^n \frac{1}{n!}.$$

This is actually the first $n+1$ terms of the standard infinite series for e^{-1} , where as usual $e = 2.71828\dots$ is the base of natural logarithms. So, as $n \rightarrow \infty$, the proportion of bijections on a set of size n that are fixed-point-free converges to

$$e^{-1} = 0.367989\dots$$

In other words, if you choose a bijection at random from all bijections on n elements, with all bijections equally likely, then the chance that it has no fixed points converges to about 36.8% as $n \rightarrow \infty$. The convergence is rapid, so this proportion gives a very useful approximation even for moderate-sized n .

At this point, it is worth revisiting Exercises 2.9 and 2.10.

8.5 SELECTION

There are many situations where we want to choose r objects from a set of n objects. For example:

- (a) making a string of r letters, each chosen from an alphabet of n letters;
- (b) forming a queue of r people, from a crowd of n people;
- (c) requesting r meals for an event, from a catering menu containing n meal options;

(d) choosing r party guests, from among n of your friends.

Counting the number of ways of making these selections depends on the specific nature of the task.

- Does the order in which items are chosen matter?
 - It does, for (a) and (b) above; but it doesn't, for (c) and (d).
- Once an item is chosen, can we choose it again?
 - We can, in (a) and (c) above; but we cannot, in (b) and (d).

So we have two distinctions to make:

- **ordered** selection (as in (a) and (b)) versus **unordered** selection (as in (c) and (d));
- selection **with replacement** (as in (a) and (c)) versus selection **without replacement** (as in (b) and (d)).

We consider each of these.

8.6 ORDERED SELECTION WITH REPLACEMENT

Given a set A of size n , suppose we make a *sequence* of r choices from A , where each choice can be *any* member of A . So:

- Our selection is *ordered*.
- Choosing a member of A does not stop it being chosen again later. In other words, if a choice takes an element from A , we can think of that element as being *replaced*, in A , by a copy of itself, so that the element is still available for future choices. This is why we say that our selection is done *with replacement*.

In how many ways can we make such a sequence of choices?

We answered this question back in [Chapter 1](#).

The number of sequences of n elements of A is just the size of the Cartesian product

$$\underbrace{A \times A \times \cdots \times A}_{n \text{ copies of } A}.$$

This size is just n^r , as we saw on p. 26 in § 1.14.

When A is an alphabet of symbols, then we are essentially just counting strings of a given length over A . We did this on p. 7 in § 1.5.

We have seen other examples of this too. In § 2.12, we applied this technique to counting functions with a finite domain and codomain.

8.7 ORDERED SELECTION WITHOUT REPLACEMENT

Suppose that we are again making a sequence of r choices from A , but that now we cannot repeat earlier choices. So:

- Our selection is *ordered*, as in § 8.6,
- Choosing an element of A means that it is no longer available for subsequent choices. In other words, if an element is chosen, it is not *replaced*. So we say that our selection is done *without replacement*.

In how many ways can we do this?

We have done this before too, on p. 13, leading to (1.3) and (1.4). Let's recap.

For our first choice, we can choose any member of A at all. So we have n options for this. For our second choice, the first choice is no longer available to us, but every other member of A is still available. The number of options has gone down by 1, so it is $n - 1$ instead of n . So the number of ways of making the first two choices is $n \times (n - 1)$. For the third choice, we can choose any member of A except for the two we have already chosen. So we have $n - 2$ options.

The general pattern is:

- For each choice we make, all previous choices are now forbidden, but every other member of A is still available.
- When we come to the k -th choice, we have previously made $k - 1$ choices, all of which are now forbidden.
- For every choice we make, the number of available choices remaining is reduced by 1.

Therefore, the number of ways of choosing r objects, in order and without replacement, from a set of n objects is

$$\underbrace{n \cdot (n - 1) \cdot \cdots \cdot (n - r + 1)}_{r \text{ factors}}.$$

This is often called the **falling factorial** and denoted by $(n)_r$. It is also sometimes written ${}^n P_r$ or occasionally ${}_n P_r$.

In § 2.12, we applied this counting method to counting injections and bijections, where domain and codomain are finite.

8.8 UNORDERED SELECTION WITHOUT REPLACEMENT

When doing ordered selection in § 8.6–§ 8.7, it was convenient to treat selection *with* replacement first, because it was easier and because it “paved the way” for selection *without* replacement. But, for unordered selection, it turns out that selection *without* replacement is easier, so we discuss it first, in this section (albeit briefly, because we

have done it before). Then, in the next section, we will use the ideas from this section (and § 1.10) to help us count unordered selections *with* replacement.

In fact, we already know how to count unordered selections without replacement, because we did it in § 1.10. We derived expressions involving factorials and binomial coefficients in (1.5) and (1.6). The number of unordered selections — i.e., *subsets* — of r elements that can be chosen, *without* replacement, from a set of n elements is given by the binomial coefficient $\binom{n}{r}$ which can be written in a few different ways:

$$\binom{n}{r} = \frac{n!}{(n-r)!r!} = \frac{(n)_r}{r!}.$$

Alternative notation for the binomial coefficient includes nC_r or occasionally ${}_nC_r$.

Subsets chosen, from a set of size n , by unordered selection without replacement are sometimes called **combinations**, but this term is also used more broadly too, for other types of selections and also in less precise ways.

8.9 UNORDERED SELECTION WITH REPLACEMENT

When choosing without replacement (as in the previous section), each member of our set A of size n can be chosen at most once. For convenience, suppose the objects in A are numbered from 1 to n . Let x_i be the number of times the i -th object in A is chosen. Then each $x_i \in \{0, 1\}$, and we require that the total number of objects chosen is r :

$$x_1 + x_2 + \cdots + x_n = r.$$

We saw in the previous section that the total number of choices is $\binom{n}{r}$.

Now we consider choosing *with* replacement. There is no longer any limit on how many times we can choose a particular object in A , except for the overall requirement that we make exactly r choices. So each $x_i \geq 0$. The x_i can be any nonnegative integer subject to the same constraint we had earlier,

$$x_1 + x_2 + \cdots + x_n = r. \quad (8.12)$$

One way to view this is that we have r 1s added together,

$$\overbrace{1 + \cdots + 1}^{r \text{ ones}} = r,$$

and we must group the ones into sums of sizes $x_1, x_2, \dots, x_n$:

$$\overbrace{\underbrace{1 + \cdots + 1}_{x_1} \underbrace{1 + \cdots + 1}_{x_2} \cdots \underbrace{1 + \cdots + 1}_{x_n}}^{r \text{ ones altogether}}.$$

We can specify the groups by placing barriers (shown as vertical lines) between them:

$$\overbrace{1 + \cdots + 1}^{r \text{ ones altogether}} \bigg| \overbrace{1 + \cdots + 1}^{x_2} \bigg| \cdots \bigg| \overbrace{1 + \cdots + 1}^{x_n} .$$

$x_1 \qquad x_2 \qquad x_n$

The understanding here is that x_1 is the sum of the 1s before the first barrier, x_2 is the sum of the 1s between the first and second barriers, and so on, with x_i being the sum of the 1s between the $(i-1)$ -th and i -th barriers ($1 \leq i \leq n-1$) and x_n being the sum of the 1s after the $(n-1)$ -th barrier. Note that we have no 0-th barrier and no n -th barrier; they are not needed.⁴ So the number of barriers is $n-1$, i.e., one less than the number of groups.

For example, if $n=5$ and $r=3$, then we have three ones,

$$\overbrace{1 + 1 + 1}^{3 \text{ ones}} = 3,$$

to be divided into five groups of sizes x_1, x_2, x_3, x_4, x_5 . So each $x_i \geq 0$ and $x_1 + x_2 + x_3 + x_4 + x_5 = 3$. One possibility is $x_1 = 2, x_2 = 0, x_3 = 0, x_4 = 1, x_5 = 0$. We can specify this using four barriers:

$$1 + 1 \bigg| \bigg| \bigg| 1 \bigg|$$

In principle, we could count unordered selections with replacement by counting the number of ways of distributing $n-1$ barriers in between the r ones, noting that barriers are allowed to be before the first 1 (which corresponds to $x_1 = 0$) or after the last one (for $x_n = 0$), and we are allowed to have more than one barrier in between ones (which happens when some $x_i = 0$). But it is not yet clear that this different viewpoint gives us a way forward. An issue we have to deal with is the possibility that multiple barriers may be placed between some pairs of consecutive 1s.

We can deal with this by the following trick. Instead of wanting n integers $x_1, \dots, x_n$ which are all ≥ 0 and add up to r , let us instead add 1 to all these integers and require them to add up to $r+n$. For each i , define $y_i = x_i + 1$. We have

$$\begin{aligned} x_1 + x_2 + \cdots + x_n = r &\iff (x_1 + 1) + (x_2 + 1) + \cdots + (x_n + 1) = r + n \\ &\iff y_1 + y_2 + \cdots + y_n = r + n. \end{aligned}$$

So we still have a simple equation specifying the required value of a sum of n integers, but now the integers are all required to be *positive* (instead of merely nonnegative) and their sum is now $r+n$ (instead of just r). Although the equation looks different, it is really encoding the same information. There is a bijection between sequences $(x_1, x_2, \dots, x_n)$ of

⁴ If we used them, they would be before the start and after the end, respectively. There would be no flexibility in where they are placed.

nonnegative integers satisfying (8.12) and sequences $(y_1, y_2, \dots, y_n)$ of *positive* integers satisfying

$$y_1 + y_2 + \dots + y_n = r + n. \quad (8.13)$$

Keep in mind the interpretation of the numbers x_i , as the number of times the i -th object is chosen. We will interpret the numbers y_i similarly, but they have the constraint that the i -th object is chosen *at least once*.

So, the number of unordered selections, with replacement, of r objects from a set of n objects, is the same as the number of unordered selections, with replacement, of $r + n$ objects from a set of n objects *with the extra constraint* that each object is now chosen at least once.

We again depict our choice using barriers.

$$\overbrace{1 + \dots + 1 + \underbrace{1}_{y_1} \mid 1 + \dots + 1 + \underbrace{1}_{y_2} \mid \dots \mid 1 + \dots + 1 + \underbrace{1}_{y_n}}^{r+n \text{ ones altogether}}.$$

For our earlier example ($n = 5$, $r = 3$, $(x_1, x_2, x_3, x_4, x_5) = (2, 0, 0, 1, 0)$), this transformation now gives

$$1 + 1 + \underbrace{1}_{\text{blue}} \mid \underbrace{1}_{\text{blue}} \mid \underbrace{1}_{\text{blue}} \mid \underbrace{1}_{\text{blue}} \mid \underbrace{1}_{\text{blue}},$$

with $r + n = 3 + 5 = 8$ ones altogether, and the five new ones shown in *blue*.

A key observation here — which is actually the point of this trick — is that, between any two adjacent ones in our line of $r + n$ ones, *there is at most one barrier*. Furthermore, there is no barrier before the first one and after the last one. These observations follow from the requirement that each $y_i \geq 1$.

Choosing positions for the barriers (which amounts to choosing the values for $y_1, y_2, \dots, y_n$) must be done to satisfy the following conditions.

- We have $r + n - 1$ positions to choose from.
- Each of these positions can be used at most once. So, if a position is chosen, it cannot be chosen again. In other words, we are now choosing *without replacement*!
- We must choose exactly $n - 1$ of these positions.

So we have transformed a problem of unordered selection *with* replacement to one of unordered selection *without* replacement. This is a classic computer science problem-solving method: transforming a new problem to one we already know how to solve, in this case by constructing a suitable bijection.

So, all we have to do now is find the number of ways of choosing $n - 1$ objects (in this case, barrier positions) from a set of $r + n - 1$ available objects (in this case, all possible positions between consecutive ones). By § 8.8, this is just

$$\binom{r + n - 1}{n - 1}.$$

By the symmetry of binomial coefficients (see (1.2)), this is the same as

$$\binom{r+n-1}{r}.$$

8.10 EXERCISES

1. Consider the following algorithm.

```

for i from 1 to a do
  for j from 1 to b do
    for k from 1 to c do
      for l from 1 to d do
        beep!
      for m from 1 to e do
        for n from 1 to f do
          beep!
        for o from 1 to g do
          beep!
        for p from 1 to h do
          beep!
    
```

- (a) Give an expression in the positive integer variables a, b, c, d, e, f, g, h for the number of times this algorithm beeps.

- (b) If each of these eight variables is $O(n)$, give a big-O expression for the number of times the algorithm beeps.

2.

- (a) In an Australian Federal Election, a ballot paper for a House of Representatives seat has n boxes, one for each candidate. A voter must enter the numbers 1 to n , in the order of their preference, in those boxes, with exactly one number in each box. In how many ways can this be done?

- (b) In the Senate, the ballot paper again has one box for each of N candidates, but this time, voters are only required to enter numbers 1 to 12, for their twelve most preferred candidates. In how many ways can this be done?

3. Two fair dice are thrown, and each shows a number from $\{1, 2, 3, 4, 5, 6\}$. The **outcome** is the ordered pair of numbers shown.

- (a) How many possible outcomes are there?

- (b) How many outcomes are there in which the number on the first die is less than the number on the second die?
- (c) How many outcomes are there in which the sum of the two numbers is 7?
- (d) How many outcomes are there in which both numbers are ≤ 3 ?
- (e) How many outcomes are there in which neither number is 5?

4. The Melbourne city centre grid is defined by fourteen main streets:

- five “horizontal” streets: Flinders St., Collins St., Bourke St., Lonsdale St., and La Trobe St;
- nine “vertical” streets: Spencer St., King St., William St., Queen St., Elizabeth St., Swanston St., Russell St., Exhibition St., and Spring St.

These divide the city centre into $4 \times 8 = 32$ square blocks.

Suppose you are at the corner of Flinders and Swanston Streets, having just emerged from Flinders Street Station. You can’t make up your mind whether to visit Federation Square, St Paul’s Cathedral or Young & Jackson’s Pub. So you decide to go for a walk, always staying on one of these main streets, never turning back, and always going further away from your starting point.

- (a) How many routes are there, following these rules, from your starting point to the corner of William and La Trobe Streets?⁵

Express your answer using one of the expressions we have used for counting, then work out its numerical value.

- (b) How many routes of this kind are there from your starting point which do not meet Russell Street and which take you four block-lengths away? Give the total as well as the number of routes to each intersection that lies at this distance from the start. Comment on the relationship among these numbers and how they relate to some counting you did earlier in semester.

- (c) How many ways are there of walking from the south-west corner (Flinders and Spencer Streets) to the north-east corner (Spring and La Trobe Streets) using a shortest possible route?

5. Using only Basic Searches (with no quotation marks or special commands) and the inclusion-exclusion principle, determine how many Monash Library catalogue entries contain at least one of the three terms CSIRAC, SILLIAC and WREDAC.

⁵ While you are there, you can see Russell’s Old Corner Shop (now closed), which is described as Melbourne’s oldest residential building, and then take a short walk in Flagstaff Gardens to see the site of Flagstaff Observatory (1858).

- 6.
- (a) Use the inclusion-exclusion principle to determine how many positive integers ≤ 100 are not multiples of 3, 5 or 7.
- (b) Is there another simple way of working this out, that you can use to check your answer?
7. Use the inclusion-exclusion principle to determine the number of surjections from $\{a, b, c, d, e, f\}$ to $\{1, 2, 3, 4\}$.
8. Determine the numbers of n -digit positive integers that have their digits
- (a) in ascending order;
- (b) in descending order;
- (c) in nondecreasing order;
- (d) in nonincreasing order.
9. The card game poker uses a standard deck of 52 cards, divided into four **suits** $\spadesuit, \clubsuit, \diamondsuit, \heartsuit$ of 13 cards each, with the cards within each suit having a **rank**

2, 3, 4, 5, 6, 7, 8, 9, 10, J, K, Q, A,

where J, Q, K, A stand for Jack, Queen, King, Ace, respectively. These designations may all be regarded as numbers, so J, Q, K, A represent 11, 12, 13, 14 respectively.

A unique property of the Ace is that it can also stand for 1. So it can be considered to be the predecessor of 2 as well as the successor of K. But it can't play these low and high roles simultaneously, in the same hand!

A **poker hand** consists of a set of five cards from the deck. In an actual game, there are multiple players, each with a hand dealt from the same deck. But in this exercise we consider just a single hand, in isolation.

Five cards in a poker hand are **consecutive** if their ranks are in numerical sequence. Here, the Ace can either be the first of five consecutive lowest cards, A, 2, 3, 4, 5, or the last of the five consecutive highest cards, 10, J, Q, K, A. But there is no wrap-around,⁶ so, for example, the five cards Q, K, A, 2, 3 are not consecutive. This is because, as mentioned above, the Ace cannot play its low and high roles simultaneously.

How many poker hands are there of each of the following types:

- (a) no restriction at all;
- (b) a **straight flush**: five consecutive cards in the same suit;

⁶ or as some card-players say, you can't go "round the corner".

- (c) **four of a kind**: four of the five cards have the same rank (so they must all be in different suits);
- (d) **full house**: three cards of one rank, two of another;
- (e) **flush**: all five cards have the same suit, but their ranks are not *all* consecutive (although some of them might be);
- (f) **straight**: five consecutive cards, not all in the same suit (although some of them will be);
- (g) **three of a kind**: three cards of the same rank, with the other two cards being of different ranks to those three and to each other;
- (h) **pair of pairs**: two separate pairs of cards, with the cards in each pair having the same rank but the two pairs having different ranks, and a fifth card that is of a different rank to the others;
- (i) **one pair**: a single pair of cards of the same rank, with the other three cards having different ranks to the pair and to each other;
- (j) **nothing**: a hand of none of the special types (b)–(i) listed above, so its five cards are all of different ranks, their ranks are not all consecutive, and the cards are not all of the same suit.

10. How many strings of five digits are there in which

- (a) there is no restriction whatsoever;
- (b) leading 0s are not allowed;
- (c) all digits are identical;
- (d) all digits are different;
- (e) every digit is different to its predecessor, but other repetitions are allowed;
- (f) all digits are in strictly ascending order (so each digit is numerically $<$ its successor);
- (g) all digits are in nondecreasing order (so each digit is numerically $\leq$ its successor);
- (h) the ordering of digits is *monotonic*, meaning that it's either increasing or decreasing (although not necessarily strictly so); in other words, no internal digit is greater than both its neighbours, and no internal digit is less than both its neighbours;
- (i) the digits are in arithmetic progression;
- (j) the digits are in geometric progression;

- (k) the digits add up to a multiple of 9;
- (l) no digit appears in its own position (i.e., the first digit cannot be 1, the second digit cannot be 2, and so on);
- (m) in addition to the previous restriction (l), all the digits are different.

11. Specify formally the bijection given in § 8.9 ...

- from the set of x -sequences of nonnegative integers satisfying the constraints given there (including (8.12))
- to the set of y -sequences of positive integers satisfying the constraints given there (including (8.13)).

DISCRETE PROBABILITY I

Uncertainty is a fact of life. We face it whenever we look beyond what we know. It surrounds our attempts to understand the past and anticipate the future. It is inherent in some natural processes. Complex systems — in machines, organisations or societies — seem to generate it.

So, when we construct models based on the real world, we need to be able to model uncertainty and to reason about it. This provides compelling motivation for the formal study of uncertainty.

But there are other reasons, relating to computation itself.

When you first learn to program, computers are presented as *deterministic*, meaning that everything about them at any given time is completely determined by knowing the entire state of the computer (i.e., every piece of information in every input device and every piece of memory) at the immediately previous instant of time. When you write a program, each instruction specifies precisely how the information stored in the computer is to change. But it turns out that access to *random* information — produced by some unpredictable chance process — can be a powerful computational tool, and can help some algorithms do some tasks more efficiently than seems possible otherwise. So the study of uncertainty and randomness is important in algorithm *design*, as well as for modelling uncertainty in the real world.

Another reason for studying uncertainty relates to algorithm *analysis*. Once we have designed an algorithm, we want to know how it behaves on the range of inputs it might have to deal with. For example, we have already mentioned upper and lower bounds on the time an algorithm takes (p. 281, at the start of [Chapter 8](#)). An upper bound gives a guarantee that your algorithm always runs at least that quickly, which is important when making positive statements about what your algorithm can do. A lower bound prevents you from being unreasonably optimistic about the time it needs, and helps you direct your efforts away from fruitless quests for unattainable improvement. But lower and upper bounds are, by definition, extremes; they don't tell you what sort of running time is *typical*, or how long it may take *on average*. If you want to analyse the time your algorithm takes across a range of possible inputs, then you need to know how likely the various inputs are, so that you can measure (or estimate, or bound) the contribution the various inputs make to an average running time.

In discussing the importance of studying uncertainty and randomness for algorithm design and analysis, we have focused on the *time* algorithms take. But the same issues arise when studying other aspects of an algorithm's behaviour, such as the amount of memory it uses, or the quality of the solution it produces. And similar points apply to designing and analysing the data structures that algorithms work with, as well as the algorithms themselves.

The theory of probability provides the foundation for measuring uncertainty and reasoning about it. In this chapter, we introduce probability theory and learn how to use it. In the next chapter, we study random variables and probability distributions.

9.1^α THE NATURE OF RANDOMNESS

Suppose someone tosses a coin. You do not know, in advance, whether it will turn up Heads or Tails, i.e., which face will be uppermost once it has landed.¹ If the coin is *fair* — meaning that it has no bias towards either outcome, so each is equally likely — then the *probability* that it comes up Heads is $\frac{1}{2}$, and the probability that it comes up Tails is $\frac{1}{2}$ too. We can say that the *event* that the coin comes up Heads has probability $\frac{1}{2}$, which we write mathematically as

$$\Pr(\text{coin comes up Heads}) = \frac{1}{2},$$

and similarly,

$$\Pr(\text{coin comes up Tails}) = \frac{1}{2}.$$

These events take place in a setting where randomness is at play, which is why we describe them using probabilities rather than just by propositions which can only be true or false.

The nature and source of randomness is a surprisingly deep topic and a lot has been written about it. Tossing a coin is often used as an introductory example of a simple random experiment, but where does the randomness come from? During the tossing process, it is subject to physical forces from the tossing hand, the movement of the air through which it passes, and gravity. If we do sufficiently careful and precise measurements of the coin, its environment and the forces acting upon it, it may be possible in principle to determine, accurately enough, its movement during the period of the toss until it comes to rest, and therefore to determine the outcome of the toss. Of course, this is usually impractical. So, instead of treating it as a deterministic process (which it seems to be, at least to the level of detail required to determine the outcome

¹ Traditionally, Heads is the head of the monarch whose face would historically adorn one side of a coin; Tails, being the opposite of Heads, indicates the other side of the coin. Not many coins would have actual “tails” depicted on the other side. One exception is the Australian penny from 1938 to the introduction of decimal currency in 1966, which had a kangaroo, including its long tail, on the other side. So Tails could be interpreted more literally in those days.

of the toss with high confidence), we say that it is too complex to treat deterministically and model it instead as a simple random process.

This illustrates one common source of randomness: processes that are actually deterministic, but for which deterministic models are infeasible. You might say that randomness is a cloak for our computational shortcomings!

That is not to say that all physical processes that we treat as random are really just overly complex deterministic processes. In some physical processes, we have no deterministic concept (however complex) to explain them, so we treat them as random. Again, randomness might be a cloak, but for our ignorance rather than our computational shortcomings. Or, for some physical processes, we might think that they really are random in some deep fundamental sense. Quantum mechanics is often interpreted as treating the measurement of physical systems as inherently random.

In other settings, the role of randomness in expressing our ignorance is very natural. For example, a coin might be lying on the ground, some distance away, and you cannot see whether it shows Heads or Tails. There is no random experiment here; either the coin shows Heads, or it shows Tails, and someone close to it may know which one it is, but you don't know yet. So you may model your knowledge of the state of the coin by saying that its probability of showing Heads is $\frac{1}{2}$.

We will focus on randomness as an aspect of processes that produce outcomes we are interested in. For such processes, we will often describe the outcomes they produce as random, too.

9.2^α PROBABILITY

Informally, the *probability* of something is a real number in the unit interval $[0,1]$ that measures how likely it is, where something that's impossible has probability 0 and something that's certain has probability 1.

Probability is usually considered in the context of an *experiment*. Here, an **experiment** is a process that is at least partly random and can give rise to any outcome from some set of all possible outcomes. The set of all possible outcomes is called the **sample space**. We can view this as just a *universal set*.

An **event** is just a subset of the sample space.

The simplest situation in which we can define probability is when the sample space U is finite and all its elements are equally likely. In this scenario, the probability $\Pr(A)$ of an event $A \subseteq U$ is given by

$$\Pr(A) = \frac{|A|}{|U|}. \quad (9.1)$$

So the probability of A is just the proportion of members of U that belong to A . If you choose a member of U at random, with an equal chance of choosing each one, then $\Pr(A)$ measures how likely your chosen element is to belong to A .

How do you work this out? You just need to work out the sizes of the sets. To do this, you can use all the techniques of counting that we have discussed so far, and many others. The study of probability is intimately related to the study of counting.

We have two extreme special cases of the definition (9.1):

$$\Pr(\emptyset) = 0, \quad \Pr(U) = 1.$$

To get a handle on the probability of an event, it is natural to try an experimental approach. If you repeatedly choose members of U , with each choice having no influence at all on subsequent choices, then you can count the number of times your chosen element is in A . Intuitively, you would anticipate that

$$\Pr(A) \approx \frac{\text{number of choices that belong to } A}{\text{total number of choices}}. \quad (9.2)$$

This seems plausible, but it is not guaranteed that the approximation is close. So, ratios of empirical counts like (9.2) do not *define* probability; they merely *estimate* it, and these estimates may or may not be good ones, depending on the number of choices you make and how chance plays out as you make your choices. As the number of choices increases, you would intuitively expect the approximation in (9.2) to get better and better. This belief can be made precise and then proved. In fact,

$$\Pr(A) = \lim_{\text{total \# choices} \rightarrow \infty} \left(\frac{\# \text{ choices that belong to } A}{\text{total \# choices}} \right). \quad (9.3)$$

We will discuss the estimation of probabilities later, but now return to defining and exactly calculating them.

We illustrate the definitions with some familiar examples.

Suppose our random experiment is the toss of a fair coin. The two possible outcomes are Heads and Tails, so our sample space is

$$U = \{\text{Heads}, \text{Tails}\}.$$

The possible events here are the various subsets of U :

$$\emptyset, \quad \{\text{Heads}\}, \quad \{\text{Tails}\}, \quad \{\text{Heads}, \text{Tails}\}.$$

Their probabilities are:

$$\begin{aligned}\Pr(\emptyset) &= 0, \\ \Pr(\{\text{Heads}\}) &= \frac{|\{\text{Heads}\}|}{|U|} = \frac{1}{2}, \\ \Pr(\{\text{Tails}\}) &= \frac{1}{2} \quad (\text{similarly}), \\ \Pr(\{\text{Heads, Tails}\}) &= \frac{2}{2} = 1.\end{aligned}$$

The throw of a fair die gives six equally likely outcomes, with sample space

$$U = \{1, 2, 3, 4, 5, 6\}.$$

We can calculate that, for example,

$$\begin{aligned}\Pr(\{5\}) &= \frac{1}{6}, \\ \Pr(\text{outcome is odd}) &= \Pr(\{1, 3, 5\}) = \frac{3}{6} = \frac{1}{2}, \\ \Pr(\text{outcome is a square}) &= \Pr(\{1, 4\}) = \frac{2}{6} = \frac{1}{3}, \\ \Pr(\text{outcome is not 1}) &= \Pr(\{2, 3, 4, 5, 6\}) = \frac{5}{6}.\end{aligned}$$

So far, we have considered probability in a context where the sample space is finite and every element of it is equally likely. Using our definition (9.1), an event consisting of a single element $x \in U$ has probability

$$\Pr(\{x\}) = \frac{|\{x\}|}{|U|} = \frac{1}{|U|}.$$

So we can assign to each *element* x a probability of $1/|U|$,

$$\Pr(x) = \frac{1}{|U|}.$$

and calculate the probability of any *event* A as the sum of the probabilities of its elements:

$$\Pr(A) = \sum_{x \in A} \Pr(x). \quad (9.4)$$

In our current scenario, where every element is equally likely and has probability $1/|U|$, this equation (9.4) agrees with our earlier definition (9.1). But our new equation (9.4) is more general. We can now deal with situations where the elements of the sample space need not all have the same probability.

We suppose, then, that the elements of the sample space each have a probability, which must be a number in the unit interval $[0,1]$, and that these probabilities add to 1. The probability of an element $x \in U$ is denoted by $\Pr(x)$. We require that, for all $x \in U$,

$$0 \leq \Pr(x) \leq 1,$$

and our constraint on the sum of these probabilities may be written

$$\sum_{x \in U} \Pr(x) = 1. \quad (9.5)$$

We emphasise that this sum is over *every element* in the entire sample space, however large or small that sample space may be.

We now *define* the probability $\Pr(A)$ of any event $A \subseteq U$ to be

$$\Pr(A) = \sum_{x \in A} \Pr(x). \quad (9.6)$$

This is more general than our previous definition of probability, which only applies to finite sample spaces with all elements having the same probability. We have, again, the extreme cases

$$\Pr(\emptyset) = 0, \quad \Pr(U) = 1.$$

For example, in the board game Scrabble², there are 100 tiles each with an English letter, except that two tiles are blank, and these tiles are all in a bag so that players can choose them at random. Suppose, at the start of the game, you choose a letter by drawing a random tile from the bag. The numbers of tiles of each type gives the following probabilities for the letters (or blank, denoted $\square$).

letter	prob.	letter	prob.	letter	prob.
A	0.09	J	0.01	S	0.04
B	0.02	K	0.01	T	0.06
C	0.02	L	0.04	U	0.04
D	0.04	M	0.02	V	0.02
E	0.12	N	0.06	W	0.02
F	0.02	O	0.08	X	0.01
G	0.03	P	0.02	Y	0.02
H	0.02	Q	0.01	Z	0.01
I	0.09	R	0.06	$\square$	0.02

² Scrabble is one of the most popular word games in the world, and is good for improving spelling and vocabulary. The letter frequencies in the game are based on an empirical count from text in some particular issues of newspapers when the game was designed, although the sample used may not have been huge.

The sample space can be taken to be the set of all English letters together with the blank, so has size 27, but this time the probabilities of its members are not all the same. Applying our definition (9.6) gives, for example,

$$\begin{aligned}
 \Pr(\text{vowel}) &= \Pr(\{A, E, I, O, U\}) \\
 &= \Pr(A) + \Pr(E) + \Pr(I) + \Pr(O) + \Pr(U) \\
 &= 0.09 + 0.12 + 0.09 + 0.08 + 0.04 \\
 &= 0.42
 \end{aligned}$$

In fact, not only does our new definition (9.6) deal with finite sample spaces where the probabilities are not all the same, but it can also deal with some infinite sample spaces. Suppose our sample space is $\mathbb{N}$ and the probability of an element $n \in \mathbb{N}$ is given by

$$\Pr(n) = \frac{1}{2^n}.$$

For this to be valid, we need the probabilities of all the elements in the sample space to add up to 1, so that (9.5) is satisfied. In this case, we have

$$\sum_{x \in U} \Pr(x) = \sum_{n \in \mathbb{N}} \Pr(n) = \sum_{n \in \mathbb{N}} \frac{1}{2^n} = \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \frac{1}{2^4} + \cdots = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \cdots$$

But this is just the sum of an infinite geometric series, with first term $a = \frac{1}{2}$ and common ratio $r = \frac{1}{2}$. So, by (6.45), the sum is

$$\frac{a}{1-r} = \frac{\frac{1}{2}}{1-\frac{1}{2}} = \frac{\frac{1}{2}}{\frac{1}{2}} = 1.$$

So all the probabilities do indeed add up to 1, and (9.5) is satisfied.

We can calculate the probabilities of events using (9.6). For example,

$$\begin{aligned}
 & \Pr(\text{a randomly chosen number is odd}) \\
 &= \Pr(\{\text{odd numbers}\}) \\
 &= \sum_{k=1}^{\infty} \frac{1}{2^{2k-1}} \quad (\text{using the fact that the odd numbers are } \{2k-1 : k \in \mathbb{N}\}) \\
 &= \sum_{k=1}^{\infty} \frac{1}{2} \frac{1}{2^{2k}} \\
 &= \sum_{k=1}^{\infty} \frac{2}{4^k} \quad (\text{which is now the sum of a geometric series with } a = \frac{1}{2} \text{ and } r = \frac{1}{4}) \\
 &= \frac{\frac{1}{2}}{1 - \frac{1}{4}} \quad (\text{by (6.45)}) \\
 &= \frac{\frac{1}{2}}{\frac{3}{4}} \\
 &= \frac{2}{3}.
 \end{aligned}$$

This should *not* be interpreted as saying that two-thirds of positive integers are odd! It is just saying that, *under the particular probabilities we have assigned to positive integers*, the probability of a randomly chosen positive integer being odd is $\frac{2}{3}$. Assigning different probabilities to the numbers could give a very different probability of oddness.

Here is another example.

$$\begin{aligned}
 \Pr(\text{a randomly chosen number is } \leq 10) &= \Pr(\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}) \\
 &= \sum_{n \in \{1, 2, \dots, 10\}} \frac{1}{2^n} \\
 &= \sum_{n=1}^{10} \frac{1}{2^n} \\
 &= \frac{1}{2} + \frac{1}{2^2} + \dots + \frac{1}{2^{10}}.
 \end{aligned}$$

This is the sum of a *finite* geometric series, with ten terms and the same a and r as before. So, using (6.44), the sum is

$$a \left(\frac{1 - r^{10}}{1 - r} \right) = \frac{1}{2} \left(\frac{1 - (\frac{1}{2})^{10}}{1 - \frac{1}{2}} \right) = \frac{1}{2} \left(\frac{1 - (\frac{1}{2})^{10}}{1 - \frac{1}{2}} \right) = 1 - \frac{1}{2^{10}} = 1 - \frac{1}{1024} = \frac{1023}{1024} \simeq 0.999.$$

The probabilities we have assigned to the positive integers in this example are far from uniform. Can you imagine giving all positive integers the *same* probability? If so,

what would that probability be? Would the probabilities sum to 1, as required? If not, we cannot really call them probabilities.

If it's too tricky to assign the same probability to all the positive integers, how close can we get? Suppose you want to give a decreasing sequence of probabilities to the positive integers such that they all sum to 1. What is the most slowly-decreasing sequence you can come up with that does this, while still giving every positive integer a positive probability?

9.3^α CHOICE OF SAMPLE SPACE

We have seen in the previous section that, to define probabilities of events, we need to have a sample space. Each element of the sample space must have a defined probability, with those probabilities summing to 1. Events correspond to subsets of the sample space, and the probability of an event is just the sum of the probabilities of its elements.

Choice of sample space is therefore fundamental. You need it to be an accurate model of the situation you are studying, so that the probabilities of events tell you about their likelihood in that situation.

Suppose you are playing Monopoly, where two dice are thrown and the numbers they show are added to give the number of steps you take on that move. So the length of your move is an integer in $\{2, 3, \dots, 12\}$. It is tempting to make this set the sample space. But what probabilities should we assign to its elements?

With some thought or experimentation, it soon becomes clear that the elements of $\{2, 3, \dots, 12\}$ are not all equally likely, so we should not just give them each a probability of $1/11$. (Doing so would define a sample space that is valid *in itself*, in that it satisfies the definition of a sample space. But the probabilities calculated from it do not align with the actual probabilities of the various totals obtained when throwing two dice. So these uniform probabilities are incorrect, as a model of this situation.)

To determine what the correct probabilities should be, we need to go deeper. Although the data we are interested in is just the total of the numbers shown on the two dice, the random process of throwing two dice gives a larger set of outcomes, namely a number from $\{1, 2, 3, 4, 5, 6\}$ on each die. So the full set of outcomes, from throwing two dice, is the Cartesian product

$$\underbrace{\{1, 2, 3, 4, 5, 6\}}_{\text{possible results from first die}} \times \underbrace{\{1, 2, 3, 4, 5, 6\}}_{\text{possible results from second die}},$$

which is the set of all pairs of results, one from the first die and another from the second die. It helps to visualise these outcomes in a 6×6 table:

(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)
(2,1)	(2,2)	(2,3)	(2,4)	(2,5)	(2,6)
(3,1)	(3,2)	(3,3)	(3,4)	(3,5)	(3,6)
(4,1)	(4,2)	(4,3)	(4,4)	(4,5)	(4,6)
(5,1)	(5,2)	(5,3)	(5,4)	(5,5)	(5,6)
(6,1)	(6,2)	(6,3)	(6,4)	(6,5)	(6,6)

We assume throughout that both dice are fair, i.e., each of the six outcomes of each die is equally likely and has probability $1/6$. We also assume throughout that the two dice are not linked in any way; the outcome of one does not influence the outcome of the other. It follows that all the pairs of outcomes are equally likely too, and since there are $6 \times 6 = 36$ pairs of outcomes, they must have probability $1/36$ each.

This gives us a more fine-grained sample space for the throw of two dice, with 36 elements (instead of just 11 for the possible totals), and it now has uniform probabilities. (Uniformity is not an essential feature for probabilities of elements of sample spaces, but we like it when it happens, as it makes life easier, provided it yields an accurate model.)

Furthermore, we can now calculate probabilities for the total of two dice. For example, what is the probability that the total is 8? This corresponds to the following subset of our new sample space, consisting of all pairs whose sum is 8:

$$\{(2,6), (3,5), (4,4), (5,3), (6,2)\}.$$

Since this subset has five elements, and since each element has probability $1/36$, we have

$$\Pr(\text{total is 8}) = \frac{1}{36} + \frac{1}{36} + \frac{1}{36} + \frac{1}{36} + \frac{1}{36} = \frac{5}{36}.$$

We can do a similar calculation for every possible outcome in the set $\{2,3,\dots,12\}$.

It is helpful to envisage this event on our diagram of the sample space. The elements of this event are circles in **green** in the next diagram:

(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)
(2,1)	(2,2)	(2,3)	(2,4)	(2,5)	(2,6)
(3,1)	(3,2)	(3,3)	(3,4)	(3,5)	(3,6)
(4,1)	(4,2)	(4,3)	(4,4)	(4,5)	(4,6)
(5,1)	(5,2)	(5,3)	(5,4)	(5,5)	(5,6)
(6,1)	(6,2)	(6,3)	(6,4)	(6,5)	(6,6)

We see that this event, where the total of the two dice is 8, is a diagonal of five pairs. In fact, all the other possible totals correspond to diagonals parallel to this one. We can see that a total of 2 has only one element, so its probability is $1/36$, and that as the totals increase, their probabilities increase too, with the probability increasing by $1/36$ for each increment of the total until the total is 7, which corresponds to the longest diagonal and maximises the probability:

$$\Pr(\text{total is } 7) = \frac{6}{36} = \frac{1}{6}.$$

Then, as the total keeps increasing, the probability decreases at the same rate until we reach the highest possible total, 12, with a probability of $1/36$.

total of two dice	2	3	4	5	6	7	8	9	10	11	12
probability	$\frac{1}{36}$	$\frac{2}{36}$	$\frac{3}{36}$	$\frac{4}{36}$	$\frac{5}{36}$	$\frac{6}{36}$	$\frac{5}{36}$	$\frac{4}{36}$	$\frac{3}{36}$	$\frac{2}{36}$	$\frac{1}{36}$

Once we have determined these probabilities of the totals 2,3,...,12, we can then use them along with the smaller sample space $\{2, 3, \dots, 12\}$ to compute probabilities of events pertaining solely to values of the total of the two dice (e.g., whether the total is 8, or even, or prime, or ≤ 5 , etc.). For example,

$$\Pr(\text{total is } \leq 5) = \frac{1}{36} + \frac{2}{36} + \frac{3}{36} + \frac{4}{36} = \frac{10}{36} = \frac{5}{12}. \quad (9.7)$$

But keep in mind that:

- To work out the probabilities of the totals in the first place, we *needed* the larger sample space, consisting of the 36 pairs with uniform probabilities.

- If we want to determine the probabilities of other events relating to the throw of two dice, we will need the larger sample space we have introduced. For example, in Monopoly there are specific consequences (including possible imprisonment!) when the two dice show the same number. This is not catered for by a sample space based only on the sum of the two numbers. But, using our larger sample space, we see that

$$\begin{aligned}
 \Pr(\text{the two dice show the same number}) &= \Pr(\{(1,1), (2,2), (3,3), (4,4), (5,5), (6,6)\}) \\
 &= \frac{6}{36} \\
 &= \frac{1}{6}.
 \end{aligned}$$

This example illustrates a more general point. While a sample space with nonuniform probabilities is ok in principle, and indeed natural in many situations, there is often a larger sample with uniform probabilities that underlies it in some way. Uniform probabilities can be simpler to deal with than nonuniform ones, so a sample space with uniform probabilities may make some probability calculations easier even if the sample space itself is larger.

Having said that, our first priority is to use a sample space and associated probabilities that model the situation at hand as accurately as possible. So we will not go for uniform probabilities if they remove us from the reality we are trying to model.

Our earlier Scrabble example is another case where a larger sample space, with uniform probabilities, could be used. In that example, the sample space we used was the set of all English letters together with the blank, making 27 elements in all, and we used nonuniform probabilities based on the numbers of tiles of each type. However, the individual tiles themselves can be treated as elements of a sample space of size 100, each with probability 0.01. To compute the probability of a vowel, we count up all tiles bearing a vowel, of which there are 42 ($9 \times A$, $12 \times E$, $9 \times I$, $8 \times O$, $4 \times U$), and divide this by the total number of tiles, 100, to obtain

$$\Pr(\text{vowel}) = \frac{|\{\text{tiles with a vowel}\}|}{|\{\text{all tiles}\}|} = \frac{42}{100} = 0.42,$$

using (9.1). This particular calculation may or may not seem easier than the one we did earlier. But the key point here is that the larger sample space underpins the one we used earlier. The probabilities in our table, such as 0.09 for A and so on, were derived from the observation that we had 100 equally likely tiles, of which nine were A. So, in effect, we used the larger sample space (of individual tiles rather than letters) to work out, and justify, the probabilities we assigned to the letters in the smaller sample space. So, again, it is the larger sample space (with its uniform probabilities) which is “what’s really going on”; the smaller sample space summarises those aspects of the larger space that

are relevant to the problem at hand, and its probabilities are obtained by calculations from the larger sample space.

9.4^α MUTUALLY EXCLUSIVE EVENTS

Two events are **mutually exclusive** if they are disjoint.

Since *events* are just subsets of the sample space, this definition just says that two events are disjoint if their *sets* are disjoint. So, two events A and B are mutually exclusive if and only if $A \cap B = \emptyset$.

This means that, if one of the events occurs, then the other cannot possibly occur. Maybe neither of them occurs. But they can't *both* occur.

If A and B are mutually exclusive, then the probability of their union is just the sum of their separate probabilities:

$$A \cap B = \emptyset \implies \Pr(A \cup B) = \Pr(A) + \Pr(B), \quad (9.8)$$

or, in other words,

$$\Pr(A \sqcup B) = \Pr(A) + \Pr(B), \quad (9.9)$$

since $A \sqcup B$ is only defined if the events are disjoint. This follows from:

$$\begin{aligned} \Pr(A \sqcup B) &= \sum_{x \in A \sqcup B} \Pr(x) && \text{(by definition of probability, (9.4))} \\ &= \left(\sum_{x \in A} \Pr(x) \right) + \left(\sum_{x \in B} \Pr(x) \right) \\ &\quad \text{(since each } x \in A \sqcup B \text{ belongs to exactly one of } A \text{ or } B) \\ &= \Pr(A) + \Pr(B) \\ &\quad \text{(again using the definition of probability, twice).} \end{aligned}$$

For example, if we throw two dice and ask about the probability that their sum is ≤ 5 or ≥ 10 (perhaps because we are playing monopoly and want to avoid landing on a

dangerous block of four consecutive squares). We already know that $\Pr(\text{total is } \leq 5) = 10/36$ (see (9.7)) and can similarly work out that $\Pr(\text{total } \geq 10) = 6/36$. Then

$$\begin{aligned}
 \Pr(\text{total is } \leq 5 \text{ or } \geq 10) &= \Pr((\text{total is } \leq 5) \cup (\text{total is } \geq 10)) \\
 &= \Pr(\text{total is } \leq 5) + \Pr(\text{total is } \geq 10) \\
 &\quad \text{(by (9.8), since the events are disjoint)} \\
 &= \frac{10}{36} + \frac{6}{36} \\
 &= \frac{16}{36} \\
 &= \frac{4}{9}.
 \end{aligned}$$

Our expression (9.8) for the probability of the disjoint union of two sets extends to disjoint unions of any number of sets.

Sets $A_1, A_2, \dots, A_n$ are **mutually disjoint**, or **pairwise disjoint**, if every pair of them is disjoint, i.e., for all $i, j \in \{1, 2, \dots, n\}$ we have $A_i \cap A_j = \emptyset$.

Let $n \in \mathbb{N}$ and let $A_1, A_2, \dots, A_n$ be mutually disjoint sets. Then

$$\Pr(A_1 \sqcup A_2 \sqcup \dots \sqcup A_n) = \Pr(A_1) + \Pr(A_2) + \dots + \Pr(A_n). \quad (9.10)$$

This important general principle may be captured in words by saying that the probability of any disjoint union is the sum of the probabilities of the events. More succinctly, *probability is additive over disjoint unions*.

Decomposing events into disjoint unions of simpler events is a very powerful tool in probability. In fact, we have come very close to using it already. When calculating the probability that a random Scrabble tile is a vowel using the sample space of all 100 tiles each with probability 0.01 (p. 316 in § 9.3^a), we used our knowledge of the numbers of tiles bearing each vowel. Converting these to probabilities, and expressing the event “the tile is a vowel” as a disjoint union of simpler events, we have

$$\begin{aligned}
 \Pr(\text{tile is vowel}) &= \Pr(\{\text{vowel tiles}\}) \\
 &= \Pr(\{\text{A-tiles}\} \sqcup \{\text{E-tiles}\} \sqcup \{\text{I-tiles}\} \sqcup \{\text{O-tiles}\} \sqcup \{\text{U-tiles}\}) \\
 &= \Pr(\{\text{A-tiles}\}) + \Pr(\{\text{E-tiles}\}) + \Pr(\{\text{I-tiles}\}) + \Pr(\{\text{O-tiles}\}) + \Pr(\{\text{U-tiles}\}) \\
 &\quad \text{(by (9.10))} \\
 &= 0.09 + 0.12 + 0.09 + 0.08 + 0.04 \\
 &= 0.42.
 \end{aligned}$$

The very definition of probability may be viewed in terms of the additivity of probability over disjoint unions. An event is just the disjoint union of all the *singleton sets*

(i.e., sets of one element) obtained from the elements of the event, and its probability is just the sum of the probabilities of those singleton sets.³

Later we will see more applications of the fact that probability is additive over disjoint unions.

9.5 OPERATIONS ON EVENTS

Since events are just subsets of the sample space, we can apply set operations to them to form other events. We have already seen the disjoint union in the previous section.

The complement $\bar{A}$ of an event A occurs precisely when A does not occur. So A and its complement $\bar{A}$ are mutually exclusive, and their union is the entire sample space. So

$$\begin{aligned}\Pr(A \sqcup \bar{A}) &= \sum_{x \in A \sqcup \bar{A}} \Pr(x) && \text{(by definition of probability)} \\ &= \sum_{x \in U} \Pr(x) && \text{(by definition of set complementation)} \\ &= 1, && \text{(by (9.5)),}\end{aligned}$$

in keeping with the certainty that either A or $\bar{A}$ occurs.

We can also apply our previous expression for the probability of mutually exclusive events, (9.8), to A and $\bar{A}$. This gives

$$\Pr(A \sqcup \bar{A}) = \Pr(A) + \Pr(\bar{A}).$$

Equating these two expressions for $\Pr(A \sqcup \bar{A})$, we have

$$\Pr(A) + \Pr(\bar{A}) = 1.$$

Therefore

$$\Pr(\bar{A}) = 1 - \Pr(A). \quad (9.11)$$

Examples:

- Some children play a pencil-and-paper “dice cricket” game where each throw of a fair die gives the number of runs scored against one delivery, except that 5 is out. So the probability of *not* being out, from one specific delivery (i.e., one throw of the die), is

$$\Pr(\text{not out}) = 1 - \Pr(\text{out}) = 1 - \frac{1}{6} = \frac{5}{6}$$

³ Strictly speaking, the probability of a set is the sum of the probabilities of its individual *elements*; it is the probabilities of the individual elements that are the starting point, and it is *from them* that the probabilities of events are defined in (9.6). But, of course, the probability of a singleton set equals the probability of its sole element. So it is true that the probability of an event is just the sum of the probabilities of all the singleton events (i.e., singleton sets) inside it.

- In drawing one of the 100 letter tiles in Scrabble, the probability of drawing a blank tile is $2/100$, so

$$\Pr(\text{not drawing a blank}) = 1 - \Pr(\text{drawing a blank}) = 1 - \frac{2}{100} = \frac{98}{100} = 0.98.$$

More generally, suppose $A \subseteq B$ and consider the set difference $B \setminus A$. In this case, $B = A \sqcup (B \setminus A)$. Therefore

$$\Pr(B) = \Pr(A \sqcup (B \setminus A)) = \Pr(A) + \Pr(B \setminus A). \quad (9.12)$$

So we have

$$A \subseteq B \implies \Pr(B \setminus A) = \Pr(B) - \Pr(A). \quad (9.13)$$

It also follows from (9.12) that

$$A \subseteq B \implies \Pr(A) \leq \Pr(B). \quad (9.14)$$

This could be paraphrased as saying that you cannot make something *less* likely by creating *more* ways for it to happen. Not surprising, but good to know!

Example:

In drawing our first Scrabble tile, what is the probability that we get a letter tile (not a blank) that has a consonant? We already know that the probability of a vowel is 0.42 and the probability of a non-blank tile is 0.98. Define events A and B for these two occurrences:

A : the tile we draw is a vowel

B : the tile we draw is a letter (not a blank).

Then $A \subseteq B$, and $B \setminus A$ is the event whose probability we seek:

$B \setminus A$: the tile we draw is a consonant.

Since $A \subseteq B$, we have

$$\Pr(\text{consonant}) = \Pr(B \setminus A) = \Pr(B) - \Pr(A) = 0.98 - 0.42 = 0.56,$$

where the second equality here is by (9.13).

When $A \not\subseteq B$, the expression (9.13) no longer applies. But we still have $A \cap B \subseteq B$, so we can still say that

$$\Pr(B \setminus A) = \Pr(B) - \Pr(A \cap B). \quad (9.15)$$

Our earlier expression (9.13) for the situation $A \subseteq B$ is a special case of this, since $A \subseteq B$ implies $A = A \cap B$.

This raises the general question of how to determine $\Pr(A \cap B)$. This is intimately related to determining $\Pr(A \cup B)$. We consider these probabilities now, in the general

situation where events A and B that are not necessarily mutually exclusive, so their sets are not necessarily disjoint, and neither is necessarily a subset of the other.

Consider $\Pr(A \cup B)$. The union $A \cup B$ is not necessarily a *disjoint* union of A and B ; the sets may intersect, and their intersection $A \cap B$ may be small or large, and its probability matters here. Nonetheless, we can still find a way to express $A \cup B$ as a disjoint union of certain of its subsets. See if you can work out how to do this, and how to use this to work out $\Pr(A \cup B)$. Before showing how it's done, we pause to reflect on the methods we have been using so far in this section.

In working out probabilities, it often helps to decompose events into a disjoint union of simpler, mutually exclusive events, as we discussed on p. 318 in § 9.4^a. In terms of sets, we are just expressing a set as a partition of simpler sets. We have seen further instances of this already in this section:

- We expressed the sample space as the disjoint union of A and $\bar{A}$ in order to work out the relationship between the probabilities of A and $\bar{A}$ in (9.11).
- When $A \subseteq B$, we expressed B as the disjoint union of A and $B \setminus A$ in (9.12), in order to derive an expression for $\Pr(B \setminus A)$ in (9.13).

So, let us continue in this vein and consider $\Pr(A \cup B)$. Since

$$A \cup B = (A \setminus B) \sqcup (A \cap B) \sqcup (B \setminus A),$$

we have

$$\Pr(A \cup B) = \Pr(A \setminus B) + \Pr(A \cap B) + \Pr(B \setminus A). \quad (9.16)$$

Now,

$$\begin{aligned} \Pr(A) &= \Pr(A \setminus B) + \Pr(A \cap B) \\ \Pr(B) &= \Pr(A \cap B) + \Pr(B \setminus A). \end{aligned}$$

Adding these two equations, we obtain

$$\Pr(A) + \Pr(B) = \Pr(A \setminus B) + 2\Pr(A \cap B) + \Pr(B \setminus A).$$

Combining this with (9.16) gives

$$\Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B). \quad (9.17)$$

Rearranging, we also have

$$\Pr(A \cap B) = \Pr(A) + \Pr(B) - \Pr(A \cup B). \quad (9.18)$$

So, if we already know the probabilities of events A and B , then we need to know the probability of *one* of the union and intersection, in order to be able to work out the probability of the other.

These equations (9.17) and (9.18) are reminiscent of the relationship between the *sizes* of two sets and their union and intersection: see (1.13) and Exercises 1.9 and 1.10.

In fact, if we take our expressions from those exercises and divide each side by the size of the universal set (i.e., in this context, the sample space), then we have

$$\frac{|A \cup B|}{|U|} = \frac{|A|}{|U|} + \frac{|B|}{|U|} - \frac{|A \cap B|}{|U|}, \quad (9.19)$$

$$\frac{|A \cap B|}{|U|} = \frac{|A|}{|U|} + \frac{|B|}{|U|} - \frac{|A \cup B|}{|U|}. \quad (9.20)$$

Each quotient here is just the probability of the set shown in the numerator in the special case when all elements of the sample space are equally likely. So, really, (9.19) and (9.20) are just special cases of (9.17) and (9.18), respectively.

When working out $\Pr(A \cap B)$, it can sometimes help to partition one of the events, say B , into a disjoint union of other events, say $B_1, B_2, \dots, B_n$:

$$B = B_1 \sqcup B_2 \sqcup \dots \sqcup B_n.$$

Observe that

$$A \cap (B_1 \sqcup B_2) = (A \cap B_1) \sqcup (A \cap B_2).$$

This is just an application of the Distributive Law together with the observation that $A \cap B_1$ and $A \cap B_2$ are disjoint (since they are subsets of the disjoint sets B_1 and B_2 , respectively).

For three events, we have

$$A \cap (B_1 \sqcup B_2 \sqcup B_3) = (A \cap B_1) \sqcup (A \cap B_2) \sqcup (A \cap B_3).$$

See [Figure 9.1](#).

This observation generalises to n mutually exclusive events:

$$A \cap (B_1 \sqcup B_2 \sqcup \dots \sqcup B_n) = (A \cap B_1) \sqcup (A \cap B_2) \sqcup \dots \sqcup (A \cap B_n).$$

Therefore,

$$\begin{aligned} \Pr(A \cap B) &= \Pr(A \cap (B_1 \sqcup B_2 \sqcup \dots \sqcup B_n)) \\ &= \Pr((A \cap B_1) \sqcup (A \cap B_2) \sqcup \dots \sqcup (A \cap B_n)) \\ &= \Pr(A \cap B_1) + \Pr(A \cap B_2) + \dots + \Pr(A \cap B_n), \end{aligned} \quad (9.21)$$

by (9.10). So, one way to work out $\Pr(A \cap B)$ is to work out each $\Pr(A \cap B_i)$, where $1 \leq i \leq n$, and then just add these probabilities. That requires working out n probabilities of intersections of events, but in some situations it is possible to choose the partition $B = B_1 \sqcup B_2 \sqcup \dots \sqcup B_n$ in such a way that working out the probabilities $\Pr(A \cap B_i)$ is much easier than working out $\Pr(A \cap B)$.

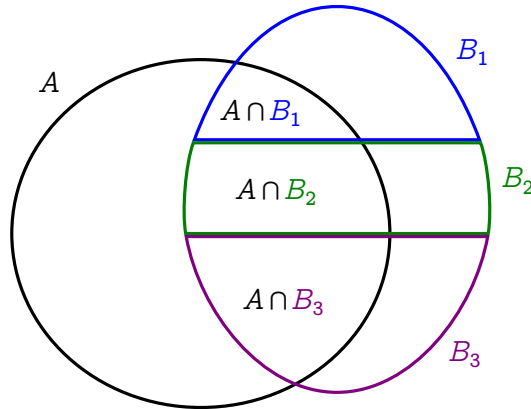


Figure 9.1: Events A and B , with B partitioned into B_1 , B_2 , B_3 . This means that $A \cap B$ is partitioned into $A \cap B_1$, $A \cap B_2$, $A \cap B_3$.

For example, what is the probability that a three-letter English word starts with 'c' and has a vowel as its second letter? This kind of question arises in word games and also, in more elaborate forms, in language modelling. Suppose the three-letter word is chosen uniformly at random from the set of all three-letter words in a standard word list. Then

$$\Pr(\text{1st letter is 'c' and 2nd letter is a vowel}) = \Pr((\text{1st letter is 'c'}) \cap (\text{2nd letter is a vowel})).$$

Let the events A and B be

$$\begin{aligned} A &= \text{1st letter is 'c'} \\ B &= \text{2nd letter is a vowel.} \end{aligned}$$

Then we can partition B into five events:

$$\begin{aligned} B_1 &= \text{2nd letter is 'a'}, \\ B_2 &= \text{2nd letter is 'e'}, \\ B_3 &= \text{2nd letter is 'i'}, \\ B_4 &= \text{2nd letter is 'o'}, \\ B_5 &= \text{2nd letter is 'u'}, \end{aligned}$$

and

$$B = B_1 \sqcup B_2 \sqcup B_3 \sqcup B_4 \sqcup B_5.$$

We can then use (9.21):

$$\begin{aligned}
 \Pr(A \cap B) &= \Pr((\text{1st letter is 'c'}) \cap (\text{2nd letter is a vowel})) \\
 &= \Pr((\text{1st letter is 'c'}) \cap (\text{2nd letter is 'a'})) + \\
 &\quad \Pr((\text{1st letter is 'c'}) \cap (\text{2nd letter is 'e'})) + \\
 &\quad \Pr((\text{1st letter is 'c'}) \cap (\text{2nd letter is 'i'})) + \\
 &\quad \Pr((\text{1st letter is 'c'}) \cap (\text{2nd letter is 'o'})) + \\
 &\quad \Pr((\text{1st letter is 'c'}) \cap (\text{2nd letter is 'u'})) \\
 &= \Pr(\text{first two letters are 'ca'}) + \\
 &\quad \Pr(\text{first two letters are 'ce'}) + \\
 &\quad \Pr(\text{first two letters are 'ci'}) + \\
 &\quad \Pr(\text{first two letters are 'co'}) + \\
 &\quad \Pr(\text{first two letters are 'cu'}) \\
 &= \frac{10}{1443} + \frac{0}{1443} + \frac{3}{1443} + \frac{14}{1443} + \frac{7}{1443} \\
 &= \frac{34}{1443} \\
 &\approx 0.024.
 \end{aligned}$$

Here we have used the word list at `/usr/share/dict/words` (i.e., filename `words`, directory path `/usr/share/dict/`) in the virtual Linux system in your Ed Workspace. This has 1443 three-letter words. It is not hard to use computational tools such as `grep` and `wc` (see Module 0 Applied Session) to calculate the answer directly in this case. But it is also worth thinking about how you might solve this problem manually, using a physical dictionary. Looking up the first two letters of a word enables you to narrow down the options a lot, and makes manual calculation feasible, showing the advantage of using (9.21). Even when using computational tools, smart partitioning of events can help calculate some probabilities more efficiently.

At this point, it is worth studying this probability of $A \cap B$ alongside the probabilities of the events A and B themselves, which are:

$$\begin{aligned}
 \Pr(A) &= \Pr(\text{1st letter is 'c'}) = \frac{44}{1443} \approx 0.030, \\
 \Pr(B) &= \Pr(\text{2nd letter is a vowel}) = \frac{760}{1443} \approx 0.527.
 \end{aligned}$$

Something to consider, to prepare for studying conditional probability later: based *only* on these probabilities of A , B and $A \cap B$, do you think that having first letter 'c' makes it *more* or *less* likely that the second letter is a vowel? Why?

An important special case of (9.21) arises when $B = U$, i.e., B is the entire sample space, so $\Pr(B) = 1$ and $A \cap B = A \cap U = A$. Then we have

$$\begin{aligned} U &= B_1 \sqcup B_2 \sqcup \cdots \sqcup B_n \\ \implies \Pr(A) &= \Pr(A \cap B_1) + \Pr(A \cap B_2) + \cdots + \Pr(A \cap B_n), \end{aligned} \quad (9.22)$$

by (9.21). This is sometimes called the **Law of Total Probability**.

9.6 INCLUSION-EXCLUSION FOR PROBABILITIES

The relationship between probabilities of events, their unions and intersections ((9.17) and (9.18)) can be extended to three events.

$$\begin{aligned} \Pr(A \cup B \cup C) &= \Pr(A \cup B) + \Pr(C) - \Pr((A \cup B) \cap C) \\ &\quad \text{(applying (9.17) to the sets } A \cup B \text{ and } C, \text{ instead of } A \text{ and } B) \\ &= \Pr(A) + \Pr(B) - \Pr(A \cap B) + \Pr(C) - \Pr((A \cup B) \cap C) \\ &\quad \text{(applying (9.17) to } A \text{ and } B, \text{ in } \Pr(A \cup B)) \\ &= \Pr(A) + \Pr(B) - \Pr(A \cap B) + \Pr(C) - \Pr((A \cap C) \cup (B \cap C)) \\ &\quad \text{(applying the Distributive Law to } (A \cup B) \cap C, \text{ in the last term)} \\ &= \Pr(A) + \Pr(B) - \Pr(A \cap B) + \Pr(C) - (\Pr(A \cap C) + \Pr(B \cap C) - \Pr((A \cap C) \cap (B \cap C))) \\ &\quad \text{(applying (9.17) to } A \cap C \text{ and } B \cap C) \\ &= \Pr(A) + \Pr(B) - \Pr(A \cap B) + \Pr(C) - \Pr(A \cap C) - \Pr(B \cap C) + \Pr((A \cap C) \cap (B \cap C)) \\ &= \Pr(A) + \Pr(B) - \Pr(A \cap B) + \Pr(C) - \Pr(A \cap C) - \Pr(B \cap C) + \Pr(A \cap B \cap C) \\ &\quad \text{(since } (A \cap C) \cap (B \cap C) = A \cap B \cap C) \\ &= \Pr(A) + \Pr(B) + \Pr(C) - \Pr(A \cap B) - \Pr(A \cap C) - \Pr(B \cap C) + \Pr(A \cap B \cap C) \quad (9.23) \\ &\quad \text{(rearranging slightly, for neatness).} \end{aligned}$$

Compare this expression with the expression for $|A \cup B \cup C|$ given in (8.2), and which you derived in Exercise 1.11.

The structure of the expression (9.23) is

$$\begin{aligned} \Pr(A \cup B \cup C) &= (\text{sum of the probabilities of } A, B, C) \\ &\quad - (\text{sum of the probabilities of all intersections of two of the sets } A, B, C) \\ &\quad + (\text{the probability of the intersection of all three of the sets } A, B, C). \end{aligned}$$

Note in particular the alternation in signs.

So we can write our expression for $\Pr(A \cup B \cup C)$ as

$$\Pr(A \cup B \cup C) = \sum_{k=1}^3 (-1)^{k+1} \cdot (\text{sum of probabilities of all intersections of } k \text{ of the sets } A, B, C).$$

This generalises to arbitrary numbers of sets.

Theorem 44 (Inclusion-Exclusion for probabilities, version 1). For all n ,

$$\Pr(A_1 \cup A_2 \cup \dots \cup A_n) = \sum_{k=1}^n (-1)^{k+1} \cdot (\text{sum of probabilities of all intersections of } k \text{ sets}) \quad (9.24)$$

□

This expresses the probability of a union of events in terms of probabilities of all possible intersections. It can be proved by induction (at greater length than many of our proofs by induction).

We can also express the probability of an intersection in terms of probabilities of all possible unions.

Theorem 45 (Inclusion-Exclusion for probabilities, version 2). For all n ,

$$\Pr(A_1 \cap A_2 \cap \dots \cap A_n) = \sum_{k=1}^n (-1)^{k+1} \cdot (\text{sum of probabilities of all unions of } k \text{ sets}) \quad (9.25)$$

□

[Theorem 44](#) and [Theorem 45](#) are really just generalisations of the Inclusion-Exclusion principle for counting, which we studied in § 8.3. There, we used it to find the size of a union of sets from the sizes of all possible intersections formed by them ([Theorem 42](#)), and also to find the size of an intersection of sets from the sizes of all possible unions of them ([Theorem 43](#)).

In fact, the proof by induction of the Inclusion-Exclusion principle for probability follows exactly the same pattern as the proof by induction we gave ([Theorem 42](#)) for the Inclusion-Exclusion principle for counting. Although the notation looks a bit different, in essence the only change is that each element contributes some specific constant amount (its probability) to the probability of any set containing it, rather than each element contributing 1 to the size of any set containing it; apart from this, the structure of the algebra and the reasoning is the same in the proofs of both Inclusion-Exclusion principles.

If all elements in our sample space are equally likely, then the probability of an event is just its size (as a set) divided by the size of the sample space (provided the sample space is finite). In this case, [Theorem 44](#) and [Theorem 45](#) just become

$$\begin{aligned}\frac{|A_1 \cup A_2 \cup \dots \cup A_n|}{|U|} &= \sum_{k=1}^n (-1)^{k+1} \cdot \left(\frac{\text{sum of sizes of all intersections of } k \text{ sets}}{|U|} \right), \\ \frac{|A_1 \cap A_2 \cap \dots \cap A_n|}{|U|} &= \sum_{k=1}^n (-1)^{k+1} \cdot \left(\frac{\text{sum of sizes of all unions of } k \text{ sets}}{|U|} \right).\end{aligned}$$

Removing the common denominator $|U|$ throughout, we just obtain [Theorem 42](#) and [Theorem 43](#). This shows that the Inclusion-Exclusion principle for counting is a special case of the Inclusion-Exclusion principle for probability.⁴

9.7 INDEPENDENT EVENTS

Intuitively, it is natural to consider two events to be “independent” of each other if the occurrence, or not, of one has no effect at all on the occurrence, or not, of the other.

For example, a toss of a coin at the start of a cricket match in Chennai seems independent of a coin toss at the start of a football match in Melbourne. Shaking two dice in the same cup and then throwing them — as done in Monopoly and other board games — is considered to give independent outcomes from the two dice; the physical interaction between them, during the shaking and throwing process, does not seem to influence their probabilistic behaviour.

Tying the notion of “independence” to the specific mechanism used is fraught with difficulty, because of the huge variety of processes that can be modelled using probability. Moreover, sometimes events that seem closely related can turn out, perhaps surprisingly, to behave independently as far as probability is concerned (and we will see examples of this).

Fortunately, there is a simple definition of independence which depends solely on the probabilities of the events involved, without referring at all to the mechanisms by which the events occur.

Two events A and B are **independent** if the probability that they both occur is the product of their separate probabilities:

$$\Pr(A \cap B) = \Pr(A) \cdot \Pr(B). \quad (9.26)$$

⁴ Conversely, if all elements of a finite sample space are equally likely, then probabilities are just sizes of sets divided by the size of the sample space; in other words, they are just counts that have been scaled so that the sample space itself is scaled to 1. So, in this scenario, the Inclusion-Exclusion principle for counting *implies* the Inclusion-Exclusion principle for probability. So, in fact, the two Inclusion-Exclusion principles are equivalent in a precise sense.

For example, when throwing a pair of dice, suppose we are interested in the first die showing 3 and the second die showing 5. Then

$$\begin{aligned} A &= \{\text{pairs } (x, y) \text{ with } x = 3\} \\ &= \{(3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (3, 6)\}, \\ B &= \{\text{pairs } (x, y) \text{ with } y = 5\} \\ &= \{(1, 5), (2, 5), (3, 5), (4, 5), (5, 5), (6, 5)\}, \\ A \cap B &= \{(3, 5)\}. \end{aligned}$$

So

$$\begin{aligned} \Pr(A) &= \Pr(\{(3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (3, 6)\}) = \frac{6}{36} = \frac{1}{6}, \\ \Pr(B) &= \Pr(\{(1, 5), (2, 5), (3, 5), (4, 5), (5, 5), (6, 5)\}) = \frac{6}{36} = \frac{1}{6}, \\ \Pr(A) \cdot \Pr(B) &= \frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36}, \\ \Pr(A \cap B) &= \Pr(\{(3, 5)\}) = \frac{1}{36}. \end{aligned}$$

Since $\Pr(A \cap B) = \Pr(A) \cdot \Pr(B)$, the events A and B are independent.

The independence of these two events is not too surprising, intuitively, if we accept that the two dice behave separately when thrown. In this case, the mechanism itself *suggests* independence. But independence can be more subtle than this.

Now suppose that the two events we are interested in are (i) the dice totalling seven, and (ii) the second die showing the “capped successor” of the first, meaning it’s one greater than the first up to a maximum of 6, so that the capped successor of x is $x + 1$ unless $x = 6$, in which case it’s 6. We have

$$\begin{aligned} A &= \{\text{pairs } (x, y) \text{ with } x + y = 7\} \\ &= \{(1, 6), (2, 5), (3, 4), (4, 3), (5, 2), (6, 1)\}, \\ B &= \{\text{pairs } (x, y) \text{ with } y = \text{capped successor of } x\} \\ &= \{\text{pairs } (x, y) \text{ with } y = \max(x + 1, 6)\} \\ &= \{(1, 2), (2, 3), (3, 4), (4, 5), (5, 6), (6, 6)\}. \end{aligned}$$

These events may not “look” so independent. They each depend on *both* dice; we cannot attribute them to two separate dice and appeal to the apparently separate nature of the two dice throws. But consider their intersection

$$A \cap B = \{(3, 4)\}$$

and the probabilities

$$\begin{aligned}
 \Pr(A) &= \Pr(\{(1,6), (2,5), (3,4), (4,3), (5,2), (6,1)\}) = \frac{6}{36} = \frac{1}{6}, \\
 \Pr(B) &= \Pr(\{(1,6), (2,5), (3,4), (4,3), (5,2), (6,6)\}) = \frac{6}{36} = \frac{1}{6}, \\
 \Pr(A) \cdot \Pr(B) &= \frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36}, \\
 \Pr(A \cap B) &= \Pr(\{(3,4)\}) = \frac{1}{36}.
 \end{aligned}$$

Again we have $\Pr(A \cap B) = \Pr(A) \cdot \Pr(B)$, so the events A and B are actually independent.

This example illustrates that, to determine independence, we must reason from the probabilities of the events A , B and $A \cap B$, and not rely just on intuition about the mechanisms that cause the events to occur.

To illustrate how delicate independence can be, let us consider the same event A as before (the total of the two dice is seven) but a slight change to B so that we now require the numbers on the two dice to be equal. The events are

$$\begin{aligned}
 A &= \{\text{pairs } (x, y) \text{ with } x + y = 7\} \\
 &= \{(1,6), (2,5), (3,4), (4,3), (5,2), (6,1)\}, \quad \text{as above;} \\
 B &= \{\text{pairs } (x, y) \text{ with } y = x\} \\
 &= \{(1,1), (2,2), (3,3), (4,4), (5,5), (6,6)\}.
 \end{aligned}$$

Superficially, this seems a small change from the previous example: A is unchanged, while B is very similar to what it was before, requiring now that the number on the second dice is *equal* to that on the first dice instead of being its capped successor. (In a sense, we have only replaced a difference of +1 or 0 by a difference of 0.) The probabilities of these two events are the same (with $\Pr(B)$ having been calculated on p. 316 in § 9.3^a):

$$\begin{aligned}
 \Pr(A) &= \frac{1}{6}, \\
 \Pr(B) &= \frac{1}{6}. \\
 \Pr(A) \cdot \Pr(B) &= \frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36}, \quad \text{as above.}
 \end{aligned}$$

But their intersection is now empty: two integers that add up to seven cannot be equal!

$$\begin{aligned}
 A \cap B &= \emptyset, \\
 \Pr(A \cap B) &= 0.
 \end{aligned}$$

Therefore $\Pr(A \cap B) \neq \Pr(A) \cdot \Pr(B)$, so these events A and B are *not* independent.

Compare *mutually exclusive* events (§ 9.4^α) with *independent* events.

- The probability of the union of *mutually exclusive* events is the sum of their individual probabilities.
- The probability of the intersection of *independent* events is the product of their individual probabilities.

Just as we said that probability is *additive* over *disjoint unions*, we can now say that probability is *multiplicative* over *intersections* of events.

It is important not to confuse *mutual exclusive* with *independent*. Do not be tricked by Venn diagrams! The fact that two mutually exclusive events “look separate from each other” on a Venn diagram does *not* mean they are independent. In fact, if two events A and B are mutually exclusive, then they are *not* independent, provided they both have positive probability. To see this, we just work from the definitions. By definition of mutual exclusivity,

$$\Pr(A \cap B) = 0,$$

yet, if both events have positive probability, then

$$\Pr(A)\Pr(B) > 0.$$

Therefore

$$\Pr(A \cap B) \neq \Pr(A)\Pr(B),$$

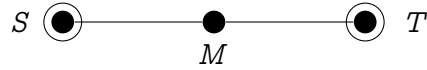
so the definition of independence is not satisfied, so the events are not independent. Intuitively, this actually makes sense, since the occurrence of A prevents B from occurring.

Independence cannot be depicted on a Venn diagram as simply as mutual exclusivity. If two events are independent, then we know they cannot be disjoint (provided each has positive probability). That’s a *necessary* condition for independence, but not a *sufficient* one. Determining independence is not just a matter of checking the emptiness, or not, of regions in a Venn diagram; it comes down to the precise relationship, given in (9.26), between the probabilities of the three sets A , B and $A \cap B$.

One of the main uses of independence is to help work out the probabilities of complex events. As usual, when confronted with a complex problem, we want to reduce it to simpler problems. A complex event, being representable as a set, can typically be described by combining simpler sets using standard set operations. So, if an event is an intersection of *independent* events, then we can work out the probabilities of those simpler events separately and then multiply them to get the probability of the more complex event we are interested in. This may be viewed as yet another instance of “divide-and-conquer” problem-solving.

Example: network reliability (I)

Suppose devices S and T are connected by two communications links in series, as shown in the following diagram.



Suppose also that

- each link may survive or fail, according to some random process;
- for each link, the probability that a link survives is p , and the probability that it fails is therefore $1 - p$, and that these probabilities are the same for each link;
- the links behave independently of each other. This means that, if A is an event determined solely by the first link SM , and B is an event determined solely by the second link MT , then $\Pr(A \cap B) = \Pr(A)\Pr(B)$.

What is the probability that S and T can communicate with each other? For this to be possible, we need both links between them to survive, otherwise there is no path between them. So

$$\begin{aligned}
 & \Pr(\text{there is a path from } S \text{ to } T \text{ using } \textit{surviving} \text{ links}) \\
 &= \Pr(\text{links } SM \text{ and } MT \text{ both survive}) \\
 &= \Pr((\text{link } SM \text{ survives}) \cap (\text{link } MT \text{ survives})) \\
 &= \Pr(\text{link } SM \text{ survives}) \cdot \Pr(\text{link } MT \text{ survives}) \\
 &\quad \text{(since the survival of each link is independent of the other link)} \\
 &= p \cdot p \\
 &= p^2.
 \end{aligned}$$

Example: network reliability (II)

Now consider the following simple communications network, under the same scenario: links behave independently and identically, with each having a survival probability of p .



Now what is the survival probability? We will work this out in two different ways.

First method:

Observe that, for there to be a path linking S and T , we just need at least one of the two links to survive.

$$\begin{aligned}
 & \Pr(\text{there is a path from } S \text{ to } T \text{ using } \textit{surviving} \text{ links}) \\
 &= \Pr(\text{at least one of the top and bottom links survives}) \\
 &= \Pr(\text{top link survives, or bottom link survives, or both}) \\
 &= \Pr((\text{top link survives}) \cup (\text{bottom link survives})) \\
 &= \Pr(\text{top link survives}) + \Pr(\text{bottom link survives}) \\
 &\quad - \Pr((\text{top link survives}) \cap (\text{bottom link survives})), \tag{9.27}
 \end{aligned}$$

by (9.17). Now, the survival of the separate links are independent events, so

$$\Pr((\text{top link survives}) \cap (\text{bottom link survives})) = \Pr(\text{top link survives}) \cdot \Pr(\text{bottom link survives}).$$

Substituting back into (9.27), we have

$$\Pr(\text{there is a path from } S \text{ to } T \text{ using surviving links}) = p + p - p^2 = 2p - p^2.$$

Second method:

Let us now start with

$$\begin{aligned}
 & \Pr(\text{there is a path from } S \text{ to } T \text{ using surviving links}) \\
 &= 1 - \Pr(\text{there is no path from } S \text{ to } T \text{ using surviving links}) \tag{9.28}
 \end{aligned}$$

and work out the probability of the complementary event, that there is no path from S to T , instead.

For this complementary event to happen, the top and bottom links *must both fail*. Therefore

$$\begin{aligned}
 & \Pr(\text{there is no path from } S \text{ to } T \text{ using surviving links}) \\
 &= \Pr(\text{top and bottom links both fail}) \\
 &= \Pr((\text{top link fails}) \cap (\text{bottom link fails})) \\
 &= \Pr(\text{top link fails}) \cdot \Pr(\text{bottom link fails}) \\
 &\quad \text{(since the failure of each link is independent of the other link)} \\
 &= (1 - p) \cdot (1 - p) \\
 &= (1 - p)^2.
 \end{aligned}$$

Substituting back into (9.28), we obtain

$$\begin{aligned}\Pr(\text{there is a path from } S \text{ to } T \text{ using surviving links}) &= 1 - (1 - p)^2 \\ &= 1 - (1 - 2p + p^2) \\ &= 2p - p^2.\end{aligned}$$

This agrees with the result of our first method, above. This second method is somewhat simpler in this case, and illustrates the benefit of being alert to the possibility of computing the probability of an event by computing the probability of its complement instead.

9.8 CONDITIONAL PROBABILITY

The probability of an event quantifies how likely it is in a given setting. But we often need to find the probability of the same event in more than one setting. New information may come to light, which may affect our view of how likely an event is, so we may need to recalculate our probabilities. Or we might have several different competing explanations of some measurement or observation we have made, so we want to calculate the probability of our observation under these various explanations in order to compare them.

Conditional probability gives us tools to do this.

Suppose we are interested in an event A . Its probability is $\Pr(A)$, calculated using the probabilities of elements of the sample space.

Suppose then that another event B occurs. This might be a new development, a change in the circumstances. Or it might be just an improvement in our information about the situation. Or we may simply be supposing event B to occur (even if we don't know whether it occurs or not) in order to reason about what might happen if it did occur. Whatever the motivation, we still want to know the probability of A , but *under the condition* that B occurs, and this may well change the probability of A . We assume here that $B \neq \emptyset$ and $\Pr(B) > 0$.

Our consideration of what can happen is now more restricted in scope than it was previously. In particular, elements of the sample space outside B are now excluded. We have to restrict our calculations to elements of the sample space that are in B . In effect, B is the new sample space.

But the elements of B cannot have the same probabilities that they had in our original sample space, because those probabilities add up to $\Pr(B)$, which in general can be < 1 . This violates the fundamental requirement that the sum of the probabilities of all elements of the sample space must be 1.

So, in order to use B as a new sample space, we need new probabilities for its elements. These new probabilities should still be proportional to the original probabilities: if one element was twice as likely as another in the original sample space, then this should still be true now. What we will do, then, is to scale all the probabilities of the elements by

the same constant factor so that they add up to 1. The appropriate scaling to use is to divide the probability of each element by $\Pr(B)$.

For any $x \in B$, we write $\Pr(x | B)$ for the probability of element x when B is used as the sample space. Then, using our usual notation $\Pr(x)$ for the probability of element x in the original sample space, we have

$$\Pr(x | B) = \frac{\Pr(x)}{\Pr(B)}.$$

We should check that these probabilities satisfy the requirements for probabilities of elements in a sample space. Firstly, they are clearly nonnegative (since the original probabilities $\Pr(x)$ are nonnegative and $\Pr(B) > 0$). Secondly, they sum to 1, because

$$\sum_{x \in B} \Pr(x | B) = \sum_{x \in B} \frac{\Pr(x)}{\Pr(B)} = \frac{1}{\Pr(B)} \sum_{x \in B} \Pr(x) = \frac{1}{\Pr(B)} \cdot \Pr(B) = 1.$$

So we can indeed use these probabilities for the elements of B when treating B as the new sample space.

Let $X \subseteq B$, and suppose we want its probability under the condition that B occurs. We denote this by $\Pr(X | B)$. To work this out, we just use our restricted sample space, B , with appropriately scaled probabilities $\Pr(x | B)$ for its elements. The definition of probability, (9.6), gives

$$\Pr(X | B) = \sum_{x \in X} \Pr(x | B) = \sum_{x \in X} \frac{\Pr(x)}{\Pr(B)} = \frac{1}{\Pr(B)} \sum_{x \in X} \Pr(x) = \frac{1}{\Pr(B)} \cdot \Pr(X) = \frac{\Pr(X)}{\Pr(B)}.$$

We emphasise that this formula only works if $X \subseteq B$.

Let's return to event A , which is a subset of the original sample space but not necessarily of B .

To study A in this restricted scenario (assuming event B), we must now exclude from consideration all elements of A that are not in B . In other words, we exclude $A \setminus B$, and focus just on $A \cap B$, since the elements of $A \cap B$ are the only way A can occur if we are restricting to B .

So to calculate the probability of A given B , we work out the probability of $A \cap B$ using the restricted sample space B with its scaled probabilities. So we use our earlier expression for $\Pr(X | B)$ with $X = A \cap B$. Therefore we have

$$\Pr(A | B) = \Pr(A \cap B | B) = \frac{\Pr(A \cap B)}{\Pr(B)}.$$

The main outcome from this discussion is the following expression for conditional probability:

$$\Pr(A|B) = \frac{\Pr(A \cap B)}{\Pr(B)} \quad (9.29)$$

Here are some important special cases:

- $\Pr(B|B) = 1$.
- $\Pr(\emptyset|B) = 0$.
- If A and B are disjoint then $\Pr(A|B) = 0$.

Equation (9.29) is most useful when we need to work out conditional probability and the probabilities of $A \cap B$ and B are available or can be calculated. In other situations, we may already have the conditional probability and can then use it to work out $\Pr(A \cap B)$, by rearranging (9.29):

$$\Pr(A \cap B) = \Pr(A|B)\Pr(B). \quad (9.30)$$

This makes sense, intuitively: for A and B to both occur, we need B to occur and, *given* that B occurs, we also need A to occur.

An especially important situation is when A and B are independent. This happens if and only if the conditional probability equals the unconditional probability.

Theorem 46. Events A and B are independent if and only if $\Pr(A|B) = \Pr(A)$.

Proof. ($\Rightarrow$)

Suppose A and B are independent. Then

$$\begin{aligned} \Pr(A|B) &= \frac{\Pr(A \cap B)}{\Pr(B)} \\ &= \frac{\Pr(A)\Pr(B)}{\Pr(B)} && \text{(by independence)} \\ &= \Pr(A). \end{aligned}$$

($\Leftarrow$)

Suppose $\Pr(A|B) = \Pr(A)$. Then

$$\frac{\Pr(A \cap B)}{\Pr(B)} = \Pr(A),$$

from which it follows that $\Pr(A \cap B) = \Pr(A)\Pr(B)$, i.e., A and B are independent. $\square$

Example 1:

On p. 324 in § 9.5, we asked if having first letter ‘c’ makes it *more* or *less* likely that a random three-letter word has a vowel as its second letter. We now answer this, using the probabilities that we calculated there.

Recall that

$$\Pr(\text{2nd letter is a vowel}) = \frac{760}{1443} \approx 0.527 \quad (9.31)$$

We need to compare this with the appropriate conditional probability, which is the probability of the second letter being a vowel *given that* the first letter is ‘c’.

$$\begin{aligned} \Pr(\text{2nd letter is a vowel} \mid \text{1st letter is 'c'}) &= \frac{\Pr((\text{2nd letter is a vowel}) \cap (\text{1st letter is 'c'}))}{\Pr(\text{1st letter is 'c'})} \\ &\quad \text{(by (9.29))} \\ &= \frac{34/1443}{44/1443} \quad \text{(see p. 324 in § 9.5)} \\ &= \frac{34}{44} \\ &\approx 0.773. \end{aligned}$$

By comparing this with the unconditional probability that the second letter is a vowel, in (9.31), we see that the conditional probability is greater than the unconditional one:

$$\Pr(\text{2nd letter is a vowel}) < \Pr(\text{2nd letter is a vowel} \mid \text{1st letter is 'c'}).$$

We conclude that the first letter being ‘c’ makes it *more* likely that the second letter is a vowel. This agrees with our intuition. In fact, it does more: it replaces vague intuition by a precise quantitative statement.

Example 2:

We saw on p. 316 that the probability of drawing a tile with a vowel from a bag of 100 Scrabble tiles is 0.42. But this does not mean that 42% of *letter tiles* are vowels, since there are also two blank tiles. The probability that a random tile is a vowel, given that it is not a blank, is

$$\begin{aligned} \Pr(\text{vowel} \mid \text{not blank}) &= \frac{\Pr(\{\text{vowel tiles}\} \mid \{\text{non-blank tiles}\})}{\Pr(\{\text{non-blank tiles}\})} \\ &= \frac{\Pr(\{\text{vowel tiles}\} \cap \{\text{non-blank tiles}\})}{\Pr(\{\text{non-blank tiles}\})} \\ &= \frac{\Pr(\{\text{vowel tiles}\})}{\Pr(\{\text{non-blank tiles}\})} \\ &\quad \text{(since every vowel tile is non-blank)} \\ &= \frac{0.42}{0.98} \\ &\approx 0.43. \end{aligned}$$

9.9 BAYES' THEOREM

The previous section introduced the conditional probability $\Pr(A | B)$ of event A given event B . What if we put the same two events the other way round, and ask about $\Pr(B | A)$? This is also a valid conditional probability, and we now pin down the relationship between the two conditional probabilities.

Theorem 47 (Bayes' Theorem).

$$\Pr(A | B) = \frac{\Pr(B | A) \Pr(A)}{\Pr(B)}.$$

Proof. Recall (9.30):

$$\Pr(A \cap B) = \Pr(B) \Pr(A | B).$$

Similarly,

$$\Pr(A \cap B) = \Pr(A) \Pr(B | A).$$

Equating the right-hand sides of the two previous equations gives

$$\Pr(B) \Pr(A | B) = \Pr(A) \Pr(B | A).$$

Then dividing each side by $\Pr(B)$ gives

$$\Pr(A | B) = \frac{\Pr(B | A) \Pr(A)}{\Pr(B)}.$$

□

So, to convert $\Pr(B | A)$ to $\Pr(A | B)$, just multiply it by the ratio $\Pr(A)/\Pr(B)$ of the probabilities of the events. To remember which ratio to use (so you don't accidentally use $\Pr(B)/\Pr(A)$ instead), keep in mind that the ratio you want is the one which, when written in-line using “/”, has A and B in the same order as they are in the conditional probability you're aiming for. So, if you're aiming for $\Pr(A | B)$, the ratio you want has A and B in that same order, i.e., $\Pr(A)/\Pr(B)$.

One of the main applications of Bayes' Theorem is to capture how beliefs change when new information becomes available.

A magician keeps three coins in their pocket, to help with their various tricks. One of the three coins is a fair coin, with Heads and Tails having probability $\frac{1}{2}$ each. Another has Heads on each side, and the third has Tails on each side. One of the three coins is chosen at random, with each of the three being equally likely to be chosen.

$$\Pr(\text{Fair}) = \Pr(\text{DoubleHead}) = \Pr(\text{DoubleTail}) = \frac{1}{3}.$$

This chosen coin is then tossed once, and the outcome observed. We see only the outcome on the upper face of the coin; we do not get to turn it over and see what was on the other side.

Let A be the event that the chosen coin is the fair coin. Before we see the outcome of a toss, our knowledge about A is captured by

$$\Pr(A) = \Pr(\text{Fair}) = \frac{1}{3}.$$

Now suppose the coin comes up Heads. We can work out its probability in (at least) two different ways. Firstly,

$$\begin{aligned} \Pr(\text{Heads}) &= \Pr(\text{Heads} \cap \text{Fair}) + \Pr(\text{Heads} \cap \text{DoubleHead}) + \Pr(\text{Heads} \cap \text{DoubleTail}) \\ &\quad \text{(by the Law of Total Probability, (9.22))} \\ &= \Pr(\text{Heads} | \text{Fair}) \Pr(\text{Fair}) + \\ &\quad \Pr(\text{Heads} | \text{DoubleHead}) \Pr(\text{DoubleHead}) + \\ &\quad \Pr(\text{Heads} | \text{DoubleTail}) \Pr(\text{DoubleTail}) \\ &\quad \text{(using (9.30) three times)} \\ &= \frac{1}{2} \cdot \frac{1}{3} + 1 \cdot \frac{1}{3} + 0 \cdot \frac{1}{3} \\ &= \frac{1}{6} + \frac{1}{3} + 0 \\ &= \frac{1}{2}. \end{aligned}$$

The second method of working this out is much quicker, and uses symmetry. Observe that the whole set-up with the three coins and their uniform probabilities is completely symmetric in Heads and Tails, so that any outcome from an experiment done with this set-up must have the same probability as the outcome obtained by swapping Heads and Tails throughout. Therefore, for a single toss of the coin chosen uniformly at random, we have

$$\Pr(\text{Heads}) = \Pr(\text{Tails}) = \frac{1}{2}.$$

Note how this insight from symmetry enables us to bypass the details of the earlier calculation, which gave the same result.

Now that we have seen the tossed coin come up Heads, we would like to know how likely it is that the coin is Fair. So we would like to determine

$$\Pr(\text{Fair} | \text{Heads}).$$

We already know

$$\begin{aligned}\Pr(\text{Heads} \mid \text{Fair}) &= \frac{1}{2} && \text{(by definition of a fair coin),} \\ \Pr(\text{Fair}) &= \frac{1}{3} && \text{(since the three coins are equally likely to be chosen),} \\ \Pr(\text{Heads}) &= \frac{1}{2} && \text{(as determined above).}\end{aligned}$$

Therefore, by Bayes' Theorem,

$$\Pr(\text{Fair} \mid \text{Heads}) = \frac{\Pr(\text{Heads} \mid \text{Fair}) \Pr(\text{Fair})}{\Pr(\text{Heads})} = \frac{\frac{1}{2} \cdot \frac{1}{3}}{\frac{1}{2}} = \frac{1}{3}.$$

So, from a single coin-toss, we do not change our belief about how likely it is that the coin is fair. But what about the coin being a DoubleHead or DoubleTail? Certainly a DoubleTail coin cannot ever show Heads, so the observation of Heads, even from just a single coin toss, rules out this possibility entirely:

$$\Pr(\text{DoubleTail} \mid \text{Heads}) = 0.$$

We can satisfy ourselves that this is in accord with Bayes' Theorem:

$$\Pr(\text{DoubleTail} \mid \text{Heads}) = \frac{\Pr(\text{Heads} \mid \text{DoubleTail}) \Pr(\text{DoubleTail})}{\Pr(\text{Heads})} = \frac{0 \cdot \frac{1}{3}}{\frac{1}{2}} = 0.$$

Finally, what does our observation of Heads tell us about the probability that the chosen coin is DoubleHeads? As a shortcut, we can use the fact that our three conditional probabilities,

$$\Pr(\text{Fair} \mid \text{Heads}), \quad \Pr(\text{DoubleHead} \mid \text{Heads}), \quad \Pr(\text{DoubleTail} \mid \text{Heads}),$$

must sum to 1, since the three outcomes are mutually exclusive and cover all possibilities for the chosen coin:

$$\Pr(\text{Fair} \mid \text{Heads}) + \Pr(\text{DoubleHead} \mid \text{Heads}) + \Pr(\text{DoubleTail} \mid \text{Heads}) = 1.$$

Therefore

$$\begin{aligned}\Pr(\text{DoubleHead} \mid \text{Heads}) &= 1 - \Pr(\text{Fair} \mid \text{Heads}) - \Pr(\text{DoubleTail} \mid \text{Heads}) \\ &= 1 - \frac{1}{3} - 0 \\ &= \frac{2}{3}.\end{aligned}$$

So we have changed our belief about how likely it is that the coin is DoubleHeads. Before we observed the coin toss outcome, our belief was summarised by $\Pr(\text{DoubleHead}) = \frac{1}{3}$. But now, we have $\Pr(\text{DoubleHead} | \text{Heads}) = \frac{2}{3}$, so we believe that the DoubleHead coin is twice as likely as before.

We can still work this last conditional probability out using Bayes' Theorem, for practice.

$$\Pr(\text{DoubleHead} | \text{Heads}) = \frac{\Pr(\text{Heads} | \text{DoubleHead}) \Pr(\text{DoubleHead})}{\Pr(\text{Heads})} = \frac{1 \cdot \frac{1}{3}}{\frac{1}{2}} = \frac{2}{3},$$

which agrees with our calculation above.

The process we have gone through here, of using Bayes' Theorem to update the probability we assign to an event we are interested in, is very powerful, even though the mathematical rule we are using is simple. This process find applications throughout science, engineering, economics, and indeed any field where data is analysed and beliefs must be quantified and then revised in the light of observations. It is a fundamental tool in machine learning and statistics. Every modern computer scientist must be proficient in its use.

There is some standard terminology that is used when Bayes' Theorem is applied in this way. The probability of an event *before* an observation is made is called its **prior probability**, and its conditional probability *after* the observation is made is called its **posterior probability**. In the last use of Bayes' Theorem above, we began with

$$\text{prior probability:} \quad \Pr(\text{DoubleHead}) = \frac{1}{3}.$$

We then observed a coin toss giving Heads, and determined:

$$\text{posterior probability:} \quad \Pr(\text{DoubleHead} | \text{Heads}) = \frac{2}{3},$$

which was different. In this case, the posterior probability was larger than the prior probability, but it can also be smaller. When we considered DoubleTail, we had

$$\begin{aligned} \text{prior probability:} \quad \Pr(\text{DoubleTail}) &= \frac{1}{3}, \\ \text{posterior probability:} \quad \Pr(\text{DoubleTail} | \text{Head}) &= 0. \end{aligned}$$

We saw that it is also possible for the prior and posterior probabilities to be equal:

$$\begin{aligned} \text{prior probability:} \quad \Pr(\text{Fair}) &= \frac{1}{3}, \\ \text{posterior probability:} \quad \Pr(\text{Fair} | \text{Head}) &= \frac{1}{3}. \end{aligned}$$

Notice that the expressions Bayes' Theorem gives for our three probabilities

$$\Pr(\text{Fair} \mid \text{Heads}), \quad \Pr(\text{DoubleTail} \mid \text{Heads}), \quad \Pr(\text{DoubleHead} \mid \text{Heads}),$$

each have $\Pr(\text{Heads})$ in their denominator. So calculating $\Pr(\text{Heads})$ is necessary if we want to calculate these three probabilities exactly. But what if we just want to *compare* the three probabilities, *relative to each other*? Maybe we just want to know which of the three probabilities is the largest. Or maybe we just want to make relative statements, like

“Given that we have observed Heads, the Double-Head coin is twice as likely as the Fair coin.”

In these cases, we don't need to know $\Pr(\text{Heads})$, because it just serves as a scaling factor. If we don't use it, then we won't get exact conditional probabilities any more; instead, we get three quantities that are in the same ratios to each other as the probabilities. We can compute

$$\begin{array}{llll} \Pr(\text{Fair} \mid \text{Heads}) & \propto & \Pr(\text{Heads} \mid \text{Fair}) \Pr(\text{Fair}) & = \frac{1}{2} \cdot \frac{1}{3} = \frac{1}{6}, \\ \Pr(\text{DoubleTail} \mid \text{Heads}) & \propto & \Pr(\text{Heads} \mid \text{DoubleTail}) \Pr(\text{DoubleTail}) & = 0 \cdot \frac{1}{3} = 0, \\ \Pr(\text{DoubleHead} \mid \text{Heads}) & \propto & \Pr(\text{Heads} \mid \text{DoubleHead}) \Pr(\text{DoubleHead}) & = 1 \cdot \frac{1}{3} = \frac{1}{3}. \end{array}$$

Each of these statements says that the left-hand side is *proportional* to the right-hand side. The same constant of proportionality is used in each case, namely $1/\Pr(\text{Heads})$, but this constant factor is now omitted from the calculation. The three numbers we obtain are no longer probabilities; they are still ≥ 0 , but they no longer sum to 1. But they are in the same ratios with each other as the probabilities were. We can see from these numbers that $\Pr(\text{DoubleHead} \mid \text{Heads})$ is the largest of the three probabilities, and that the double-headed coin is twice as likely as the fair coin.

So, in some situations where we only want to do comparisons rather than compute the probabilities exactly, we may not need to compute $\Pr(\text{Heads})$.

Having made that important point, let's return to considering the exact calculation of the probabilities, and in particular the calculation of the denominator $\Pr(\text{Heads})$. In the three-coins example, we first calculated $\Pr(\text{Heads})$ as a sum of terms of the form $\Pr(\text{Heads} \mid A) \Pr(A)$, where A is each of the three coins: $A \in \{\text{Fair}, \text{DoubleHead}, \text{DoubleTail}\}$. So we had

$$\begin{aligned} \Pr(\text{Heads}) &= \Pr(\text{Heads} \mid \text{Fair}) \Pr(\text{Fair}) + \\ &\quad \Pr(\text{Heads} \mid \text{DoubleHead}) \Pr(\text{DoubleHead}) + \\ &\quad \Pr(\text{Heads} \mid \text{DoubleTail}) \Pr(\text{DoubleTail}). \end{aligned}$$

This is very typical of applications of Bayes' Theorem, so much so that the theorem is often presented in the following form.

Theorem 48 (Bayes' Theorem, extended version). If $U = A_1 \sqcup A_2 \sqcup \cdots \sqcup A_n$ then, for each $j \in \{1, 2, \dots, n\}$,

$$\Pr(A_j | B) = \frac{\Pr(B | A_j) \Pr(A_j)}{\sum_{i=1}^n \Pr(B | A_i) \Pr(A_i)}.$$

Proof. If $U = A_1 \sqcup A_2 \sqcup \cdots \sqcup A_n$ then, by the Law of Total Probability (9.22),

$$\Pr(B) = \sum_{i=1}^n \Pr(B | A_i) \Pr(A_i). \quad (9.32)$$

(We are using (9.22) with the names A and B interchanged throughout, but that makes no difference to the underlying mathematics.)

Using Theorem 47 with A_j instead of A , with the substitution (9.32), we obtain

$$\Pr(A_j | B) = \frac{\Pr(B | A_j) \Pr(A_j)}{\Pr(B)} = \frac{\Pr(B | A_j) \Pr(A_j)}{\sum_{i=1}^n \Pr(B | A_i) \Pr(A_i)}.$$

□

An important special case is when $n = 2$. We have two events, which we can call A and $\bar{A}$, and

$$\Pr(A | B) = \frac{\Pr(B | A) \Pr(A)}{\Pr(B | A) \Pr(A) + \Pr(B | \bar{A}) \Pr(\bar{A})}.$$

9.10 EXERCISES

1. A random word is chosen from a list of all five-letter English words, and then one of that word's five letters is chosen.

(a) What is a suitable sample space for this experiment? What should the probabilities of its elements be?

(b) Challenge: use your Linux skills and `/usr/share/dict/words` to determine the probability of each English letter in this experiment.

2. A fair coin is tossed three times. The outcome is the sequence of results of the coin tosses. For convenience, we denote Heads and Tails by H and T, respectively.

Specify a suitable sample space, with associated probabilities, for this experiment.

3. A fair coin is tossed repeatedly until it comes up Heads for the first time. Then the tossing stops. The outcome is the sequence of the results of the tosses. For example, if the first Head occurs on the third toss, then the outcome is TTH.

Specify a suitable sample space, with associated probabilities, for this experiment.

4. Suppose two fair dice are thrown and their numbers are added. Let T be the total of the numbers on the two dice.

- (a) What is the probability that T is prime?
- (b) What is the minimum value of d such that the probability that the total differs from the middle value, 7, by at least d is at most 0.1? Symbolically, we are asking for the minimum d such that

$$\Pr(|T - 7| \geq d) \leq 0.1.$$

5. Find the probability that a random poker hand contains no diamonds.

6. A random positive integer ≤ 100 is chosen, with all choices being equally likely. What is the probability that the chosen number is divisible by 3 but not by 9?

7. (Birthday Paradox).

Let $n \in \mathbb{N}$. Suppose n people are chosen at random.

- (a) What is the probability that at least two of them share a birthday?
- (b) What is the minimum value of n such that it is more likely than not that there are at least two people in the set that share a birthday?

8. Prove that, for any events A and B ,

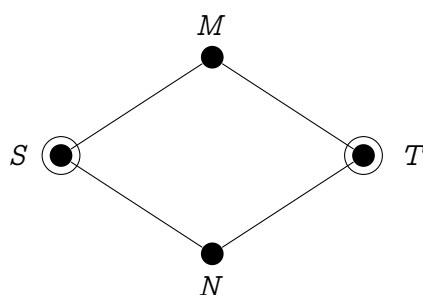
$$\Pr(A \cup B) = 1 - \Pr(\overline{A} \cap \overline{B}).$$

9. Prove by induction on n that, for all $n \in \mathbb{N}$ and any mutually disjoint sets $A_1, A_2, \dots, A_n$,

$$\Pr(A_1 \sqcup A_2 \sqcup \dots \sqcup A_n) = \sum_{k=1}^n \Pr(A_k).$$

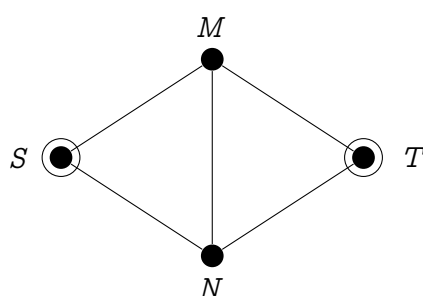
10. Suppose events A, B, C are independent. Must A also be independent of $B \cap C$? Justify your answer, with a proof (if the answer is Yes) or a counterexample (if the answer is No).

11. Recall the network reliability problems in § 9.7. Now consider the following network. Again, all links behave independently and have identical survival probability p .



- (a) Give an expression, in terms of p , for the probability that there is a path of surviving links from S to T .
- (b) Compare your answer to (a) with the answer to Exercise 4.8(b). Discuss the relationship between your answers to the two questions.

Now let's upgrade this communications network by adding a link between M and N , giving the following network.



- (c) Give an expression, in terms of p , for the probability that there is a path of surviving links from S to T in this upgraded network.
- (d) Compare your answer to (c) with the answer to Exercise 4.8(c). Discuss the relationship between your answers to the two questions.

12. Determine the probability that a random permutation on a set of two elements is fixed-point-free.

Then do the same for a set of three elements, and then a set of five elements.

13. What is the probability that a Scrabble letter is a vowel, given that it is in the first half of the alphabet?

14. Prove that, for any two events A and B , the two conditional probabilities $\Pr(A|B)$ and $\Pr(B|A)$ are equal if and only if the events have the same probability.

15. Consider again the network of four nodes and five links from Exercise 11. Suppose that $p = \frac{1}{2}$.

1. What is the probability that there is a path from S to T ?
2. What is the probability that there is a path from M to N ?

16. You are at a cricket match and the ball is hit high towards a fielder. You think, will they catch it? The fielder could be any of four players: Alex, Chris, Kim and Sam. You can't tell the difference between them in their white cricket uniforms at this distance. You assess that Alex has a probability of catching of 0.9, and a probability of dropping the catch of 0.1. Each of Chris, Kim and Sam has catching probability 0.4 and dropping probability 0.6.

- (a) What is the probability that the ball is caught?
- (b) Suppose now that you see that the ball is caught. What is the probability that the fielder who caught it was Alex? What is the probability that it was not Alex?

17. A serious crime is committed in a big city, and police are trying to identify the perpetrator. There are 5 million people who cannot be ruled out. A degraded fragment of DNA from the criminal is found at the scene. There is no other evidence. A random person has only a one-in-a-million chance of matching the DNA fragment. The fragment is compared with a database and a match is found with someone who provided DNA for a different reason in a completely irrelevant context some years ago. How likely is it that this person committed the crime? Are they guilty, beyond reasonable doubt?

18. In the three-coins example considered from page 337 onwards in § 9.9, suppose that, instead of observing just one coin toss, we observe two coin tosses instead.

For each of the following observations of the outcomes of the two coin tosses, determine the prior and posterior probabilities of the chosen coin being each of the three possibilities: Fair, DoubleHead, DoubleTail.

- (a) both tosses are Heads;
- (b) both tosses are Tails;
- (c) the first toss is Heads, the second is Tails.

DISCRETE PROBABILITY II

We continue our study of discrete probability by looking at random variables and probability distributions, including the four most important discrete probability distributions. These give us tools for modelling and analysing a huge variety of random processes.

10.1^α RANDOM VARIABLES

For a given sample space, there are many numerical quantities we might be interested in. For example, when throwing two dice — with each member of sample space $\{1, 2, 3, 4, 5, 6\} \times \{1, 2, 3, 4, 5, 6\}$ having probability $1/36$ — we might be interested in their sum (if playing Monopoly), or their product, or their maximum. The idea of a numerical quantity determined from a random member of a sample space is captured by the notion of a *random variable*.

A **random variable** is a function from a sample space to the real numbers. So, if U is the sample space, it's just a function $f: U \rightarrow \mathbb{R}$.

We think of a random variable working as follows.

1. First, a random member $x \in U$ of the sample space is chosen, according to the probabilities defined on U .
2. Then the function f is applied to x , giving the real number $f(x)$.
 - This part of the process is deterministic, meaning that it is *not* random: $f(x)$ is entirely *determined* by x . All the randomness in a random variable comes from the initial random selection of x from U .

We can also use a random variable repeatedly. If we do so, then each time we choose a random member of the sample space U , we do so *independently* of all previous choices we may have made.

Other examples of random variables:

- Points on a random Scrabble letter. The sample space consists of 100 tiles, each with probability $1/100$ (see p. 316). Given a tile, the function gives the number of points shown on that tile (or 0 for a blank).
- The length of a word chosen at random from a dictionary or word list.

- The number of tests a program passes, during testing on a fixed number of randomly chosen inputs.
- The number of visits to a specific website in a given time period.
- The lifetime of a device, in days, until it fails.
- The score made by a Test cricket batter in their next innings.

It's common to denote a random variable by a capital letter, and often one near the end of the alphabet, such as X , Y or Z , but that is not a formal requirement. This symbol is usually thought of as containing a random value, where that value is obtained by taking a random member of the sample space and then applying the function to it.

For example, if Z is a random variable representing the sum of the numbers shown when throwing two dice, then Z contains a value chosen randomly from $\{2, 3, 4, \dots, 11, 12\}$, and these values have the probabilities given on p. 315. If we pick any possible value $k \in \{2, 3, 4, \dots, 11, 12\}$, then we write

$$Z = k$$

for the event that the random variable Z has the value k . Since it's an event, it has a probability. We can use the usual definition of the probability of an event to work out the probability of the event $Z = k$. We just add up the probabilities of the elements of the sample space that belong to the event.

Here, the sample space members are the pairs (i, j) , where $i, j \in \{1, 2, 3, 4, 5, 6\}$. So we add up the probabilities of all these pairs such that $i + j = k$:

$$\Pr(Z = k) = \sum_{(i,j): i+j=k} \Pr((i,j)).$$

For example, if $k = 9$, then we have the event $Z = 9$, which consists of the pairs $(3, 6), (4, 5), (5, 4), (6, 3)$. Its probability is given by

$$\begin{aligned} \Pr(Z = 9) &= \Pr(\{(3, 6), (4, 5), (5, 4), (6, 3)\}) = \sum_{(i,j): i+j=9} \Pr((i,j)) \\ &= \Pr((3, 6)) + \Pr((4, 5)) + \Pr((5, 4)) + \Pr((6, 3)) \\ &= \frac{1}{36} + \frac{1}{36} + \frac{1}{36} + \frac{1}{36} \\ &= \frac{4}{36} \\ &= \frac{1}{9}. \end{aligned}$$

We have defined random variables as having codomain $\mathbb{R}$, following usual practice. This is because we will do computations with them that require real number operations, such as arithmetic operations and comparing size. But there are situations where people

use “random variables” with other codomains, such as $\mathbb{Z}_n$ or sets. This is fine as long as you don’t try to do operations with them that are not defined in the codomain being used. For example, you can’t find average values of “random variables” that are sets, since averaging needs addition and division and neither of these operations is defined on sets.

10.2^α PROBABILITY DISTRIBUTIONS

A **probability distribution** is a function that assigns a probability to every member of a set in such a way that the probabilities add up to 1.

We have seen probability distributions before. Every time we define a sample space, we must specify a *probability distribution* on its elements, where each element is given a probability and these must add up to 1. (The probabilities must, of course, also be in the interval $[0,1]$; this follows from calling them *probabilities*.)

Now, we use probability distributions more widely. We are particularly interested in probability distributions for random variables.

For any random variable X , each of its possible values k has a probability, written $\Pr(X = k)$ and equal to the sum of the probabilities of all elements of the sample space for which $X = k$ (see the end of the previous section). The probabilities of the values of X together constitute the **probability distribution** of the random variable X .

Once we specify the set of values a random variable X can take, together with its probability distribution, we have specified everything we need to know about X in order to use it. In the background, there is still a formal definition of X in terms of a sample space and a function from the sample space to the set of possible values. And we can go back to that formal definition if we want to (because any probability distribution contains probabilities, and these must ultimately come from a sample space). But, from now on, when we use random variables, we will mostly just refer to the set of possible values they can take together with the probability distribution on those numbers.

Keep in mind that

events	have	probabilities,
		<i>but</i>
random variables	have	probability <i>distributions</i> .

So, for example, if k is some fixed number in the codomain of random variable X , then it’s ok to write $\Pr(X = k)$ or to refer to the probability that $X = k$ or the probability of the value k (if the random variable X is clear from the context). And it’s ok to refer to the probability distribution of the random variable X . But it’s not ok to write “ $\Pr(X)$ ” or refer to the “probability of X ” when X is a random variable, since a random variable is not an event and therefore does not just have one single probability. And it’s not ok to refer to the probability distribution of $X = k$, since k is just a single specific value of X , and $X = k$ is an event; as such, $X = k$ has a probability, but it does not have an

entire probability *distribution*.

Although there are some similarities between sample spaces and random variables, there are also important differences, both in the way we define them and in their purpose and motivation.

- When defining sample spaces, we divide the range of possible outcomes up into elementary, “atomic” outcomes. We try to make the sample space elements as simple as we can, so that any event can be described by some set of them, and there is no requirement for these sample space elements to be numbers. We want the probabilities of individual elements to be easy to calculate, and we often try to get a sample space where all elements have the same probability. The main aim is for the sample space to be a good model of *the underlying random process*.
- When defining random variables, our priority is to capture *useful numerical functions of random data*. These might not be easy to calculate, and they typically lump many elements of a sample space together. The main aim is for the random variable to be a good model of *what we are interested in* about the random data.

Sometimes we need to combine random variables. One of the most common operations we want to do, to combine random variables, is to add them. We have seen one example of this, when we added the numbers obtained from throwing two dice.

So let’s consider what sums of random variables look like.

Let X and Y be random variables. Their sum is denoted by $X + Y$. This, too, is a random variable, but its probability distribution is, in general, different to the distributions of X and Y . To work out the probability that this new random variable $X + Y$ takes a specific value k , we have to look at all possible *pairs* of values of X and Y whose sum is k , and add up their probabilities:

$$\Pr(X + Y = k) = \sum_{(i,j): i+j=k} \Pr((X = i) \wedge (Y = j)),$$

where the sum is over all pairs (i, j) such that $i + j = k$.

For example, let X and Y be the number shown on two separate fair dice. Then $Z = X + Y$ is their total. We studied this on p. 348 and worked out the probability that $X + Y = 9$. See also p. 313 in § 9.3^α.

Two random variables X and Y are **independent** if, for every possible value x of X and every possible value y of Y , the events $X = x$ and $Y = y$ are independent. In other words, for all values x and y we have

$$\Pr((X = x) \wedge (Y = y)) = \Pr(X = x) \cdot \Pr(Y = y). \quad (10.1)$$

Examples:

- If two dice are thrown, and X and Y are the numbers shown on the first and second die respectively, then X and Y are considered independent (unless something dodgy is going on).
- Let X be the rank and let Y be the suit of a random card drawn from a standard deck of 52 playing cards. Then X and Y are independent.
- A random variable is not independent of itself! This seems intuitively obvious, and we can also see it from (10.1) (using the pair X, X instead of the pair X, Y).
- If X is a word chosen at random from a dictionary and Y is a word chosen at random from the same dictionary page as X , then X and Y are not independent.

10.3^α EXPECTATION

A random variable takes many different values, each with some probability. Sometimes, though, we want to work with a single number that is somehow representative of the entire set of possible values, taking into account their probabilities. This representative number should, in some sense, correspond to what we *expect* the random variable to give us, or to be *typical* of the values it gives. There are several different ways to do this; indeed, the terms “expected value” and “typical value” are often interpreted differently. But the most important and widely used representative value of a random variable is its *expected value*, also called its *expectation* or *mean*. This is based on the familiar notion of the *average* of a set of numbers.

You know that, to compute the average of a set of numbers, you just add them up and divide by how many of them there are. So, the average of the three numbers 2, 5, 6 is

$$\frac{2+5+6}{3} = \frac{13}{3} = 4\frac{1}{3}.$$

Another way to put it is that each number is multiplied by $\frac{1}{3}$ and then they are added. So the three numbers 2, 5, 6 each have a coefficient or “weight”, and the three coefficients are fractions and add up to 1. In this case, the three coefficients are all the same: $\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$.

There might be repetition among the numbers we are averaging. The average of 2, 5, 5, 6 (with 5 appearing twice) is

$$\frac{2+5+5+6}{4} = \frac{1}{4} \cdot 2 + \frac{2}{4} \cdot 5 + \frac{1}{4} \cdot 6 = 0.5 + 2.5 + 1.5 = 4.5.$$

We may view this as using the same three numbers 2, 5, 6 as before, but with different coefficients: $\frac{1}{4}, \frac{1}{2}, \frac{1}{4}$. These coefficients are still fractions and they still add up to 1.

These coefficients are reminiscent of probabilities, since they belong to the unit interval $[0, 1]$ and add up to 1. So, taken together, the coefficients can be viewed as a probability distribution.

We can use this viewpoint to define the average of any set of numbers on which there is a probability distribution. And a set of numbers with a probability distribution is nothing more or less than a random variable, as we saw in § 10.2^α. So, for any random variable X , we define its **expectation** $E(X)$ to be

$$E(X) = \sum_k k \cdot \Pr(X = k), \quad (10.2)$$

where the sum is over all possible values of X . So we just multiply each value by its probability, and add up all these products.

The expectation is also called the **expected value** or the **mean**. It can also be called the *average*, although it is more usual to reserve that term for averages obtained from actual observed data rather than from probability distributions.

Examples:

- Let X be the number obtained from throwing a fair die once. Then each value in $\{1, 2, 3, 4, 5, 6\}$ has probability $1/6$. So

$$E(X) = 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + 3 \cdot \frac{1}{6} + 4 \cdot \frac{1}{6} + 5 \cdot \frac{1}{6} + 6 \cdot \frac{1}{6} = \frac{1+2+3+4+5+6}{6} = \frac{21}{6} = \frac{7}{2} = 3\frac{1}{2}.$$

- Suppose we have a *biased* coin for which Heads is twice as likely as Tails:

$$\begin{aligned} \Pr(\text{Heads}) &= \frac{2}{3}, \\ \Pr(\text{Tails}) &= \frac{1}{3}. \end{aligned}$$

Let X be the number of Heads in a single toss, so

$$X = \begin{cases} 1, & \text{with probability } \frac{2}{3}; \\ 0, & \text{with probability } \frac{1}{3}; \end{cases}$$

Then

$$E(X) = 1 \cdot \frac{2}{3} + 0 \cdot \frac{1}{3} = \frac{2}{3} + 0 = \frac{2}{3}.$$

- In Scrabble, every letter tile displays (as a subscript) the points that letter is worth, if used in a word.¹ Blanks are worth 0 points.

¹ There are other factors involved when the game is played. The actual points you get from playing a letter may be doubled or tripled when played on certain special squares on the board, and may be doubled if used to make two words in a single turn. We ignore those factors. We focus only on the face value of a tile.

letter	points	prob.	letter	points	prob.	letter	points	prob.
A	1	0.09	J	8	0.01	S	1	0.04
B	3	0.02	K	5	0.01	T	1	0.06
C	3	0.02	L	1	0.04	U	1	0.04
D	2	0.04	M	3	0.02	V	4	0.02
E	1	0.12	N	1	0.06	W	4	0.02
F	4	0.02	O	1	0.08	X	8	0.01
G	2	0.03	P	3	0.02	Y	4	0.02
H	4	0.02	Q	10	0.01	Z	10	0.01
I	1	0.09	R	1	0.06	□	0	0.02

Let X be the number of points on a randomly chosen Scrabble tile. Then

$$\begin{aligned}
 E(X) &= 1 \cdot 0.09 + 3 \cdot 0.02 + 3 \cdot 0.02 + 2 \cdot 0.04 + 1 \cdot 0.09 + 4 \cdot 0.02 + \dots \\
 &\quad \dots + 8 \cdot 0.01 + 4 \cdot 0.02 + 10 \cdot 0.01 + 0 \cdot 0.02 \\
 &= 1.87.
 \end{aligned}$$

- You spend \$5 on a lottery ticket. Your ticket is one of 10,000 sold. First prize is \$20,000, second prize is \$5,000 and third prize is \$2,000. What is your expected prizemoney? What is your expected profit?

Let P be the random variable representing your prizemoney. The values of P , with their probabilities, are

P , in \$	probability
20,000	0.0001
5,000	0.0001
2,000	0.0001
0	0.9997

We have

$$\begin{aligned}
 E(P) &= 20,000 \cdot 0.0001 + 5,000 \cdot 0.0001 + 2,000 \cdot 0.0001 + 0 \cdot 0.9997 \\
 &= 2 + 0.5 + 0.2 + 0 \\
 &= 2.7.
 \end{aligned}$$

So your expected prizemoney is \$2.70. To find your expected profit, deduct the cost of your ticket. Therefore your expected profit is

$$\$2.70 - \$5.00 = -\$2.30.$$

So your expected loss is \$2.30.

Some simple facts about expectation:

- The expectation of a constant is itself. In other words, if a random variable doesn't actually vary — i.e., it just takes one single constant value, with probability 1 — then that single value is its expectation. Symbolically: if c is a constant then

$$E(c) = c. \quad (10.3)$$

- If we scale a random variable by a constant factor, then its expectation is scaled by the same factor. If X is a random variable and α is a constant, then αX is the random variable formed from X by multiplying each of its values by α . So, for each value k of X , then αX takes value αk with probability $\Pr(X = k)$. In other words,

$$\Pr(\alpha X = \alpha k) = \Pr(X = k).$$

Then the expectation of αX is given by

$$E(\alpha X) = \alpha E(X). \quad (10.4)$$

We often want to add random variables together. We have already seen an example of this: adding the numbers on two dice to form a total, when playing Monopoly. We now show that we can calculate the expectation of a sum of random variables by just working out the expectations of all the random variables separately and then adding them up.

Theorem 49 (Linearity of Expectation). For any random variables X and Y , the expectation of their sum is the sum of their expectations:

$$E(X + Y) = E(X) + E(Y).$$

Proof.

$$\begin{aligned} E(X + Y) &= \sum_k k \cdot \Pr(X + Y = k) \\ &= \sum_k k \sum_{(i,j): i+j=k} \Pr((X = i) \wedge (Y = j)) \\ &= \sum_k \sum_{(i,j): i+j=k} k \cdot \Pr((X = i) \wedge (Y = j)), \end{aligned}$$

with the last step being possible because k is a constant as far as the inner sum is concerned (even though it varies in the outer sum). Now the two summations are both out the front (on the left), and together they just amount to summing over *all* pairs (i, j) , without restriction. Then we can replace k by $i + j$. Therefore

$$E(X + Y) = \sum_{(i,j)} (i + j) \cdot \Pr((X = i) \wedge (Y = j)).$$

So now we are summing over all pairs (i, j) where i is a possible value for X and j is a possible value for Y . We can organise this sum as an outer sum over all possible i and an inner sum over all possible j , where the possibilities for j in the inner sum are unaffected by the choices of i in the outer sum. So we can replace the single summation $\sum_{(i,j)}$ over all pairs by these two nested summations $\sum_i \sum_j$. Or we could put the nested summations the other way round, as in $\sum_j \sum_i$. Right now, we do the former.

$$\begin{aligned}
 E(X+Y) &= \sum_i \sum_j (i+j) \cdot \Pr((X=i) \wedge (Y=j)) \\
 &= \sum_i \sum_j (i \cdot \Pr((X=i) \wedge (Y=j)) + j \cdot \Pr((X=i) \wedge (Y=j))) \quad (10.5) \\
 &\quad \text{(by the distributive law: } (i+j)P = iP + jP, \\
 &\quad \text{where } P \text{ is the probability shown)} \\
 &= \left(\sum_i \sum_j i \cdot \Pr((X=i) \wedge (Y=j)) \right) + \left(\sum_i \sum_j j \cdot \Pr((X=i) \wedge (Y=j)) \right) \\
 &\quad \text{(grouping all the “first summands” of (10.5) together,} \\
 &\quad \text{and likewise with all the “second summands”)} \\
 &= \left(\sum_i \sum_j i \cdot \Pr((X=i) \wedge (Y=j)) \right) + \left(\sum_j \sum_i j \cdot \Pr((X=i) \wedge (Y=j)) \right) \\
 &\quad \text{(reversing the nesting order in the second summation,} \\
 &\quad \text{which we can do because } i \text{ and } j \text{ vary independently).}
 \end{aligned}$$

Now, in the first nested summation, the factor i in the inner sum over j does not depend on j at all, so as far as that inner sum is concerned, it is fixed. So it can be taken outside the inner sum as a fixed common factor. Similarly, in the second nested summation, the factor j in the inner sum over i does not depend on i , so that factor j can be taken outside the inner sum. (Neither of these factors can be taken outside the outer sum, though, as these factors are the variables used in those outer summations, so they each vary as their outer sum is being done.) So we have

$$E(X+Y) = \left(\sum_i i \sum_j \Pr((X=i) \wedge (Y=j)) \right) + \left(\sum_j j \sum_i \Pr((X=i) \wedge (Y=j)) \right) \quad (10.6)$$

But consider the inner sums.

$$\sum_j \Pr((X=i) \wedge (Y=j)) = \Pr(X=i), \quad (10.7)$$

by the law of total probability, (9.22). Here we are using the fact that the events $Y = j$, considered for all j , together cover all possibilities, and are mutually exclusive. Similarly,

$$\sum_i \Pr((X = i) \wedge (Y = j)) = \Pr(Y = j). \quad (10.8)$$

Using (10.7) and (10.8) in (10.6), we have

$$E(X + Y) = \sum_i i \Pr(X = i) + \sum_j j \Pr(Y = j).$$

The first term here is just $E(X)$ and the second term is just $E(Y)$. Therefore

$$E(X + Y) = E(X) + E(Y).$$

□

Linearity of expectation is a very simple rule. It's one of those rare cases where something simple that you *hope* might be true actually *is* true! It is remarkably general, too: it does not even require the two random variables to be independent; you can check that we did not assume independence of X and Y at any stage in the proof. It also turns out to be surprisingly powerful.

Linearity of expectation is another instance of working something out for a complex object by working it out for components of that object and then combining the answers.

We can also ask about the expectation of a product of random variables. This time, we need independence to get a simple rule.

Theorem 50. If two random variables X and Y are independent, then the expectation of their product is the product of their expectations:

$$E(XY) = E(X)E(Y).$$

Proof. By definition of expectation, we have

$$E(XY) = \sum_k k \cdot \Pr(XY = k) \quad (10.9)$$

where the sum is over all possible products of values of X and Y . We can express the event $XY = k$ as a partition into events of the form $(X = x) \wedge (Y = y)$ where $xy = k$, so that $\Pr(XY = k)$ is just the sum of the probabilities of those smaller events:

$$\Pr(XY = k) = \sum_{x,y: xy=k} \Pr((X = x) \wedge (Y = y)),$$

where the sum is over all pairs x, y whose product is k . Substituting this into (10.9), we obtain

$$\begin{aligned}
 E(XY) &= \sum_k k \cdot \sum_{x,y:xy=k} \Pr((X=x) \wedge (Y=y)) \\
 &= \sum_k \sum_{x,y:xy=k} k \Pr((X=x) \wedge (Y=y)) \\
 &\quad \text{(moving } k \text{ inside the inner sum)} \\
 &= \sum_k \sum_{x,y:xy=k} xy \Pr((X=x) \wedge (Y=y)) \\
 &\quad \text{(replacing } k \text{ by } xy \text{ in the inner sum).}
 \end{aligned}$$

Now these two summations, together, just involve summing over all possible pairs x, y , without regard to their sum (since the inner sum is over all pairs with a specific product k , but the outer sum is over all possible values of k). So

$$E(XY) = \sum_{x,y} xy \Pr((X=x) \wedge (Y=y)). \quad (10.10)$$

Since we are assuming that X and Y are independent, we have, for every x and y ,

$$\Pr((X=x) \wedge (Y=y)) = \Pr(X=x) \Pr(Y=y).$$

Substituting this into the right-hand side of (10.10), we have

$$\begin{aligned}
 E(XY) &= \sum_{x,y} xy \Pr(X=x) \Pr(Y=y) \\
 &= \sum_x \sum_y xy \Pr(X=x) \Pr(Y=y) \\
 &\quad \text{(just writing the sum over all pairs } x, y \text{ as a sum over } x \text{ and a sum over } y) \\
 &= \sum_x \sum_y x \Pr(X=x) \cdot y \Pr(Y=y) \\
 &= \sum_x x \Pr(X=x) \sum_y y \Pr(Y=y) \\
 &\quad \text{(since } x \Pr(X=x) \text{ does not depend on } y, \text{ so it can be taken outside the inner sum)} \\
 &= \left(\sum_x x \Pr(X=x) \right) \cdot \left(\sum_y y \Pr(Y=y) \right) \\
 &= E(X) \cdot E(Y).
 \end{aligned}$$

□

10.4 MEDIAN

We mentioned earlier that the expectation gives a single number that represents what we expect from a random variable. Although a random variable is inherently unpredictable, the expectation gives us a rough idea where it tends to sit on the real number line. We also mentioned that we might want a number that is *typical* of the values taken by the random variable, and we noted in passing that the terms “expected value” and “typical value” might be interpreted differently.

The *median* is intended to capture this notion of a “typical” value of a random variable.

The **median** of a random variable X is a real number m such that

$$\begin{aligned}\Pr(X \leq m) &\geq \frac{1}{2}, \\ \Pr(X \geq m) &\geq \frac{1}{2}.\end{aligned}$$

So we can think of m as lying exactly “in the middle of X ” *as far as its probability distribution is concerned*.

In saying this, we are *not* saying that m lies exactly half-way between the smallest and largest values that X might take. So the median must not be confused with the **mid-range** which is defined by

$$\text{mid-range} = \frac{(\text{smallest value}) + (\text{largest value})}{2}.$$

The mid-range is easy to calculate, but it ignores most of the information in X , including the probabilities (except that the minimum and maximum values have nonzero probability). So it is not very useful in general, although it can be a convenient short-cut in some situations where the distribution of X is symmetric (since in such cases the expectation and median both equal the mid-range).

When we say that the median m lies “in the middle of X ”, we intend that this be understood *probabilistically*. The median is the value for which at least half the probability is on one side, and at least half the probability is on the other side.

It is for this reason that the median is often considered to be *typical* of the values of the random variable.

Examples

- Let X be the number obtained from tossing a single die. Then its median lies between 3 and 4, which we can determine from the probabilities or simply from the symmetry of X . It makes sense in this case to define the median to be 3.5. In general, when the median falls naturally in a gap, its value can be chosen to be the middle of that gap. There are also more detailed ways of calculating the median in such cases, but we do not consider them here.

- Let X be the number of points on a random Scrabble letter. The median of X is 1, since

$$\Pr(X = 0) = 0.02,$$

$$\Pr(X = 1) = 0.68,$$

$$\Pr(X \geq 2) = 0.3,$$

so $\Pr(X \leq 1) = 0.7 \geq 0.5$ and $\Pr(X \geq 1) = 0.98 \geq 0.5$, satisfying the definition. You should satisfy yourself that no other median value works in this case.

- Let P be the lottery prizemoney random variable defined in the example on p. 353. Its median is 0, since $\Pr(P \geq 0) = 1 \geq 0.5$ and $\Pr(P \leq 0) = 0.9997 \geq 0.5$.

The expectation and median each have their pros and cons.

- The median is less vulnerable than the expectation to changes to the extreme values of the random variable. If the lowest (or highest) value changes a bit, then the median is usually unaffected. This is different to the expectation, which can be affected by a small change to any value of the random variable.
- The median does not mix with arithmetic operations as well as the expectation. For example, the median of a sum of two random variables does not, in general, equal the sum of their medians. So there is no analogue of Linearity of Expectation to help simplify calculations. This means that the expectation is usually preferred for random variables that might be used in arithmetical calculations.
- The expectation can be quite different to most actual values of the random variable. For example, consider a random variable which can take any of the five values 1, 2, 3, 4, 90, with probability $\frac{1}{5}$ each. Then its expectation is

$$1 \cdot \frac{1}{5} + 2 \cdot \frac{1}{5} + 3 \cdot \frac{1}{5} + 4 \cdot \frac{1}{5} + 90 \cdot \frac{1}{5} = \frac{100}{5} = 20.$$

But its median is 3. Clearly, in this case, the median is more like the other values, and therefore more “typical” of them, than the expectation, which is a long way from any of the values.

Both the expectation and the median of a random variable X give a numerical indicator of where X may be located on the number line. For this reason, they can each be described as a *measure of location*. The mid-range is another measure of location. But none of these measures says anything about how widely spread, or not, the values of X are. This is often important to know, so we need a new measure to try to capture it.

10.5 MODE

Our last measure of location is the *mode*. We only consider it briefly, but it is important terminology to know.

The **mode** of a random variable X is the value g for which $\Pr(X = g)$ is greatest. This may not be unique.

The mode may be described as the most frequent, or most popular, value of X . This is useful, since there are times when we just want to know what the most likely value is.

The mode's probability gives us an upper bound on the probability of every other value of X . But it does not say anything else about how the distribution behaves at other values. It is quite possible for the mode to be a long way from the mean and/or the median. So it may not be a good representative of the random variable as a whole. Nonetheless, for many important well-behaved distributions, the mode is not too far from the mean and median.

10.6 VARIANCE

In many situations, it is important to have some measure of how much a random variable varies. The values of the variable might be tightly concentrated or loosely smeared out. The expectation or median tell us nothing about this variability. So we now introduce a measure of variability, the *variance*, and its close relative, the *standard deviation*.

The *variance* of a random variable X measures how far its values tend to be, on average, from its expected value. In measuring this, it gives greater weight to values that are further from the expected value. It does this by using the *square* of the difference from the expected value, and taking the expectation of this squared difference. We now define it formally.

Let $\mu = E(X)$ be the expected value of X . Then the **variance** $\text{Var}(X)$ is defined by

$$\text{Var}(X) = E((X - \mu)^2). \quad (10.11)$$

This has many important properties, some of which we will come to soon. But we have to keep in mind that, if the values of X have units on some scale (e.g., if X arises from a physical measurement), then $\text{Var}(X)$ does not have the same units as X ; rather, its units are the *square* of the units of X , since it is an expectation of *squares*. For example, if X is a distance in *metres*, then the units of $\text{Var}(X)$ are *square metres*, which is a unit of area rather than length. So, although the variance does indicate how widely X varies around μ — with a larger variance indicating wider variation — it does not do so *on the same scale*. In order to get a measure of variation that is on the same numerical scale as X and has the same units, we can use the **standard deviation**, which is defined by

$$\text{standard deviation of } X = \sqrt{\text{Var}(X)}.$$

The variance and standard deviation go hand-in-hand. Whenever one of them is used, it is likely that the other one will be used at some stage too. We are not asked to choose between them! They are both indispensable.

Examples

- Let X be the number given by the throwing a single die. We know that $\mu = E(X) = 3.5$ (see p. 352). So

$$\begin{aligned}
 \text{Var}(X) &= E((X - \mu)^2) \\
 &= E((X - 3.5)^2) \\
 &= \sum_{k=1}^6 (k - 3.5)^2 \cdot \Pr(X = k) \\
 &= (1 - 3.5)^2 \cdot \Pr(X = 1) + (2 - 3.5)^2 \cdot \Pr(X = 2) + (3 - 3.5)^2 \cdot \Pr(X = 3) + \\
 &\quad (4 - 3.5)^2 \cdot \Pr(X = 4) + (5 - 3.5)^2 \cdot \Pr(X = 5) + (6 - 3.5)^2 \cdot \Pr(X = 6) \\
 &= (-2.5)^2 \cdot \frac{1}{6} + (-1.5)^2 \cdot \frac{1}{6} + (-0.5)^2 \cdot \frac{1}{6} + 0.5^2 \cdot \frac{1}{6} + 1.5^2 \cdot \frac{1}{6} + 2.5^2 \cdot \frac{1}{6} \\
 &= 6.25 \cdot \frac{1}{6} + 2.25 \cdot \frac{1}{6} + 0.25 \cdot \frac{1}{6} + 0.25 \cdot \frac{1}{6} + 2.25 \cdot \frac{1}{6} + 6.25 \cdot \frac{1}{6} \\
 &= \frac{6.25 + 2.25 + 0.25 + 0.25 + 2.25 + 6.25}{6} \\
 &= \frac{17.5}{6} \\
 &= 2\frac{11}{12} \\
 &\approx 2.92.
 \end{aligned}$$

Therefore the standard deviation is

$$(\text{Var}(X))^{1/2} \approx 1.71.$$

Another way to calculate the variance is given by the following theorem.

Theorem 51. Let X be a random variable with $\mu = E(X)$. Then

$$\text{Var}(X) = E(X^2) - \mu^2.$$

This theorem may be written in the visually neat form,

$$\text{Var}(X) = E(X^2) - E(X)^2.$$

Proof. The variance is just the expectation of $(X - \mu)^2$. Let's expand $(X - \mu)^2$ before taking its expectation. Observe that

$$(X - \mu)^2 = X^2 - 2X\mu + \mu^2. \quad (10.12)$$

So we can work out the variance, $E((X-\mu)^2)$, by taking the expectation of the expanded form on the right of (10.12). When doing that, we can use [Theorem 49](#) (Linearity of Expectation).

$$\begin{aligned}
 \text{Var}(X) &= E((X-\mu)^2) \\
 &= E(X^2 - 2X\mu + \mu^2) && \text{(by (10.12))} \\
 &= E(X^2) - E(2X\mu) + E(\mu^2) && \text{(by Linearity of Expectation)} \\
 &= E(X^2) - 2\mu E(X) + \mu^2 && \text{(by (10.4) and (10.3))} \\
 &= E(X^2) - 2\mu^2 + \mu^2 && \text{(since } \mu = E(X)) \\
 &= E(X^2) - \mu^2.
 \end{aligned}$$

□

You can use either way of calculating the variance — the definition (10.11), or [Theorem 51](#) — according to your preference, or whichever is easier for the data you have.

You may wonder why, when defining the variance, we use the average *squared* difference rather than just the differences themselves. What would happen if we just took the expectation of the difference $X - \mu$, rather than the expectation of $(X - \mu)^2$? By Linearity of Expectation and (10.3), we have

$$E(X - \mu) = E(X) - E(\mu) = E(X) - \mu = \mu - \mu = 0.$$

So this tells us nothing! The problem with this approach was that the total positive and negative differences end up cancelling each other out. We could, instead, take the expectation of the *absolute* difference, which is called the **mean absolute deviation**:

$$\text{mean absolute deviation} = E(|X - \mu|).$$

In fact, this is occasionally used, and it can be treated as being on the same scale as X , with the same units. But its mathematical properties are not as strong as the variance or standard deviation, so we do not consider it further.

Theorem 52. If two random variables X and Y are independent, then the variance of their sum is the sum of their variances:

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y).$$

Proof.

$$\begin{aligned}
 \text{Var}(X+Y) &= E((X+Y)^2) - E(X+Y)^2 \\
 &= E(X^2 + 2XY + Y^2) - (E(X) + E(Y))^2 \\
 &= E(X^2) + 2E(XY) + E(Y^2) - (E(X)^2 + 2E(X)E(Y) + E(Y)^2) \\
 &= E(X^2) + 2E(XY) + E(Y^2) - E(X)^2 - 2E(X)E(Y) - E(Y)^2 \\
 &= E(X^2) - E(X)^2 + E(Y^2) - E(Y)^2 + 2(E(XY) - E(X)E(Y)) \\
 &= \text{Var}(X) + \text{Var}(Y) + 2(E(XY) - E(X)E(Y)).
 \end{aligned}$$

If X and Y are independent, then $E(XY) = E(X)E(Y)$, by [Theorem 50](#). So the final term above, $2(E(XY) - E(X)E(Y))$, is zero. Therefore

$$\text{Var}(X+Y) = \text{Var}(X) + \text{Var}(Y).$$

□

Comments:

- The variance is easier to work with, when adding random variables, than the standard deviation, for which we have

$$\text{StdDev}(X+Y) = \sqrt{(\text{StdDev}(X))^2 + (\text{StdDev}(Y))^2}.$$

- The quantity $E(XY) - E(X)E(Y)$, used near the end of the above proof, is important in its own right. It is the **covariance** of the random variables X and Y , denoted by $\text{Cov}(X, Y)$:

$$\text{Cov}(X, Y) = E(XY) - E(X)E(Y).$$

This is zero when X and Y are independent, and can be zero in some other situations too. If it is nonzero then it indicates some kind of linear relationship between the two random variables, though not usually an exact one; the relationship may be approximate and probabilistic.

Since the standard deviation measures how far X tends to be from its mean μ , we would expect that X is unlikely to be too many standard deviations away from μ , and that the further away from μ we go, the less likely X is to appear there.

This intuition can be made precise. We use t for how far away from the mean we want to go (in terms of numbers of standard deviations). We will put an upper bound on the probability of being at least that far away.

Theorem 53 (Chebyshev's Inequality). For any random variable X with expectation μ and variance σ^2 , and any $t \in \mathbb{R}^+$, the probability that X is at least t standard deviations away from its mean is at most $1/t^2$:

$$\Pr(|X - \mu| \geq t\sigma) \leq \frac{1}{t^2}.$$

Proof. The variance of X is $E((X - \mu)^2)$. This expectation is a sum over all values of X , but we'll compare that with what we get by only taking values that are far enough away from the mean.

$$\begin{aligned} \text{Var}(X) &= E((X - \mu)^2) \\ &= \sum_k (k - \mu)^2 \Pr(X = k) \\ &\geq \sum_{k: |k - \mu| \geq t\sigma} (k - \mu)^2 \Pr(X = k) \\ &\quad \text{(we now only sum over those } k \text{ that are } \geq t \text{ standard deviations} \\ &\quad \text{away from the mean)} \\ &\geq \sum_{k: |k - \mu| \geq t\sigma} (t\sigma)^2 \Pr(X = k) \\ &\quad \text{(since, if } |k - \mu| \geq t\sigma, \text{ then } (k - \mu)^2 \geq (t\sigma)^2) \\ &= (t\sigma)^2 \sum_{k: |k - \mu| \geq t\sigma} \Pr(X = k) \\ &\quad \text{(since } (t\sigma)^2 \text{ does not depend on } k, \text{ so can be taken outside the sum)} \\ &= (t\sigma)^2 \Pr(|X - \mu| \geq t\sigma). \end{aligned}$$

Therefore, since $\text{Var}(X) = \sigma^2$,

$$\sigma^2 \geq (t\sigma)^2 \Pr(|X - \mu| \geq t\sigma).$$

Therefore

$$\Pr(|X - \mu| \geq t\sigma) \leq \frac{1}{t^2}.$$

□

So, the larger $t\sigma$ is, the less likely it is that X is that far away from the mean, and the theorem gives an upper bound on the probability of being that far away.

This theorem's main virtue is its generality. It applies to *any* random variable at all. In practice, when we have a random variable with a specific well-behaved probability distribution, stronger statements may be possible; there may be smaller bounds on the probability of being a given distance from the mean. But, if we don't know much about how a given random variable is distributed, or if it's hard to analyse it, then we can

always fall back on Chebyshev's Inequality.

Having introduced many of the basic concepts pertaining to random variables, especially expectation and variance, we now look at some important and useful random variables and their associated probability distributions.

10.7 UNIFORM DISTRIBUTION

Let a and b be integers with $a \leq b$. The **uniform distribution** gives the same probability to all integers between a and b inclusive, and zero probability to all other integers. The number of integers in the interval $[a, b]$ is $b - (a - 1) = b - a + 1$, so they each get probability $1/(b - a + 1)$. All other integers have probability zero.

If a random variable x has the uniform distribution as its probability distribution, we say it is **uniformly distributed**, and its probabilities are given by

$$\Pr(X = x) = \begin{cases} \frac{1}{b - a + 1}, & \text{if } a \leq x \leq b; \\ 0, & \text{otherwise.} \end{cases}$$

We can write $X \sim \text{Unif}_{\mathbb{Z}}(a, b)$ to mean that the random variable X is uniformly distributed over the integer interval $[a, b] \cap \mathbb{Z}$. The subscript $\mathbb{Z}$ may be omitted if it is clear from the context that X can only take integer values in the interval $[a, b]$.

A plot of the discrete uniform distribution, for $a = 2$ and $b = 6$, is shown in [Figure 10.1](#).

Here are some examples of uniformly distributed random variables, some of which we have seen before.

- Let X be the outcome of the toss of a fair coin, with outcomes encoded as 0 and 1 for Tails and Heads respectively. Then $X \sim \text{Unif}(0, 1)$.
- Let X be the number shown on a fair die after it is thrown. Then $X \sim \text{Unif}(1, 6)$.
- Let X be the age of an adult student who has not reached the minimum age for a full Victorian Driver's Licence. This means that their age could be 18, 19, 20 or 21 (since the minimum age for a full licence is 22). In the absence of any other information about the student or their driving history, we might use $X \sim \text{Unif}(18, 21)$.

Sometimes, we use a uniform distribution because we have reason to believe that all possible values are indeed equally likely, as for a fair die or a fair coin.

We also use a uniform distribution to model ignorance or uncertainty, as we did in the third example above. Suppose we know that an integer-valued random variable always takes values in $[a, b]$, but *we know nothing more about it*. So we don't know where its most popular or least popular values are; we don't know whether it tends to lie closer to a or closer to b or somewhere in the middle; we don't know whether its

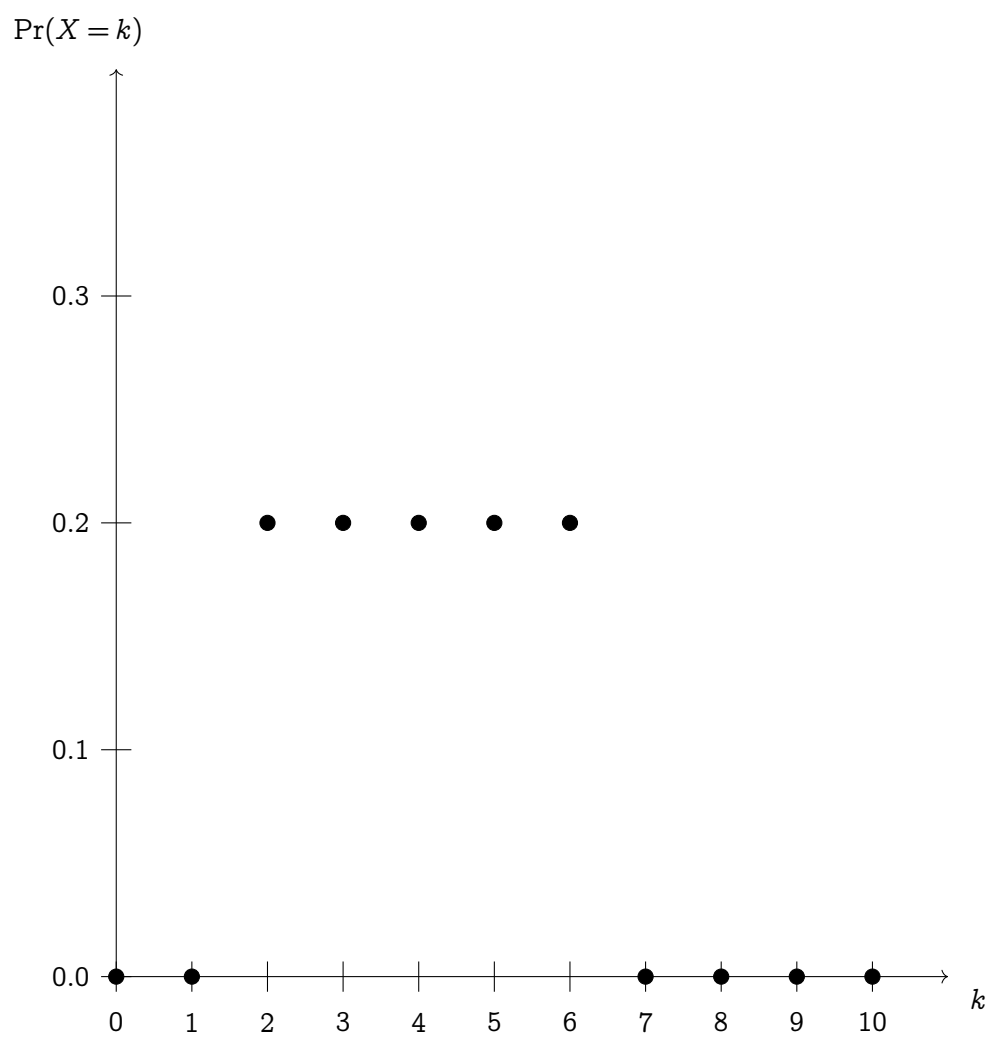


Figure 10.1: The discrete uniform distribution $\text{Unif}(2, 6)$.

distribution is symmetric or not; we don't know if the probabilities vary smoothly or jump around wildly as we move along the interval from a to b ; we don't know how many "peaks" or "troughs" a graph of its probabilities may have; in fact, we don't know anything at all beyond the fact that all its values lie within that interval. Then the uniform distribution is the natural one to use. It is the distribution that best captures our prior ignorance or uncertainty, since it makes no assumptions about any value of X being more or less likely than any other value.²

The expectation of a random variable $X \sim \text{Unif}(a, b)$ must be exactly in the middle of the interval, by symmetry:

$$E(X) = \frac{a+b}{2}. \quad (10.13)$$

You can also confirm this by working from the definition and doing the algebra.

For the case of a single throw of a die, we have $a = 1$ and $b = 6$, and on p. 352 we worked out the expectation to be $(a+b)/2 = 3.5$.

The median is also $(a+b)/2$, by symmetry.

The variance needs more work. The simplest approach is probably to work out $E(X^2)$, using the definition of expectation, and then deduct $E(X)^2$ (with $E(X)$ given by (10.13) above), and use Theorem 51. You may also need Exercise 3.8. You should obtain

$$\text{Var}(X) = \frac{(b-a+1)^2 - 1}{12}.$$

For the throw of a die, this formula gives $((b-a+1)^2 - 1)/12 = ((6-1+1)^2 - 1)/12 = 35/12 = 2\frac{11}{12} \approx 2.92$. This agrees with our computation on p. 361.

10.8 BINOMIAL DISTRIBUTION

A **Bernoulli trial** is a random experiment with two possible outcomes, **success** and **failure**. The probabilities of these outcomes are denoted by p and $q = 1 - p$ respectively:

$$\begin{aligned} p &= \text{Pr}(\text{success}), \\ q &= \text{Pr}(\text{failure}) = 1 - p. \end{aligned}$$

This very simple random experiment can be used to model a huge variety of situations. The outcomes "success" and "failure" can be renamed according to the needs of the situation; the names themselves are just conventional, and are not an essential feature of the model. We could, instead, call the outcomes "yes" and "no", or "on" and "off", or 1

² The notion of *uncertainty* of a random variable can be quantified, using the concept of *entropy*. You should learn about entropy later in your computer science studies. It is a foundational concept in *information theory* and was first introduced by Claude Shannon in coding theory, where data is encoded for sending over a communications channel, and the encoding scheme used must be as efficient as possible while protecting against some level of random "noise" on the channel. It is also used in cryptography, data compression, and machine learning. One of the basic theorems about entropy, as a measure of uncertainty, is that the entropy of a distribution over an integer interval is maximised by the uniform distribution on that interval.

and 0. Coin tosses can be regarded as Bernoulli trials in which the outcomes are Heads and Tails. Mostly, our coins have been fair, with $p = q = \frac{1}{2}$, but biased coins can also be modelled, using $p \neq \frac{1}{2}$ (see, e.g., p. 352). In our network reliability example on p. 330 in § 9.7, each edge was a Bernoulli trial in which the outcomes represent the survival or failure of links in a network, with link outcomes being independent and the probability of survival being p for all links.

When we refer to some number or sequence of Bernoulli trials, they are understood to be *independent* and *identically distributed*. So, the outcome of any one of them is independent of all previous trials, and each trial uses the same success probability p .

A Bernoulli trial can be described by a $\{0, 1\}$ -valued random variable:

$$X = \begin{cases} 1, & \text{with probability } p; \\ 0, & \text{with probability } 1 - p. \end{cases}$$

This is about the simplest random variable that can be defined that has any randomness at all. We can work out its mean and variance:

$$\begin{aligned} E(X) &= 0 \cdot \Pr(X=0) + 1 \cdot \Pr(X=1) = 0 \cdot (1-p) + 1 \cdot p = p, & (10.14) \\ \text{Var}(X) &= E(X^2) - E(X)^2 & (\text{by Theorem 51}) \\ &= E(X^2) - p^2 & (\text{using } E(X) \text{ from (10.14)}) \\ &= (0^2 \cdot \Pr(X=0) + 1^2 \cdot \Pr(X=1)) - p^2 \\ &= (0 \cdot \Pr(X=0) + 1 \cdot \Pr(X=1)) - p^2 \\ &= p - p^2 \\ &= p(1-p). & (10.15) \end{aligned}$$

Suppose we have n Bernoulli trials with success probability p . How many successes are there? This is a random variable Z whose set of possible values is $\{0, 1, 2, \dots, n-1, n\}$. If k is a value in this set, what is the probability that we have k successes (and therefore $n-k$ failures)?

Each success has probability p and each failure has probability $1-p$. Since the trials are independent, a given sequence of outcomes must have probability

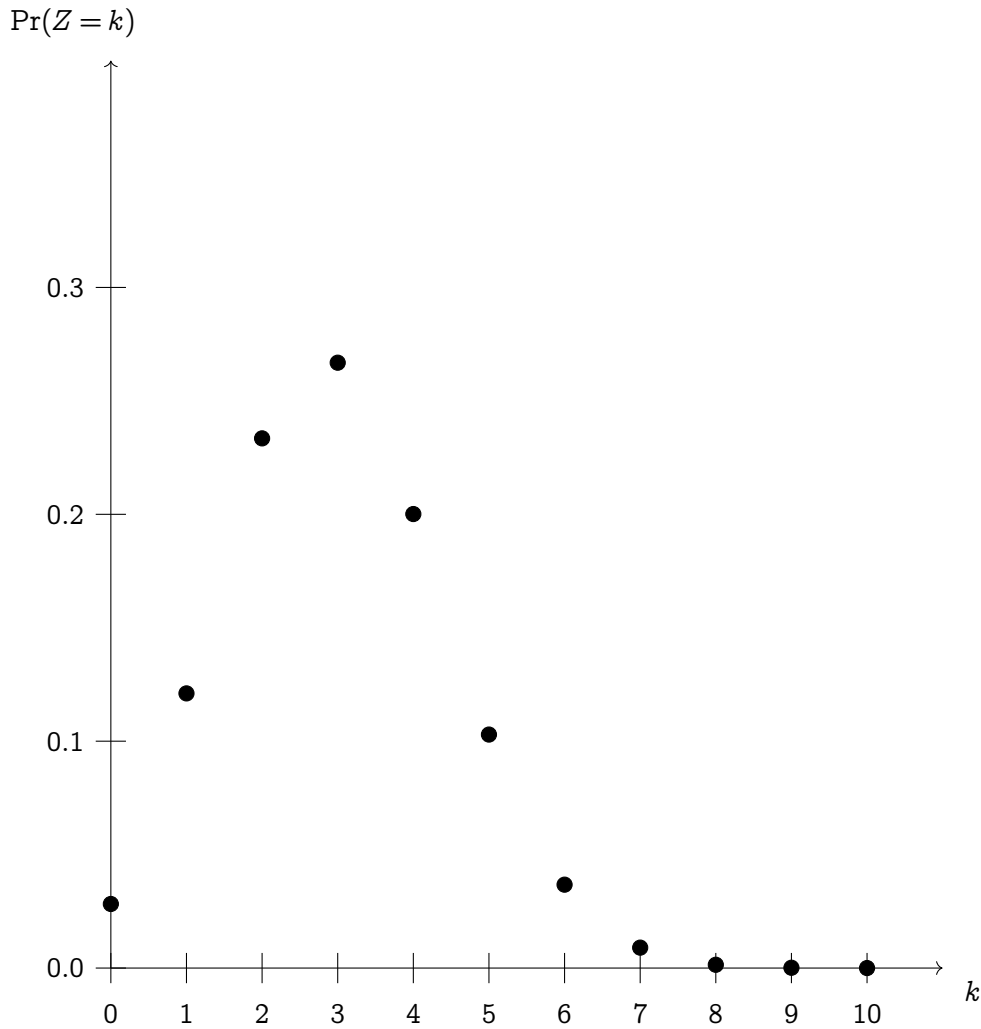
$$p^{\# \text{ successes}} (1-p)^{\# \text{ failures}}.$$

So a sequence with k successes and $n-k$ failures has probability

$$p^k (1-p)^{n-k}.$$

The number of sequences of n outcomes consisting of k successes and $n-k$ failures is just the number of ways of choosing which k of the trials are the successes. This is $\binom{n}{k}$. Therefore

$$\Pr(Z=k) = \binom{n}{k} p^k (1-p)^{n-k}. \quad (10.16)$$

Figure 10.2: The binomial distribution $\text{Bin}(10, 0.3)$.

This probability distribution is known as the **binomial distribution**. We can say that the random variable Z has the binomial distribution based on n Bernoulli trials and success probability p by writing

$$Z \sim \text{Bin}(n, p).$$

A plot of the binomial distribution, for $n = 10$ and $p = 0.3$, is shown in [Figure 10.2](#). Note that, for the highest values of k (i.e., $k = 9, 10$), the probabilities are nonzero even though the points appear to be on the horizontal axis.

For example, suppose we want to know the probability that we get exactly six successes from $n = 10$ Bernoulli trials each with success probability $p = 0.3$. This is the

same scenario as for the plot in [Figure 10.2](#), where we now seek the distribution's value, $\Pr(Z = 6)$, when $k = 6$. Using [\(10.16\)](#), we have

$$\Pr(Z = 6) = \binom{10}{6} 0.3^6 (1 - 0.3)^{10-6} = 210 \cdot 0.3^6 \cdot 0.7^4 \approx 0.037.$$

We now consider the expectation of a binomially distributed random variable, $Z \sim \text{Bin}(n, p)$.

This can be calculated directly using the definition of expectation [\(10.2\)](#) with the binomial probability [\(10.16\)](#). This takes some time, effort and care, and is good practice in doing the required algebra and manipulating summations. But there is a *much* easier way, using Linearity of Expectation ([Theorem 49](#)).

The binomial random variable Z is the number of successes in n Bernoulli trials, each with probability p . For each $i \in \{1, 2, \dots, n\}$, define the $\{0, 1\}$ -valued random variable X_i to be the outcome of the i -th Bernoulli trial, where the outcome is written as 1 for success and 0 for failure. Since X_i counts the number of successes in just the i -th trial, adding them up over all i gives the total number of successes over n trials:

$$Z = X_1 + X_2 + \dots + X_n.$$

So the expectation of Z is given by

$$\begin{aligned} E(Z) &= E(X_1 + X_2 + \dots + X_n) \\ &= E(X_1) + E(X_2) + \dots + E(X_n) && \text{(by Linearity of Expectation)} \\ &= \underbrace{p + p + \dots + p}_{n \text{ copies}} && \text{(by (10.14))} \\ &= np. \end{aligned}$$

Since Bernoulli trials are independent, we can compute the variance by adding up the variances of all the X_i (by [Theorem 52](#)):

$$\begin{aligned} \text{Var}(Z) &= \text{Var}(X_1 + X_2 + \dots + X_n) \\ &= \text{Var}(X_1) + \text{Var}(X_2) + \dots + \text{Var}(X_n) && \text{(by Theorem 52)} \\ &= np(1 - p). \end{aligned}$$

Therefore the standard deviation of Z is $\sqrt{np(1 - p)}$.

10.9 POISSON DISTRIBUTION

The binomial distribution is often used when the number n of trials is very large. We should be on the lookout for efficient approximations when working with large numbers.

Fortunately, there are two very useful approximations to the binomial distribution that can be used in two types of situations with large n (although they do not cover all possible scenarios). We consider the first of these in this section.

Let X be a random variable that can take any nonnegative integer value. We say that X has **Poisson distribution with parameter μ** if, for all $k \in \mathbb{N}_0$,

$$\Pr(X = k) = \frac{e^{-\mu} \mu^k}{k!}. \quad (10.17)$$

We can write $X \sim \text{Poisson}(\mu)$ to mean that X has Poisson distribution with parameter μ . In (10.17), e is the base of natural logarithms, as usual.

We should first check that this is a valid probability distribution. The values are nonnegative, but do they add up to 1? We can use the infinite series for e^x :

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^i}{i!} + \cdots = \sum_{i=0}^{\infty} \frac{x^i}{i!}, \quad (10.18)$$

for all $x \in \mathbb{R}$. The total of all the probabilities in the Poisson distribution is

$$\begin{aligned} \sum_{k=0}^{\infty} \Pr(X = k) &= \frac{e^{-\mu} \mu^0}{0!} + \frac{e^{-\mu} \mu^1}{1!} + \frac{e^{-\mu} \mu^2}{2!} + \frac{e^{-\mu} \mu^3}{3!} + \cdots \\ &= e^{-\mu} \left(\frac{\mu^0}{0!} + \frac{\mu^1}{1!} + \frac{\mu^2}{2!} + \frac{\mu^3}{3!} + \cdots \right) \\ &= e^{-\mu} \left(1 + \mu + \frac{\mu^2}{2!} + \frac{\mu^3}{3!} + \cdots \right) \\ &= e^{-\mu} \cdot e^{\mu} && \text{(by (10.18))} \\ &= 1, \end{aligned}$$

so we can indeed call it a probability distribution.

A plot of the Poisson distribution, for $\mu = 3.0$, is shown in Figure 10.3. Again, the higher values of k give very small positive values.

For a numerical example, suppose we seek the probability that a Poisson random variable with parameter $\mu = 3$ (as in the plot) has value ≤ 2 . Then

$$\begin{aligned} \Pr(X \leq 2) &= \Pr(X = 0) + \Pr(X = 1) + \Pr(X = 2) \\ &= \frac{e^{-3} 3^0}{0!} + \frac{e^{-3} 3^1}{1!} + \frac{e^{-3} 3^2}{2!} \\ &= e^{-3} + e^{-3} \cdot 3 + e^{-3} \cdot \frac{9}{2} \\ &\approx 0.04980 + 0.14936 + 0.22404 \\ &\approx 0.423 \end{aligned}$$

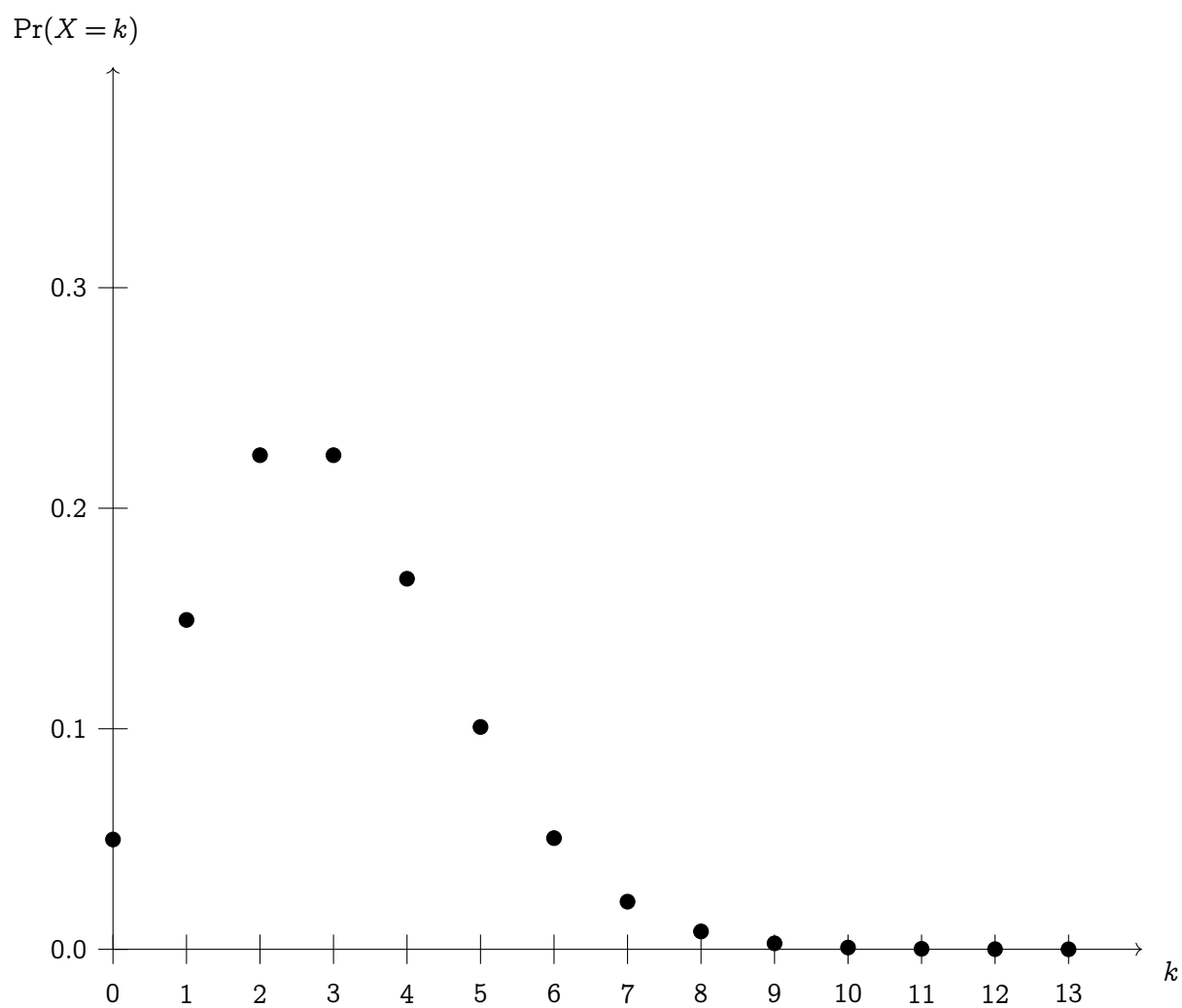


Figure 10.3: The Poisson distribution $\text{Poisson}(3)$.

Let's now give the expectation and variance of the Poisson distribution in general.

Theorem 54. If X is a Poisson random variable with parameter μ ,

$$\begin{aligned} E(X) &= \mu, \\ \text{Var}(X) &= \mu, \\ \text{StdDev}(X) &= \sqrt{\mu}. \end{aligned}$$

Proof.

$$\begin{aligned} E(X) &= \sum_{k=0}^{\infty} k \Pr(X = k) \\ &= \sum_{k=0}^{\infty} k \frac{e^{-\mu} \mu^k}{k!} \\ &= e^{-\mu} \sum_{k=1}^{\infty} k \frac{\mu^k}{k!} \\ &\quad \text{(since } e^{-\mu} \text{ does not depend on } k, \text{ so it can be taken outside the sum;} \\ &\quad \text{we have also started the sum at } k=1, \text{ since for } k=0 \text{ we have } k\mu^k/k! = 0) \\ &= e^{-\mu} \sum_{k=1}^{\infty} \frac{\mu^k}{(k-1)!} \\ &= e^{-\mu} \cdot \mu \sum_{k=1}^{\infty} \frac{\mu^{k-1}}{(k-1)!} && \text{(taking one factor } \mu \text{ outside the sum)} \\ &= e^{-\mu} \cdot \mu \sum_{i=0}^{\infty} \frac{\mu^i}{i!} && \text{(writing the sum in terms of } i = k-1 \text{ rather than } k) \\ &= e^{-\mu} \cdot \mu \cdot e^{\mu} \\ &= \mu. \end{aligned}$$

A similar argument can be used to show that $\text{Var}(X) = \mu$. We leave that as an exercise. It then follows that the standard deviation is $\sqrt{\mu}$. $\square$

The symbol μ for the Poisson parameter was chosen as a reminder that it is actually the expectation, and also the variance, of the distribution.

The Poisson distribution arises in situations where

- a random variable could conceivably take any nonnegative integer value (or it has a finite upper bound which is very large), *and*
- its value is counting the number of occurrences of some event, *and*
- occurrences of these events do not affect each other.

Many Poisson random variables are based on counting how many times an event happens within some time interval, in situations where these events are independent of each other and can happen at any point in time. The parameter μ depends on the situation, including the length of the time interval.

Here are some examples of random variables with Poisson distributions.

- Telephone calls: the number of calls received (by a phone, call centre or exchange) during a given time interval.
- Website visits: the number of visits to a website within a given time interval.
- Busking: the number of coins tossed into a busker's cap during a given time interval.
- Radioactive decay. Suppose you have a quantity of radioactive material, the atoms of which emit *alpha particles* (which are essentially Helium nuclei, and consist of two protons and two neutrons). The emissions by the atoms are independent of each other. The number of alpha particles emitted by a given quantity of material in a given period of time follows a Poisson distribution. The parameter μ depends on the particular element (and isotope) as well as the quantity of material and the length of the time interval during which observations are made.

One important application of the Poisson distribution is as an approximation to the Binomial distribution. If n is large and the Binomial mean np is comparatively small, then the Poisson distribution with parameter $\mu = np$ is usually a good approximation to $\text{Bin}(n, p)$.

10.10 GEOMETRIC DISTRIBUTION

In the binomial distribution (§ 10.8), we count the number of successes in a finite sequence of Bernoulli trials. We now consider the *geometric distribution*, where we count the number of trials needed to get the *first* success, however long that may take.

Suppose we have a sequence of Bernoulli trials, each with success probability p . Let $k \in \mathbb{N}$, so we are supposing here that the sequence of trials can be arbitrarily long. What is the probability that the k -th trial is the first successful one? For this to happen,

- the first $k - 1$ trials must result in failure, which has probability $(1 - p)^{k-1}$, since the trials are independent; *and*
- the k -th trial must result in success, which has probability p .

We multiply these together (again using independence) to obtain

$$\Pr(\text{first success is at } k\text{-th trial}) = (1 - p)^{k-1}p.$$

This motivates the following definitions.

The **geometric distribution** assigns, to $k \in \mathbb{N}$, the probability $(1-p)^{k-1}p$. We give expressions for the first few values in the following table.

k	1	2	3	4	
probability	p	$(1-p)p$	$(1-p)^2p$	$(1-p)^3p$	

As usual, we check that these are indeed probabilities. They are clearly nonnegative, since $0 \leq p \leq 1$. So we now check that they sum to 1. Adding the probabilities gives an infinite series

$$p + (1-p)p + (1-p)^2p + (1-p)^3p + \cdots.$$

This is an infinite *geometric* series with first term $a = p$ and common ratio $r = 1-p$, so its sum is

$$\frac{a}{1-r} = \frac{p}{1-(1-p)} = \frac{p}{1-1+p} = \frac{p}{p} = 1,$$

as required. So they are indeed probabilities.

Let X be a random variable taking positive integer values. X is **geometrically distributed** if, for some $p \in [0, 1]$ and every $k \in \mathbb{N}$,

$$\Pr(X = k) = (1-p)^{k-1}p.$$

We write $X \sim \text{Geom}(p)$ to mean that X is geometrically distributed with parameter p .

Note that, although our geometric random variable represents the number of trials *until* the first success (so the first successful trial is included in the count), other sources you read may use the term instead for the number of failures *before* the first success. It does not matter very much, as one of these random variables is just a shift, by one, of the other. But it is important to be aware of this.

A plot of the geometric distribution, for $p = 0.3$, is shown in [Figure 10.4](#).

For an example, suppose we have $p = 0.3$, as in the plot, and we seek the probability that $X = 6$ (so $k = 6$).

$$\Pr(X = 6) = (1-0.3)^{6-1} \cdot 0.3 = 0.7^5 \cdot 0.3 \approx 0.050.$$

We now give the expectation and variance for the general case.

Theorem 55. If $X \sim \text{Geom}(p)$ then

$$\begin{aligned} E(X) &= \frac{1}{p}, \\ \text{Var}(X) &= \frac{1-p}{p^2}. \end{aligned}$$

Proof. (outline)

$$E(X) = \sum_{k=1}^{\infty} k \Pr(X = k) = \sum_{k=1}^{\infty} k(1-p)^{k-1}p = p \sum_{k=1}^{\infty} k(1-p)^{k-1}, \quad (10.19)$$

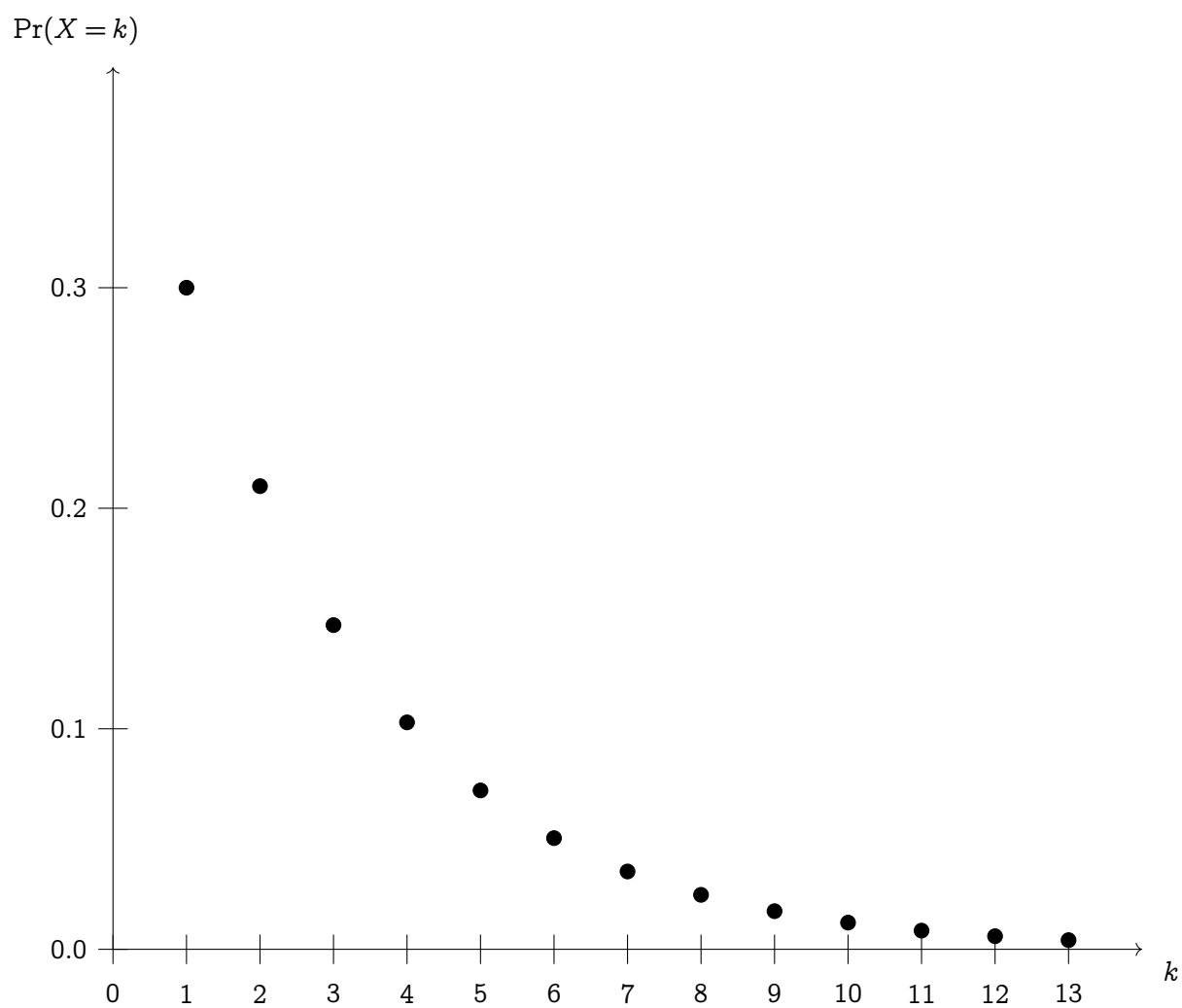


Figure 10.4: The geometric distribution $\text{Geom}(0.3)$.

since the factor p , inside the sum, does not depend on k , so can be taken outside the sum.

The sum here is not a type of sum we have studied before. If the terms being added had no coefficient k , then we would have an infinite geometric series. If, instead, the exponent $k - 1$ were removed, we would have an infinite arithmetic series. But, as it stands, this infinite sum is not of either of those types.

Nonetheless, there is something familiar about it. Study the expression inside the summation in (10.19). Where have you seen an expression of the form

$$kx^{k-1}$$

before?

See if you can remember, before going to the next page!

Hopefully, the last expression on the previous page rang a bell.

The expression kx^{k-1} is the derivative³ of x^k , with respect to x :

$$\frac{d}{dx}x^k = kx^{k-1}.$$

Here we have $x = 1 - p$; if we differentiate $(1 - p)^k$ with respect to p , then we must introduce a minus sign, by the Chain Rule:

$$\frac{d}{dp}(1 - p)^k = -k(1 - p)^{k-1}, \quad \text{therefore} \quad k(1 - p)^{k-1} = -\frac{d}{dp}(1 - p)^k.$$

Substituting this back into (10.19), we have

$$\begin{aligned} E(X) &= p \sum_{k=1}^{\infty} -\frac{d}{dp}(1 - p)^k \\ &= -p \frac{d}{dp} \left(\sum_{k=1}^{\infty} (1 - p)^k \right) \\ &\quad \text{(since a sum of derivatives is the derivative of the sum).} \end{aligned}$$

This sum is now the sum of a geometric series with first term $a = 1 - p$ and common ratio $r = 1 - p$. So this sum is given by

$$\frac{a}{1 - r} = \frac{1 - p}{1 - (1 - p)} = \frac{1 - p}{p} = \frac{1}{p} - 1.$$

Therefore

$$\begin{aligned} E(X) &= -p \frac{d}{dp} \left(\frac{1}{p} - 1 \right) \\ &= -p \left(\frac{-1}{p^2} \right) \\ &= \frac{1}{p}. \end{aligned}$$

This completes our derivation of the expectation of the geometric distribution.

We omit the derivation of the variance, which follows similar lines but is more detailed. □

The geometric distribution has the important property of being *memoryless*, which we now explain.

³ It may be surprising to encounter calculus here, since our focus is on *discrete* probability. But mathematics is not obliged to respect the walls that humans find it convenient to erect between its parts! It is common for tools from continuous mathematics, such as calculus, to be applied to the study of discrete probability and other parts of discrete mathematics. In fact, the traffic goes both ways.

Suppose you conduct a series of Bernoulli trials, with success probability p , and let X be the number of trials until the first success. Then $X \sim \text{Geom}(p)$. Now suppose it so happens, by chance, that the first 5 trials are all failures. How much longer do we have to wait for success? The time for this wait, *after* the fifth trial and *given* that the first five trials are all failures, is *also* geometrically distributed, and with the *same* success probability.

This is just a consequence of the independence of the trials. If we have seen five failures, then independence implies that this has no influence whatsoever on subsequent trials. In other words, the trials “forget” previous outcomes. Our waiting time to success behaves exactly as if we are right at the very start of our sequence of trials.

There is a common fallacy about these situations called the “law of averages”, which is the belief that a sequence of failures makes success more likely and/or that a sequence of successes makes failure more likely. It is possible to define sequences of random experiments where such a law does indeed hold, but it *does not hold* for a sequence of *independent* trials with the same success probability.

So, in general, the memoryless property means that, if $X \sim \text{Geom}(p)$ and $t \in \mathbb{N}$, then the distribution of $X - t$ *given* that $X \geq t$ is also geometric with probability p .

In fact, it can be shown that this property *characterises* the geometric distribution: any memoryless random variable with values in $\mathbb{N}$ must be geometrically distributed.

Some examples of random variables with geometric distribution:

- Program running time: the number of iterations of a loop in a program, where the stopping condition depends on a random quantity satisfying some condition. We assume that the random quantities are independent in each iteration and identically distributed. (Even quantities that are not explicitly random can sometimes be usefully modelled using randomness. See the discussion in § 9.1^a.)
- Time to failure: the number of time intervals (seconds/hours/etc) that pass until the failure of a specific component in a device/machine/system. Here we assume that the component has the same failure probability in each time interval, and that failures in different time intervals are independent. Note that here we are inverting the roles of success and failure, so we wait until the first failure rather than until the first success. This is just a matter of renaming the outcomes, and does not affect the underlying theory.
- Cricket (batting): the number of balls faced by a batter in a Test cricket match.
- Cricket (bowling): the number of balls bowled until the first wicket falls in a Test cricket match.

10.11 THE COUPON COLLECTOR'S PROBLEM

Suppose now that, instead of asking just for the time until the first success, we ask for the time until we have seen *both* success *and* failure. So we are asking for the time until we have seen both possible outcomes. If we are tossing a coin, we are asking for the number of tosses until we have seen both Heads and Tails. For simplicity, we focus on the case where both outcomes are equally likely, i.e., $p = \frac{1}{2}$.

We know that we need at least two tosses, because after just one toss we have only seen one of the two outcomes. We could have both outcomes after two tosses, as in the toss sequences HT or TH, or it could take three or more tosses, e.g., HHT, TTH, HHHT, TTTT, TTTTTH, and so on.

Let the random variable Z be the number of tosses until we have seen both outcomes. We know that $Z \geq 2$. After the first toss (whatever its outcome may be), how long do we have to wait until we see the other outcome too? This random variable is just $Z - 1$. After the first toss, we just wait until the other outcome appears, and that outcome has probability $\frac{1}{2}$. So our waiting time, $Z - 1$, is geometrically distributed with probability $p = \frac{1}{2}$:

$$Z - 1 \sim \text{Geom}\left(\frac{1}{2}\right).$$

Therefore

$$E(Z - 1) = \frac{1}{\frac{1}{2}} = 2. \quad (10.20)$$

We can use this to calculate the expected total waiting time, from the very start, until we have seen both outcomes. This is the expectation of Z , which is

$$\begin{aligned} E(Z) &= E(1 + (Z - 1)) && \text{(since } Z = 1 + Z - 1\text{)} \\ &= 1 + E(Z - 1) && \text{(by a simple application of Linearity of Expectation)} \\ &= 1 + 2 && \text{(by (10.20))} \\ &= 3. \end{aligned}$$

The **coupon collector's problem** is the traditional name given to the extension of this problem to an arbitrary number of equally likely outcomes.

It derives its name from a situation where items in a commercial product line (breakfast cereal boxes, in the original scenario) each contain one of n possible coupons, and a prize is available to someone (perhaps the first person) who collects all n coupons. Assume that all n coupons are equally likely to appear in a given cereal box, and that the numbers of them are so large that we can consider the coupons to be chosen with replacement from an infinite supply. If you want to collect all n different coupons, how many of the items do you have to buy in order to achieve your aim? This usually means you get repeats of some coupons, but as soon as you have found your last coupon, you stop.

We model this as a sequence of independent trials where, instead of just one or two successful outcomes from each trial, we have n possible outcomes, and that all outcomes are equally likely, so each has probability $1/n$.

Let random variable Z be the number of trials until we have seen each possible outcome at least once. We do not care about the order in which the outcomes occur; we don't mind which outcome is the first one we see, which is the second, etc, and which is the last to be seen, as long as we do see them all eventually. As soon as the last outcome (whichever one that may be) occurs, we stop, and the time taken (i.e., number of trials) to reach this point is the value of Z .

For the event $Z = k$, we need the k -th trial to be the first time some particular outcome occurs. This means that no previous trial gives that outcome, and also that the previous trials together include all other outcomes at least once. Deriving an exact expression for this probability, $\Pr(Z = k)$, is nontrivial, and we will not do it now. We focus on the expectation of this random variable, which tells us how long we expect to wait, on average, until all possible outcomes have occurred.

The first trial immediately gives one of the n outcomes (though it could be any of them). So, immediately, we have seen our first outcome, and there are $n - 1$ we have yet to see. One down, $n - 1$ to go!

Then we must wait some period of time until we get another outcome. How long do we have to wait until we get an outcome that's different to the first one? For each trial after the first,

$$\Pr(\text{outcome is } \underline{\text{the same as}} \text{ the first trial}) = \frac{1}{n},$$

since each outcome has probability $1/n$, so the chance that a specific later trial agrees with the first trial is $1/n$. Therefore

$$\Pr(\text{outcome is } \underline{\text{different to}} \text{ the first trial}) = 1 - \frac{1}{n}.$$

When we are waiting for a different outcome to that of the first trial,

- we regard a different outcome (to the first trial) as a “success”, so the success probability is $1 - 1/n$;
- we regard the same outcome (as the first trial) as a “failure”, so the failure probability is $1/n$.

So, with this new view of success/failure, the trials can be viewed as Bernoulli trials with

$$\begin{aligned} p &= 1 - \frac{1}{n}, \\ 1 - p &= \frac{1}{n}. \end{aligned}$$

So the time to success from these trials has geometric distribution with $p = 1 - 1/n$. Denote this time by X_2 . So

$$X_2 \sim \text{Geom}\left(1 - \frac{1}{n}\right).$$

Since X_2 is the minimum time *after* the first trial until we get a different outcome to the first trial, the *total* time *from the start* until we have seen two different outcomes is

$$1 + X_2,$$

since we have one trial for the first outcome (whatever that is), followed by X_2 trials for an outcome different to the first outcome.

After the second outcome has occurred, how many trials are there until we get a third outcome? We represent this by a new random variable X_3 . We can again model this using a geometric distribution, but with different concepts of “success” and “failure”, and different probabilities. Now, “failure” is having one of the first two outcomes again, and “success” is having any other outcome. So

$$\begin{aligned} \Pr(\text{“success”}) &= \Pr(\text{trial outcome is } \underline{\text{different to}} \text{ the first two outcomes to appear}) = 1 - \frac{2}{n}, \\ \Pr(\text{“failure”}) &= \Pr(\text{trial outcome is } \underline{\text{the same as}} \text{ the first two outcomes to appear}) = \frac{2}{n}. \end{aligned}$$

So

$$X_3 \sim \text{Geom}\left(1 - \frac{2}{n}\right).$$

The time *from the start* until we have seen three different outcomes is

$$1 + X_2 + X_3.$$

We can continue in this vein. For each $k \in \{1, 2, \dots, n\}$, define the random variable X_k by

X_k = number of trials *after* $(k - 1)$ different outcomes have occurred, *until* the k -th outcome occurs.

So X_k describes the number of Bernoulli trials in a sequence, where

$$\begin{aligned} \text{“success”} &= \text{trial outcome is one of the first } k - 1 \text{ different outcomes;} \\ \text{“failure”} &= \text{trial outcome is } \underline{\text{not}} \text{ one of the first } k - 1 \text{ different outcomes,} \end{aligned}$$

and the sequence consists of some number of “failures” followed by a single “success”. Therefore

$$X_k \sim \text{Geom}\left(1 - \frac{k-1}{n}\right). \quad (10.21)$$

The extreme cases here are

- $k = 1$: in this case, success is certain, failure is impossible, and $X_1 = 1$. This is the situation right at the start, when we do not have any previous trials, so the very first trial is new, and gives an outcome we haven't seen before. Whatever that outcome is, it becomes the first outcome in the sequence. We can plug $k = 1$ into (10.21), which gives $X_1 \sim \text{Geom}(1)$, but that's just a fancy way of saying that X_1 always equals 1.

- $k = n$: in this case, all outcomes have already occurred except for one, and we are just waiting for that last outcome, which has probability

$$\Pr(\text{"success"}) = 1 - \frac{n-1}{n} = \frac{1}{n}$$

in each trial. So

$$X_n \sim \text{Geom}\left(\frac{1}{n}\right).$$

The total time, from start to finish, is therefore

$$Z = 1 + X_2 + X_3 + \cdots + X_k + \cdots + X_{n-1} + X_n.$$

This is a sum of random variables, and they are all geometrically distributed, although with different probabilities. As we mentioned earlier, our priority is to work out how long we expect to wait until we have seen all outcomes. So we won't work out the entire probability distribution of Z , but rather will focus on its expectation.

Fortunately, this expectation can be calculated using linearity of expectation together with our knowledge of the expectations of the individual geometric random variables X_k .

$$\begin{aligned}
E(Z) &= E(1 + X_2 + X_3 + X_4 + \cdots + X_{n-1} + X_n) \\
&= 1 + E(X_2) + E(X_3) + E(X_4) + \cdots + E(X_{n-1}) + E(X_n) \\
&= 1 + \frac{1}{1 - \frac{1}{n}} + \frac{1}{1 - \frac{2}{n}} + \frac{1}{1 - \frac{3}{n}} + \cdots + \frac{1}{1 - \frac{n-2}{n}} + \frac{1}{1 - \frac{n-1}{n}} \\
&\quad \text{(by (10.21) and Theorem 55)} \\
&= \frac{n}{n} + \frac{1}{\frac{n-1}{n}} + \frac{1}{\frac{n-2}{n}} + \frac{1}{\frac{n-3}{n}} + \cdots + \frac{1}{\frac{n-(n-2)}{n}} + \frac{1}{\frac{n-(n-1)}{n}} \\
&= \frac{n}{n} + \frac{n}{n-1} + \frac{n}{n-2} + \frac{n}{n-3} + \cdots + \frac{n}{2} + \frac{n}{1} \\
&= n \left(\frac{1}{n} + \frac{1}{n-1} + \frac{1}{n-2} + \frac{1}{n-3} + \cdots + \frac{1}{2} + \frac{1}{1} \right) \\
&= n \left(\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n-2} + \frac{1}{n-1} + \frac{1}{n} \right) \\
&\quad \text{(just reversing it, so it looks more familiar)} \\
&= nH_n,
\end{aligned}$$

where H_n is the n -th *harmonic number*, being the sum of the reciprocals of the first n positive integers:

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n-2} + \frac{1}{n-1} + \frac{1}{n}.$$

We learned about the harmonic numbers in § 6.15. Recall the approximation in (6.48), which we repeat here:

$$H_n \approx \log_e n + \gamma.$$

It follows that

$$E(Z) \approx n(\log_e n + \gamma),$$

and in fact it is usually a very good approximation to simply use

$$E(Z) \approx n \log_e n.$$

The coupon collector's problem arises in many different contexts.

- A computer program can produce any one of n possible outputs, and for random inputs these are equally likely. The program is complex, so you have no feasible way of crafting an input that produces a given output. (Perhaps you do not have access to its inner workings, but must treat it as a black box.) Nonetheless, you must test it thoroughly, to ensure that each possible output can be produced without crashing. So you test the program by running it on a sequence of random

inputs. How many tests do you expect to have to do, until you have seen every possible output?

- A proposed random number generator is supposed to produce each of the first n positive integers with equal probability. You run it and observe how long it takes to produce each of these numbers for the first time, and finally, how long it takes before you have seen all numbers at least once. Comparing your observations with theory (i.e., with your knowledge of how long you expected the process to take) may help you assess whether the numbers are sufficiently random for your purposes. (This is the general idea behind the *coupon collector's test* for random number generators.)
- In biology, the problem has been used to estimate the number of different species of some type of life form in some area. This is a kind of “inverse” of the coupon collector’s problem, since we use sequences of species actually observed (which may be very long and contain many repetitions) to make inferences about how many species there are in total, and we may need to use a more general setting where the distribution of outcomes (species) is not uniform. Extensive theory has been developed for this.

10.12 EXERCISES

1. Suppose two independent fair dice are thrown, with one represented by the random variable X and the other represented by the random variable Y . Calculate the probability distribution of $|X - Y|$.

2. Suppose X and Y are independent $\{0,1\}$ -valued random variables where X is unbiased, so

$$\Pr(X=0) = \Pr(X=1) = \frac{1}{2},$$

but the probability distribution of Y is unknown.

What can you say about the probability distribution of $X \oplus Y$, by which we mean $X + Y \bmod 2$?

3. Suppose one of the four bitstrings 000, 011, 101, 110 is chosen uniformly at random. So each of these four strings of three bits has probability $1/4$ of being chosen. Define $\{0,1\}$ -valued random variables X, Y, Z as follows:

$$\begin{aligned} X &= \text{first bit of the chosen bitstring;} \\ Y &= \text{second bit of the chosen bitstring;} \\ Z &= \text{third bit of the chosen bitstring.} \end{aligned}$$

- (a) Prove that (i) X and Y are independent; (ii) X and Z are independent; (iii) Y and Z are independent,
- (b) Determine whether or not Z is independent of the other two, i.e., if Z is independent of the pair (X, Y) .
- (c) Similarly, determine whether or not Y is independent of the other two, and determine whether or not X is independent of the other two.

4. You want to play a game that needs one die, but you do not have one. Your friend suggests using five fair coins instead, observing the number of Heads (which can be any number in $\{0, 1, 2, 3, 4, 5\}$), and adding one to get a number in $\{1, 2, 3, 4, 5, 6\}$, and using this final number in place of the number shown by a die.

Discuss this suggestion.

Can you think of any other way in which a set of fair coins (possibly more than six) can be used to simulate the throw of a fair die?

5. Define the random variable W as follows.

k	$\Pr(W = k)$
0	$\frac{1}{2}$
1	$\frac{1}{3}$
2	$\frac{1}{6}$

Determine the expectation, median, mode, variance and standard deviation of W .

6. Find the variance of the sum of the numbers shown on two fair dice.

What is the probability that this sum is at least two standard deviations away from its mean? Compare this with what Chebyshev's Inequality says about this case.

7. Suppose $X \sim \text{Bin}(16, 0.5)$.

- (a) Your friend claims that, because $p = \frac{1}{2}$, half the trials must be successes and half must be failures. What is the probability that that actually happens?
- (b) What is the probability that X is at least two standard deviations away from the mean? (Use a calculator/spreadsheet/program as needed.)
- (c) What is the probability that X is at least three standard deviations away from the mean?
- (d) How do these exact probabilities compare with the bounds given by Chebyshev's Theorem?

8. During a meteor shower, meteors can arrive randomly at any time and are independent of each other. The Eta Aquariids meteor shower (on now!) has — at its peak and in idealised conditions — an average rate of 0.83 meteors per minute that are visible to the unaided eye.⁴

- (a) What is the distribution of the number of meteors seen in a given minute around the peak of the shower?
- (b) What is the probability that no meteors are seen in a given minute?
- (c) How long should an observer be prepared to wait for, if they want to have a probability of 0.95 of seeing a meteor? (You'll need a calculator, spreadsheet or program for this.)

9. Each time a bowler on the opposing team bowls to a Test cricket batter, the probability of the batter being dismissed (thereby ending their innings) is 1.25%, with the outcomes from different balls being independent.

- (a) What is the distribution of the number of balls the batter faces until they are dismissed?
- (b) What is the probability they get a “golden duck” (i.e., dismissed by the first ball they face)?
- (c) What is the expected length of their innings?
- (d) Suppose the batter has faced 200 balls without being dismissed. How many further balls would you expect them to face until being dismissed?

10. For each of the random variables listed below, state which of the following distributions is the best model for it: uniform, binomial, Poisson, geometric. If none of these seem to fit, discuss why, and suggest an appropriate distribution.

- (a) the face value of the top card in a well-shuffled standard deck of 52 playing cards, where the face values of Ace, Jack, Queen and King are defined to be 1, 11, 12 and 13, respectively, and the face value of any other card is the number shown on it.
- (b) the number of days in a particular week on which your regular morning train to work arrives late, where train arrival times on different days are independent and identically distributed, and a train is late if it arrives at least a minute after its scheduled arrival time.

⁴ This rate is idealised in the sense that it assumes perfect atmospheric conditions and gives the number of meteors that would be seen if they were all high in the sky, at the best time of night for viewing the shower. In reality, viewing conditions seldom come close to that ideal. You have to be much more patient than the “official” rates may seem to indicate!

- (c) the number of cosmic rays that hit the CPU of your laptop during your next class;
- (d) the number of working days you have to wait, from tomorrow onwards, until your regular morning train arrives on time (with same assumptions as for (b)).
- (e) the number of cereal packets you have to buy until you get one of your three favourite coupons (under the original coupon collector's scenario).
- (f) the number of cowrie shells that land aperture-up when seven of them are thrown in a game of Pachisi⁵, assuming the cowrie shells are identical and behave independently.
- (g) a random digit from the first 1,002 digits after the decimal point in the decimal representation of $3/7$.
- (h) a random digit from the first 1,000 digits after the decimal point in the decimal representation of $5/11$.
- (i) a random digit from the first 10^{100} digits of the decimal representation of π .

11. You met the Birthday Paradox in Exercise 9.7. There, the focus was on how many people you need to meet in order for it to be more likely than not that at least two of those people have the same birthday. Now let's ask a different question, but under the same assumptions we made then.

Suppose you are recording birthdays of members of a customer loyalty scheme, so that your company can send them greetings on their birthday each year. How many people do you expect to have to enrol in the scheme until their birthdays cover every day of the year?

⁵ an ancient Indian ancestor of Ludo, Trouble and Cờ cá ngựa

GRAPH THEORY I

Graphs are abstract models of *networks*. They can be used to model any system consisting of components that interact in some way. For example, they can model social networks, molecules, maps, electronic circuits, transport networks, the web, communications networks, software systems, timetabling requirements, and much else. In each case, we have a set of nodes or *vertices* together with links or *edges* between certain pairs of vertices. Table 11.1 lists a number of network types, identifying the vertices and edges for each. We met some of these previously, on p. 66 in § 2.13.

type of network	vertices	edges
social network	people	friendships
phone call network	phones	phone calls
the web	web pages	hyperlinks
molecule	atoms	bonds
map	countries	borders
railway network	towns/cities	train lines
road network	intersections	roads
timetabling	units (i.e., subjects)	unit pairs with a student in common
electrical circuit	junctions	wires
polyhedron	vertices	edges

Table 11.1: Some types of networks, with their vertices and edges.

Like other mathematical models, graphs are *abstractions*, so they don't represent *everything* about a system. They are intended to capture the structure of the *interactions*, without incorporating the details of how those interactions work or what the nodes do. This means a lot of information is thrown away. For example, in a graph that models a social network, we don't record people's height or where each pair of friends

first met. Nonetheless, the information retained in the graph enables many important problems about the network to be solved.

Problems we can tackle using graphs include:

- In a social network, who has the most friends? Who has the most central position in the network? What is the largest clique of people, who all know each other? Can you identify subcommunities of people within the network?
- What's the minimum number of cities you need to travel through in order to drive between two specific cities? Is there a tour that visits every city and returns to its starting point, and if so, which of these has the fewest repeat visits to cities?
- Given an electronic circuit, can it be embedded in a silicon wafer so that no wires cross each other? (This is important for avoiding short circuits and reducing manufacturing costs.)
- Given a computer network, how can it be displayed on a screen with the fewest edge crossings? (This is important for constructing network diagrams that are readable and help people understand networks better.)
- How many different ways are there of travelling by train between two specific cities?
- How many different timeslots are needed for an exam timetable in which no student has a clash?

This chapter introduces the basic concepts of graph theory and some foundational results relating to vertex degrees and various kinds of paths and cycles.

11.1^α BASIC DEFINITIONS

A *graph* consists of a set of vertices and a set of edges. The set of vertices can be any set, and is intended to represent the objects we are interested in. Each edge must be a pair of vertices.

We now state this definition more formally.

A **graph** is a pair (V, E) where V is a set and E is a set of unordered pairs of elements of V . Each member of V is called a **vertex** and each member of E is called an **edge**.

If G is a graph, we may write $G = (V, E)$ to make clear that V is its set of vertices and E is its set of edges. We can also write $V(G)$ for the set of vertices of G and $E(G)$ for the set of edges of G .

So, to specify a graph, we need to specify its vertex set and edge set. Each member of the edge set is an unordered pair of vertices. So, in a graph $G = (V, E)$, an edge $e \in E$ between two vertices $v, w \in V$ is the set $\{v, w\}$ containing just those two vertices.

For example, let's define a graph $G = (V, E)$ with

$$\begin{aligned} \text{vertex set: } V &= \{a, b, c, d, e, f, g\}; \\ \text{edge set: } E &= \{\{a, b\}, \{a, c\}, \{b, c\}, \{c, d\}, \{f, g\}\}. \end{aligned}$$

This graph has seven vertices and five edges. It is depicted in Figure 11.1, where the dots represent vertices and the lines represent edges.

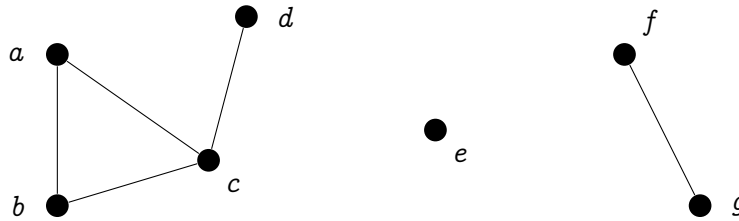


Figure 11.1: A graph G .

We say that two vertices v, w of a graph are **adjacent** if $\{v, w\}$ is an edge in the graph. We also use the convenient shorthand $v \sim w$ for this, since it is a bit briefer than writing $\{v, w\} \in E$, but they mean the same thing. If v and w are not adjacent, then we can write $v \not\sim w$. For example, in our graph G in Figure 11.1, we have $a \sim b$, $a \sim c$, $b \sim c$, $c \sim d$, and $f \sim g$. For any other pair v, w of distinct vertices of G , we have $v \not\sim w$. For example, $d \not\sim f$.

If v is a vertex, then every vertex w that is adjacent to v is called a **neighbour** of v . The set of all neighbours of v is the **neighbourhood** of v .

If v and w are adjacent, then they are said to be **endpoints**, or **endvertices**, of the edge $\{v, w\}$ between them.

Adjacency refers to the relationship between *two vertices* that share an edge. We use a different term for describing the relationship between a *vertex* and an *edge*. If vertex v is one of the two vertices in edge e — which means that there is some other vertex x for which $e = \{v, x\}$ — then we say that v is **incident** with e .

The same term is used to describe the relationship between *two edges* that have a common vertex. Let $\{v, w\}$ and $\{v, x\}$ be edges that meet at the vertex v , with $w \neq x$. Then we say that these two edges are **incident** with each other.

In the graph of Figure 11.1:

- Vertex c is incident with edges $\{a, c\}$, $\{b, c\}$ and $\{c, d\}$, but not with any others.
- Edge $\{a, c\}$ has endpoints a and c . This edge is incident with $\{a, b\}$, since they meet at vertex a , and it is also incident with $\{b, c\}$ and $\{c, d\}$, since they meet at vertex c .
- Edge $\{f, g\}$ is incident with vertices f and g (its endpoints), but it is not incident with any other edges.

A vertex is **isolated** if it does not belong to (i.e., is not incident with) any edges. This happens if and only if it is not adjacent to any other vertices. In the graph in Figure 11.1, vertex e is isolated.

A vertex is a **leaf** if it belongs to exactly one edge. In Figure 11.1, there are three leaves: d , f and g .

11.2^α TYPES OF GRAPHS

Under our definition of graphs,

- no vertex is adjacent to itself. Since an edge is a *set* of *two* vertices, those vertices must be distinct (else we'd have a set consisting of the same vertex appearing twice, but duplicates are not allowed in sets).
- no two edges can have the same pair of endpoints. To see this, suppose we have two edges with the same two endpoints v and w . Then the two edges must both be $\{v, w\}$. (Writing one as $\{v, w\}$ and the other as $\{w, v\}$ makes no difference, because order does not matter in a set, so they are still really the same set, even though we have written them differently.) And since the graph's edges are considered to be a *set* (recalling our definition of a graph as (V, E) , where E is the *set* of edges), there cannot be duplicate edges in that set.

In graph theory, a graph that satisfies both these conditions is called a **simple graph**. We will focus on simple graphs in this unit. We will usually omit the adjective “simple”, and will just assume that our graphs are all simple graphs unless we say otherwise.

In focusing on simple graphs, we are not saying that they are the only type that matters. There are broader classes of graphs in which the two conditions above are relaxed.

- Sometimes, *loops* are allowed. A **loop** is an edge whose two endpoints are identical. It therefore joins a vertex to itself, and is shown in diagrams as a closed curve through the vertex.
- Sometimes, we allow more than one edge between a pair of vertices. We call these **multiple edges** or **parallel edges**.

There are other classes of graphs in which we allow the vertices and/or edges to carry other information. A **weighted graph** has a number on each edge, which could represent a length, or size, or cost of some kind.

In our graphs, the edges have no *direction*. An edge $\{v, w\}$, being a set, has no notion of order between v and w . Such a graph is said to be **undirected**. So far, we have been working entirely with undirected graphs, and we will omit the adjective “undirected” and just assume that graphs are of this type unless otherwise stated.

But there are many situations where the order of vertices in an edge does matter. For example, a one-way street in a road network has a strict order between its endpoints. A hyperlink between two webpages goes from the linking page to the linked page. A phone call has a caller and a receiver. To model these situations, we use ordered pairs for edges, instead of unordered pairs. So, a **directed edge** from vertex v to vertex w is an ordered pair (v, w) . A directed edge is also called an **arc**. A **directed graph** is defined just as we have done, except that we change from *unordered* pairs to *ordered* pairs for edges. In other words, a directed graph is a graph in which all edges are directed. It is also possible to define “mixed” or “hybrid” graphs in which edges may be either directed or undirected.

All these different types of graphs are used as models in a wide variety of practical contexts. We focus on simple (undirected) graphs because

- they are simple! Relatively so, anyway.
- they appear as special cases in most other classes of graphs, so we’d have to master them anyway if we want to learn about other classes of graphs.
- they are still very widespread, as models of practical situations;
- they are still complex enough to capture all the main computational issues that arise in more general classes of graphs.

11.3^ω GRAPHS AND RELATIONS

We first encountered graphs informally — under the more informal term *network* — when discussing binary relations in § 2.13. (See p. 66.) The relationship between graphs and binary relations is very close.

Adjacency is a binary relation defined on the set of vertices of a graph. It consists of all ordered pairs (v, w) such that v is adjacent to w . Our graphs are undirected, so if v is adjacent to w then w is adjacent to v . In other words, the pair (v, w) belongs to the adjacency relation if and only if (w, v) also belongs to it. This tells us that adjacency is a *symmetric* binary relation. However, it is not reflexive. In fact, *no* vertex is adjacent to itself, so the adjacency relation does not contain any pairs (v, v) where the two vertices in the pair are the same. So we say that the adjacency relation is *irreflexive*.¹ We can think of the adjacency binary relation as being obtained by replacing each edge $\{v, w\}$ by the two ordered pairs (v, w) and (w, v) .

So simple graphs may be regarded as irreflexive symmetric binary relations. Every simple graph gives rise to such a relation, via adjacency. Conversely, every irreflexive symmetric relation may be used to define a simple graph whose vertex set is the domain of the relation.

¹ This is a stronger condition than just “not reflexive”; it is not merely a logical negation of reflexivity, but a kind of extreme opposite of it.

If we discard irreflexivity, then we are allowing (but not enforcing) loops. If, instead, we discard symmetry, then we have directed graphs rather than undirected graphs. If we discard both of these, then we have a directed graph which may have loops but is not allowed to have multiple edges. (Here, we mean that, if we have an edge (v, w) from vertex v to vertex w , we do not have any *extra* edges from v to w , i.e., no other edges “parallel” to (v, w) and *in the same direction*. But we do allow the reverse edge (w, v) . In directed graphs, forbidding multiple edges does not forbid us from having both (v, w) and (w, v) .) So we may regard any binary relation as a directed graph, possibly with loops but with no multiple edges. Similarly, any directed graph with no multiple edges gives rise to a binary relation.

11.4^α REPRESENTING GRAPHS

It is natural to display graphs as diagrams, as we did in [Figure 11.1](#), when they are small enough for this to be practical. But diagrams have their limits. Many graphs from real applications are too large to be depicted in a diagram on a page or a device screen. In any case, to run algorithms on graphs, we need to be able to store them in computer memory. To do this, we need formal, purely symbolic ways of representing graphs.

We now consider the four main ways of representing graphs symbolically. We will not discuss the details of using these representations in programs; that will be considered in FIT2004 Algorithms and Data Structures.

11.4.1 *Edge list*

Our first representation corresponds to the formal definition of graphs that we have just given.

An **edge list** of a graph is just a list of its edges. Typically, the order does not matter, so we can think of this as a set, although inside a computer information is always stored in some specific order.

To represent a graph completely, listing the edges alone is not sufficient, in general. This is because it’s possible for a vertex to belong to no edges. Such a vertex will not be apparent just from looking at all the edges. So, we should specify both the vertex set and the set of edges. This just means specifying exactly the information required in our formal definition of graphs.

An edge list does not necessarily group edges together according to the vertices they share. This may mean it is less efficient, for some tasks, than other representations we will cover shortly.

11.4.2 *Adjacency matrix*

The **adjacency matrix** of a graph G is an $n \times n$ array A of bits in which

- n is the number of vertices of G ,
- the rows of the array correspond to the vertices of G ;
- the columns of the array also correspond to the vertices of G ;
- for every two vertices v and w , the entry in the v -row and w -column of A is

$$\begin{aligned} 1, & \quad \text{if } v \sim w; \\ 0, & \quad \text{if } v \not\sim w. \end{aligned}$$

We think of 1 as meaning “edge” and 0 as meaning “no edge”. Another way to put this is to say that every entry of the matrix gives the *number of edges* between the two vertices. For simple graphs, this number must be 0 or 1, so we can represent it by a single bit.

Here is an adjacency matrix for the graph G in [Figure 11.1](#). We show the vertices corresponding to each row and column, for convenience, but they are not part of the matrix itself.

$$\begin{array}{c} a \quad b \quad c \quad d \quad e \quad f \quad g \\ \begin{array}{l} a \\ b \\ c \\ d \\ e \\ f \\ g \end{array} \left(\begin{array}{ccccccc} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{array} \right) \end{array}$$

We see, for example, that $a \sim c$, since the entry in the a -row and c -column is 1, while $c \not\sim f$, since the entry in the d -row and f -column is 0.

We make two observations about the adjacency matrix.

- The entries in the main diagonal (top left to bottom right) are all 0, because a vertex is never adjacent to itself.
- The matrix is symmetric about the main diagonal. This means that, for all v and w , the entry in row v and column w is the same as the entry in row w and column v . This means that the matrix is completely determined by all the entries that lie above the main diagonal. Similarly, it is completely determined by all the entries that lie below the main diagonal.

As we often do with tables of data, it is natural to ask about the sums of the entries in each row and each column, and the sum of all entries in the entire matrix.

- For a given vertex v , what information about it is captured by the number of 1s in the v -row of the adjacency matrix? What about the v -column? We return to this question in [§ 11.7](#).

- Consider the number of 1s in the entire adjacency matrix. The 1s indicate the edges in the graph, but each edge is counted twice: edge $\{v, w\}$ gives 1 in the entry for the v -row and w -column, and also for its “mirror image” entry in the w -row and v -column. So the number of 1s in the adjacency matrix is exactly twice the number of edges of the graph.

The rules for adjacency matrices can be relaxed, if we are working with a wider class of graphs than just simple graphs. If we allow loops, then some of the diagonal entries can be 1. If we have a directed graph, then the entry in row v and column w need not equal the entry in row w and column v . If we allow multiple edges between two vertices, then we can have entries that are neither 0 nor 1; the entry for row v and column w can give the number of edges between v and w .

We referred to an adjacency matrix as an *array*, and it can be represented in programs using two-dimensional array structures, which most programming languages have. It can also be thought of as a “table”, with n rows and n columns, with the row and column headings not included in the counts of rows and columns.

But there is a reason why we called it an adjacency matrix rather than an adjacency “array” or “table”. The use of the term “matrix” is not just about *the way it stores* the adjacency information in a two-dimensional $n \times n$ way. If that were all, then the terms “2D array” and “table” would serve just as well. The term “matrix” is also about *what you can do with it*. In mathematics, we can do operations with matrices, to calculate important numbers from them or form other matrices. For adjacency matrices of graphs, these operations can shed light on the graph, revealing aspects of its structure that might have been hard to determine otherwise. They are also used in some algorithmic problems on graphs. We don’t cover these graph-theoretic applications of matrices in this unit. But it is good to be aware that they are “out there”, even if you mainly just use adjacency matrices as a method for storing graphs.

One virtue of the adjacency matrix is that you can efficiently test whether or not two given vertices are adjacent: you just have to look up an entry in a matrix, and most computer representations of matrices (using 2D arrays) enable this to be done very quickly. But other operations — like searching through all the neighbours of a vertex — can be done more efficiently using adjacency lists, which we come to next (§ 11.4.3).

The adjacency matrix takes the same amount of space regardless of how many edges the graph has. This may not be a problem, if the graph has many edges. But many real-world networks are relatively “sparse”, meaning (roughly speaking) that most pairs of vertices do not have an edge between them. If you are dealing with a class of large sparse graphs, then the adjacency matrix representation may take up too much space in memory.

11.4.3 Adjacency list

An **adjacency list** specifies the vertices in some order and, with each vertex, it gives a list of all the vertices it is adjacent to.

For example, the graph G in Figure 11.1 has the following adjacency list representation.

```

a  :  b, c
b  :  a, c
c  :  a, b, d
d  :  c
e  :
f  :  g
g  :  f

```

Each line nominates a vertex, followed by a list of all its neighbours. We have used a colon here to separate each vertex, at the start of its line, from the list of all its neighbours. But that is just a superficial detail; other ways of conveying this information can be used, provided it is done clearly and consistently. We have used punctuation for human readers, but when graphs are represented as adjacency lists in computers, the separation of the different types of information is done differently, using data structures such as arrays or lists which you will learn about in programming units.

We can see from the adjacency list that, for example, c is adjacent to a , b and d , so that the neighbourhood of c is $\{a, b, d\}$. We can also readily see that e has no neighbours and that the graph has three leaves, namely d , f and g .

This is probably the most widely used representation of graphs in computers. It enables efficient searching of neighbourhoods of vertices, which is a common task in many graph algorithms. It is compact, more so than the adjacency matrix.

11.4.4 Incidence matrix

An **incidence matrix** of a graph G is an $m \times n$ array A of bits such that

- m is the number of edges of G , and n is the number of vertices of G ;
- the rows of the array correspond to the edges of G ;
- the columns of the array correspond to the vertices of G ;
- for every edge e and vertex v , the entry in the e -row and v -column is

$$\begin{array}{ll} 1, & \text{if } e \text{ is incident with } v; \\ 0, & \text{if } e \text{ is not incident with } v. \end{array}$$

We think of 1 as meaning “incident” and 0 as meaning “not incident”.

As with the adjacency matrix, we call this a “matrix” because some standard matrix operations can be done with it in order to study the graph.

This representation of graphs is seldom used in computers. But it is theoretically important and has been used to prove theorems about graphs.

11.5 SUBGRAPHS

Suppose we want to focus our attention only on a portion of a graph G . If this portion is a graph in its own right, we say it is a *subgraph* of G . This is not yet a precise formal definition, because we have not defined what we mean by a “portion” of a graph.

Let $G = (V, E)$ be a graph. A **subgraph** of G is a graph $F = (U, D)$, with vertex set U and edge set D , such that

$$U \subseteq V, \quad \text{and} \quad D \subseteq E.$$

In other words, every vertex of F is also a vertex of G , and every edge of F is also an edge of G . When F is a subgraph of G , we write $F \leq G$. In effect, we are “overloading” the $\leq$ symbol so that, as well as standing for ordinary numerical inequality, it also stands for the subgraph relation. Context should make it clear which is meant.

For example, let $F = (U, D)$ be the graph defined by

$$\begin{aligned} U &= \{a, b, c, e, g\}, \\ D &= \{\{a, b\}, \{a, c\}\}, \end{aligned}$$

shown in Figure 11.2. Then F is a subgraph of G , and we can write $F \leq G$.



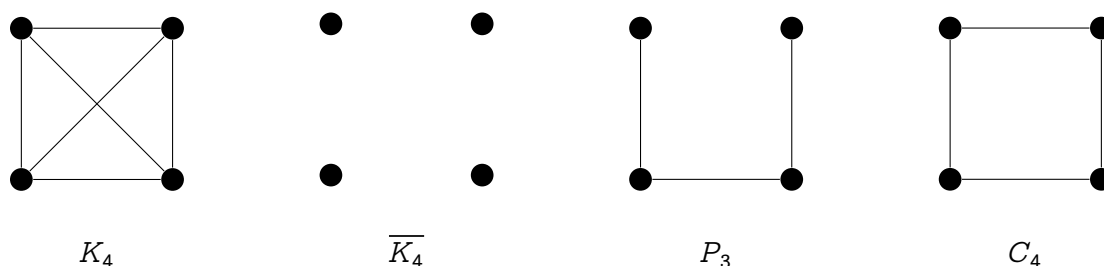
Figure 11.2: A subgraph H of the graph G of Figure 11.1.

A graph is a subgraph of itself: $G \leq G$. You can check that the definition is satisfied. But there are times when we want to exclude this possibility and only consider those subgraphs that are not the whole graph. We say that F is a **proper subgraph** of G if it is a subgraph of G but is not equal to G . Symbolically, $F \leq G$ and $F \neq G$. We can write this as $F < G$.

11.6 SOME SPECIAL GRAPHS

The **complete graph** on n vertices, denoted by K_n , has every pair of vertices joined by an edge.

At the other extreme, the **null graph** on n vertices, denoted by $\overline{K_n}$, has no edges at all.

Figure 11.3: K_4 , $\overline{K_4}$, P_3 , C_4 .

The **path graph** of length $n - 1$, denoted by P_{n-1} , has n vertices and $n - 1$ edges with the property that the vertices can be put in a sequence so that two vertices are adjacent if and only if they are consecutive in the sequence.

The **cycle** of length n , denoted by C_n , has n vertices and n edges and can be formed from a path graph on the same vertices by adding a new edge between the first and last vertex of the path.

Examples of these graphs, with four vertices, are shown in [Figure 11.3](#).

11.7 DEGREE

The **degree** of a vertex is the number of neighbours it has. Since our graphs are *simple*, it also equals the number of edges that are incident with the vertex.

We denote the degree of vertex v in graph G by

$$\deg_G(v).$$

If the graph G is clear from the context, we may drop the subscript and just write $\deg(v)$.

This gives us some alternative descriptions of some concepts we introduced earlier.

- A vertex is isolated if and only if it has degree 0.
- A vertex is a leaf if and only if it has degree 1.

If n is the number of vertices of a graph G , then the degrees of the vertices of G can be any of $0, 1, 2, \dots, n - 1$. There are n different possibilities for the degrees here. But it turns out that we never get *all* these different degrees in the same graph. The proof is a good example of proof by cases.

Theorem 56. In every graph, there are two vertices with the same degree.

Before reading the proof of this theorem, you should draw some small graphs and work out the degrees of their vertices. Try and construct a graph with all vertex degrees being different, and try to get a sense of why this doesn't seem to be possible.

Proof. Let G be any graph, and let n be its number of vertices.

We use proof by cases.

We consider two cases, depending on whether or not G has a vertex of degree $n - 1$.

Case 1: G does not have a vertex of degree $n - 1$.

In this case, the possible degrees that vertices of G can have are $0, 1, 2, \dots, n - 2$. There are $n - 1$ different numbers in this list. So there are only $n - 1$ different degrees that the vertices of G can have. Yet there are n vertices in G . So at least two of them must have the same degree.

Case 2: G has a vertex of degree $n - 1$.

Let v be a vertex of G that has degree $n - 1$. The only way a vertex in an n -vertex graph can have degree $n - 1$ is if it is adjacent to all other vertices. This means that all other vertices have v as a neighbour, so their degrees are all ≥ 1 . Therefore G has no vertex of degree 0. Therefore all the degrees, of all the vertices of G , must be in the set $\{1, 2, \dots, n - 1\}$. This means there are only $n - 1$ possible degrees that a vertex of G can have. But there are n vertices in G . Therefore there must be at least two vertices in G that have the same degree. $\square$

Now we study the sum of all the vertex degrees of a graph. One reason for studying this is because it enables us to determine the average degree of a vertex in the graph, which helps in understanding what the graph looks like locally.

Our main theorem on this is called the Handshaking Lemma. The name comes the following scenario. At a social function with n people, some pairs of people shake hands when they first meet, others do not. How many handshakes occur? One way to determine this is to find out, from each person, how many times they shook hands. Adding up these individual numbers of handshakes counts each handshake twice: it takes two hands to shake! Think of the people as vertices and the handshakes as edges.

Theorem 57 (Handshaking Lemma). For every graph $G = (V, E)$, the sum of the degrees of the vertices of a graph is twice its number of edges:

$$\sum_{v \in V} \deg(v) = 2m,$$

where m is the number of edges of G .

Proof. If you rephrase the above handshaking explanation in terms of vertices and degrees, then you have a proof of this theorem.

Alternatively, think of the adjacency matrix of G (see § 11.4.2). For each vertex, the number of 1s in the v -column is just the degree of v . Since the columns are disjoint and together make up the entire matrix (i.e., they partition the matrix entries), adding up the column counts gives the total number of 1s in the matrix. But we noted in § 11.4.2 that the number of 1s in the matrix is $2m$. Therefore the sum of all the degrees equals $2m$. $\square$

From this, we immediately obtain the average degree of the vertices of a graph.

Corollary 58. For any graph G ,

$$\text{average degree of } G = \frac{2m}{n},$$

where n is the number of vertices of G and m is its number of edges.

Proof.

$$\text{average degree of } G = \frac{\text{sum of degrees}}{\text{number of vertices}} = \frac{\sum_{v \in V(G)} \deg(v)}{n} = \frac{2m}{n},$$

by [Theorem 57](#). □

The average degree of a graph is therefore very simple to calculate. You only need to know the two most fundamental parameters of the graph: its number of vertices, and its number of edges. These parameters are “global” in the sense that they pertain to the graph as a whole rather than any part of it. But, once you have them, this simple calculation of $2m/n$ gives you the average degree, which tells you something about the “local” structure of the graph, namely what happens, on average, in the immediate vicinity of the vertices.

In principle, the average degree can be as low as 0, for graph with no edges, and as high as $n - 1$, for complete graphs on n vertices. In practice, real-world graphs tend to have average degrees that are much lower than the maximum possible. The worldwide human social network currently has about 8,000,000,000 people, but the average degree is tiny by comparison. (The average degree depends on how you define the edges. If the graph uses close friendships only, the average degree is said to be about 4.)

Another consequence of the Handshaking Lemma follows from the fact that the sum of the degrees is even.

Corollary 59. Every graph has an even number of vertices of odd degree.

Proof. Since $2m$ is even (where m is the number of edges of the graph), the sum of all the degrees is even (by [Theorem 57](#)). Now, the sum of the even degrees (i.e., those vertex degrees that are even numbers) is also even, since the sum of even numbers is always even. It follows that the sum of the odd degrees is even too.

Recall that

- the sum of two odd numbers is even,
- therefore the sum of an even number of odd numbers is even;
- the sum of an odd number of odd numbers is odd.

It follows that there must be an even number of odd degrees, else the sum of the odd degrees would be odd, whereas we observed above that it's even. $\square$

[Theorem 56](#) and [Corollary 59](#) give some necessary conditions for a set of numbers to be a valid set of degrees of some graph. They show that not every set of numbers within the required range (0 to $n - 1$) can be the set of degrees of a graph.

11.8 MOVING AROUND

We often want to move from one part of a graph to another, using the edges to step from vertex to vertex.

A **walk** from a vertex v to a vertex w is a sequence of vertices and edges,

$$v_0, e_1, v_1, e_2, v_2, \dots, v_{k-1}, e_k, v_k, \quad (11.1)$$

where

- $v_0 = v$, meaning that the walk starts at v ,
- $v_k = w$, meaning that the walk finishes at w ,
- each v_i is a vertex of G ,
- each e_i is an edge of G ,
- $\forall i : e_i = \{v_{i-1}, v_i\}$. This means that the edge e_i links the two vertices listed on either side of it. So, to go from v_{i-1} to v_i , we step along edge $e_i = \{v_{i-1}, v_i\}$.

The **length** of a walk is the number of edges in it. The shortest possible walk is the walk of length zero, which consists of just a single vertex and no edges.

For example, consider in the graph in [Figure 11.4](#). Here are some examples of walks in this graph.

$a, \{a, b\}, b, \{b, d\}, d, \{d, e\}, e, \{e, f\}, f$	a walk of length 4 from a to f
$a, \{a, b\}, b, \{b, d\}, d, \{d, c\}, c, \{c, b\}, b, \{b, e\}, e$	a walk of length 5 from a to e
$b, \{b, e\}, e, \{e, f\}, f, \{e, f\}, e, \{e, d\}, d$	a walk of length 4 from b to d
$a, \{a, b\}, b, \{b, c\}, c, \{c, a\}, a$	a closed walk of length 3 from a to a
$a, \{a, b\}, b, \{b, c\}, c, \{c, b\}, b, \{b, c\}, c, \{c, a\}, a$	a closed walk of length 5 from a to a
$a, \{a, c\}, c, \{c, b\}, b, \{b, d\}, d, \{d, e\}, e, \{e, b\}, b, \{b, a\}, a$	a closed walk of length 6 from a to a
$b, \{b, c\}, c, \{c, d\}, d, \{d, b\}, b, \{b, a\}, a, \{a, c\}, c, \{c, d\}, d, \{d, e\}, e$	a walk of length 7 from b to e
c	a walk of length 0 from c to c

Note, in particular, two key *freedoms* we have in walking around a graph.

- We are allowed to visit a vertex more than once.

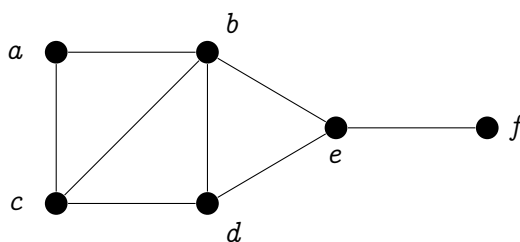


Figure 11.4: A graph.

- We are even allowed to move along an edge more than once, and there is no requirement to always move along a particular edge in the same direction. For example, we could move along the edge $\{a, b\}$ from a to b , and later in the walk we could move along it from b to a (and/or we could go along it from a to b again).
 - Note that, if we use an edge twice, we must necessarily use each of its endpoints at least twice too. But it is possible, in many graphs, to visit some vertices more than once without using any edge more than once.

For simple graphs (which are our focus), there can only be one edge between two specific vertices. So, if our walk steps from vertex a to vertex b , then it must do so along the edge $\{a, b\}$, and there is only one such edge. So, in fact, the edge we use to go from a to b is completely determined by the two vertices. This means that, to specify a walk in a *simple* graph, it is sufficient to specify its sequence of vertices. So, our walk in (11.1) could be specified by just listing the vertices in order:

$$v = v_0, v_1, v_2, \dots, v_{k-1}, v_k = w.$$

Let us do this for our examples of walks for the graph in Figure 11.4.

a, b, d, e, f	a walk of length 4 from a to f
a, b, d, c, b, e	a walk of length 5 from a to e
b, e, f, e, d	a walk of length 4 from b to d
a, b, c, a	a closed walk of length 3 from a to a
a, b, c, b, c, a	a closed walk of length 5 from a to a
a, c, b, d, e, b, a	a closed walk of length 6 from a to a
b, c, d, b, a, c, d, e	a walk of length 7 from b to e
c	a walk of length 0 from c to c

But if we are walking in graphs that may have multiple edges, then we need to specify the edges between the vertices as well as the vertices themselves.

A walk is **closed** if its start and end vertices are the same. With our above notation, this is when $v = w$, i.e., $v_0 = v_k$. In our examples of walks in the graph of Figure 11.4, the closed walks are the following.

a, b, c, a
 a, b, c, b, c, a
 a, c, b, d, e, b, a
 c

We now consider walks that are more restricted, in the sense that we forbid one or both of the two freedoms we mentioned above.

A **trail** is a walk in which no edge is used more than once. Using (11.1), this means that all the edges

$$\{v_0, v_1\}, \{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{k-1}, v_k\}$$

are different. We cannot get around this restriction by going along an edge once in one direction and then again later in the other direction; this still counts as using the edge twice, which is forbidden for trails.

So, for trails, the second of our two walking freedoms (i.e., the freedom to use an edge more than once) is prohibited. But we are still allowed to repeat vertices if we wish (although we don't have to).

In the graph of Figure 11.4, the following walks are trails:

a, b, d, e, f
 a, b, d, c, b, e
 a, b, c, a
 a, c, b, d, e, b, a
 c

The following walks are *not* trails, for the reasons indicated.

b, e, f, e, d	re-uses edge $\{e, f\}$
a, b, c, b, c, a	re-uses edge $\{b, c\}$
b, c, d, b, a, c, d, e	re-uses edge $\{c, d\}$

A **closed trail** is just a closed walk that is also a trail. So, it finishes at the same vertex where it starts, and uses each edge at most once (with repeat visits to vertices being allowed).

In our list of examples for the graph of Figure 11.4, the following walks are closed trails:

a, b, c, a
 a, c, b, d, e, b, a
 c

A **path** is a trail in which no vertex or edge is repeated. So, both our walking freedoms (reusing vertices and reusing edges) are now prohibited.

In the graph of Figure 11.4, the following walks are paths:

a, b, d, e, f
 c

Note that the path a, b, d, e, f from a to f is not a *shortest* path from a to f . The shortest path from a to f is a, b, e, f , which has length 3. In this graph, there is a unique shortest path from a to f , but this does not always happen. For example, there are two shortest paths from a to d .

To help remember the distinctions between walks, trails and paths, imagine hiking in the countryside and think of how these three terms get stronger in what they imply about where you can go. You may be able to walk wherever you like²; it's something *you* do, rather than something that is laid out for you. A trail may be a simple, rough track, more definite than an arbitrary walk but maybe lacking in official status. People will tend to stay on a trail rather than wander off it. A path may be a more definite, and may have been laid down by others or by some authority. Perhaps it has a special surface; perhaps there is more expectation that you stick to it; perhaps it is more well documented. So, in ordinary usage, the terms "walk", "trail" and "path" get *stronger*, and more restrictive, as you go from one to the next. Similarly, in graphs, the terms get stronger and more restrictive, too. (The analogy here is very loose. It's only designed to help remember the order in which the terms get more restrictive. But this should still help you remember which is which.)

Whenever there is a walk between two vertices in a graph, there is also a path between them. In fact, the shortest walk between two vertices is always a path. (Why?)

The **distance** between two vertices is the length of the shortest path between them. This is the same as the length of the shortest walk between them, by the observations in the previous paragraph.

The prohibition on repeating vertices in paths means that a path cannot start and end at the same vertex. But we also want a way to talk about "paths" that can do that.

A **cycle** is a closed trail in which no vertex except the first and last is repeated. Because it is closed, the first and last vertex must be the same, but that vertex must not appear anywhere else on the cycle. We might think of a cycle as a "closed path" although, strictly speaking, that term is self-contradictory as a path cannot be closed.

Although a cycle is much more restricted, in its allowed repetitions, than an arbitrary closed walk, it is interesting to note that, when their lengths are *odd*, the existence of one implies the existence of the other.

Theorem 60. Let G be any graph. G has an odd closed walk if and only if it has an odd cycle.

Proof.

($\Leftarrow$)

² subject, of course, to any rules in force, e.g., in nature reserves or on private property

Suppose G has an odd cycle. Now, every cycle is a closed walk. So an odd closed cycle is an odd closed walk. Therefore G has an odd closed walk.

($\Rightarrow$)

Suppose G has an odd closed walk. Let W be the shortest odd closed walk in G .

The sequence of vertices of W may be written as

$$v_0, v_1, v_2, \dots, v_{k-1}, v_k = v_0,$$

where k is odd.

If no vertex of G except the start/end vertex repeats in W , then W is already an odd cycle, and we are done.

We now prove, by contradiction, that no vertex of G except the start/end vertex repeats in W . From this it will follow that W is actually an odd cycle.

Assume, by way of contradiction, that there is a vertex that is repeated in W . We don't want $v_0 = v_k$ to count as a repeat here, so we actually mean a vertex in the slightly shorter list

$$v_0, v_1, v_2, \dots, v_{k-1}$$

that is repeated.

Let v_i be the *first* vertex in this shorter list that appears again later in the list (but not earlier). Suppose it appears later in the shorter list as v_j . So $v_i = v_j$, and $0 \leq i < j \leq k-1$.

Using the vertices v_i and v_j , we can divide the closed walk W into two shorter closed walks:

- $v_i, v_{i+1}, \dots, v_{j-1}, v_j$.
 - $v_j, v_{j+1}, \dots, v_{k-1}, v_0, v_1, \dots, v_{i-1}, v_i$.
- Keep in mind here that $v_0 = v_k$.

Recall that the length of W is odd. Since we just divided W into two “subwalks”, the length of W must equal the sum of the lengths of those two subwalks. Therefore, one of those subwalks has odd length and the other has even length. Both are *closed* walks, so the one that has odd length is an odd closed walk. Since both the subwalks are shorter than W , we have constructed an odd closed walk in G that is shorter than W . This contradicts our choice of W as the shortest odd closed walk in G . So our initial assumption, that there is a vertex v_i with $i < k$ that is repeated in W , is wrong. Therefore there is no vertex repetition in W except for $v_0 = v_k$. Therefore W is an odd cycle. $\square$

This theorem would not be true if we replaced “odd” by “even” in the theorem statement. To see this, consider the complete graph on two vertices, K_2 . Call its vertices v and w . This graph has no cycles at all (even or odd). But it has an even closed walk: v, w, v . This closed walk is not a cycle, because the edge $\{v, w\}$ is used twice.

It is instructive to read through the proof of [Theorem 60](#) again to find the point where the proof would break down if “odd” were replaced by “even” throughout.

A cycle of length 3 in a graph is called a **triangle**. This is the shortest possible cycle a simple graph can have. But in a multigraph, we can have a cycle of length 2 formed by two different edges between the same pair of vertices. If a graph has a loop at a vertex, then that is a cycle of length 1.

Now let’s consider the other extreme of cycle length.

A cycle in a graph is **Hamiltonian** if it includes every vertex. Hamiltonian cycles are used in planning routes which must visit every location in some set, without visiting any location twice, and returning to the starting point. For example, suppose you must visit every city in some region exactly once, returning to your starting point, while using available transport links (roads or train lines, as the case may be) to go between cities. Then your problem is to find a Hamiltonian cycle in the graph whose vertices represent cities with edges representing available transport links between cities. In practice, each edge may have an associated distance or cost, and you may also want to find a Hamiltonian cycle with minimum total cost (assuming the total cost of the cycle is the sum of the costs of its edges). The problem of finding the Hamiltonian cycle of minimum total cost goes back to the 19th century and is traditionally known as the Travelling Salesman Problem (TSP).

11.9 CONNECTIVITY

Two vertices v and w in a graph G are **connected** if there is a path from v to w in G . This in turn holds if and only if there is a walk from v to w in G .

Because of this terminology, which relates vertices that may be a long way away from each other in a graph, the term “connected” must not be used as a synonym for “adjacent”.

A graph G is **connected** if, for every pair of vertices v and w , there is a path from v to w in G .

A **component** of G is a maximal connected subgraph of G , meaning that is not a proper subgraph of any other connected subgraph of G .

For example, in the graph in [Figure 11.1](#) has three components: one on the left, one in the middle, and one on the right. Here they are in (vertex-set, edge-set) format.

- $(\{a, b, c, d\}, \{\{a, b\}, \{a, c\}, \{b, c\}, \{c, d\}\})$
- $(\{e\}, \emptyset)$
- $(\{f, g\}, \{\{f, g\}\})$

The subgraph $(\{a, b, c\}, \{\{a, b\}, \{a, c\}, \{b, c\}\})$ — which is the triangle on the left in [Figure 11.1](#) — is not a component of G . Although it is a connected subgraph of G , it

is not a *maximal* connected subgraph of G because it is a proper subgraph of another connected subgraph of G , namely the first of the three listed above.

Connectivity gives a binary relation on the vertices of a graph. Let $G = (V, E)$ be a graph. Define the relation R on V as follows. For all $v, w \in V$,

$$vRw \iff \text{there is a walk from } v \text{ to } w \text{ in } G.$$

We claim this is an equivalence relation on V .

- It is reflexive because, for any vertex v , the zero-length walk relates v to itself, so vRv .
- It is symmetric because a walk from v to w can be reversed to give a walk from w to v . So $vRw \Rightarrow wRv$.
- It is transitive because, if uRv and vRw , then we have a walk from u to v and another walk from v to w , and we can put them together at v to make a walk from u to w , showing that uRw .

Since R is an equivalence relation, its equivalence classes partition V ([Theorem 12](#)). In fact, the equivalence classes are the vertex sets of the components of G .

11.10 BIPARTITE GRAPHS

A graph $G = (V, E)$ is *bipartite* if its vertex set V can be written as a disjoint union

$$V = A \sqcup B$$

of two parts, A and B , such that every edge of the graph has one endpoint in A and the other in B .³

We often draw bipartite graphs with all the vertices in one part on the left side and all the vertices in the other part on the right, with edges going between them as required. Two examples are given in [Figure 11.5](#).

A **complete bipartite graph** is a bipartite graph in which every pair of vertices on different sides are adjacent. This means that, if the two sides are A and B (as above), then the edge set is

$$\{\{a, b\} : a \in A \wedge b \in B\}.$$

If the two sides have p and q vertices then we denote the complete bipartite graph by $K_{p,q}$. The graph in [Figure 11.5](#) is the complete bipartite graph $K_{2,3}$.

Bipartite graphs are widely used, especially to model situations where there are two kinds of entities and we must consider how to allocate them to each other subject to constraints of some kind. For example, if A is a set of jobs and B is a set of candidates, then

³ This might not be a partition of V because one of the parts may be empty, although that could only happen if G had no edges. As long as the graph has at least one edge, then the sets A and B are both nonempty and they form the two parts of a partition of V .

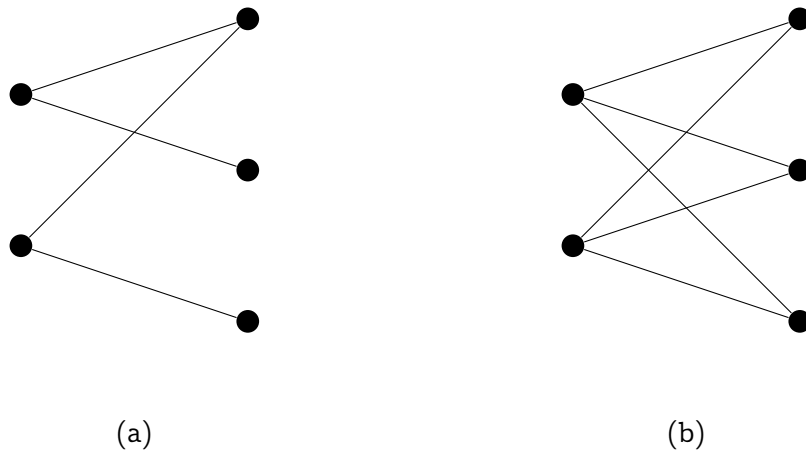


Figure 11.5: Two bipartite graphs. The one on the right, (b), is the complete bipartite graph $K_{2,3}$.

the edges of the graph may represent which candidates are suitable (or have applied) for which jobs. If A is a set of students and B is a set of subjects, edges could be used to represent which student is doing which subject, or which student wants to enrol in which subject. (The diagram of the employer function in [Figure 2.1](#) shows a graph of this type.)

Another way of viewing bipartite graphs is via 2-colourings.

A **2-colouring** of a graph $G = (V, E)$ is a function that assigns one of two colours to each vertex of G such that adjacent vertices are assigned different colours.

Theorem 61. A graph is bipartite if and only if it has a 2-colouring.

Proof.

($\Rightarrow$)

Suppose G is bipartite, and let A and B be the two sides as above, so all edges have one endpoint in A and the other in B . Define a function which colours all vertices in A Black and all vertices in B White. Then every edge has one endpoint coloured Black (because it is in A) and the other endpoint coloured White (because it is in B). Therefore adjacent vertices always get different colours. Therefore this function is a 2-colouring of G .

($\Leftarrow$)

Suppose $G = (V, E)$ has a 2-colouring $f: V \rightarrow \{\text{Black}, \text{White}\}$. Let $A = f^{-1}(\text{Black})$ be the set of all vertices coloured Black, and let $B = f^{-1}(\text{White})$ be the set of all vertices coloured White. Every vertex of V belongs to one of these two sets, since every vertex is mapped to one of those two colours. Furthermore, no vertex can belong to both sets, because f is a function and therefore cannot assign two different values to any element of its domain. Therefore $V = A \sqcup B$. Also, since adjacent vertices get different colours under

f , adjacent vertices must belong to different sets (A or B); they cannot both belong to the same set, for then they would be given the same colour by f . These properties of the two parts A and B show that G is bipartite with A and B as the two sides. $\square$

Theorem 62. A graph is bipartite if and only if it has no odd closed walk.

Proof.

($\Rightarrow$)

If G is bipartite, then every walk must alternate between vertices in A and vertices in B . So, it goes from a vertex in A , to a vertex in B , to one in A , to one in B , and so on, possibly with repetition of vertices and edges. At every stage, if we are at a vertex in A , then two steps later we are back in A . This alternation ensures that a *closed* walk has even length. Therefore G has no odd closed walk.

($\Leftarrow$)

If G has no odd closed walk, then let us colour the vertices of G as follows. In each component of G , we do the following.

1. Choose one vertex in the component as the “ground vertex” of that component; this will be the first vertex in the component to be coloured. Call it g .
2. Colour g Black.
3. Then colour every other vertex v of the component as follows.

$$\text{colour of } v = \begin{cases} \text{Black,} & \text{if the distance from } g \text{ to } v \text{ is even;} \\ \text{White,} & \text{if the distance from } g \text{ to } v \text{ is odd.} \end{cases}$$

In fact, colouring g Black is a special case of this, since 0 is even.

Once this is done, for all components of G , we will have given a colour to every vertex of G .

We claim that this is a 2-colouring of G . We prove this by contradiction.

Assume, by way of contradiction, that our assignment of colours is not a 2-colouring of G . Then there must be two adjacent vertices that get the same colour. Let X be this colour, which could be Black or White. Let v and w be these two adjacent vertices, each with colour X . Since they are adjacent, they must belong to the same component of G . Let g be the ground vertex for that component.

Key observation: the distances from g to v , and from g to w , have the *same parity*. In other words, they are both even or both odd. This is because the colours of v and w are completely determined by the parity of their distances from g , so the fact that they get the same colour implies that the parities of these distances are the same. Since these two distances have the same parity, *their sum is even*.

Let P be a shortest path in G from g to v , and let Q be a shortest path in G from g to w . The lengths of these two paths have the same parity, as explained in the previous paragraph. So

$$\text{length of } P + \text{length of } Q$$

is even. Now consider the *walk* formed by starting at v , going along P to g , then stepping along edge $\{v, w\}$ to w , then going along Q from w back to g , finishing up at g . This is a closed walk, since it finishes where it starts. Its length is

$$\text{length of } P + \text{length of } Q + 1,$$

where we now have an extra $+1$ because of the edge $\{v, w\}$. We have already seen that the sum of the lengths of P and Q is even. It follows that the length of this closed walk is odd.

So we have constructed an odd closed walk in G . But this is a contradiction, since this part of the proof starts with a graph with no odd closed walk. Therefore our assumption, that our assignment of colours is not a 2-colouring of G , is wrong. Therefore it is, in fact, a 2-colouring. Therefore G is 2-colourable. Therefore G is bipartite (using [Theorem 61](#)).⁴ $\square$

Corollary 63. A graph is bipartite if and only if it has no odd cycle.

Proof. This follows from [Theorem 60](#) and [Theorem 62](#). $\square$

11.11 EULER TOURS

An **Euler tour** is a closed trail that uses every edge. Since it is a *trail*, this means each edge gets used exactly once. Since it is closed, it finishes at its starting vertex.

This concept goes back to what is often considered to be the start of graph theory.⁵

In the city of Königsberg in Prussia (now Kaliningrad, in Russia), there were seven bridges linking two islands in the River Pregel to each other and to the two sides of the river. It was a popular pastime to try to do a walk which crossed each bridge exactly once and returned to its starting point. No-one had succeeded in doing this, so it was widely believed to be impossible, but no-one had pinned down why it couldn't be done. The great Swiss-German mathematician Leonhard Euler tackled this problem, modelled it using a graph, developed some theory, and solved it, showing that such a tour around the seven bridges was impossible. (Euler also developed some of the number theory we

⁴ We only used 2-colourings in this part of the proof since it seemed easiest and neatest to write about giving colours to vertices rather than putting them in A or B . But it would not have made much difference. Had we done everything in terms of A and B , we would not have needed to appeal to [Theorem 61](#) at the end.

⁵ This may be true for graph theory as a field of study. But there are graph-theoretic definitions and concepts going back much further in time. For example, connectivity is used in the ancient game of Go, known as Igo in Japan, Baduk in Korea and Wéiqí in China. This game was invented in China about 2,500 years ago, and can be played on any graph.

have done: § 7.12 and Theorem 38.) He first presented his work to the St Petersburg Academy of Sciences in 1735 and it was published in 1736.

A map showing the seven bridges, taken from Euler's paper, is shown in Figure 11.6.⁶

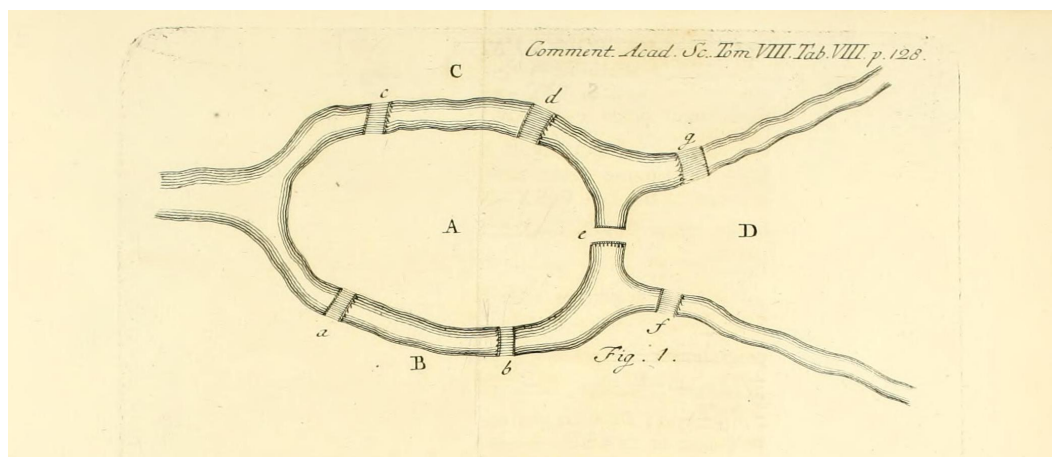


Figure 11.6: Diagram of the Seven Bridges of Königsberg, from Leonhard Euler's paper published in 1736.

From this geographic setting, Euler constructed the graph shown in Figure 11.7. This is a classic case of abstraction: identifying the essential elements of the problem at hand (the relationship between the bridges and the land masses) and omitting all irrelevant information (the sizes and shapes of the land masses, the lengths and widths of the bridges, the curves of the river, etc.).

The graph is a precise model of the Königsberg Bridges problem, since a walk around all the bridges of the required type exists if and only if the graph has an Euler tour.

At this point, you can try to construct an Euler tour for this graph and, through this exploration, try and gain some insight as to why it is not possible.

Euler not only solved the problem for the Königsberg Bridges graph, but provided a general characterisation of graphs that have Euler tours, giving a simple test that does not require an exhaustive search.

Theorem 64 (Euler, 1736). A connected graph G has an Euler tour if and only if every vertex has even degree.

Proof.

($\Rightarrow$)

⁶ Diagram taken from: Leonhard Euler, *Solutio problematis ad geometriam situs pertinentis*, *Commentarii Academiae Scientiarum Imperialis Petropolitanae*, vol. 8 (1736) 128–140, with figures between pp. 158 & 159; this copy is from the copy in the Biodiversity Heritage Library, <https://www.biodiversitylibrary.org/>.

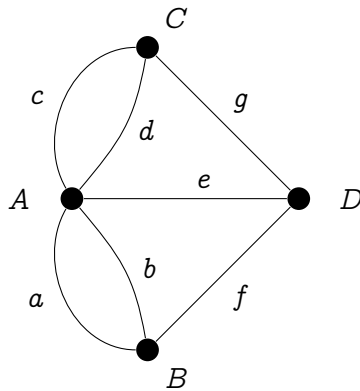


Figure 11.7: A graph representing the Seven Bridges of Königsberg.

Let G be a connected graph with an Euler tour. An Euler tour is a trail, and when a trail passes through a vertex, it uses two of the edges incident with that vertex: one on the way in, and one on the way out. An Euler tour uses each edge of the graph exactly once. It follows that it must revisit each vertex as many times as necessary in order to use each of its incident edges exactly once, and it uses two edges for each visit, so the number of incident edges at the vertex must be a multiple of 2. Therefore each vertex degree is even.

This reasoning applies to the vertex where the tour starts and ends, too, provided we consider the tour to be entering the vertex at the end (via the last edge of the tour) and leaving it at the start (via the first edge of the tour). Alternatively, we can treat that vertex as a special case, noting that we use one of its incident edges at the start and another at the very end, making two edges so far. Then, every other visit to that vertex uses two of its incident edges, so again, the number of incident edges at that vertex must be a multiple of 2, by the reasoning of the previous paragraph.

($\Leftarrow$)

Let G be a connected graph in which every vertex has even degree.

We give an algorithm for constructing an Euler tour for G .

1. Initialisation:
 - a) Choose any vertex v of G .
 - b) Let T be the trivial closed trail consisting just of the vertex v , with no edges.
2. While there is at least one edge we have not yet used in T :
 - a) Find a vertex in T that has at least one edge that we have not yet used in T . Call it v .
 - There must be at least one such vertex. To see this, let e be any edge that we have not yet used (which might be elsewhere in the graph; it might

not be incident with any vertex of T). If e has an endpoint in T , then this endpoint is already a vertex in T with at least one incident unused edge. So suppose both the endpoints of T are not in T . Let u be one of the endpoints of T and let w be any vertex in T . Since G is connected, there is a path P from u to w in G . Let v be the first vertex of T that we encounter on P , as we go along it from u towards w . (This might be w itself, if the path doesn't meet T until it ends up at w , or it might be somewhere else in T .) The edge of P that we go along to arrive at v for the first time cannot be in the closed trail T , else its other endpoint is in T too and we would have met T earlier. So this vertex v is a vertex in T with at least one incident edge not in T .

- b) We now construct a new closed trail U that starts and finishes at v and which has no edges in common with T (although it can use vertices of T , including v). Starting at v , walk along an edge incident at v that is not in T . (We established above that this is possible.) Continue walking along edges not in T . Since T is a closed trail, it uses an even number of edges at each vertex, so there will always be an even number of unused edges at each vertex. So, if in U we enter a vertex via an unused edge, there must be another unused edge which we can use in order to leave that vertex. Therefore we can always keep going, unless we return to v , in which case it's possible we may have no more unused edges from v to use. This new closed trail can only stop at v , and because the graph is finite, it must therefore stop at v eventually. So we have constructed a new closed trail, U , starting and finishing at v , which has no edges in common with T (because we only ever used edges that were unused by T).
 - c) We now combine the closed trails T and U to make a new closed trail. Starting anywhere on T , we follow T until we first reach v . Then we follow U all the way until we return to v for the last time, having stepped along all edges of U . Then we resume following T , and we follow it all the rest of the way until we return to our starting vertex for the last time, having used every vertex of T .
 - d) We now redefine T , so that it is now the new name for this combined closed trail, formed from the old T and the extra closed trail U .
3. The algorithm finishes when there is no edge of G that has not been used by the closed trail T . Then, the closed trail T necessarily uses each edge of G exactly once, and it is therefore an Euler tour. So we output T .

□

Motivated by this theorem, we call a graph **Eulerian** if it contains an Euler tour. With this terminology, [Theorem 64](#) says that a connected graph is Eulerian if and only if every vertex of it has even degree.

Theorem [Theorem 64](#) also works for graphs with multiple edges. In fact it also works for graphs that may have loops, provided that each loop contributes 2 to the degree of its vertex. (Why?)

Euler tours can be used in planning routes in other contexts, too.

Suppose you and a colleague are delivering items into mailboxes outside people's houses in some area. You walk along one side of each street, and your colleague walks along the other side in the same direction, keeping roughly alongside each other. You would prefer to not walk again along any stretch of road where you have already done your deliveries, because that seems like wasted time and effort.

You can model this situation by a graph, where vertices represent intersections of streets and edges represent segments of streets between intersections (with these segments having no other intersections in the middle of them, otherwise they are really more than one segment). You would like to find an Euler tour in this graph.

The Euler tour problem has various generalisations that also have practical applications. For example, you could try to find a closed walk which includes every edge and re-uses the fewest edges. Or the edges might be weighted, and you could try to find the closed walk of minimum total weight.

If a connected graph G does not have an Euler tour, then it must have some vertices of odd degree. By [Corollary 59](#), the number of vertices of odd degree must be *even*. Suppose that the number of vertices of odd degree is $2k$, where $k \in \mathbb{N}$. We can, if we wish, pair these odd-degree vertices off with each other, however we like. Let us call the vertices in the i -th pair v_i and w_i . So the pairs are

$$\begin{array}{ccc} v_1 & \text{and} & w_1, \\ v_2 & \text{and} & w_2, \\ \vdots & \vdots & \vdots \\ v_k & \text{and} & w_k, \end{array}$$

making $2k$ odd-degree vertices altogether. Note, each of these pairs may or may not be adjacent.

Now suppose we add new edges linking each of these pairs of vertices: one between v_1 and w_1 , another between v_2 and w_2 , and so on, making k new edges altogether. This may create some multiple edges, but that's ok. Once we've done this, *all degrees are even*. By [Theorem 64](#) (or rather, its extension to multigraphs), this new graph has an Euler tour. This tour includes each of the edges of G exactly once, plus each of the k new edges exactly once. If we then delete all those new edges, then the tour is broken into k trails such that each edge of G appears exactly once in exactly one of these trails (and not at all in any of the others). In other words, these trails give a *partition* of the edges of G . Each of these k Euler trails must start at one vertex of odd degree and end at another vertex of odd degree.

We have shown that the edge set of a graph G with $2k$ vertices of odd degree can be partitioned into k trails, and that the $2k$ endpoints of the trails are precisely the $2k$ odd-degree vertices of the graph. Furthermore, it is not possible to partition the edges of G into fewer than k trails. (Proving this, using the concepts we have been discussing, is a good exercise.)

We have outlined the proof of the following extension of [Theorem 64](#).

Theorem 65. For any $k \in \mathbb{N}$, the edges of a graph can be partitioned into k trails if and only if the graph has at most $2k$ vertices of odd degree. The open trails in such a partition start and end at odd-degree vertices. $\square$

11.12 EXERCISES

1. Let V and E be two finite sets. We'll call the members of V "vertices" and the members of E "edges".

Suppose that

- v, w and z are variables with domain V ;
- e , and f are variables with domain E ;
- *incident* is a binary predicate whose first argument has domain V and whose second argument has domain E .

It is our *intention* to use these ingredients (i.e., the given sets V and E , the variables and the predicate) to describe graphs and some of their properties. In particular, we *intend* that $\text{incident}(v, e)$ means that the vertex v is incident with the edge e . But we need to establish logical rules to ensure that these intentions are actually carried out.

Write expressions in predicate logic with the following meanings.

- (a) Every edge is incident with at least one vertex.
- (b) No edge is incident with exactly one vertex (i.e., there are no loops).
- (c) Every edge is incident with exactly two vertices.
- (d) There are no multiple edges.
- (e) (V, E) represents a simple graph.
- (f) There are no isolated vertices.

2. Consider the four types of graph representation we looked at in [§ 11.4^α](#): edge lists, adjacency matrices, adjacency lists, and incidence matrices. Suppose that

- the graphs we want to represent have n vertices and m edges,
- their vertices are named 1 to n ,
- we represent vertex numbers in binary, using $\lfloor \log_2 n \rfloor + 1$ bits for each vertex number. (This is the number of bits required to represent n in binary, and we assume that smaller numbers have extra leading zeros if necessary to ensure that all vertex numbers have the same number of bits.)

Under these assumptions, determine the total number of bits used to represent a graph using

- an edge list, with the vertex sets and edge sets listed in full;
- an adjacency matrix;
- an adjacency list;
- an incidence matrix.

In each case, then give a simple upper bound in big-O notation, again just in terms of n and m , and without using the floor or ceiling functions.

3. Rewrite the proof of [Theorem 56](#) so that the two cases are based on whether or not the graph has an isolated vertex.
4. Determine (a) the number of edges, and (b) the average degree of the graph of the caffeine molecule.
Here, vertices represent atoms and edges represent bonds. The graph has some multiple edges, in the form of double bonds. Its chemical formula is $C_8H_{10}N_4O_2$. In molecules, the **valency** of an atom is the number of bonds it has; this is just the degree of the corresponding vertex of the graph that represents the molecule.⁷ The valencies of Carbon, Nitrogen, Oxygen and Hydrogen are 4,3,2,1, respectively.
5. Prove that the shortest walk between two vertices is always a path.
6. Prove that every graph with minimum degree at least 2 has a cycle.
7. Prove that the distance between two vertices in a graph satisfies the **triangle inequality**. This means that, for every triple of vertices u, v, w in any graph G ,

$$d_G(u, w) \leq d_G(u, v) + d_G(v, w),$$

where $d_G(x, y)$ denotes the distance between vertices x and y in G .

⁷ In fact, some graph theorists borrowed the term *valency* from chemistry and used it instead of “degree”, but that is uncommon these days.

8. How many 2-colourings does a bipartite graph with n vertices, m edges and k components have?
9. The following puzzle once circulated at a Victorian high school.

Consider the four-vertex multigraph in [Figure 11.8](#). Determine if it has a trail that includes every edge, and also if it has an Euler tour.

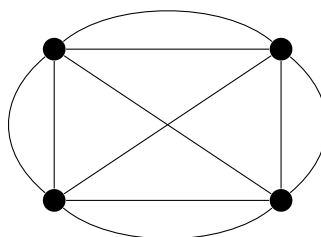


Figure 11.8: A multigraph.

10. Each of the five platonic solids (tetrahedron, cube, octahedron, dodecahedron, icosahedron) has a skeleton consisting of its vertices and edges, which is a graph. In fact, the graph terminology “vertex” and “edge” came from their use for polyhedra.

For each of these five graphs: find a Hamiltonian cycle; determine if it is Eulerian, and if it is, find an Euler tour.

11. Let Q_n be the graph whose vertices are all strings of n bits, with two vertices being adjacent if they differ in just one bit.

- Draw the cube, Q_3 .
- Find a Hamiltonian cycle in Q_3 .
- Describe a method for constructing a Hamiltonian cycle in Q_n , for each n . Think recursively.
- Find an Eulerian cycle in Q_4 .

12. Using [Theorem 64](#) for simple graphs, how would you prove it for multigraphs (where multiple edges are allowed)?

To answer this, don’t re-do the *proof* of [Theorem 64](#). Treat that theorem just as a black box. So, your task is to show that a connected multigraph is Eulerian if and only if every vertex has even degree, *using* [Theorem 64](#) somehow. Since [Theorem 64](#) is only about simple graphs, you should try to construct, from any given multigraph G , a

simple graph H such that applying [Theorem 64](#) to H helps prove this extension of the theorem for G .

GRAPH THEORY II

We continue our study of graphs by looking at two of the most important classes of graphs: trees (and the closely related class of forests), which are ubiquitous in computer science, and planar graphs, which are fundamental in many applications including network layout and information visualisation.

We finish our exploration of graph theory by focusing on fun, through some games that can be played on any graph. These games have a rich theory and are the source of many good puzzles and challenges as well as being interesting to play.

12.1^α TREES

A **tree** is a connected graph without cycles.

An example of a tree is given in [Figure 12.1](#).

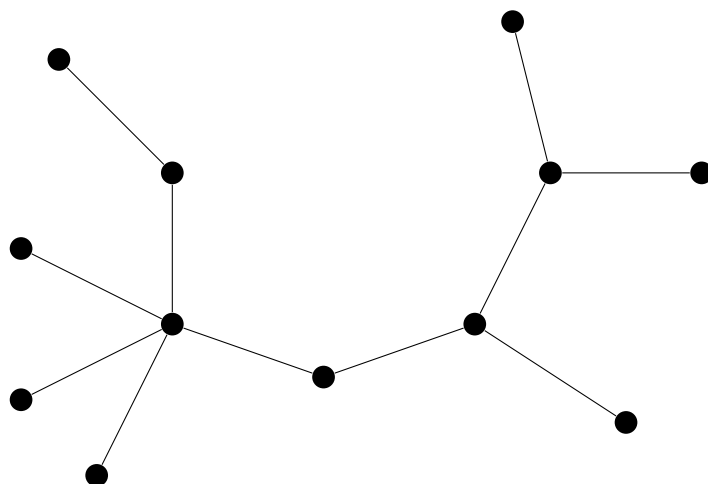


Figure 12.1: A tree.

Some of the contexts in which trees are used as abstract models include:

- communications networks. It can be useful to identify a subset of the links which, together, still enable any node to communicate with any other node. A minimal subset of links that does this turns out to be a tree that reaches every node.
- hierarchies. In a hierarchy, each object depends on a unique “higher” object, while also potentially having other objects that depend on it. Examples include organisational hierarchies¹, taxonomies in biology, and parts of family trees. Computer science itself is rich in hierarchies:
 - In many operating systems (and historically inspired by Unix), the files are organised into directories which form a hierarchy which is a tree.
 - In *object-oriented* programming languages, the possible data types — represented by *classes* — are organised into *inheritance hierarchies*. This means that objects that belong to a class *inherit* the functionality specified in other classes higher up the hierarchy. This kind of programming has proved very beneficial for promoting reusability and maintainability of software, and is especially useful when writing programs for simulating real-world processes and systems.

For inheritance hierarchies to truly be trees, they must conform to *single inheritance*, where each class can only have one superclass. Many object-oriented languages are restricted to single inheritance. But some languages allow *multiple inheritance*, where a class can have more than one superclass. In such cases, the inheritance hierarchies are not trees.

Python supports object-oriented programming.

- decision-making. Suppose there is a collection of situations, each requiring a choice to be made between several options. Each choice leads to another situation, requiring another choice, and so on, until some final outcome is reached. If each situation can be reached by only one sequence of choices, then a tree is a suitable model.
- text analysis. The classical example of this is the use of trees in grammars, for human languages or programming languages. A *parse tree* shows how a string of text can be generated according to grammatical rules.
- trees, in the botanical sense!

Trees are used extensively as data structures in computer programs, to help sort, store and retrieve information.

In some of these examples, the trees have a special vertex called the **root** on which everything else ultimately depends. For example, in a directory hierarchy in Unix or Linux, there is a *root directory*, denoted simply by /, which (unlike all other directories)

¹ assuming each employee has only one supervisor

has no parent directory. Try finding it in your Linux environment, in an Ed Workspace, and explore its contents and subdirectories. In graph theory, a tree with a root is called a **rooted tree**.

We will focus on normal (unrooted) trees, because they are important in their own right, and because they are fundamental to using and understanding rooted trees.

12.2 PROPERTIES OF TREES

When mentioning communications networks (p. 422 in § 12.1^α), we said that trees arise as minimal networks that keep everything connected. We now formalise and prove this, as a general statement about trees.

Suppose a graph $G = (V, E)$ is connected, but deleting *any* edge of the graph disconnects it (but we retain all the vertices). Such a graph can be called a *minimal connected graph on V* , since it's connected but every proper subgraph *with the same vertex set* is disconnected. It turns out that minimal connected graphs on V are the same as trees with vertex set V .

Theorem 66. A graph $G = (V, E)$ is a tree if and only if it is a minimal connected graph on V .

Proof.

($\Rightarrow$)

Let G be a tree. By definition, it is connected. If it is *not* a *minimal* connected graph on V , then there is some edge e of G such that the graph remains connected even if e is deleted. Let v and w be the endpoints of e , so that $e = \{v, w\}$. Since G remains connected after e is deleted, there must be a path from v to w that does not use the edge e . But adding e to this path creates a cycle in G . Since G contains a cycle, it is not a tree, which is a contradiction.

($\Leftarrow$)

Let G be a minimal connected graph on V .

Since G is already connected, we only need to show that it has no cycle. Then we will know it is connected and has no cycle, which means it's a tree.

Assume, by way of contradiction, that G has a cycle C . Let e be any edge in C . Now, let v and w be any vertices in G . Since G is connected, there is a walk from v to w in G . If this walk includes e , then we can construct an alternative walk from v to w that avoids e by going all the way round C instead. Since we can do this for any v and w , it follows that every pair of vertices in G are connected by a walk that avoids e .

Therefore, in the proper subgraph of G obtained from it by deleting e , every pair of vertices are connected by a walk (and hence by a path). Therefore this proper subgraph of G is connected. It also has the same vertex set, V , as G . So this subgraph contradicts the minimality of G . Therefore our assumption, that G has a cycle, was wrong. So G has no cycle. $\square$

So, in the context of a communications network, a tree may be viewed as a minimal or “bare bones” network that links all nodes. This view is reinforced by the observation that, in a tree, every pair of vertices is connected by a *unique* path. In fact, this property gives another characterisation of trees (see [Exercise 5](#)).

Trees are very diverse. The two most “extreme” trees on n vertices are

- the path graph P_{n-1} , with n vertices and $n-1$ edges. This tree contains the longest path of any tree on n vertices. No other tree on n vertices has a path of length $n-1$. The maximum vertex degree of P_{n-1} is 2; every other tree on n vertices has higher maximum degree.
- the *star graph* consisting of one central vertex and $n-1$ leaves, each adjacent to the central vertex. Using complete bipartite graph notation, this is $K_{1,n-1}$. By contrast with the path graph, its paths are extremely short. It has no path of length > 2 ; no other tree has this property. Its maximum degree is $n-1$; every other tree on n vertices has lower maximum degree.

[Figure 12.2](#) shows three trees on five vertices: a path, a star, and another shown between them. In fact, this is a complete list of the structures that trees on five vertices can have; every tree on five vertices may be identified as one of these three (possibly after renaming and redrawing to make the relationship clear).

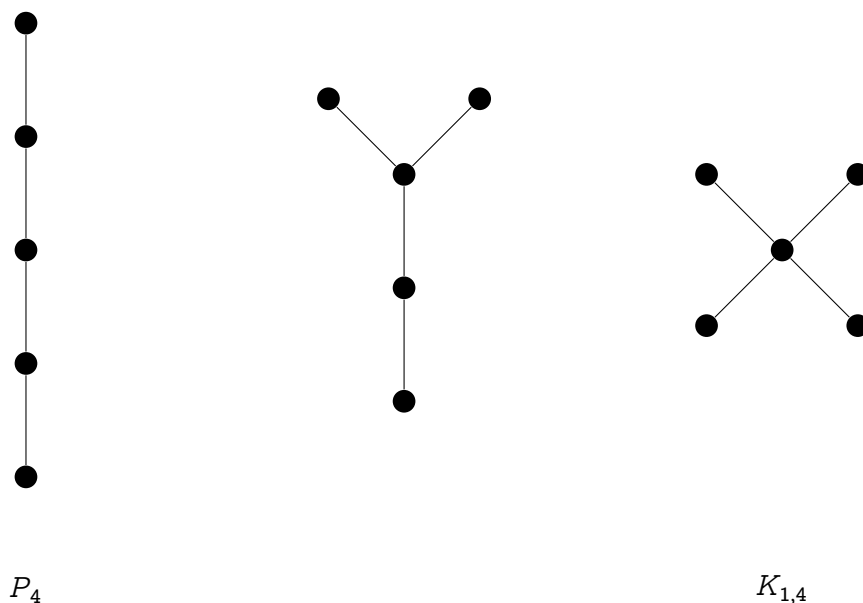


Figure 12.2: Three trees on five vertices.

See if you can go beyond [Figure 12.2](#) by drawing all possible trees on *six* vertices. You will start to see more of the variety of shapes they have. You should also study their

structure and see if you can make some general observations about the structure of trees. How many edges does a tree on n vertices have? What values can their average degree take? What values can the maximum degree of a vertex in a tree take? What values can the minimum degree take? What possible values can the length of the longest path take?

We now answer some of these questions. First, we consider minimum degree.

Theorem 67. Every tree with at least two vertices has a leaf

Proof. See Exercise 11.6. □

We now consider the number of edges in a tree.

Theorem 68. For every $n \in \mathbb{N}$, every tree on n vertices has $n - 1$ edges.

Proof. Let $P(n)$ be the following statement:

Every tree on n vertices has $n - 1$ edges.

We prove, by induction on n , that $P(n)$ is true for all n .

Inductive Basis:

When $n = 1$, we only have one vertex, and there is only one graph with one vertex, and it has no edge. This is the simplest tree, and its number of edges is 0, which is $1 - 1$, so the number of edges is indeed $n - 1$ in this case. So $P(0)$ is true.

Inductive Step:

Let $k \geq 1$.

Assume that $P(k)$ holds, i.e., that every tree on k vertices has $k - 1$ edges. This is our Inductive Hypothesis.

We need to show that $P(k + 1)$ holds. This is a statement about all trees on $k + 1$ vertices, so it's a universal statement. So our proof strategy for the Inductive Step is to consider a general tree on $k + 1$ vertices and prove that it has the required property, looking out for a chance to apply the Inductive Hypothesis.

Let T be any tree on $k + 1$ vertices. By Theorem 67, T has a leaf.

We now make a new tree S from T by deleting a leaf from it (which means we remove the leaf vertex and also its incident edge). S is indeed a tree, because removing a leaf from a graph never disconnects it and never creates a cycle.

This new tree S has one fewer vertex and one fewer edge than T , because of the deletion of the leaf (and its incident edge). Since T has $k + 1$ vertices, this means S has k vertices. This in turn means that we can apply the Inductive Hypothesis to S .

By the Inductive Hypothesis, S has $k - 1$ edges.

Now we use the relationship between S and T . Since S has one fewer edge than T (due to the way it was constructed from T), the fact that S has $k - 1$ edges tells us that the number of edges of T is

$$(k - 1) + 1 = k.$$

Therefore T has k edges. So $P(k+1)$ holds.

Summarising, we started with the assumption that $P(k)$ holds and deduced from this that $P(k+1)$ holds. In other words, we proved $P(k) \Rightarrow P(k+1)$. So we have completed the Inductive Step.

Conclusion:

Therefore, by the Principle of Mathematical Induction, $P(n)$ holds for all $n \in \mathbb{N}$. $\square$

12.3 FORESTS

A **forest** is just a graph with no cycles.

So a forest is like a tree, except that we don't require it to be connected.

Even though a forest does not have to be connected, it will (like any graph) have components, and those components are all connected. The components cannot have any cycles, since any cycle in a component is also a cycle in the whole graph. So each component in a forest is connected and has no cycles. Therefore each component of a forest is a tree.

So a forest is just a collection of trees, and this explains where the name comes from.

The graph in [Figure 11.2](#), early in the previous chapter, is a forest with three components. Every tree is a forest, even though it only has one component. So the graphs in [Figure 11.5\(a\)](#) and [Figure 12.1](#) are forests (because they are trees).²

[Theorem 68](#) can be extended to forests.

Theorem 69. For every $n \in \mathbb{N}$, every forest with n vertices and k components has $n - k$ edges.

Proof. Let F be a forest and let $F_1, F_2, \dots, F_k$ be its components.

Let n and m be the numbers of vertices and edges, respectively, of F . For each i , let n_i and m_i be the numbers of vertices and edges, respectively, of the i -th component, F_i .

We can express the number of vertices of F as the sum of the numbers of vertices of the components,

$$n = n_1 + n_2 + \dots + n_k = \sum_{i=1}^k n_i. \quad (12.1)$$

We can treat the numbers of edges similarly:

$$m = m_1 + m_2 + \dots + m_k = \sum_{i=1}^k m_i. \quad (12.2)$$

By [Theorem 68](#) we have, for each i ,

$$m_i = n_i - 1.$$

² But the graph in [Figure 11.5\(b\)](#) is certainly not a tree or a forest.

Let us use these observations to work out the number of edges of F .

$$\begin{aligned}
 m &= \sum_{i=1}^k m_i && \text{(by (12.2))} \\
 &= \sum_{i=1}^k (n_i - 1) && \text{(by applying Theorem 68 to each component)} \\
 &= \left(\sum_{i=1}^k n_i \right) - \left(\sum_{i=1}^k 1 \right) \\
 &= n - k && \text{(by applying (12.1) to the first sum).}
 \end{aligned}$$

□

12.4 SPANNING TREES

Let G be a connected graph. A **spanning tree** of G is a subgraph of G which

- is a tree, and
- includes every vertex of G .

In Figure 12.3, we show a graph at the top, followed by two further drawings of it with two different spanning trees of the graph. (This graph is the same as the one in Figure 11.4.) See if you can find some other spanning trees for this graph.

Since a tree is a minimal connected graph on its vertex set (Theorem 66), a *spanning tree* of G is a minimal connected subgraph of G that includes every vertex of G . In other words, it's a subgraph of G in which every vertex is connected to every other vertex *but* deleting any edge from the *subgraph* disconnects the *subgraph*.

We can find a spanning tree of a connected graph G by deleting edges (but not vertices) until we can't delete any more edges without disconnecting the subgraph. We can usually do this in many different ways. Often, this process will give different spanning trees, but sometimes we will get the same spanning tree by deleting the same edges but doing so in a different order.

For example, in the graph of Figure 12.3, suppose we delete edge $\{a, b\}$, then $\{c, d\}$, then $\{b, e\}$. You can check that, at each stage in this process, the remaining graph is still connected (and includes all the vertices, since we are not deleting vertices). After we have deleted all these three edges, we have the spanning tree in the middle diagram. Order of deletion does not matter here; we could have instead deleted $\{c, d\}$, then $\{b, e\}$, then $\{a, b\}$, and we would still end up with the same spanning tree.

If, instead, we delete $\{d, e\}$, then $\{a, c\}$, then $\{c, d\}$, then we get the spanning tree shown at the bottom of the figure.

This always works, for any connected graph. It follows that every connected graph has a spanning tree.

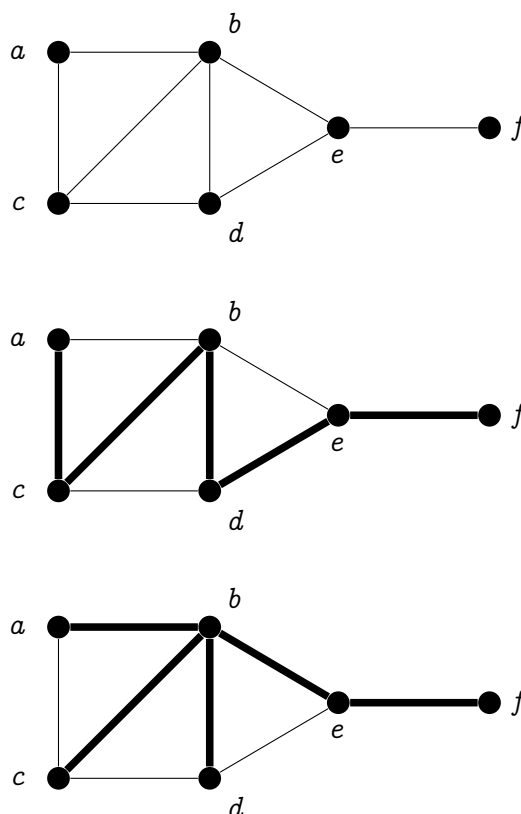


Figure 12.3: A graph (top), and two of its spanning trees, shown with thick edges (middle and bottom).

This method of finding a spanning tree starts with the entire graph and deletes edges until we are left with a spanning tree. Another method of finding a spanning tree in a graph works in the opposite way: we start with *nothing* and *build up*. We now describe this.

At the start, we pick any edge of the graph. (If the graph has no edge, then it has just one vertex and nothing else, otherwise it is not connected. A one-vertex graph is its own spanning tree, so in that case there is no need to do anything.) Then we keep trying to add edges, to the graph we have built up so far, provided that this *does not create a cycle*. While we are doing this, the subgraph we are building must be a forest (as it has no cycle), but it might not be a connected subgraph, so it might not be a tree. Eventually, the process stops, when it is no longer possible to add another edge without creating a cycle. When that happens, it turns out that we will have a spanning tree of the original graph.

Let us now set this out as an algorithm.

1. Input: a connected graph G .

2. $X := \emptyset$
3. while $\exists e \in E \setminus X$ such that $X \cup \{e\}$ does not contain a cycle:
 - 2.1. $e :=$ any edge $e \in E \setminus X$ such that $X \cup \{e\}$ does not contain a cycle
 - 2.2. $X := X \cup \{e\}$
4. Let F be the graph (V, X) , which is a subgraph of G .
Output: F .

This algorithm starts with a set of edges containing no cycle, and it only ever adds edges that do not create a cycle. Therefore the output subgraph F , with vertex set V and edge set X , has no cycles. Therefore it is a forest.

Since G is connected, the output subgraph F is connected too. (It is a good exercise to try to prove this.) Since F is a connected forest, it is a tree, and since it is also a subgraph of G that includes all vertices of G , it is a spanning tree of G .

If G is disconnected, then the algorithm can be applied to each component of G to construct a spanning tree in each component.

Suppose you have a set V of nodes that need to be joined up somehow into a network. Applications include planning for transport or communications. There is a set E of possible links that could be built between pairs of nodes, and each possible link carries a cost. How do you find the minimum cost network that connects all the nodes?

We model this by a graph $G = (V, E)$, where the vertices represent nodes in the network and the edges represent pairs of nodes which could be linked. We assume G is connected, otherwise it's impossible to find a subgraph that connects all the vertices. For each edge e there is a cost $w(e) \in \mathbb{R}^+$ which represents the cost of building that link. We want to find a subset X of the edges such that

- (V, X) is connected, so that every pair of vertices has a path using only edges in X ; and
- the total cost $w(X)$ is minimum, where the total cost of X is just the sum of all the costs of the edges in X :

$$w(X) = \sum_{e \in X} w(e).$$

In order to minimise the total cost, we do not want to include any unnecessary edges in X . Suppose X contains an edge e such that $X \setminus \{e\}$ still connects all vertices of G . Then we prefer to omit e , since that reduces the total cost by $w(e)$, the cost of e . So, any minimum-cost subgraph (V, X) will not only be a connected subgraph of G , but will also be a *minimal* connected subgraph that includes all vertices of G . As we remarked

early in this section, a minimal connected subgraph that includes all vertices is just a spanning tree of G . So what we want is a minimum-cost spanning tree of G .

Our earlier algorithm for finding a spanning tree takes no notice of any costs on the edges. So, in general, it won't find a minimum-cost spanning tree.

We can easily modify the algorithm to make use of the edge costs. In the key step where the next edge is chosen (step 2.1), we could choose the edge of minimum cost, among all those not in X which would not create a cycle if we chose them. This algorithm is known as **Kruskal's Greedy Algorithm**, after Joseph Kruskal who introduced it in 1956. It is *greedy* because it always makes the choice that gives the greatest immediate benefit, rather than looking further ahead to see what gives greatest benefit overall.

Here is the Greedy Algorithm in full, which is the same as the previous algorithm except for the part shown in blue, where we choose the next edge greedily instead of arbitrarily.

Kruskal's Greedy Algorithm

1. Input: a connected graph G .
2. $X := \emptyset$
3. while $\exists e \in E \setminus X$ such that $X \cup \{e\}$ does not contain a cycle:
 - 2.1. $e :=$ an edge $e \in E \setminus X$ such that $X \cup \{e\}$ does not contain a cycle, and $w(e)$ is minimum among all edges with this property
 - 2.2. $X := X \cup \{e\}$
4. Let F be the graph (V, X) , which is a subgraph of G .
Output: F .

Remarkably, this algorithm is actually *guaranteed* to find a minimum cost spanning tree.

Theorem 70. For every weighted graph G , Kruskal's Greedy Algorithm finds a minimum cost spanning tree in G . $\square$

This is not at all obvious, and very unexpected! Usually, in designing algorithms, a greedy approach does not give optimum solutions. Repeatedly maximising short-term benefit seldom gives maximum long-term benefit. For example, if you want to find the shortest path between two vertices in a weighted graph, with edge weights representing lengths or travel times, then a greedy approach will not work, in general, and may even fail to find *any* path. (Try it on some examples.)

We will not prove [Theorem 70](#), but finding a proof is a good challenge for a strong, keen student.

The Greedy Algorithm's combination of simplicity, efficiency and optimality makes it a very powerful tool for programmers and problem-solvers. Although it is rare for

greedy algorithms to find optimal solutions, the benefits when that does happen can be enormous. So it is important to be able to recognise those situations. A whole branch of mathematics, *matroid theory*, is based on the study of structures for which the greedy algorithm always finds optimum solutions.

12.5 PLANARITY

Graphs often need to be presented on a flat, two-dimensional surface.

- To help people study graphs visually, they are usually displayed on a screen or drawn/printed on paper.
- Transport networks involve laying out routes between locations in a geographical area.
- Electrical circuits are laid out on flat surfaces such as circuit boards or wafers, with wires routed along these surfaces.

If possible, it is desirable to avoid, or at least minimise, edge-crossings. This is because they introduce costs. In diagrams, the relationships among components may be obscured by edge-crossings, so these crossings impose a cognitive cost on human readers. In transport networks, edge-crossings may need to be implemented as bridges, tunnels or intersections, which may be expensive to build or operate. In electrical circuits, edge-crossings introduce extra manufacturing costs in order to prevent short-circuits between wires.

It is therefore important to understand graphs that can be represented on a flat, two-dimensional surface with no crossings at all. Geometrically, this means drawing them on the plane such that

- the vertices are represented as distinct points on the plane;
- each edge $\{v, w\}$ is drawn as a curve between v and w which does not intersect itself anywhere, meets v and w only at the ends of the curve, and meets no other vertices at all;
- the curves representing different edges cannot meet each other at all, except if the two edges are incident at a common vertex, in which case they only meet at that vertex.

A graph that can be drawn in this way is **planar**. Such a drawing in the plane is called a **planar drawing** of the graph, or a **plane graph**. These definitions extend to multigraphs too.

If you review the graphs illustrated in this and earlier chapters, you will find that most of them are planar, and are drawn without crossings, so that the illustrations give planar drawings of them. A few particular cases are worth comment.

- The drawing of K_4 on the left of Figure 11.3 is not planar, since it has an edge-crossing. But the graph K_4 itself is planar. Can you find a planar drawing of it?
- The drawings of two bipartite graphs in Figure 11.5 each have edge-crossings, so they are not planar drawings. But both graphs are planar. Can you show this, using an appropriate drawing?
- The multigraph in Figure 11.8 is planar, although the drawing given there is not a planar drawing since there is an edge-crossing in the middle. How can you redraw it so that there are no crossings?

Null graphs are of course planar, since they have no edges to form crossings with. Paths, cycles and trees are always planar. What about two other special families: complete graphs, and complete bipartite graphs?

- complete graphs: K_1, K_2, K_3 and K_4 are all planar. For K_1 and K_2 , this is because they do not have enough edges to form edge-crossings. For K_3 , it is a special case of the fact that all cycles are planar. Hopefully you have just shown that K_4 is planar, using a different drawing to the one given on the left in Figure 11.3. But what about K_5 ?
- complete bipartite graphs: $K_{1,k}$, the star graph, is a tree, and therefore planar. $K_{2,2}$ is really the same as the 4-cycle, C_4 , and is therefore planar. Hopefully you have now redrawn $K_{2,3}$ (see Figure 11.5(b)) to show that it is planar. But what about the next two cases, $K_{2,4}$ and $K_{3,3}$?

Figure 12.4 shows K_5 and $K_{3,3}$.

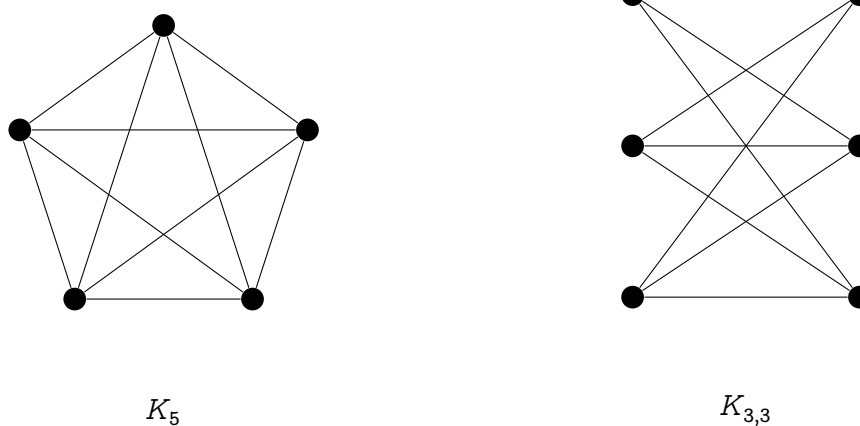


Figure 12.4: K_5 (left) and $K_{3,3}$ (right).

The planarity, or otherwise, of K_5 and $K_{3,3}$ has been the basis of numerous puzzles.

- Five towns are planning a rail network in which each pair of towns has a direct route between them. Can this be done without building any crossings?
- An old puzzle based on $K_{3,3}$ has the vertices on the left side representing houses and those on the right side representing utility services, typically water, electricity and gas. The puzzle is to connect each house to each utility so that none of the pipes or wires carrying the services to the houses cross each other.

We will soon be able to determine whether or not these two specific graphs are planar without doing an exhaustive search of the many different ways of drawing them.

Every planar drawing of a graph G divides the plane up into *regions*. Informally, these are the areas of the plane that are surrounded by vertices and edges but have no vertices and edges within them. You can imagine cutting the plane along all the edges (including their endpoints), leaving the plane to fall apart into separate “pieces”. These pieces are the regions of the plane graph. Each region is referred to as a **face** of the plane graph.

Each face has the property that any two points that lie inside a face (and not on any of the vertices or edges that surround the face) can be joined by a curve that also lies entirely within the face. So the curve does not meet any vertex or edge of G . In fact, a face may be defined formally as a *maximal* subset of points in the plane such that any two points in the subset can be joined by a curve that does not meet any vertices or edges of G .

The region of the plane “outside” the graph is considered to be a face just as much as the other regions. It is called the **outer face**, and must not be forgotten when counting the faces in the graph.

Consider the plane graph in Figure 12.5. This graph divides the rest of the plane into four faces, shown in the figure as F_1 , F_2 , F_3 and F_4 . Here, F_4 is the outer face.

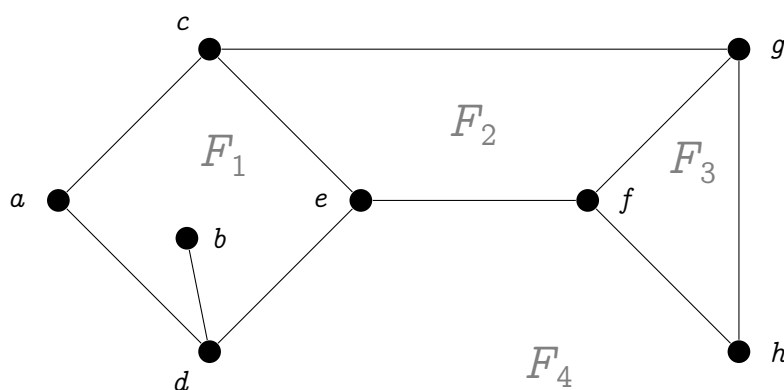


Figure 12.5: A plane graph with four faces: F_1 , F_2 , F_3 and the outer face F_4 .

Each face has a **boundary** consisting of those vertices and edges that are next to it. You can walk along this boundary, all around the face, much as you might walk around the boundary of a park or field. You can do so in either of two directions: one direction keeps the face on your *left* as you walk around it, while the other keeps the face on your *right* as you walk around it. Either way, and wherever you start, this walk will eventually return to your starting point, and it actually constitutes a *closed walk* in the graph.

The boundaries of the faces in [Figure 12.5](#) are as follows.

face	boundary	comment
F_1	a, c, e, d, b, d, a	uses edge $\{b, d\}$ twice
F_2	c, g, f, e, c	this boundary is actually a cycle
F_3	f, g, h, f	so is this
F_4	a, d, e, f, h, g, c, a	outer face; also a cycle in this case.

Consider any edge e in a plane graph, and imagine walking along it. As you do so, you can look to your left or to your right.

- On your left is a face that has that edge in its boundary. We imagine a small portion of that face that lies alongside the edge on your left, and call it a **side** of that edge.
- On your right there is also a face with that same edge in its boundary. Again, imagine a small portion of that face lying alongside the edge on your right, and call it a **side** of that edge.

Often, the faces we see on either side of an edge are two different faces, as in [Figure 12.6\(a\)](#). But they can also be the same face, as in [Figure 12.6\(b\)](#). In any case, each edge has *two sides*, and these sides may belong to different faces or to the same face.

Observe that each edge belongs to at most two faces. If we construct a boundary walk around every face, then each edge appears exactly twice in the full set of boundary walks. This could happen in either of two ways:

- The edge might appear once in the boundary walk around one face, and once in the boundary walk around a different face.
- The edge might appear twice in the boundary walk around a single face.

In each case, the edge appears in no other boundary walk around any other face.

You can, if you wish, check that each edge of the plane graph in [Figure 12.5](#) appears twice in the set of boundaries of the four faces, using the list of boundary walks we gave above. For example, edge $\{e, f\}$ appears once in the boundary of F_2 and once in the boundary of F_4 , while edge $\{b, d\}$ appears twice in the boundary of F_1 .

The numbers of vertices, edges and faces in a plane graph satisfy an equation which, again, is due to Euler.

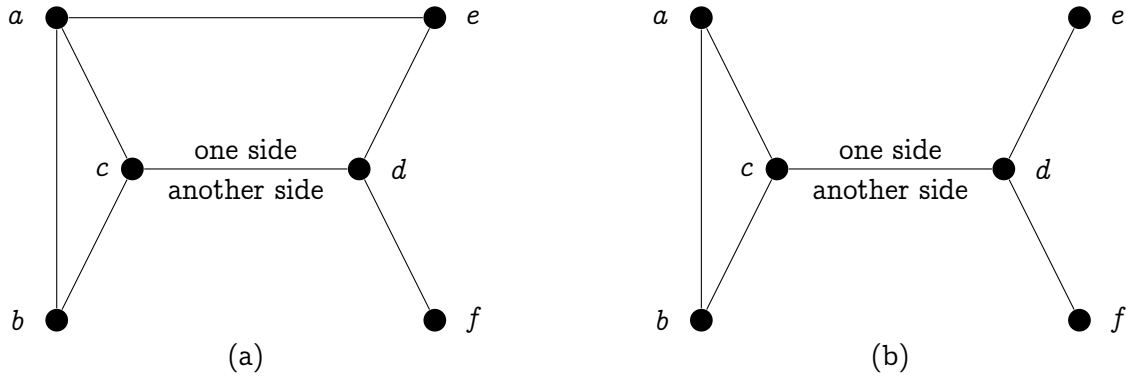


Figure 12.6: Two plane graphs, each showing the two sides of an edge $\{c, d\}$. (a) The two sides are in different faces. (b) The two sides are in the same face.

Theorem 71 (Euler's Theorem). Let G be a connected plane graph, and let n , m and f be its numbers of vertices, edges and faces, respectively. Then

$$n - m + f = 2.$$

Proof. We prove by induction on m that, for any graph,

$$n - m + f = 2, \tag{12.3}$$

with n, m, f defined as in the statement of the theorem.

Inductive Basis:

We don't need to consider any $m < n - 1$, since no graph with fewer than $n - 1$ edges can be connected. This is because trees are *minimal* connected graphs on a given vertex set, and they have $n - 1$ edges by [Theorem 68](#).

If $m = n - 1$, then G must be a tree, since it is connected. When a tree is drawn in the plane, the only face is the outer face, consisting of the entire plane except for the points representing the vertices and the edges of the tree. So, in this case, G has one face, so $f = 1$. Then, using $m = n - 1$ and $f = 1$, we have

$$n - m + f = n - (n - 1) + 1 = n - n + 1 + 1 = 2.$$

So (12.3) holds in this case.

Inductive Step:

Let $m \geq n - 1$.

Assume that, for every graph with m edges,

$$(\# \text{ vertices}) - (\# \text{ edges}) + (\# \text{ faces}) = 2.$$

This is our Inductive Hypothesis.

Let G be any connected graph with n vertices, $m+1$ edges and f faces. We need to prove that $n - (m+1) + f = 2$.

Since G is connected and has more than $n-1$ edges, it cannot be a tree and therefore must contain a cycle. Let C be a cycle in G and let e be any edge of C . Now, C divides the plane into two parts, the “inside” of C and the “outside” of C .³ This means that the two sides of the edge e are in two separate faces of G (otherwise it would be possible to move from one side of e to the other side of e without crossing C , which would be a contradiction).

Consider now the graph H formed from G by deleting the edge e but keeping its endpoints and all other vertices and edges of G . Its drawing in the plane is unchanged except that the edge e has disappeared. The absence of e means that the two separate faces that were on either side of e are now merged into a single face. So

$$\# \text{ faces of } H = (\# \text{ faces of } G) - 1 = f - 1.$$

Since we deleted one edge from G to get H , the number of edges of H is also one less than the number of edges of G , so

$$\# \text{ edges of } H = (\# \text{ edges of } G) - 1 = (m+1) - 1 = m.$$

Although the deletion of e has reduced the numbers of edges and faces, it has not changed the number of vertices:

$$\# \text{ vertices of } H = \# \text{ vertices of } G = n.$$

Now, since H has only m edges, we can apply the Inductive Hypothesis to it. So we have

$$(\# \text{ vertices of } H) - (\# \text{ edges of } H) + (\# \text{ faces of } H) = 2.$$

Substituting the expressions we have derived for these three quantities, we have

$$(\# \text{ vertices of } G) - ((\# \text{ edges of } G) - 1) + ((\# \text{ faces of } G) - 1) = 2.$$

Therefore

$$(\# \text{ vertices of } G) - (\# \text{ edges of } G) + (\# \text{ faces of } G) = 2,$$

so (12.3) holds for G as well. This completes the Inductive Step.

³ We are treating this fact as intuitively obvious and not justifying it further. But it is actually surprisingly deep, and is not trivial to prove rigorously. It is called the **Jordan Curve Theorem**.

Conclusion:

Therefore, by Mathematical Induction, (12.3) holds for all m .

□

Corollary 72. For any planar graph G with $n \geq 3$ vertices and m edges,

$$m \leq 3n - 6.$$

Proof. Since G is planar, it has a planar drawing. Let f be the number of faces in some planar drawing of G .

We are going to use the faces of G , together with (i) what we can say about the sizes (i.e., numbers of edges) of each of those faces, and (ii) Euler's Theorem, to derive an inequality between m and n .

Since G is simple, every face has at least three sides. So, for each face, we can pick any three of its sides and mark them. A side can only be marked from the face it belongs to, so each side gets marked at most once. Each edge has exactly two sides, and the number of sides is twice the number of edges. So the number of sides that get marked by this process is $\leq 2m$. Since the number of marks is $3f$, it follows that

$$3f \leq 2m,$$

which gives

$$f \leq \frac{2m}{3}.$$

But Theorem 71 tells us that $n - m + f = 2$, which after rearranging gives

$$-n + m + 2 = f.$$

Combining these, we have

$$-n + m + 2 \leq \frac{2m}{3},$$

which is equivalent to

$$-3n + 3m + 6 \leq 2m.$$

From this we obtain

$$m \leq 3n - 6.$$

□

This fact is already strong enough to settle our earlier question about the planarity or otherwise of K_5 .

Corollary 73. K_5 is nonplanar.

Proof. K_5 has five vertices and ten edges: $n = 5$ and $m = 10$. We have $3n - 6 = 3 \cdot 5 - 6 = 15 - 6 = 9 < 10$, so

$$m > 3n - 6.$$

Therefore, by [Corollary 72](#), K_5 is not planar. $\square$

But [Corollary 72](#) is not strong enough to answer our question about $K_{3,3}$.

We can get a stronger bound on the number of edges in a planar graph when the graph has no triangles.

Corollary 74. For any triangle-free planar graph G with $n \geq 3$ vertices and m edges,

$$m \leq 2n - 4.$$

Proof. The proof is very similar to the proof of [Corollary 72](#). The key difference is that, when we mark edges around each face, we know that each face has at least *four* sides, because G has no triangles, so we mark *four* of them. Instead of the inequality $3f \leq 2m$, we have

$$4f \leq 2m,$$

which simplifies to

$$2f \leq m.$$

Combining this with Euler's Theorem, $n - m + f = 2$, gives

$$m \leq 2n - 4.$$

$\square$

We are now in a position to settle our question about $K_{3,3}$.

Corollary 75. $K_{3,3}$ is nonplanar.

Proof. $K_{3,3}$ has six vertices and nine edges: $n = 6$ and $m = 9$. We have $2n - 4 = 2 \cdot 6 - 4 = 12 - 4 = 8 < 9$, so

$$m > 2n - 4.$$

Note also that, since $K_{3,3}$ is bipartite, it has no odd cycles, by [Corollary 63](#). In particular, it has no cycles of length 3, i.e., it has no triangles. Therefore, by [Corollary 74](#), $K_{3,3}$ is not planar. $\square$

12.6^ω GAMES ON GRAPHS

We finish our exploration of graph theory with some games you can play on graphs.

12.6.1^ω Shannon's Switching Game

The Shannon Switching Game is named after Claude Shannon, one of the most important and influential scientists of the twentieth century. His first major contribution was in his Masters thesis, where he introduced the algebra of switching. He went on to lay

the foundations of information theory, including the theory of how to communicate information clearly and efficiently in the presence of noise. Then, in related work, he laid the mathematical foundations of cryptography.

Throughout his work, he was driven by curiosity and maintained a great sense of fun. This game is one of his many inventions.

We start with any connected graph in which there are two special vertices designated s and t . There are two players, Cut and Join, who take turns. They do different things on their turns, and have different aims. On their turn:

- **Cut** crosses out an edge, with the aim of ensuring that there is no path between s and t in G . Once an edge is crossed out, it cannot be used in such a path.
- **Join** thickens an edge, with the aim of ensuring that there is a path of thickened edges between s and t in G .

Once an edge is crossed out, it can never be thickened, and once an edge is thickened, it can never be crossed out.

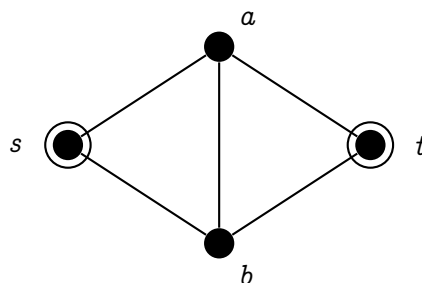
The game ends in one of two ways.

- As soon as there is a path of thickened edges between s and t , Join wins. (It is not required that *all* the thickened edges form such a path, but only that the thickened edges *include* a thickened path between s and t .)
- As soon as there is no possibility of such a path — because s and t are in different components of the graph of uncrossed-out edges — Cut wins.

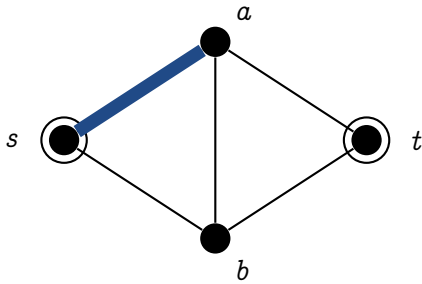
As soon as either player wins, the game stops.

We have not stipulated who goes first. This can be agreed between the players in advance.

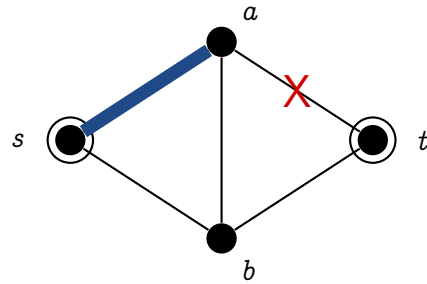
Let's illustrate one possible play of the game, on the following graph, with Join going first.



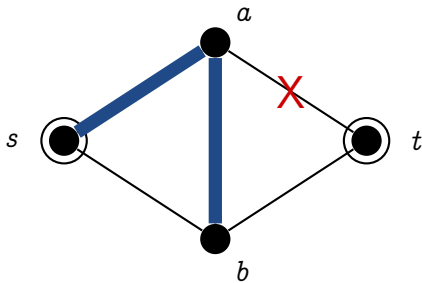
Join's first move: thicken $\{s, a\}$.



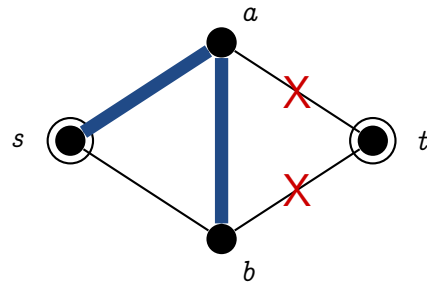
Cut's first move: cross out $\{a, t\}$.



Join's second move: thicken $\{a, b\}$.



Cut's second move: cross out $\{b, t\}$.



At this point, Cut wins the game, because once edges $\{a, b\}$ and $\{b, t\}$ are deleted, the terminals s and t are in different components of the remaining graph.

But Join played poorly in this game. In particular, their second move was a serious blunder! What should they have done? Could they have won? Do they have a winning strategy in this game?

The theory of connectivity (§ 11.9) tells us that there is always a winner in this game. There is no possibility of a draw. Suppose the players keep playing until all edges are either thickened or crossed out, and consider the graph consisting of all the original vertices but only the thickened edges. Either there is a path between s and t in this graph, or there isn't. If there is such a path, then since this path only has thickened edges, Join wins. If there is no such path, then Cut wins. As soon as a thickened-edge path is created or becomes impossible, the game is over, so the game could finish before all edges are used, but in any case, there is always a winner.

At any stage while playing the game, there is at least one move for the player whose turn it is that is best possible for them. It may be that one of the players has a *winning strategy*. What would it mean for the player who starts the game to have a winning strategy on a particular graph? This means that:

they have a **1st** move such that:
 for each of their **opponent's 1st** moves,
 they have a **2nd** move such that,
 for each of their **opponent's 2nd** moves,
 they have a **3rd** move ...
 ...and so on, ...

eventually leading to a win for the starting player no matter what their opponent does. This is not saying that they can force a win from every conceivable configuration in the game, but rather that they can force a win, eventually, from the starting configuration in the game (in which no edges are thickened or crossed out). It's also not saying that they will win even if they play badly. A player with a winning strategy may well lose if they depart from that strategy.

On the other hand, it's possible that the second player to move (i.e., the one who does not start the game) has a winning strategy. This would mean that:

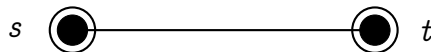
for each of their **opponent's** possible **1st** moves,
 they have a **1st** move in reply, such that
 for each of their **opponent's** possible **2nd** moves,
 they have a **2nd** move in reply, such that
 for each of their **opponent's** possible **3rd** moves, ...
 ...and so on, ...

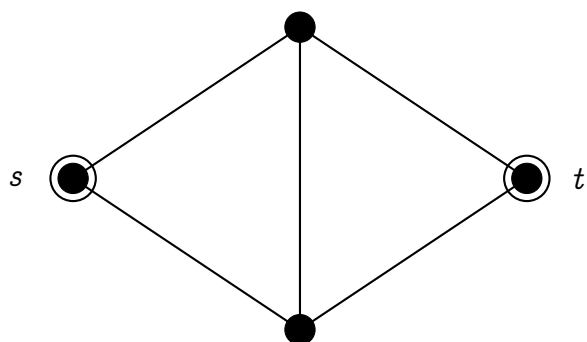
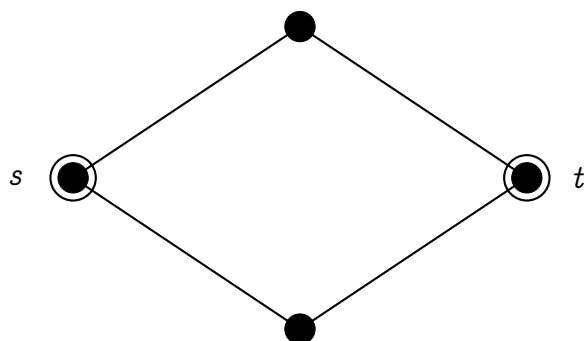
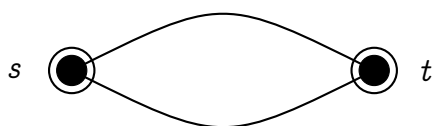
eventually leading to a win for the second player no matter what their opponent (the first player) does.

Since every play of the game always results in a win for one of the two players, it follows that if one of the two players does not have a winning strategy, then the other does. This may, or may not, depend on who goes first. So, for any given graph, there may be four possibilities:

- Join has a winning strategy, whether they go first or second.
- Cut has a winning strategy, whether they go first or second.
- The first player has a winning strategy, whether that's Join or Cut.
- The second player has a winning strategy, whether that's Join or Cut.

For each of the following graphs (or multigraphs), see if you can determine which of those four cases apply.





Try making some other graphs, and playing the Shannon Switching Game on them, and try to determine who has a winning strategy in each.

After you have done this for the above graphs and maybe some others, you will be able to classify each graph into one of our four categories, according to who has a winning strategy: Join, Cut, first player, or second player. You should notice that there is one category with no graphs: there is actually *no graph* for which the *second* player *always* has a winning strategy regardless of whether Join or Cut starts the game.

This is not just an empirical observation from the graphs you have tried. (Such observations are important in formulating conjectures, but of course they are not proofs.)

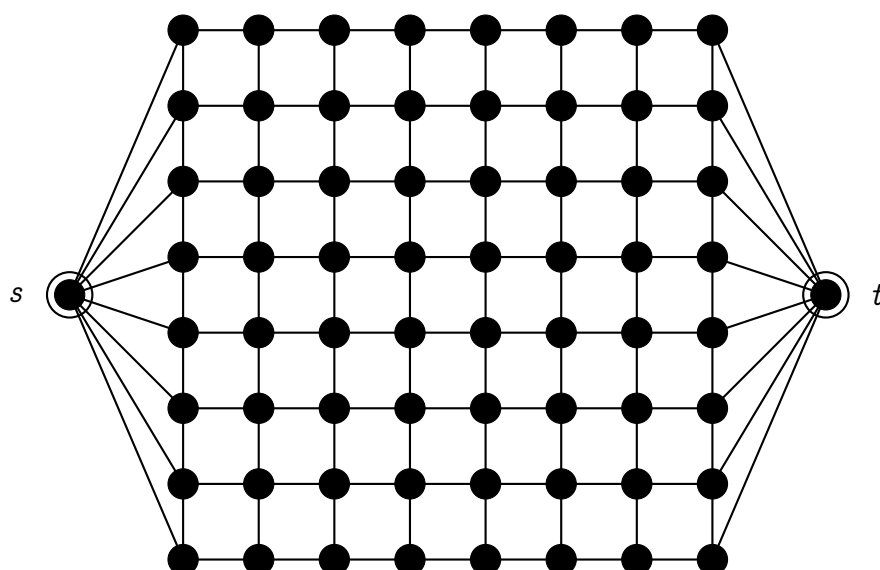
It is, in fact, a general property of this game. To see this, imagine that the second player always has a winning strategy, regardless of who starts. Then what can the first player do? Consider the following strategy for the first player: make *any initial move*, and from then on, play as if they are the second player, which should result in them winning if they use one of the winning strategies for the second player. This gives a contradiction: we started by assuming that the second player can force a win, and then showed how this same strategy could be exploited by the first player to force a win. So, in fact, there can be no graph for which the second player always has a winning strategy.

The reason this argument works is that, in this game, it is *never a disadvantage to move*. For Join, it can never be a disadvantage to thicken an edge (compared with doing nothing at all), and for Cut, it can never be a disadvantage to cross out an edge. In other words, Join thickening an edge can never be better, for Cut, than Join doing nothing, and Cut crossing out an edge can never be better, for Join, than Cut doing nothing. (In this game, you are not allowed to do nothing on your move, but it wouldn't matter if you *were* allowed to do nothing, because if you play sensibly, you would always choose to make an actual move — joining or cutting, depending on your role — rather than doing nothing.)

This argument won't work in all two-player games. For example, in Chess, Draughts (Checkers), Reversi (Othello) and Backgammon, situations can arise where any move by a player is disadvantageous for them. Other games where moving is *never* a disadvantage include Noughts-and-Crosses (Tic-tac-toe). In that particular case, draws are possible, so we cannot argue that there is always a winning strategy for one of the two players.

You can just enjoy playing the game for fun. You can also try to develop some theory for it. If you'd like a challenge, try to characterise those graphs that belong to each of our three categories: Join wins, Cut wins, or first player wins. This is not easy, but the characterisation uses some concepts discussed in this chapter. Such a characterisation should be based purely on the structure of the graph, without examining all possible plays of the game on the graph (of which there is, in general, a huge number). But, of course, it's ok to play the game as many times as you like, to help explore how it works and to help discover what it is, about the structure of graphs, that helps a particular player win.

Here is a much more complex graph you can play the game on if you wish.



A commercial version of this game, using this last graph, has been produced. This is the game Bridg-It, invented by David Gale and manufactured by Hasenfeld in 1960.



<http://abstractstrategy.com/bridg-it.html>

12.6.2^ω *Slither*

The game of **Slither** was invented by David L. Silverman and described by Martin Gardner in 1972.⁴

The game can be played on any graph, as follows. Two players take turns choosing edges so that the set of chosen edges always forms a path in the graph. Initially, no edges are chosen. The first player chooses any edge they like, which is a path of length 1. Then the second player must choose an edge which is incident with the first edge and

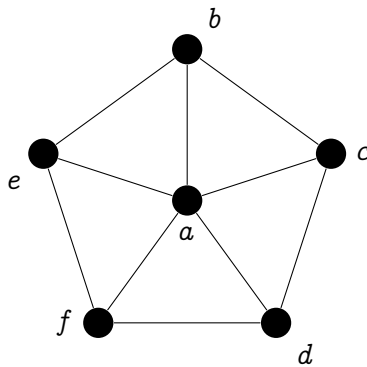
⁴ Martin Gardner, Mathematical games, *Scientific American* **226** (no. 6) (June 1972) 114–118.

creates a path of length 2. Then the first player chooses any edge that extends the path created so far. And so on, for as long as possible. In order to extend the path chosen so far, each new edge chosen must satisfy the following conditions:

- One of its endpoints must be one of the two endpoints of the path chosen so far.
 - Either of those endpoints can be used, at each turn. Players are not required to keep extending the path at the same end.
- The other endpoint of the new edge must be a vertex that does not yet belong to the path. That new vertex then becomes an end of the path.

The game ends when no legal move is possible. When that happens, the last player to have made a legal move wins. So, the first player who cannot make a legal move loses.

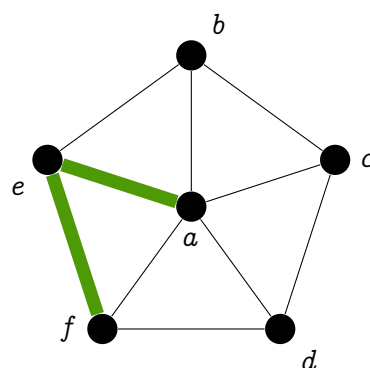
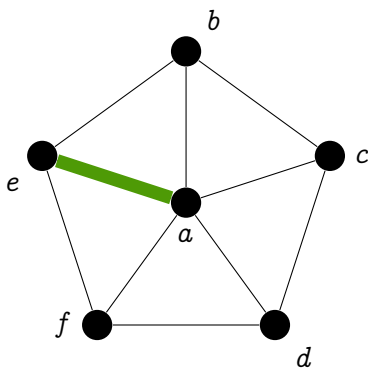
We now give a possible play of the game, on the following graph.



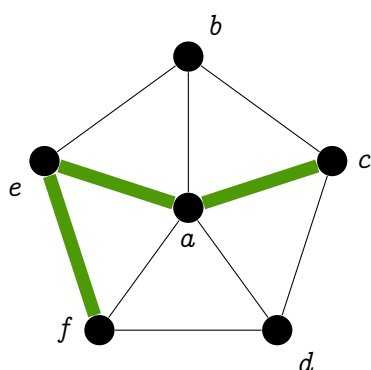
The chosen edges are shown as thick green edges.

First player's first move: $\{a, e\}$.

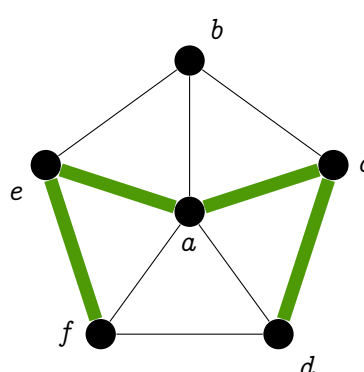
Second player's first move: $\{e, f\}$.



First player's second move: $\{a, c\}$.



Second player's second move: $\{c, d\}$.



In this play of the game, the game stops at this point because no further legal move is possible. The path cannot be extended at either end without destroying the path property. The player to move (the first player, in this case) cannot choose $\{d, f\}$, because that creates a cycle of chosen edges: a, e, f, d, c, a . Similarly, neither $\{a, f\}$ nor $\{a, d\}$ can be chosen, because they also create cycles of chosen edges (triangles, in fact). The edges $\{b, c\}$, $\{a, b\}$ and $\{b, e\}$ cannot be chosen because they are not incident with an endpoint (d or f) of the chosen path.

Could the first player have done better? Does the first player have a winning strategy when the game is played on this graph?

This game must end in a win for one or the other of the two players. For each graph, we have one of two possibilities:

- The first player has a winning strategy.
- The second player has a winning strategy.

Try the game on a few small graphs, and classify them as to whether you think the first or second player has a winning strategy. This time, you should find that there are graphs of each type: some for which the first player has a winning strategy, and others for which the second player has a winning strategy.

Then try and investigate how to determine, just from the graph itself and without doing an exhaustive search of all possible plays of the game, who has a winning strategy on a given graph.

One important difference between this game and the Shannon Switching Game is that, in this game, the two players have the same role, since their effect on an edge, when they choose it, is the same. (This contrasts with the Shannon Switching Game, where one player thickens edges and the other player crosses them out.) Of course, the details of the games are very different, too. But this point about the different roles of the players is a very fundamental one in the theory of games. A game is said to be

impartial if, in any given configuration, the options available to each player would be the same if it were their turn to move. A game is **partisan** if it is not impartial. So, with this terminology, the Shannon Switching Game is a partisan game, while Slither is an impartial game.

12.7 EXERCISES

1. Draw all trees that have six vertices. Your set of drawings should be comprehensive, so that every tree on six vertices can be seen to be identical to one of those in your set (potentially after some relabelling and redrawing).

2. The **diameter** of a graph is the length of the longest path in it. This is the maximum, over all pairs of vertices v, w , of the length of any path between v and w .

Prove that every tree of diameter ≥ 3 has a vertex v such that

$$2 \leq \deg(v) \leq n - 2,$$

where n is the number of vertices of the graph.

3. If a tree has k vertices of degree 3 and all other vertices are leaves, how many leaves does it have?

4.

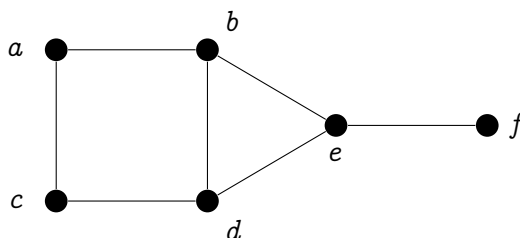
(a) Prove that every tree with at least three vertices has a vertex of degree at least 2.

(b) Is there a number N such that every tree with at least N vertices has a vertex of degree at least 3?

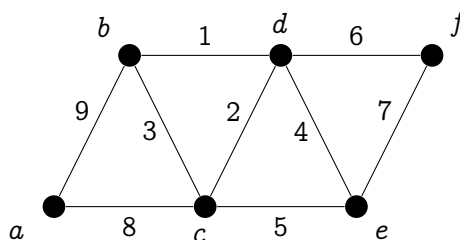
5. Prove that a graph G is a tree if and only if for every pair u, v of vertices, there is a unique path between them in G .

6. Give an upper bound on the average degree of a tree.

7. How many spanning trees does the following graph have?



8. Apply Kruskal's Greedy Algorithm to find a minimum weight spanning tree in the following weighted graph.



9. Prove that $K_{2,n}$ is planar for all n .
10. Check that Euler's Theorem holds for the following graphs:
- the cycle C_n ;
 - the wheel graph W_n , which is obtained from C_n by adding one new vertex (the "hub") that is adjacent (via "spokes") to every vertex in C_n (the "rim");
 - K_4 ;
 - the cube Q_3 (Exercise 11.11);
 - the octahedron;
 - the dodecahedron;
 - the icosahedron.
- 11.
- Give an upper bound on the average degree of a planar graph.
 - Give an upper bound on the average degree of a planar graph with no triangles.
 - Give an upper bound on the average degree of a bipartite planar graph.
12. Prove that every planar graph has a vertex of degree ≤ 5 .
- 13.
- Let $d \in \mathbb{N}$. A graph is **d -regular** if every vertex has degree d .

- (a) Find the 3-regular planar graph with the fewest vertices.
- (b) Find the 3-regular bipartite planar graph with the fewest vertices.
- (c) Find the 3-regular nonplanar graph with the fewest vertices.
- (d) Describe an infinite family of connected 4-regular planar graphs.
- (e) What is the minimum number of vertices that a 5-regular planar graph can have? Can you find such a graph?
- (f) Does a 6-regular planar graph exist? Justify your answer.